ARCH LINUX 分册教材

第九册

安全与加密

加固、认证、加密存储、沙箱与密钥管理

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

一台自己搭起来的 Arch，装好、能开机、能上网，并不等于它守得住。家目录里的私钥对任何能碰到磁盘的人都可见，一次来路不明的构建脚本也可能把任意代码请进系统。本册处理的是这些“装完以后才开始最重要”的问题：谁被允许做什么，别人拿走硬盘时数据还剩多少隐私，以及程序行为越界时用什么把它关起来。

读完本册，你应当能为自己定一个明确的威胁模型——防小偷、防入侵，还是防自己误操作——并据此挑选用得上的措施：给谁提权、在哪一层加密磁盘、要不要给笔记本配硬件令牌、要不要把某个联网程序关进沙箱。你也会知道哪些流行做法其实没有增益。

在九册中的位置

这是全套的最后一册，后面没有续篇。它把前八册里散落的安全细节收拢成一条线：第二册装好的系统、包管理与软件包签名，在这里换成密钥管理的视角再讲一遍；第三册的分区、文件系统与 initramfs，是静态数据加密的直接底座；第四册配好的网络与远程登录，正是本册要加固的对象。硬件令牌、指纹这类内容，还要回头对照第五册与第六册的外设和桌面。

本册不重复基础操作，你可以随时翻回前面某册查命令。

先把总纲读完

建议“先总后分”。第 1 章把用户与用户组、提权方式、权能、权限掩码、授权代理这些日常加固手段铺开，是后面各章共用的词汇表，值得完整读一遍；跳过它，后面的配置会处处看不懂。之后按场景分支：笔记本与个人工作站从静态数据加密和沙箱入手，再补密码、密钥与第二因素；服务器方向则先读加密库与 TLS，再走身份验证与访问控制。

另有几章各自独立成篇，更像手册条目——智能卡、指纹、入侵检测之类，没有对应硬件或需求时可以整章跳过。

前置知识与几个误区

前置由第二、三、四册提供：会读 systemd 服务、能看懂分区与挂载、能用 SSH 连上一台机器，剩下的不作要求。加密部分会在启动早期完成解锁，出问题时要能自己进恢复环境。

常见的误区有四个。第一，把“加密”当成“安全”：加密只在关机或未解锁时保护数据，登录之后它几乎不设防。第二，以为装上 sudo 就万事大吉，而真正的风险常是口令

复用和随手执行陌生脚本。第三，以为容器或沙箱等同于虚拟机，它们本质上共用同一个内核。第四，在没有备份加密头、没有记牢恢复口令时就动手做全盘加密——这一步出错，代价是整块磁盘再也打不开。

安全分层

没有任何单点能兜住全部



安全是分层叠加的结果，任何单点措施都不足以抵御针对性攻击

从固件到数据的五层安全防线

目　录

第 1 章　安全（18 节）

1.1 TPM2-PK11　1.2 PRoot　1.3 Systemd/run0　1.4 Haveged　1.5 Su　1.6 Doas　1.7 Wireshark　1.8 Capabilities　1.9 实时进程管理　1.10 Umask　1.11 MAC 地址欺骗　1.12 安全地擦除磁盘/提示和技巧　1.13 Polkit　1.14 ClamAV　1.15 安全地擦除磁盘　1.16 用户和用户组　1.17 Sudo　1.18 安全

第 2 章　静态数据加密（11 节）

2.1 Dm-crypt　2.2 Dm-crypt/加密非root文件系统　2.3 Fscrypt　2.4 Dm-crypt/准备磁盘　2.5 Systemd-cryptenroll 2.6 Dm-crypt/交换分区加密　2.7 Dm-crypt/系统配置　2.8 静态数据加密　2.9 Dm-crypt/特殊应用　2.10 Dm-crypt/设备加密　2.11 Dm-crypt/加密整个系统

第 3 章　沙箱（8 节）

3.1 Systemd/沙箱　3.2 Bubblewrap　3.3 Bubblewrap/示例　3.4 Firejail　3.5 Linux 容器　3.6 Podman　3.7 Docker　3.8 Systemd-nspawn

第 4 章　加密（5 节）

4.1 Vault　4.2 Tomb　4.3 Paperkey　4.4 SHA 散列算法　4.5 Dnscrypt-proxy

第 5 章　TLS（4 节）

5.1 Mbed TLS　5.2 GnuTLS　5.3 Network Security Services　5.4 OpenSSL

第 6 章　身份验证（4 节）

6.1 Pam oath　6.2 Pam autologin　6.3 Howdy　6.4 Kerberos

第 7 章　OpenPGP（2 节）

7.1 软件包维护者指导原则　7.2 Makepkg

第 8 章　密码学（2 节）

8.1 Hashcat　8.2 随机数生成

第 9 章　认证（2 节）

9.1 身份管理　9.2 LDAP 认证

第 10 章　访问控制（2 节）

10.1 访问控制列表　10.2 AppArmor

第 11 章　Smartcards（1 节）

11.1 YubiKey

第 12 章　TOMOYO Linux（1 节）

12.1 TOMOYO Linux

第 13 章　入侵检测（1 节）

13.1 Rkhunter

第 14 章　密码管理器（1 节）

14.1 Pass

第 15 章　指纹识别（1 节）

15.1 Fprint

第 16 章　通用第二因素（1 节）

16.1 通用第二因素

安全

安全不是一个开关，而是一条从密码、CPU、内存、存储贯穿到账户设置的链路。本章先讲清威胁与信任边界，再逐项落到能照着做的配置：账户一侧是《用户和用户组》《Sudo》《Doas》《Su》《Polkit》，文件一侧是《Umask》与《Capabilities》，数据处置则归《安全地擦除磁盘》及其提示和技巧。

建议以《安全》为纲，读完概念与账户设置，再依次走《用户和用户组》《Sudo》《Doas》《Su》《Polkit》这条提权与授权主线，这五节必读；《Umask》《Capabilities》补齐权限细节，也不宜跳过；《run0》《PRoot》《TPM2-PK11》《Wireshark》《ClamAV》《Haveged》《实时进程管理》《MAC 地址欺骗》各管一类具体需求，可略读，用时再查。最容易踩的坑是把某个机制当万能钥匙：覆写并不等于数据彻底消失，用户组权限与 umask 谁先生效常被搞反，PRoot 与 run0 也都不构成安全边界。

1.1　TPM2-PK11

难度 入门 | **篇幅** 2.1 KB | **所属** 安全

本节将讲到

- SSH 用法

tpm2-pk11-gitAUR\[损坏的链接：package not found\] 为 TPM 芯片提供了 PKCS#11 后端。这可以用来保护您的 SSH 密钥。

注意：截至 2017 年 11 月，仅支持 OpenSSH 客户端。

SSH 用法

创建密钥：

```
$ mkdir ~/.tpm2 && cd ~/.tpm2
$ tpm2_createprimary -A e -g 0x000b -G 0x0001 -C po.ctx
$ tpm2_create -c po.ctx -g 0x000b -G 0x0001 -o key.pub -O key.priv
$ tpm2_load -c po.ctx -u key.pub -r key.priv -n key.name -C obj.ctx
$ tpm2_evictcontrol -A o -c obj.ctx -S 0x81010010
$ rm key.name *.ctx
```

创建配置文件并为您的设置进行更改：

```
$ cp config.sample ~/.tpm2/config
```

提取公钥：

```
$ ssh-keygen -D libtpm2-pk11.so
```

使用您的 TPM 密钥：

```
$ ssh -I libtpm2-pk11.so ssh.example.com
```

或在 `~/.ssh/config` 中将 PKCS#11 模块添加到您的 ssh 配置中：

```
Host *
    PKCS11Provider libtpm2-pk11.so
```

1.2 PRoot

难度 入门 | **篇幅** 2.4 KB | **所属** 安全

本节将讲到

- 安装
- 用法
- 安全性

PRoot是一个能在用户空间内运行的程序, 功能类似于chroot, `mount --bind`和binfmt_misc , 能让root用户使用备用根目录运行程序, 就像chroot "jail"一样。 PRoot在由于缺少root权限而无法使用chroot的情况下尤其有用。

安装

PRoot可以从prootAUR包安装。 *pacstrap*可以在运行*proot*之前用Arch环境初始化目录。

用法

安装完成之后, PRoot不需要root权限。 和chroot一样, 你必须为PRoot提供一个新目录作为新的根目录。 如果没有指定shell程序，PRoot默认将启动`/bin/sh`。 虚拟文件系统无需手动挂载，PRoot会自动处理这个问题。

```
proot -r ~/mychroot/
```

此时，将启动一个shell， `/` 对应于主机上的 `~/mychroot/` 文件夹。

可以使用 `-b` 选项明确地绑定变量：

```
proot -b /bin/bash:/bin/sh
```

这使得主机的/bin/bash在来宾的/bin/sh处可用

PRoot在内部使用qemu用户模式模拟器来允许程序在PRoot内运行，即使程序是为主机系统以外的体系结构编译的。

安全性

与chroot一样，PRoot仅提供文件系统级隔离。 PRoot "jail" 中的程序共享相同的内核，硬件，进程空间和网络子系统。 chroot和PRoot并不是如同虚拟机管理程序或半虚拟化程序的，真正的虚拟化程序替代品。

1.3 Systemd/run0

难度 入门 | **篇幅** 2.5 KB | **所属** 安全

本节将讲到

- 安装
- 配置
- 使用
- 参见

翻译状态：

- 本文（或部分内容）译自 Systemd/run0，最近一次同步于 2026-10-08，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/run0_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

run0(1) 是一个提权工具。其作为一种 systemd-run(1) 的另一种调用模式实现。run0 为 sudo 机制提供了一种更安全、更稳健的替代方案，由 systemd PID 1 执行命令（“权限向下剥离”而非“向上窃取权限”），使用 polkit 验证而无需依赖 SUID 二进制文件进行验证。

安装

自 systemd v256 起，run0 包含于 systemd包 软件包中。

安装 polkit包 软件包以处理授权。

配置

run0 没有独立的配置文件，所有配置均在 Polkit#配置 和 Polkit#认证规则 中完成。

默认情况下 Polkit#管理员身份认证 会将 `wheel` 组用户视为管理员。因此考虑将你的用户加入 `wheel` 组，以便于使用自己的身份进行认证。

使用

要使用 run0，需要在命令和参数前加上 `run0` 和空格

```
$ run0 cmd
```

例如使用 pacman 升级软件包：

```
$ run0 pacman -Syu
```

若需启用交互式 shell （类似于 `sudo -i` 或 `su -l`），只需要执行 `run0`。

使用 `--empower` 参数以当前用户身份运行特权命令，以避免完整的 root 上下文。这在需要以特权身份运行命令，同时确保所有创建的文件和目录权限仍归属当前用户时很有用（此方法加入自 systemd v259）。

`systemd-run` 的选项同样适用于 `run0`，参见：systemd-run(1) 和 run0(1)。

提示：参见安全#无 SUID 获取更高级的示例。

参见

1.4 Haveged

难度 入门 | **篇幅** 3.6 KB | **所属** 安全

本节将讲到

- 安装
- 检查当前的熵
- 其它选择
- Virtual machines
- 参阅

翻译状态：

- 本文（或部分内容）译自 Haveged，最近一次同步于 2014-12-13，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

haveged 项目的目的是提供一个简单易用的不可预测随机数生成器，基于 HAVEGE 算法。Haveged 可以解决在某些情况下，系统熵过低的问题。

警告：此程序无法保证熵的质量(，). 如果对安全要求较高，请考虑使用硬件随机数生成器 rng-tools包.

安装

安装软件包 haveged包.

启动和启用服务 `haveged.service`。

检查当前的熵

要检查是否需要 Haveged, 使用下面命令查看当前收集到的熵:

```
# cat /proc/sys/kernel/random/entropy_avail
```

如果结果比较低 (<1000)，建议安装 haveged. 否则加密程序会等待系统有足够的熵。例如如果使用软件热点，网速会比较慢。

安装 haveged 之后，可以再次查看系统熵看下有无提升。

其它选择

Unless you have a specific reason to not trust any hardware random number generator on your system, you should try to use them with the rng-tools first and if it

turns out not to be enough (or if you do not have a hardware random number generator available), then use Haveged.

Virtual machines

As discussed at Is it appropriate to use haveged as a source of entropy on virtual machines?, it can be contested whether haveged provides quality entropy within a virtual environment. Haveged relies on the rdtsc instruction, which may be virtualized within a virtual machine resulting in lower quantity entropy. On some hypervisors, it is possible to disable the virtualization of rdtsc, which would in theory allow haveged to provide higher quality entropy.

To disable the virtualization of the rdtsc instruction in VMware ESXi, add the setting `monitor_control.virtual_rdtsc = "FALSE"` to the virtual machine's .vmx configuration file. VMware recommends the setting for use when performing measurements that require a precise source of real time in the virtual machine.

参阅

- http://www.issihosts.com/haveged
- 如何通过 haveged 增加系统的熵

1.5 Su

难度 入门 | **篇幅** 4.2 KB | **所属** 安全

本节将讲到

- 安装
- 用法
- 提示和技巧

su) 核心应用程序（替代用户）用来假设系统中另一个用户的身份，默认为root。

参阅 PAM 可以找到配置 **su** 其他特性的方法。

安装

su 是 util-linux包 包的一部分。

用法

要切换到其他用户身份，将要切换的用户名传递给 su，像这样：

```
$ su username
```

默认情况下，当以普通用户身份运行时，系统会提示您输入要成为的用户的密码。以root身份运行 *su* 时，不需要密码。

如果没有传入用户名，su 默认切换为 root 用户，要求输入的密码也应该是 root 用户的密码。

更多信息，请参阅su(1)

提示和技巧

“登录至”其他用户

su 的默认行为是保持在当前目录中并保持原始用户的环境变量（而不是切换到新用户的环境变量）。

这一特性的优劣需要注意以下重要的对比因素：

- 系统管理员可以使用普通用户的 shell 而不是自己的。特别是在有些时候，解决用户问题的最有效的方法，就是登录到该用户的帐户以重现问题或进行调试。
- 但是通常情况下，root 用户不能登录普通用户的 shell 并使用该用户的环境变量进行操作，而是用自己的环境变量操作，这在很多情况下是不可取的，甚至是危险的。在无意中使用普通用户的 shell 时，root 可能会安装程序，或对系统进行其他更改，而这些更改与使用 root 帐户时所做的结果不同。例如，可能会安装某个程序，使得普通用户能够意外地损坏系统或未经授权访问某些数据。

因此，建议系统管理员以及被授权使用 su 的任何其他用户（建议只有极少数用户），始终保持用 `-l` 或 `--login` 选项运行 su 命令的习惯。它有两个作用：

1. 通过登录至目标用户，从当前工作目录切换到目标用户的主目录（比如切换到 root 用户就是 `/root`）。
2. 根据目标用户的首选设置（比如 `~/.bashrc` 或是 bash shell 的其他配置文件） 切换到目标用户的环境变量。也就是说，当前工作目录和环境将会切换到和目标用户登录新会话时一样的目录和环境（而不是仅接管现有用户的会话）。

因此，管理员通常应该这样使用 su：

```
$ su -l
```

添加用户名 root 结果一样：

```
$ su -l root
```

对于任何其他用户（如名为 archie 的用户），同样可以做到：

```
# su -l archie
```

你可能希望在 `~/.bashrc` 里为这个规则添加一个 alias：

```
alias su="su -l"
```

提示：有时你会遇到使用 `su -` 而不是 `su -l` 或 `su --login` 来登录用户。不提倡这种简写是因为 `-` 操作会遇到解析限制，参阅 su(1) § DESCRIPTION。

su 和 wheel 用户组

BSD su 默认仅允许 `wheel` 用户组成员切换至 root 身份。而 GNU su 默认没有这一特性，可以使用 PAM 来模拟这一特性。将 `/etc/pam.d/su` 和 `/etc/pam.d/su-l` 中相应的行取消注释：

```
auth required pam_wheel.so use_uid
```

1.6 Doas

难度 入门 | **篇幅** 4.5 KB | **所属** 安全

本节将讲到

- 安装
- 使用
- 配置
- 提示与技巧

OpenDoas 是 OpenBSD 的 doas 命令的移植版本，与 sudo 相比，它以体积小而闻名。与 *sudo* 一样，*doas* 用于假定系统中另一个用户的身份。

安装

安装 opendoas包。

使用

要以非特权用户的身份开始使用 *doas*，必须对它进行适当配置。参见#配置。

要使用 *doas*，只需在命令及其参数前加上 `doas` 和一个空格：

```
$ doas cmd
```

例如，要使用 pacman：

```
$ doas pacman -Syu
```

要进入具有 root 提示的交互式 shell：

```
$ doas -s
```

更多内容参见 doas(1)：

配置

安装 OpenDoas 会附带 PAM 模块，但没有包括默认配置或示例。

为了允许 wheel 组的成员以其他用户的身份运行命令，请创建一份具有以下内容的配置文件：

```
/etc/doas.conf
permit :wheel
```

注意：配置文件必须以换行结束。

`/etc/doas.conf` 的所有者和组应均是 `0`，文件权限应设置为 `0400`：

```
# chown -c root:root /etc/doas.conf
# chmod -c 0400 /etc/doas.conf
```

要检查 `/etc/doas.conf`是否存在语法错误，请运行：

```
# doas -C /etc/doas.conf && echo "config ok" || echo "config error"
```

警告：必须保证 `/etc/doas.conf` 没有语法错误！

允许 `plugdev` 组的成员作为根用户无需密码运行 smartctl。

```
/etc/doas.conf
permit nopass :plugdev as root cmd /usr/bin/smartctl
```

`/etc/doas.conf` 的一般语法形式为：

```
permit|deny [options] identity [as target] [cmd command [args ...]]
```

更多细节请阅读 doas.conf(5)。

提示与技巧

doas 持久化

doas 提供持久化支持：在用户成功认证后，在一段时间内不再要求输入密码。默认情况下是禁用的，用 `persist` 选项启用它：

```
/etc/doas.conf
permit persist :wheel
```

注意：持久化默认禁用，因为它比较新，有潜在的危险。在最初的 *doas* 中，一个内核 API 被用来设置和清除超时。这个 API 是 OpenBSD 特有，其他操作系统上没有类似的 API。作为一种变通方法，持久化是使用类似于 *sudo* 的时间戳文件实现。

从 sudo 平滑过渡到 doas

为了从 *sudo* 平滑过渡到 *doas* 并保持向下兼容，可以在环境中添加：

```
alias sudo='doas'
alias sudoedit='doas rnano'
```

或者，将 *doas* 链接到 *sudo* 通常所在位置（但是它无法提供`sudoedit`命令）：

```
# ln -s $(which doas) /usr/bin/sudo
```

doas-sudo-shimAUR 或许可以帮您平滑过渡，可惜无法提供`sudoedit`命令。

注意：默认情况下 *sudo* 会保留一些环境变量，而 *doas* 不会，最明显的是 XAUTHORITY、LANG 和 LC_ALL。这意味着你将无法在 X 下启动图形应用程序，也无法在没有额外配置的情况下访问用户的区域设置。例如，要允许 *wheel* 组的成员运行图形应用程序，并使用 setenv 访问用户的 locale：

```
/etc/doas.conf
```

```
permit setenv { XAUTHORITY LANG LC_ALL } :wheel
```

Bash tab 补全

默认情况 Bash 只对当前或参考目录下的文件和目录进行 tab 补全。要让 Bash 像完成单独的命令一样完成参数（同时利用其他命令的 tab 补全设置），可以在用户的 `.bashrc` 或全局的 `/etc/bash.bashrc` 中添加以下内容：

```
~/.bashrc
complete -cf doas
```

1.7 Wireshark

难度 入门 | **篇幅** 5.0 KB | **所属** 安全

本节将讲到

- 安装
- 以普通用户身份抓包
- 一些抓包技巧

Wireshark 是一款自由且开源的包分析器。可用于网络排错、网络分析、软件和通讯协议开发以及教学等。

安装

Wireshark 软件包分成了 CLI 版本和依赖 CLI 版本的前端界面.

- CLI 版本 - 安装软件包 wireshark-cli包。
- TUI 前端 - 安装软件包 termshark包。
- Qt 前端 - 安装软件包 wireshark-qt包。

注意：Wireshark 3.0 已经移除了 GTK 界面

以普通用户身份抓包

以 root 身份运行 Wireshark 是不安全的。Arch Linux 官方仓库中的 Wireshark 已经实现了权限分离，其使用的抓包工具 `/usr/bin/dumpcap` 是唯一有权限进行数据包抓取的进程，仅能被 root 或 `wireshark` 群组用户执行。

wireshark-cli包 的安装脚本还会为 `/usr/bin/dumpcap` 设置数据包捕获能力（capability），使其无需 root 权限也能进行数据包捕获。

如果希望普通用户能够正常运行 Wireshark，必须将其添加到 `wireshark` 用户组中（参见用户和用户组#用户组管理）。

使用如下命令可以将对应用户添加到 `wireshark` 用户组（中文 wiki 注）：

```
# usermod -a -G wireshark 用户名
```

重启或注销对应用户重新登陆即可生效。

一些抓包技巧

你可以通过使用 capture filters 和 display filters 准确获取你想抓取的数据包。

注意：capture filters 语法请参考 pcap-filter(7)，display filters 语法参考 wireshark-filter(4)。

过滤 TCP 包

只查看所有的 TCP 数据包，在 *Filter* 栏输入 `tcp`，或者在终端中输入：

```
$ tshark -f "tcp"
```

过滤 UDP 包

只想查看所有的 UDP 数据包，在 *Filter* 栏输入 `udp`，或者在终端输入：

```
$ tshark -f "udp"
```

查看指定 IP 地址的数据包

将 `1.2.3.4` 替换为要查看的 IP 地址。

- 只想查看发到某个特定地址的数据包，输入 `ip.dst == 1.2.3.4`。
- 只想查看从某个特定地址发出的数据包，输入 `ip.src == 1.2.3.4`。
- 要查看某个特定地址的所有数据包，输入 `ip.addr == 1.2.3.4`。

排除特定 IP 地址的数据包

将 `1.2.3.4` 替换为要排除的 IP 地址。

```
ip.addr != 1.2.3.4
```

查看局域网数据包

如果只想查看局域网数据包，使用

```
 (ip.src==10.0.0/8 AND ip.dst==10.0.0/8) OR (ip.src==172.16.0.0/12 AND
ip.dst==172.16.0.0/12) OR (ip.src==192.168.0.0/16 and ip.dst==192.168.0.0/16)
```

这样可以过滤出所有来自私有网络地址的数据包。

查看指定端口的数据包

查看某两个或更多端口的流量：

```
tcp.port==80||tcp.port==3306
tcp.port==80||tcp.port==3306||tcp.port==443
```

使用 dumpcap 捕获数据包

dumpcap 是 Wireshark 的一部分，可以使用该工具在无图形界面的情况下捕获数据包。*dumpcap* 可以在 tmux 或 GNU Screen 中使用，以在一个处于 Detached 状态的会话中捕获数据包。

可以使用 `-h` 参数查看 dumpcap 的所有选项。

下面的例子展示了如何捕获循环缓冲区（ringbuffer）中内容。下面的命令会以每个文件 100MB 的大小捕获20个`.pcap`文件，并且会在捕获第21个文件时替换最早创建的文件并依次继续。因此此命令会保持运行，而不会持续消耗磁盘空间以致磁盘容量耗尽。

```
# dumpcap -i 1 -b filesize:100000 -b files:20 -w mycapture.pcapng
```

- `-i` - 接口名称或编号（`dumpcat -D` 命令列出的编号）
- `-b filesize:` - 创建的`.pcap`文件的大小，以 kB 为单位
- `-b files:` - 在输出指定的文件数量后覆盖旧文件
- `-w` - 输出文件名，上例中输出会保存到 `mycapture*identifier*.pcapng`

1.8 Capabilities

难度 进阶 | **篇幅** 6.4 KB | **所属** 安全

本节将讲到

- 实现
- 管理和维护
- 其他从capabilities受益的程序
- 实用命令
- 为程序临时授予能力

能力（**capability**）是 POSIX 1003.1e 标准引入的功能，手册说明参见 capabilities(7)。它允许使用更小的粒度控制超级管理员权限，可以避免 root 权限的滥用。软件开发者应当尽可能为二进制文件赋予最小权限，而不是过度使用强大的 setuid。Arch Linux 中很多软件包用了 capability，例如 fping包 中的 `fping` 就使用了 `CAP_NET_RAW` （能力的一种）。这样一来，普通用户也可以正常执行 `fping`，效果与使用 **setuid** 相似，同时减少了 `fping` 的潜在安全隐患。

实现

在 Linux 中，能力通过 *security* 名空间下的扩展属性（详见xattr(7)）实现。主流 Linux 文件系统均支持扩展属性 ，包括Ext2，Ext3，Ext4，Btrfs，JFS，XFS 和 Reiserfs 等。

下面的示例使用 `getcap` 显示“fping”的能力（Capabilities），以及通过 `getfattr` 显示其编码后的“fping”的能力信息。

```
$ getcap /usr/bin/fping
/usr/bin/fping = cap_net_raw+ep
$ getfattr -d -m "^security\\." /usr/bin/ping
# file: usr/bin/ping
security.capability=0sAQAAAgAgAAAAAAAAAAAAAAAAAAA=
```

部分程序能够自动保留文件的扩展属性，但更多的情况下需要单独指定参数，详情可以参考页面文件权限与属性#保留扩展属性。

在 Arch 中, 能力可以通过包的安装脚本（如 fping包 中的 ）设置。

管理和维护

如果一个软件包设置了过度宽松的能力，那应当被视为一个错误（bug）并上报, 而不是在这里列举出来。 同时，因为Arch不支持 MAC/RBAC ，所以设置 `CAP_SYS_ADMIN` 或 `CAP_DAC_OVERRIDE` ，不应当被视为错误（bug）。

警告：过多的能力会可能会导致权限问题。可以在 Brad Spengler的帖子中查看例子和解释：False Boundaries and Arbitrary Code Execution.

其他从capabilities受益的程序

下面列举的软件中没有使用 setuid 属性的文件，但是他们中的部分命令需要 root 权限才能正常使用。通过设置能力，普通用户可以在不提升权限的情况下使用这些软件。

下文中能力后面的 `+ep` 表示相关能力标记为*生效（effective）*和*允许（permitted）*。更多信息请参阅 capabilities(7) § File capabilities

软件	执行命令（以 root 权限执行）
beep(1)	`setcap cap_dac_override,cap_sys_tty_config+ep /usr/bin/beep`
chvt(1)	`setcap cap_dac_read_search,cap_sys_tty_config+ep /usr/bin/chvt`
iftop(8)	`setcap cap_net_raw+ep /usr/bin/iftop`
mii-tool(8)	`setcap cap_net_admin+ep /usr/bin/mii-tool`
nethogs(8)	`setcap cap_net_admin,cap_net_raw+ep /usr/bin/nethogs`
wavemon(1)	`setcap cap_net_admin+ep /usr/bin/wavemon`

在 ArchLinux 中，部分软件（例如 mtr包）已经在其打包所使用的 `.install` 文件中配置了所需的“能力”。 因此部分软件不需要手动执行上面的命令。

实用命令

找到具有 setuid-root 权限的文件:

```
$ find /usr/bin /usr/lib -perm /4000 -user root
```

找到具有 setgid-root 权限的文件:

```
$ find /usr/bin /usr/lib -perm /2000 -group root
```

为程序临时授予能力

capsh(1)可以以指定的能力运行程序，而不需要修改文件扩展属性。

下面的例子展示了如何为 gdb(1) 赋予 CAP_SYS_PTRACE 能力来调试程序：

```
$ sudo -E capsh --caps="cap_setpcap,cap_setuid,cap_setgid+ep cap_sys_ptrace+eip" --
keep=1 --user="$USER" --addamb="cap_sys_ptrace" --shell=/usr/bin/gdb -- -p <pid>
```

上面的命令用 -E 参数是 sudo 命令中，保留当前用户环境变量的参数。详细说明参阅 sudo(8)。

nc(1) 监听低端口号的例子：

```
$ sudo -E capsh --caps="cap_setpcap,cap_setuid,cap_setgid+ep cap_net_bind_service+eip"
--keep=1 --user="$USER" --addamb="cap_net_bind_service" --shell=/usr/bin/nc -- -lvtn
123
Listening on 0.0.0.0 123
```

systemd

可以使用 AmbientCapabilities 和 CapabilityBoundingSet 为 systemd 单元分配能力，相较于在二进制文件上设置能力，这种方式更安全。参见 systemd.exec(5)。

参阅

- Man Pages: capabilities(7), setcap(8), getcap(8)
- Wikibooks:Grsecurity/Appendix/Capability Names and Descriptions
- Seccomp BPF (SECure COMPuting with filters)

1.9 实时进程管理

难度 进阶 | **篇幅** 6.6 KB | **所属** 安全

本节将讲到

- 配置
- 硬实时与软实时
- 权能管理
- 技巧和窍门
- See also

本文提供了在实时运行时确定进程线程优先级的有关信息，而不仅限于启动时。本文将展示如何控制单个进程或特定组运行的所有进程的CPU、内存和其他资源的利用率。

虽然许多最近的处理器功能强大到足以同时播放十几个视频或音频流，但仍有可能会有一个线程占用处理器半秒以完成另一项任务。 这会导致音频或视频流的短暂中断，视频或音频流也可能因此而不同步。 这对于休闲音乐听众来说很烦人，但对于内容制作者、作曲家或视频编辑者来说，这个问题要严重得多，因为它会打断他们的工作流程。

简单的解决方案是为音频和视频进程提供**更高的优先级**。虽然普通用户可以为进程设置更高的*nice*值——这意味着该进程的优先级较低，但只有 root 可以设置较低的值并以低于 0 的*nice*值启动进程。这可以保护普通用户不会影响到那些耗时较长而对系统而言很重要的进程。这在多用户机器上尤为重要。

配置

Arch Linux 默认启用实时优先级排序。可以使用PAM和systemd实现对系统、组和用户范围的配置。

realtime包组包组提供了额外的工具来修改中断请求)和进程的实时调度政策。

注意：当使用自定义内核时，需确保支持`PREEMPT`以使用上述方法。

配置PAM

`/etc/security/limits.conf`文件提供了对`pam_limits`PAM模块的配置。`pam_limits`设定了系统资源的调用上限(参见 limits.conf(5))。

提示： It is advised to externalize the configuration of `pam_limits` to separate files below `/etc/security/limits.d` as those take precedence over the main configuration file.

`pam_limits`提供了两种类型的资源限制**hard limits**和**soft limits**。硬件限制由`root`设置，并由内核强制实施。软件限制可以被用户配置，需要在硬件限制允许的范围内。

安装realtime-privileges包包，把用户添加到`realtime`组，并提供合理的默认值（参考示例Professional audio)。

配置systemd服务

由systemd系统服务生成的进程需要进行专门的配置以与`limits.conf`匹配。更多信息可以参考systemd.exec(5)中的systemd.exec(5) § CREDENTIALS和systemd.exec(5) § PROCESS PROPERTIES两节。

硬实时与软实时

实时的含义是能够及时运行而不会被其他的任何进程中断的进程。尽管如此，这个进程的运行周期也可能被抛弃，电源不足或更高优先级的进程可能是导致这个错误的潜在原因。为了解决这个问题，会对实时的质量进行放缩。这篇文章主要考虑**软**实时。硬实时与其说被需要，不如说被期待。

权能管理

realtime-lsm 模块授予属于特定 UID 的用户获得更高capabilities的权利。 rlimit 方式的工作原理类似，但它可以控制得更精细。PAM 中有一项新功能，可用于控制每个用户或每个组级别的权能。 在当前版本 (0.80-2) 中，这些值没有被设置为开箱即用，仍然会产生问题。使用 PAM，您可以将实时优先权授予特定用户或特定用户组。 PAM 的概念使人们可以想象，将来会有在每个应用程序级别上授予权限的方法，然而至今，这还不可能。

技巧和窍门

支持PAM的登录

参见 Start X at login.

为了使您的系统使用PAM limit设置，您需要使用支持PAM的登录方法或登录管理器。几乎所有的图形化登录管理器都是支持PAM的，Arch默认的登录方式也支持PAM。您可以在`/etc/pam.d`中搜索相关设置:

```
$ grep pam_limits.so /etc/pam.d/*
```

如果您没有得到返回，说明您现在的登录方式没有使用PAM。您可以使用一个登录管理器来使用PAM(例如PolicyKit)。我们期待像这样的输出：

```
/etc/pam.d/crond:session    required     pam_limits.so
/etc/pam.d/login:session           required      pam_limits.so
/etc/pam.d/polkit-1:session         required        pam_limits.so
/etc/pam.d/system-auth:session    required  pam_limits.so
/etc/pam.d/system-services:session    required     pam_limits.so
```

我们看到`login`, PolicyKit, 和其他的应用都需要pam_limits.so模块，这说明PAM的限制被正确的实施。

Console/autologin

参见: Automatically log in some user to a virtual console on startup

如果您希望不使用图形化登录，您仍有实施PAM limit的方法。您需要编辑`pam`对`su`(由coreutils包提供)的配置文件：

```
/etc/pam.d/su
 ...
 session               required        pam_limits.so
```

参见此论坛 post.

See also

- getrlimit(2)
- https://docs.kernel.org/scheduler/sched-design-CFS.html
- https://docs.kernel.org/block/ioprio.html
- IO Benchmarking: How, why and with what

- SYSSTAT utilities home page

1.10 Umask

难度 进阶 | **篇幅** 8.3 KB | **所属** 安全

本节将讲到

- 模式掩码的含义
- 显示当前掩码值
- 设置掩码值
- 参见

掩码工具用于控制文件创建模式掩码，它决定新创建文件的文件权限位的初始值。这个工具的行为是由 POSIX 标准化，并在 POSIX 开发者手册中描述。因为*掩码值*会影响当前的 shell 执行环境，所以它通常作为 shell 的内置命令实现。

模式掩码的含义

模式掩码包含在新创建的文件上不应该设置的权限位，因此它是在新创建的文件上设置的权限位的逻辑非。如果掩码值中的某些位被设置为`1`，新创建文件的相应权限将被禁用。因此，掩码值就像一个过滤器，可以剥离权限位，有助于设置文件的默认访问权限。

在新创建的文件上设置权限位的结果是用位法实质非蕴涵（也称为否定）计算的，可以用逻辑符号表示。

```
R: (D & (~M))
```

也就是说，产生的权限 `R` 是默认权限 `D` 的位与和文件创建模式屏蔽 `M` 的位非的结果。

注意：

- 目录的默认创建权限为`777`。
- 因为 Linux 不允许以执行权限创建文件，文件的默认创建权限为`666`。
- 在Linux下，只使用掩码值的文件权限位--见 umask(2)。掩码值中的 *suid*、*sgid* 和 *sticky* 位被忽略。

例如，让我们假设文件创建模式掩码为`027`。这里，每个数字的位数表示为：

- `0`代表未在新创建的文件上设置的*用户*权限位
- `2`代表新创建文件未设置的*组*权限位
- `7`代表新创建文件未设置的*其他*权限位

根据下表提供的信息，这意味着对于一个新创建的文件，例如由`User1`用户和`Group1`组所拥有，`User1`对新创建的文件拥有所有可能的权限（八进制值`7`）。`Group1`组的

其他用户没有写权限（八进制值5），而其他用户对新创建的文件没有任何权限（八进制值0）。因此，在本例中采取027掩码值，文件将以750的权限创建。

<table class="wikitable"><caption></caption><tbody><tr><th>八进制</th><th>二进制</th><th>含义
</th></tr><tr><td><code>0</code></td><td><code>000</code></td><td>无权限
</td></tr><tr><td><code>1</code></td><td><code>001</code></td><td>只执行
</td></tr><tr><td><code>2</code></td><td><code>010</code></td><td>只写
</td></tr><tr><td><code>3</code></td><td><code>011</code></td><td>可写和执行</td></tr><tr><td><code>4</code></td><td><code>100</code></td><td>只读
</td></tr><tr><td><code>5</code></td><td><code>101</code></td><td>可读和执行</td></tr><tr><td><code>6</code></td><td><code>110</code></td><td>可读和写</td></tr><tr><td><code>7</code></td><td><code>111</code></td><td>可读，写和执行</td></tr></tbody></table>

显示当前掩码值

要显示当前掩码值，只需调用 `umask`，不需要指定任何参数。默认的输出方式取决于实现，但通常为八进制。

```
$umask
0027
```

当使用POSIX标准化的 `-S` 选项时，掩码值将使用符号表示来显示。然而，**符号表示值总是用八进制值的逻辑补码**，即在新创建的文件上要设置的权限位：

```
$ umask -S
u=rwx,g=rx,o=
```

设置掩码值

注意：可以根据具体情况设置掩码值。例如，桌面用户可能会发现对其主文件夹的限制权限已足够（`useradd -m` 默认创建具有 `700` 权限的目录），因为其他人无法访问其中的所有文件。如果这不切实际（例如，在使用 Apache HTTP Server 时），并且公共文件存储在私有文件中，那么请考虑限制掩码值。

可以通过 *umask* 命令设置掩码值。指定模式掩码的字符串遵循与 chmod 的模式参数相同的语法规则（详见 POSIX 开发者手册）。

全系统的掩码值可以在 `/etc/profile` (e.g. `/etc/profile.d/umask.sh`) 或默认的 shell 配置文件中设置，比如 `/etc/bash.bashrc`。大多数 Linux发行版，包括 Arch，都在 `/etc/login.defs` 将掩码值的默认值设置为`022` 。也可以用 `pam_umask.so`设置掩码值，但它可能被 `/etc/profile` 或类似文件覆盖。

如果需要设置不同的值，可以直接编辑该文件，从而影响所有用户，或者从 shell 的用户配置文件中调用 `umask`，例如 `~/.bashrc`，只修改你的掩码值，但这些修改只在下次登录后生效。要想只在当前会话中改变掩码值，只需运行 `umask` 并输入你想要的值。例如，运行 `umask 077` 会给你新文件的读写权限，以及新文件夹的读写和执行权限。

如 pam_umask(8) § DESCRIPTION 中提到的, `umask=value` 也可以被用在 `/etc/passwd` 中的 Users and groups#User database 部分。参见 setting UMASK in GECOS field 的讨论。

设置 KDE / Plasma 的掩码值

通过 `/etc/profile` 设置掩码值对 KDE / Plasma 会话不再有效，因为这些会话是以 systemd 用户单元的形式启动。

掩码值可以通过 `pam_umask.so` 或 systemd 的附加配置片段设置。

```
/etc/systemd/system/user@.service.d/override.conf
[Service]
UMask=0002
```

使用 `pam_umask.so` 可以在一个环境为文本控制台和图形 KDE 会话设置全系统的掩码值。在 `/etc/profile` 或 systemd 配置中的任何改动都可以省略。因此，需要在 `/etc/pam.d/login` 和 `/etc/pam.d/systemd-user` 都包含的配置文件中启用 `pam_umask.so`。

在 `/etc/pam.d/system-login` 中添加以下一行：

```
# session    optional   pam_umask.so          umask=022
```

Site-specific

编辑 `/etc/login.defs` 并更新 `UMASK` 值。例如：

```
/etc/login.defs
UMASK 077
```

参见

- POSIX 开发者手册:
- 掩码值(可在 umask(1p) 阅读)
- chmod（扩展说明）(可在 chmod(1p) 阅读)
- 027 掩码值：安全和简单的折中方案

1.11 MAC 地址欺骗

难度 进阶 | **篇幅** 8.6 KB | **所属** 安全

本节将讲到

- 手动更改
- 自动更改
- 故障排除
- 另见

这篇文章文章将会介绍几个修改 MAC 地址的方法。

手动更改

有两种方法可以修改 MAC 地址：安装并配置 iproute2包 或 macchanger包。下面来说明一下这两种方法。

iproute2

首先，你可以用下面的命令来检查当前的 MAC 地址

```
# ip link show interface
```

`*interface*` 是你的网卡的名字

我们现在要关注的是跟在“link/ether”后面的那一串带冒号的十六进制字节。它看起来可能是这样：

```
link/ether 00:1d:98:5a:d1:3a
```

修改 MAC 地址的第一步是禁用网卡，它可以通过下面的命令来完成：

```
# ip link set dev interface down
```

接下来，我们要开始修改我们的 MAC 地址。只要每个字节都是十六进制值就可以，但有的网络运营商可能会拒绝为不正确的 MAC 分配 IP 地址。所以，除非你是你连接的网络的管理员，否则你应该使用真实的 MAC 地址前缀（一般是前三个字节），剩下三个字节可以随便设置（只要是十六进制值）。如果想了解更多内容，请访问 Wikipedia:Organizationally unique identifier.

要更改 MAC 地址，我们要运行这个命令：

```
# ip link set dev interface address XX:XX:XX:XX:XX:XX
```

这6位 `*XX:XX:XX:XX:XX:XX*` 就是你要设置的 MAC 地址。

最后一步是重新启用网卡，输入这行命令：

```
# ip link set dev interface up
```

如果你想验证你的 MAC 地址是否成功修改，只需要再次运行 `ip link show *interface*` ，然后检查“link/ether”后面的值。如果成功修改，“link/ether”后面应该跟着你刚刚设置的 MAC 地址。

macchanger

另一个方法是通过 macchanger包 (a.k.a., the GNU MAC Changer)。它有一些方便的功能，比如改变 MAC 地址以匹配某个运营商，或者完全随机化地址。

从官方仓库里安装这个包 macchanger包 .

由于更改 MAC 地址基于网卡，我们需要用网卡名来替换每行命令中的 `*interface*` 。

用这行命令我们可以将 MAC 地址完全随机化：

```
# macchanger -r interface
```

要随机化当前 MAC 地址的后三位字节 (这样运营商会认为这个 MAC 地址是注册过的 MAC 地址，就可以避免被断网的风险)，你可以运行这个命令

```
# macchanger -e interface
```

要把 MAC 地址改成指定的值，请运行：

```
# macchanger --mac=XX:XX:XX:XX:XX:XX interface
```

把 `*XX:XX:XX:XX:XX:XX*` 改成你想要的 MAC 地址。

最后，如果想把 MAC 地址恢复成出厂值，运行这个：

```
# macchanger -p interface
```

注意：在更改 MAC 地址的时候，设备将无法使用（无论是以任何方式连接，或是试图启用这个设备）

自动更改

systemd-networkd

systemd-networkd 支持通过 link files 设置 MAC 地址（细节请查看 systemd.link(5)）.

要设置一个静态 MAC 地址，修改这个文件为：

```
/etc/systemd/network/00-default.link
[Match]
MACAddress=原始 MAC

[Link]
MACAddress=更改后的 MAC
NamePolicy=kernel database onboard slot path
```

如要在每次启动时随机化 MAC 地址，把 `MACAddress=*更改后的 MAC*` 改成 `MACAddressPolicy=random`。

systemd-udevd

udev 允许你创建 udev 规则 来更改 MAC 地址。使用 `address` 参数来用原始 MAC 地址来匹配设备，然后用 *ip* 命令来更改 MAC 地址:

```
/etc/udev/rules.d/75-mac-spoof.rules
ACTION=="add", SUBSYSTEM=="net", ATTR{address}=="XX:XX:XX:XX:XX:XX", RUN+="/usr/bin/ip link set dev %k address YY:YY:YY:YY:YY:YY"
```

其中，`XX:XX:XX:XX:XX:XX` 是原始 MAC 地址，`YY:YY:YY:YY:YY:YY` 是目标 MAC 地址

systemd 单元

创建单元

下面写了两个用 Systemd 来在启动时更改 MAC 地址的例子，其中一个用 *ip* 来设置静态 MAC，另一个用 *macchanger* 来设置随机 MAC。systemd 的 `network-pre.target` 可以确保 MAC 地址在网络管理器如 netctl、NetworkManager、systemd-networkd 或 dhcpcd 启动之前就已经更改好。

iproute2

设置静态 MAC 地址的 Systemd 单元:

```
/etc/systemd/system/macspoof@.service
[Unit]
Description=MAC Address Change %I
Wants=network-pre.target
Before=network-pre.target
BindsTo=sys-subsystem-net-devices-%i.device
After=sys-subsystem-net-devices-%i.device

[Service]
Type=oneshot
ExecStart=/usr/bin/ip link set dev %i address 36:aa:88:c8:75:3a
ExecStart=/usr/bin/ip link set dev %i up

[Install]
WantedBy=multi-user.target
```

macchanger

设置随机 MAC 地址的 Systemd 单元，同时保留原始的运营商字节。请确保您已安装了 macchanger包:

```
/etc/systemd/system/macspoof@.service
[Unit]
Description=macchanger on %I
Wants=network-pre.target
Before=network-pre.target
BindsTo=sys-subsystem-net-devices-%i.device
After=sys-subsystem-net-devices-%i.device

[Service]
ExecStart=/usr/bin/macchanger -e %I
Type=oneshot

[Install]
WantedBy=multi-user.target
```

或者用 `-r` 选项来使 MAC 地址完全随机化，参见 #macchanger。

启用服务

将所需的网络接口 (如：`eth0`) 附加到服务名称后面 (如 `macspoof@eth0.service`)，然后启用服务.

重启，或者按照适当的顺序重启依赖的服务。如果你是局域网管理员，请通过路由器检查其中的静态或 DHCP 地址表，验证 MAC 是否已成功修改。

netctl 接口

你可以使用 netctl hook 来在每次启动或重启网卡的时候运行特定命令。把 `*interface*` 替换为你的网络接口:

```
/etc/netctl/interfaces/interface
#!/usr/bin/env sh
/usr/bin/macchanger -r interface
```

使脚本可执行:

```
chmod +x /etc/netctl/interfaces/interface
```

来源: akendo.eu

NetworkManager

参见 NetworkManager#Configuring MAC address randomization.

故障排除

连接到 DHCPv4 网络失败

如果您无法连接到 DHCPv4网络，而且您使用的是 NetworkManager 默认的 dhcpcd，你可能需要修改 dhcpd 配置来续租。

另见

- Wikipedia:MAC spoofing
- Macchanger GitHub page
- DebianAdmin 上的文章 with more *macchanger* options

1.12 安全地擦除磁盘/提示和技巧

难度 进阶 | **篇幅** 14.3 KB | **所属** 安全

本节将讲到

- 擦除单个文件
- 覆写目标
- 擦除可用空间
- 另见

主文章请见安全地擦除磁盘。

本文额外介绍了一些擦除工具，可替代专用擦除工具，并加快擦除速度。

警告：某些文件系统的保留块不能通过创建文件的方式擦除，但可以使用工具 `tune2fs` 禁用。

擦除单个文件

警告：如果曾对存储文件的分区进行了 碎片整理、调整大小或移动操作，或者文件曾在同一设备上被复制过，那么擦除效果将受到影响。若数据存储在加密容器上，并擦除整个加密容器，则文件恢复将变得困难得多，但加密可能会大幅降低 磁盘性能（除非使用硬件加密）。另见 安全地擦除磁盘#数据残留。

注意：要在擦除时保留文件的访问和修改时间不变，在修改文件前先使用stat(1)获取文件当前的时间信息，之后可使用touch(1)手动修改文件时间。另请参阅不同文件系统对元数据和时间戳的支持情况。

本节将介绍擦除单个文件的两种基本方法。此外，还有一种专门用于反取证的高级方法，其非常耗时，并且只能通过专用工具完成，本文不作介绍。

- 在删除前用随机数据覆写，或在不改变文件大小的情况下用其他内容替换原内容。
- 使用专用工具擦除文件和文件系统存储的元数据
- 搜索整个磁盘，找到并清除已删除文件的残留部分，同时不对其它文件造成影响。

可以使用常见的 Linux 工具在不改变文件大小的情况下覆写文件：

- 使用 `shred -x <目标文件>`，`-x`选项会用伪随机数据替换文件的内容，且不改变文件大小。`-u` 选项会在覆写后删除文件。
- 使用 `mkfs` 可以将文件格式化为文件系统，挂载该文件系统后，任何对文件系统的修改都将反映到文件上，在文件系统上存储其它内容，可达到覆盖原始文件内容的目的。
- `dd` 可创建指定大小、内容的文件，若目标文件已存在，则直接会覆写。通过组合 `skip` 和 `seek` 选项，可以将整个文件或文件的一部分替换为其他内容。在操作前，需要知道文件大小，以避免意外扩容文件。文件大小可以使用 `du -b <文件名> | cut -f1` 或 `stat -c "%s" <文件名>`获取。注意使用dd时，必须指定 `iflag=fullblock` 选项，详见 Dd#部分读取：复制的数据量小于要求的大小。
- 使用 perl包 工具可以用单个符号（symbol）替换文件内容，同时避免文件大小发生变化。

要擦除文件的元数据，可以不断创建新文件，直到填满分区，这将迫使文件系统覆盖旧的文件元数据条目，以存储新创建文件的条目。此外，也可以使用专用工具。详见#擦除可用空间。

组合使用提到的文件创建和转换工具擦除文件内容，可达到避免文件被恢复、误导恢复工具及其使用者（可通过覆写随机数据或预先定义好的其它内容实现）的目的。具体的组合方式可依情况和需求而定。

注意：

- 若存储文件的文件系统还有可用空间，则简单的覆写操作可能无法清除文件的某些部分。
- 若要覆写特定文件的内容，同时确保该文件在硬盘上的逻辑位置不变，可以先创建一个或多个文件将文件所在分区填满，再使用合适的工具重写或替换目标文件的内容。不过，如果分区的可用空间很多，而要覆盖的文件很少，这会消耗大量时间。

例如：

使用Perl，用 . 替换文件中所有内容的：

```
$ perl -p -i -e 's\[^*]\.\g' <文件名>
```

使用dd：

```
$ <覆盖内容> | dd bs=<目标文件大小（字节）> count=1 iflag=fullblock of=<目标文件> seek=0
```

或者，也可使用stdout重定向。这种方式在创建文件时速度更快，但无法使用 `seek` 选项来跳过目标文件中的某些部分：

```
$ <覆盖内容> | dd bs=<目标文件大小（字节）> count=1 iflag=fullblock > <目标文件>
```

提示：如果覆盖内容长度小于目标文件，需将其与下文介绍的 `while` 循环结合使用。

另见：

- 转换文本文件的字符集
- Advanced Bash-Scripting Guide - 描述使用 bash 脚本创建文件的高级方法
- sed alternatives - 一个 perl 示例，在替换文件内容方面比 sed 效果更好

覆写目标

防止擦除已挂载的分区

选择要擦除的设备时需要格外小心，简单的拼写错误就可能导致错误的目标被擦除，从而损坏系统。为了降低风险，可以使用脚本来包装选定的擦除工具。例如：

```
if [[ -e "$1" && -b "$1" ]];then
NOT_safe="$(lsblk -o "NAME,MOUNTPOINT" ${1//[0-9]/} | grep -e / -e '\]')";
 if [[ -z "$NOT_safe" ]];then
# 在这里你可以使用任何擦除工具擦除目标
# 擦除目标由命令行传入，并存储在变量"$1"中
#
  else
  echo 'Not allowed to destroy if any of the partitions is mounted: '"$NOT_safe"
 fi
fi
```

使用dd进行高级擦除

在使用dd时，可以利用OpenSSL，使用随机密码进行AES加密，并将密文用于擦除目标，以实现随机擦除：

```
# DEVICE="/dev/sdX"
# PASS=$(tr -cd '[:alnum:]' < /dev/urandom | head -c128)
```

```
# openssl enc -aes-256-ctr -pass pass:"$PASS" -nosalt < /dev/zero | dd obs=64K ibs=4K of=$DEVICE oflag=direct status=progress
```

上述命令使用/dev/urandom生成128字节的加密密钥，并使用 CTR 模式的 AES-256 算法来加密 /dev/zero 的输出。相比直接使用伪随机源，使用加密算法可能可以获得更快的写入速度。最终设备将被 AES 密文填充。

上述命令将块大小设置为 64K，通常比默认的 512 字节更快。还可以尝试更大的块大小，以找到适合硬件的最佳传输速率。参考 及其中的参考文献。

类似的方法还可参见 Dm-crypt/准备磁盘#dm-crypt_擦除空设备或分区。

使用模板文件

除了用零填充，还可以使用内容无关紧要的文件进行填充。若使用mkfs格式化了目标文件，需要挂载该文件系统，并使用特定内容填满该文件系统（可以使用无关紧要的文件，或相关工具生成的重复输出）。

一种方法是反复使用模板文件进行擦除，直到设备末尾。若使用此方法，当出现到达设备末尾的错误时，必须手动使用停止键来中断 while 循环，因而不推荐使用：

```
$ while [ 1 -lt 2 ];do cat file1-to-use.as-template file2-to-use.as-template /tmp/templatefiles/* ;done > /dev/sd"XY"
```

若使用dd，如果通过选项正确设置了要擦除的大小，就可以安全地进行重复擦除，而不会出现空间不足的错误。这可通过将while循环产生的模板文件输出作为dd的输入实现。此外，通过设置skip 和 seek 选项的值，还能确定要擦除的范围。除了例子中使用的cat外，还可以使用head(1)、tail(1)将模板文件的部分内容输出到标准输出。

```
while [ 2 -gt 1 ]; do
 if [ -z "$(pidof dd)"  ];then
  break ;
 fi;
cat file1-to-use.as-template file2-to-use.as-template /tmp/templatefiles/* ;
done | dd of=/dev/sd"XY" seek=<起始扇区> bs=<扇区大小> count=<擦除扇区数量>
```

提示：若要重复覆写同一区域，可以在 while ... done | dd ... 外层新增一个循环，使用新增循环的循环变量，在每次循环中使用不同的模板文件进行覆写。

注意：如果用于覆写的预定义模板文件数量很多，可以利用 SquashFS 来压缩文件和文件夹，并存放到挂载的 内存盘 中。这样能最大限度地减少磁盘读取次数，但同时会增加 CPU 使用率。此外，也可以为模板文件创建压缩包，将压缩包复制到内存盘中，并使用能够将解压后的内容输出到标准输出的解压缩工具。使用压缩包的缺点是，难以在每次覆写时只是用压缩包中的部分模板文件。不过，使用压缩包也存在优点：若在创建压缩包时使用了分卷压缩，可以直接将某一分卷作为覆写内容覆写文件

另见：

- sector size - 文件创建和扇区大小

擦除可用空间

警告：若正在为块设备加密准备可用空间，不适合使用此方法。

可以通过以下几种方式擦除可用空间：

- 将输出重定向到一个文件，而不是分区或设备。
- 在循环中，使用随机文件名或目录名创建多个文件副本（例如，使用cp），直到占满所有可用空间。
- 使用带有文件加密功能的工具，并使用随机密码、随机文件名进行加密。某些文件压缩工具支持加密功能，且可以通过选项设置压缩方法、文件类型等，有的还支持分卷压缩。通过使用循环，在循环中随机设置压缩选项，可以用加密数据填满可用空间，以覆写以前的数据。
- 使用专门用于擦除可用空间的程序，例如：

wipefreespace — 安全地擦除文件系统上的可用空间，以防止已删除的敏感数据被恢复。

: https://sourceforge.net/projects/wipefreespace/ || wipefreespaceAUR

zerofree — 扫描并以零填充文件系统中的非零空闲块

: https://frippery.org/uml/ || zerofreeAUR

使用 dd

可以使用 dd 创建一个文件来填满可用空间：

```
dd if=<填充数据源> of=junk
sync
rm junk
```

<填充数据源> 可以是 /dev/urandom 随机流或 /dev/zero 全零流。

注意需要确保数据已同步到磁盘后，再删除该文件。

使用 7-Zip

```
Password="$(dd if=/dev/random bs=128 count=1 iflag=fullblock)"
DestinationFile="$(${RANDOM/0/1}$(date "+%s")/${RANDOM/0/1}))"
7z a -t7z -mhe=on -p"${Password}" -mx=0 -v1m ${DestinationFile} source
```

所用选项的详细说明参见 7z --help。

source 可以是一个包含随机数据的文件，也可以是一个设备（例如 /dev/urandom），或者另一个块设备或其上的分区（例如 /dev/sd"XY"）。注意当使用块设备时，该块设备中不应含有敏感数据，并且，块设备中已在文件系统层面删除的数据也可能会被读取并压缩，以填充目标位置。

注意：

- 不必设置压缩级别，只需将数据存储到压缩文件中即可，以最大限度地减少 CPU 使用率并更快地填满可用空间。
- 如果使用单个文件作为源，可以将其放入 内存盘 或 /tmp 文件夹。/tmp文件夹使用tmpfs，用于将文件存储到内存中，可加快读取速度。

借助 timeout 命令创建多个文件

若在循环中使用timeout命令，并设置随机的等待时间。则timeout将在随机时间后中断文件的创建，从而使得创建的文件大小随机化。不过该方法速度较慢。

在循环中使用带有随机等待时间的 timeout 命令会中断命令，从而留下一个随机大小的文件。这是一种很慢的方法，但也是一种可行的替代方案。此外，对于文件名，可以在随机部分之前附加特定的预定义前缀。

```
AA=${RANDOM/0/1};
timeout $((AA/100)) cat /dev/urandom > filename${RANDOM}.tmp;
```

另见

- limits for the file creation on the different file systems.
- meta-data in the filesystem may keep information about file after it was deleted.
- forensic software uses meta-data to recovery and what need to do for wiping of the meta-data.

1.13 Polkit

难度 进阶 | **篇幅** 14.9 KB | **所属** 安全

本节将讲到

- 安装
- 配置
- 范例
- 参阅

来自 polkit 主页：

: polkit 是一个应用程序级别的工具集，通过定义和审核权限规则，实现不同优先级进程间的通讯：控制决策集中在统一的框架之中，决定低优先级进程是否有权访问高优先级进程。

Polkit 在系统层级进行权限控制，提供了一个低优先级进程和高优先级进程进行通讯的系统。和 sudo 等程序不同，Polkit 并没有赋予进程完全的 root 权限，而是通过一个集中的策略系统进行更精细的授权。

Polkit 定义出一系列操作，例如运行 GParted, 并将用户按照群组或用户名进行划分，例如 wheel 群组用户。然后定义每个操作是否可以由某些用户执行，执行操作前是否需要一些额外的确认，例如通过输入密码确认用户是不是属于某个群组。

安装

安装 polkit包 包。

身份认证组件

Polkit 的权限管理是基于用户或群组进行配置，而身份认证组件的作用就是让会话用户证明自己是某个用户或属于某个群组。

图形化环境Cinnamon、Deepin、GNOME、GNOME Flashback、KDE、LXDE、LXQt、MATE、theShell、Xfce 和 Hyprland 各自都已有认证组件。请按照下列清单确认安装了对应的身份认证组件，并且在登录时自动启动它。

其他桌面环境需要从下列实现中选用一种，polkit包 软件包提供了一个名为“pkttyagent”的基于文本方式的认证代理，作为后备方案。

- lxqt-policykit包，提供了 `/usr/bin/lxqt-policykit-agent`
- lxsession包，提供了 `/usr/bin/lxpolkit`
- mate-polkit包，提供了 `/usr/lib/mate-polkit/polkit-mate-authentication-agent-1`
- polkit-efl-gitAUR，提供了 `/usr/bin/polkit-efl-authentication-agent-1`
- polkit-gnome包，提供了 `/usr/lib/polkit-gnome/polkit-gnome-authentication-agent-1`
- polkit-kde-agent包，提供了 `/usr/lib/polkit-kde-authentication-agent-1`
- ts-polkitagentAUR, 提供了 `/usr/lib/ts-polkitagent`
- xfce-polkit-gitAUR，提供了 `/usr/lib/xfce-polkit/xfce-polkit`
- hyprpolkitagent包，提供了 `/usr/libexec/hyprpolkitagent`

配置

警告：不要更改包文件的默认权限，因其可能在软件包更新时被覆盖。

Polkit 定义了两种不同的内容：

- **操作（Actions）**：在 `/usr/share/polkit-1/actions` 中定义，文件是 XML 格式，以 `.policy` 结尾。每个**操作**都有一个默认的权限集合（例如，你

需要标识为管理员以使用 GParted 操作）。默认值是可以修改的，但是不应该通过修改操作文件实现。

- **认证规则（Authorization rules）**：用 JavaScript 语法定义，文件以 `.rules` 结尾。有两个目录可放置规则文件：第三方的包将文件放置在 `/usr/share/polkit-1/rules.d`（尽管很少见），本地配置应该放置在 `/etc/polkit-1/rules.d`。

Polkit 没有取代系统已有的权限系统，而是在已有的群组和管理员上进行管控。**.rules** 文件指定了一个用户的子集合，涉及到一个或多个**操作**文件中指定的操作，并规定这些用户可以执行哪些操作，需要满足哪些限制。举例来说，GParted 默认规则要求所有用户认证为管理员之后才能使用，可以用规则文件修改默认规则，规定某个用户不需要管理员身份认证就可以执行操作，也可以完全禁止某个用户使用 GParted。

注意：如果用户不是通过 polkit 申请权限，比如通过命令行直接以 root 权限执行，这里的禁止设定就无法起作用。所以应该用 polkit 给低权限用户更高的权限，而不应该用 polkit 限制高权限用户可以执行的操作。出于安全考虑，sudoers仍然是一种方法。

操作

提示：要在图形程序中显示 Polkit 操作，可以安装软件包 polkit-explorer-gitAUR。

polkit 中的可用操作是安装的软件包决定的。有些在多种桌面环境下都可以使用，文件命名为 *（org.freedesktop.\）*，*有些只能在特定桌面下使用，文件命名类似* (org.gnome.\)，有些操作是单个程序特有的，命名类类似 *(org.archlinux.pkexec.gparted.policy)*。`pkaction` 命令会显示所有定义在 `/usr/share/polkit-1/actions` 操作。

通过下面几个常用的操作类型，可以了解 polkit 到底能做什么：

- **systemd-logind** *(org.freedesktop.login1.policy)* 定义用户是否有权限进行关机、重启、挂起、休眠等操作，即使有其它用户登录时， polkit 也能管控某个用户的上述权限。
- **udisks** *(org.freedesktop.udisks2.policy)* 定义文件系统挂载、加密磁盘打开等操作。
- **NetworkManager** *(org.freedesktop.NetworkManager.policy)* 定义网络打开和关闭， wifi 和移动网络间的切换。

每个操作都定义在 .policy 文件的 `<action>` 标签中。例如 `org.gnome.gparted.policy` 包含一个操作：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE policyconfig PUBLIC
```

```
 "-//freedesktop//DTD PolicyKit Policy Configuration 1.0//EN"
 "http://www.freedesktop.org/software/polkit/policyconfig-1.dtd">
<policyconfig>

  <action id="org.gnome.gparted">
    <message>Authentication is required to run the GParted Partition Editor</message>
    <icon_name>gparted</icon_name>
    <defaults>
      <allow_any>auth_admin</allow_any>
      <allow_inactive>auth_admin</allow_inactive>
      <allow_active>auth_admin</allow_active>
    </defaults>
    <annotate key="org.freedesktop.policykit.exec.path">/usr/bin/gparted</annotate>
    <annotate key="org.freedesktop.policykit.exec.allow_gui">true</annotate>
  </action>

</policyconfig>
```

id 属性是发送给 D-Bus 的命令，**message** 属性用来在身份认证时向用户解释当前动作，**icon_name** 是图标。

defaults 标签下定义权限。包含三种设置：**allow_any**、**allow_inactive** 和 **allow_active**。 Inactive 会话是远程会话（例如 SSH、VNC 等。）active 会话是本地终端或图形界面直接登录机器的会话。allow_any 同时包含两种会话。

对每个设置，都有如下选项：

- *no*：不允许用户执行操作，不需要身份认证。
- *yes*：用户可以不进行认证就执行操作。
- *auth_self*：需要认证，但是用户可以只输入自己的密码，不需要属于管理员。
- *auth_admin*：需要用户认证为管理员。
- *auth_self_keep*：和 auth_self 类似，认证状态会保持一段时间。
- *auth_admin_keep*：和 auth_admin 类似，认证状态会保持一段时间。

这些设置是默认设置，只要没有被配置规则覆盖，适用于所有用户。

从上面的 GParted 操作示例可以看出，不管用户是本地还是远程，都需要先认证为管理员之后才能使用 GParted。

认证规则

认证规则可以覆盖默认的设置，个人使用的单个系统设置，应该放到 `/etc/polkit-1/rules.d` 目录。

`addRule()` 方法可以增加一个函数，输入操作和用户，只要进行权限检查，这个函数就会被调用。所有函数会按添加顺序依次调用，只要遇到第一个 return 返回。所以，要将规则放到其它规则前，需要将规则文件放到 `/etc/polkit-1/rules.d` 的其它规则之前，最早的检查是 `00-early-checks.rules`。

.rules 文件的层级是完全自解释的：

```
/* Allow users in admin group to run GParted without authentication */
polkit.addRule(function(action, subject) {
    if (action.id == "org.archlinux.pkexec.gparted" &&
        subject.isInGroup("admin")) {
        return polkit.Result.YES;
    }
});
```

上面函数检查操作 ID *（是否 org.archlinux.pkexec.gparted）*，再确认用户群组*（是否属于 admin ）*，如果是，返回 "yes"。

管理员身份认证

`addAdminRule()` 方法会添加一个在每个管理员认证时被执行的函数。此函数用来规定什么用户可被视作系统管理员。函数的输入是操作和用户，函数按顺序依次执行，直到第一个 return。

系统默认的配置位于 `50-default.rules`，如果要修改这个值，需要把自定义的身份确认函数加到 50 之前，比如 `40-default.rules`。

```
/etc/polkit-1/rules.d/50-default.rules
polkit.addAdminRule(function(action, subject) {
    return ["unix-group:wheel"];
});
```

需要配置的是 return 返回值：输入谁的密码之后就被认为是系统管理员。如果用户自己属于管理员群组，只需要输入自己的密码。如果只有 root 是管理员，需要输入 root 密码。

Arch 的默认设置中会将所有 **wheel** 群组用户视作管理员，如果用下面的规则文件，那么用户需要输入 root 用户密码才会被认为是管理员。

```
/etc/polkit-1/rules.d/49-rootpw_global.rules
/* Always authenticate Admins by prompting for the root
 * password, similar to the rootpw option in sudo
 */
polkit.addAdminRule(function(action, subject) {
    return ["unix-user:root"];
});
```

范例

调试/输出

提示：需要先在配置文件`/usr/lib/systemd/system/polkit.service`中，删除`ExecStart`命令中的`--no-debug`参数，`polkit.log()`才会生效。

下面的规则会输出关于所请求的访问的详细信息。

```
/etc/polkit-1/rules.d/00-log-access.rules
polkit.addRule(function(action, subject) {
    polkit.log("action=" + action);
    polkit.log("subject=" + subject);
});
```

禁用挂起和休眠

下面规则禁止所有用户通过 Polkit 进行挂起和休眠。

```
/etc/polkit-1/rules.d/10-disable-suspend.rules
polkit.addRule(function(action, subject) {
    if (action.id == "org.freedesktop.login1.suspend" ||
        action.id == "org.freedesktop.login1.suspend-multiple-sessions" ||
        action.id == "org.freedesktop.login1.hibernate" ||
        action.id == "org.freedesktop.login1.hibernate-multiple-sessions")
    {
        return polkit.Result.NO;
    }
});
```

跳过口令提示

要模拟 sudo 的 `NOPASSWD` 选项，完全根据 user/group 身份进行认证，可以在 `/etc/polkit-1/rules.d/` 中创建规则进行设置。参考：示例.

全局规则

创建下列文件：

```
/etc/polkit-1/rules.d/49-nopasswd_global.rules
/* Allow members of the wheel group to execute any actions
 * without password authentication, similar to "sudo NOPASSWD:"
 */
polkit.addRule(function(action, subject) {
    if (subject.isInGroup("wheel")) {
        return polkit.Result.YES;
    }
});
```

请将 `wheel` 替换为需要的群组。 设置完成后，所有操作通过 Polkit 授权时，都不需要密码。因此，请仔细选择授权的群组。

针对特定的动作设置

创建文件：

```
/etc/polkit-1/rules.d/49-nopasswd_limited.rules
/* Allow members of the wheel group to execute the defined actions
 * without password authentication, similar to "sudo NOPASSWD:"
 */
polkit.addRule(function(action, subject) {
    if ((action.id == "org.archlinux.pkexec.gparted" ||
        action.id == "org.libvirt.unix.manage") &&
        subject.isInGroup("wheel"))
    {
        return polkit.Result.YES;
    }
});
```

示例中 `action.id` 选择了 GParted 和 Libvirt，可以根据需要进行选择。`||` 操作符是“或”操作，`&&` 是“与”操作。

Udisks

文件管理器在挂载磁盘时可能要求输入密码，或报告 *Not authorized* 或类似错误，详情请查看：Udisks#Configuration.

允许一般用户管理某个 systemd 单元

通过检查 polkit 策略中的某些值，可以指定某些用户和群组管理 systemd 的权限。例如下面配置允许一般用户启动和停止 wpa_supplicant：

```
/etc/polkit-1/rules.d/10-wifimanagement.rules
polkit.addRule(function(action, subject) {
    if (action.id == "org.freedesktop.systemd1.manage-units") {
        if (action.lookup("unit") == "wpa_supplicant.service") {
            var verb = action.lookup("verb");
            if (verb == "start" || verb == "stop" || verb == "restart") {
                return polkit.Result.YES;
            }
        }
    }
});
```

参阅

- Polkit 手册页面
- 使用 PolKit 授权\[失效链接 2021-11-15 ⓘ\] (openSUSE Leap 15.2 安全指南)
- Polkit 身份验证框架

1.14 ClamAV

难度 高级 | **篇幅** 19.6 KB | **所属** 安全

本节将讲到

- 安装
- 配置
- 更新病毒库
- 启动 ClamAV + OnAccessScanning 守护进程
- 测试

Clam AntiVirus 是一款 UNIX 下开源的 (GPL) 反病毒工具包。它提供了包括一个灵活且可扩展的多线程守护进程，一个命令行扫描程序及用于自动更新数据库的高级工具在内的多个实用程序。ClamAV 主要用被在文件和邮件服务器上，它通过内置的特征库来检测恶意软件，且不能被当作传统的终端安全套件来使用。

由于多重原因，Linux 反恶意软件产品的现状并不理想：

1. 使用有限：和 Windows 相比，Linux 的用户和终端数较少，导致公司不愿为 Linux 平台开发产品。
2. 安于现状：很多人认为 Linux 本身就是安全的，导致缺乏恶意软件防护的意识。这也导致主动防御机制上的空缺。
3. 缺乏功能：现有工具往往缺乏 Windows 反恶意软件产品中常见的一些高级功能，因此在 Linux 上的效果较差。

由于基于 Linux 的服务器和物联网设备数量不断增加，Linux 上的恶意软件数量也在不断增加，可能的攻击面也在不断扩大，因此这种情况尤为糟糕。

ClamAV 是当前 Linux 上为数不多的，且积极开发中的反恶意软件解决方案之一。

安装

安装 clamav包 软件包。

该操作将安装以下工具：

```
clamd：ClamAV 守护进程
clamonacc：访问实时保护
clamdscan：一个简易的扫描客户端
clamdtop：针对 clamd 的资源监控界面
freshclam：特征库更新守护进程
clamconf：用于创建和检查配置文件的工具
```

所有 ClamAV 相关的工具、服务和守护进程都通过套接字与 clamd 进行通信。

默认情况下，这是通过一个名为“LocalSocket”的本地套接字完成的。

ClamAV 还提供了通过使用网络套接字进行远程通信的功能，该套接字名为“TCPSocket”。

警告：在 LocalSocket 改为使用 TCPSocket 时，请注意与 clamd 端口安全相关的警告。通常来说会使用本地 unix 套接字，但请注意该端口未经验证或保护。

更多信息请参考：

https://blog.clamav.net/2016/06/regarding-use-of-clamav-daemons-tcp.html

https://docs.clamav.net/manual/Usage/Scanning.html#clamd-v0101 (Daemon and then ClamD section)

另一个需要注意的问题是，在使用 LocalSocket 时，clamd 需要在一个有权限扫描受监控文件的用户下运行。

配置

正常情况下应已有默认配置文件，否则可以通过 clamconf 手动进行生成：

```
# clamconf -g freshclam.conf > freshclam.conf
# clamconf -g clamd.conf > clamd.conf
# clamconf -g clamav-milter.conf > clamav-milter.conf
```

下列文件中包含了对应的配置项：

- freshclam：`/etc/clamav/freshclam.conf`
- clamd：`/etc/clamav/clamd.conf`
- clamd 邮件过滤：`/etc/clamav/clamav-milter.conf`

另外，你可以通过执行 `clamconf` 来检查配置文件是否可用。

默认安装方式将创建正常的默认配置，如 clamav 用户/用户组和所需的 clamd 配置。

还可以设置其它建议配置：

```
/etc/clamav/clamd.conf
# Log time with each message.
# Default: no
LogTime yes

# Log additional information about the infected file, such as its
# size and hash, together with the virus name.
ExtendedDetectionInfo yes

# Run as another user (clamd must be started by root for this option to work)
# Default: don't drop privileges
User clamav

# Maximum depth directories are scanned at.
# Default: 15
MaxDirectoryRecursion 20

DetectPUA yes
HeuristicAlerts yes
ScanPE yes
ScanELF yes
ScanOLE2 yes
ScanPDF yes
ScanSWF yes
ScanXMLDOCS yes
ScanHWP3 yes
ScanOneNote yes
ScanMail yes
ScanHTML yes
ScanArchive yes
Bytecode yes
AlertBrokenExecutables yes
AlertBrokenMedia yes
AlertEncrypted yes
AlertEncryptedArchive yes
AlertEncryptedDoc yes
AlertOLE2Macros yes
AlertPartitionIntersection yes
```

启用实时保护（访问时扫描，OnAccessScan）

访问时扫描是一个实时保护守护进程，会在读取/写入或执行文件时对其进行扫描。它可被配置为在检测到问题时发出提示或进行屏蔽。

可以通过编辑 `/etc/clamav/clamd.conf` 来配置访问时扫描；使用该访问时扫描需要进行以下更改：

```
/etc/clamav/clamd.conf
# Exclude the UID of the scanner itself from checking, to prevent loops
OnAccessExcludeUname clamav
```

建议进行以下更改将将访问时扫描器设为仅提醒模式：

```
/etc/clamav/clamd.conf
# Set the mount point where to recursively perform the scan,
# this could be every path or multiple path (one line for path)
OnAccessMountPath /

# Alternatively, add some directories instead of mount points
# OnAccessIncludePath /home
```

```
# Prevention doesn't work with OnAccessMountPath.
# It works with OnAccessIncludePath, as long as /usr and /etc are not included.
# Including /var while activating prevention is also not recommended, because
# this would slow down package installation by a factor of 1000.
OnAccessPrevention no

# Perform scans on newly created, moved, or renamed files
OnAccessExtraScanning yes

# Optionallyexclude root-owned processes
# OnAccessExcludeRootUID true
```

为警报弹出通知提醒

目前为止，ClamAV 会静默记录所有检测结果，但不会通知用户。可以通过下面的操作在检测到问题时提醒用户。

首先，在 clamd 配置中添加一行：

```
/etc/clamav/clamd.conf
VirusEvent /etc/clamav/virus-event.bash
```

接下来，允许 *clamav* 用户通过 sudo 以任何带有自定义环境变量的用户身份运行 *notify-send*：

```
/etc/sudoers.d/clamav
clamav ALL = (ALL) NOPASSWD: SETENV: /usr/bin/notify-send
```

然后创建 `/etc/clamav/virus-event.bash` 文件，添加以下内容并将其设为可执行：

```
/etc/clamav/virus-event.bash
#!/bin/bash
PATH=/usr/bin
ALERT="Signature detected by clamav: $CLAM_VIRUSEVENT_VIRUSNAME in
$CLAM_VIRUSEVENT_FILENAME"

# Send an alert to all graphical users.
for ADDRESS in /run/user/*; do
    USERID=${ADDRESS#/run/user/}
    /usr/bin/sudo -u "#$USERID" DBUS_SESSION_BUS_ADDRESS="unix:path=$ADDRESS/bus"
PATH=${PATH} \
        /usr/bin/notify-send -w -u critical -i dialog-warning "Virus found!" "$ALERT"
done
```

该文件使你可以自定义 clamd 访问时扫描服务检测到问题时弹出的消息内容。

默认情况下，*clamonacc* 会将刚访问的文件名称传递给 *clamav* 进行扫描。问题是，如果 *clamav* 用户无法访问对应的文件，就不能使用该方式进行扫描。作为替代，可以让 *clamonacc*（其始终以根用户权限运行）传递文件描述符。参考以下内容编辑 `clamav-clamonacc.service`：

```
[Service]
ExecStart=
ExecStart=/usr/sbin/clamonacc -F --fdpass --log=/var/log/clamav/clamonacc.log
```

最后，启动/启用或重启 `clamav-clamonacc.service` 和 `clamav-daemon.service`。

参考：#启动 ClamAV + OnAccessScanning 守护进程

如果 AppArmor 拒绝了 *clamd*，可以将其设为 complain 模式：

```
# aa-complain clamd
```

更新病毒库

通过下列命令更新病毒库:

```
# freshclam
```

如果你处于代理后，需编辑 `/etc/clamav/freshclam.conf` 并更新 HTTPProxyServer，HTTPProxyPort，HTTPProxyUsername 和 HTTPProxyPassword 的信息。

病毒库保存在下列文件中：

```
/var/lib/clamav/daily.cld
/var/lib/clamav/main.cld
/var/lib/clamav/bytecode.cvd
```

For automatic updates first create and set the requires freshclam.log file:

```
touch /var/log/clamav/freshclam.log
chmod 600 /var/log/clamav/freshclam.log
chown clamav /var/log/clamav/freshclam.log
```

启动/启用 `clamav-freshclam.service` 或 `clamav-freshclam-once.timer` 来保持病毒库为最新。

`clamav-freshclam.service` 将启动 `freshclam` 为守护进程模式，默认每天进行 12 次检查（每 2 小时一次）。检查频率可以在 `/etc/clamav/freshclam.conf` 中进行修改。

`clamav-freshclam-once.timer` 将启动 `freshclam` 为每天检查一次。检查频率可在 `/usr/lib/systemd/system/clamav-freshclam-once.timer` 中进行修改。

注意：

- 以守护进程模式启动的 freshclam 在每次启动服务时会进行检查。
- 每小时检查超过 1 次将被 CDN 屏蔽 24 小时。
- `.timer` 单元将遵循计划配置，与设备重启或服务重启无关。
- 可参考 #添加更多病毒库/特征库 一节添加病毒特征数据库

启动 ClamAV + OnAccessScanning 守护进程

该操作将把所有病毒特征加载到内存中。以 2024 年 2 月为例，这些特征共将占用至少 1.6GB 内存空间。在定期更新特征时，将短时占用多一倍的内存空间。

注意：

- 在首次启动服务前需要先运行 `freshclam`，否则将出现报错，并无法正常启动 ClamAV。

- 如果你只需要使用独立扫描器进行扫描，就无需启动守护进程。更多信息可参考下方的扫描病毒部分。
- 如果没有启用访问时扫描，那守护进程不会进行任何操作，详细信息请参考 #启用实时保护（访问时扫描，OnAccessScan）

对应的服务是 `clamav-daemon.service`。启动它并启用使它自动在启动时运行。

另外请启动并启用 `clamav-clamonacc.service` 以开启实时访问保护。

测试

为了确保 ClamAV 和定义安装正确，请使用 *clamscan* 扫描 EICAR 测试文件（无病毒代码的无害签名）。

```
$ curl https://secure.eicar.org/eicar.com.txt | clamscan -
```

输出**必须**包括

```
stdin: Win.Test.EICAR_HDB-1 FOUND
```

实时防护

您可以下载 eicar 文件并保存在您配置 clamonacc 监控的目录中。例如：

```
$ cd /home/user/Downloads/
$ wget https://secure.eicar.org/eicar.com.txt
$ cat eicar.com.txt
```

添加更多病毒库/特征库

ClamAV 可以使用来自其他存储库或安全厂商的病毒/特征库。

可通过安装 clamav-unofficial-sigsAUR（参见 GitHub 描述）或是 python-fangfrischAUR（参见线上文档）来添加最重要的部分。两者都会添加来自如 MalwarePatrol，SecuriteInfo，Yara，Linux Malware Detect 等著名供应商的特征/病毒库。

注意：必须启动 `clamav-freshclam.service` 才能从 ClamAV 镜像更新官方特征信息。

选项 #1：配置 Fangfrisch

Fangfrisch 在设计上目标成为更安全，灵活和方便的 clamav-unofficial-sigs 替代品，且无需太多配置（`/etc/fangfrisch/fangfrisch.conf`）。

最重要的是，与 clamav-unofficial-sigs 不同，Fangfrisch 用不需要以 root 权限运行

通过以下命令创建库结构：

```
# sudo -u clamav /usr/bin/fangfrisch --conf /etc/fangfrisch/fangfrisch.conf initdb
```

启用 `fangfrisch.timer`（系统层级）。

选项 #2：配置 clamav-unofficial-sigs

启用 `clamav-unofficial-sigs.timer`。

这将根据 `/etc/clamav-unofficial-sigs` 目录下的配置文件信息定期更新非官方特征信息。

可通过如下方法手动更新特征：

```
# clamav-unofficial-sigs.sh
```

可参考 `/etc/clamav-unofficial-sigs/user.conf` 来修改默认设置。

MalwarePatrol 病毒库

如果你想使用 MalwarePatrol 病毒库，需要先在 https://malwareblocklist.org 注册账号（需付费）。

在 `/etc/clamav-unofficial-sigs/user.conf` 中，修改以下内容来启用该功能：

```
malwarepatrol_receipt_code="YOUR-RECEIPT-NUMBER" # 输入收据编号
malwarepatrol_product_code="8" # 免费账号设为 8，高级客户设为 15
malwarepatrol_list="clamav_basic" # clamav_basic 或是 clamav_ext
malwarepatrol_free="yes" # 免费账号设为 yes，高级客户设为 no
```

来源：https://www.malwarepatrol.net/clamav-configuration-guide/

扫描病毒

有两种方法进行按需扫描：

使用独立扫描器

`clamscan` 可用以扫描文件, 用户目录亦或是整个系统：

```
$ clamscan myfile
$ clamscan --recursive --infected /home/archie
# clamscan --recursive --infected --exclude-dir='^/sys|^/dev' /
```

- 如果希望 `clamscan` 删除感染的文件，请使用 `--remove` 参数。
- 使用 `-l *path/to/file*` 参数可以将 `clamscan` 的日志写入 log 文件。

如果希望 `clamscan` 删除感染的文件，请使用 `--remove` 参数，也可以使用 `--move=/dir` 来隔离文件。

你可能会想要 `clamscan` 扫描大文件，在这一情况下，可将 {{ic|1=--max-filesize=4000M} 和 `--max-scansize=4000M` 添加到命令中。'4000M' 指的是最大值，可按需降低。

`-l /path/to/file` 选项将把 `clamscan` 的日志输出为文本文件，方便定位受感染文件。

使用守护进程

`clamdscan` 与上一选项类似，但使用了守护进程，因此必须要守护进程运行中才能使用。由于守护进程会读取 `/etc/clamav/clamd.conf` 中指定的配置，大多数选项都将被忽略。

使用 milter

Milter 会检查 sendmail 服务器中的邮件是否含有病毒。根据需要编辑 `/etc/clamav/clamav-milter.conf`，例如：

```
/etc/clamav/clamav-milter.conf
MilterSocket /tmp/clamav-milter.socket
MilterSocketMode 660
FixStaleSocket yes
User clamav
MilterSocketGroup clamav
PidFile /run/clamav/clamav-milter.pid
TemporaryDirectory /tmp
ClamdSocket unix:/run/clamav/clamd.ctl
LogSyslog yes
LogInfected Basic
```

创建 `/etc/systemd/system/clamav-milter.service`：

```
/etc/systemd/system/clamav-milter.service
[Unit]
Description='ClamAV Milter'
After=clamav-daemon.service
Restart=Always

[Service]
Type=forking
ExecStart=/usr/bin/clamav-milter --config-file /etc/clamav/clamav-milter.conf

[Install]
WantedBy=multi-user.target
```

如需使用如 logrotate 等自动化程序停止服务，可能会需要不同的 `Restart=` 配置。

启用并启动 `clamav-milter.service`。

对于 Postfix，将以下内容添加到 `/etc/postfix/main.cf`：

```
/etc/postfix/main.cf
smtpd_milters = unix:/tmp/clamav-milter.socket
milter_default_action = tempfail
```

检查 journalctl 来确认 postfix 对 `clamav-milter.socket` 的访问权限是否正常。如有问题，请将 postfix 添加到 `clamav` 组中。

小贴士

使用多线程运行

使用 clamscan

通过命令行使用 `clamscan` 扫描文件或文件夹时只会使用单 CPU 线程。如果不考虑时间或者希望电脑不卡，就没什么大问题。但如果你要扫描大型目录，或者需要快速扫描 U 盘，就最好使用全部 CPU 核心来加速进程。

`clamscan` 设计上为单线程，因此可以使用 `xargs` 进行多线程扫描：

```
$ find /home/archie -type f -print0 | xargs -0 -P $(nproc) clamscan
```

在这一例子中，`xargs` 的 `-P` 参数同时调用了尽可能多的 CPU 核心（数量由 `nproc` 提供）。`--max-lines` 和 `--max-args` 参数可以更精细地调整线程间的负载分配。

因为每个进程都是独立的，且都会单独加载特征文件，所以该操作会消耗大量内存。单个线程会消耗大概 1G 或以上的内存，且可能会由于内存不足（OOM）导致系统卡住。可以考虑使用 clamdscan。

使用 clamdscan

如果你已经启动了 `clamd` 守护进程，可以使用 `clamdscan`（参考 #启动 ClamAV + OnAccessScanning 守护进程）：

```
$ clamdscan --multiscan --fdpass /home/archie
```

其中 `--multiscan` 参数会让 `clamd` 使用可用线程并行扫描文件夹中的内容。由于守护进程是以 `clamav` 用户和用户组身份运行的，因此必须使用 `--fdpass` 参数将文件描述符权限传递给 `clamd`。

`clamdscan` 的可用线程数是由 `/etc/clamav/clamd.conf` 中的 `MaxThreads` 参数决定的（默认为 10）clamd.conf(5)。只有在指定了 `--multiscan` 选项时，`clamdscan` 才会使用多线程扫描，否则将保持使用单线程。

启用 TCPSocket

如果你在 `/etc/clamav/clamd.conf` 中启用了 TCPSocket，需要一并编辑 `clamav-daemon.socket`（参考 FS#57669），将 systemd socket 文件中的配置改为对应的 IP 地址和端口：

```
/etc/systemd/system/clamav-daemon.socket.d/override.conf
[Socket]
ListenStream=
ListenStream=/run/clamav/clamd.ctl
ListenStream=127.0.0.1:3310
```

最后重启 `clamav-daemon.socket`，就能看到 Clamd 绑定到了 TCP 3310 端口：

```
# ss -tulpn | grep clamd
```

```
tcp    LISTEN 0      4096       127.0.0.1:3310      0.0.0.0:*    users:
(("clamd",pid=599,fd=4),("systemd",pid=1,fd=44))
```

疑难解答

注意： 确保运行 `clamscan` 的用户拥有病毒库文件（`/var/lib/clamav/*.c?d`）的读取权限。

Error: Clamd was NOT notified

如果你在运行 freshclam 命令之后出现下列信息：

```
WARNING: Clamd was NOT notified: Cannot connect to clamd through
/var/lib/clamav/clamd.sock connect(): No such file or directory
```

为 clamav 添加一个 sock 文件：

警告：参考 #安装 中的警告了解 clamd 端口安全的相关信息：

```
# touch /run/clamav/clamd.ctl
# chown clamav:clamav /run/clamav/clamd.ctl
```

然后， 编辑 `/etc/clamav/clamd.conf`，去掉该行注释:

```
LocalSocket /run/clamav/clamd.ctl
```

保存文件并重启 `clamav-daemon.service`。

Error: No supported database files found

当启动守护进程时出现下列错误信息：

```
LibClamAV Error: cli_loaddb(): No supported database files found
in /var/lib/clamav ERROR: Not supported data format
```

这意味着 `/etc/clamav/clamd.conf` 中的 `DatabaseDirectory` 设置与 `/etc/clamav/freshclam.conf` 中的 `DatabaseDirectory` 设置不匹配。`/etc/clamav/freshclam.conf` 指向了 `/var/lib/clamav`，而 `/etc/clamav/clamd.conf` 指向了 `/usr/share/clamav`（默认）或其它目录。你需要编辑 `/etc/clamav/clamd.conf`，将其中的 `DatabaseDirectory` 改为与 `/etc/clamav/freshclam.conf` 一致，之后就能正常启动 *clamav* 了。

Error: Can't create temporary directory

如果提示如下错误并给出包含 UID 和 GID 的提示，

```
# can't create temporary directory
```

请修改权限：

```
# chown UID:GID /var/lib/clamav && chmod 755 /var/lib/clamav
```

参考

- Wikipedia:ClamAV

1.15 安全地擦除磁盘

难度 高级 | **篇幅** 22.6 KB | **所属** 安全

本节将讲到

- 常见用途
- 数据残留
- 选择擦除目标
- 选择擦除数据源
- 选择块大小

擦除磁盘是通过向磁盘写入新数据，直至将旧数据完全覆盖完成的。

提示：本文中“磁盘”的概念可拓展到回环设备。

常见用途

清除设备上的数据

完全擦除磁盘最常见的应用是，在需要丢弃或出售设备时，清除磁盘上可能仍存在的未加密敏感数据。擦除数据后，可抵御使用简单文件恢复软件恢复敏感数据的尝试。

若追求擦除速度，可以向设备写入`/dev/zero`产生的全零数据，或其它通过简单模式生成数据。不过，在某些情况下，使用具有一定随机性的数据可能更好，详见#数据残留。

对于设备已经被擦除的部分，不可能使用常规的系统功能（例如，标准ATA/SCSI命令）和硬件接口获取到擦除前的数据。若文件恢复软件需要尝试从被擦除的部分恢复数据，则其必须使用依设备而异的专有功能。

对于机械硬盘，要从被擦除的部分恢复数据，至少需要使用非公开的设备指令，或修改设备控制器/固件，以从设备通常不可访问的部分读取信息。例如，可能可以从被重映射的扇区（记录在S.M.A.R.T.中的设备损坏的扇区）读取到擦除前的数据。

在擦除数据方面，不同的物理存储技术存在不同问题。其中最值得注意的是基于闪存的设备和老式磁存储设备（机械硬盘硬盘、软盘、磁带）。

为块设备加密做准备

若要将磁盘某部分用于存储块设备加密数据，建议使用密码学随机数生成器（下文中简称为RNG）生成的#随机数据进行擦除。

另见随机数生成。

警告：若将某个分区作为块设备加密的容器，且该分区之前存储的是非加密/随机性低的数据，则可能可以通过数据特征分析出加密分区的使用情况，使得块设备加密的强度被削弱到文件系统加密级别。因此，若重视加密强度，不应当使用全零、简单模式生成的数据或其它非随机的数据擦除分区。

数据残留

即使有意移除/擦除了磁盘上的数据，数据的表示可能仍然存在于设备中，详见 Wikipedia:Data remanence。

操作系统、程序、文件系统的影响

操作系统、运行中的程序或日志文件系统，可能会在你不知情的情况下隐式将未加密数据分散拷贝到磁盘的任意角落。不过，只有当需要执行像前文提到的那种高级操作（例如为出售设备而彻底销毁数据，或为全盘加密做准备）并直接操作整个磁盘时，这个潜在的风险才显得至关重要。

若能准确地确定数据在磁盘上的位置，且能确定其并没有被显式或隐式地复制其它地方，可以通过向对应位置写入伪随机数据来快速、彻底地擦除数据。

例如，在擦除LUKS密钥槽位时，cryptsetup就将向槽位所在位置写入`/dev/urandom`产生的伪随机数据。

硬件相关问题

闪存

由于闪存（包括固态硬盘）的写入放大等特性，可靠的擦除较为困难。

由于硬件控制器和操作系统之间的透明抽象层，在操作系统层面对数据的操作和控制器实际的操作可能完全不同。操作系统层面对数据的覆写不会实际覆盖闪存中的数据，因而难以可靠地擦除文件或设备上的特定数据块。

此外，设备可能还提供其它功能，例如所有SandForce固态硬盘都启用了透明压缩。若使用全零或简单重复性模式擦除，则设备可能将写入压缩。若发现擦除的速度远超预期，可能是透明压缩的作用。

使用较简单的设备即可拆解闪存设备，直接不经设备控制器地读取闪存上的数据，从而进行数据恢复。许多数据恢复公司都以较低价格提供对闪存的数据恢复服务。

更多信息详见：

- 固态硬盘/存储单元擦除
- 可靠地擦除基于闪存技术的固态硬盘中的数据
- #选择擦除目标

已识别坏扇区

若硬盘将某一扇区标记为坏扇区，则其将隔离该扇区，使得不可能再通过软件写入该扇区。因此，即使在软件层面进行了全盘写入，该扇区的数据也不会被覆盖。好在，由于扇区大小所限，坏扇区中理论可恢复的数据一般只有几KiB。

磁性残留

对于现代高密度磁存储设备，使用全零或随机数据进行一次全盘写入后，设备上就不再有任何有恢复意义的数据。因而，在如今，无需再进行多次擦除。 虽然在磁盘上可能留下单个比特的磁性残留痕迹，但将这些残留痕迹组合成有意义的字节基本上是不可行的。 另见、、。

选择擦除目标

使用fdisk查找用户具有读取权限的所有设备。

在输出中留意以`/dev/sd*X*`开头的行。

例如，对于一个安装了Linux系统的机械硬盘，输出如下：

```
# fdisk -l
Disk /dev/sda: 250.1 GB, 250059350016 bytes, 488397168 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00ff784a

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1   *        2048      206847      102400   83  Linux
/dev/sda2          206848   488397167   244095160   83  Linux
```

对于写入了Arch Linux引导镜像的4GB U盘，输出如下：

```
# fdisk -l
Disk /dev/sdb: 4075 MB, 4075290624 bytes, 7959552 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x526e236e

   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1   *           0      802815      401408   17  Hidden HPFS/NTFS
```

若担心不小心损坏了计算机上的其它重要数据，可以考虑使用隔离的环境进行操作。例如，可以将要擦除的硬盘连接到虚拟环境（VirtualBox，VMWare，QEMU等），在虚拟环境中进行操作；或使用只连接了需要擦除的硬盘的单独的计算机，并使用Live介质（USB，CD，PXE等）启动；也可使用脚本避免不小心擦除已挂载的设备。

选择擦除数据源

在擦除敏感数据时，擦除数据源的数据模式应当符合要求。

全零数据

在大多数情况下，使用`/dev/zero`或其它简单数据模式擦除设备都是安全的。特别是对于现代机械硬盘，这是一种快速且恰当的擦除方式。

然而，若使用全零或简单数据模式擦除的速度异常地快，说明设备可能使用了透明压缩功能，此时很多磁盘块可能并未被擦除。部分#闪存设备存在此问题。

若要将擦除的部分用于存储加密数据，则不应当使用全零或其它简单数据模式擦除，而应当使用随机数据擦除（见下节），以避免削弱加密强度。

警告：全零或其它简单数据模式擦除可能因透明压缩而失效，因而对于闪存设备、固态硬盘应当慎重使用，且不应用于为存储加密数据做准备。

随机数据

`/dev/urandom`的随机数据由Linux内核生成，可作为快速且密码学安全的伪随机数来源。若要详细了解随机数和伪随机数来源，参见随机数生成。

曾经，内核的随机数生成器速度较慢，因而常常使用加密数据流作为伪随机数据来源。加密数据流通常通过使用随机密钥加密`/dev/zero`获得。虽然加密数据流从理论上来讲仍是安全的，但随着内核随机数生成器的改进，其相比直接使用内核随机数生成器已经没有任何优势，且存在临时随机密钥被意外保存的风险。

警告：对于固态硬盘，若计划启用TRIM支持，则无需使用随机数据擦除。在启用TRIM支持后，由于擦除的部分将被标记为空闲空间，将逐渐被设备清除，使得随机数据擦除没有意义。

选择块大小

另见zhwp:Dd_(Unix)#块大小#.E5.9D.97.E5.A4.A7.E5.B0.8F)，blocksize io-limits。

若使用的机械硬盘采用先进磁盘格式，推荐在擦除时手动设置块大小，而不是默认的512字节设置。采用符合硬盘物理扇区大小的设置，可提高擦除速度。可通过`dd`命令的`bs`选项设置块大小（例如，`bs=4096`将设定块大小为4KiB）。

可使用`fdisk`获取设备的逻辑/物理块大小。或者，也可以通过sysfs暴露的信息获取：

```
/sys/block/sdX/size
/sys/block/sdX/queue/physical_block_size
/sys/block/sdX/queue/logical_block_size
/sys/block/sdX/sdXY/alignment_offset
/sys/block/sdX/sdXY/start
/sys/block/sdX/sdXY/size
```

警告：上述方法显示的是设备向内核报告的块大小。对于某些设备，这可能不准确。例如，大多数固态硬盘都报告物理块大小而512字节，小于设备实际的擦除单元大小。

提示：genwipe.shAUR脚本可用于辅助计算使用`dd`擦除设备时需要传递的参数。只需向脚本传入要擦除的设备即可，例如：`genwipe.sh /dev/sd*XY*`。

手动计算要擦除的块数

块设备被划分为扇区，可通过扇区数量乘扇区大小得到设备的大小。

例如，在使用dd命令擦除硬盘上的某一分区时，可按如下方式传递扇区数量、扇区大小等参数：

```
# dd if=<擦除数据源> of=/dev/sdX bs=<扇区大小> count=<扇区数量> seek=<分区起始扇区> status=progress
```

下面将使用具体实例进行阐述。对于/dev/sd*X*设备，fdisk的输出如下：

```
# fdisk -l /dev/sdX
Disk /dev/sdX: 1.8 TiB, 2000398934016 bytes, 3907029168 sectors
Disk model: ST3500413AS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
...
Device     Boot      Start         End          Sectors     Size  Id Type
/dev/sdX1            2048          3839711231   3839709184  1,8T  83 Linux
/dev/sdX2            3839711232    3907029167   67317936    32,1G  5 Extended
```

- 输出的第一行显示了设备大小（按字节计）、逻辑扇区数量。
- 此外，也可通过blockdev --getsize64 /dev/sd*XY*获取整个设备或设备上某一分区的大小。
- 输出中“Units”开头的行显示了设备的逻辑扇区大小。也可通过设备大小（按字节计）除以逻辑扇区数量得到逻辑扇区大小（echo $((2000398934016 / 3907029168))）。
- 输出中“Sector size”开头的行显示了逻辑/物理扇区大小。使用物理扇区大小进行写入，可提高速度。
- 可通过设备大小（按字节计）除以物理扇区大小得到物理扇区数量（echo $((2000398934016 / 4096))）。

注意：

- 在本例中，将使用逻辑扇区大小进行擦除。
- 通过计算下一分区的起始扇区数与前一分区的结束扇区数的差值，可以使用dd命令擦除未分区的磁盘空间。

例如，要擦除/dev/sdX1，并使用逻辑扇区大小作为擦除的块大小：

- 使用要擦除的分区的起始扇区数、结束扇区数定位：

```
# dd if=<擦除数据源> of=/dev/sdX bs=${BytesInSector} count=${End - Start} seek=${Start} status=progress
```

根据fdisk的输出，代入Start=2048，End=3839711231，BytesInSector=512即可。

- 或者，使用逻辑扇区数：

```
# dd if=<擦除数据源> of=/dev/sdX1 bs=${BytesInSector} count=${LogicalSectors}
status=progress
```

根据fdisk的输出，代入BytesInSector=512，LogicalSectors=3839709184即可。

也可以擦除整个设备，并使用物理扇区大小作为擦除的块大小：

```
# dd if=<擦除数据源> of=/dev/sdX bs=${PhysicalSectorSizeBytes}
count=${AllDiskPhysicalSectors} seek=0 status=progress
```

根据fdisk的输出，代入AllDiskPhysicalSectors=488378646，PhysicalSectorSizeBytes=4096即可。

注意：在擦除整个设备，或擦除单个分区，但已经通过sd*XY*指定了分区的范围时，无需手动指定count=选项。dd将写满整个区域，不过会在写入结束时显示剩余空间不足的错误消息。

覆写目标设备

可以使用多种工具对目标设备进行覆写。若只需要擦除一个文件，参见安全地擦除磁盘/提示和技巧#擦除单个文件。

输出重定向

通过输出重定向，可以创建文件、覆写分区中的空闲空间、擦除单一分区或整个设备。下文的例子中使用/dev/zero以全零填充设备，若需要随机擦除，应当改用/dev/urandom。

如下展示了如何将其它工具的标准输出重定向到块设备中，达到覆写目的：

```
# cat /dev/zero > /dev/sdXY
cat: write error: No space left on device
# xz -z0 /dev/zero -c > /dev/sdXY
xz: (stdout): Write error: No space left on device
# dd if=/dev/zero status=progress > /dev/sdXY
dd: writing to 'standard output': No space left on device
20481+0 records in
20480+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 2.29914 s, 4.6 MB/s
```

dd

另见dd、安全地擦除磁盘/提示和技巧#擦除单个文件。

警告：dd在执行操作前不会二次确认。一定要**仔细确认**输出指向了正确的设备或分区。确保of=...选项指向的是目标设备，而不是系统磁盘。

使用/dev/zero全零流向磁盘每个可寻址位置写入零：

```
# dd if=/dev/zero of=/dev/sdX bs=4096 status=progress
```

或者，使用/dev/urandom随机流：

```
# dd if=/dev/urandom of=/dev/sdX bs=4096 iflag=fullblock status=progress
```

当dd提示No space left on device并结束时，表示擦除完成：

```
dd: writing to '/dev/sdX': No space left on device
```

```
7959553+0 records in
7959552+0 records out
4075290624 bytes (4.1 GB, 3.8 GiB) copied, 1247.7 s, 3.3 MB/s
```

对于容量较大的目标设备，可参见如下提速技巧：

- 安全地擦除磁盘/提示和技巧#使用dd进行高级擦除使用OpenSSL
- 安全地擦除磁盘/提示和技巧#使用模板文件使用非随机数据覆写（例如，重复写入一个文件的内容），随机性不足，但速度快
- Dm-crypt/准备磁盘#dm-crypt_专用方案使用dm-crypt

cp

文件复制命令cp(1)不会检查目标位置的类型，因而也可以用于覆写设备：

```
# cp /dev/zero /dev/sdXY
 cp: error writing ‘/dev/sdXY’: No space left on device
 cp: failed to extend ‘/dev/sdXY’: No space left on device
```

pv

pv包工具可在覆写时显示进度条、已用时间、预计剩余时间。将选定的擦除数据源作为参数传递给pv(1)，并使用`-o`/`--output`选项指定要覆写的目标。例如，使用`/dev/zero`全零填充`/dev/sd*X*`：

```
# pv /dev/zero -o /dev/sdX
```

wipe

`wipe`是wipe包的一部分，专门用于擦除文件。例如，要擦除特定目标：

```
$ wipe -r /path/to/wipe
```

参见wipe(1)。注意：该工具上次更新于2009年，其SourceForge 页面显示当前已处于**不维护**状态。

shred

shred是coreutils包的一部分。其专门用于安全地删除单个文件或整个设备。被删除的文件或设备即使理论上能够被恢复，其难度也非常高，并且需要使用专用设备。默认情况下，`shred`将使用伪随机数据执行3次覆写操作。可手动指定覆写操作执行的次数。

使用默认参数调用`shred`，这将在擦除时显示进度条：

```
# shred -v /dev/sdX
```

`shred`也可用于擦除单个分区。例如，要擦除设备上的第一个分区，使用`shred -v /dev/sd*X*1`。

此外，也可设定只执行1次覆写操作，选择`/dev/urandom`作为熵源，并在结束前再使用全零覆盖一次：

```
# shred --verbose --random-source=/dev/urandom -n1 --zero /dev/sdX
```

scrub

scrub将按指定的顺序，用不同的模式多次覆盖文件或设备，使得数据恢复更加困难。

如下命令使用默认参数调用`scrub`，覆写整个设备（模式1）。覆写使用的模式组合遵循NNSA Policy Letter NAP-14.x规范。这是效率最高的方式。

```
$ scrub /dev/sdX
```

如下命令使用默认参数调用`scrub`，覆写特定文件（模式2）。覆写使用的模式组合遵循NNSA Policy Letter NAP-14.x规范。实际覆写的字节数将向上取整到能够完全覆盖文件占用的最后一个文件系统块。注意，使用该模式时有一些需要注意的地方，详见手册。

```
$ scrub /path/to/file # file是常规文件
```

如下命令使用默认参数调用`scrub`，此时将创建一个目录，在目录中创建文件直到占满文件系统，之后再覆写创建的文件（模式3）。覆写使用的模式组合遵循NNSA Policy Letter NAP-14.x规范。对于每一个文件，实际覆写的字节数将向上取整到能够完全覆盖文件占用的最后一个文件系统块。注意，使用该模式时有一些需要注意的地方，详见手册。

```
$ scrub /path/to/dir # dir是目录
```

更多信息和用法，参见手册。

badblocks

badblocks是e2fsprogs包的一部分，其通过反复向设备写入、读取数据检测坏扇区。由于其写入操作对存储的数据而言是破坏性的，因而也能用于擦除设备。在默认情况下，其将执行4次读取、写入循环，因而可能需要花费较长时间。

```
# badblocks -wsv /dev/device
```

hdparm

警告：不要尝试向通过USB连接的设备发送Secure Erase ATA命令。详见该回复以及

hdparm支持ATA Secure Erase功能，该功能由设备固件处理，作用大致相当于使用全零填充整个设备，并包括软件无法访问的区域。其相当于现代版本的“低级格式化”命令。对于固态硬盘，有报道称执行该命令后，性能可恢复到出厂水平，但可能不足以完全擦除数据（参见#闪存）。

某些设备支持**Enhanced Secure Erase**功能，其具体实现方式依厂商而异。若`hdparm -I`的输出显示，使用Enhanced Secure Erase相比普通Secure Erase的用时显著更短，则说明设备可能支持硬件加密功能，而Enhanced Secure Erase将通过擦除加密密钥保护数据安全。

有关ATA Secure Erase的更多信息，参见固态硬盘/存储单元擦除、Linux ATA wiki。

blkdiscard

参见固态硬盘/存储单元擦除#使用blkdiscard的通用方法。

另见

- Wipe free space in Linux

1.16 用户和用户组

难度 高级 | **篇幅** 23.9 KB | **所属** 安全

本节将讲到

- 概览
- 权限与属主
- Shadow
- 文件列表
- 用户管理

GNU/Linux 通过用户和用户组实现访问控制——包括对文件访问、设备使用的控制。Linux 默认的访问控制机制相对简单直接，不过还有一些更加高级的机制，包括 ACL、Capabilities 和 LDAP 认证.

概览

*用户*一般指使用计算机的人。在本文语境中，该词指用来识别用户的用户名称，既可以是 Mary 或 Bill 这样的真名，也可以是 Dragonlady, Pirate 这样的昵称。关键是，计算机给每个账户分配了特定的名称，而用户则使用这些名称访问计算机。除了人之外，一些系统服务也以有部分限制，由享有部分特权的用户账户身份运行。

由于安全需要，「用户管理」应运而生，以加以明确限制各个用户账户的权限。超级用户 root 于计算机里拥有至高无上的管理权限，所以一般只作管理用。非特权用户则可以用 su 或 sudo 程序以临时获得特权。

个体可以拥有多账户，只不过彼此名称当然不同。但有一些用户名称已事先被系统占用，比如 "root".

此外，任意用户可能从属某个「用户组」。此外用户也能够新加入某些已经存在的用户组，以获取该组所拥有的特权。

注意：新手请务必谨慎地使用这方面的工具，并且要避免对除自己以外的其他已存在用户发生误操作。

提示：「**root**」重定向至此。关于「根目录」的内容，参见分区#根分区。

权限与属主

摘自 In UNIX Everything is a File （UNIX 中一切皆文件）：

: 集众多灵感及理念之大成，UNIX 操作系统打造出了它的设计、接口、文化甚至革新。重中之重，有一句道理：「一切皆文件」可谓 UNIX 的真谛之一。

: 根据这一设计原则，必须要有统一的模型，用以管理对大量 I/O 资源的访问：文档、目录、磁盘、CD-ROM、调制解调器、键盘、打印机、显示器和终端等等，甚至也包括了进程、网络之间的通信。而解决之策，就是为所有这些资源提成一个抽象层，UNIX 之父们称之为「文件」。所有文件都通过一致的 API 以提供访问，因此光只用同一套简单的命令，就可以读写磁盘、键盘、文档以及网络设备。

摘自 Extending UNIX File Abstraction for General-Purpose Networking （针对常规的网络应用，扩展出 UNIX 文件抽象层）：

: UNIX 及兼容系统提供了一个即基本又强悍的抽象层——文件。很多系统服务和设备的应用程序接口，一开始都被设计为文件或文件系统之类的东西。这赋予程序全新的姿态——通过文件抽象层，我们就可以以全新的方式使用众多现成的、且用途单一的小工具。例如 cat 工具，原本只用来读取文件，再将其内容发送到标准输出，但现在它也可以直接访问特殊设备文件（通常在 `/dev` 目录中），加以读取 I/O 设备。在很多系统上，音频记录和播放也可以通过一令执行：分别是 `cat /dev/audio > myfile` 以及 `cat myfile > /dev/audio`.

GNU/Linux 系统中的每一个文件都从属一个用户（属主）和一个用户组（属组）。另外，还有三种类型的访问权限：读（read）、写（write）、运行（execute）。我们可以针对文件的属主、属组、而设置相应的访问权限。再次，我们可以通过 ls 命令的长列表格式以查询文件属主、属组和权限：

```
$ ls /boot/ -l
total 18492
drwxr-xr-x 3 root root    12288 Aug 21 03:27 grub
-rw-r--r-- 1 root root 12487150 Aug 29 18:24 initramfs-linux-fallback.img
-rw-r--r-- 1 root root  2990626 Aug 29 18:23 initramfs-linux.img
-rw-r--r-- 1 root root  3440576 Aug 26 15:17 vmlinuz-linux
```

第一列是文件访问权限（例如，文件`initramfs-linux.img`的权限为`-rw-r--r--`），第三列和第四列分别是属主和属组（本例中所有文件属主都是*root*用户，属组都是*root*组）。

```
$ ls -l /media/
total 16
drwxrwx--- 1 root vboxsf 16384 Jan 29 11:02 sf_Shared
```

上述例子中，`sf_Shared`目录由*root*用户和*vboxsf*组所有。使用`stat`命令也可以查看文件所有权和权限：

属主：

```
$ stat -c %U /media/sf_Shared/
root
```

属组：

```
$ stat -c %G /media/sf_Shared/
vboxsf
```

访问权限：

```
$ stat -c %A /media/sf_Shared/
drwxrwx---
```

访问权限由三组字符组成，分别代表属主、属组、其他人的权限。例如，`-rw-r--r--`表示属主有读写权限、但无运行权限（`rw-`），属组用户和其他用户只有读取权限（`r--`）。又如，`drwxrwx---`表示文件属主和属组用户有读、写、执行权限（`rwx`），而禁止其他用户任何访问（`---`）。第一个字符”d“代表文件类型（目录）。

通过`find`命令查找属于某个用户或某个组的文件：

```
# find / -group "用户组名"
# find / -group "用户组编号"
# find / -user "用户"
```

文件的属主、属组可以通过`chown`命令更改。文件的权限可以通过`chmod`命令修改。

详情参见：chown(1)、chmod(1)、Linux 文件权限。

Shadow

Arch Linux 的用户、群组和密码管理工具由 shadow包 提供，这个软件包是 base包 元软件包的依赖。

文件列表

警告：不要手动编辑这些文件。有些工具可以更好的处理锁定、避免数据库错误。

文件	作用
/etc/shadow	保存用户安全信息
/etc/passwd	用户账户信息
/etc/gshadow	保存组账号的安全信息
/etc/group	定义用户所属的组
/etc/sudoers	可以运行 sudo 的用户
/home/*	主目录

用户管理

使用`who`命令，可以查看目前已登录的用户。要查看系统上的用户，以 root 执行 `passwd -Sa` 输出的数据格式可以参考 passwd(1)。

使用`useradd`命令添加用户：

```
# useradd -m -G "附加组" -s "登录shell" "用户"
```

- `-m`/`--create-home`：创建用户主目录`/home/[用户名]`；在自己的主目录内，即使不是root用户也可以读写文件、安装程序等等。
- `-G`/`--groups`：用户要加入的附加组列表；使用逗号分隔多个组，不要添加空格；如果不设置，用户仅仅加入初始组。
- `-s`/`--shell`：用户默认登录shell的路径；启动过程结束后，默认启动的登录shell在此处设定；请确保使用的shell已经安装，默认是 Bash。

警告：为了登录，登录 shell 必须位于 `/etc/shells` 中, 否则 PAM 的 `pam_shell` 模块会阻止登录请求。不要使用 `/usr/bin/bash` 替代 `/bin/bash`, 除非这个路径已经在 `/etc/shells`中正确配置.

- 有时候需要禁止某些用户执行登录动作，例如用来执行系统服务的用户。将shell设置成 `/usr/bin/nologin` 就可以禁止用户登录。(`nologin(8)`).
- 需要用 *passwd* 为新用户设置密码。
- 如果设置了用户初始组的名称或数字ID；该组必须是存在的；如果没有设置该选项，`useradd`会根据`/etc/login.defs`文件中的USERGROUPS_ENAB环境变量进行设置。默认(`USERGROUPS_ENAB yes`) 会用和用户名相同的名字创建群组，`GID` 等于 `UID`。

更多选项说明请查看 useradd(8)。

添加登录用户

要用默认设置添加一个名为*archie*的用户：

```
# useradd -m archie
```

提示：使用 `useradd --defaults` 可以查看 shell 的默认值。默认是 Bash。使用 `-s`/`--shell` 选项可以设置其他值。`/etc/shells` 记录了可以使用的登录 shell。

此命令会自动创建 `archie` 群组，并成为 `archie` 的默认登录群组。建议每一个用户都设置自己的默认群组，因为umask 默认值是 `002`, 所以同一个默认群组的用户会有创建文件的写权限。参阅 User Private Groups\[失效链接 2021-11-19 ⓘ\]。

要赋予一个群组某个目录的写权限，可以在父目录中设置：

```
# chmod g+s our_shared_directory
```

通过下列命令设置用户密码，虽然不是必须的，还是强烈建议设置密码

```
# passwd [用户名]
```

如果需要临时更改GID，也可以使用 newgrp 命令在运行时将用户的默认 GID 更改为其他 GID。例如，执行 `newgrp *"组名"*` 后，用户创建的文件将关联到该*组名*的 GID，而无需重新登录。若要恢复默认 GID，只需执行不带*组名*的 `newgrp` 命令即可。

添加系统用户

为进程、守护进程分配不同的系统用户可以更安全的管控目录及文件的访问。下面命令创建一个不创建 `home` 目录的非登录用户(可以加入 `-U` 参数创建一个和用户名相同的群组):

```
# useradd -r -s /usr/bin/nologin username
```

如果用户需要特殊的 UID/GID，可以加入 `-u`/`--uid` 和 `-g`/`--gid` 选项：

```
# useradd -r -u 850 -g 850 -s /usr/bin/nologin username
```

修改登录名或主目录

更改用户主目录：

```
# usermod -d /my/new/home -m username
```

`-m` 选项会自动创建新目录并移动内容。

更改用户登录名：

```
# usermod -l newname oldname
```

警告：确保你不是使用你要修改的用户名登录，同时按下(`Ctrl`+`Alt`+`F1`)打开一个新的终端，使用root用户登录，或用其他用户登录后使用su命令登录为root用户。

操作得当的话，在Arch（或其他Linux发行版）中更改用户名是安全的，并且很简单。你可以更改用户所属的组。按照以下步骤进行，可以保留受影响用户的UID和GID，而不会搞乱你已经设置好的文件权限。还有一种方法是手动编辑 `/etc/passwd` 文件。

- 如果要使用sudo，请更新文件`/etc/sudoers`把新的用户（以root登录使用visudo命令）添加进去。
- 如果修改了`~/.bashrc`的PATH环境变量，并把新的用户添加进去。。
- 更改用户名后，我不得不重新安装Thunderbird扩展（Enigmail）。
- 系统（桌面快捷方式，脚本等）里使用了旧的用户主目录的地方，都需要进行修改。要在脚本中避免这样的问题，可以使用`~`或`$HOME`变量来表示主目录。

其他用户管理示例

将用户加入 *群组*，用逗号分隔:

```
# usermod -aG 群组 username
```

警告：如果不使用 `-a` 选项，用户会离开没有列在*群组*的其它群组。

要修改用户的登录 shell:

```
# usermod -s /bin/bash username
```

通过下列命令设置*GECOS*字段（用户信息，例如用户全名）：

```
# chfn [用户名]
```

（这样将会以交互式模式启动`chfn`）

此外，可以设置 GECOS comment：

```
# usermod -c "Comment" username
```

使用userdel命令删除用户：

```
# userdel -r [用户名]
```

-r选项表示一并删除用户主目录和邮件。

提示：adduserAUR 可以以交互的方式执行 *useradd*, *chfn* 和 *passwd*，参考 FS#32893.

用户信息存储

本地用户信息储存在/etc/passwd文件中。要查看系统上所有用户账户：

```
$ cat /etc/passwd
```

一行代表一个用户，格式如下，每行分七个部分，用英文冒号“:”分开：

```
account:password:UID:GID:GECOS:directory:shell
```

此处：

- account：用户名，不能为空，而且要符合标准的*NIX命名规则。
- password：加密的密码，可以使用一个小写的"x"（不带括号）表示密码保存在/etc/shadow文件里。
- UID GID：每个用户和组有一个对应的UID和GID（用户ID和组ID）。Arch里面，第一个非 root 普通用户的默认 UID 是 1000，后续创建的用户UID也应大于1000，特定用户的GID应该属于指定的首要组，组的ID数值列在/etc/group文件里。
- GECOS：可选的注释字段，通常记录用户全名
- directory：用于登录命令设置$HOME环境变量。某些服务的用户主目录设置为"/"是安全的，但不建议普通用户设置为此目录。
- shell：是用户默认登录的shell，通常是Bash，还可选择其他的命令解释器，默认是"/bin/bash"（不带括号），如果你用的是别的shell，在这里设置其路径，此部分是可选的，可留空。

注意：Arch Linux 使用*影子*密码。passwd文件对所有人可读，在里面存储密码（无论是否加密过）是很不安全的。在password字段，通常使用一个占位字符（x）代替。加密过的密码储存在/etc/shadow文件，该文件对普通用户限制访问。

示例：

```
jack:x:1001:1003:Jack Smith,some comment here,,:/home/jack:/bin/bash
```

示例分解说明：用户登录名为jack，密码保存在/etc/shadow，UID为1001，首要组的ID是1003 (users组)，全名Jack Smith并加了一些注释，主目录是/home/jack，使用Bash作为默认shell。

pwck 命令可以用来验证用户数据库是否完整。它可以在检查的同时用 GID 排序用户列表，在比较时很有用：

```
# pwck -s
```

警告：Arch Linux的默认文件会由新版本的'filesystem包 软件包创建为.*pacnew*文件。除非Pacman输出了需要您采取行动的相关信息， 这些 .*pacnew* 可以且应当被删除。systemd-sysusers(8) 或者软件包安装脚本会自动添加或重新添加新的，所需的默认用户和组。

用户组管理

`/etc/group`文件储存了系统中用户组的信息，详情参见：group(5)。还有一个很少使用的 `gshadow`，详情请参考 gshadow(5).

使用`groups`命令查看用户所在组的名称：

```
$ groups [用户名]
```

若省略用户名，默认显示当前用户所在组。

`id`命令提供额外的信息，包括用户UID以及相关用户组GID：

```
$ id [用户名]
```

查看所有组：

```
$ cat /etc/group
```

使用`groupadd`创建新的组：

```
# groupadd [组名]
```

使用`gpasswd`将用户添加到组：

```
# gpasswd -a [用户名] [组名]
```

更改用户所属的组名，不变更`GID`：

```
# groupmod -n newname oldname
```

删除用户组：

```
# groupdel [组名]
```

将用户从组中移除：

```
# gpasswd -d [用户名] [组名]
```

如果用户已登录，必须重新登录使更改生效。

可以使用 *grpck* 命令验证系统组文件的完整性。

警告：Arch Linux的默认文件会由新版本的'filesystem包 软件包创建为.*pacnew*文件。除非Pacman输出了需要您采取行动的相关信息， 这些 .*pacnew* 可以且应当被删除。systemd-sysusers(8) 或者软件包安装脚本会自动添加或重新添加新的，所需的默认用户和组。

群组列表

本节解释来自 filesystem包 的常用组的用途。有很多其他组，在安装相关包时会使用正确的 GID 创建。请到相关软件的首页上了解更多信息。

注意：删除软件包并不会自动移除软件包自动创建的用户/组（UID/GID）。这是有意为之的设计，因为如果不这样做，在软件包使用期间创建的任何文件都将变为无主文件，从而构成潜在的安全风险。

用户组

组	影响文件	作用
adm		通常用来给予系统日志读权限，可以读取 journal 文件。
ftp	`/srv/ftp/`	访问 FTP 服务器.
games	`/var/games`	访问一些游戏。
log		访问 syslog-ng 创建的 `/var/log/` 日志文件.
http	`/srv/http/`	访问 HTTP 服务器文件.
rfkill	**不再使用!** 控制无线设备的电源 (被 rfkill 使用).	
sys		可以管理 CUPS 打印服务。
systemd-journal	`/var/log/journal/*`	以只读方式访问系统日志，和 `adm` 和 `wheel` 不同 . 不在此组中的用户仅能访问自己生成的信息。
users		标准用户组。
uucp	`/dev/ttyS[0-9]+`, `/dev/tts/[0-9]+`, `/dev/ttyUSB[0-9]+`, `/dev/ttyACM[0-9]+`, `/dev/rfcomm[0-9]+`	串口和 USB 设备，例如猫、手柄 RS-232/串口。
wheel		管理组，可以访问 journal 文件和 CUPS 打印服务，可以用户 sudo 和 su 命令权限管理(需要额外设置)。

系统组

下列组系统使用，一般不被 Arch 用户使用：

组	影响文件	作用
avahi		
clamav	`/var/lib/clamav/*`, `/var/log/clamav/*`	Clam AntiVirus 使用.
dbus	`/var/run/dbus/*`	
kmem	`/dev/port`, `/dev/mem`, `/dev/kmem`	
locate	`/usr/bin/locate`, `/var/lib/locate`, `/var/lib/mlocate`, `/var/lib/slocate`	见 Locate。
lp		访问并口设备（如打印机）。

组	影响文件	作用
	/dev/lp[0-9]*, /dev/parport[0-9]*	
mail	/usr/bin/mail	
mpd	/var/lib/mpd/*, /var/log/mpd/*, /var/run/mpd/*, 可选的音乐目录	MPD 组.
nobody		无权限的组。
ntp	/var/lib/ntp/*	NTPd 组.
proc	/proc/*pid*/	A group authorized to learn processes information otherwise prohibited by hidepid= mount option of the proc file system. The group must be explicitly set with the gid= mount option.
root	/*	完全的系统管理和控制 (root, admin)
smmsp		sendmail 群组.
tty	/dev/tty, /dev/vcc, /dev/vc, /dev/ptmx	访问 /dev/ACMx
utmp	/run/utmp, /var/log/btmp, /var/log/wtmp	
vboxsf	虚拟系统的共享目录	VirtualBox使用.

systemd 之前的群组

在 Arch 迁移到 systemd 之前，用户需要主动加入以下组，才能访问对应设备。This way has been deprecated in favour of udev marking the devices with a uaccess tag\[失效链接 2022-09-23 ⓘ\] and *logind* assigning the permissions to users dynamically via ACLs according to which session is currently active. 注意在 session 没有损坏时这才能工作 (见 General troubleshooting#Session permissions 以检查）。

There are some notable exceptions which require adding a user to some of these groups: for example if you want to allow users to access the device even when they are not logged in. However, note that adding users to the groups can even cause some functionality to break (for example, the audio group will break fast user switching and allows applications to block software mixing).

如下组是 systemd 之前使用，目前已经没有任何作用，使用后还可能对功能有影响:

组	作用	
audio	/dev/audio, /dev/snd/*, /dev/rtc0	直接访问声音硬件(ALSA 和 OSS),JACK 也用来给予用户实时处理权限。
camera		访问 Digital Cameras.
disk	/dev/sda[1-9], /dev/sdb[1-9]	直接访问不受 *optical*, *floppy* 和 *storage* 组控制的块设备. 除非有特殊需要, 否则不建议将一般用户添加至该组.
floppy	/dev/fd[0-9]	访问软盘驱动器。
lp		访问打印设备，管理打印任务。

组	作用	
	`/etc/cups`, `/var/log/cups`, `/var/cache/cups`, `/var/spool/cups`	
network		改变网络设置的权限，比如使用 NetworkManager 的权限.
optical	`/dev/sr[0-9]`, `/dev/sg[0-9]`	访问光学设备，比如CD，DVD。
scanner	`/var/lock/sane`	访问扫描仪硬件。
storage		访问可移动储存器，例如 USB 硬盘、flash 存储器、MP3 播放器等；用户可以通过 D-Bus 挂载设备。
sys		管理 CUPS 中的打印机.
video	`/dev/fb/0`, `/dev/misc/agpgart`	访问视频捕获和硬件加速设备。例如framebuffer (X 不属于这个组也能使用).

不再使用的组

Group	Affected files	Purpose
bin	none	Historical
daemon		
lock		Used for lockfile access. Required by e.g. gnokiiAUR.
mem		
network		Unused by default. Can be used e.g. for granting access to NetworkManager (see NetworkManager#Set up PolicyKit permissions).
power		
uuidd		
users		The primary group for users when user private groups are not used (generally not recommended), e.g. when creating users with `USERGROUPS_ENAB no` in `/etc/login.defs` or the `-N`/`--no-user-group` option of *useradd*.

1.17 Sudo

难度 高级 | **篇幅** 24.7 KB | **所属** 安全

本节将讲到

- 安装
- 使用
- 配置
- 使用技巧
- 疑难解答

Sudo 使得系统管理员可以授权特定用户或用户组作为 root 或其他用户执行命令，同时还能够对命令及其参数提供审核跟踪。

Sudo 是对于使用 su 以 root 用户身份执行命令的一个替代。与 su 启动 root shell 并允许用户执行之后的所有命令不同，sudo 仅对单个命令临时授予权限。通过仅在需要时授权，sudo 可以减少因输错命令或命令存在问题导致系统损坏的可能性。

sudo 也能以其他用户身份执行命令，并且会将用户执行的命令及失败的权限申请记录到 journal，以供后续安全审计使用。

安装

安装 sudo包 软件包。

使用

在正确配置之前，普通用户无法使用 `sudo`。参见#配置。

要使用 ***sudo***，只需在命令和其参数前加上 `sudo` 和空格：

```
$ sudo cmd
```

例如，要使用 pacman：

```
$ sudo pacman -Syu
```

参见 sudo(8)。

Login shell

并不是所有命令都可以在前面加上 *sudo* 就能以其它用户身份执行。特别是在使用重定向)和命令替换时，必须通过 `sudo -iu *user*`（目标用户是 root 时可以省略 `-u *user*`）使用 login shell。

以下为例，其可以在完整的 shell 环境中执行，但无法直接附加 *sudo* 执行：

```
$ sudo wpa_supplicant -B -i interface -c <(wpa_passphrase MYSSID passphrase)
Successfully initialized wpa_supplicant
Failed to open config file '/dev/fd/63', error: No such file or directory
Failed to read or parse configuration '/dev/fd/63'
```

配置

默认框架

sudoers(5) § SUDOERS OPTIONS 在 `/etc/sudoers` 文件中列出了所有可以被 `Defaults` 命令使用的选项。

查看 获得格式优化后的 `sudoers` 选项列表（解析自 1.8.7 源码）。

查看 sudoers(5) 获得更多信息，例如密码超时设置。

查看当前设置

命令 `sudo -ll` 可以显示当前的 sudo 配置; 命令 `sudo -lU *user*` 可以查看某个特定用户的设置。

使用 visudo

sudo 的配置文件是 `/etc/sudoers`，**必须**使用 visudo(8) 编辑该文件。*visudo* 会锁定 `sudoers` 文件，将修改保存到临时文件中，并在将其复制到 `/etc/sudoers` 前检查语法问题。

警告：

- 必须确保 `sudoers` 没有语法问题！任何错误都会导致 sudo 不可用。**必须**使用 *visudo* 编辑该文件防止出错。
- visudo(8) 警告说，将 *visudo* 配置为尊重用户编辑器的环境变量设置可能会是一个安全漏洞，因为它允许拥有 *visudo* 权限的用户通过将环境变量设置为其他内容从而以根用户身份运行任意命令而不被日志所记录。

visudo 调用的默认编辑器是 *vi*。sudo包 包在编译时启用了 `--with-env-editor`，会采用环境变量 `SUDO_EDITOR`，`VISUAL` 和 `EDITOR` 的设置。如果设置了`VISUAL`，就不会使用 `EDITOR`。

要在当前 shell 会话中使用 nano 作为 *visudo* 编辑器，可以设置 `EDITOR=nano` 环境变量；如果要临时使用其他编辑器，在调用 *visudo* 前设置环境变量即可。

```
# EDITOR=nano visudo
```

你还可以编辑 `/etc/sudoers` 的副本，然后使用 `visudo -c */copy/of/sudoers*` 进行检查。该操作可以避免 *visudo* 锁定文件。

要永久设置编辑器，请参考定义本地环境变量。要仅修改 *visudo* 的系统级编辑器设置，可以在 `/etc/sudoers` 中添加以下内容（假设你要使用 nano 作为默认编辑器）：

```
# Set default EDITOR to restricted version of nano, and do not allow visudo to use
EDITOR/VISUAL.
Defaults      editor=/usr/bin/rnano, !env_editor
```

设置示例

要允许指定用户在命令前添加 `sudo` 时获得完全 root 权限，在配置文件中加入该行：

```
用户名    ALL=(ALL:ALL) ALL
```

要限定主机名为 `HOST_NAME` 的设备上的指定用户可以执行任意命令：

```
用户名    HOST_NAME=(ALL:ALL) ALL
```

要允许 wheel 用户组成员使用sudo：

```
%wheel      ALL=(ALL:ALL) ALL
```

提示：创建新的管理员经常被描述为为 `wheel` 组启用 sudo 访问并将管理员加入其中，这是因为默认 Polkit 就认为 wheel 组中的成员是管理员。如果用户不是 `wheel` 组的成员，使用 Polkit 的软件可能会要求验证 root 密码而非用户密码。

禁用询问用户 `USER_NAME` 的密码：

警告：任何以您的用户名运行的程序都可以无需密码就执行 sudo。

```
Defaults:USER_NAME      !authenticate
```

仅允许主机 `HOST_NAME` 上的用户 `USER_NAME` 执行特定命令：

```
USER_NAME HOST_NAME=/usr/bin/halt,/usr/bin/poweroff,/usr/bin/reboot,/usr/bin/pacman -Syu
```

注意：最后的设置会覆盖前面的设置，所以限定多的配置应该放到配置文件的后面。如果用户位于 `%wheel` 用户组内，就需要将该行放于对应组配置之后。

仅允许主机 `HOST_NAME` 上的用户 `USER_NAME` 执行特定命令，但不用输入密码：

```
USER_NAME HOST_NAME=
NOPASSWD: /usr/bin/halt,/usr/bin/poweroff,/usr/bin/reboot,/usr/bin/pacman -Syu
```

更详细的 `sudoers` 范例请参考 `/usr/share/doc/sudo/examples/sudoers`。此外，更多信息参见 sudoers(5)。

sudoers 文件默认权限

`sudoers` 文件的属主和属组 ID 必须都是 0，文件权限位是 0440（-r--r-----）。如果你不小心改变了默认权限，应当立即恢复它们，否则 sudo 将会出错：

```
# chown -c root:root /etc/sudoers
# chmod -c 0440 /etc/sudoers
```

使用技巧

禁用密码输入超时

比较常见的烦人问题是有个后台终端上长时间运行的进程，该进程以正常权限运行，只有在需要时才提升权限。这时就会出现 sudo 密码输入提示，该提示很容易会被忽略并导致超时，此时进程就会死亡，已完成的工作也会丢失，或者最多只是被缓存。常见的建议是启用无密码 sudo 或延长 sudo 记住密码的超时时间。这两种方法都会带来负面的安全影响。这里的解决方案应是将**输入提示**超时禁用，且不会造成任何合理的安全影响：

```
Defaults passwd_timeout=0
```

bash 自动补全支持

**这一章节正在考虑移除。**

原因: 没理由不使用 bash-completion包。 (在 Talk:Sudo 讨论)

要为 sudo 命令提供完整的 bash 自动补全功能，请安装 bash-completion包 软件包。如果由于某些原因无法安装该包，则可在 .bashrc 中添加以下内容，以获得部分 sudo 命令的自动补全功能：

```
complete -cf sudo
```

Passing aliases

The following is only relevant if the bash completion is not available (either full or reduced as described above): Aliases in Zsh and Bash are normally only expanded for the first word in a command. This means that your aliases will not normally get expanded when running the `sudo` command. One way to make the next word expand is to make an alias for sudo ending with a space. Add the following to your shell's configuration file:

```
alias sudo='sudo '
```

zshmisc(1) § ALIASING describes how this works:

: If the replacement text ends with a space, the next word in the shell input is always eligible for purposes of alias expansions.

As well as bash(1) § ALIASES:

: If the last character of the alias value is a blank, then the next command word following the alias is also checked for alias expansion.

Add terminal bell to the password prompt

To draw attention to a sudo prompt in a background terminal, users can simply make it echo a bell character:

```
Defaults passprompt="^G[sudo] password for %p: "
```

Note the `^G` is a literal bell character. E.g. in vim, insert using the sequence `Ctrl+v Ctrl+g`. If `Ctrl+v` is mapped, e.g. for pasting, one can usually use `Ctrl+q` instead. In nano, `Alt+v Ctrl+g`.

Another option is to set the `SUDO_PROMPT` environment variable. For example, add the following to your shell configuration file:

```
export SUDO_PROMPT=$'\a[sudo] password for %p: '
```

禁用单终端 sudo

警告：这将使所有进程都能使用你的 sudo 会话。

如果觉得每开一个终端使用 sudo 都要输密码很烦，可以将 `timestamp_type` 设为 `global`：

```
Defaults timestamp_type=global
```

减少要求输入密码的次数

如果觉得每 5 分钟（默认值）都要重新输密码很烦，可以将 `timestamp_timeout` 改成更长的值（单位为分钟）：

```
Defaults timestamp_timeout=10
```

如果在较长的脚本中使用 sudo 命令，且不想在超时后等待用户输入，则可以通过循环运行 `sudo -v` 来刷新超时（而 `sudo -K` 则会立即取消超时）。

环境变量

如果你使用了大量环境变量，或者你通过 `export http_proxy="..."` 配置了代理设置，除非执行 sudo 时使用了 `-E`/`--preserve-env` 选项，否则这些变量不会被传递到根用户下。

```
$ sudo -E pacman -Syu
```

推荐方法是将要传递的环境变量添加到 `env_keep` 中：

```
/etc/sudoers
Defaults env_keep += "ftp_proxy http_proxy https_proxy no_proxy"
```

使用 root 密码

通过将 `targetpw`（目标用户，默认为 root）或 `rootpw` 添加到 `/etc/sudoers` 的 Defaults 一行中，可以让 sudo 询问 root 账户的密码，而不是用户密码：

```
Defaults targetpw
```

为避免向用户暴露 root 账户密码，可以将该设置限定到特定组：

```
Defaults:%wheel targetpw
%wheel ALL=(ALL) ALL
```

禁止 root 登录

用户也许希望禁止使用 root 登录。没有了 root 用户，黑客就必须要知道 sudoer 的用户名和密码，具体实例请参考 OpenSSH#拒绝。

警告：

- 注意，禁用 root 登陆后有可能会把你自己锁在系统外。默认情况下没有安装 sudo，其默认配置也不允许无密码使用 root 权限和使用用户密码获取 root 权限。在禁用 root **之前**，务必确保已正确配置至少一个用户的权限！
- 如果你在 sudoers 中配置了默认使用 rootpw，就不要执行下列任一命令禁用 root 登陆：
- 如果你已经被锁在系统外了，请参考重置 root 密码。

使用 `passwd` 命令锁住 root 用户：

```
# passwd -l root
```

使用以下命令解锁 root 用户：

```
$ sudo passwd -u root
```

或者，编辑 `/etc/shadow` 文件，将 root 的加密口令列替换为 `!*`：

```
root:!*:12345::::::
```

要再次允许 root 登陆，重新设置其密码即可：

```
$ sudo passwd root
```

**本文或本章节的事实准确性存在争议。**

原因： In most cases when a user ends up in an emergency shell they are using the initramfs, will not use the following configuration, unless added to the `FILES` in mkinitcpio's configuration.（在 Talk:Sudo 中讨论）

In case of system emergency, the recovery prompt is going to ask your for a root password, making it impossible to log into recovery shell. To automatically unlock the root account in case of emergency add `SYSTEMD_SULOGIN_FORCE=1` environment variable to `rescue.service` using a drop-in file:

```
/etc/systemd/system/rescue.service.d/SYSTEMD_SULOGIN_FORCE.conf
[Service]
Environment=SYSTEMD_SULOGIN_FORCE=1
```

提示：要在禁用 *root* 账号后使用 root 交互终端，请使用 `sudo -i`。

kdesu

KDE 下可能会使用 kdesu 以 root 权限执行图形程序。默认情况下，即使 root 账户被禁用，kdesu 仍可能会尝试使用 su 切换 root。可以配置 kdesu 以使用sudo，创建或编辑 `~/.config/kdesurc`，加入：

```
[super-user-command]
super-user-command=sudo
```

或者使用下面命令：

```
$ kwriteconfig6 --file kdesurc --group super-user-command --key super-user-command sudo
```

示例：通过 sudo 强化安全

假设你创建了 3 个用户：admin，devel 和 archie。用户“admin”用于 journalctl，systemctl，mount，kill 和 iptable；“devel”用于安装软件包和编辑配置文件；“archie”是用户登录使用的账户。为了允许“archie”进行重启，关机和使用 netctl，需要进行以下步骤：

编辑 `/etc/pam.d/su` 和 `/etc/pam.d/su-l`。Require user be in the wheel group, but do not put anyone in it.

```
#%PAM-1.0
auth            sufficient      pam_rootok.so
# Uncomment the following line to implicitly trust users in the "wheel" group.
#auth           sufficient      pam_wheel.so trust use_uid
# Uncomment the following line to require a user to be in the "wheel" group.
auth            required        pam_wheel.so use_uid
auth            required        pam_unix.so
account         required        pam_unix.so
session         required        pam_unix.so
```

仅允许 'ssh' 用户组下的用户通过 SSH 登录，只有“archie”在该组内：

```
# groupadd -r ssh
# gpasswd -a archie ssh
# echo 'AllowGroups ssh' >> /etc/ssh/sshd_config
```

重启 `sshd.service`。

将用户添加到其它组中：

```
# for g in power network ;do ;gpasswd -a archie $g ;done
```

```
# for g in network power storage ;do ;gpasswd -a admin $g ;done
```

设置配置文件的权限，以使 devel 能够进行编辑：

```
# chown -R devel:root /etc/{http,openvpn,cups,zsh,vim,screenrc}
Cmnd_Alias  POWER       =   /usr/bin/shutdown -h
now, /usr/bin/halt, /usr/bin/poweroff, /usr/bin/reboot
Cmnd_Alias  STORAGE     =   /usr/bin/mount -o nosuid\,nodev\,noexec, /usr/bin/umount
Cmnd_Alias  SYSTEMD     =   /usr/bin/journalctl, /usr/bin/systemctl
Cmnd_Alias  KILL        =   /usr/bin/kill, /usr/bin/killall
Cmnd_Alias  PKGMAN      =   /usr/bin/pacman
Cmnd_Alias  NETWORK     =   /usr/bin/netctl
Cmnd_Alias  FIREWALL    =   /usr/bin/iptables, /usr/bin/ip6tables
Cmnd_Alias  SHELL       =   /usr/bin/zsh, /usr/bin/bash
%power      ALL         =   (root)  NOPASSWD: POWER
%network    ALL         =   (root)  NETWORK
%storage    ALL         =   (root)  STORAGE
root        ALL         =   (ALL)   ALL
admin       ALL         =   (root)  SYSTEMD, KILL, FIREWALL
devel      ALL         =   (root)  PKGMAN
archie     ALL         =   (devel) SHELL, (admin) SHELL
```

完成配置后，就基本不需要以 root 身份登录了。

“archie”可以连接到家里的 Wi-Fi：

```
$ sudo netctl start home
$ sudo poweroff
```

“archie”无法以其他用户身份使用 netctl：

```
$ sudo -u admin -- netctl start home
```

如果“archie”需要使用 journalctl 或杀死其它进程，可以切换到对应用户：

```
$ sudo -i -u devel
$ sudo -i -u admin
```

但“archie”无法切换到根用户：

```
$ sudo -i -u root
```

如果“archie”想以 admin 身份启动 gnu-screen 会话，可以参考以下操作：

```
$ sudo -i -u admin
[admin]$ chown admin:tty `echo $TTY`
[admin]$ screen
```

让 sudo 使用 /etc/sudoers.d 中的文件

sudo 可以解析 `/etc/sudoers.d/` 目录中的文件，这样就不需要编辑 `/etc/sudoers`，可以单独将配置写入到新文件中，然后放入到该目录。该方式的优点包括：

- 不需要编辑 `sudoers.pacnew` 文件；
- 如果新配置有问题，可以删除这个文件而不用编辑 `/etc/sudoers`（但注意下方警告）。

该目录下的文件格式与 `/etc/sudoers` 一致。如需直接编辑这些文件，可以使用 `visudo -f /etc/sudoers.d/*somefile*`，具体请参考 sudoers(5) § Including other files from within sudoers。

/etc/sudoers.d/ 目录中的文件是按字母顺序加载的，. 或 ~ 开头的文件会被跳过。为避免排序问题，文件名应以两位数字开头，例如：01_foo。

注意：注意文件中各项的排序，确保不会被互相覆盖。

警告：/etc/sudoers.d/ 中的文件和 /etc/sudoers 一样，格式错误将导致 sudo 无法正常运行。因此，强烈建议同样使用 visudo。

编辑文件

sudo 提供了 sudoedit 命令（与 sudo -e 等效），能以当前用户身份及配置运行编辑器时修改仅 root 有权编辑的文件。

要编辑文件，需将 SUDO_EDITOR 设为编辑器名称，并将文件名传递给 sudoedit，例如：

```
$ SUDO_EDITOR=vim sudoedit /etc/file
```

设置编辑器的方式请参考#使用 visudo 和 sudo(8) § e，但需要注意潜在的安全问题。

如果一次性向 sudo 传入多个文件名，则会在编辑器中同时打开所有文件：

```
$ SUDO_EDITOR=vimdiff sudoedit /etc/file /etc/file.pacnew
```

启用嘲讽

通过 visudo 在 sudoers 文件中添加下行可以启用嘲讽彩蛋：

```
/etc/sudoers
Defaults insults
```

在用户输错密码时，会将 Sorry, try again. 替换成各种嘲讽。

启用密码输入反馈

默认情况下，输入密码是不会有视觉反馈的，目的是增强安全性。如果你想启用视觉反馈，可以添加下行进行启用：

```
/etc/sudoers
Defaults pwfeedback
```

彩色密码提示

在 shell 初始化文件中设置 SUDO_PROMPT 环境变量和 tput(1) 可以自定义彩色和/或粗体密码提示。

以将 Password: 显示为红色粗体为例：

```
export SUDO_PROMPT="$(tput setaf 1 bold)Password:$(tput sgr0) "
```

也可以使用不同颜色来显示默认消息：

```
export SUDO_PROMPT="$(tput setab 1 setaf 7 bold)[sudo]$(tput sgr0) $(tput setaf
6)password for$(tput sgr0) $(tput setaf 5)%p$(tput sgr0): "
```

详细信息请参考在终端输出彩色和 Bash/Prompt customization。

使用 U2F

将 U2F 与 sudo 搭配使用，可以通过物理触碰进行批准，还能基本消除公众场合下被肩窥)的风险。

具体信息请参考通用第二因素#无密码 sudo。

写入到受保护文件

在使用 sudo 时，无法使用 >/>> 写入到受保护文件，这时候可以使用 tee：

```
$ input stream | sudo tee --option protected_file_1 protected_file_2...
```

Vim 环境内

A similar concept is useful when you forgot to start Vim with sudo when editing a file owned by an other user. In this case you can do the following inside Vim to save the file:

```
:w !sudo tee %
```

You can add this to your `~/.vimrc` to make this trick easy-to-use with `:w!!` mapping in command mode:

```
~/.vimrc
" Allow saving of files as sudo when I forgot to start vim using sudo
 cmap w!! w !sudo tee > /dev/null %
```

The `> /dev/null` part explicitly throws away the standard output since we do not need to pass anything to another piped command.

More detailed explanation of how and why this works can be found in How does the vim “write with sudo” trick work? article on StackOverflow.

疑难解答

SSH TTY 问题

**本文或本章节可能需要合并到#配置。**

远程执行命令时，SSH 默认不会分配 tty。没有分配 tty，sudo 就无法在获取密码时关闭回显。使用 SSH 的 `-t` 选项可以强制分配 tty。

`Defaults` 的 `requiretty` 选项可以仅允许有 tty 的用户才能使用 sudo：

```
# Disable "ssh hostname sudo <cmd>", because it will show the password in clear text.
You have to run "ssh -t hostname sudo <cmd>".
#
#Defaults    requiretty
```

权限 Umask

**本文或本章节可能需要合并到#配置。**

Sudo 会统一用户的 umask 值和它自己的 umask（默认是 0022）。这会防止 sudo 创建的文件权限比该用户 umask 允许的更广。在没有使用自定义 umask 时，该配置比

较合理，但可能导致 sudo 创建的文件权限与 root 下直接创建的权限不同。如果这导致了问题，sudo 提供了一个方法来修复 umask，即使想要的 umask 比用户指定的 umask 权限还要多。添加下面内容（使用 `visudo`）会覆盖 sudo 的默认行为：

```
Defaults umask = 0022
Defaults umask_override
```

这会将 sudo 的 umask 设置为 root 的默认 umask（0022），覆盖掉默认行为，无论用户的 umask 设置成什么都会使用这里设定的值。

1.18 安全

难度 高级 | **篇幅** 88.3 KB | **所属** 安全

本节将讲到

- 概念
- 密码
- CPU
- 内存
- 存储

本文包含了加固 Arch Linux 系统的常用建议与最佳实践。

概念

- 收紧安全措施有可能达到使系统无法使用的程度。安全性与便利性需要得到平衡。诀窍在于建立一个安全且有用的系统。
- 最大的威胁是（并且一直都会是）用户。
- 最小权限原则：系统的每一部分应该只能访问到它确实需要的东西，除此之外的则不可以。
- 纵深防御：多个独立的层次能带来更好的安全性。当一层防护被攻破时，另一层应该能够阻止攻击。
- 保持一点点的偏执和多疑。如果有件事看起来太好了，不像是真的，那可能确实如此。
- 永远无法令系统 100% 安全，除非把机器从网络上断开，关掉电源，锁进保险柜，用混凝土封住并不再使用它。
- 为失败做好准备。预先为安全措施被攻破的情况制定可供执行的计划。

密码

密码是一个安全系统的关键。它可以保护你的用户账户、加密的文件系统和 SSH/GPG 密钥。密码也是让计算机信任使用者的主要方式，所以安全性的很大一部分就在于选择高强度的密码并保护它们不被泄露。

选择安全的密码

密码必须足够复杂，不能轻易地被猜中（比如和个人信息有关的密码），也不能轻易地被社会工程或暴力尝试等方法破解。强密码的要点在于*长度*和*随机性*。在密码学中，密码的质量被称为它的熵安全性。

不安全的密码包括：

- 个人可识别信息（如：狗的名字，出生日期，区号，最喜欢的视频游戏）
- 对单词简单地替换一些字符（如：`k1araj0hns0n`），因为现代字典攻击可以轻松对付它们
- 在基本单词或常见字符串的前后加上数字，符号或其他字符（如：`DG091101%`）
- 常见句子或词典中单词的序列（如：`photocopyhauntbranchexpose`），包括对其字符进行替换（如：`Ph0toc0pyh4uN7br@nch3xp*se`）
- 任何一个最常见的密码

最好的选择是由随机来源生成的长密码（越长越好）。使用长密码很重要。弱哈希算法会使得一个8字符密码的哈希值在几小时之内便被攻破。

像 pwgen包 或 apgAUR 这样的工具可生成随机密码，不过这些密码可能很难记住：为了记住它们，一种方法（仅针对经常使用的密码）是生成一个长密码并记住一小段（这一小段要能保证最低限度的安全），暂时把完整的密码写下来。在一次次的密码输入过程中，尝试着记住从一小段到一大段乃至全部密码，随着时间推移，密码就会随着肌肉记忆根深蒂固。这种方法很难，但是能保证密码不会出现在破解用的字典里，也可以抵御“智能地”组合单词并替换部分字符的暴力破解手段。

除了密码管理，keepassxc包 还提供密码/口令生成功能。用户可以在图形用户界面中自定义生成密码。还支持基于字典的口令。

还有个方法可以产生安全性好的、看起来随机的密码，那就是从一句句子的每一个词中提炼出一个符号。 例如 “the girl is walking down the rainy street” 这句话可以转换为 `t6!WdtR5` 或是更加复杂的 `t&6!RrlW@dtR,57`。 这种方法可以更为简单地帮助记忆密码，但是请注意，不同字母出现在单词开头的概率不同 (Wikipedia:字母频率)。

另外一种有效的技术是写下随机生成的密码并将其存储在*安全*的地方，例如钱包、挎包或文件保险箱中。不少人在保护其物理贵重物品免受盗窃方面通常做得很好，并且与数字安全实践相比，大多数人更容易理解物理安全最佳实践。

将助记符和随机技术结合起来，通过密码管理器保存长的随机生成的密码也非常有效。密码管理器需要用一个易记的“主密码”来访问，这个密码只能用于此目的。主密码必须记住，绝不能保存。这就要求在系统上安装密码管理器，以便轻松访问密码（根据情况，这可以被视为不方便或安全特性）。一些密码管理器还有智能手机应用，可以用来显示用于在没有安装该密码管理器的系统上进行手动输入的密码（如果这是常见的使用情况，那么你仍然可以为每个服务使用易于输入但安全的密码，而不是完全随机的密码，参见下文）。注意，如果你忘记了主密码，密码管理器就引入了一个单点故障。

有些密码管理器根据主密码和你想要登录的服务名称计算包含的密码，而不是加密它们，使得在新的系统上无需同步任何数据也可以使用它。

使用一长串相互无关的单词作为密码或许是有效的方法。原理在于，如果使用足够长的短语，从密码长度所获得的熵就可以抵消使用字典词汇而失去的熵。这个 XKCD 漫画描绘了此方法中对于熵的权衡，考虑到短语中每个单词可能的单词集的限制。如果你使用的单词集很大（数千个单词）并且您从中选择 5-7 个甚至更多随机单词，那就算攻击者知道你所选择的可能单词集和选择的单词数量，这种方法仍能提供很大的熵。参见Diceware。

参阅 The passphrase FAQ 或 Wikipedia:Password strength ，获取额外背景信息。

维护密码安全

一旦你选择了一个强密码，就一定要保证它的安全。当心键盘记录器（软件层面 和 硬件层面），屏幕记录器，社会工程)，肩窥)，并避免对不同的服务器（网站）使用重复密码，以防止不安全的服务器泄漏超出其范围的信息。密码管理器可以帮助管理大量复杂密码：将密码从管理器复制粘贴到其他程序中时，记住每次粘贴完都清除复制缓冲区，并确保密码不会“意外地”保存在任何类型的文件中（例如，不要将它们粘贴在普通的终端命令中，因为这些命令会存到 `.bash_history` 之类的文件里）。

最好不要因为安全性强的密码难记而选择不安全的密码，密码是它们之间的一种平衡。与拥有许多相似的弱密码相比，更好的做法是建立一个加密的安全密码数据库，数据库由密钥和一个强主密码保护着。把密码写下来也许同样有效，可以避开软件中的潜在漏洞，同时也需要保证物理安全。

衡量密码强度的另一个指标是它不能够被轻易从其他地方恢复。

如果你使用与登录密码相同的磁盘加密密码（这在登录时自动挂载加密分区或目录很有用），请确保 `/etc/shadow` 也在加密分区上，或者/并 使用强哈希算法（即 sha512/bcrypt，而不是 md5）来存储密码哈希（详细信息请参阅 SHA 散列算法）。

提示：Arch Linux 已于 2013 年将默认散列算法切换到 yescrypt。若未自定义过默认配置，仅需使用 `passwd` 执行一次密码修改，即可应用新的默认算法。

如果要备份密码数据库，请勿将备份的副本存储在密码保护之下（比如存储在加密的驱动器或需要身份验证的远程存储服务），而解锁副本的密码又存储在副本中，这样在需要时将无法访问它（相当于把房间的钥匙锁在了房间里）。一个有用的技巧是，使用主密码的简单哈希来加密存储密码数据库的驱动器或帐户。维护一份记录备份位置的列表：如果有一天你觉得主密码已被泄露，一是要更改所有数据库备份的密码，二是要使用从新主密码派生的新哈希来保护存有数据库的位置。

以安全的方式控制数据库的版本可能非常复杂：如果你这样做，那你必须有办法更新所有数据库版本的主密码。主密码泄露时，你可能并不能马上知道：为了降低其他人在你意识到主密码泄露之前发现密码的风险，你可以选择定期更改主密码。如果你担心自己失去了对数据库副本的控制权，则需要根据主密码的熵，在暴力破解主密码所需的时间内更改数据库副本中包含的所有密码。

密码的散列值

散列函数（又称哈希函数）是一种单向不可逆函数。其设计旨在于确保无法通过输出结果（散列值，又称哈希值）逆向推导出原始输入。优秀的散列算法（又称哈希算法）需要具备极强抗碰撞性（即很难找到两个不同输入产生相同的散列值）与雪崩效应（输入微调即导致输出结果剧烈变化）。

密码散列函数（Cryptographic hash function；例如：bcrypt）则专用于存储密码，为防止攻击者通过彩虹表、暴力穷举等手段破解密码，密码散列函数除了前述不可逆性外往往使用密钥拉伸技术或加盐（Salting）机制。

密钥派生函数（Key derivation function, KDF；例如：yescrypt、scrypt、PBKDF2、Argon2）是从一个或多个值（如主密钥、密码）中派生出一个或多个安全密钥（如 AES 密钥、密码散列值）的加密算法。现代 KDF 使用密钥拉伸技术，通过增加算法执行步骤资源消耗，使派生出的密钥在抗暴力破解能力上更优，因此，KDF 可适用于多种应用场景，其中便包括作为密码散列函数。

默认情况下，Arch 将用户密码散列值存储在仅 root 可读的 `/etc/shadow` 文件中，与存储在所有人可读的 `/etc/passwd` 文件中的其他用户参数分开，请参阅 Users and groups#用户信息存储。另请参阅 #限制 root 权限。

使用 **passwd** 命令来设置密码，该命令会调用系统的 crypt) 函数对密码进行密钥拉伸，然后将它们保存在 `/etc/shadow` 中。密码在处理中还会被加盐（salting），以

抵御彩虹表攻击。若需了解其底层原理，参见密码在 Linux 中的存储方式（理解 shadow 工具集的散列机制）。

由于密码散列值遵循特定的格式，因此可针对后续新执行的 *passwd* 命令配置不同的加密方法与参数。这也意味着，`/etc/shadow` 文件中存储的各个用户散列值可以是系统所支持的各种散列函数的混合体。

若需了解有关格式、散列方法和参数的更多信息，请参考 crypt(5)。

`/etc/login.defs` 文件用于配置默认密码散列方法 `ENCRYPT_METHOD YESCRYPT` 及其参数 `YESCRYPT_COST_FACTOR`。

例如，调高默认的 `YESCRYPT_COST_FACTOR` 参数值，将导致从密码推导哈希值所需的计算时间呈对数级增长。此时间增长对试图破解密码的第三方和进行用户登录认证的系统同样有效。

相比之下，SHA-512 哈希函数的计算时间受其参数呈线性影响。关于 Arch 之前的默认配置，请参考 SHA 散列算法。需要注意，yescrypt 算法在内部结合了 SHA-256、HMAC 和 PBKDF2 来计算密码哈希，其主要目的在于融合这些经过广泛测试的常用函数的优点，以提升抗攻击能力。由于 SHA 在各类场景中的广泛应用，硬件层面已对其提供了加速支持。这意味着纯 SHA 哈希的计算性能已大幅提升，因而直接将其用作密码哈希函数已逐渐被时代淘汰。

禁用空密码

允许空密码进行身份验证会扩大系统的攻击面。例如CVE-2020-27780 或 dirtyfrag (CVE-2026-43284 , CVE-2026-43500)——后者即为通过恶意篡改 `/etc/shadow` 并利用 `pam_unix` 中启用的 `nullok` 选项来实现攻击。 若要禁用空密码，请移除 `/etc/pam.d/system-auth` 的 `nullok` 选项：

```
/etc/pam.d/system-auth
auth        [success=1 default=bad]     pam_unix.so           try_first_pass
```

若要列出当前密码为空的用户，请执行：

```
# getent shadow | awk -F: '($2=="") {print $1}'
```

若输出结果中包含本地系统无对应账号的用户（例如通过 OpenLDAP 进行身份验证的用户），这些用户将不受移除 `nullok` 选项的影响。

用 pam_pwquality 强制使用强密码

PAM 为“可插拔身份验证模块”（Pluggable Authentication Modules）的缩写。*pam_pwquality* 提供针对字典攻击的保护，并可用于配置在整个系统中实施的密码策略。它基于 *pam_cracklib*，故也向后兼容其选项。

安装 libpwquality包 软件包。

警告：*root* 账户默认不会受此策略影响。

注意：

- 可以用 *root* 帐户来帮助想绕过策略的用户设置他们的密码。这在设置临时密码时很有用。
- 目前有关密码的安全指南（例如来自 NIST 以及其他机构的指南）并不建议强制使用特殊字符，因为它们通常只会导致可预测的更改。

举个例子，假设要强制执行下面的策略：

- 如果密码错误，则提示 2 次输入密码（retry 选项）
- 最小长度为10个字符（minlen 选项）
- 输入新密码时，新密码应至少有 6 个字符与旧密码不同（difok 选项）
- 至少 1 个数字（dcredit 选项）
- 至少 1 个大写字母（ucredit 选项）
- 至少 1 个小写字母（lcredit 选项）
- 至少 1 个除上述之外的其他字符（ocredit 选项）
- 不能包含单词 "myservice" 和 "mydomain"
- 为 root 应用此策略

编辑 `/etc/pam.d/passwd` 文件，把它改成：

```
#%PAM-1.0
password required pam_pwquality.so retry=2 minlen=10 difok=6 dcredit=-1 ucredit=-1
ocredit=-1 lcredit=-1 [badwords=myservice mydomain] enforce_for_root
password required pam_unix.so use_authtok yescrypt shadow
```

`password required pam_unix.so use_authtok` 用来告诉 *pam_unix* 模块不要显示输入密码的提示符，而采用 *pam_pwquality* 所提供的。

更多信息可以参考 pam_pwquality(8) 和 pam_unix(8) 手册页。

CPU

微码

关于如何为CPU微码安装重要安全更新的信息见微码。

硬件漏洞

有些 CPU 存在硬件漏洞。这些漏洞的列表见关于硬件漏洞的 Linux 内核文档，其中也包含了修补方法选择指南，有助于针对特定场景对内核自定义，从而修补这些漏洞。

要检查是否受到已知漏洞影响，请运行：

```
$ grep -r . /sys/devices/system/cpu/vulnerabilities/
```

大部分情况下，更新内核和微码能够修补漏洞。

同步多线程（超线程）

同步多线程（SMT），在英特尔 CPU 上又称超线程，这一硬件功能可能是 L1 Terminal Fault 与微架构数据采样（Microarchitectural Data Sampling）漏洞的来源。Linux 内核和微码更新含有针对已知漏洞的补丁，但是如果存在不受信任的虚拟化客户机，则对于某些 CPU 可能仍然需要禁用 SMT。

注意：禁用 SMT 的受益对象主要为宿主机/虚拟机监视器（Hypervisor）。对于普通系统而言，禁用 SMT 几乎无法带来安全受益。

SMT 往往可以在系统固件中关闭。更多信息见主板或系统文档。通过添加以下内核参数，也可以在内核中禁用 SMT：

```
mitigations=auto,nosmt
```

内存

Linux 内核提供了内存管理的相关设置。仓库中的内核启用了 Linux 地址空间布局随机化(ASLR)。

```
$ cat /proc/sys/kernel/randomize_va_space
2
```

运行时配置中 2 是最大值。但是还可以进一步配置，参见#用户空间 ASLR 比较。

内核的内存分配还支持操作时对内存进行清理的配置，此项功能性能开销较大，尤其在频繁分配与释放内存的场景下。但在对数据驻留时间有严格要求（如防范内存数字取证）的场景中，该选项仍具参考价值。

例如，linux包 软件包的默认设置为：

```
# dmesg | grep -i 'mem auto-init'
mem auto-init: stack:all(zero), heap alloc:on, heap free:off
```

CPU 栈清空（`stack:all(zero)`）在编译时设定，而内存池清理（`heap allow:on` 和 `heap free:off`）可通过内核参数 `init_on_alloc` 和 `init_on_free` 进行配置。例如，在添加 `init_on_free=1` 之后：

```
# dmesg | grep -i 'mem auto-init'
mem auto-init: stack:all(zero), heap alloc:on, heap free:on
```

加固 malloc

hardened_mallocAUR 是 glibc 函数 malloc() 加固过的替代品。此项目最开始由 GrapheneOS 的 Daniel Micay 开发，集成在 Android 的 Bionic) 和 musl，他也内置了对 x86_64 标准 Linux 发行版的支持。

存储

静态数据加密

静态数据加密，最好是使用强密码的全磁盘加密，是保护数据免受物理恢复的唯一方法。这在计算机关闭或相关磁盘卸载时提供了数据机密性。

然而，一旦计算机启动并且驱动器被挂载，其数据将变得与未加密的驱动器一样容易受到攻击。因此，最佳做法是在不再需要数据分区时立即卸载它们。

你还可以使用存储在 TPM 中的密钥对驱动器进行加密，尽管它过去存在可以通过总线嗅探攻击提取密钥的漏洞。

某些程序，如 dm-crypt，允许用户将 Loop file 加密为虚拟卷。当系统只有特定的部分需要加密时，这种方法可以替代全盘加密。

虽然数据静态加密wiki 中比较的基于块设备或文件系统的加密类型对于保护物理媒体上的数据很有用，但大多数不能用于保护无法控制的远程系统上的数据（例如云存储）。在某些情况下，个别文件加密会很有用。

以下是一些加密文件的方法：

- age包 是一款简单易用的文件加密工具。该工具支持多接收者模式以及使用 SSH 密钥进行加密，非常适合用于安全文件共享。
- 一些归档和压缩工具还提供基本的加密功能。由于部分工具出于兼容性考虑可能会使用弱加密算法（尤其是在使用 zip 文件格式时），因此在未充分了解特定工具及其配置的安全属性前，切勿盲目依赖此类功能。典型示例包括7-zip（`-p`参数）和zip包（`-e`参数）。
- GnuPG 亦可用于加密文件。然而，安全地使用该工具需要大量专业培训与知识储备，且其包含的 GnuPG 专属拓展与 OpenPGP 标准并不兼容。为防止常见的用户操作失误，GunPG 维护者建议使用图形化界面程序 kleopatra包，而非 GPG 命令行工具。

文件系统

如果用 sysctl 启用了内核的 `fs.protected_hardlinks` 和 `fs.protected_symlinks` 选项，内核就可以防止出现硬链接和软链接（符号链接）相关的安全问题。因此将全局可写的目录独立出来不再具有安全方面的优势。

使包含全局可写目录的文件系统保持独立 仍然可以作为一种防止恶意程序填充垃圾内容使磁盘空间耗尽的粗略手段。但是，把 `/var` 或 `/tmp` 所在分区塞满也足以使系统

停止响应。处理这种问题的更灵活的机制是存在的（比如磁盘配额），并且某些文件系统自身就带有相关功能（Btrfs 的子卷可以设置配额）。

挂载点

根据最小权限原则，挂载文件系统时应该采用最为严格的挂载选项（在不损失功能的情况下）。

相关的挂载选项有：

- `nodev`: 文件系统中，禁止解释任何字符或屏蔽特殊设备。
- `nosuid`: 禁止 set-user-identifier 或 set-group-identifier 标志位生效。
- `noexec`: 禁止文件系统里的任何二进制文件直接运行。
- 在 `/home` 上设置 `noexec` 选项会禁用可执行脚本，影响 Wine* 、PyCharm 、 Steam 、 .NET等软件正常运行。
- Wine 不需要 `exec` 标志来打开 Windows 二进制文件。仅当 Wine 本身安装在 `/home` 中时才需要它。
- 为了保持 Steam 正常工作，你可以通过在 fstab 中添加以下内容来让 `/home/user/.local/share/Steam` 以 `exec`标志挂载：

```
/home/user/.local/share/Steam  /home/user/.local/share/Steam  none defaults,bind,user,nodev,nosuid,exec,nofail  0  0
```

- 部分软件包（比如编译 nvidia-dkms包）可能需要 `/var` 目录下有 `exec` 权限。

用于存放数据的文件系统应该坚持使用 `nodev`、`nosuid` 和 `noexec` 挂载。

可能的文件系统划分参考：

- `/var`
- `/home`
- `/dev/shm`
- `/tmp`
- `/boot`

提示：使用 GPT 分区自动挂载 时，ESP 和 XBOOTLDR 分区始终使用`noexec,nosuid,nodev`挂载选项。

无 SUID

若能接受使用 Systemd/run0 替代 sudo 以及移除所有其他 SUID 和文件系统能力（Capabilities）而带来部分功能缺失，那么将包括根系统在内的所有文件系统都挂载为 `nosuid` 是完全可行的。

首先，找出系统中的 SUID 和 SGID 文件。

接着，排查带有文件系统能力（Capabilities）的二进制文件：

```
$ getcap -r /
```

由于移除了 `nvidia-modprobe` 的 SUID 权限，配备 NVIDIA 显卡的系统需要添加以下配置单元以进行补偿：

```
/etc/systemd/system/local-nvidia.service
[Service]
Type=simple
ExecStart=/usr/bin/nvidia-modprobe -m -l -s

[Install]
WantedBy=multi-user.target
```

在无 SUID 的配置中，还可以全局启用`NoNewPrivileges=yes`，这同样会禁用 SUID/SGID 以及文件系统的能力（Capabilities）：

```
/etc/systemd/system.conf.d/local-nnp.conf
[Manager]
NoNewPrivileges=yes
```

也可以尝试从 PID 1 及其子进程，或者从单个单元（Unit）中移除特定的能力（Capabilities）。参见 Capabilities 和 en:systemd/Sandboxing#Capabilities。

快照

当使用 Btrfs、LVM 或 ZFS 等文件系统快照功能时，必须注意：快照可能会保留用户认为已被删除的敏感信息。在配置了 Snapper 等自动快照工具时尤为如此，因为它们会定期或在特定系统事件发生时自动创建快照。以下是一些 `/home/` 目录下的敏感信息如何在快照中残留的典型场景：

- 已删除的文件和目录：即便文件或目录已从当前文件系统中删除，它们仍可能存在于历史快照中。在大多数情况下这符合备份预期，但需仔细考量是否应当保留如 `.local/share/Trash/`、`.history` 等文件和目录。
- 临时文件与缓存：应用程序生成的临时文件和缓存数据可能会被包含在快照中。例如，保存在加密目录中的文件在被打开时，可能会产生缩略图（`.cache/thumbnails`）或工作副本，这些内容进而可能会被写入快照。同理，浏览记录（如 `.mozila/`、`.config/chromium/` 等）也可能在被彻底清理之前就已经被保存在了某次快照中。

如果系统支持，建议考虑将此类目录完全排除在快照范围之外。例如，若使用 Btrfs，可根据实际需求，为 `.cache/`、`.local`、`.var/`或任何其他目录创建独立的子卷（Subvolume）。

注意：在某些情况下，将 `.local/share/Trash` 移动到独立的子卷可能会导致回收站功能失效，例如在GNOME/文件中。

文件访问权限

**本文或本章节的事实准确性存在争议。**

原因： `chmod go-r` 不会"取消所有权限"，其仅会移除读（read）权限（在 \[\] 中讨论）

默认的文件权限对几乎所有文件都赋予了读权限，修改文件权限可以在取得了非 root 账户（如`http` 或 `nobody` 账户）的攻击者面前隐藏有价值的信息。

您可以使用 chmod 取消组和其他人的所有权限：

例如：

```
# chmod go-r path_to_hide
```

警告：不要广泛应用这个原则。请一次尝试一种配置，确保它值得隐藏，并且不会破坏程序功能。如果依赖于组，您可能需要从命令中删除`g`（或者如果已经运行，则使用`chmod g+r *path*`重新添加权限）。

需要考虑的一些路径是：

- `/boot` ：引导目录，其中可能包含传统的 vmlinuz 和 initramfs 镜像，或是统一内核镜像（UKI）。请注意，在使用 systemd 的 GPT 分区自动挂载时，默认会采用更安全的权限设置。
- `/etc/nftables.conf` ：nftables 配置，适用于 nftables包 和 iptables-nft 包。
- `/etc/iptables`: 旧版 iptables 配置，适用于iptables-legacy包 。

可以修改默认的 Umask `0022` 来为新建的文件提高安全性。NSA RHEL5 安全指南建议将 umask 设置为 `0077` 以获得最充分的安全性，这将使得新文件无法被创建者之外的用户读取。要修改 umask，参见 Umask#Set the mask value。如果你使用 sudo，可以考虑将其配置为使用默认的 root umask。

SUID 和 SGID 文件

了解系统中哪些文件启用了 Setuid（SUID）或 Setgid（SGID）权限至关重要。若要搜索所有设置了 SUID 或 SGID 权限的文件，请运行：

```
$ find / -perm "/u=s,g=s" -type f 2>/dev/null
```

常见的已设置 SUID 权限的相关文件示例：

- unix_chkpwd
- chage、expiry、gpasswd、passwd、sg（shadow包）
- fusermount3、fusermount2
- pkexec
- ssh-keysign

- chfn、chsh、mount、newgrp、umount、wall、write（util-linux包）
- sudo、sudo-rs包、doas、su、su-rs、ksu
- firejail
- dbus-daemon-launch-helper
- chromium-sandbox
- Xorg.wrap

这类可执行文件最显著的风险在于可能存在特权提升（提权）漏洞，具体参考 Setuid 的安全影响。

未归 root 所有但设置了 SUID 权限的文件，或是设置了 SGID 权限的文件，“通常”潜在影响较小，但如果存在漏洞，理论上仍可能造成不小的危害。通常情况下，可以通过分配能力（Capabilities）来完全避免使用 SUID 或 SGID。

提示：必须保持警惕，及时更新提供 SUID/SGID 可执行文件的软件包，以防止系统产生漏洞；另请参阅无 SUID 这一替代方案

备份

备份对于安全至关重要，因为通过备份，系统可以在遭遇勒索软件、数据损坏、误删除以及系统受损时进行恢复，从而避免永久性的数据丢失。参见系统备份

SATA SSD 冻结模式

SATA 固态硬盘（SSD）在从睡眠状态唤醒时，极易受到安全擦除单元（ATA SECURITY ERASE UNIT）命令的攻击。若需降低此风险，参阅固态硬盘#从睡眠中唤醒时设置 SATA SSD 为冻结模式。

账户设置

不要日常使用 root 账户

根据最小权限原则，不要日常使用 root 账户。给每个使用系统的人建立一个没有特权的用户账户。有关临时获取特权的方法，参见应用程序列表/安全#提权。

在失败的登录尝试后强制延时一段时间

将下方的行添加至 `/etc/pam.d/system-login` 即可在失败的登录尝试后延时至少 4 秒：

```
/etc/pam.d/system-login
auth optional pam_faildelay.so delay=4000000
```

注意：此行必须作为文件中的第一行。

`4000000` 是延时的微秒数。

除 pam_faildelay 外，其他 PAM 模块也可以建议此类延迟；如果多个模块同时指定了延迟，PAM 将采用其中最长的时间。

特别是 pam_unix 和 pam_faillock，它们默认会设置至少 2 秒的最低延迟。

若要完全移除该延迟，你需要为这些模块的任何 auth 配置行添加 nodelay 参数，例如：

```
/etc/pam.d/system-auth
auth        [success=1 default=bad]     pam_unix.so          try_first_pass nullok
nodelay
```

在三次登录尝试失败后封锁用户

至 pambase包 20200721.1-2时，pam_faillock.so 默认启用，在15分钟内3次失败的登录尝试之后会封锁用户10分钟（见 FS#67644）。这一封锁只适用于密码认证（如登录及 sudo），通过 SSH 的公钥认证仍然会被接受。为防止彻底拒绝服务，该封锁对于 root 禁用。

要解锁用户，执行：

```
$ faillock --reset --user username
```

默认情况下，封锁机制为每个用户一个文件，位于 /run/faillock/。删除或清空此文件及可解锁该用户——该目录是由 root 所有，但文件是由该用户所有，故 faillock 命令只会清空文件，因此并不需要 root。

pam_faillock.so 模块可通过文件 /etc/security/faillock.conf 配置。封锁参数：

- unlock_time——封锁时间（单位为秒，默认10分钟）。
- fail_interval——导致封锁的失败尝试时间范围（单位为秒，默认15分钟）。
- deny——封锁前允许的登录失败次数（默认为3）。

提示：锁定的主要目的是减缓暴力攻击，使其变得不可行。因此，如果由于密码输入错误而导致的锁定变得过于频繁，请优先考虑放宽尝试次数，而不是减少锁定时间。

注意：deny = 0 会禁用封锁。

默认情况下，重新启动后所有用户锁都会丢失。如果攻击者可以重新启动计算机，则锁定持续存在会更安全。要使锁定持续存在，请
将 /etc/security/faillock.conf 中的 dir 参数更改
为 /var/lib/faillock 。

注意：如果你将锁定设置为持久化，受 polkit 127 引入的变更影响：你可能需要放宽其辅助代理（helper agent）的沙箱限制，以使其维持正常功能。最佳方法是通过 `}systemctl edit polkit-agent-helper@.service` 为其 systemd 服务单元创建一个配置补丁，并添加如下内容：

```
[Service]
ReadWritePaths=/var/lib/faillock
```

更改无需重启即可生效。更多配置选项见 faillock.conf(5) ，如启用 root 账户封锁、在中心化登录（如 LDAP）情况下禁用等。

限制进程数量

在有很多用户或存在不可信用户的系统上，限制每个用户同时可运行的进程数很重要，这样可以防止 fork bombs 以及其他拒绝服务攻击。`/etc/security/limits.conf` 配置文件决定了每个用户或组可以运行的进程数，默认情况下为空（除了有用的注释）。将以下行添加到此文件将限制所有用户每位最多运行 100 个活动进程，除非他们使用 `prlimit` 命令将此次会话的最大值显式地提高到 200。可以根据用户应当运行的进程数量或当前管理的系统的硬件条件来确定这些值。

```
* soft nproc 100
* hard nproc 200
```

当前每个用户运行的进程数量可以使用 `ps --no-headers -Leo user | sort | uniq -c` 查看。这可以帮助确定合适的用户进程限额。参见en:limits.conf。

显示服务器

尽量使用 Wayland 代替 Xorg 作为显示服务器。Xorg 的设计早于现代安全实践，并且被许多人认为是不安全的。例如，Xorg 应用程序可以在不活动时记录击键。

在 Wayland 中，Xwayland 兼容层将自动使用无 root 权限的 Xorg。如果必须运行 Xorg，建议避免以 root 身份运行，参考Xorg#没有 root 权限的 Xorg，并确保其不监听抽象套接字（Abstract sockets）和 TCP 套接字。

限制 root 权限

按照系统的设计，root 用户是系统中最强大的用户。审计 root 用户帐户也很困难。因此，重要的是尽可能多地限制root用户帐户的使用。有多种方法可以保持 root 用户的权力，同时限制其造成损害的能力。

用 sudo 替代 su

出于下述原因，使用 sudo 进行特权访问比 su 更好：

- sudo 记录了普通权限用户运行每个特权命令的日志。
- root 用户的密码不需要告知每个请求 root 权限的用户。

- `sudo` 可以防止用户意外地以 *root* 身份执行无需该权限的命令，因为 sudo 并没有为 root 创建完整的终端。这符合最小权限原则。
- 可以为单个用户启用单个程序的 root 权限，而不用为了运行一个程序启用该用户对 root 的完整访问权。

有关更多信息，参见Sudo#配置。

使用 sudo 编辑文件

参见Sudo#编辑文件。另外，你也可以使用像`rvim`或`rnano`这样限制了部分高级功能的的编辑器，因而可以安全地在 root 环境下运行。

限制 root 登录

正确配置 sudo 后，完全的 root 权限就可以被严格限制或停用，且不会损失太多可用性。要禁用 root 而允许使用 sudo，可以运行 `passwd -l root`。

仅允许特定用户

PAM 的 `pam_wheel.so` 模块可以做到仅允许 `wheel` 组中的用户使用 su包登录。参见Su#su 和 wheel 用户组。

拒绝 SSH 登录

即使你不想禁止 root 用户在本地登录，也最好禁止 root 通过 SSH 登录。这样做的目的是在用户可以在远程完全破坏系统之前添加额外的一层安全保护。

使用 access.conf 指定允许的登录组合

警告：如果正在使用 GNOME 49 及更高版本，应确保*gdm*用户组能够从本地登录。这可以通过添加`+:(gdm):LOCAL`规则来实现。

当用户尝试通过 PAM进行登录时,系统会检查 `/etc/security/access.conf` 文件，并从中查找第一个 与其登录属性匹配的组合。随后，系统将根据该匹配规则决定允许还是拒绝此次登录尝试。

```
+:root:LOCAL
-:root:ALL
```

我们可以为特定的组和用户设置规则。在下面的示例中，用户 `archie`、`wheel` 组和 `adm` 组的所有成员都被允许进行本地登录，而所有其他登录尝试均被拒绝：

```
+:archie:LOCAL
+:(wheel):LOCAL
+:(adm):LOCAL
-:ALL:ALL
```

更多信息请参阅 access.conf(5)

强制访问控制

强制访问控制（Mandatory Access Control, MAC）是一种安全策略，它与 Arch 以及大部分 Linux 发行版默认使用的 自主访问控制 （Discretionary Access Control, DAC）有很大的不同。MAC 本质上是对照着一个安全规则集，检查程序的每一个可能对系统造成影响的操作。与 DAC 方式相比，用户不能修改 MAC 的规则集。使用几乎任 何强制访问控制系统都将显著提高计算机的安全性，尽管它们的实现方式存在差异。

基于路径的MAC

基于路径的访问控制是一种简单的访问控制形式，它根据文件的路径提供相应的权限。这种方式有缺点，就是当文件改变路径以后相应的权限并没有随着文件一起移动，或是在给无法访问的文件添加硬链接将使文件可被访问。从积极的方面来说，基于路径的 MAC 可以在更广泛的文件系统上实现，这与基于标签的另一种可选方案是不同的。

- AppArmor 是一个由 Canonical 维护的 MAC 实现，它被看做是 SELinux 的简化版替代方案。
- TOMOYO Linux 是另一种提供访问控制的系统，简单而易用。它的使用方式和内部实现都被设计得足够简单，需要的依赖也很少。

基于标签的MAC

基于标签的访问控制意味着每份文件都带有扩展属性用于决定它的安全权限。虽然这类系统应该比基于路径的控制更灵活，但它只适用于支持这些扩展属性的文件系统。

- SELinux 是一个基于 美国国家安全局(NSA) 的项目，用于提高 Linux 的安全性。它完整实现了 MAC，将系统用户与角色独立开来。它实现了一个非常强大的多级 MAC 策略，可以在系统扩大和改变得超出其原始配置时也能轻松控制系统。

访问控制表 (ACLs)

访问控制列表（Access Control Lists, ACLs）是以某种方式将规则直接附加到文件系统的一种可选方法。ACLs 将程序的实际操作与允许的操作对照，来实现访问控制。

内核加固

内核自我保护 / 漏洞防护

linux-hardened包 相较于 linux包 使用了基本内核加固补丁集和更多安全相关的编译时配置选项。也可以使用自定义编译选项来把握安全性和性能之间的平衡，而不是使用强调安全性的默认选项。

然而需要注意的是，当使用该内核时，某些软件包（例如throttled包）将无法正常工作。

如果你用了内核代码树以外的驱动，比如 NVIDIA，可能会需要切换到它的 DKMS 包。

用户空间 ASLR 比较

linux-hardened包 包为用户空间进程提供了改进的 ASLR（Address Space Layout Randomization，地址空间布局随机化）实现。paxtest包 命令可用于获取所提供熵的估计值：

64 位进程

```
linux-hardened 5.4.21.a-1-hardened
Anonymous mapping randomization test     : 32 quality bits (guessed)
Heap randomization test (ET_EXEC)        : 40 quality bits (guessed)
Heap randomization test (PIE)            : 40 quality bits (guessed)
Main executable randomization (ET_EXEC)  : 32 quality bits (guessed)
Main executable randomization (PIE)      : 32 quality bits (guessed)
Shared library randomization test        : 32 quality bits (guessed)
VDSO randomization test                  : 32 quality bits (guessed)
Stack randomization test (SEGMEXEC)      : 40 quality bits (guessed)
Stack randomization test (PAGEEXEC)      : 40 quality bits (guessed)
Arg/env randomization test (SEGMEXEC)    : 44 quality bits (guessed)
Arg/env randomization test (PAGEEXEC)    : 44 quality bits (guessed)
Offset to library randomisation (ET_EXEC): 34 quality bits (guessed)
Offset to library randomisation (ET_DYN) : 34 quality bits (guessed)
Randomization under memory exhaustion @~0: 32 bits (guessed)
Randomization under memory exhaustion @0 : 32 bits (guessed)
linux 5.5.5-arch1-1
Anonymous mapping randomization test     : 28 quality bits (guessed)
Heap randomization test (ET_EXEC)        : 28 quality bits (guessed)
Heap randomization test (PIE)            : 28 quality bits (guessed)
Main executable randomization (ET_EXEC)  : 28 quality bits (guessed)
Main executable randomization (PIE)      : 28 quality bits (guessed)
Shared library randomization test        : 28 quality bits (guessed)
VDSO randomization test                  : 20 quality bits (guessed)
Stack randomization test (SEGMEXEC)      : 30 quality bits (guessed)
Stack randomization test (PAGEEXEC)      : 30 quality bits (guessed)
Arg/env randomization test (SEGMEXEC)    : 22 quality bits (guessed)
Arg/env randomization test (PAGEEXEC)    : 22 quality bits (guessed)
Offset to library randomisation (ET_EXEC): 28 quality bits (guessed)
Offset to library randomisation (ET_DYN) : 28 quality bits (guessed)
Randomization under memory exhaustion @~0: 29 bits (guessed)
Randomization under memory exhaustion @0 : 29 bits (guessed)
linux-lts 4.19.101-1-lts
Anonymous mapping randomization test     : 28 quality bits (guessed)
Heap randomization test (ET_EXEC)        : 28 quality bits (guessed)
Heap randomization test (PIE)            : 28 quality bits (guessed)
Main executable randomization (ET_EXEC)  : 28 quality bits (guessed)
Main executable randomization (PIE)      : 28 quality bits (guessed)
Shared library randomization test        : 28 quality bits (guessed)
VDSO randomization test                  : 19 quality bits (guessed)
Stack randomization test (SEGMEXEC)      : 30 quality bits (guessed)
Stack randomization test (PAGEEXEC)      : 30 quality bits (guessed)
Arg/env randomization test (SEGMEXEC)    : 22 quality bits (guessed)
```

```
Arg/env randomization test (PAGEEXEC)     : 22 quality bits (guessed)
Offset to library randomisation (ET_EXEC): 28 quality bits (guessed)
Offset to library randomisation (ET_DYN) : 28 quality bits (guessed)
Randomization under memory exhaustion @~0: 28 bits (guessed)
Randomization under memory exhaustion @0 : 28 bits (guessed)
```

32 位进程（运行在 x86_64 内核上）

```
linux-hardened
Anonymous mapping randomization test      : 16 quality bits (guessed)
Heap randomization test (ET_EXEC)         : 22 quality bits (guessed)
Heap randomization test (PIE)             : 27 quality bits (guessed)
Main executable randomization (ET_EXEC)   : No randomization
Main executable randomization (PIE)       : 18 quality bits (guessed)
Shared library randomization test         : 16 quality bits (guessed)
VDSO randomization test                   : 16 quality bits (guessed)
Stack randomization test (SEGMEXEC)       : 24 quality bits (guessed)
Stack randomization test (PAGEEXEC)       : 24 quality bits (guessed)
Arg/env randomization test (SEGMEXEC)     : 28 quality bits (guessed)
Arg/env randomization test (PAGEEXEC)     : 28 quality bits (guessed)
Offset to library randomisation (ET_EXEC): 18 quality bits (guessed)
Offset to library randomisation (ET_DYN) : 16 quality bits (guessed)
Randomization under memory exhaustion @~0: 18 bits (guessed)
Randomization under memory exhaustion @0 : 18 bits (guessed)
linux
Anonymous mapping randomization test      : 8 quality bits (guessed)
Heap randomization test (ET_EXEC)         : 13 quality bits (guessed)
Heap randomization test (PIE)             : 13 quality bits (guessed)
Main executable randomization (ET_EXEC)   : No randomization
Main executable randomization (PIE)       : 8 quality bits (guessed)
Shared library randomization test         : 8 quality bits (guessed)
VDSO randomization test                   : 8 quality bits (guessed)
Stack randomization test (SEGMEXEC)       : 19 quality bits (guessed)
Stack randomization test (PAGEEXEC)       : 19 quality bits (guessed)
Arg/env randomization test (SEGMEXEC)     : 11 quality bits (guessed)
Arg/env randomization test (PAGEEXEC)     : 11 quality bits (guessed)
Offset to library randomisation (ET_EXEC): 8 quality bits (guessed)
Offset to library randomisation (ET_DYN) : 13 quality bits (guessed)
Randomization under memory exhaustion @~0: No randomization
Randomization under memory exhaustion @0 : No randomization
```

手动配置

在 linux-hardened包 之外的内核上，也可以通过使用以下 sysctl 参数来达到相同的效果：

```
/etc/sysctl.d/51-aslr.conf
vm.mmap_rnd_bits = 32
vm.mmap_rnd_compat_bits = 16
```

限制访问 proc 文件系统中的内核指针

注意：linux-hardened包 默认设置是 `kptr_restrict=2` 而不是 `0`。

将 `kernel.kptr_restrict` 设置为 1 将针对没有 `CAP_SYSLOG` 的普通用户隐藏 `/proc/kallsyms` 中的内核符号地址，这使得利用内核漏洞动态解析地址/符号变得更加困难。这对预编译好的 Arch Linux 内核没有多大帮助，因为攻击者可以直接下载到内核包并从那里手动获取符号，但如果你正在编译自己的内核，这可以帮助减轻本地根攻击。这样做以后会影响非 root 用户运行部分 perf包 命令（虽然很多 perf包 功能本身就需要 root 权限）。更多信息请参见 FS#34323。

将 `kernel.kptr_restrict` 设置为 2 将隐藏 `/proc/kallsyms` 中的内核符号地址，而忽略用户的权限。

```
/etc/sysctl.d/50-kptr-restrict.conf
kernel.kptr_restrict = 1
```

BPF 加固

BPF 是一种用在运行时动态向内核加载并执行字节码的系统。它被广泛应用于许多 Linux 内核子系统中，例如网络（如 XDP、tc）、跟踪（如 kprobes、uprobes、tracepoints）以及安全（如 seccomp）。此外，它在高级网络安全、性能分析和动态跟踪方面也大有裨益。

BPF 最初是伯克利包过滤器（Berkeley Packet Filter）的缩写，因为最初的经典 BPF 仅用于 BSD 的数据包捕获工具。该技术最终演变为扩展 BPF（eBPF），不久后被简称为 BPF（不再作为缩写）。不应将 BPF 与 iptables 或 netfilter 等数据包过滤工具混淆，尽管 BPF 可以用于实现此类数据包过滤工具。

BPF 代码既可被解释执行，也可以使用即时编译器（JIT）进行编译。Arch 内核在构建时启用了`CONFIG_BPF_JIT_ALWAYS_ON`，该选项会禁用 BPF 解释器，并强制所有 BPF 均使用 JIT 编译。这使得攻击者更难利用 BPF 来扩大对 SPECTRE（幽灵）这类漏洞的攻击。有关更多细节，参见引入 CONFIG_BPF_JIT_ALWAYS_ON 的内核补丁。

内核包含一项针对 JIT 编译 BPF 的加固特性，该特性可以缓解某些类型的 JIT 喷射（JIT spraying）攻击，但代价是会损失部分性能，并会使许多 BPF 程序的跟踪与调试功能失效。可以通过将`net.core.bpf_jit_harden`设置为`1`（仅对非特权代码启用加固）或`2`（对所有代码启用加固）来开启此功能。

有关更多信息，参见内核文档中的 `net.core.bpf_*` 设置。

提示：

- linux-hardened包 默认将该值设置为 `net.core.bpf_jit_harden=2` 而非 `0`。
- 默认情况下，非特权用户也可以运行 BPF 程序。若要更改此行为，请设置 `kernel.unprivileged_bpf_disabled=1`。

ptrace 作用域

ptrace(2) 系统提供了一种手段，使得一个进程（“追踪者”）可以观察和控制另一个进程（“被追踪者”）的执行情况，并检查或更改被追踪者的内存和寄存器。`ptrace` 通常被 *gdb*、*strace*、*perf*、*reptyr* 等调试及跟踪工具使用。然而，它也为恶意进程读取其他进程的数据并获取控制权提供了可乘之机。

Arch 默认启用了 Yama LSM，它提供了一个 `kernel.yama.ptrace_scope` 内核参数。该参数默认设置为 `1`（受限模式），这会阻止追踪者对受限作用域之外的进程执行 `ptrace` 调用，除非追踪者拥有特权或具备 `CAP_SYS_PTRACE` 能力。与传统权限相比，这在安全上是一次显著的提升。如果没有这个模块，在没有引入额外的安全层（如pid_namespaces(7)）的情况下，以相同用户运行的各进程之间将毫无隔离可言。

注意：默认情况下，若需使用依赖 `ptrace` 的工具，仍可通过以特权进程身份运行它们来实现（例如使用Sudo）。

如果无须使用调试工具，可以考虑将 `kernel.yama.ptrace _scope` 设置为 `2`（仅限管理员）或 `3`（完全禁用 `ptrace`）以加固系统。

注意：某些反作弊或数字版权管理（DRM）实现需要依赖 `ptrace` 才能工作，包括 Wine 环境下的 Easy Anti-Cheat 和 Ubisoft Connect。将此参数设置为 `2` 或更高可能会导致使用这些方案的游戏无法启动。

隐藏进程标识符

**本文或本章节的事实准确性存在争议。**

原因：全局启用 `hidepid` 并不是 systemd 所支持的操作方式；此外，在 systemd 作为服务管理器运行的操作系统上，全局启用它在安全层面上并不会带来任何实质性的提升。（在 Talk:安全 中讨论）

警告：

- 这可能会导致某些应用程序出现问题，例如在沙盒和 Xorg 中运行的应用程序（请参阅解决方法）。
- 当所使用的 systemd包 版本大于 237.64-1 时，这可能会导致 D-Bus、PulseAudio 和 bluetooth 出现问题。

其他用户的进程通常可以在 `/proc` 访问到，而内核有能力向非特权用户隐藏这些进程，按照此处记录的方法以 `hidepid=` 和 `gid=` 选项挂载 `proc` 文件系统即可。

这使得入侵者收集正在运行的进程信息变得异常艰难，同样艰难的还包括判断是否存在特权运行的守护进程、其他用户是否正在运行敏感程序，甚至无法捕捉到其他用户运行的任何程序，更无法捕捉到特定的程序是否正在运行（假设该程序不会因为其自身行为暴露自己），并且作为额外的优势，那些通过外部参数传递敏感信息的、写得不那么好的程序现在可以防范本地窃听了。

由 filesystem包 软件包提供的 `proc` 用户组相当于一个白名单，包含了有权访问其他用户进程信息的用户。如果用户或服务需要访问除自身以外的 `/proc/<pid>` 目录，请将他们加入该用户组。

例如，把除了 `proc` 组中用户之外的其他用户的进程信息都隐藏起来：

```
/etc/fstab
proc   /proc  proc   nosuid,nodev,noexec,hidepid=2,gid=proc    0       0
```

要使用户会话正常工作，需要为 *systemd-logind* 添加例外：

```
/etc/systemd/system/systemd-logind.service.d/hidepid.conf
[Service]
SupplementaryGroups=proc
```

限制模块加载

默认的 Arch 内核启用了 `CONFIG_MODULE_SIG_ALL`，这会对作为 linux包 软件包一部分构建的所有内核模块进行签名。这使得内核只能加载带有有效签名的模块，也就是说，本地编译的树外（out-of-tree）模块或由 nvidia-open包 等软件包提供的模块将被无法加载。可以使用 `modinfo` 来验证当前加载的模块是否包含签名；而手动验证签名则稍微复杂一些。

可以通过设置 `module.sig_enforce=1` 内核参数来限制内核模块的加载。更多信息可以在内核文档中找到。

此外，可以将不需要的特定模块列入黑名单，具体示例可以参考secureblue。

可以通过 sysctl 限制能力（Capabilities），以加固内核自动加载模块机制。

禁用模块加载

某些安全漏洞（例如 Copy Fail）存在于大多数用户并不需要的内核模块中。禁用所有无关模块的加载可以防止此类漏洞被利用。要使此方案可行，内核通常必须处于稳定状态，这意味着一旦系统完全启动，通常就不再需要加载任何模块。当内核达到该状态时，可以切换 modules_disabled 选项，以不可逆地禁用模块的加载和卸载，直至下一次重启后失效。

禁用 kexec

Kexec 允许替换当前正在运行的内核。

```
/etc/sysctl.d/51-kexec-restrict.conf
kernel.kexec_load_disabled = 1
```

提示：在 linux-hardened包 中，kexec 默认已被禁用

Kernel 锁定模式（Lockdown）

Linux 支持一项可选的锁定（Lockdown）特性，旨在强化 UID 0（root）与内核之间的边界。启用此功能后，某些依赖对硬件或内核进行底层访问的应用程序可能会停止工作。

要使用锁定功能，必须先初始化其 LSM（Linux 安全模块）并设置一种锁定模式。

所有官方支持的内核都会初始化该 LSM，但默认都不会强制执行任何锁定模式。

提示：可以通过运行 `cat /sys/kernel/security/lsm` 来验证已初始化的 LSM。

锁定模式有两种运行级别：

- `integrity`（完整性）：禁止允许用户空间修改正在运行的内核的特性（例如 kexec、bpf）。
- `confidentiality`（机密性）：在完整性的基础上，进一步禁用允许用户空间从内核提取机密信息的内核特性。

除非特定的威胁模型另有要求，否则建议使用 `integrity` 模式。

若要在运行时启用内核锁定，请运行：

```
# echo mode > /sys/kernel/security/lockdown
```

若要在引导时启用内核锁定，请使用内核参数 `lockdown=*mode*`。

注意：

- 内核锁定在运行时无法被禁用。
- 内核锁定会禁用休眠（挂起到磁盘）功能。
- 在早于 6.17 的 kernel_lockdown(7) 手册页版本中，错误地写到“如果系统在 EFI 安全启动模式下引导，锁定将被自动启用”。这既不是上游内核的行为，也不是 Arch 打包的内核的行为。

另请参见 kernel_lockdown(7)。

Linux 内核运行时守护（LKRG）

LKRG（lkrg-dkmsAUR）是一个用于对内核进行完整性检查并检测漏洞利用尝试的内核模块。

禁用紧急 Shell

**本文或本章节的事实准确性存在争议。**

原因：屏蔽（Masking）`emergency.target` 和 `emergency.service` 对这些单元被添加到 initramfs 并在早期用户空间（early userspace）中运行不会产生任何影响。即使它们存在于 initramfs 中，出于“安全原因”，mkinitcpio 的 systemd 钩子也会锁定 root 账户（参见FS#70408）。如果确实需要解决相关文章中提到的问题，解决方法应当是阻止 `rescue.target`、`rescue.service`、`emergency.target` 和 `emergency.service` 被添加到 initramfs 镜像中。（在 Talk:安全 中讨论）

紧急 Shell（Emergency shell）用于在引导过程中对机器进行交互式故障排除。然而，它也是攻击者可以用来访问 TPM 等安全资源的工具。有关实际案例，参见这篇文章。禁用紧急 Shell 可以增加攻击的难度，但代价是移除了一款用于排查早期引导故障的工具。

若要禁用紧急 Shell，参见Systemd#在远程主机上关闭救援（emergency）模式。

禁用非特权用户命名空间

除 linux-hardened包 外，在所有官方支持的内核中，非特权user_namespace(7)（用户命名空间）的使用默认都是启用的。非特权用户命名空间极大地扩大了本地特权提升的攻击面；参见 AppArmor 的 Wiki 和 FS#36969。

为缓解此风险，可以执行以下操作之一：

- 使用已具备安全默认设置的 linux-hardened包 内核，或
- 将 `kernel.unprivileged_userns_clone` Sysctl 参数设置为 `0`。

注意，这可能会破坏诸如 nsjail包 等应用程序。在此设置下，基于 Chromium 的应用程序需要为 `chrome-sandbox` 开启 SUID 位才能正常工作。

限制系统服务

可以对系统服务允许执行的操作进行限制。使用 `systemd-analyze security` 命令可以审查机器上 systemd 服务单元的安全属性，阅读专属文章 en:systemd/sandbox 以了解更多信息。

沙箱程序

另请参阅 Wikipedia:Sandbox (computer security))。

Firejail

Firejail是一种易于使用的用于沙盒化应用程序和服务器的工具。它原本是为浏览器和面向互联网的应用程序创建的，但现在已经支持大量的应用程序。为了建立具有各种特性的沙盒环境，它被安装为一个suid二进制文件，并根据黑名单和白名单为目标应用程序构建一个沙盒化的运行时环境。

bubblewrap

bubblewrap 是为无特权容器工具（如 Flatpak）开发的沙箱应用程序，其资源占用和复杂性都远远小于Firejail。虽然它缺少某些功能，如文件路径白名单，但bubblewrap确实提供了bind挂载以及创建用户/IPC/PID/网络/cgroup命名空间的功能，并且可以支持简单和复杂的沙箱。对于 linux-hardened包 内核，你需要使用 bubblewrap-suid包

Bubblejail沙箱基于bubblewrap，并提供了一个面向资源的权限模型，用户可以通过图形界面进行权限的调整。

Portable

Portable 是一个沙箱框架，它利用 bubblewrap 和许多其他工具来锁定运行中的应用程序。该框架旨在为打包者提供便利，同时为用户保证效率，并且默认切断安全漏洞并监控后台进程。

关于由 portable 沙箱化的应用程序仓库，参见 portable-arch。

如果沙箱化的应用程序没有使用 Portal 文件选择器，portable 可以将文件传递给沙箱（通过传递 `--actions share-files` 参数）。

Portable 在 GNOME 上功能完全正常，而其他桌面环境可能会缺少诸如高级后台监控和屏幕截图 Portal 等少量特性

chroots

也可以手动构建chroot囚禁来创建沙箱化的进程环境。其相比其他沙箱技术的功能有限；它的沙箱程度仅限于文件路径隔离。

gVisor

由 Google 主导的 gVisor 项目提供了一款沙箱应用程序，重点专注于遵循 OCI 倡议的容器技术，例如 Docker 和 Kubernetes。它通过拦截绝大多数发往内核的系统调用，并将自身伪装为用户机内核（Guest Kernel），从而实现容器及单个应用程序与宿主机的隔离。

与其他拦截型沙箱项目的一个关键区别在于，gVisor 使用 Go 语言重新实现了系统调用，具体详见其设计概述。有关已重新实现的系统调用列表细节，可以在 git 中查看。关于使用示例、局限性以及特殊特性，请参阅此项目的项目文档。

该应用程序可以通过 gvisor-gitAUR 和 gvisor-binAUR 获取。

Linux容器

当你需要比其他选项提供的更多分离时（简略于 #完全虚拟化选项），Linux Containers是另一个很好的选择。LXC在现有内核之上运行，采用伪chroot，并拥有自己的虚拟硬件。

完全虚拟化选项

如果你计划运行风险应用程序或浏览危险网站，使用完全虚拟化选项如VirtualBox, KVM，Xen或 Qubes OS (基于Xen)也可以提高隔离和安全性。

网络与防火墙

防火墙

虽然源里面的 Arch 内核能够启用 Netfilter 的 nftables（以及它的前身 iptables），但其服务默认是不启用的。强烈建议配置防火墙来保护系统上运行的服务。许多资料（包括 ArchWiki）没有明确说明哪些服务值得保护，因此启用防火墙是一个很好的预防措施。

- 参阅 nftables 来获取一般信息。
- 参阅 Uncomplicated Firewall 来获取配置一个基础防火墙的指南。
- 参阅 Category:Firewalls 来获取设置 netfilter 的其他方法。
- 参阅 Ipset 以设置 IP 地址黑名单，内容可参考来自 Bluetack 的名单。

开放端口

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：“开放端口”（Open ports）并不是一个好标题，因为它忽略了应用程序可能绑定的接口和地址。从防火墙角度来看，即使此时没有应用程序在监听，端口也可能是“开放”的/（在Talk:安全讨论）

一些服务会在开放的网络端口上监听入站流量。只将这些服务绑定到严格必要的地址和接口是非常重要的。远程攻击者可能能够利用有缺陷的网络协议访问暴露的服务。即使在绑定到localhost的进程中，这种情况也可能发生。

总的来说，如果一个服务只需要对本地系统可访问，那么就绑定到一个Unix域套接字(unix(7))或者一个回环地址，比如`localhost`，而不是非回环地址，比如`0.0.0.0/0`。

如果一个服务需要通过网络对其他系统可访问，那么就通过严格的防火墙规则控制访问，并且尽可能配置认证、授权和加密。

你可以使用`ss -l`来列出所有当前开放的端口。要显示所有**正在监听的**进程及其**数值化的 tcp** 和 **udp** 端口号：

```
# ss -lpntu
```

查看 ss(8) 以获取更多选项。

内核参数

作用于网络的内核参数可以使用 Sysctl 来设置。要查询具体方法，请参阅 Sysctl#TCP/IP stack hardening。

SSH

为了减轻暴力攻击，建议强制使用基于密钥的身份验证。对于 OpenSSH，请参阅 OpenSSH#保护。另外，Fail2ban 或 Sshguard 通过监控日志并写入 iptables 规则提供了较少形式的保护，但这样做可能会使服务器拒绝服务，因为攻击者可以伪装成管理员的地址并发送欺骗性的数据包。

你可以用双因素身份验证来进一步强化身份验证。Google Authenticator（Google 身份验证器）使用一次性密码 (OTP) 提供两步验证过程。

拒绝 root 登录也是一种很好的做法，既可以跟踪入侵，也可以在 root 访问之前添加额外的安全层。对于 OpenSSH，请参阅 OpenSSH#Deny。

Mozilla公开发布了一个OpenSSH配置指南，其中设置了更为详尽的审计日志记录，并限制了密码。

DNS

默认的域名解析（DNS）配置具有很高的兼容性，但存在安全弱点。查看域名解析#隐私与安全获得更多信息。

代理

代理通常用作应用程序和网络之间的额外层，对来自不可信来源的数据进行清理。从攻击面上看，一个以较低权限运行的小型代理的攻击面显著小于以最终用户权限运行的复杂应用程序。

例如，DNS解析器在glibc包中实现，该解析器与应用程序（可能以root身份运行）链接，因此DNS解析器中的错误可能导致远程代码执行。通过安装DNS缓存服务器，例如dnsmasq，它可以起到代理的作用，可以防止这种情况发生。

管理TLS证书

请查看 TLS#信任管理。

物理安全

只要有足够的时间和资源，对计算机的物理访问就等同于 root 权限的访问。然而，通过设置足够的障碍，可以获得较高的*实际*安全级别。

攻击者可以通过简单地连接一个恶意的IEEE 1394（FireWire）、雷电或PCI Express设备，即可在下次启动时完全控制您的计算机，因为这些设备默认被赋予全内存访问权限。 对于雷电，您可以完全限制直接内存访问或仅限于已知设备，参见雷电#用户

设备授权。对于Firewire和PCI Express，防止此类事件或硬件自身的修改（例如在驱动器上刷新恶意固件）您能做的很少。但是，大多数攻击者并非这么有知识和决心。

#静态数据加密可以防止计算机被盗时对您的数据的访问，但是资源充足的攻击者可以安装恶意固件，在您下次登录时获取此数据。

锁定BIOS

在BIOS中添加密码可以防止他人启动至可移动媒体，这基本上相当于对您的计算机拥有根访问权限。您应确保您的驱动程序在启动顺序中排在第一，并尽可能禁用其他驱动程序的启动功能。

引导加载程序 (Bootloader)

保护您的引导加载程序非常重要。一个未受保护的启动加载器可以绕过任何登录限制：例如通过设置`init=/bin/sh`内核参数以直接启动到shell，它会使得任何用户的登录限制完全无用。

Syslinux

Syslinux支持对启动加载器进行密码保护。它允许您设置菜单项密码 或者 全局启动加载器密码。

GRUB

GRUB也支持启动加载器密码。请查看GRUB/技巧和窍门#用密码保护 GRUB 菜单以获取详细信息。它还支持 #加密的/boot，这可以加密GRUB的配置，内核和 initramfs ，只剩下启动加载器代码中的一部分未加密。

systemd-boot

当启用了#安全启动（Secure Boot）时，systemd-boot会自动禁用对内核参数的编辑功能。此外，也可以在 systemd-boot 中设置带密码保护的内核参数编辑器，以使用更传统的密码验证方案。

安全启动

安全启动是UEFI的功能，允许验证你的计算机启动的文件。这有助于防止一些邪恶女仆攻击，如替换启动分区内的文件。通常，计算机会携带由供应商（OEM）授予的密钥，不过，密钥可以被删除，并使计算机进入*设置模式*，允许用户导入并管理自己的密钥。

安全启动页面会引导你如何通过使用自己的密钥来设置安全启动。

可信平台模块 (TPM)

TPMs 是带有嵌入式加密密钥的硬件微处理器。这构成了大多数现代电脑的基本信任根（源），允许对启动链执行端到端验证。它们可用作内部智能卡，验证计算机上运行的固件，并允许用户将密码插入防篡改和抗暴力破解的存储器中。

在可移动闪存驱动器上的启动分区

一个流行的想法是将启动分区放在闪存驱动器上，以便在没有它的情况下系统无法启动。提倡这个想法的人通常会使用全盘加密，有些人还会使用放在启动分区的分离的加密头。

自动注销

如果您使用Bash或Zsh，您可以设置 `TMOUT` 在超时后自动从shell注销。

例如，以下操作将在虚拟控制台（但不是X11中的终端模拟器）自动退出：

```
/etc/profile.d/shell-timeout.sh
TMOUT="$(( 60*10 ))";
[ -z "$DISPLAY" ] && export TMOUT;
case $( /usr/bin/tty ) in
    /dev/tty[0-9]*) export TMOUT;;
esac
```

如果你确实希望每一个Bash/Zsh提示符（甚至在X内）都有超时，使用：

```
$ export TMOUT="$(( 60*10 ))";
```

注意，如果有一些命令在shell中运行（例如：一个SSH会话或没有`TMOUT`支持的其他shell），这将不起作用。但是，如果你主要是用VC来重启冷冻的GDM/Xorg作为root，那么这会非常有用。

保护免受恶意USB设备的攻击

内核提供了用于停用 USB 端口的设置，以保护计算机免受恶意 USB 设备（又称 BadUSB、PoisonTap 或 LanTurtle）的侵害。这些设置可以在运行时配置，并通过 sysctl 实现自动化。

如果需要更精细的控制，可以安装 USBGuard。这是一个基于设备属性实现基础白名单和黑名单功能的软件框架。

易失性数据收集

开启状态的计算机可能会受到易失性数据收集的威胁。将计算机完全关闭，在无需使用时或者计算机的物理安全暂时受到破坏时（例如，通过安全检查点），是一种最好的做法。

软件包

验证

如果没有正确地对包进行签名，包管理器就有可能受到攻击，甚至可能会影响原本具有签名机制的包管理器。Arch 默认采用软件包签名机制，并依赖与 5 个受信任的主密钥的信任网络。详情请参阅 pacman/Package signing。

升级

定期升级系统非常重要。

关注漏洞警报

可订阅由国家漏洞数据库提供的Common Vulnerabilities and Exposure（CVE）安全警报更新，在NVD Download网页上可以找到。

如果您通过主要仓库或AUR之外的其他方式安装软件，您还应考虑订阅您使用的软件的 发布通知。一些软件有您可以订阅的安全通知邮件列表。源代码托管网站通常提供可以接收新版本发布消息的RSS源。参见 Arch 安全团队#资源。

重新构建软件包

软件包可以去除不需要的功能并重新构建，这样可以缩小受攻击范围。例如，bzip2包可以在没有 `bzip2recover` 的情况下重新构建，以试图规避 CVE-2016-3189 漏洞。强化安全的自定义编译参数也可以手动或通过包装器在编译时加入。

**本文或本章节可能需要合并到Arch 打包准则/安全。**

附注： 有关安全相关的构建标志，其有自己的文章。（在 Talk:安全 中讨论）

**本文或本章节的事实准确性存在争议。**

原因： 复制粘贴自从 3 年前的博客文章。编译器标志特定于 GCC，有些几乎与安全无关。（在 Talk:安全 中讨论）

参数（Flag）	用途（Purpose）
-D_FORTIFY_SOURCE=2	检测运行时缓冲区溢出
-D_GLIBCXX_ASSERTIONS	C++ 字符串和容器的运行时边界检查
-fasynchronous-unwind-tables	提高回溯（Backtrace）的可靠性
-fexceptions	启用基于表的线程取消机制
-fpie -Wl,-pie	为可执行文件启用完全地址布局空间随机化（ASLR）
-fpic -shared	共享库无文本重定位
-fplugin=annobin	生成用于加固质量控制的数据
-fstack-clash-protection	提高栈溢出检测的可靠性
-fstack-protector, -fstack-protector-all or -fstack-protector-strong	栈粉碎保护（Stack Smashing Protector）
-grecord-gcc-switches	在调试信息中存储编译器参数
-mcet -fcf-protection	控制流完整性（CFI）保护

参数（Flag）	用途（Purpose）
-Werror=format-security	拒绝潜在不安全的文件格式化字符串参数
-Werror=implicit-function-declaration	拒绝隐式函数声明（缺少函数原型）
-Wl,-z,defs	检测并拒绝欠链接（Underlinking）
-Wl,-z,now	禁用延迟绑定（Lazy binding）
-Wl,-z,relro	重定位后将相关段设为只读

- 参数和信息源

参考资料

- unixchad 的视频：《捍卫隐私的真正方法：Firejail，全盘加密，GPG和ufw》
- Arch Linux Security Tracker
- CentOS Wiki: OS Protection
- Hardening the Linux desktop
- Hardening the Linux server
- Linux Foundation: Linux workstation security checklist
- privacytools.io Privacy Resources
- Red Hat Enterprise Linux 7 Security Guide
- Securing Debian Manual
- The paranoid #! Security Guide

第 2 章　共 11 节　318 KB

静态数据加密

静态数据加密处理的是设备脱离本机后的暴露面：盘被拆走、被盗或报废时，里面的内容还能不能被读出来，又不拖累启动与日常使用。主线是 dm-crypt，fscrypt 只是覆盖目录级加密的替代路线。

必读《静态数据加密》《Dm-crypt》《Dm-crypt/设备加密》《Dm-crypt/加密整个系统》，再用《Dm-crypt/系统配置》补早期解密与疑难解答；只加密数据盘或单个文件容器，读到《Dm-crypt/加密非root文件系统》即可，交换分区、特殊应用、Systemd-cryptenroll 可按需再翻。最容易踩的坑是备份：LUKS 头部或密钥文件一丢，口令再对也打不开数据，所以先按《Dm-crypt/准备磁盘》安全擦除，再把头部备份放到加密区之外。也别以为根分区加密完就结束，未加密的 boot、休眠所需的交换分区、TRIM 与性能的取舍，都会把强度打回去。

2.1　Dm-crypt

难度 入门 | **篇幅** 4.1 KB | **所属** 静态数据加密

本节将讲到

- 用法
- 示例方案
- 另请参见

dm-crypt 是 Linux 内核的设备映射器（device mapper）的加密模块（crypto target）。根据Wikipedia:dm-crypt词条的定义：

: 它是 Linux 内核中的透明磁盘加密子系统... 该模块通过设备映射器框架实现，可堆叠在其他设备映射器转换层之上。因此能够加密整个磁盘（包括可移动介质）、分区、软件 RAID 卷、逻辑卷以及文件。加密后的存储单元表现为块设备，可用于承载文件系统、作为交换空间或 LVM 物理卷。

用法

/准备磁盘 : 处理诸如安全擦除硬盘驱动器数据和分区的 dm-crypt 特定点等操作。 **/设备加密** : 介绍如何通过 cryptsetup 命令手动使用 dm-crypt 加密系统。 它涵盖了使用 dm-crypt 的加密选项的示例，处理密钥文件的创建，用于密钥管理以及备份与恢复的 LUKS 特定命令。 **/系统配置** : 说明在加密系统时如何配置mkinitcpio、内核参数和crypttab文件。 **/交换分区加密** : 介绍如何在需要时向加密系统添加交换分区。交换分区也必须加密，以保护系统交换出的任何数据。本部分详细介绍了无需休眠支持和需要休眠支持的方法。 **/特殊应用** : 处理一些特殊的操作，如保护未加密的boot分区，使用GPG、LUKS、OpenSSL加密的密钥文件，通过网络启动和解锁，为SSD设置

discard/TRIM，使用encrypt钩子解密多个设备。 **/登录时挂载（英语：Dm-crypt/Mounting_at_login）**

示例方案

/加密非根文件系统：如果你需要加密一个不用于启动系统的设备，比如一个分区（英文）或一个文件容器（英文）。 **/加密整个系统**：如果您想加密整个系统，特别是根分区。本文涵盖了使用带有**LUKS**扩展的**dm-crypt**、**plain**模式加密以及加密和**LVM**的情况。

另请参见

- dm-crypt - 项目主页
- cryptsetup - LUKS主页、FAQ，这是最重要的参考资料。

2.2 Dm-crypt/加密非root文件系统

难度 进阶 | **篇幅** 7.1 KB | **所属** 静态数据加密

本节将讲到

- 总览
- 分区
- 文件容器

以下是一些用 dm-crypt 加密辅助（secondary）文件系统（即，非根文件系统）的例子。

总览

加密辅助文件系统不会对操作系统、程序文件进行加密，因此通常只用于单独保护敏感数据。对于USB驱动器等外部介质，进行加密可确保在不同计算机上使用时，数据安全不受威胁。此外，也可按照数据访问人员将数据分类，分别使用不同的密钥加密。

dm-crypt 是块级别加密层，其将加密整个块设备（例如，分区和回环设备）。

若要加密单个文件，需要使用文件系统层级的加密。例如使用eCryptfs 或 EncFS。有关保护隐私数据的更多信息，参见静态数据加密。

分区

本例介绍的是对 `/home` 分区的加密，但相关方法也可以应用到其它包含用户数据的非根分区。

提示：可以为每个用户的/home目录单独分配一个分区，也可以所有用户的/home目录共享一个分区。

下文假定用于/home目录的分区已经创建。若该分区曾用于其它用途，因而包含了数据，可能需要使用dm-crypt专用方案安全擦除分区。注意，只擦除一个分区，而不擦除整个磁盘，可能仍会泄露一些数据。

建立 LUKS 头：

```
# cryptsetup options luksFormat device
```

*device*应对应已创建的分区。*options*的取值参见Dm-crypt/设备加密#LUKS_模式的加密选项。

使用设备映射器对设备进行解密，以对加密的分区进行操作：

```
# cryptsetup open device name
```

分区解密之后，会被映射成块设备 /dev/mapper/*name*。之后，建立文件系统：

```
# mkfs.fstype /dev/mapper/name
```

若所有用户共享/home目录，则可将创建好的文件系统挂载到/home；若该文件系统是某个用户专用的，则挂载到/home/*username*。参见 #手动挂载和卸载。

提示：卸载并再挂载一次文件系统，可确定映射有无问题。

手动挂载和卸载

挂载分区：

```
# cryptsetup open device name
# mount /dev/mapper/name /mnt/home
```

卸载：

```
# umount /mnt/home
# cryptsetup close name
```

提示：GVFS 也可以挂载加密的分区。若使用支持 gvfs 的文件管理器（例如，Thunar）挂载加密分区，则会弹出密码输入框。对于其他桌面环境，可使用zulucryptAUR进行图形化挂载。

自动解锁并挂载

有三种不同方法来自动化解锁分区并挂载文件系统。

启动时解锁

配置 /etc/crypttab 文件，systemd会在系统启动过程中自动解析该文件，并以此解密相应设备。若要挂载所有用户共用的home分区，或要自动挂载其它加密的块设备，推荐使用本方法。

可参考Dm-crypt/系统配置#crypttab。此外，Dm-crypt/系统配置#在启动时挂载中提供了示例配置。

用户登录时解锁

- 使用pam_exec：若需要为每个用户挂载单独的加密home分区，推荐使用该方式。
- 使用pam_mount。

文件容器

Cryptsetup 对设备进行操作。因此，若要使用文件容器，需要配置回环设备。好在，在后台，cryptsetup可自动管理回环设备(请见 Gentoo:Custom_Initramfs#Encrypted_keyfile)，无需手动调用 `losetup`。

首先，使用恰当的随机数生成器用 dd 创建一个加密容器：

```
$ dd if=/dev/urandom of=bigsecret.img bs=100M count=1 iflag=fullblock
```

这会创建大小为 100 MiB 的 `bigsecret.img` 文件。

注意，不要遗漏`iflag=fullblock`选项。否则，可能出现“部分读”问题。详见Dd#部分读取：复制的数据量小于要求的大小。

要避免之后调整容器大小的麻烦，在创建文件容器时，确保其大小大于要加密的文件的总大小、文件系统内部数据/元数据开销和LUKS头的总和。若使用LUKS，则仅其元数据就可能需要多达16MiB的空间。创建一个小于 LUKS头（16MiB）大小的文件容器会在打开设备时报错`Requested offset is beyond real size of device bigsecret.img`。

之后的操作和#分区中的相同，但要将 `device` 替换为 `bigsecret.img`。

cryptsetup 会自动寻找可用的回环设备设备并将文件附加到设备上。在卸载之后，文件容器应该相应地关闭，cryptsetup之后会分离使用的回环设备。

提示：使用dm-crypt加密容器非常灵活。可查看Tomb的功能和文档详细了解。Tomb提供了一系列dm-crypt脚本，以便快速灵活地处理加解密操作。

使用 losetup 手动挂载和卸载

首先，查找未使用的回环设备：

```
# losetup --find
```

然后，将文件容器附加到回环设备，例如，要附加到`loop0`：

```
# losetup /dev/loop0 bigsecret.img
```

注意：如果出现`/dev/loop0: No such file or directory`错误，需要先以 root 用户执行命令`modprobe loop`以加载内核模块。较新的内核中（内核版本3.2及更高），回环设备是按需创建的。可通过以 root 用户执行 `losetup -f` 命令请求创建新的回环设备。

之后，按照常规方式进行cryptsetup设置即可：

```
# cryptsetup open /dev/loop0 secret
# mount -t ext4 /dev/mapper/secret /mnt/secret
```

卸载容器时，需要以相反的顺序执行对应的操作：

```
 # umount /mnt/secret
 # cryptsetup close secret
```

若要分离使用的回环设备:

```
 # losetup --detach /dev/loop0
```

2.3 Fscrypt

难度 进阶 | **篇幅** 12.0 KB | **所属** 静态数据加密

本节将讲到

- 替代方案
- 准备工作
- 加密目录
- 锁定/解锁目录
- 加密主目录

fscrypt 是一个用于管理 ext4, F2FS, UBIFS, CephFS 和 Lustre) 文件系统加密的用户空间工具.

内核中上述文件系统的 加密特性有时也被叫做“Fscrypt”。

在此加密特性中，不同目录可以用不同的密钥加密。在被加密的目录中，所有文件的内容、文件名和符号链接，以及子目录都将被加密。而元数据，如时间戳、文件大小，文件属性，则不会被加密。

由于本文提供的内容仅适用于fscrypt（以及可选的配套的pam_fscrypt），多数内容不适用于其他同样用于配置加密的用户空间工具，例如 systemd-homed.

替代方案

dm-crypt（LUKS）支持用一个密码加密整个分区。对于磁盘加密，这通常是更好的选择，因为它能加密分区上所有的内容，包括元数据。FSClect最适合加密特定目录，也能使不同的加密目录能够分别解锁，如按用户加密的主目录。

eCryptfs 使用Fscrypt不支持的文件系统堆叠加密，这使得其内存的利用率极高。它还使用更先进的加密算法，且无需root权限即可设置，无需提权。但eCryptfs已不再活跃，其主要用户（Ubuntu 用户和 Chrome OS用户）已迁移到其他解决方案。

参阅静态数据加密,以了解有关其他方案的更多信息,以及相关功能。

注意：

- fscrypt和dm-crypt可以结合使用,且每个加密层都分别具有不同用途。例如,文件系统本身可以通过dm-crypt使用一种安全性较差的方案进行加密,例如将TPM绑定到“安全启动”或或者一个所有用户都知道的密码,而fscrypt也可以将每个用户的主目录以该用户设置的密码加密。
- e2fsprogs包的e4crypt可以作为fscrypt的替代品。但不推荐使用，因为e4crypt缺少很多基本功能，而且已经不再活跃。
- f2fs-tools包的f2fstools只是e4crypt的复制品，同样不建议使用

准备工作

内核

所有官方支持的内核均支持ext4， F2FS， UBIFS和CephFS的加密。

对于用户自定义的内核,请确保`CONFIG_FS_ENCRYPTION=y`已被设置。

文件系统

对于ext4，目标文件系统必须具备`encrypt` 启用了功能标志。要启用它,请运行:

```
# tune2fs -O encrypt /dev/device
```

提示：创建新文件系统时,可以使用`mkfs.ext4 -O encrypt`以立即启用`encrypt`。

F2FS

对于F2FS, 使用`mkfs.f2fs -O encrypt`以创建带加密标志的文件系统，或使用`fsck.f2fs -O encrypt`为现有的文件系统添加。

用户空间工具

安装fscrypt包。然后运行:

```
# fscrypt setup
```

这会在创建文件`/etc/fscrypt.conf` 和目录`/.fscrypt`。

如果目标文件系统不是根文件系统，运行:

然

```
# fscrypt setup --all-users mountpoint
```

其中 `*mountpoint*` 是该文件系统的挂载点, 如 `/home`.

`--all-users` 选项允许用户选择要加密的目录，也可以在创建配置后更改，更多信息请参见 fscrypt documentation。

这将会创建`*mountpoint*/.fscrypt`以存放配置和保护器。

警告：

- 永远不要删除`.fscrypt`目录; 否则所有被加密的文件都将丢失.

- 在backup, 备份,修复和恢复时一定要先尝试一下。

自动解锁目录

要(可选)在登录时自动解锁使用登录密码的保护目录，并保持其保护目录与登录密码的更改同步，请调整系统 PAM 配置以启用pam_fscrypt。

在`/etc/pam.d/system-login`的auth 部分附加以下行:

```
/etc/pam.d/system-login
auth       optional   pam_fscrypt.so
```

然后在`session include system-auth` 的*session* 部分**之前**插入:

```
/etc/pam.d/system-login
session    [success=1 default=ignore]  pam_succeed_if.so  service = systemd-user quiet
session    optional                    pam_fscrypt.so
```

注意：第一行取自https://github.com/google/fscrypt/issues/95, 是针对 systemd --user 会话的绕过方法，因为它未能正确关闭PAM会话，否则会在登出时阻止锁定目录。

最后，在`/etc/pam.d/passwd`后插入:

```
/etc/pam.d/passwd
password    optional    pam_fscrypt.so
```

加密目录

要加密一个空目录，运行:

```
$ fscrypt encrypt dir
```

按照提示创建或选择“保护器”。保护器是关于保护目录密钥的秘密信息。保护器的类型包括:

- “custom_passphrase”，即用户自定义密码。
- “pam_passphrase”，即用户登录密码。使用此类保护器的目录，当用户登录时（如果启用了该保护器），pam_fscrypt就会自动解锁它们。考虑到Arch Linux使用yescrypt哈希算法， 使用此类保护器前应先查阅上游安全建议。

在这两种情况下，密码都支持在设置加密后更改，或修改保护器。对于“pam_passphrase”，fscrypt 会为其自动生成第二个保护器，即恢复密码。

E

custom_passphrase的示例:

```
$ fscrypt encrypt private/
Should we create a new protector? [y/N] y
Your data can be protected with one of the following sources:
1 - Your login passphrase (pam_passphrase)
2 - A custom passphrase (custom_passphrase)
3 - A raw 256-bit key (raw_key)
Enter the source number for the new protector [2 - custom_passphrase]: 2
Enter a name for the new protector: Super Secret
Enter custom passphrase for protector "Super Secret":
Confirm passphrase:
```

```
"private/" is now encrypted, unlocked, and ready for use.
```

pam_passphrase的示例:

```
$ fscrypt encrypt private/
Should we create a new protector? [y/N] y
Your data can be protected with one of the following sources:
1 - Your login passphrase (pam_passphrase)
2 - A custom passphrase (custom_passphrase)
3 - A raw 256-bit key (raw_key)
Enter the source number for the new protector [2 - custom_passphrase]: 1
Enter login passphrase for testuser:
"private/" is now encrypted, unlocked, and ready for use.
```

注意：注释加密只能在空目录上启用。要加密以有目录，可以使用下面的方法:

```
$ mkdir new_dir
$ fscrypt encrypt new_dir
$ cp -a -T old_dir new_dir
$ find old_dir -type f -print0 | xargs -0 shred -n1 --remove=unlink
$ rm -rf old_dir
```

要注意是移动文件（命令mv），而不是复制，因为未加密的文件很容易恢复。在某些情况下，尤其是在使用SSD时，这些文件在被销毁和删除后，有可能从磁盘中恢复。所以最好从一开始就加密数据。

锁定/解锁目录

要解锁加密的目录，运行:

```
$ fscrypt unlock dir
```

FSCcrypt会提示输入密码短语。

要锁定加密目录，运行:

```
$ fscrypt lock dir
```

加密主目录

警告：

- 登录密码保护器的安全性取决于`/etc/shadow`中密码的哈希算法。使用该保护器之前，请参见#Encrypt a directory。
- 如果主目录被加密了，那么在主目录解锁之前，可能无法连接该用户的SSH.

要加密主目录，首先要完成 准备工作，同时应启用 *pam_fscrypt*。

然后创建一个加密目录做为新的主目录:

```
# mkdir /home/newhome
# chown user:user /home/newhome
# fscrypt encrypt /home/newhome --user=user
```

S

使用\--user选项以使用登录密码保护主目录。

将原主目录下的所有内容复制到加密的主目录:

```
# cp -a -T /home/user /home/newhome
```

提示：如果没有足够的磁盘空间复制内容到加密目录，可以考虑使用`mv -T /home/*user* /home/newhome`代替。然而该命令并不安全，因此强烈建议在执行之前备份。

如果使用了cp命令，那么切换主目录之前应当先检查新目录在登录时有没有自动解锁，最简单的方法就是重启并重新登录，然后运行:

```
$ fscrypt status /home/newhome
"/home/newhome" is encrypted with fscrypt.

Policy:   d80f252996aae181204403043b0ada25
Options:  padding:32 contents:AES_256_XTS filenames:AES_256_CTS policy_version:2
Unlocked: Yes

Protected with 1 protector:
PROTECTOR         LINKED  DESCRIPTION
5952c84ebaf0f98d  No      login protector for testuser
```

若输出`Unlocked: No`，可能是PAM配置错误，也可能是选错了保护器的类型。

如果没有问题，就可以替换主目录:

```
# mv /home/user /home/oldhome
# mv /home/newhome /home/user
# reboot
```

如果之后的使用正常，可以删除原来的主目录:

```
# find /home/oldhome -type f -print0 | xargs -0 shred -n1 --remove=unlink
# rm -rf /home/oldhome
```

提示：运行shred是可选的, 但十分推荐，原因详见 安全地擦除磁盘#shred。

Lxc内的加密

Lxc内部支持使用Fscrypt，或者更一般地在mount_namespaces(7) 中使用Fscrypt，该功能在v0.2.8.以添加

容器停止时设定目录

容器内的systemd/用户单元可以在容器停止时锁定加密目录:

```
~/.config/systemd/user/lock-directory.service
[Unit]
Description=lock encrypted directory on shutdown
DefaultDependencies=no
Before=shutdown.target

[Service]
Type=oneshot
ExecStart=/bin/sh -c "/usr/bin/fscrypt lock /home/facade/target"
TimeoutStartSec=0

[Install]
WantedBy=shutdown.target
```

故障排除

要解决一般问题，参见 https://github.com/google/fscrypt/blob/master/README.md#troubleshooting以及在Github提交issues.

ext4: 从加密目录移动文件

可以将加密文件从它已解锁的父目录中移动出去，但这种方法支持。不佳当相应的保护器锁定时可能会引发许多后续问题，当保护器被移除时，甚至可能丢失数据。参见 https://github.com/google/fscrypt/issues/393。

若要永久解密文件，可以将其从已解锁的加密目录复制到普通的未加密目录中。

参见

- Documentation for the fscrypt tool
- Documentation for Linux native file encryption
- The design document for the fscrypt tool

2.4 Dm-crypt/准备磁盘

难度 进阶 | **篇幅** 12.5 KB | **所属** 静态数据加密

本节将讲到

- 安全擦除驱动器数据
- dm-crypt 专用方案
- 分区

在加密一个驱动器前，建议先用随机的数据将设备内容覆写来安全地擦除磁盘内容。要想避免加密攻击或不必要的文件恢复，写入的数据最好与之后 dm-crypt 写入的数据难以区分。关于更深入的讨论，请见静态数据加密#准备磁盘。

安全擦除驱动器数据

在决定使用何种方式安全擦除驱动器数据时，记住：如果驱动器要持续地作为加密设备使用，这只需要执行一次。

警告：

- 在开始前对必要数据进行恰当的备份！！
- 如果您要使用 TRIM，切勿用随机数据覆写固态硬盘。第一次执行 TRIM 后，未使用的块将被标记为空，并最终被擦除，使已执行的操作失效。

注意：在清除大量数据时，这个过程会耗费数小时至数天

提示：

- 继续操作前，请检查您的先进格式化设备是否采用最佳逻辑扇区大小。出于加密性能的考虑，cryptsetup 强烈推荐将磁盘的逻辑扇区大小改为 4096 字节。
- 在数 TB 的设备上，填充加密驱动器的过程可能需要耗费一整天。为了不让机器在操作过程中无法使用，从已经安装在另一个驱动器上的系统中执行此操作，比在 Arch 安装用的 live 系统中执行此操作要好。

- 对于固态硬盘,要想尽量减少闪存的缓存，可以考虑按 固态硬盘记忆单元清理 中的 *优先* 节执行。

通用方案

关于如何擦除并准备好驱动器的详细指导，参考安全地擦除磁盘#闪存。

dm-crypt 专用方案

下面两个方案专门用于 dm-crypt ，之所以提到它们是因为它们很快而且可以在分区设置后执行。

cryptsetup 常见疑问(item 2.19 "*How can I wipe a device with crypto-grade randomness?*")提到了一个非常简单的操作可以使用一个已存在的 dm-crypt 卷，像一个简单的伪随机数生成器生成随机数据来擦除一个底层块设备上所有可访问的空闲空间。而且据说这可以避免使用方式的暴露。这是因为加密数据实际上与随机数据难以区分。

dm-crypt 擦除空设备或分区

首先，在待加密的单个分区（格式为 `sdXY`）或整个设备（格式为 `sdX`）上创建临时加密容器:

```
 # cryptsetup open --type plain --key-file /dev/urandom --sector-size 4096 /dev/block-device to_be_wiped
```

可以这样验证它是否存在:

```
# lsblk
NAME          MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINT
sda             8:0    0  1.8T  0 disk
└─to_be_wiped 252:0    0  1.8T  0 crypt
```

用0擦除容器。不需要使用 `if=/dev/urandom` 因为加密密钥已经将数据变成随机的了。

```
# dd if=/dev/zero of=/dev/mapper/to_be_wiped status=progress bs=1M
dd: writing to '/dev/mapper/to_be_wiped': No space left on device
```

提示：要想对操作进行检查,在创建擦除容器前将分区清零。在执行擦除命令后 `blockdev --getsize64 */dev/mapper/container*` 可以作为 root 获取准确的容器大小。在容器被关闭后 *od* 可以用于检查擦除操作是否覆盖了被清零的分区。举个例子,执行命令 `# od -j $((*containersize - blocksize*)) /dev/nvme0n1` 来从头至尾检查擦除是否执行。

最后，关闭临时的容器:

```
# cryptsetup close to_be_wiped
```

在加密整个系统时，下一步是#分区。如果只是加密一个分区，继续按照加密非 root 文件系统#分区执行。

dm-crypt 在安装后擦除可用空间

在安装前没时间执行擦除步骤的用户，即使在加密系统已启动，文件系统已挂载的情况下也可以做到类似的效果。但请注意相关文件系统是否有保留空间，例如，对于 root 用户，或即使是 root 用户执行的擦除操作也会被限制的其他磁盘配额机制：底层块设备的一些部分可能完全不会写入。

要想执行擦除操作，通过在加密容器写入一个文件来临时填充分区的剩余可用空间:

```
# dd if=/dev/zero of=/file/in/container status=progress
dd: writing to '/file/in/container': No space left on device
```

缓存同步到磁盘，然后删除文件来回收可用空间。

```
# sync
# rm /file/in/container
```

对每个块设备的分区和其中的文件系统都要重复一次以上操作。举个例子，设置 LUKS 上的 LVM 时，对每个逻辑卷都要执行这个操作。

dm-crypt 在安装后擦除可用空间(通过重新加密)

作为替代，对于想要在不重新安装的情况下完全擦除可用空间的用户，可以通过重新加密 LUKS 设备来达成。需要在每个 LUKS 设备上执行一次。但是请注意这个过程可能很慢(在桌面 HDD 上大约 50MB/s)。

擦除 LUKS header

使用 LUKS 的分区由两部分组成：标头和加密数据。标头包含密钥和加密参数，如果没有这些密钥和参数，几乎不可能恢复数据。如果创建新分区或停用硬盘，只需删除该部分而无需擦除整个设备。注意事项参见本小节末。

警告：操作是不可逆的，请小心执行。

要想擦除所有密钥，使用下列命令:

```
# cryptsetup erase device
```

通过以下命令确保没有激活的密钥槽:

```
# cryptsetup luksDump device
```

此外，通过在密钥被擦除后调用 arch:wipefs 可以移除 LUKS header 本身以避免 `cryptsetup` 之后检测到它:

```
# wipefs -a device
```

加密数据保留在原位。截至2020年，没有已知的实用方法解密这些数据，但是在未来可能会有所不同。由用户来衡量安全，隐私与正确擦除整个设备的时间。

在某些存储介质（尤其是闪存介质）上，可能无法可靠地覆盖数据。带有密钥的 LUKS 标头可能会保存在操作系统无法访问的位置（如驱动器缓存）。如果担心出现

这种情况，则必须使用ATA 安全擦除。该操作会擦除设备上的所有区块，包括软件看不到的区块。详情请参见 cryptsetup FAQ 5.19。

分区

注意：如果您要使用OPAL 硬件加密，且之前未为硬盘设置 OPAL 管理密码，则需要使用 cryptsetup-erase(8) 删除硬盘上的所有数据，才能创建新的 OPAL 管理密码。请按照arch:Self-encrypting drives#Secure disk erasure中的说明，在分区前执行擦除操作。

只有在加密整个系统时，才需用到本节。在安全地覆写驱动器后，必须慎重选择一个恰当的分区方案：要考虑到 dm-crypt 的要求，以及各种抉择对管理目标系统的影响。

还要考虑另一个重要因素：如何处理交换分区和系统挂起？参见 Dm-crypt/交换分区加密。

启动分区与 EFI 系统分区

在几乎)所有情况下，必须单独留一个盛装内核与 initramfs 的未加密分区，以供引导加载程序访问（启动过程各部分的详情，参见Arch_的启动流程） 。

鉴于 mkinitcpio 默认把内核与 initramfs 置于 `/boot/` 下，因而通常把这个未加密的分区挂载在 `/boot` 下。对于 `/boot` 分区的要求与限制的解释，见于分区##/boot。

在 UEFI 系统上，可以把 EFI 系统分区挂载在 /boot 下。鉴于按照定义，EFI 系统分区必须是主分区表中的未加密物理分区，这样做可以避免创建额外的未加密分区。EFI 系统分区特有的要求与限制，见于 EFI 系统分区一文。将 EFI 系统分区挂载在 `/boot` 下时，其同时也受 boot 分区的要求制约。

如果您仍然担忧留一个未加密分区的安全问题，参见 Dm-crypt/特殊应用#保护未加密的boot分区.

物理分区

在最简单的情况下，加密层可以直接基于物理分区;对于如何创建物理分区，请见分区。就像在未加密系统中，一个 root 分区就足够了，此外还需要一个上述的 `/boot` 分区。使用这个方法可以选择哪些分区加密，哪些分区不加密，无论涉及的磁盘数量多少，都可以正常工作。之后还可以添加或移除分区，但是调整分区的大小会被分区所基于的磁盘限制。最后请注意每个加密分区的开启都需要单独的密钥，即使这可以通过使用 `crypttab` 文件在启动时自动化，请见 Dm-crypt/系统配置#crypttab。

堆叠块设备

如果需要更高的灵活性，dm-crypt 可以和其他例如 LVM 和 RAID的堆叠块设备共存。加密容器可以位于其他堆叠块设备的之下或基于其他堆叠块设备上:

- 如果 LVM/RAID 设备创建在加密层上，那么在同一加密分区中可以自由地添加，移除文件系统和调整文件系统大小，所有的存储设备都只需要一个密钥或密码。因为加密层基于物理分区，但是这样就不能使用 LVM 和 RAID 跨越多个磁盘的能力。
- 如果加密层创建在 LVM/RAID 设备上，它之后依然有可能重组文件系统，但是复杂性会提高，因为加密层需要相应地调整。而且，每个加密设备的打开都需要一个密钥或密码。不过这是在跨越多个磁盘上创建加密文件系统的唯一选择。

Btrfs 子卷

Btrfs 的内建子卷功能可以和 dm-crypt 一起使用，如果不需要其他文件系统，则完全取代对 LVM 的需求。

加密的启动分区 (GRUB)

无需未加密分区的例外情况：在 BIOS/MBR 上使用 GRUB 时。此时 GRUB （未加密地）储存在 MBR 引导代码区域，而 MBR 之后的间隔，以及其他分区均可加密。

参见 dm-crypt/加密整个系统#加密的boot分区(GRUB))。

2.5 Systemd-cryptenroll

难度 进阶 | **篇幅** 13.8 KB | **所属** 静态数据加密

本节将讲到

- 安装
- 列出密钥槽
- 擦除密钥槽
- 注册密码
- 注册硬件设备

systemd-cryptenroll(1)：

: systemd-cryptenroll 是一个用于将硬件安全令牌和设备注册到 LUKS2 加密卷中的工具，之后便可在启动过程中使用它们解锁该卷。

systemd-cryptenroll 允许将smartcards、通用第二因素 令牌、可信平台模块安全芯片以及普通密码注册到 LUKS 设备中。之后，systemd-cryptsetup@.service(8) 可以使用已注册的令牌解锁这些设备。

安装

systemd-cryptenroll 是 systemd包 的组成部分，并随该软件包提供。不过，要将硬件设备用作密钥，还需要安装额外的软件包：

- 要使用 PKCS#11 令牌，请安装 libp11-kit包，可能还需要 opensc包。
- 要使用 FIDO2 令牌，请安装 libfido2包。
- 要使用 TPM2 设备，请安装 tpm2-tss包。

列出密钥槽

systemd-cryptenroll 可以列出 LUKS 设备中的密钥槽，功能类似于 `cryptsetup` 的 luksDump，但输出格式更加易于阅读。

```
# systemd-cryptenroll /dev/disk
SLOT TYPE
   0 password
   1 recovery
   2 tpm2
```

擦除密钥槽

```
# systemd-cryptenroll /dev/disk --wipe-slot=SLOT
```

其中，*SLOT* 可以是：

- 单个密钥槽的索引，如#列出密钥槽所示。
- 密钥槽类型，这将擦除该类型的所有密钥槽。有效类型包括 `empty`、`password`、`recovery`、`pkcs11`、`fido2` 和 `tpm2`。
- 以上各项的组合，以逗号分隔。
- 字符串 `all`，用于擦除设备上的所有密钥槽。此选项只能在同时注册另一个设备或密码时使用。

`--wipe-slot` 操作可以与所有注册选项组合使用，便于更新已有的设备注册信息：

```
# systemd-cryptenroll /dev/disk --wipe-slot=fido2 --fido2-device=auto
```

注册密码

普通密码

此操作等同于 `cryptsetup luksAddKey`。

```
# systemd-cryptenroll /dev/disk --password
```

恢复密钥

systemd-cryptenroll(1)：

: 恢复密钥与密码基本相同，但它由计算机生成，而不是由用户自行选择，因此能够保证较高的熵。恢复密钥使用易于输入的字符集，也可以通过扫描屏幕上的二维码获取。

恢复密钥用于在硬件令牌不可用时作为备用解锁方式，并且可以在任何需要普通密码的场合代替普通密码使用。

```
# systemd-cryptenroll /dev/disk --recovery-key
```

注册硬件设备

`--*type*-device` 选项必须指向对应设备类型的有效设备路径。可以向该选项传入 `list` 参数，获取可用设备列表。或者，如果只连接了一台所需类型的设备，可以使用 `auto` 选项自动选择。

注意：将硬件令牌注册到 LUKS2 卷后，还必须配置系统，以便在适当的时候使用它们。对于根文件系统等需要在早期用户空间解锁的卷，请参阅 Dm-crypt/系统配置#可信平台模块与FIDO2密钥；对于其他分区，请参阅 Dm-crypt/系统配置#在后期用户空间中解密。

PKCS#11 令牌或智能卡

令牌或智能卡必须包含一对 RSA 密钥，用于加密所生成的卷解锁密钥。

```
# systemd-cryptenroll /dev/disk --pkcs11-token-uri=device
```

FIDO2 令牌

任何支持“hmac-secret”扩展的 FIDO2 令牌都可以与 *systemd-cryptenroll* 配合使用。以下示例将一个 FIDO2 令牌注册到加密的 LUKS2 块设备中，仅要求通过用户在场验证进行身份认证：

```
# systemd-cryptenroll /dev/disk --fido2-device=device --fido2-with-client-pin=no
```

此外，*systemd-cryptenroll* 还支持使用令牌内置的用户验证方式：

- `--fido2-with-user-presence` 指定解锁前是否验证用户在场（例如触摸令牌），默认为 yes。
- `--fido2-with-user-verification` 指定解锁前是否要求用户验证，默认为 no。

注意：

- 如果令牌不支持这些功能，上述选项将不起作用。
- 有关两者区别的更多信息，请参阅 用户在场与用户验证。

默认情况下，生成 FIDO2 凭据时使用的加密算法为 *es256*，即采用 NIST P-256 曲线和 SHA-256 的椭圆曲线数字签名算法（ECDSA）。如果需要，并且 FIDO2 令牌支持，也可以在注册时指定其他加密算法。

注意：对于担忧 ECDSA 的用户，这样做也可能更合适。详情参阅 SSH 密钥#ECDSA。

假设此前已经注册过一个 FIDO2 令牌，而用户希望再注册一个令牌，则可以使用以下命令生成采用 Curve25519 和 SHA-512 的 EdDSA 凭据（*eddsa*），并使用此前注册的令牌而非密码来认证设备：

```
# systemd-cryptenroll /dev/disk --fido2-device=device --fido2-credential-algorithm=eddsa --unlock-fido2-device=auto
```

注意：必须将两个令牌都连接到系统，才能成功完成注册。

可信平台模块

systemd-cryptenroll 原生支持将 LUKS 密钥注册到 TPM。使用此功能需要满足以下条件：

- 必须安装 tpm2-tss包。
- 使用 LUKS2 设备（目前也是 cryptsetup 默认使用的类型）。
- 如果打算在根分区上使用此方法，需要对 initramfs 进行一些调整（高级配置参阅 Dm-crypt/系统配置#使用systemd-cryptsetup-generator）：
- mkinitcpio 用户：启用 `systemd` 和 `sd-encrypt` Mkinitcpio#钩子（HOOKS）。

> **注意**：钩子的排列顺序非常重要。非标准顺序可能导致系统无法启动（此时需要在 arch-chroot 环境中重新生成 initramfs 才能恢复）。正确顺序的示例参阅 Dm-crypt/系统配置#示例。

- dracut 用户：启用 `tpm2-tss` Dracut#dracut 模块。

首先，运行以下命令列出已安装的 TPM 及其使用的驱动程序：

```
$ systemd-cryptenroll --tpm2-device=list
```

提示：

- 如果计算机安装了多个 TPM，请在后续步骤中使用 `--tpm2-device=*/path/to/tpm2_device*` 指定要使用的设备。
- 考虑使用 Trusted Platform Module#PCR policies，而不是直接将机密绑定到原始 PCR 值。

只需一条命令，就可以同时将密钥注册到 TPM 和 LUKS 卷。以下示例会生成一个新的随机密钥，将其添加到卷中，使其能够与现有密钥一样用于解锁，并将这个新密钥绑定到 PCR 7（安全启动状态）：

```
# systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=7 /dev/sdX
```

其中，`/dev/*sdX*` 是加密 LUKS 卷的完整路径。如果 LUKS 卷使用密钥文件而非密码解锁，请使用 `--unlock-key-file=*/path/to/keyfile*`。

有关 Linux 中常见的 PCR 测量内容，请参阅 systemd-cryptenroll(1) 和 可信平台模块#访问_PCR_寄存器。根据需要调整 `--tpm2-pcrs=7`（多个参数之间使用 `+` 符号分隔）。

警告：

- 绑定到 PCR 7 时，请确保 安全启动已启用且处于用户模式，否则未经授权的启动设备可能解锁加密卷。
- 如果固件证书发生变化，PCR 7 的状态也可能改变，从而导致用户无法解锁设备。这种变化可能由 fwupd 间接引起，也可能由轮换安全启动密钥直接引起。
- 仅绑定到启动前测量的 PCR（PCR 0–7）会带来恶意操作系统攻击的风险。攻击者可以创建一个恶意分区，并复制真实根文件系统的元数据（例如分区 UUID），使其伪装成原始分区。随后，initramfs 会尝试将恶意分区挂载为根文件系统（解密失败时将回退到密码输入），而启动前 PCR 的值保持不变。由攻击者控制文件内容的恶意根文件系统仍然可能获取真实根分区的解密密钥。详情参阅 Brave New Trusted Boot World 和 BitLocker 文档。
- 对于根卷，一种解决方案是使用 `--tpm2-pcrs=*other_pcrs*+15:sha256=0000000000000000000000000000000000000000000000000000000000000000` 绑定到初始为空的 PCR 15。如果设置了任何 `rd.luks` 内核参数，或者在 `/etc/crypttab` 中使用了 `x-initrd.attach` 选项，还需要在 `rd.luks.options=` 或 `/etc/crypttab` 的第四个字段中添加 `tpm2-measure-pcr=yes` 选项；如果依赖 GPT 分区自动挂载，则无需这样做。在早期用户空间解锁根卷后，PCR 15 将发生变化，已注册的密钥也将无法再被取回。
- 另一种更简洁的解决方案参阅 Dm-crypt/系统配置#固定 LUKS 卷。

绑定哪些 PCR，需要根据具体情况在易用性和安全限制之间取得平衡。例如，你可能希望更新 UEFI 固件时无需手动干预 安全启动状态，或者希望使用不同的启动设备。另一个例子是，微软的 BitLocker 倾向于使用 PCR `7+11`，但也可能采用其他 PCR 组合。

注意：

- 除了要求 TPM 状态正确外，还可以要求输入 PIN。只需在上述命令中添加 `--tpm2-with-pin=yes` 选项，并在提示时输入 PIN。
- *systemd-cryptenroll* 不会在请求输入 PIN 之前检查 TPM 测量值，因此应考虑使用专用 PIN，因为当前环境可能并不可信。

要检查新密钥是否已成功注册，可以转储 LUKS 配置，并查找 `systemd-tpm2` 令牌条目，以及 *Keyslots* 部分中新增的密钥槽条目：

```
# cryptsetup luksDump /dev/sdX
```

要测试密钥是否有效，请在 LUKS 卷处于关闭状态时运行以下命令：

```
# systemd-cryptsetup attach mapping_name /dev/sdX none tpm2-device=auto
```

其中，`*mapping_name*` 是为卷解锁后所创建的映射指定的名称。

有关如何在启动时解锁卷，请参阅 Dm-crypt/系统配置#crypttab 和 Dm-crypt/系统配置#可信平台模块与FIDO2密钥。

注意：虽然可以在 `/etc/crypttab` 中使用 LUKS 卷的 UUID 代替设备路径，但 *systemd-cryptenroll* 命令本身目前只支持路径名。

更多信息和示例请参阅 systemd-cryptenroll(1) 和 crypttab(5)。

参见

- Lennart 的博客：在 systemd 248 中使用 TPM2、FIDO2 和 PKCS#11 安全硬件解锁 LUKS2 卷

2.6 Dm-crypt/交换分区加密

难度 进阶 | **篇幅** 15.2 KB | **所属** 静态数据加密

本节将讲到

- 无需休眠支持
- 需要休眠支持
- 已知问题

本文介绍了多种加密交换分区的方法，可满足不同的需求。若在每次重启时都重新对交换分区进行加密（使用不同密钥）的方式，可避免换出到交换分区上的敏感文件片段因未被覆盖而长期存在，从而更好地保护数据。然而，若使用重新加密，则无法使用休眠功能。

无需休眠支持

若无需使用休眠功能，则可以通过配置`/etc/crypttab`，在每次启动时以随机密码，使用dm-crypt plain模式进行解密。在关机时，随机密码将被丢弃，设备上只含有加密的、无法访问的交换数据。

要使用该配置，只需要取消注释`/etc/crypttab`中以`swap`开头的一列，并修改“设备”（device）参数指向交换分区，例如：

```
/etc/crypttab
# <name>   <device>            <password>          <options>
swap       /dev/sdX#    /dev/urandom    swap,cipher=aes-xts-plain64,size=512,sector-size=4096
```

上述配置将解密`/dev/sd*X#*`，并映射到`/dev/mapper/swap`。之后，像普通的swap分区一样，将`/dev/mapper/swap`添加到`/etc/fstab`中，即可完成交换分区配置。若之前已经配置了非加密的交换分区，需要将其禁用（或直接将其在fstab中的条目指向`/dev/mapper/swap`）。上面给出的选项应当足以应对大多数场景。若需要了解每个选项的详细解释，可参见crypttab(5)、point cryptsetup FAQ 2.3。

警告：上述配置中“设备”参数对应的设备的数据将被**永久清除**。不要使用内核的简单命名（例如，`/dev/sda`, `/dev/sdb`）指定设备，简单命名的顺序可能在每次启动时都不同，导致设备数据被意外清除。建议使用如下方式指定：

- 使用`by-id`、`by-path`。然而，当出现硬件变化时，两者仍可能会改变，详见块设备持久化命名#通过_id_和_通过路径。
- 使用PARTLABEL、PARTUUID。
- 使用LVM逻辑卷名称。
- 使用#文件系统UUID和文件系统标签中介绍的方法。由于每次启动时，都将重新加密设备，并使用`mkswap`重新格式化swap分区， **不能**直接使用文件系统标签和文件系统UUID，参见cryptsetup FAQ。但可通过间接手段达成目的。

注意：有时，交换分区配置可能失败，参见systemd issue 10179。

若要使用`by-id`持久化命名，首先，找到swap分区对应的设备：

```
# find -L /dev/disk -samefile /dev/sdaX
/dev/disk/by-id/ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-partX
/dev/disk/by-id/wwn-0x60015ee0000b237f-partX
```

之后，修改配置文件，使用持久化引用（若上述命令返回多个结果，任选一个即可）：

```
/etc/crypttab
# <name>  <device>                                                  <password>
<options>
swap      /dev/disk/by-id/ata-WDC_WD2500BEVT-22ZCT0_WD-WXE908VF0470-partX  /dev/urandom
swap,cipher=aes-xts-plain64,size=512,sector-size=4096
```

重启并激活加密交换后，使用`swapon -s`命令可以看到交换分区对应映射后的设备（例如，`/dev/dm-1`）；使用`lsblk`命令，可看到交换分区的`FSTYPE`为**crypt**。由于每次启动时都重新加密并格式化交换分区，`/dev/mapper/swap`的文件系统UUID会在每次启动时变化。

注意：若swap分区之前曾用作LUKS加密卷，则crypttab不会覆盖该LUKS加密卷以创建加密的swap分区。这是一项安全措施，用于避免在crypttab中错误指定swap分区对应的设备时，造成数据丢失。若确定要使用之前的LUKS加密卷作为swap分区，必须先覆盖LUKS头。

文件系统UUID和文件系统标签

在crypttab中使用内核简单命名（例如，`/dev/sdX#`）、ID命名（例如，`/dev/disk/by-id/ata-SERIAL-partX`）指定加密交换分区的目标设备有较大风险，设备名称、分区结构的细微变化都可能导致指向错误的设备，从而造成数据丢失。使用PARTLABEL、PARTUUID（与分区，而不是文件系统、设备绑定）的风险相对较小，但若打算将目标设备用作其它用途，而忘记移除crypttab中的条目，数据仍可能被覆盖（文件系统改变，分区不改变）。

更可靠的方法是使用文件系统标签或文件系统UUID。但在默认情况下，每次重启时，dm-crypt和`mkswap`都将覆盖目标分区上的内容，导致文件系统标签和文件系统UUID

被移除。不过，可以指定swap分区的起始偏移量，以在设备开始处放置另一个空的、容量较小的文件系统。该文件系统仅用于提供持久性的文件系统UUID和文件系统标签。

使用指定文件系统标签创建文件系统：

```
# mkfs.ext2 -L cryptswap /dev/sdX# 1M
```

设备名后的`1M`参数限制文件系统的大小为1MiB，为之后加密的交换分区留出空间。

```
# blkid /dev/sdX#
/dev/sdX#: LABEL="cryptswap" UUID="b72c384e-bd3c-49aa-b7a7-a28ea81a2605" TYPE="ext2"
```

这样，无论设备名称、分区数量如何变化，始终可以使用文件系统UUID或文件系统标签指向`/dev/sdX#`。

接下来，配置`/etc/crypttab`和`/etc/fstab`条目即可：

```
/etc/crypttab
# <name> <device>         <password>    <options>
swap     LABEL=cryptswap  /dev/urandom  swap,offset=2048,cipher=aes-xts-
plain64,size=512,sector-size=4096
```

注意偏移量的指定（`offset=2048`），偏移量的单位为512字节的扇区（不受dm-crypt扇区大小的影响），因此`offset=2048`相当于1MiB，跳过了提供UUID的文件系统的区域，因此不影响文件系统UUID和文件系统标签，并正确处理了数据对齐。

```
/etc/fstab
# <filesystem>    <dir>  <type>  <options>  <dump>  <pass>
/dev/mapper/swap  none   swap    defaults   0       0
```

在本配置中，只会尝试使用特定文件系统标签的设备作为加密交换分区，而不受设备名称影响。若打算将加密交换分区用作其它用途，当格式化时，文件系统标签被覆盖，因此`/etc/crypttab`不会在下次启动时意外覆盖新的文件系统及数据。

在桌面环境中关闭休眠

桌面环境可能无法自动检测到swap分区已被随机密码加密，不能用作休眠，仍错误显示休眠按钮。

若使用Xfce，可通过如下命令隐藏“休眠”、“混合睡眠”按钮：

```
$ xfconf-query -c xfce4-session -np /shutdown/ShowHibernate -t bool -s false
$ xfconf-query -c xfce4-session -np /shutdown/ShowHybridSleep -t bool -s false
```

需要休眠支持

下面介绍在需要进行休眠的情况下，设置加密swap的三种方法。若已经使用其中一种方法完成设置，需要注意系统换出的敏感数据可能在交换分区/交换文件中保留较长时间，直到被覆盖。若要降低相关风险，可以设置系统任务，在特定时机（例如，系统关机时）重新加密交换分区/重置交换文件。

使用交换文件

交换文件可放置在加密设备的文件系统中。依照Swap#交换文件中的方法创建交换文件，并完成休眠设置即可。

若采用加密根文件系统的配置，并使用基于systemd的initrd，则配置加密交换文件非常简单、灵活。在完成配置加密的根文件系统后，只需要按常规方式创建并激活交换文件，并将其添加到`/etc/fstab`中即可，无需额外配置。

注意：在设置`resume=`内核参数时，注意其必须指向解密后产生的映射设备，该映射设备包含存储有交换文件的文件系统。

使用交换分区

注意：若要重用当前正在被系统使用的交换分区，必须先以root身份执行`swapoff /dev/*device*`将其禁用，并在/etc/crypttab中移除相关条目。

使用cryptsetup-luksFormat(8)创建容纳交换分区的加密容器：

```
# cryptsetup luksFormat --label swap /dev/device
```

打开该容器，并映射到`/dev/mapper/swap`：

```
# cryptsetup open /dev/disk/by-label/swap swap
```

在映射设备上创建swap文件系统：

```
# mkswap /dev/mapper/swap
```

若不使用Systemd#GPT分区自动挂载，需要将映射设备加入`/etc/fstab`中：

```
/dev/mapper/swap none swap defaults 0 0
```

要启用休眠，添加`resume=/dev/mapper/swap`内核参数。详见电源管理/挂起与休眠#传递休眠位置到_initramfs。

使用TPM

通过在TPM中存储密钥，可自动完成交换分区的加密与解密。

可以使用systemd-cryptenroll将密钥注册到LUKS容器和TPM中。之后，清除之前创建的密码槽位：

```
# systemd-cryptenroll --tpm2-device auto /dev/device
# systemd-cryptenroll --wipe-slot password /dev/device
```

可通过如下命令验证结果：

```
# systemd-cryptenroll /dev/device
SLOT TYPE
   0 tpm2
```

在initramfs中解密分区

若要从加密的交换分区中唤醒，必须在initramfs中解密该分区。

mkinitcpio

基于systemd的initramfs

若使用基于systemd的inintramfs（使用sd-encrypt钩子），则可使用如下任意一个方案：

- 正确设置`rd.luks.uuid=`内核参数以解密交换分区；
- 编辑`crypttab.initramfs`，并重新生成 initramfs。

例如，若使用TPM加密交换分区：

```
/etc/crypttab.initramfs
swap UUID=56f8ee97-54b3-4a65-9282-688deb922527 none tpm2-device=auto
```

基于busybox的initramfs

若使用基于busybox的initramfs（使用encrypt钩子），需进行如下操作。

若交换分区所在设备与根文件系统的不同，则其不会被`encrypt`钩子打开。由于从休眠中唤醒的过程在读取`/etc/crypttab`前进行，不能在`/etc/crypttab`中进行配置以打开加密的交换分区，而需要修改`/etc/mkinitcpio.conf`并创建自定义钩子，以在唤醒开始之前打开加密的交换分区。

注意：本节只适用于`encrypt`钩子，其只能解密单个设备（archlinux/mkinitcpio/mkinitcpio#231）；若使用`sd-encrypt`钩子，则可以解密多个设备（包括加密的交换分区），详见Dm-crypt/系统配置#systemd-gpt-auto-generator。

mkinitcpio-openswapAUR中包含了所需的钩子及配置文件。

警告：在系统休眠且还未恢复时，在initramfs中挂载文件系统非常危险且可能导致数据丢失。若使用密钥文件解密，密钥文件不应保存到任何休眠时已挂载的文件系统上。

注意：为了降低文件系统损坏的可能性，上述包中的脚本使用了`-o ro,noload`挂载选项。这些选项适用于ext4文件系统，若使用其它文件系统，需修改`openswap.conf`，使用作用相似的选项。

将`openswap`钩子添加到`/etc/mkinitcpio.conf`的`HOOKS`数组中，注意应当添加到`filesystem`钩子之前，`encrypt`钩子之后。再在`openswap`钩子之后添加`resume`钩子。

```
HOOKS=(... encrypt openswap resume filesystems ...)
```

最后，重新生成 initramfs。

启动时，`openswap`钩子将打开加密的交换分区，以便内核从中唤醒。若使用自定义钩子进行唤醒操作，需确保在`HOOKS`数组中，相关钩子放置在`openswap`之后。注意，由于在initrd阶段便挂载了交换分区，`/etc/crypttab`中无需包含交换分区相关条目。

dracut

创建密钥文件：

```
# dd bs=512 count=4 if=/dev/random iflag=fullblock | install -m
0600 /dev/stdin /etc/cryptsetup-keys.d/swap.key
```

将密钥文件添加到LUKS卷：

```
# cryptsetup luksAddKey /dev/device /etc/cryptsetup-keys.d/swap.key
```

修改dracut配置，加入`resume`模块，并将`swap.key`文件加入到initramfs中（详见Dracut#休眠）：

```
/etc/dracut.conf.d/resume-from-hibernate.conf
add_dracutmodules+=" resume "
install_items+=" /etc/cryptsetup-keys.d/swap.key "
```

重新生成 initramfs。

在内核参数中添加与交换分区对应的`rd.luks.name`、`rd.luks.key`参数。

修改后的内核命令行应当与下例相似：

```
kernel /vmlinuz-linux cryptdevice=/dev/sda2:root root=/dev/mapper/root
resume=/dev/mapper/swap rd.luks.name=fd839505-3213-4603-9a70-c5a96a24768f=swap
rd.luks.key=/etc/cryptsetup-keys.d/swap.key ro
```

在LUKS上配置LVM

若交换分区位于LVM卷组内，且对应卷组在initramfs中被激活，只需要按照电源管理/挂起与休眠#休眠中的步骤进行配置即可。

已知问题

- 系统日志中出现形如`Stopped (with error) /dev/dm-1`的错误。详见systemd issue 1620。

2.7 Dm-crypt/系统配置

难度 高级 | **篇幅** 32.4 KB | **所属** 静态数据加密

本节将讲到

- 在早期用户空间解密
- 在后期用户空间中解密
- 疑难解答

在早期用户空间解密

若要从加密的根文件系统启动，initramfs需要包含能在早期用户空间（即initramfs阶段）解密卷的工具。一般使用内核参数指定要解密的卷。

下列小节介绍了如何配置mkinitcpio以生成所需的initramfs，以及所需的内核参数。

mkinitcpio

依据具体的配置，需要启用如下mkinitcpio钩子中的部分：

busybox	systemd	Use case
encrypt	sd-encrypt	若根分区是加密的，或者需要在根分区挂载**之前**挂载其它加密分区，则需要此钩子。在其它情况下，初始化脚本（例如，/etc/crypttab）将解锁非根分区，故无需此钩子。注意，此钩子必须放置在udev / systemd钩子**之后**。
–	sd-encrypt-opensc	扩展 sd-encrypt 钩子的功能，以支持使用 PKCS#11 令牌或智能卡 解锁 LUKS 卷。
keyboard	需要，用于使得在早期用户空间可以使用键盘。 > **提示**：对于外设连接经常改变（即，后续启动时连接的外设可能和生成initramfs镜像时不一致）的系统（例如，使用外置键盘的笔记本、headless系统），建议将本钩子放置在autodetect之前，以便始终包含所有键盘驱动。否则，若在生成镜像时未连接外置键盘，但在启动时连接，则外置键盘可能由于缺少驱动而无法正常工作。	

其它可能需要的钩子参考相应配置方案的指南。

> **注意**：在修改/etc/mkinitcpio.conf后，应当重新生成 initramfs。

示例

对于基于busybox的initramfs，使用encrypt钩子时，典型的配置如下：

```
/etc/mkinitcpio.conf
...
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt lvm2 filesystems fsck)
...
```

对于基于systemd的initramfs，使用sd-encrypt钩子时，典型的配置如下：

```
/etc/mkinitcpio.conf
...
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt lvm2 filesystems fsck)
...
```

内核参数

encrypt钩子和sd-encrypt钩子需要设定的内核参数不同。但root、resume参数的设定方式相同。

root

root=参数指定了解密后的、实际的根文件系统：

```
root=device
```

- 若直接在加密设备上创建根文件系统，则为/dev/mapper/*dmname*。
- 若在LVM上层使用加密的逻辑卷，则设置同上。
- 若在加密设备上使用LVM，则为root=/dev/<卷组名称>/<逻辑卷名称>。

提示：

- 若使用sd-encrypt钩子以及GPT分区自动挂载，则无需指定root=参数。详见在单一分区上配置LUKS。
- 若使用GRUB，并通过*grub-mkconfig*生成grub.cfg配置文件，则无需手动设置root=参数。*grub-mkconfig*将自动检测解密后的根文件系统的UUID，并将其添加到grub.cfg中。

resume

```
resume=device
```

- *device* 应当为用于休眠的设备文件或加密后的swap文件系统。对于单独的swap分区，该参数的值形如/dev/mapper/swap。详见Dm-crypt/交换分区加密。

encrypt钩子

注意：相比sd-encrypt钩子，encrypt不支持如下功能：

- 解密多个加密设备 (archlinux/mkinitcpio/mkinitcpio#231)。在initramfs中只能解密一个设备。
- 使用分离的LUKS头 (archlinux/mkinitcpio/mkinitcpio#234)。
- 设置某些crypttab支持的加密选项。

cryptdevice

指定在冷启动时包含加密的根分区的设备（加密容器）。encrypt钩子通过该参数判断哪个设备包含了加密的系统。

```
cryptdevice=device:dmname:options
```

- *device*：包含加密容器的设备。强烈建议使用块设备持久化命名指定。
- *dmname*：解密后产生的设备映射器名称（**d**evice-**m**apper name），设备解密后，可通过/dev/mapper/*dmname*访问解密后的数据。
- *options*：（可选），按逗号分隔的选项（例如，启用TRIM支持）。若无需额外指定选项，省略该参数（即，使用cryptdevice=*device*:*dmname*）。
- 若加密的根分区在LVM上，则应当首先激活LVM，并设置*device*为/dev/<卷组名称>/<逻辑卷名称>。

提示：对于固态硬盘，可能需要启用Discard/TRIM 支持。

cryptkey

提示：若使用密码解密设备，则无需指定`cryptkey=`参数。在启动时，将提示输入密码。

该参数指定了`encrypt`钩子用于解锁`*cryptdevice*`的密钥文件。若密钥放置在默认位置（详见下），无需手动指定此参数。

取决于密钥文件的存储位置，该参数有三种指定方式。

若密钥文件存储在某设备上的文件系统中，作为该文件系统中的一个文件：

```
cryptkey=device:fstype:path
```

- `*device*`：存储密钥文件的设备。强烈建议使用块设备持久化命名指定。
- `*fstype*`：存储密钥文件的设备的文件系统。或者，指定`auto`表示自动探测文件系统类型。
- `*path*`：在该文件系统中，密钥文件的绝对路径。

例如：`cryptkey=LABEL=usbstick:vfat:/secretkey`

若密钥文件直接对应某一设备上的一段比特流：

```
cryptkey=device:offset:size
```

offset表示比特流起始位置相对于设备首部的偏移量，size表示比特流长度。offset、size的单位都是字节。例如：

```
cryptkey=UUID=ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ:0:512
```

从指定设备开头处读取512字节，作为密钥文件内容。

提示：若要使用的设备路径包含字符“:”，需要使用反斜杠“\”转义。例如，对于ID为`usb-123456-0:0`的USB设备，可设置`cryptkey=/dev/disk/by-id/usb-123456-0\:0:0:512`。

若密钥文件已包含在initramfs中：

```
cryptkey=rootfs:path
```

例如：`cryptkey=rootfs:/secretkey`。

注意，若没有指定`cryptkey`，将默认尝试从initramfs中读取`/crypto_keyfile.bin`。

另见Dm-crypt/设备加密#密钥文件。

crypto

该参数专门用于传递dm-crypt plain模式的加密选项。

参数格式如下：

```
crypto=hash:cipher:keysize:offset:skip
```

参数内容直接与cryptsetup选项有关。参见Dm-crypt/设备加密#plain模式的加密选项。

即使设备采用plain模式的默认选项加密，也必须指定crypto参数。但该参数的每个部分都可以留空：

```
crypto=::::
```

该参数的具体例子如下：

```
crypto=sha512:twofish-xts-plain64:512:0:
```

使用systemd-cryptsetup-generator

systemd-cryptsetup-generator是用于解锁加密设备的systemd单元生成器。其将读取部分内核参数、/etc/crypttab以进行生成。关于其的详细叙述和支持的选项，参见systemd-cryptsetup-generator(8)。

若使用sd-encryptmkinitcpio钩子、systemd dracut模块，则systemd-cryptsetup-generator将在initramfs阶段运行。

下面将介绍部分systemd-cryptsetup-generator读取的内核参数。

提示：

- 若系统中存在/etc/crypttab.initramfs文件，则其将被复制到initramfs中的/etc/crypttab，以便指定需要在initramfs阶段解密的设备。该文件的语法详见#crypttab。若不存在/etc/crypttab.initramfs文件，则initramfs中将不包含/etc/crypttab。此时必须通过下面介绍的内核参数指定要解密的设备。
- 与rd.luks参数不同，/etc/crypttab.initramfs不局限于使用UUID指定设备。可以使用任何块设备持久化命名方法。
- 若所有前提条件满足，则可以使用GPT分区自动挂载，在/etc/crypttab.initramfs中可使用/dev/gpt-auto-root-luks代指加密的根分区，而无需手动指定UUID、PARTUUID等设备标识符。详见#systemd-gpt-auto-generator。
- 在解密过程中，输入的密码将被systemd-cryptsetup(8)缓存到内核钥匙环中，故若多个设备使用相同的密码，只需要输入一次密码。钥匙环中缓存的密码还可用于在后期用户空间中解锁crypttab中指定的其它设备。

注意：

- 所有rd.luks相关参数都可多次指定，以解密多个设备。
- rd.luks相关参数仅支持解密LUKS设备。若要解密dm-crypt plain模式加密的设备，必须在/etc/crypttab.initramfs中指定。语法详见#crypttab。
- 在修改/etc/crypttab.initramfs后，需要重新生成 initramfs。

警告：若同时使用/etc/crypttab//etc/crypttab.initramfs配置文件和luks.*/rd.luks.*内核参数，则只有在内核参数中指定了的设备会被解密。对于crypttab中指定，但内核参数未指定的设备，将不会解密，并显示Not creating device 'devicename' because it was not specified

on the kernel command line.警告。若要解密/etc/crypttab中指定的所有设备，不要设置luks.*内核参数，但仍可使用rd.luks.*内核参数指定initramfs阶段要解密的设备。若要解密/etc/crypttab.initramfs中指定的所有设备，不要设置任何luks.*、rd.luks.*内核参数。

rd.luks.name /rd.luks.uuid

有两个独立的内核参数用于在启动阶段解锁LUKS设备

请注意，应引用 LUKS 容器的 UUID，而不是 PARTUUID，也不是已解锁的/映射文件系统的 UUID。

```
rd.luks.name=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=name
```

指定要打开的LUKS分区的UUID，并为已解锁的/映射设备分配一个显式的名称（例如/dev/mapper/name）。

```
rd.luks.uuid=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
```

只指定LUKS的UUID且未提供显式名称的话，解密后的设备将被映射到/dev/mapper/luks-*XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX*。

提示：使用rd.luks.name或者rd.luks.uuid，不要同时用两个。

rd.luks.key

指定用于解密特定UUID的分区所需要的密钥文件。与encrypt钩子的cryptkey参数不同，密钥文件位置没有默认值。

若密钥文件已包含在initramfs中：

```
rd.luks.key=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=/path/to/keyfile
```

或者：

```
rd.luks.key=/path/to/keyfile
```

提示：若密钥文件已经存储到/etc/cryptsetup-keys.d/*name*.key，则无需指定rd.luks.key参数。

若密钥文件在另一设备上：

```
rd.luks.key=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=/path/to/keyfile:UUID=ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ
```

将UUID=*ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ*替换为密钥文件所在设备的UUID。

警告：

- 若存储密钥文件的设备使用的文件系统与根文件系统不同，则必须在initramfs中包含所需内核模块。
- 默认情况下，若rd.luks.key指定了位于另一设备上的密钥文件，但该设备在启动过程中不可用，则不会回退到使用密码解密。若需要回退，在rd.luks.options参数中指定keyfile-timeout=选项。例如，若要等待该设备10秒，使用：

```
rd.luks.options=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=keyfile-timeout=10s
```

注意：包含密钥文件的卷不会在启动过程结束后自动卸载。详见systemd issue 28021。

rd.luks.options

```
rd.luks.options=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=options
```

或者：

```
rd.luks.options=options
```

指定用于解密特定UUID的分区时的额外选项。若不指定UUID，则指定的选项将用于所有解密过程，除非在其它地方（例如，crypttab）单独为该解密过程指定了选项。

该参数作用与crypttab的选项字段相同，和crypttab选项字段的格式也相同，都使用逗号分隔，使用`*option*=*value*`指定带有值的选项。该参数与`encrypt`钩子的`cryptdevice`参数类似。

例如：

```
rd.luks.options=timeout=10s,discard,password-echo=no,tries=1
```

超时时间

在启动时，能用于输入密码解密卷的时间受两个选项影响：

- `rd.luks.options=timeout=*mytimeout*`：询问密码的超时时间
- `rootflags=x-systemd.device-timeout=*mytimeout*`：systemd等待根文件系统所在设备出现的时间（默认为90秒）

若要完全关闭超时，将两个超时时间都设为0：

```
rd.luks.options=timeout=0 rootflags=x-systemd.device-timeout=0
```

密码回显

当输入密码时，systemd-cryptsetup默认将对每个输入的字符回显星号（*）。而`encrypt`钩子在输入时将不回显。要禁用回显，设置`password-echo=no`选项：

```
rd.luks.options=password-echo=no
```

可信平台模块与FIDO2密钥

若系统有TPM2芯片，或者使用了FIDO2兼容的安全密钥。可以使用它们而非密码、密钥文件来解密卷。

除了`rd.luks.uuid`/`rd.luks.name`外，还需添加如下参数：

- 若使用TPM2芯片：`rd.luks.options=*XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX*=tpm2-device=auto`
- 若使用FIDO2密钥：`rd.luks.options=*XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX*=fido2-device=auto`

此外，也可以在`/etc/crypttab.initramfs`中指定上述参数：

```
/etc/crypttab.initramfs
```

```
root  UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX  none  tpm2-device=auto
```

上述配置中，使用UUID指定加密设备名称，解密后的设备使用`root`挂载名称（解密后的设备可通过`/dev/mapper/root`访问），不使用密码解密，而是从TPM2设备获取密钥。

注意，若要使用TPM2解密，在密码字段必须填写`none`或者`-`，而不能留空。否则，`tpm2-device=auto`将被识别为密码，在启动时，将尝试使用该“密码”解密，并在解密失败时要求手动输入密码，而不尝试使用TPM2解密。

若通过UUID指定设备，请确保其是在磁盘上实际存在的加密容器的UUID，而不是解密后的根文件系统的UUID。

固定 LUKS 卷

可以使用 `fixate-volume-key=` 选项，在 crypttab 配置中固定 LUKS 卷密钥的预期哈希值。该选项允许指定预期的卷密钥哈希值，并在哈希值不匹配时阻止系统激活该卷。预期的哈希值对应于使用 `tpm2-measure-pcr=yes` 选项时，在 TPM2 的 SHA256 PCR 寄存器组中测量得到的摘要。

此功能有助于防御恶意操作系统利用从真实根文件系统复制的元数据（例如分区 UUID）来冒充原始分区的攻击。为了获得最佳安全性，应将此选项与经过签名的 统一内核映像 配合使用。

要确定 `fixate-volume-key=` 选项所需的预期哈希值：

1. 启用 `tpm2-measure-pcr=yes`。
2. 重新启动系统。
3. 运行以下命令获取哈希值：

```
# grep volume-key /run/log/systemd/tpm2-measure.log | jq -r --seq '.digests[].digest'
```

最终的内核命令行选项通常如下所示：

```
rd.luks.options=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=tpm2-device=auto,tpm2-measure-pcr=yes,fixate-volume-key=YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
```

或者，也可以使用 `/etc/crypttab`：

```
/etc/crypttab
root  UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX  none  x-initrd.attach,tpm2-device=auto,tpm2-measure-pcr=yes,fixate-volume-key=YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
```

rd.luks.data

当使用分离的LUKS头时，使用该参数可指定存储加密数据的块设备。同时必须在`rd.luks.options`中指定LUKS头的位置。

详见Dm-crypt/特殊应用#在加密系统时使用分离的LUKS头。

systemd-gpt-auto-generator

若满足Systemd#GPT分区自动挂载中的所有要求，则可以不设置`rd.luks`相关内核参数。systemd-cryptsetup-generator将自动探测并尝试解密LUKS加密的根分区。

udev将创建`/dev/gpt-auto-root`、`/dev/gpt-auto-root-luks`符号链接，分别指向解密后的根分区、存储根分区加密容器的分区。可在相关配置文件中使用，作为对相关分区的持久性引用。

在指定crypttab选项时，可以在`/etc/crypttab.initramfs`中使用`/dev/gpt-auto-root-luks`作为设备名称。例如，一个不设置额外选项、使用自动解密的配置文件如下：

```
/etc/crypttab.initramfs
root  /dev/gpt-auto-root-luks  none  tpm2-device=auto,fido2-device=auto
```

在后期用户空间中解密

crypttab

与fstab相似，`/etc/crypttab`（encrypted device table，加密设备表）包含了一系列需要在启动过程中挂载的文件系统，不同之处在于`crypttab`中指定的是需要被解密的加密设备。可以使用该文件自动挂载加密的swap设备、非根文件系统。

`crypttab`在`fstab`之前处理，以便在尝试挂载加密容器中的文件系统之前，先解密加密容器。

注意，`crypttab`在系统启动后（后期用户空间，挂载根文件系统后）才被处理，因此在加密了根分区的情况下，不能用于替代mkinitcpio钩子、内核参数配置。`crypttab`将由systemd-cryptsetup-generator(8)处理。

该文件语法详见crypttab(5)。此外，下文还提供了一些例子。在#在启动时挂载节中还包含了使用UUID挂载加密设备的方法。

警告：

- 若设置了`nofail`选项，则密码输入界面可能会在输入过程中消失。因此，只应在使用密钥文件时设置`nofail`选项。
- 对于dm-crypt plain模式加密的设备，必须指定`plain`选项，以便systemd-cryptsetup(8)正确识别。参见systemd issue 442。

```
/etc/crypttab
# 示例crypttab文件。字段：名称、底层设备、密码、cryptsetup选项

# Mount /dev/lvm/swap re-encrypting it with a fresh key each reboot
swap  /dev/lvm/swap  /dev/urandom  swap,cipher=aes-xts-plain64,size=256,sector-size=4096

# Mount /dev/lvm/tmp as /dev/mapper/tmp using plain dm-crypt with a random passphrase, making its contents unrecoverable after it is dismounted.
```

```
tmp  /dev/lvm/tmp  /dev/urandom  tmp,cipher=aes-xts-plain64,size=256

# Mount /dev/lvm/home as /dev/mapper/home using LUKS, and prompt for the passphrase at
boot time.
home  /dev/lvm/home

# Mount /dev/disk/by-uuid/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx as /dev/mapper/backup
using LUKS, with a passphrase stored in a file.
backup  UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  /home/alice/backup.key

# Unlock /dev/disk/by-partuuid/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx using the only
available TPM, naming it myvolume
myvolume  PARTUUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  none  tpm2-device=auto
```

修改crypttab后，若要立即测试并应用更改，先使用daemon-reload重新加载systemd配置，然后启动新生成的`systemd-cryptsetup@*name*.service`单元。

```
# cryptsetup status name
/dev/mapper/name is active.
  type:    ...
  cipher:  ...
  keysize: ... bits
  key location: ...
  device:  /dev/sdxN
  sector size:  ...
  offset:  ... sectors
  size:    ... sectors
  mode:    ...
  flags:   ...
```

有关`systemd-cryptsetup@*name*.service`单元的更多信息，详见#按需挂载。

提示：

- 若使用GUID分区表（GPT），并正确设置分区类型UUID，则在使用systemd时，对于特定挂载点，可以自动挂载，而无需编写crypttab、fstab条目。详见Systemd#GPT分区自动挂载。
- 使用sd-encrypt钩子时，启动过程中手动输入的密码将由systemd-cryptsetup(8)缓存到内核钥匙环中（包括initramfs阶段输入的密码）。若`crypttab`中的某一条目对应设备的密码与之前的条目相同，则可设置第三个选项为`none`，将自动使用缓存的密码进行解密。如果密码未被缓存，或者使用的是 `encrypt` 钩子，则系统会提示输入密码。

注意：`crypttab`中设置的`timeout`选项只决定**输入密码**的超时时间。此外，systemd还有**等待设备可用**的超时时间（默认为90秒）。这两个超时时间是独立的。因此，即使`crypttab`中的`timeout`选项设置值长于90秒（设置为0表示无限时），systemd仍然只会等待某个设备90秒。要修改systemd的等待时间，可在fstab中，为相应的设备设置`x-systemd.device-timeout`选项（详见systemd.mount(5)）。建议对于启动时要挂载的加密设备，将`crypttab`中的`timeout`与fstab中的`x-systemd.device-timeout`设置为相同的值。

使用密钥文件解密

若根文件系统是加密的，可在其中存储解密其它设备的密钥文件。由于根文件系统是加密的，因此在系统关机时，密钥文件是安全的。存储密钥文件的好处在于，在启动时，可通过设置crypttab，自动通过密钥文件解密其它设备，而无需输入密码。

例如，解密指定UUID的设备：

```
/etc/crypttab
```

```
home-crypt  UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX  /etc/cryptsetup-keys.d/home-crypt.key
```

提示：

- 若没有指定密钥文件，systemd-cryptsetup(8)会自动尝试从`/etc/cryptsetup-keys.d/*name*.key`、`/run/cryptsetup-keys.d/*name*.key`读取密钥文件。
- 若使用dm-crypt plain模式，需要在`/etc/crypttab`中指定相关加密选项。参见#crypttab节中提到的systemd相关注意事项。

之后，使用`/etc/crypttab`中定义的设备映射名称，添加对应的`/etc/fstab`条目：

```
/etc/fstab
/dev/mapper/home-crypt  /home  ext4  defaults  0 2
```

`/dev/mapper/home-crypt`已经是持久性的、唯一的分区映射名，可直接使用，而无需通过UUID指定。若要使用UUID指定，应当注意加密容器中文件系统的UUID和包含加密容器的分区的UUID不同。

挂载多层映射的块设备

在启动时，systemd的生成器将自动处理多层映射的块设备。

例如，可以先建立RAID，在此之上创建加密卷，再在加密卷上创建LVM逻辑卷，形成多层映射：

```
$ lsblk -f
─sdXX                  linux_raid_member
│ └─md0                crypto_LUKS
│   └─cryptedbackup    LVM2_member
│     └─vgraid-lvraid  ext4               /mnt/backup
└─sdYY                 linux_raid_member
  └─md0                crypto_LUKS
    └─cryptedbackup    LVM2_member
      └─vgraid-lvraid  ext4               /mnt/backup
```

启动过程中，将询问密码，并自动挂载所有层级的设备。

对于非根分区，只要正确设置了相应crypttab、fstab条目，就无需额外添加mkinitcpio钩子或修改相关配置。注意`/etc/crypttab`只适合于处理非根分区的挂载。若在挂载在RAID上的根分区时使用了`mdadm_udev`钩子，则需要额外配置`/etc/madadm.conf`并更新initramfs，以正确加载RAID配置。

按需挂载

若在`/etc/crypttab`中有如下条目：

```
/etc/crypttab
externaldrive UUID=... none noauto
```

则若要打开`externaldrive`，除了直接使用cryptsetup外：

```
# cryptsetup open UUID=... externaldrive
```

还可启动`systemd-cryptsetup@externaldrive.service`。这样，无需在打开加密卷时手动输入加密选项。若需要密码进行解锁，启动单元时将提示输入。

有关单元文件由systemd-cryptsetup-generator(8)生成。可通过如下命令查看所有生成的单元文件：

```
$ systemctl list-unit-files | grep systemd-cryptsetup
```

非阻塞挂载

若在crypttab中指定了某一设备，即使该设备并不需要在启动时就完成挂载，在默认情况下，cryptsetup服务仍会阻塞地等待该设备完成挂载，再继续启动进程。可以通过指定`nofail`选项避免阻塞等待。添加该选项后，将从指定设备的挂载服务（systemd mount service）中移除`Before=cryptsetup.target`，避免启动过程阻塞。若设备不被启动过程依赖，且使用密钥文件解锁，则可使用`nofail`选项加快启动过程。

```
/etc/crypttab
data UUID=d0d0d110-0a71-4ed6-936a-304969ea36af /etc/cryptsetup-keys.d/data.key nofail
```

疑难解答

系统在启动时卡住/不显示输入密码提示符

若使用了Plymouth，需要确保使用了正确的模块（详见Plymouth#mkinitcpio），否则Plymouth将覆盖密码输入提示符，导致无法解密设备，系统无法启动。

无法使用键盘输入密码/无法从文件系统上读取密钥文件

若需要使用键盘输入密码（特别是键盘通过USB拓展坞连接时），或要从特定文件系统上读取密钥文件（且该文件系统在生成initramfs时未挂载），可能需要手动向mkinitcpio的`MODULES`数组中添加需要的模块。详见Mkinitcpio#模块（MODULES）。键盘、文件系统需要的模块名称可参见Mkinitcpio/极简_initramfs#整理模块。

对于外设连接经常改变（即，后续启动时连接的外设可能和生成initramfs镜像时不一致）的系统，需要将`keyboard`钩子放置到`autodetect`钩子**之前**，以包含所有键盘有关模块，而不是只包含在生成initramfs时连接到计算机的键盘的模块。详见Mkinitcpio#常用钩子。

文件传输无响应

向加密的机械硬盘（HDD）复制大文件时，可能会导致文件传输过程无响应。该问题的根本原因在于 dm-crypt 的工作队列：这些队列可能在短时间内被大量请求迅速填满，而请求的产生速度超过了底层加密物理设备的写入速度。即使使用响应性良好的 I/O 调度器也无济于事，因为 dm-crypt 的工作队列作为独立的缓冲区，位于 I/O 栈其余部分之前。可以通过禁用工作队列来缓解此问题。

2.8 静态数据加密

难度 高级 | **篇幅** 45.9 KB | **所属** 静态数据加密

本节将讲到

- 为什么要使用加密?
- 准备
- 加密是怎么运作的
- 用于磁盘加密场景的数据备份

本文讨论静态数据加密软件，该软件即时加密/解密写入/读取块设备、磁盘分区或目录的数据。块设备的示例是硬盘驱动器、闪存驱动器和 DVD。

静态数据加密仅应被视为操作系统现有安全机制的附属品——专注于保护物理访问，同时依赖系统的其他部分提供网络安全和基于用户的访问控制等功能。

对于全盘加密 (Full-disk encryption，FDE)，参阅Dm-crypt/加密整个系统。

为什么要使用加密?

静态数据加密确保文件始终以加密的形式存储在磁盘上。这些文件只有在系统运行并由受信任的用户解锁时才以可读形式提供给操作系统和应用程序（使用中的数据或传输中的数据）。未经授权的人直接查看磁盘内容，只会发现混乱的随机数据，而不是实际的文件。

例如，这可以防止未经授权的数据查看，当电脑或硬盘：

- 处于当你离开时不受信任的人可能获得使用机会的场所
- 丢失或被盗，如笔记本电脑、上网本或外部存储设备
- 在维修店里
- 淘汰后被丢弃

此外，静态数据加密还可用于增加一些安全性，以防止未经授权的篡改操作系统的尝试——例如，可以获得物理访问权限的攻击者趁你不在的时候安装键盘记录程序（keyloggers）或特洛伊木马（Trojan）。

警告：静态数据加密并不能保护你的数据免受所有威胁。

你仍面临以下潜在风险：

- 攻击者可能在系统运行期间，且你已经解锁并挂载了磁盘的加密部分时入侵系统（如通过互联网）。

- 攻击者可能在计算机运行期间（即使已启用屏幕锁定程序），获取对设备的物理访问权限，或者在计算机停止运行后的极短时间内，借助相关资源实施冷启动攻击。
- 政府机构不仅拥有轻易实施上述攻击的资源，还可能通过各种胁迫手段强迫你交出密钥或密码短语。执法机构若怀疑你可能隐藏了某些可疑信息，这种行为可能是合法的。
- 橡胶管密码分析法（Rubber-hose cryptanalysis），即暴力威胁。另请参阅 XKCD #538

为了抵御能够在你使用系统之前对其进行篡改的专业攻击者，必须配置极高强度的磁盘加密方案（例如，具备完整性校验且无明文引导分区的全系统加密）。即便如此，这也无法防范所有类型的篡改（例如硬件键盘记录器）。最优的应对策略或许是基于硬件的全盘加密与可信计算。

警告：静态数据加密也无法防止他人直接擦除你的磁盘。建议定期备份，以确保数据安全。

系统数据加密

虽然仅加密用户数据本身（通常位于家目录中，或在数据 DVD 等可移动媒体上）是最简单且侵入性最小的方法，但是它有一些明显的缺点。 在现代计算机系统中，有许多后台进程可能会在硬盘驱动器的非加密区域中缓存和存储有关用户数据或部分数据本身的信息，例如：

: - 交换分区（swap）

- （可能的补救措施：禁用交换，或使用加密过的交换分区）
- `/tmp`（用户应用程序创建的临时文件）
- （可能的补救措施：避免此类应用程序；在ramdisk内挂载`/tmp`）
- `/var` （日志文件和数据库等；例如，mlocate将所有文件名的索引存储在`/var/lib/mlocate/mlocate.db`中)

解决办法就是对系统和用户数据都进行加密，阻止未经授权的对可能被系统缓存的隐私数据的物理访问。不过，这样也带来了一个缺点——必须在启动时对磁盘的加密部分进行解锁。系统数据加密的另一个好处是，它给具有物理访问权限的人安装诸如键盘记录器或 rootkits 之类的恶意软件带来了麻烦。

准备

选择设置

根据你的目的（请阅读上文#为什么要使用加密？）及系统参数选取合适的加密设置。

此外，你需要回答下列问题：

你想抵御什么样的“攻击者”？

- 在你的系统被关闭、被盗等情形下窥探你的磁盘的普通用户
- 可在你使用你的系统前后反复获得读写权限的专业密码分析员
- 任何介于这两者之间的人

你想加密什么？

- 仅用户数据
- 用户数据和系统数据
- 仅机密数据，也就是你的数据的子集

交换分区（swap）、`/tmp` 等，应如何处理？

- 禁用或挂载为 ramdisk
- 被加密的交换分区
- 将交换文件作为全盘加密的一部分
- 单独加密交换分区

应如何解锁磁盘的加密部分？

- 使用密码
- 同账户登录密码
- 不同于账户登录密码
- 使用密钥文件（例如将其置于随身携带的或保存在安全地方的U盘上）
- 二者兼有

***何时*解锁磁盘的加密部分？**

- 引导前
- 引导过程中
- 登录账户时
- 按需手动解锁（**账户登录后**）

应如何满足多用户需求？

- 不需要
- 使用每个用户都知道的共享密码（或密钥文件）
- 用于磁盘的同一加密部分的，被独立分发的、可撤销的密码（或密钥文件）
- 为不同的用户分配磁盘的加密部分

接下来你可以做出所需的技术选择（参阅上文#可用方法，以及下文#加密是怎么运作的），关于：

- 块设备加密与文件系统叠加加密的对比
- 密钥管理
- 密码及操作模式
- 元数据存储
- “下层目录”的位置（在文件系统叠加加密的情况下）

准备磁盘

在一（部分）磁盘上设置加密之前，先考虑安全地擦除它。这包括用零字节或随机字节的流覆盖整个驱动器或分区，并且出于以下一个或两个原因而做：

防止恢复以前存储的数据

当文件系统创建或修改这些特定扇区所保存的数据时，磁盘加密不会改变单个扇区仅按需覆盖的事实（请参阅下面的#加密是怎么运作的。文件系统认为“当前未使用”的扇区不会被触及，并且可能仍包含来自先前文件系统的数据残余。确保无法恢复之前存储在驱动器上的所有数据的唯一方法是手动擦除它。 为此，使用零字节或随机字节都没有关系（尽管使用零字节擦除会快得多）。

防止泄露加密驱动器上的使用模式

理想情况下，磁盘的整个加密部分应该与均匀随机数据无法区分。这样，未经授权的人就无法知道哪些扇区以及多少个扇区实际上包含加密数据——这本身可能是一个理想的目标（作为真正机密性的一部分），也可以作为抵御试图破解加密的攻击者的额外屏障。 为了实现这个目标，使用高质量随机字节擦除磁盘至关重要。

第二个目标仅与块设备加密结合使用才有意义，因为在文件系统叠加加密的情况下，无论如何都可以轻松定位加密数据（以独特的加密文件的形式出现于宿主文件系统中）。另请注意，即使你只打算加密特定文件夹，如果你想删除以前以未加密形式存储在该文件夹中的文件，则必须擦除整个分区（由于磁盘碎片）。如果同一分区上还有其他文件夹，则必须备份它们，然后再将它们移回。

确定要执行哪种磁盘擦除后，请参阅文章安全地擦除磁盘以获取技术说明。

提示：在决定使用哪种方法对硬盘驱动器进行安全擦除时，请记住，只要驱动器被用作加密驱动器，就不需要多次执行此操作。

可用方法

所有静态数据加密方法都是这样操作的：虽然磁盘实际存放的是加密数据，但只要加密容器（即磁盘中存放加密数据的逻辑部分）被“解锁”并被挂载，那么操作系统和应用程序就会把它 "看成 "相应的普通的可读数据。

为此，用户需要提供一些“机密信息”（通常采用密钥文件和/或密码短语的形式），从这些信息中可以派生出实际的加密密钥（并在会话期间存储在内核密钥环中）。

如果你对这种操作完全不熟悉，还请阅读下文#加密是怎么运作的的部分。

可用的静态数据加密方法可以按其操作层分为两种类型：

栈式文件系统加密

作为堆叠在已有文件系统之上的层而实现的文件系统叠加加密方案，使所有写入启用加密的文件夹的文件在底层文件系统将其写入磁盘之前就被即时加密，并在文件系统从磁盘读取它们时被解密。这样，文件以加密的形式存储在宿主文件系统中（意味着它们的内容，通常还有它们的文件/文件夹名称，都被长度大致相同的随机数据所取代），但除此之外，它们仍然以正常的文件/符号链接/硬链接等形式存在于该文件系统中，如同没有加密一样。

它的实现方式是，为了解锁宿主文件系统中存储原始加密文件的文件夹（「下层目录」），它被挂载（使用一个特殊的堆叠的伪文件系统）到它本身或可选的不同位置（「上层目录」），然后相同的文件以可读的形式出现——直到它再次被卸载，或系统被关闭。

此类别中可用的加密方案是eCryptfs和EncFS。

云存储优化

如果你正在部署文件系统叠加加密，以实现与第三方控制的位置(如云存储服务)的零知识同步（zero-knowledge synchronization），你可能需要考虑替代 eCryptfs 和 EncFS 的方案，因为他们并不是为了通过 Internet 传输文件这一目的而优化的。有一些解决方案是为了这个目的而设计的：

- gocryptfs
- cryptomatorAUR 或 cryptomator-binAUR （跨平台）
- cryfs包

请注意，一些云存储服务直接通过他们自己的客户端应用程序提供零知识加密（zero-knowledge encryption）。

块设备加密

另者，块设备加密（block device encryption）方法操作于文件系统层之下，确保写入某个块设备（即整个磁盘、分区或充当回环设备的文件）的所有内容都是加密的。这意味着当块设备处于脱机（offline）状态时，它的整个内容看起来就像一大团随机数据（a large blob of random data），无法确定它包含什么样的文件系统和数据。通过一种特殊方式将受保护的容器（在本例中为块设备）挂载到任意位置，再次访问数据。

Arch Linux 中提供了以下几种“块设备加密”方案：

loop-AES : loop-AES 是 cryptoloop 的继任者，是一种安全、快速的系统加密解决方案。然而，loop-AES 被认为不如其他选择易用（less user-friendly），因为它需要非标准内核支持（non-standard kernel support）。

dm-crypt : dm-crypt 是由 Linux 内核提供的标准设备映射器加密（standard device-mapper encryption）功能。喜欢全面掌握分区及密钥管理的各个方面的人直接用它就行了。对 dm-crypt 的管理是通过 cryptsetup包 用户空间实用程序完成的。它可用于以下类型的块设备加密：**LUKS（默认）**、**plain**，并且具备有限的用于 **loopAES** 和 **Truecrypt** 设备的功能。

- 默认情况下使用的 LUKS 是一个额外的便利层，它将 dm-crypt 所需的所有设置信息存储在磁盘本身上，并抽象分区和密钥管理，以提高易用性和加密安全性。
- plain 模式下的 dm-crypt，作为原始的内核功能，并没有采用便利层。用它来应用（与 LUKS 类型）同样的加密强度是比较困难的。这样做时，会产生更长的密码或密钥文件（二者被统称为 keys）。但是，它具有其他优点，描述于下文 #块设备加密与文件系统叠加加密的对比\[损坏的链接：无效的章节\]之中。

TrueCrypt/VeraCrypt : 一种可移植（portable）格式，支持整个磁盘/分区或文件容器的加密，兼容所有主流操作系统。对 TrueCrypt 的开发于2014年5月被其开发者停止。其分支 VeraCrypt 于2016年被审计。

关于选定的操作层的实际含义，参阅下文#块设备加密与文件系统叠加加密的对比\[损坏的链接：无效的章节\]，以及对 eCryptfs的总体描述。参阅Category:Encryption，了解下表比较的方法的可用内容，以及未包括在表中的其他工具。

块设备加密与文件系统叠加加密的对比

	块设备加密	原生/叠加式文件系统加密
加密	块设备	文件

	块设备加密	原生/叠加式文件系统加密
加密数据的容器可能是……	磁盘、磁盘分区，或作为回环设备使用的文件	现有文件系统中的目录
与文件系统的关系	在文件系统层下运行：不关心加密块设备的内容是文件系统、分区表、LVM 设置还是其他任何内容	通过文件系统原生功能，或向现有文件系统添加额外的叠加加密层来实现加密。两者都会在文件写入/读取时自动加密/解密。，以在文件写入或读取时自动进行加密/解密
加密文件元数据（文件数、目录结构、文件大小、权限、最后修改时间等）	是 （使用 'discard' 可能会暴露文件的大小）	部分 （文件名会被加密，其他元数据是否可见取决于具体功能实现）
可被用于可用于定制加密整个硬盘（包括分区表）（包括分区表）	是	否
可被用于加密交换空间	是	否
可被用作加密数据容器而无需预先分配固定大小的加密数据容器	否 （使用 'discard' 可能允许稀疏分配的容器，代价是暴露文件的大小）	是
可被用于保护无法直接访问块设备的现有文件系统，例如NFS 或 Samba 共享、云存储等	否1	是2
允许对加密文件进行允许对加密文件进行离线文件级备份	否	是

1. 理论上在这些文件系统中，一个单独的文件也可以被用作容器（通过虚拟回环设备），但这样实际上就不再是在使用该文件系统（以及它提供的功能）了。
2. 对于 ZFS 不适用。对于其他文件系统原生加密方式，可能会存在挂载限制。

比较表

“dm-crypt +/- LUKS” 一栏表示 dm-crypt 在 LUKS（"+"）和 plain（"-"）加密模式下的功能。如果某个特定功能需要使用 LUKS，则以“（使用 LUKS）”表示。同样，“（不使用 LUKS）”表示使用 LUKS 会阻碍实现该功能，应使用 plain 模式。

概要（Summary）	**Loop-AES**	**dm-crypt +/- LUKS**	**VeraCrypt**	**ZFS**	**fscrypt**	**eCryptfs**	**EncFS**	**gocryptfs**
加密类型	块设备	块设备	块设备	原生文件系统或块设备	原生文件系统	堆叠式文件系统	堆叠式文件系统	堆叠式文件系统
说明（Note）	存在时间最长的一种；可能是速度最快的；适用	Linux 上块设备加密的事实标准；非常灵活	TrueCrypt 的受维护分支，支持 TrueCrypt 和 VeraCrypt 卷	加密功能相对较新（2019 年）；通过 ZVOL 提供加密块设备	Chrome OS 和 Android 加密的默认方案	比 EncFS 稍快；单个加密文件可在不同系统间移动	最容易使用的一种；支持非 root 用户管理	EncFS 的潜在继任者

概要（Summary）	Loop-AES	dm-crypt +/- LUKS	VeraCrypt	ZFS	fscrypt	eCryptfs	EncFS	gocryptfs
	于旧系统							
在 Arch Linux 中的可用性	需要手动编译、自定义内核	**内核模块：**已随默认内核提供；**工具：**device-mapper包、cryptsetup包	veracrypt包	ZFS#安装	**内核模块：**已随默认内核提供；**工具：**fscrypt包	**内核模块：**已随默认内核提供；**工具：**ecryptfs-utils包	encfs包	gocryptfs包
许可证	GPL	GPL	Apache License 2.0，部分代码受 TrueCrypt License v3.0 约束	CDDL	GPL（内核）、Apache 2.0（用户空间工具）	GPL	GPL	MIT
加密实现	内核空间	内核空间	内核空间	内核空间	内核空间	用户空间（FUSE）	用户空间（FUSE）	用户空间（FUSE）
加密元数据存储	？	使用 LUKS：LUKS 标头	（已解密）设备的开始/结束位置 (格式规范)					

演讲/ 幻灯片)

(格式规范)

演讲/ 幻灯片)

可用功能（Usability features）	Loop-AES	dm-crypt +/- LUKS	VeraCrypt	ZFS	fscrypt	eCryptfs	EncFS	gocryptfs
非 root 用户可以创建/销毁加密数据容器	否	否	否	是	是	受限	是	是
提供GUI	否	否	是	否	否	否	是 可选	

使用 systemd 和 /etc/crypttab

使用 systemd

AES-Twofish、AES-Twofish-Serpent、Serpent-AES、Serpent-Twofish-AES、Twofish-Serpent

性能特性（Performance features）	Loop-AES	dm-crypt +/- LUKS	VeraCrypt	ZFS	fscrypt	eCryptfs	EncFS	gocryptfs
多线程支持	？	是 						

兼容性与流行度（Compatibility & prevalence）	Loop-AES	dm-crypt +/- LUKS	VeraCrypt	ZFS	fscrypt	eCryptfs	EncFS	gocryptfs
支持的 Linux 内核版本	2.0 或更新版本	CBC 模式自 2.6.4 起支持，ESSIV 自 2.6.10 起支持，LRW 自 2.6.20 起支持，XTS 自 2.6.24 起支持	？	2.6.32 或更新版本（截至 0.8.3）	4.1 或更新版本	？	2.4 或更新版本	？
加密数据也可以从 Windows 访问	？	？	是	是				

OpenZFS on Windows (仓库)

（需要管理员权限）

cppcryptfs 移植版

OpenZFS on OS X (仓库)

（测试版质量）

ZFS on FreeBSD （原生； 仓库）

示例

在实践中，结果可能是这样的：

示例 1：仅利用用户的家目录（home directory）里的一个被用 EncFS 加密的路径名为 `~/Private` 的虚拟文件夹的简单的（内部硬盘驱动器）用户数据加密

- 被 on-disk 存储的文件的加密版本位于 `~/.Private`（encrypted versions of the files stored on-disk in `~/.Private`）
- 使用专用密码按需解锁

示例 2：部分系统加密，每个用户的主目录用 ECryptfs 加密

- 使用账户登录密码，在各个用户登录时被解锁
- 具有用LUKS的dm-crypt加密的 `swap` 和 `/tmp` 分区，使用每个会话自动生成的一次性密钥
- 禁止 **slocate**（及类似的应用程序）索引或缓存 `/home` 的内容

示例 3：系统加密——具有用LUKS的dm-crypt加密的整个硬盘驱动器，`/boot` 分区除外（不过，用 GRUB 引导时，`/boot` 可被加密）

- 使用密码或带有密钥文件的 U 盘，在引导期间解锁
- 每个用户可能有不同的密码/密钥——可独立撤销的
- 可用 LUKS on LVM 实现跨驱动器加密或灵活性分区布局

示例 4：隐藏式/plain 系统加密——具有 plain dm-crypt 加密的整个硬盘驱动器

- USB-boot，使用专用密码 + 带有密钥文件的U盘
- 在挂载前检查数据完整性
- `/boot` 分区位于上述U盘上

示例 5：文件容器加密——将预分配的文件用作用户数据的加密容器

- 用Dm-crypt/加密不具有根目录的文件系统#文件容器加密类似 `~/.volume.img` 这样的文件
- 通过将整个容器复制到远程主机上进行不定期的备份

当然还有许多其他的组合。你应该仔细计划什么样的设置适合你的系统。

加密是怎么运作的

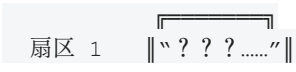
本节旨在作为对常规磁盘加密的核心设置的概念和过程的高级介绍。

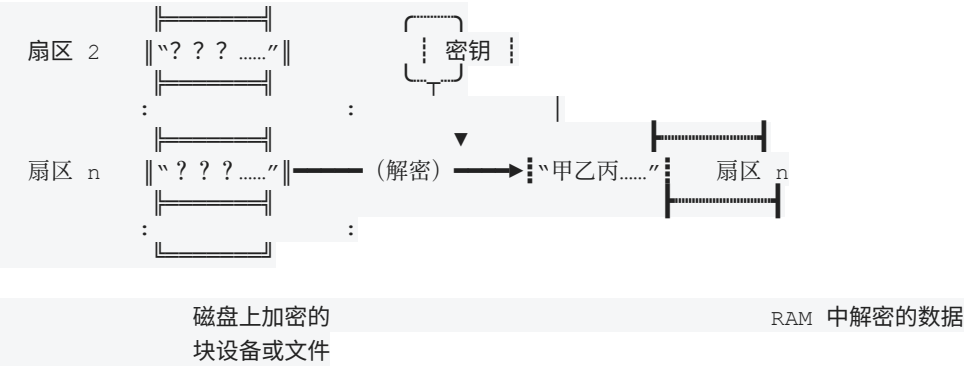
它不涉及技术或数学细节（请查阅相应的文献），但应该让系统管理员大致了解不同的设置选择（尤其是关于密钥管理）如何影响可用性和安全性。

基本原则

出于磁盘加密的目的，每个块设备（或文件系统叠加加密情况下的单个文件）被划分为等长的**扇区**，例如 512 字节（4,096 位）。然后在每个扇区的基础上进行加密/解密，因此磁盘上块设备/文件的第 n 个扇区将存储原始数据的第 n个扇区的加密版本。

每当操作系统或应用程序从块设备/文件请求某个数据片段时，包含数据的整个扇区（或扇区们）将被从磁盘读取，被即时解密，并被临时存储在内存中：





同样，在每次写入操作中，所有受影响的扇区都一定会被彻底地重加密（而其余扇区不受影响）。

为了能够对数据进行解密/加密，磁盘加密系统需要知道与之相关的唯一的秘密 "钥匙"。每当要挂载相关的加密块设备或文件夹时，必须提供与其相对应的密钥（以下称为“主密钥”）。

密钥的熵对于加密的安全性至关重要。一个随机生成的一定长度的字节串，例如32字节（256比特），虽具有所需的安全性，却没法记忆，也无法在挂载期间手动应用。

出于这个原因，有两种技术被用作辅助手段。第一种是应用密码学来增加主密钥的熵值，通常涉及一个单独的人类友好的口令。对于不同类型的加密，#比较表列出了各自的特点。第二种方法是创建一个具有高熵的密钥文件，并将其存储在与要加密的数据驱动器分开的介质上。

另请参阅维基百科：认证加密（英文页面）。

密钥、密钥文件和密码短语

以下是如何使用密钥文件来存储、加密保护主密钥的示例：

存储在明文密钥文件中

简直（以可读形式）将主密钥存储在文件中是最简单的选择。该文件——被称为“密钥文件”——可以放置于你保存在安全位置的 U 盘上，并且仅在你想要挂载磁盘的加密部分时（例如在启动或登录期间）将它连接到计算机。

以受密码保护的形式存储在密钥文件或磁盘本身中

主密钥（以及加密数据）可以使用一段必须记下来且每次你想要挂载加密的块设备或文件夹时都必须输入的保密密码短语（secret passphrase）进行保护。有关详细信息，请参阅下面的#加密的元数据。

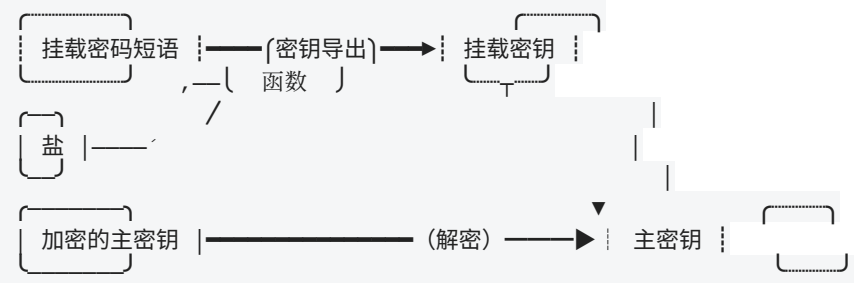
给每个会话随机生成一个临时的

在某些情况下，例如在加密交换空间（swap）或 `/tmp` 分区时，根本不需要保留永久的主密钥。可以给每个会话随机生成一个新的一次性密钥，而无需任何用户交互。这意味着一旦被卸载，写入相关分区的所有文件都不能被**任何人**再次解密——在那些特定的用例中，这非常好。

加密的元数据

加密技术经常使用密码函数来增强主密钥本身的安全性。在挂载加密设备时，密码或密钥文件通过这些（密码函数）传递，只有结果才能解锁主密钥以解密数据。

一个常见的设置是（通过一种“密钥导出函数（key derivation function）”）对密码短语（passphrase）应用所谓的“密钥拉伸（key stretching）”，并使用生成的增强密码短语作为挂载密钥来解密（之前以加密的形式存储的）实际的主密钥：



密钥导出函数（如 PBKDF2 或 scrypt）刻意地放慢速度（它应用一个哈希函数的多次迭代，例如 HMAC-SHA-512 的 1000 次迭代），因此通过暴力破解来找到密码短语变得不可行。对于认证用户的正常用例，每个会话只需要计算一次，因此小的减速不是问题。 它（密钥导出函数）还需要一个额外的数据块，即所谓的“**盐**”，作为一个参数——在设置磁盘加密期间随机生成一次，并被作为加密元数据的一部分不受保护地存储。因为每次设置的值都不同，攻击者无法使用密钥导出函数的预计算表来加速暴力破解。

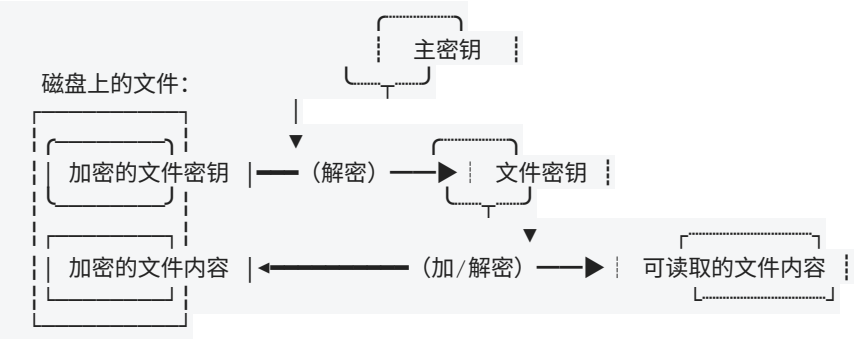
加密的主密钥可以与加密的数据一起存储在磁盘上。这样，加密数据的机密性完全取决于保密密码短语。

可以通过将加密的主密钥存储在密钥文件中来获得额外的安全性，例如U 盘。这提供了**双重身份验证(two-factor authentication)**：现在访问加密数据需要只有你**知道**的内容（密码短语），以及只有你**拥有**的内容（密钥文件）。

实现双重身份验证的另一种方法是增强上述密钥检索方案，以数学方式将密码短语与从（位于U 盘或类似设备上的）一个或多个外部文件读取的字节数据“结合”起来，然后将其传递给密钥导出函数。任何文件都在考虑范围之中，例如有助于#合理推诿\[损坏的链接：无效的章节\]的普通的JPEG图像。不过，在这种情况下，它们仍然被称为“密钥文件”。

在其导出后，只要挂载了加密的块设备或文件夹，主密钥就会被安全地存储在内存中（例如，在内核密钥环中）。

不过，它通常不直接用于对磁盘数据进行加/解密。 例如，在文件系统叠加加密的情形中，每个文件都可以自动分配它自己的加密密钥。每当文件要被读取/修改时，这个文件密钥首先需要用主密钥进行解密，然后它本身才能用于对文件内容进行加/解密。



以类似的方式，在文件系统叠加加密的情形下，可以使用单独的密钥（例如，每个文件夹一个）来加密文件名。

在块设备加密的情况下，每个设备使用一个主密钥，因此使用所有数据。一些方法提供了为同一设备分配多个密码短语/密钥文件的特性，而另一些方法则没有。有些使用上述函数来保护主密钥，有些则把密钥安全的控制权完全交给用户。通过 dm-crypt 在 plain 或 LUKS 模式下使用的加密参数解释两个例子。

当比较两种模式使用的参数时，我们注意到 dm-crypt plain 模式有关于如何定位密钥文件的参数（如`--keyfile-size`、`--keyfile-offset`）。dm-crypt LUKS 模式不需要这些，因为每个块设备在开头都包含一个带有加密元数据的标头。标头包括使用的密码、加密的主密钥本身以及派生解密所需的参数。后面的参数又来自于主密钥初始加密时使用的选项（如`--iter-time`、`--use-random`）。

对于不同技术的优/缺点，请参考#比较表或浏览特定页面。

另请参阅：

- Wikipedia:Passphrase
- Wikipedia:Key (cryptography))
- Wikipedia:Key management
- Wikipedia:Key derivation function

密码和操作模式

用于在相对于给定加密密钥彼此对应的未加密和加密数据（所谓的“明文（plaintext）”和“密文（ciphertext）”）之间进行转换的实际算法称为“**密码（cipher）**”。

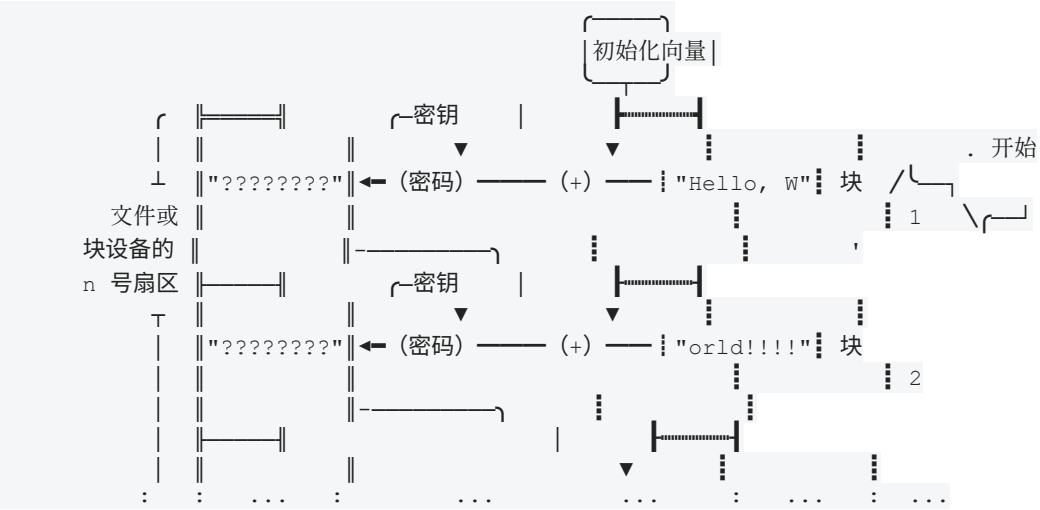
磁盘加密采用“块密码（block cipher）”，它对固定长度的数据块进行操作，例如16 字节（128 位）。在撰写本文时，主要使用的是：

	块大小	密钥大小	注释
AES	128 bits	128、192 或 256 bits	经 NSA 批准用于保护“机密”和“绝密”机密的美国政府信息（当使用 192 或 256位的密钥大小时）
Blowfish)	64 bits	32–448 bits	最早公开的无专利安全密码之一，因此在 Linux上非常成熟
Twofish	128 bits	128、192 或 256 bits	作为 Blowfish的继任者而开发，但尚未得到广泛使用
Serpent)	128 bits	128、192 或 256 bits	被认为是入围AES决赛的五个算法中最安全的.

加/解密扇区（见上文\[损坏的链接：无效的章节\]）是通过将其划分为与密码的块大小相匹配的小块来实现的，并遵循特定的规则集（所谓的“**操作模式（mode of operation）**”）将密码连续地应用于各个块。

仅将其不加以修改地单独应用于每个区块（被称为 "**电子编码本（electronic codebook，ECB）**"模式）是不安全的。因为如果相同的 16 字节明文总会产生相同的16 字节密文，攻击者可轻易识别出存储在磁盘上的密文中的加密模式。

实践中使用的最基本（且常见的）操作模式是“**密码分组链接（cipher-block chaining，CBC）**”。使用此模式加密扇区时，每个明文数据块以数学方式与前一个块的密文组合，然后使用密码对其进行加密。对于第一个块，由于它之前没有密文，因此使用了一个特殊的预生成的数据块（data block），该数据块与扇区的加密的元数据（cryptographic metadata）一起存储，称为“**初始化向量 (initialization vector, IV)**”：



磁盘上的密文 RAM 中的明文

类似地，在解密时，程序是反过来的。

对应于每个扇区的唯一的初始化向量的生成是值得注意的。最简单的选择是把现成的数值（如扇区号码）以某种可预见的方式计算。但这将给可以重复访问系统的攻击者执行所谓的水印攻击以可乘之机。为了预防这种情况，可以使用一种称为 “加密的盐扇区初始化向量 （Encrypted salt-sector initialization vector，**ESSIV**）”的方法来生成初始化向量，使它们在潜在攻击者看来完全随机。

还有许多其他更复杂的操作模式可用于磁盘加密，它们已经提供了针对此类攻击的内置安全性（因此不需要 ESSIV）。 有些还可以额外保证加密数据的真实性（即确认它没有被无权访问密钥的人修改/损坏）。

另请参见：

- Wikipedia:Disk encryption theory
- Wikipedia:Block cipher
- Wikipedia:Block cipher modes of operation

流密码

流密码以数据流的方式工作，不需要“操作模式”这样的概念。然而，磁盘加密所需的特性（例如“wide-block”）意味着需要在其基础上添加额外层。就 Linux 系统上的磁盘加密而言，主要例子是 Adiantum 配置中的 XChaCha12，即 xchacha12,aes-adiantum，专为硬件不支持 AES 加速的低端系统设计。Adiantum 在此类硬件上运行速度快，并且能够保证加密数据的真实性。

合理推诿

参见 Wikipedia:Plausible deniability.

用于磁盘加密场景的数据备份

为防止数据丢失，制作用户数据的备份。一般来说，你的被加密数据的备份也应该加密。

块设备加密

有多种选择：你可以将加密容器（encryption container）所在的磁盘块设备（disk block device）备份为映像（image），例如 `/dev/sd*x*`；也可以备份加密容器内的文件系统，例如 `/dev/mapper/*dm_name*`；亦可备份里面的文件，例如使用 rsync。 以下部分列出了每个选项的优点和缺点。

磁盘块设备备份

磁盘块设备备份会：

- 按原样加密，具有与工作副本（working copy）相同的安全级别
- 包含你的 LUKS 标头
- 总是和磁盘块设备一样大
- 难以使用高级备份策略，例如增量备份、压缩或重复数据删除
- 易于恢复到新磁盘因其亦恢复加密容器

文件系统或文件备份

文件系统或文件备份，它：

- 不按原样加密
- 应在通过网络传输之前或保存到磁盘时进行加密。需做额外的工作
- 未必具有与工作副本相同的安全级别
- 不包含你的LUKS标头
- 仅与文件系统上的已用空间一样大，比如看看partclone
- 可使用高级备份策略，例如增量备份、压缩或重复数据删除
- 需要手动将加密容器恢复到新磁盘，例如通过恢复 LUKS 标头的备份

LUKS 标头备份

如果使用 LUKS，则可以对 LUKS 标头进行备份：定期检查与同步这些备份是有意义的，尤其是在密码已被撤销的情况下。

但是，如果你有数据备份，并且想要恢复它，你可以使用 cryptsetup 从头开始重新创建 LUKS 加密分区，然后恢复数据，因此备份 LUKS 标头不如备份数据重要。

2.9 Dm-crypt/特殊应用

难度 高级 | **篇幅** 49.9 KB | **所属** 静态数据加密

本节将讲到

- 保护未加密的boot分区
- 使用GPG、LUKS、OpenSSL加密的密钥文件
- 远程解锁分区
- 设置用于重启的一次性密钥
- 固态硬盘的Discard/TRIM支持

保护未加密的boot分区

即使在加密了根分区的情况下，磁盘上仍存在两处未加密的区域：/boot和主引导记录（MBR）。通常情况下，引导加载程序需要读取/boot，BIOS需要读取主引导记录，若两者被加密，则无法解密dm-crypt容器，进而完成启动过程。不过，GRUB支持解密经过加密的/boot，详见dm-crypt/加密整个系统#加密的boot分区(GRUB))。

本节介绍了可提高启动过程安全性的方法。

警告：虽然对/boot分区和主引导记录进行额外保护可避免许多针对启动过程的攻击方法。但系统仍可能面临其它威胁，例如BIOS/UEFI/固件被篡改，键盘记录器，冷启动攻击等。这些攻击方式不在本文的讨论范围中。有关全盘加密的系统信任问题，参见。

注意：对于UEFI启动的系统，在配置得当时（例如，使用统一内核映像），只有EFI 系统分区是未加密的。配合安全启动，可确保启动链的安全性。该方式的安全性与本节中介绍的其它方法相同，且配置简单。详见dm-crypt/加密整个系统#使用TPM2和安全启动在单一分区上配置。

从可移动设备引导

使用单独的引导设备启动系统非常直接，并能极大提高系统在面临特定类型攻击时的安全性。在加密了根分区的系统中，易受到攻击的部分为：

- 主引导记录
- /boot分区

要成功启动系统，上述部分必须保持未加密状态。要避免上述未加密区域被篡改，建议将它们存储到可移动介质（例如，USB设备）中，并从可移动介质中引导，而不是直接从硬盘引导。只要保证相应的可移动介质的安全，就可确保相关区域不被篡改，提高解密系统时的安全性。

如下内容假定已完成系统配置，并使用单独boot分区，挂载到/boot。若未还未完成系统配置，参见Dm-crypt/系统配置#内核参数，注意在配置过程中，用可移动设备替换硬盘。

注意：必须确保系统支持从选择的可移动介质（USB设备、移动硬盘、SD卡等）启动。

准备可移动设备（/dev/sdx）：

```
# gdisk /dev/sdx # 按需格式化。也可以使用cgdisk、fdisk、cfdisk、gparted等分区工具
# mkfs.ext2 /dev/sdx1 # BIOS启动的系统
# mkfs.fat -F 32 /dev/sdx1 # UEFI启动的系统
# mount /dev/sdx1 /mnt
```

将已有/boot分区内容复制到可移动设备中：

```
# cp -ai /boot/* /mnt/
```

将可移动设备挂载到/boot，并更新fstab：

```
# umount /boot
# umount /mnt
# mount /dev/sdx1 /boot
# genfstab -p -U / > /etc/fstab
```

更新GRUB配置。grub-mkconfig应当能自动检测新分区的UUID。但若配置了自定义启动选项，可能需要手动更新相应条目。

```
# grub-mkconfig -o /boot/grub/grub.cfg
# grub-install /dev/sdx # 安装到可移动设备，而不是硬盘中。用于BIOS启动的系统
# grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=grub # 用于UEFI启动的系统
```

重启以测试新的配置。注意在BIOS/UEFI中选择正确的启动设备。若系统启动失败，仍可以从硬盘启动以修复存在的问题。

chkboot

警告：chkboot可**发现并提示**/boot遭到篡改（**tamper-evident**），但**不能防止**篡改（**tamper-proof**）。chkboot脚本运行前，输入的密码可能已经被篡改过的引导加载程序、 内核、initrd读取。因此，当chkboot完整性检查失败时，数据安全性已经无法保证。

根据ct-magazine中一篇文章（Issue 3/12, page 146, 01.16.2012, ）的内容，下述脚本将检查/boot中文件的SHA-1散列值是否改变、/boot分区的inode、占用的磁盘块信息，以及主引导记录。该脚本不能完全阻止特定类型的攻击，但极大提升了攻击难度。脚本自身的配置文件不存储在未加密的/boot中。因此，对于攻击者，在系统锁定/关机的情况下，难以发现系统将在启动时检查/boot分区的内容。然而，对于有备而来的攻击者，可修改固件以在系统内核之上执行代码，劫持对boot分区的访问，呈现出文件未被篡改的假象。通常来讲，在固件以上的层级的安全措施都不能确保启动链可信，也不能确保发现并提示篡改迹象。

下述脚本可在此处获取（作者：Juergen Schmidt, ju at heisec.de；许可：GPLv2）。此外，也可通过chkbootAUR安装。建议使用AUR包，其提供了额外的辅助脚本。

Juergen Schmidt的原始脚本

/usr/local/bin/chkboot_user.sh需要在登录后立即执行，故需要加入自动启动中。（例如，对于KDE：系统设置 -> 系统 -> 自动启动；对于GNOME 3，使用gnome-session-properties）。

由于在Arch Linux中，/boot的内容经常改变（例如，更新内核），建议使用如下脚本进行系统更新，以在系统更新时同时更新chkboot设置：

```
#!/bin/sh
#
# 注意：将<user> 替换为你的用户名，并使用sudo执行该脚本，以确保pacman、chkboot有所需的权限
#
echo "Pacman update  Quickcheck before updating" &
sudo -u <user> /usr/local/bin/chkboot_user.sh
/usr/local/bin/chkboot.sh
sudo -u <user> /usr/local/bin/chkboot_user.sh
```

```
echo "Pacman update  Syncing repos for pacman"
pacman -Syu
/usr/local/bin/chkboot.sh
sudo -u <user> /usr/local/bin/chkboot_user.sh
echo "Pacman update  All done, let us roll on ..."
```

AUR包

在安装对应AUR包后，启动`chkboot.service`。

可能需要在mkinitcpio钩子列表的最后添加`chkboot`，以便每次mkinitcpio重新生成initramfs时更新chkboot记录的哈希值。可通过修改`/etc/mkinitcpio.conf`，在`HOOKS`列表的最后添加`chkboot`实现。

AUR包中还附带了`chkboot-desktopalert`脚本，在检测到意料之外的`/boot`修改时以图形化窗口的形式弹出警告。可将该脚本加入桌面环境的启动项中。

mkinitcpio-chkcryptoboot

警告：本钩子不会加密GRUB的核心代码（MBR部分）、EFI stub。若攻击者通过修改引导加载程序，在运行时篡改内核或initramfs，本钩子不能保证安全性。

mkinitcpio-chkcryptobootAUR是一个mkinitcpio钩子，将在早期用户空间中检查完整性，并在发现系统可能被篡改的痕迹时建议用户不要输入密码。本方案的安全性来自加密的boot分区)和加密的根分区。并要求boot分区的密码和根分区的不同。加密的boot分区将由GRUB的`cryptodisk.mod`模块解密。这样，可保护initramfs和内核不受到离线攻击。并且，即使在受到篡改的机器上输入了`/boot`分区的密码，在chkcryptoboot钩子的提示下，也能确保根分区的安全性。（假设chkcryptoboot检测到了篡改，并且钩子本身没有在运行时被篡改）

本钩子需要>=2.00版本的grub包。要保证安全性，需要使用专用的、有独立密码的加密`/boot`分区。

安装

安装 mkinitcpio-chkcryptobootAUR。编辑`/etc/default/chkcryptoboot.conf`以进行配置。若要检测攻击者是否用自制的boot分区替换了原本的boot分区，设置`CMDLINE_NAME`、`CMDLINE_VALUE`为某一秘密值。例如，可以按照安装过程中显示的建议使用两个哈希值。此外，还需要在`/etc/default/grub`中修改内核参数，传递相应的`CMDLINE_NAME`、`CMDLINE_VALUE`。之后，修改`/etc/mkinitcpio.conf`的`HOOKS=`行，在`encrypt`**之前**插入`chkcryptoboot`。最后重新生成 initramfs。

技术概述

mkinitcpio-chkcryptobootAUR同时包含了安装、运行时mkinitcpio钩子。安装钩子将在每次重新生成initramfs时运行。对于UEFI启动的系统，将计算EFI stub（`$esp/EFI/grub_uefi/grubx64.efi`）的哈希值；对于BIOS启动的系统，将计算GRUB安装到的磁盘的首446字节的哈希值，并将哈希值保存到initramfs中（initramfs保存在加密的`/boot`分区中）。当系统启动时，在GRUB中输入`/boot`分区的密码后，运行时钩子将重新计算对应的哈希值，将计算出的值与保存的值比对，若匹配，才提示输入根分区密码解密根分区。若不匹配，将显示类似如下的错误：

```
CHKCRYPTOBOOT ALERT!
CHANGES HAVE BEEN DETECTED IN YOUR BOOT LOADER EFISTUB!
YOU ARE STRONGLY ADVISED NOT TO ENTER YOUR ROOT CONTAINER PASSWORD!
Please type uppercase yes to continue:
```

除了计算引导加载程序的哈希值之外，钩子还将在运行时以及启动过程完成后，根据`/etc/default/chkcryptoboot.conf`中的配置检查当前运行内核的内核参数，以确认GRUB配置文件、boot分区未被非法替换。

注意，对于BIOS启动的系统，本钩子计算并检查的是GRUB第一阶段引导加载程序（安装在引导设备的前446字节）的哈希值，并未检查第二阶段引导加载程序（`core.img`）。

AIDE

除了上述脚本外，也可以使用AIDE进行哈希值检查。AIDE的配置文件非常灵活，可进行高度自定义配置。

STARK

上述介绍的其它方法虽然可以满足大部分用户的需求，但并未解决所有与未加密的`/boot`分区有关的安全问题。为了构建经过完全验证的启动链，学术界提出了一种名为POTTS的实现，旨在实施 STARK 认证框架。

POTTS 概念验证（proof-of-concept）使用 Arch Linux 作为基础发行版，并实现了一个包含以下组件的系统启动链：

- POTTS - 用于展示一次性验证消息的启动菜单。
- TrustedGrub - GRUB Legacy实现，通过TPM芯片的PCR寄存器验证内核和initramfs。
- TRESOR - 内核补丁，实现了AES加密，并在启动过程中，将主密钥保存在CPU寄存器中，而非内存中。

基于Arch Linux （2013-01及之后版本的ISO镜像）的安装配置方法已作为研究的一部分发布。注意，在尝试之前，应当了解软件仓库中并没有相关实现工具，并且该方法的日常维护非常繁琐。

使用GPG、LUKS、OpenSSL加密的密钥文件

如下论坛帖子讨论了使用了双因素身份验证、GPG或OpenSSL加密的密钥文件，而不是明文密钥文件。拓展了System Encryption using LUKS with GPG encrypted keys中的讨论：

- GnuPG: 有关GPG加密的密钥文件的帖子该帖子介绍了通用指令。
- OpenSSL: 有关OpenSSL加密的密钥文件的帖子该帖子只介绍了`ssldec`钩子。
- OpenSSL: 有关OpenSSL加盐的bf-cbc加密的密钥文件的帖子该帖子介绍了`bfkf`钩子，安装过程，生成加密密钥文件的脚本。
- LUKS: 有关LUKS加密的密钥文件的帖子使用`lukskey`钩子。

注意：

- 以上方法只适用于只有两个主分区的情况：在配置加密时必要的boot分区以及主LVM分区。在LVM中，可以按需求任意创建逻辑卷（至少应当有根逻辑卷、swap逻辑卷、home逻辑卷）。只使用两个主分区的好处是只需要一个密钥文件，并且可以使用休眠功能（休眠到加密的swap分区）。若要使用休眠功能，`/etc/mkinitcpio.conf`中的钩子配置应类似：

```
HOOKS=( ... usb usbinput <etwo或者ssldec> encrypt（若使用openssl） lvm2 resume ... )
```

，并在内核参数中加入

```
resume=/dev/<VolumeGroupName>/<LVNameOfSwap>
```

。

- 若需要临时在某处保存解密后的密钥文件，不要保存到未加密的磁盘中。最好确保只保存到内存中（例如，`/dev/shm`）。
- 若要使用GPG加密的密钥文件，需要静态编译1.4版本的GnuPG。或者，使用gnupg1AUR并相应修改钩子实现。
- 对于上述第二个帖子（OpenSSL加密密钥文件），更新OpenSSL可能破坏自定义的`ssldec`钩子。

远程解锁分区

对于需要远程登录启动中的系统，在启动早期阶段远程提供解密LUKS加密的分区（例如，根分区及其它分区）的密码的情况，可以使用mkinitcpio钩子配置网络接口并启动SSH服务器。下文中介绍的某些包提供了mkinitcpio 编译钩子，以简化配置。下面将介绍了在本机控制台输入密码的基础上，增加远程解密的方法。

注意：

- 需要确保使用内核对网络设备的命名（例如，`eth0`），而不是udev的命名（例如，`enp1s0`）。
- 默认情况下，可预测网络接口名（Predictable Network Interface Names）在启动过程后期才激活并修改网络设备的命名。可使用dmesg检查有关网络的内核模块的日志以确定网络设备的原始命名（例如，`eth0`）。
- 可能需要将以太网卡或无线网卡所需的模块添加到MODULES列表中。

基于BusyBox的initramfs（使用mkinitcpio）构建

mkinitcpio-netconf包和/或mkinitcpio-pppAUR为基于BusyBox的initramfs提供了网络支持。对于SSH服务器，则可使用mkinitcpio-dropbear包或mkinitcpio-tinyssh包。注意，上述钩子不会安装shell，故可能按需安装mkinitcpio-utils包。对于上述包的任何组合方式，下述操作步骤都适用，些许不同之处将单独指出。

1. 若没有已有的SSH密钥对，在客户端系统上生成密钥对。生成的密钥对将用于登录远程主机。

> **注意：** > > - `tinyssh`只支持不设置密码的Ed25519、ECDSA密钥。若使用mkinitcpio-tinyssh包，只能选择这两种密钥类型之一。 > - 0.0.3-5版本的`mkinitcpio-dropbear`与现有的移除了dss的dropbear实现不兼容。详情及修复方法参见Github。

1. 根据选择的SSH服务器，将SSH公钥插入到远程主机的`/etc/dropbear/root_key`/`/etc/tinyssh/root_key`文件中（对于刚刚生成的密钥对，SSH公钥存储在**.pub**结尾的文件中，公钥一般被复制到远程主机以实现无密码登录）。

> **提示：**可依照此方法添加其它所需的SSH公钥。也可以直接复制远程主机的`~/.ssh/authorized_keys`文件，但需要检查文件内容以仅包含需要执行远程解锁操作的公钥。添加公钥后，需要使用`mkinitcpio`重新生成initrd。另见OpenSSH#保护

1. 修改`/etc/mkinitcpio.conf`的"HOOKS"列表，在`filesystems`文件系统钩子前添加`<netconf 和/或 ppp> <dropbear 或 tinyssh> encryptssh`三个钩子（`encryptssh`钩子代替了`encrypt`钩子）。并重新生成 initramfs。

> **注意：不需要**mkinitcpio-nfs-utils包提供的`net`钩子。

1. 配置所需的`cryptdevice=`内核参数，并在引导加载程序配置中添加恰当的`ip=`内核参数。例如，若要使用DHCP分配IP地址，可设置

```
ip=dhcp
```

，这一般要求DHCP服务器始终为需要远程解密的主机分配相同的IP地址，否则进行SSH连接较为繁琐。也可以使用静态IP地址：

```
ip=192.168.1.1::::eth0:none
```

。此外，还可指定子网掩码和网关：

```
ip=192.168.1.1::192.168.1.254:255.255.255.0::eth0:none
```

。

> **注意**：若使用0.0.4及更新版本的mkinitcpio-netconf包，还可通过在`ip=`中设置多个参数，分别配置多个网络接口。但是，不能同时设置“全局配置”和针对某一网络接口的配置（例如，不能与`ip=dhcp`混用）。必须显式地指定网络接口： > > `` > `ip=ip=192.168.1.1::::eth0:none:ip=172.16.1.1::::eth1:none` > ``

若使用DHCP，可能需要添加`netconf_timeout=`内核参数，避免在DHCP服务不可用时，netconf无限期地尝试通过DHCP获取IP地址。`ip=`参数的详细介绍可参见Mkinitcpio#使用_net。设置完成后，更新引导加载程序配置。

1. 最后，重启远程主机并尝试使用SSH连接, **显式地指定使用root用户连接**（即使远程主机的系统配置中关闭了root账户，也可在initrd中使用，以解锁远程主机）。若远程主机在启动后使用openssh包提供远程SSH连接，并且使用mkinitcpio-dropbear包提供initramfs中的SSH服务端支持，则在连接到initramfs中的SSH服务端时，不会显示SSH主机密钥变更的警告，因为mkinitcpio-dropbear包将使用和OpenSSH相同的SSH密钥（注意，dropbear不支持Ed25519密钥，若openssh使用了Ed25519密钥，连接时仍会显示密钥不匹配警告）。对于mkinitcpio-tinyssh包，其附带了`tinyssh-convert`脚本，以使用与openssh包相同的密钥（当前仅支持使用Ed25519密钥）。有时，可能需要手动使用`tinyssh-convert`将密钥复制到`/etc/tinyssh/sshkeydir`。注意，无论使用何种方式复制ssh密钥，都需要确保密钥已由ssh守护进程生成，即，至少通过systemd单元启动过一次OpenSSH。配置完成后，重启远程主机并通过SSH连接，应当能看到输入密码以解锁设备的提示符；输入密码并等待系统启动完成后，可以使用普通用户进行正常的SSH连接。

使用Wi-Fi

`netconf`钩子一般用于以太网连接。若远程主机只连接无线网络，并且要通过Wi-Fi进行远程连接以解锁设备，需要在`netconf`钩子前运行相关配置无线网络的钩子，可以使用预定义的钩子，也可以创建自定义钩子。

预定义钩子

可以安装预定义钩子：

1. 安装mkinitcpio-wifiAUR。
2. 根据网络属性创建*wpa_supplicant*配置，以设置Wi-Fi连接：

```
    wpa_passphrase "ESSID" "passphrase" > /etc/wpa_supplicant/initcpio.conf
```

1. 修改`/etc/mkinitcpio.conf`，在`netconf`钩子前添加`wifi`钩子。应当能自动检测到与Wi-Fi相关的模块。若没有自动检测到，手动将需要的模块加入`MODULES`中。
2. 添加`ip=:::::wlan0:dhcp`内核参数。
3. 重新生成 initramfs。
4. 更新引导加载程序配置。

自定义钩子

下例展示了使用USB Wi-Fi适配器，连接到WPA2-PSK保护的Wi-Fi网络的方法。对于其它配置（例如，连接到WEP网络、使用其它initramfs生成工具），可能需要相应调整某些步骤。

1. 修改`/etc/mkinitcpio.conf`：
2. 添加Wi-Fi适配器所需的内核模块。
3. 包含`wpa_passphrase`、`wpa_supplicant`二进制文件。
4. 在`net`钩子前添加`wifi`钩子（该钩子即是需要创建的自定义钩子，也可以使用其它名称。

```
        MODULES=(module)BINARIES=(wpa_passphrase wpa_supplicant)HOOKS=(base udev
autodetect ... wifi net ... dropbear encryptssh ...)
```

1. 在`/etc/initcpio/hooks/wifi`中创建`wifi`钩子：

```
    run_hook (){     # Sleep a couple of seconds so wlan0 is setup by kernel sleep 5
# Set wlan0 to up     ip link set wlan0 up # Associate with Wi-Fi network     # 1. Save
temp config file     wpa_passphrase "network ESSID" "pass phrase" > /tmp/wifi       #
2. Associate  wpa_supplicant -B -D nl80211,wext -i wlan0 -c /tmp/wifi # Sleep a couple
of seconds so that wpa_supplicant finishes connecting   sleep 5       # wlan0 should
now be connected and ready to be assigned an ip by the net hook}run_cleanuphook (){ #
Kill wpa_supplicant running in the background    killall wpa_supplicant      # Set
wlan0 link down      ip link set wlan0 down      # wlan0 should now be fully
disconnected from the Wi-Fi network}
```

1. 在`/etc/initcpio/install/wifi`中创建钩子安装文件：

```
    build (){ add_runscript}help (){cat<<HELPEOF Enables Wi-Fi on boot, for dropbear
ssh unlocking of disk.HELPEOF}
```

1. 添加`ip=:::::wlan0:dhcp`内核参数。注意移除`ip=:::::eth0:dhcp`，以避免冲突。
2. （可选）可额外添加使用`ip=:::::eth0:dhcp`内核参数以进行以太网连接的启动加载项。

3. 重新生成 initramfs。
4. 更新引导加载程序配置。

注意检查系统的Wi-Fi配置，避免系统启动完成后无法远程连接。

若在initramfs中无法连接到Wi-Fi网络，可尝试增加脚本中的sleep时间。

基于systemd的initramfs（使用mkinitcpio）构建

mkinitcpio-systemd-extrasAUR为基于systemd的initramfs提供了一整套用于网络配置、远程SSH连接的构建钩子。对于远程会话，可进行配置以使用busybox的dash完全访问initramfs环境，也可只显示密码输入提示符。

最小配置如下：

```
/etc/mkinitcpio.conf
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole sd-network
block sd-tinyssh sd-encrypt filesystems fsck)
SD_TINYSSH_COMMAND="systemd-tty-ask-password-agent --query --watch"
```

对于上述配置，当使用`mkinitcpio`构建initramfs时，将复制systemd-networkd配置文件，并尝试从已有的TinySSH、OpenSSH配置中复制/转换SSH密钥。系统需要安装tinyssh包（但不需要启用）。若系统不使用systemd-networkd，需要额外配置参数，详见mkinitcpio-systemd-extrasAUR的文档。

注意：

- 在默认情况下，**可预测网络接口名**（例如，`enp2s0`）在initramfs阶段不可用。故直接复制的systemd-networkd配置文件很可能无法正常工作。解决方法可参见mkinitcpio-systemd-extras wiki。
- 还可使用mkinitcpio-systemd-tool包实现在启动早期远程连接，并解密LUKS卷。该方式与上述介绍完全不同，需要额外添加`systemd-tool`钩子，并且在相应systemd单元的`[X-SystemdTool]`节中进行配置。有关该方式实现以及配置的文档非常有限。

基于systemd的initramfs（使用dracut）构建

若使用dracut而不是mkinitcpio构建initramfs，可能需要使用dracut-sshd。

设置用于重启的一次性密钥

还可以通过设置临时密钥文件，以在无需手动终端交互输入密码的情况下实现远程设备的重启。密钥文件需要放置在能在启动时被内核访问的位置，并设置cryptkey启动参数，同时，还需要通过"cryptsetup luksAddKey"命令添加密钥文件。

使用passless-bootAUR可方便地完成上述操作。若想要自行实现，也可参照该工具的的README。注意，在使用前，请阅读Security considerations中的讨论。

固态硬盘的Discard/TRIM支持

固态硬盘用户应当注意，在默认情况下，除非指定`discard`选项，否则dm-crypt的device mapper不会启用TRIM命令支持。

由于可能的安全问题，device mapper维护者表示永远不会在dm-crypt设备上默认启用TRIM支持。 在启用TRIM的设备上，通过释放的磁盘块信息，有可能推断出加密设备上层使用的文件系统。cryptsetup开发者在该博客中介绍了启用TRIM可能导致的问题。还需注意，相关风险可能叠加。例如，若设备仍使用之前的加密方式（cryptsetup < 1.6.0）加密（`--cipher aes-cbc-essiv`），相比现在的默认选项，释放的磁盘块信息可能泄露更多配置信息。

依据配置方式，可分情况详细讨论：

- 若设备使用默认的dm-crypt LUKS模式加密：
- 在默认情况下，LUKS头存储在设备头部，启用TRIM支持有利于保护对LUKS头的修改。例如，在撤销了一个可能泄露的LUKS密码后，在未启用TRIM支持的情况下，仍可能从设备上读取旧的LUKS头，直到其被另一操作覆盖。若设备在此情况下失窃，则理论上，攻击者可能能够使用某种方法找到旧的LUKS头，并使用泄露的LUKS密码获得主密钥解密设备。详见cryptsetup FAQ, section 5.19 What about SSDs, Flash and Hybrid Drives?、Full disk encryption on an ssd。
- 若认为启用TRIM可能造成的信息泄露比上述LUKS头有关问题更加严重，则应当保持TRIM关闭的状态。另见安全地擦除磁盘#闪存。
- 若设备使用dm-crypt plain模式加密，或者单独存储LUKS头：
- 若需要可抵赖性，则根据本节开头提到的内容，**永远不应**启用TRIM支持。否则，TRIM可能造成的信息泄露会使得plain模式加密没有意义。
- 若无需可抵赖性，则在可接受本节开头提到的安全风险的情况下，可以为了性能启用TRIM支持。

警告：在启用TRIM前，需确保相应设备完全支持TRIM指令，否则可能造成数据丢失。详见固态硬盘#TRIM。

除了启用dm-crypt的discard支持外，还应定期执行fstrim(8)；或者，在`/etc/fstab`中指定`discard`挂载选项。详见固态硬盘#TRIM。

LUKS2

对于LUKS2加密的设备，可通过打开设备时使用`--allow-discards --persistent`挂载选项启用TRIM支持。使用该命令后，`allow-discards`将写入LUKS2头中，之后打开设备时无需再手动指定。

注意：

- 通过`cryptsetup --persistent`设置新的持久性标志时，将替代所有已设置的持久性标志，而不是在原有基础上新增一个标志。故若还设置了其它标志，需要一同指定。
- 若使用不经过dm-crypt的OPAL加密（详见cryptsetup-luksFormat(8)的`--hw-opal-only`选项），则文件系统和底层磁盘间没有dm-crypt层，discard请求将直接传递到底层磁盘，无需手动启用discard支持。

```
# cryptsetup --allow-discards --persistent open /dev/sdaX root
```

若设备已经打开，上述`open`操作将会报错。此时，应当使用cryptsetup-refresh(8)命令：

```
# cryptsetup --allow-discards --persistent refresh root
```

可通过`cryptsetup luksDump`输出的LUKS2头信息确保标志已经持久性设置：

```
# cryptsetup luksDump /dev/sdaX | grep Flags
Flags:          allow-discards
```

LUKS1及plain模式

对于LUKS1或者plain模式加密的设备，需要在每次打开设备时手动指定选项启用TRIM支持。

要在启动时启用TRIM支持，设置如下内核参数。

若使用encrypt钩子：

```
cryptdevice=/dev/sdaX:root:allow-discards
```

若使用sd-encrypt钩子（基于systemd的initramfs）：

```
rd.luks.options=discard
```

注意：对于在initramfs镜像`/etc/crypttab`（对应`/etc/crypttab.initramfs`）中挂载的文件系统，设置`rd.luks.options=discard`内核选项没有作用。必须在`/etc/crypttab.initramfs`中设置`discard`选项。

对于通过`/etc/crypttab`解密的设备，指定`discard`选项，例如：

```
/etc/crypttab
luks-123abcdef-etc UUID=123abcdef-etc none discard
```

若手动使用命令解锁设备，使用`--allow-discards`。

例如，可使用`--allow-discards`手动打开设备，并手动执行fstrim命令：

```
# cryptsetup --allow-discards open /dev/sdaX root
```

总是可以通过`dmsetup table`的输出判断设备是否通过discard选项打开：

```
# dmsetup table
```

```
luks-123abcdef-etc: 0 1234567 crypt aes-xts-plain64 000etc000 0 8:2 4096 1
allow_discards
```

关闭工作队列以提高SSD性能

固态硬盘用户应当注意，在默认情况下，device mapper并未启用关闭内部读取、写入工作队列的功能。即，块设备将不使用`no_read_workqueue`、`no_write_workqueue`选项挂载。

在Cloudflare内部进行的关于总体磁盘加密性能的研究（Speeding up Linux disk encryption）中，提出了`no_read_workqueue`、`no_write_workqueue`标志。研究显示，dm-crypt内部的读取和写入队列降低了SSD的性能。尽管磁盘操作队列对于机械硬盘适用，但对于SSD，禁用操作队列，直接同步写入数据可使吞吐量翻倍，并将IO操作等待延迟降低一半。相关补丁已经被上游接受，并在5.9及更新版本的linux包中可用。

提示：linux-zen包 默认关闭了dm-crypt工作队列。

对于使用crypttab解密的LUKS设备，要关闭工作队列，按需指定相应的`no-read-workqueue`、`no-write-workqueue`选项。例如：

```
/etc/crypttab
luks-123abcdef-etc UUID=123abcdef-etc none no-read-workqueue
```

要同时关闭读取和写入工作队列，同时指定两个标志：

```
/etc/crypttab
luks-123abcdef-etc UUID=123abcdef-etc none no-read-workqueue,no-write-workqueue
```

对于LUKS2加密的设备，可以使用`--persistent`选项，将`--perf-no_read_workqueue`、`--perf-no_write_workqueue`设置为打开设备时的默认值。例如：

```
# cryptsetup --perf-no_read_workqueue --perf-no_write_workqueue --persistent
open /dev/sdaX root
```

若设备已经打开，上述`open`操作将会报错。此时，应当使用cryptsetup-refresh(8)命令。例如：

```
# cryptsetup --perf-no_read_workqueue --perf-no_write_workqueue --persistent refresh
root
```

可通过`cryptsetup luksDump`输出的LUKS2头信息确保标志已经持久性设置：

```
# cryptsetup luksDump /dev/sdaX | grep Flags
Flags:          no-read-workqueue
```

总是可以通过`dmsetup table`的输出判断为设备指定的选项：

```
# dmsetup table
luks-123abcdef-etc: 0 1234567 crypt aes-xts-plain64 000etc000 0 8:2 4096 1
no_read_workqueue
```

注意：通过`cryptsetup --persistent`设置新的持久性标志时，将替代所有已设置的持久性标志，而不是在原有基础上新增一个标志。故若还设置了其它标志，需要一同指定。

例如，通过cryptsetup同时指定no_read_workqueue、no_write_workqueue：

```
# cryptsetup --perf-no_read_workqueue --perf-no_write_workqueue --persistent refresh root
```

可以通过dmsetup table的输出判断为设备指定的选项：

```
# cryptsetup luksDump /dev/sdaX | grep Flags
Flags:          no-read-workqueue no-write-workqueue
```

使用encrypt钩子解密多个设备

注意：本节仅关注在早期用户空间（initramfs阶段）解密多个设备。对于后期用户空间（在切换root后）解密的配置，参见Dm-crypt/系统配置#在后期用户空间中解密。

提示：sd-encrypt钩子支持解密多个设备。可通过内核命令行或/etc/crypttab.initramfs指定需要解密的设备。详见Dm-crypt/系统配置#使用systemd-cryptsetup-generator。

encrypt钩子仅支持**单个**cryptdevice=条目（archlinux/mkinitcpio/mkinitcpio#231）。在有多硬盘的系统中，由于dm-crypt自身不能跨越物理设备，需要配置多个加密物理分区，此时encrypt将造成限制。例如，对于“在LUKS上配置LVM”方案，整个LVM都存在于LUKS映射器之上。对于仅有单个硬盘的系统，只需解密一个设备，不会出现问题。但是，若之后想通过添加设备，增加LVM的大小呢？在不修改encrypt钩子的情况下，无法实现这一目的。

之后的小节介绍了克服这一限制的替代方案。第一小节介绍了如何将“在LVM上配置LUKS”方案拓展到新增的硬盘上。第二小节介绍了，在不使用LVM的配置中，如何修改encrypt钩子以解密多个磁盘。

将LVM拓展到多个磁盘上

管理多个磁盘，并以此提供分区灵活性，是LVM的特色功能。若将LVM作为dm-crypt的第一个映射器（first mapper），则LVM也可与dm-crypt共同使用（“在LVM上配置LUKS”）。在此配置中，加密设备在逻辑卷中创建（可能分别对每个加密卷使用不同的密钥）。下面将介绍如何将本配置拓展到另一磁盘。

警告：请注意，虽然改变文件系统大小似乎是常规操作，操作过程中仍**可能出现问题**，同时，下面介绍的方式也可能不适用于某些特定配置。通常来讲，扩容文件系统可能遇到的问题比收缩文件系统小，尤其是在类似本例的多层映射配置中。

添加新设备

首先，可能需要按照dm-crypt/准备磁盘中的步骤准备磁盘。然后，创建用于拓展LVM的分区（例如，将设备所有空间分配到/dev/sdY1中，并设置分区类型为8E00（Linux LVM））。之后，将新磁盘/分区添加到已有的LVM卷组中，例如：

```
# pvcreate /dev/sdY1
```

```
# vgextend MyStorage /dev/sdY1
```

拓展逻辑卷

接下来，将最终分配新的磁盘空间到逻辑卷中。注意需要首先卸载要扩容的逻辑卷。如果需要扩容cryptdevice设定的根分区，需要在Arch安装ISO中执行下述操作。

在本例中，假定需要拓展用于/home（逻辑卷名称为homevol）的逻辑卷：

```
# umount /home
# fsck /dev/mapper/home
# cryptsetup close /dev/mapper/home
# lvextend -l +100%FREE MyStorage/homevol
```

在拓展逻辑卷后，拓展LUKS容器：

```
# cryptsetup open /dev/MyStorage/homevol home
# umount /home      # 在/home可能被自动挂载的情况下，作为安全措施
# cryptsetup --verbose resize home
```

最后，修改文件系统大小：

```
# e2fsck -f /dev/mapper/home
# resize2fs /dev/mapper/home
```

若需要，可重新挂载/home，其容量已被拓展：

```
# mount /dev/mapper/home /home
```

注意，cryptsetup resize操作不会影响密钥。

修改encrypt钩子以用于多个分区

注意，sd-encrypt原生支持解密多个分区。此外，若通过该方式解密的多个分区密码相同，sd-encrypt还将自动尝试输入的密码，而无需多次输入。因此，使用sd-encrypt可能比下面介绍的方式更加方便。

跨越多个分区的根文件系统

可以通过修改encrypt钩子以在启动时解密跨越多个分区的根文件系统。一种方式如下：

```
# cp /usr/lib/initcpio/install/encrypt /etc/initcpio/install/encrypt2
# cp /usr/lib/initcpio/hooks/encrypt  /etc/initcpio/hooks/encrypt2
# sed -i "s/cryptdevice/cryptdevice2/" /etc/initcpio/hooks/encrypt2
# sed -i "s/cryptkey/cryptkey2/" /etc/initcpio/hooks/encrypt2
```

添加cryptdevice2=启动选项（可能还需添加cryptkey2=选项），在mkinitcpio.conf中添加encrypt2钩子，并重新生成initramfs。详见Dm-crypt/系统配置。

非根分区

可能还有使用encrypt钩子解密非根分区的需求。Arch并不原生支持这一功能，但可通过修改/lib/initcpio/hooks/encrypt中的cryptdev、cryptname值方便地实现。（cryptdev对应要解密的分区，例如/dev/sd*；cryptname对应解密后的映射名称）。

使用encrypt钩子的优势在于其自动化了非根分区的解密。虽然可通过/etc/crypttab文件达到相同目的，但若使用外部密钥文件（即，密钥文件不存储在系统硬盘中，而在可移动设备上），需要确保USB、火线（FireWire）等外部设备在尝试挂载加密分区前被挂载，这至少需要修改/etc/fstab中的挂载顺序，相关配置较为麻烦。

当然，若cryptsetup包包更新，则需要重新修改上述脚本。此外，与/etc/crypttab不同，默认情况下仅能通过encrypt钩子解密一个分区，但通过进一步修改，应当能实现解密多个分区的功能。

在加密系统时使用分离的LUKS头

本例使用了与dm-crypt/加密整个系统#Plain_dm-crypt类似的设置，应当先阅读其中的内容。

若使用分离的LUKS头，则加密的块设备自身将只含有加密数据，而不含LUKS头信息。若攻击者不知道分离的LUKS头的存在，则可实现可抵赖加密。这与使用plain dm-crypt类似，但优势在于仍可使用多密码、密钥派生等LUKS高级功能。此外，使用分离的LUKS头还提供了某种程度上的双重因素认证，并且相比使用GPG、OpenSSL加密密钥文件更加简单，同时可以使用内建的密码提示符进行多次尝试。详见静态数据加密#加密的元数据。

在继续之前，参见可用的加密选项。使用如下命令创建加密的系统分区和LUKS头文件：

```
# dd if=/dev/zero of=header.img bs=16M count=1
# cryptsetup luksFormat --offset 32768 --header header.img /dev/sdX
```

提示：--offset选项指定了加密数据在设备上的起始位置。在设备开头预留一定空间可便于之后将分离的LUKS头重新附加到设备上。该选项的单位是512字节的扇区，详见cryptsetup-luksFormat(8)。

打开加密容器：

```
# cryptsetup open --header header.img /dev/sdX enc
```

接下来，按照“在LUKS上配置LVM”中的步骤进行。之后，在可移动设备上配置boot分区（若不将boot分区放置在可移动设备上，使用分离的LUKS头没有意义）。

将header.img移动到boot分区：

```
# mv header.img /mnt/boot
```

按照安装指南中的步骤继续安装，直到配置mkinitcpio步骤前（此时应当已经arch-chroot到加密的系统中）。

提示：由于整个系统硬盘都只含有看似随机的加密数据，没有分区表、分区，因而没有UUID、LABEL。但仍可通过id、路径进行持久化命名。例如，可以使用硬盘id（可在/dev/disk/by-id/中查看）。

有两种方法使得initramfs支持分离的LUKS头。

使用systemd钩子

设置如下内核参数：

```
rd.luks.name=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=enc rd.luks.options=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=header=/header.img:UUID=ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ
rd.luks.data=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX=/dev/disk/by-id/your-disk_id
```

- 将*XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX*替换为LUKS超级块的UUID。该UUID可通过`cryptsetup luksDump header.img`或者`blkid -s UUID -o value header.img`获得。
- 将*UUID=ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ*修改为存放分离的LUKS头文件的块设备的UUID。

除了指定`rd.luks`内核参数以外，还可通过`/etc/crypttab.initramfs`文件指定：

```
/etc/crypttab.initramfs
enc    /dev/disk/by-id/your-disk_id       none    header=/header.img:UUID=ZZZZZZZZ-ZZZZ-ZZZZ-ZZZZ-ZZZZZZZZZZZZ
```

接下来，修改`/etc/mkinitcpio.conf`以使用systemd，并添加用于读取分离的LUKS头文件存储的文件系统的模块。例如，若存储在FAT卷中：

```
/etc/mkinitcpio.conf
...
MODULES=(vfat)
...
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt lvm2 filesystems fsck)
...
```

重新生成 initramfs。

注意：

- 若使用了`/etc/crypttab.initramfs`进行配置，则无需添加上述内核参数。`/etc/crypttab.initramfs`将被复制到initramfs中的`/etc/crypttab`。
- 避免同时使用`rd.luks`内核参数和`/etc/crypttab.initramfs`文件，否则可能导致冲突。详见Dm-crypt/系统配置#使用systemd-cryptsetup-generator。

修改encrypt钩子

下文介绍了如何修改`encrypt`钩子，以便使用分离的LUKS头。

目前，需要修改`encrypt`钩子以使得`cryptsetup`使用分离的LUKS头（archlinux/mkinitcpio/mkinitcpio#234；本方法的基本思路和框架来自该BBS帖子）。将修改钩子的副本，以免mkinitcpio的更新覆盖所做的改动。

```
# cp /usr/lib/initcpio/hooks/encrypt /etc/initcpio/hooks/encrypt2
# cp /usr/lib/initcpio/install/encrypt /etc/initcpio/install/encrypt2
/etc/initcpio/hooks/encrypt2 (around line 52)
warn_deprecated() {
```

```
    echo "The syntax 'root=${root}' where '${root}' is an encrypted volume is
deprecated"
    echo "Use 'cryptdevice=${root}:root root=/dev/mapper/root' instead."
}

local headerFlag=false
for cryptopt in ${cryptoptions//,/ }; do
    case ${cryptopt} in
        allow-discards)
            cryptargs="${cryptargs} --allow-discards"
            ;;
        header)
            cryptargs="${cryptargs} --header /boot/header.img"
            headerFlag=true
            ;;
        *)
            echo "Encryption option '${cryptopt}' not known, ignoring." >&2
            ;;
    esac
done

if resolved=$(resolve_device "${cryptdev}" ${rootdelay}); then
    if $headerFlag || cryptsetup isLuks ${resolved} >/dev/null 2>&1; then
        [ ${DEPRECATED_CRYPT} -eq 1 ] && warn_deprecated
        dopassphrase=1
```

之后，修改mkinitcpio.conf，添加`encrypt2`、`lvm2`钩子。同时还需在`FILES`中添加`header.img`；在`MODULES`中添加`loop`。并按需添加其它所需内容：

```
/etc/mkinitcpio.conf
...
MODULES=(loop)
...
FILES=(/boot/header.img)
...
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block
encrypt2 lvm2 filesystems fsck)
...
```

上述配置通过将LUKS头文件复制到initramfs中，使得LUKS头在启动过程中可用，以便解密系统，而无需再挂载另一个USB设备以获取LUKS头文件，降低了配置的复杂性。配置完成后，重新生成initramfs。

接下来，配置引导加载程序，指定`cryptdevice=`，并传递新增的`header`选项：

```
cryptdevice=/dev/disk/by-id/your-disk_id:enc:header
```

对于在USB存储介质上配置的`/boot`分区，可参见dm-crypt/加密整个系统#安装完成后的操作。

2.10 Dm-crypt/设备加密

难度 高级 | **篇幅** 54.4 KB | **所属** 静态数据加密

本节将讲到

- 准备
- Cryptsetup 用法
- 加密选项
- 使用 cryptsetup 加密设备
- 专门针对LUKS的Cryptsetup操作

本节介绍如何从命令行手动使用 **dm-crypt** 来加密系统。

准备

在使用 cryptsetup包 之前，请始终确保已加载`dm_crypt`内核模块（kernel module）。

Cryptsetup 用法

cryptsetup(8) 是与 **dm-crypt** 交互的命令行工具，用于创建、访问和管理加密设备。该工具后来被扩展为支持依赖 Linux 内核设备映射器和加密模块（Linux kernel **d**evice-**m**apper and the **crypt**ographic modules）的不同加密类型。最显著的扩展是 Linux 统一密钥设置 (LUKS) 扩展，它将 dm-crypt 所需的所有设置信息存储在磁盘本身上，并抽象分区和密钥管理以提高易用性。通过设备映射器访问的设备称为块设备。有关详细信息，请参阅静态数据加密#块设备加密（Data-at-rest encryption#Block device encryption）。

该工具的使用如下：

```
# cryptsetup action options device name
```

它具有选项和加密模式的编译默认值，如果在命令行上未指定其他值，则将使用这些默认值。看看

```
$ cryptsetup --help
```

它按顺序列出了加密模式的选项、操作和默认参数。可以在手册页上找到完整的选项列表。由于需要或可选不同的参数，根据加密模式和操作，以下部分将进一步指出差异。块设备加密速度很快，但速度也很重要。由于在设置后更改块设备的加密密码很困难，因此提前检查各个参数的 dm-crypt 性能很重要：

```
$ cryptsetup benchmark
```

可以指导在安装之前决定算法和密钥大小。如果某些 AES 密码以相当高的吞吐量表现出色，那么这些密码可能是 CPU中具有硬件支持的密码。

提示：您可能想在学习时练习在虚拟机（virtual machine）中加密虚拟硬盘。

Cryptsetup 密码和密钥

加密的块设备受密钥保护。密钥可能是以下两者之一：

- 密码: 参见 Security#Passwords。
- 密钥文件, 参见 #密钥文件。

两种密钥类型都有默认的最大大小：密码最多可包含 512 个字符，密钥文件最多可包含 8192KiB。

LUKS 在这一点上需要注意的一个重要区别是，该密钥用于解锁 LUKS 加密设备的主密钥，并且可以通过 root 访问权限进行更改。其他加密模式不支持在设置后更改密钥，因为它们不使用主密钥进行加密。有关详细信息，请参阅静态数据加密#块设备加密（Data-at-rest encryption#Block device encryption）。

加密选项

Cryptsetup 支持与 dm-crypt 一起使用的不同加密操作模式：

- `--type luks` 使用默认的 LUKS 格式版本（cryptsetup<2.1.0为LUKS1，cryptsetup≥2.1.0为LUKS2）；
- `--type luks1` 用于使用 LUKS1，LUKS 的过往版本；
- `--type luks2` 用于使用 LUKS2，LUKS 的当前版本，引入了附加扩展；
- `--hw-opal-only` 用于驱动盘实现了 TCG OPAL 标准的硬件加密。见自加密驱动#使用 cryptsetup（英语：Self-encrypting drives#Using cryptsetup） 和 cryptsetup(8) § SED (SELF ENCRYPTING DRIVE) OPAL EXTENSION_OPAL_EXTENSION)。
- `--hw-opal` 用于 OPAL 硬件加密与 dm-crypt 软件加密叠叠乐。
- `--type plain` 使用 dm-crypt plain模式；
- `--type loopaes` 用于 loopaes传统模式；
- `--type tcrypt` 用于TrueCrypt 兼容模式。
- `--type bitlk` 用于 BitLocker 兼容模式。请参阅cryptsetup(8) § BITLK (WINDOWS BITLOCKER COMPATIBLE) EXTENSION_EXTENSION)。

可用的加密密码和散列的基本加密选项可用于所有模式，并依赖于内核加密后端功能。所有已加载并可在运行时用作选项的内容都可以通过以下方式查看：

```
$ less /proc/crypto
```

提示：如果列表很短，请执行`$ cryptsetup benchmark`触发加载可用模块。

下面介绍 `luks`、 `luks1`、 `luks2` 和 `plain` 模式的加密选项。请注意，表格列出了本文各个示例中使用的选项，而不是所有可用的选项。

LUKS 模式的加密选项

在 LUKS 加密模式下设置新的 dm-crypt 设备的 **cryptsetup** 操作是`luksFormat`。与名称所暗示的不同，它不会格式化设备，而是设置 LUKS设备标头并使用所需的加密选项加密主密钥。

为了使用`cryptsetup --help`列出的编译默认值创建一个新的 LUKS 容器，只需执行：

```
# cryptsetup luksFormat device
```

从 cryptsetup 2.4.0 开始，这相当于：

```
# cryptsetup luksFormat --type luks2 --cipher aes-xts-plain64 --hash sha256 --iter-time 2000 --key-size 256 --pbkdf argon2id --use-urandom --verify-passphrase device
```

默认值与下表中更高规格的加密示例进行了比较，并附有注释：

选项	Cryptsetup 2.1.0默认值	示例	注释
--cipher -c			

\-s

\-h

\-i

\-y

LUKS 功能和选项的属性在 LUKS1(pdf) 和 LUKS2(pdf) 规范中进行了描述。

提示：项目开发者的 devconfcz2016(pdf) 演示文稿总结了 LUKS2 主要规范更新的动机。另请参阅 cryptsetup 的v2 release notes

迭代时间

来自 cryptsetup FAQ§2.1 和 §3.4：

: 设置密码时计算密钥槽 \[……\] 的解锁时间。默认为 1 秒（LUKS2 为 2 秒）。 \[……\] : 密码迭代计数基于时间，因此安全级别取决于创建 LUKS 容器的系统的 CPU 能力。 \[……\] : 如果在快速机器上设置密码，然后在慢速机器上解锁，解锁时间可能会更长。

因此，最好始终在最常访问它的机器上创建一个容器。

阅读其余部分，了解如何在需要时正确调整迭代次数。

扇区大小

请参阅Advanced Format#dm-crypt.

plain模式的加密选项

在dm-crypt **plain**模式下，设备上没有主密钥，因此，不需要设置它。相反，要使用的加密选项被直接用来创建加密硬盘和命名设备之间的映射。映射可以针对一个分区或一个完整的设备创建。在后一种情况下，甚至不需要一个分区表。

使用cryptsetup 的默认参数创建**plain**模式映射：

```
# cryptsetup open --type plain options device name
```

Executing it will prompt for a password, which should have very high entropy, and the `--verify-passphrase` option can be used but is not a default. In general it is advisable to make exact note of the encryption options used for the creation,

because they can not be derived from the encrypted device, or an optional key-file, and upstream defaults may change. 执行它将提示输入密码，该密码应该具有非常高的熵。也可以添加 `--verify-passphrase` 选项（并非默认值）。一般来说，建议准确记录用于创建加密驱动器的选项，因为它们不能从加密设备或可选密钥文件中获得，而上游（简写的）默认值可能会更改。

选项	Cryptsetup 2.7.0默认值	示例	注释
--hash -h			

\-c

\-s

\-b

\-o

\-p

\-d

\-l

使用设备`/dev/sd*X*`, 上述右栏示例的结果是：

```
# cryptsetup open --type plain --cipher=aes-xts-plain64 --offset=0 --skip=2048 --key-file=/dev/sdZ --key-size=512 --sector-size 4096 /dev/sdX enc
```

警告：与使用 LUKS 加密不同，上述命令在需要重新建立映射时必须**完整**执行，因此记住密码、哈希和密钥文件的详细信息很重要。不要依赖 cryptsetup 的默认值，它们可能会随着版本迭代而改变。

现在可以检查是否已映射：

```
# fdisk -l
```

此时一个条目应当存在于`/dev/mapper/enc`。

使用 cryptsetup 加密设备

本节介绍如何使用选项来创建新的加密块设备并手动访问它们。

警告：GRUB 对 LUKS2 的支持有限；有关详细信息，请参阅GRUB#Encrypted /boot。对 GRUB 需要解锁的分区请使用 基于 PBKDF2 的 LUKS2 （`cryptsetup luksFormat --pbkdf pbkdf2`）。

使用 LUKS 模式加密设备

格式化 LUKS 分区

要将分区设置为加密的 LUKS分区，请执行：

```
# cryptsetup luksFormat device
```

然后将提示您输入密码并进行验证。

有关命令行选项，请参阅#Encryption options for LUKS mode\[损坏的链接：无效的章节\]。

您可以通过以下方式检查结果：

```
# cryptsetup luksDump device
```

您会注意到，dump 不仅显示密码头信息，还显示用于 LUKS 分区的密钥槽。

以下示例将使用 XTS 模式下的默认 AES 密码和有效的 256 位加密在 `/dev/sda1`上创建一个加密的根分区

```
# cryptsetup luksFormat -s 512 /dev/sda1
```

使用LUKS来格式化带有密钥文件的分区

在创建新的 LUKS 加密分区时，密钥文件可能会在创建时与分区相关联，使用：

```
# cryptsetup luksFormat device /path/to/mykeyfile
```

有关如何生成和管理密钥文件的说明，请参阅#密钥文件。

使用设备映射器解锁/映射 LUKS分区

一旦创建了 LUKS 分区，就可以解锁它们。

解锁过程将使用设备映射器将分区映射到新的设备名称。这提醒内核，`*device*`实际上是一个加密设备，应该通过 LUKS 使用 `/dev/mapper/*dm_name*` 寻址，以免覆盖加密数据。为防止意外覆盖，为了防止意外覆盖，请阅读在完成设置后备份cryptheader的可能性。

为了打开一个加密的LUKS分区，执行：

```
# cryptsetup open device dm_name
```

然后将提示您输入密码以解锁该分区。通常，设备映射名称描述了被映射的分区的功能。例如，以下内容解锁了根 luks 分区`/dev/sda1`并将其映射到名为`root`的设备映射器：

```
# cryptsetup open /dev/sda1 root
```

打开后，根分区设备地址将是`/dev/mapper/root` 而不是分区（例如 `/dev/sda1`）。

为了在加密层上设置 LVM，解密卷组的设备文件将类似于`/dev/mapper/root`而不是`/dev/sda1`。然后LVM 将为创建的所有逻辑卷提供附加名称如`/dev/lvmpool/root` 与`/dev/lvmpool/swap`。

为了将加密数据写入分区，必须通过设备映射名称对其进行访问。访问的第一步通常是创建文件系统。例如：

```
# mkfs.ext4 /dev/mapper/root
```

然后，设备`/dev/mapper/root`可以像其他分区一样被挂载(mount)。

要关闭 LUKS 容器，请卸载分区并执行以下操作：

```
# cryptsetup close root
```

使用 TPM 存储密钥

请参阅可信平台模块#LUKS 数据加密

使用plain模式加密设备

创建和后续访问**dm-crypt** plain模式加密只需要使用**cryptsetup** open操作，并使用正确的参数。下面的例子展示了两个非根设备的例子，但通过将两个设备堆叠起来增加了一个quirk(即，第二个是在第一个设备内部创建的)。显然，叠加加密会使开销加倍。这里的用例只是为了演示使用cipher选项的另一个示例。

第一个映射器是使用 **cryptsetup** 的plain模式默认值创建的，如上文的表中左列所述

```
# cryptsetup --type plain -v open /dev/sdxY plain1
WARNING: Using default options for cipher (aes-xts-plain64, key size 256 bits) that
could be incompatible with older versions.
WARNING: Using default options for hash (sha256) that could be incompatible with older
versions.
For plain mode, always use options --cipher, --key-size and if no keyfile is used, then
also --hash.

Enter passphrase for /dev/sdxY:
Command successful.
```

现在我们在其中添加第二个块设备，使用不同的加密参数和（可选的）偏移量，创建一个文件系统并挂载它

```
# cryptsetup --type plain --cipher=serpent-xts-plain64 --hash=sha256 --key-size=256 --
offset=10  open /dev/mapper/plain1 plain2
Enter passphrase for /dev/mapper/plain1:
# lsblk -p
 NAME
 /dev/sda
 ├─/dev/sdxY
 │ └─/dev/mapper/plain1
 │   └─/dev/mapper/plain2
 ...
# mkfs -t ext2 /dev/mapper/plain2
# mount -t ext2 /dev/mapper/plain2 /mnt
# echo "This is stacked. one passphrase per foot to shoot." > /mnt/stacked.txt
```

我们关闭堆栈（stack），以检查访问是否有效

```
# cryptsetup close plain2
# cryptsetup close plain1
```

首先，让我们尝试直接打开文件系统：

```
# cryptsetup --type plain --cipher=serpent-xts-plain64 --hash=sha256 --key-size=256 --
offset=10 open /dev/sdxY plain2
# mount -t ext2 /dev/mapper/plain2 /mnt
mount: /mnt: wrong fs type, bad option, bad superblock on /dev/mapper/plain2, missing
codepage or helper program, or other error.
       dmesg(1) may have more information after failed mount system call.
```

为什么那行不通？因为“plain2”起始块（10）仍然使用“plain1”中的密码加密。它只能通过堆叠映射器访问。不过这个错误是任意的，尝试错误的口令或错误的选项也会产生同样的结果。对于 **dm-crypt** plain 模式，open操作本身不会出错。

以正确的顺序重试：

```
# cryptsetup close plain2    # 上一次尝试的功能失调的映射器
# cryptsetup --type plain open /dev/sdxY plain1
WARNING: Using default options for cipher (aes-xts-plain64, key size 256 bits) that
could be incompatible with older versions.
WARNING: Using default options for hash (sha256) that could be incompatible with older
versions.
For plain mode, always use options --cipher, --key-size and if no keyfile is used, then
also --hash.

Enter passphrase for /dev/sdxY:
# cryptsetup --type plain --cipher=serpent-xts-plain64 --hash=sha256 --key-size=256 --
offset=10 open /dev/mapper/plain1 plain2
Enter passphrase for /dev/mapper/plain1:
# mount /dev/mapper/plain2 /mnt && cat /mnt/stacked.txt
This is stacked. one passphrase per foot to shoot.
```

dm-crypt也会处理一些混合模式的叠加加密。例如，LUKS模式可以在 “plain1”映射器上叠加。当它被关闭时，它的头会在 “plain1”中被加密。

仅可用于plain模式的是选项`--shared`。有了它，可以将单个设备分割成不同的非重叠映射器。我们在下一个示例中就是这样做的，这次为“plain2”使用与 loopaes兼容的密码模式：

```
# cryptsetup --type plain --offset 0 --size 1000 open /dev/sdxY plain1
WARNING: Using default options for cipher (aes-xts-plain64, key size 256 bits) that
could be incompatible with older versions.
WARNING: Using default options for hash (sha256) that could be incompatible with older
versions.
For plain mode, always use options --cipher, --key-size and if no keyfile is used, then
also --hash.

Enter passphrase for /dev/sdxY:
# cryptsetup --type plain --offset 1000 --size 1000 --shared --cipher=aes-cbc-lmk --
hash=sha256 open /dev/sdxY plain2
WARNING: Using default options for cipher (aes-cbc-lmk, key size 256 bits) that could
be incompatible with older versions.

For plain mode, always use options --cipher, --key-size and if no keyfile is used, then
also --hash.

Enter passphrase for /dev/sdxY:
# lsblk -p
NAME
dev/sdxY
├─/dev/sdxY
│ ├─/dev/mapper/plain1
│ └─/dev/mapper/plain2
...
```

正如设备树所示，两者都位于同一级别，即没有堆叠，“plain2”可以单独打开。

专门针对LUKS的Cryptsetup操作

密钥管理

可以为 LUKS 分区添加额外的密钥。这使用户能够创建访问密钥以进行安全的备份存储。在所谓的密钥托管中，一个密钥用于日常使用，另一个保存在托管中以获取对分区的访问权限，以防忘记日常密码或密钥文件丢失/损坏。 也可以使用不同的密钥槽向用户授予对分区的访问权限，方法是发出第二个密钥，然后再将其撤销。

一旦创建了加密分区，就会创建初始密钥槽0。（如果没有手动指定其他密钥槽）。额外的密钥槽从1到7进行编号。可以通过发出以下命令查看哪些密钥槽被使用

```
# cryptsetup luksDump /dev/device
```

其中*device*是包含LUKS头的块设备。此命令和以下本节中的所有命令也适用于头备份文件。

添加 LUKS 密钥

添加新的密钥槽是动作luksAddKey完成的。为了安全起见，在输入新密钥之前，它总是会要求输入一个有效的现有密钥（任意已有密码槽的密码），对于已经解锁的设备也是如此。

```
# cryptsetup luksAddKey /dev/device (/path/to/additionalkeyfile)
Enter any existing passphrase:
Enter new passphrase for key slot:
Verify passphrase:
```

如果给出了*/path/to/additionalkeyfile*，cryptsetup 将为 *additionalkeyfile* 添加一个新的密钥槽。否则提示输入新密码。若要使用现有的**密钥文件**来授权操作，--key-file或 -d 选项后跟“旧的” *keyfile*，这样将尝试解锁所有可用的密钥文件密钥槽：

```
# cryptsetup luksAddKey /dev/device [/path/to/additionalkeyfile] -d /path/to/keyfile
```

如果打算使用多个密钥并更改或撤销它们，则可以使用 --key-slot或 -S 选项来指定密钥槽：

```
# cryptsetup luksAddKey /dev/device -S 6
WARNING: The --key-slot parameter is used for new keyslot number.
Enter any existing passphrase:
Enter new passphrase for key slot:
Verify passphrase:
# cryptsetup luksDump /dev/device
...
Keyslots:
...
6: luks2
      Key:      512 bits
      Priority: normal
...
```

提示：LUKS2 可以为各个密码槽设置优先级，请参阅 cryptsetup-config(8)。 例如，如果上述密码槽 6 旨在在启动时解锁设备，请用 root 执行 `cryptsetup config --priority Preferred --key-slot 6 /dev/*device*` 。

为了在此示例中显示关联的操作，我们决定在继续删除之前立即更改密钥：

```
# cryptsetup luksChangeKey /dev/device -S 6
Enter passphrase to be changed:
Enter new passphrase:
Verify passphrase:
```

移除 LUKS 密钥

从加密头部中删除密钥有三种不同的操作：

- `luksRemoveKey` 通过指定其密码/密钥文件来删除密钥。请参阅 cryptsetup-luksRemoveKey(8) 。
- `luksKillSlot` 可用于从特定密钥槽中删除密钥（使用另一个有效密钥）。显然，如果您忘记了密码、丢失了密钥文件或无法访问它，这将非常有用。请参阅 cryptsetup-luksKillSlot(8)。
- `erase` 用于快速删除所有激活的密钥。请参阅 cryptsetup-erase(8)。

警告：* 以上所有操作都可用于不可撤销地删除加密设备的最后一个活动密钥！

- 在1.6.4版本中添加了`erase`命令，此操作**不会**提示输入有效密码！它不会擦除 LUKS 头，而是一次擦除所有密钥槽，因此，除非您拥有 LUKS 头的有效备份，否则您将无法重新获得访问权限。
- 对 OPAL 硬件加密的分区使用 `erase` 命令时，LUKS 头会被清空，同时 OPAL 锁定区域会被移除。与软件加密不同，这个操作不可恢复，即使存在有效的 LUKS 头备份也不行。

对于上述警告，最好知道我们要保留的密钥是有效的。一个简单的检查是使用 `-v` 选项解锁设备，该选项将指出它占用的密钥槽：

```
# cryptsetup --test-passphrase -v open /dev/device
No usable token is available.
Enter passphrase for /dev/device:
Key slot 1 unlocked.
Command successful.
```

现在我们可以使用其密码删除在上一小节中添加的密钥：

```
# cryptsetup luksRemoveKey /dev/device
Enter passphrase to be deleted:
```

如果我们对两个密钥槽使用相同的密码，那么现在将擦除第一个槽。只有再次执行它才会删除第二个。

或者，我们可以指定密钥槽：

```
# cryptsetup luksKillSlot /dev/device 6
Enter any remaining passphrase:
```

请注意，在这两种情况下，都不需要确认。

重申上面的警告：如果密钥槽 1 和 6 使用了相同的密码，那么现在两者都将消失。

备份与恢复

如果 LUKS 加密分区的头被破坏，您将无法解密您的数据。这就像忘记密码或损坏用于解锁分区的密钥文件一样，都是一种困境。损坏可能是由于您在稍后重新分区硬盘时自己的错误或第三方程序误解了分区表而造成的。因此，备份（LUKS 加密分区的）头并将其存储在另一个硬盘上可能是个好主意。

注意：如果其中一个LUKS加密分区的密码被泄露，你必须在每一个加密头（cryptheader）的副本上撤销它，即使是那些你已经备份的副本。否则，使用被泄露的密码短语的备份加密头的副本可被用来确定主密

钥，该主密钥可以用来解密关联的分区（甚至是实际的分区，而不仅仅是备份的版本）。另一方面，如果主密钥被泄露，您必须重新加密整个分区。有关详细信息，请参阅 LUKS FAQ。

使用 cryptsetup 进行备份

Cryptsetup 的`luksHeaderBackup` 操作存储LUKS 头和密钥槽区域的二进制备份：

```
# cryptsetup luksHeaderBackup /dev/device --header-backup-file /mnt/backup/file.img
```

其中`*device*`是包含LUKS 卷的分区。

你也可以将纯文本头备份到tmpfs中，并在将其写入持久性存储之前用例如GPG进行加密：

```
# mount --mkdir -t tmpfs -o noswap tmpfs /root/tmp
# cryptsetup luksHeaderBackup /dev/device --header-backup-file /root/tmp/file.img
# gpg --recipient User_ID --encrypt /root/tmp/file.img
# cp /root/tmp/file.img.gpg /mnt/backup/
# umount /root/tmp
```

提示：选项`noswap`确保了该文件系统不会交换到硬盘中去。

使用 cryptsetup 恢复

警告：恢复不正确的头或恢复到未加密的分区将导致数据丢失!该操作无法检查头是否实际上是该特定设备的**正确**头。

为了避免恢复错误的头，你可以先使用`--header` 远程连接它，来确保它真的没错。

```
# cryptsetup -v --header /mnt/backup/file.img open /dev/device test
No usable token is available.
Enter passphrase for /dev/device:
Key slot 0 unlocked.
Command successful.
# mount /dev/mapper/test /mnt/test && ls /mnt/test
# umount /mnt/test
# cryptsetup close test
```

现在检查成功，可以执行恢复：

```
# cryptsetup luksHeaderRestore /dev/device --header-backup-file ./mnt/backup/file.img
```

现在所有的密钥槽区域都被重写了；发出命令后，只有备份文件中活跃的密钥槽可用。

手动备份和恢复

头（header）始终位于设备的开头，并且无需访问 **cryptsetup** 也可以执行备份。首先，您必须找出加密分区的有效负载偏移量：

```
# cryptsetup luksDump /dev/device | grep "Payload offset"
Payload offset:       4040
```

其次检查驱动器的扇区大小

```
# fdisk -l /dev/device | grep "Sector size"
Sector size (logical/physical): 512 bytes / 512 bytes
```

现在您知道了这些值，您可以使用简单的 dd 命令备份头：

```
# dd if=/dev/device of=/path/to/file.img bs=512 count=4040
```

并安全存储。

然后可以使用与备份时相同的值执行还原：

```
# dd if=./file.img of=/dev/device bs=512 count=4040
```

重新加密设备

cryptsetup `reencrypt` 操作能够重新加密 LUKS 设备。在 LUKS2 设备上，此操作可在线进行、多重并行且抗系统故障（弹性）。在 LUKS1 设备上，此操作只能离线进行（卸载后），单进程操作且更少弹性。

请参阅 cryptsetup-reencrypt(8)了解操作模式和选项。

可以借此改变#LUKS 模式的加密选项，也可用于将现有的未加密文件系统转换为 LUKS 加密文件系统或从设备中永久删除 LUKS 加密（使用`--decrypt`，请参阅去除系统加密（英语：Removing system encryption））。对于分离的 LUKS 标头，重新加密也是可行的，但请注意 `--header` 选项的警告。不支持 LUKS 以外的模式（例如 plain 模式）的重新加密。

重新加密的一种应用可能是在密码或#密钥文件被泄露**并且**不能确定（对方）没有获得 LUKS 头的副本后再次保护数据。例如，如果只有一个口令被窃取，但没有发生对设备的物理/逻辑访问，那么只需改变相应的口令/密钥即可（#密钥管理）。

警告：始终确保有**可靠的备份**可用，并在使用该工具之前仔细检查您指定的选项！

下面展示了一个对未加密的文件系统分区进行加密以及对现有LUKS设备进行重新加密的例子。

加密现有的未加密文件系统

提示：如果您尝试加密现有的根分区，您可能需要创建一个将被挂载到`/boot`的单独的且未加密的引导分区（请参阅dm-crypt/加密整个系统#准备boot分区）。这不是绝对必要的，但有许多优点：

- 如果`/boot` 位于加密的根分区内，系统将在机器开机时要求输入两次密码。第一次发生在引导加载程序尝试读取位于加密`/boot`中的文件时，第二次发生在内核尝试挂载加密分区 。这可能不是所需的行为，可以通过使用单独且未加密的引导分区来防止。
- 如果`/boot`位于加密分区，某些系统还原应用程序（例如，timeshift包）将无法运行。

简而言之，如果需要，创建一个大小至少为 260 MiB 的分区。请参阅Partitioning#/boot。

LUKS 加密头始终存储在设备的起始（the beginning of the device）。由于现有文件系统通常会分配所有分区扇区，因此第一步是缩小它以给 LUKS头腾出空间。

默认的LUKS2头需要16兆字节。如果当前的文件系统占据了所有的可用空间，那么我们至少要缩减这么多。要将`ext4`上现有的`/dev/sd*xY*`文件系统缩小到目前可能的最小值:

```
# umount /mnt
# e2fsck -f /dev/sdxY
e2fsck 1.46.5 (30-Dec-2021)
Pass 1: Checking inodes, blocks, and sizes
...
/dev/sda6: 12/166320 files (0.0% non-contiguous), 28783/665062 blocks
# resize2fs -p -M /dev/sdxY
e2fsck 1.46.5 (30-Dec-2021)
Resizing the filesystem on /dev/sdxY to 26347 (4k) blocks.
The filesystem on /dev/sdxY is now 26347 (4k) blocks long.
```

提示：使用`-M`缩小到最小尺寸可能需要较长时间。您可能想要计算比当前大小仅小32 MiB 的大小，而不是使用 `-M`。

警告：文件系统应被缩小，而底层设备（比如说一个分区）应保持其原始大小。一些图形工具（例如GParted）可能会同时调整文件系统和分区的大小，加密后可能会发生数据丢失。

现在我们使用默认密码对其进行加密，我们不必明确指定它：

```
# cryptsetup reencrypt --encrypt --reduce-device-size 32M /dev/sdxY
WARNING!

========

This will overwrite data on LUKS2-temp-12345678-9012-3456-7890-123456789012.new
irrevocably.

Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for LUKS2-temp-12345678-9012-3456-7890-123456789012.new:
Verify passphrase:
```

完成后，整个`/dev/sd*xY*`分区都被加密，而不仅仅是文件系统被压缩的空间。作为最后一步，我们将原来的`ext4`文件系统扩展到现在加密的分区上，再次占用所有可用的空间：

```
# cryptsetup open /dev/sdxY recrypt
Enter passphrase for /dev/sdxY:
...
# resize2fs /dev/mapper/recrypt
resize2fs 1.43-WIP (18-May-2015)
Resizing the filesystem on /dev/mapper/recrypt to 664807 (4k) blocks.
The filesystem on /dev/mapper/recrypt is now 664807 (4k) blocks long.
# mount /dev/mapper/recrypt /mnt
```

文件系统现在可以使用了。您可能希望将其添加到您的crypttab。

提示：如果刚刚加密了根分区，可能需要做许多加密后的调整。

1. 配置 mkinitcpio 和内核参数。参阅Dm-crypt/系统配置#在早期用户空间解密
2. 更新 fstab 中的 / 条目，使用解锁卷的标识 (如使用 UUID).

重新加密现有的 LUKS 分区

在此示例中，现有的 LUKS 设备被重新加密。

警告：在没有**可靠备份**的情况下永远不要直接进行重新加密！若因密码泄露而考虑重新加密，则应当在创建备份、执行重新加密操作**之前**使用 `luksChangeKey`。

若要使用目标设备当前的加密选项重新加密，只需执行：

```
# cryptsetup reencrypt /dev/sdxY
```

使用不同的密码和/或散列重新加密设备时，会保留现有密钥。

若需要在重新加密的过程中，采用新的加密选项，则需要手动指定欲采用的选项。注意 LUKS2 头支持为每个密钥槽设置不同的加密选项，故重新加密过程只会在 LUKS2 卷的数据段上进行。

注意，LUKS 头的大小可能限制可用的修改操作。例如，如果设备最初使用 CBC 模式密码和 128 位密钥大小加密，则 LUKS 头将是上述4096个扇区大小的一半：

```
# cryptsetup luksDump /dev/sdxY | grep -e "mode" -e "Payload" -e "MK bits"
Cipher mode:         cbc-essiv:sha256
Payload offset:      2048
MK bits:             128
```

若要在类似情况下更新加密选项，则应当在重新加密前将卷转换为 LUKS2 格式。若在转换过程中即出现头部空间不足，则需要先使用 `--reduce-device-size` 选项进行重新加密，以为头部留出更多的空间，之后再使用新的加密选项进行重新加密。将卷转换为 LUKS2 格式会操作 LUKS 头，为头部留出更多空间需要操作文件系统数据，故需要注意上述两步操作都存在一定风险。

从LUKS1到LUKS2的转换和回退

cryptsetup 提供 `convert` 选项，用于在 LUKS1 和 LUKS2 容器类型之间进行转换。参数 `--type` 是**必需的**。建议在执行转换前先对 LUKS 头进行备份。

从 LUKS1 迁移到 LUKS2：

```
# cryptsetup convert --type luks2 /dev/sdxY
```

注意：LUKS 头大小将为 16 MiB 而不是 2MiB。

回滚到 LUKS1（例如，从带有加密 /boot 的GRUB 启动）：

```
# cryptsetup convert --type luks1 /dev/sdxY
```

注意：从 LUKS2 到 LUKS1 的转换并非总是可行的。您可能会收到以下错误：

```
Cannot convert to LUKS1 format - keyslot 0 is not LUKS1 compatible.
If the container is using Argon2, it needs to be converted to PBKDF2 to be LUKS1-compatible.

# cryptsetup luksConvertKey --pbkdf pbkdf2 /dev/sdxY
```

调整加密设备的大小

如果将使用 dm-crypt 加密的存储设备（使用dd 之类的工具）克隆到另一个更大的设备，或调整了 LUKS 容器所在磁盘分区的大小，则需要相应调整 LUKS 容器中的文件

系统的大小。注意，对于 LUKS 容器本身，若没有指定 `--size` 参数，则容器并不记录底层磁盘分区的大小，而是默认使用整个分区，故不需要手动“调整”LUKS容器的大小，而是只需要遵循调整普通分区的大小的步骤即可。

扩大包含 *ext4* 文件系统的 LUKS 设备（`/dev/sdX2`）的步骤如下：

首先，使用 Parted 或 fdisk 调整底层分区的大小（注意，若进行的是缩容，则应当先调整文件系统大小，再调整底层分区大小）。

然后，打开加密设备，并调整文件系统的大小：

警告：在调整大小前，应当备份设备中的数据，并且应当卸载要调整的文件系统。

在此示例中，目标设备是 /dev/sdX2，将使用与分区相邻的整个可用空间：

```
# cryptsetup luksOpen /dev/sdX2 sdX2
# e2fsck /dev/mapper/sdX2
# resize2fs /dev/mapper/sdX2  # 使用扩大后的底层 LUKS 分区中的所有空间
```

之后重新挂载对应的文件系统即可。

```
# mount /dev/mapper/sdX2 /mnt/enlarged_sdX2
```

注意：cryptsetup-resize 命令用于调整当前处于活跃状态的 dm-crypt 映射的大小。若调整的是*ext2/3/4*文件系统，一般不需要该命令。

在LUKS上配置LVM

详见调整“在LUKS上配置LVM”方案的大小。

回送文件系统

假设一个加密的回送文件系统存储在文件`/bigsecret`中，循环到`/dev/loop0`，映射到`secret`并挂载在`/mnt/secret`上，如 dm-crypt/Encrypting a non-root file system#File container中的例子。

如果容器文件当前已映射和/或挂载，请卸载 并且/或着 关闭它：

```
# umount /mnt/secret
# cryptsetup close secret
# losetup -d /dev/loop0
```

接下来，使用要添加的数据大小扩展容器文件。在本例中，文件将扩展 1M * 1024，即1G。

警告：一定确保使用 `oflag=append conv=notrunc` 选项，否则会覆盖文件，而不是追加到它。强烈建议在这一步骤之前做一个备份。

```
# dd if=/dev/urandom of=/bigsecret bs=1M count=1024 iflag=fullblock oflag=append conv=notrunc status=progress
```

现在将容器映射到回送设备：

```
# losetup /dev/loop0 /bigsecret
# cryptsetup open /dev/loop0 secret
```

在此之后，将容器的加密部分的大小调整为容器文件的新的最大尺寸：

```
# cryptsetup resize secret
```

最后，执行文件系统检查，如果没问题，调整它的大小（例如ext2/3/4）：

```
# e2fsck -f /dev/mapper/secret
# resize2fs /dev/mapper/secret
```

您现在可以再次挂载容器：

```
# mount /dev/mapper/secret /mnt/secret
```

完整性保护设备

如果设备是在支持完整性的情况下格式化的 (例如 `--integrity hmac-sha256`) 并且支持块设备缩小，则无法打开它，并出现以下错误： `device-mapper: reload ioctl on failed: Invalid argument.`

为了解决这个问题而不再次擦除设备，可以用以前的主密钥进行格式化（保持每个扇区的标签有效）。

```
# cryptsetup luksDump /dev/sdX2 --dump-master-key --master-key-file=/tmp/masterkey-in-tmpfs.key
# cryptsetup luksFormat /dev/sdX2 --type luks2 --integrity hmac-sha256 --master-key-file=/tmp/masterkey-in-tmpfs.key --integrity-no-wipe
# rm /tmp/masterkey-in-tmpfs.key
```

密钥文件

注意：本节描述使用明文密钥文件。如果你想加密你的密钥文件给你双重身份验证，请参阅dm-crypt/特殊应用#使用GPG、LUKS、OpenSSL加密的密钥文件了解详细信息，但仍请阅读本节。

什么是密钥文件?

密钥文件是将其数据用作解锁加密卷的密码的文件。 这意味着如果此类文件丢失或更改，则可能无法再解密该卷。

提示：在密钥文件之外定义一个密码，以便在定义的密钥文件丢失或改变的情况下对加密卷进行备份访问。

为什么要使用密钥文件?

密钥文件有很多种。所使用的每种类型的密钥文件都有以下总结的优点和缺点:

密钥文件的类型

密码

这是一个包含简单密码的密钥文件。这种类型的密钥文件的好处是，如果文件丢失，它包含的数据是已知的，并且很容易被加密卷的所有者记住。然而，缺点是这不会增加在初始系统启动期间输入密码的任何安全性。

示例： `correct horse battery staple`

{{注意|包含密码短语的密钥文件中不得有换行符。一种选择是使用以下方法来创建它：

```
# printf '%s' 'your_passphrase' | install -m 0600 /dev/stdin /etc/cryptsetup-keys.d/keyfile.key
```

如果文件包含特殊字符，例如引号、反斜杠，而不是转义这些字符，建议直接编辑密钥文件，直接输入或粘贴密码，然后用方便的perl命令一行删除结尾换行：

```
 # perl -pi -e 'chomp if eof' /etc/cryptsetup-keys.d/keyfile.key
```

随机文本

这是一个包含随机字符块的密钥文件。这种类型的密钥文件的好处是它比简单的密码更能抵抗字典攻击。在这种情况下，可以利用密钥文件的另一个优势，即所用数据的长度。由于这不是要由人记住以供输入的字符串，因此创建包含数千个随机字符作为密钥的文件是微不足道的。缺点是如果此文件丢失或更改，则很可能在没有备份密码的情况下无法访问加密卷。

理论上，随机文本可以使用任意字符集。但考虑到部分情况下键盘布局和 Unicode 支持可能不完善（例如，系统启动时的 LUKS 密码输入框、部分不支持 Unicode 的 POSIX 工具），建议使用 ASCII 中的字母和数字。可以按如下方式生成：

```
$ tr -dc '[:alnum:]' </dev/urandom | head -c64
```

示例：

```
rTCBW6j1dI2aYC5KcD6Ar38rBGN2DkWyang3RT7pdMGpdf1kRuMXi8EBHKu0BJ8X
```

而随机生成 UTF-8 编码的密钥文件，结果可能是这样的：

示例：`W[5ODó?Oéµ»9¬hjT}DÐ§íŽuLÝæÝœ§aþóx±)Ñ)léeð¤ú=èe`

二进制

这是一个已被定义为密钥文件的二进制文件。在将文件识别为密钥文件的候选文件时，建议选择相对静态的文件，例如照片、音乐、视频剪辑。这些文件的好处是它们具有双重功能，使它们更难被识别为密钥文件。与具有大量随机文本的文本文件不同，这种密钥文件对于旁观者来说看起来就像一个普通的图像文件或音乐剪辑。缺点是如果此文件丢失或更改，则很可能在没有备份密码的情况下无法访问加密卷。此外，与随机生成的文本文件相比，理论上存在随机性损失。这是因为图像、视频和音乐在相邻数据位之间具有某种内在关系，而随机文本文件不存在这种关系。然而，这是有争议的，从未被公开利用。

示例： 图片、文本、视频……

使用随机字符创建密钥文件

将密钥文件存储在文件系统上

密钥文件可以是任意内容和大小。

这里 dd 用于生成 2048 个随机字节的密钥文件，将其存储在文件`/etc/cryptsetup-keys.d/mykeyfile.key` 中：

```
# dd bs=512 count=4 if=/dev/random iflag=fullblock | install -m
0600 /dev/stdin /etc/cryptsetup-keys.d/mykeyfile.key
```

如果你计划将密钥文件存储在一个外部设备上，也可以轻松将输出文件更改为对应的目录：

```
# dd bs=512 count=4 if=/dev/random of=/run/media/user/usbstick/mykeyfile.key
iflag=fullblock
```

安全地覆盖存储的密钥文件

如果你把你的临时密钥文件存储在一个物理存储设备上，并想删除它，记住以后不要只是删除密钥文件，而是使用类似于

```
# shred --remove --zero mykeyfile
```

来安全地覆盖它。对于 FAT 或 ext2等过时的文件系统，这已经足够了，而在日志文件系统、闪存硬件和其他情况下，强烈建议擦除整个设备（英语：Securely_wipe_disk）。

将密钥文件存储在 tmpfs 中（关闭 swap）

另外，您可以挂载一个 tmpfs（关闭 swap） 来临时存储密钥文件：

```
# mount --mkdir -t tmpfs -o noswap tmpfs /root/mytmpfs
# cd /root/mytmpfs
```

优点是它驻留在 RAM 中而不是物理磁盘上，因此卸载 tmpfs 后无法恢复。将密钥文件复制到另一个安全且持久的文件系统后，再次卸载 tmpfs：

```
# umount /root/mytmpfs
```

配置 LUKS 以使用密钥文件

将密钥文件的密钥槽添加到 LUKS 头：

```
# cryptsetup luksAddKey /dev/sda2 /etc/cryptsetup-keys.d/mykeyfile.key
Enter any existing passphrase:
```

使用密钥文件手动解锁分区

打开 LUKS设备时使用`--key-file` 选项：

```
# cryptsetup open /dev/sda2 dm_name --key-file /etc/cryptsetup-keys.d/mykeyfile.key
```

在启动时解锁根分区

这只是配置mkinitcpio以包括必要的模块或文件，并配置cryptkey以及内核参数以知道在哪里找到密钥文件。

下面介绍两种情况：

1. 使用存储在外部介质（例如 U盘）上的密钥文件
2. 使用嵌入在initramfs 中的密钥文件

使用存储在外部介质上的密钥文件

配置mkinitcpio

您必须将驱动器文件系统的内核模块添加到 `/etc/mkinitcpio.conf`中的 MODULES array中。例如，如果文件系统是Ext4，则添加`ext4`；如果是FAT，则添加`vfat`：

```
MODULES=(vfat)
```

如果在启动时有关于坏的超级块和坏的码表的信息，那么你需要加载一个额外的码表模块。例如，你可能需要`nls_iso8859-1`模块来加载`iso8859-1`编码页。

重新生成 initramfs。

配置内核参数

- 对于使用encrypt钩子的基于busybox的initramfs，参阅Dm-crypt/系统配置#cryptkey。
- 对于使用sd-encrypt钩子的基于systemd的initramfs，参阅Dm-crypt/系统配置#rd.luks.key。

在 initramfs 中嵌入密钥文件

警告：只有你通过以下方式充分保护密钥文件时，才使用嵌入式密钥文件：

- 在启动过程的早期使用某种形式的身份验证。否则会发生自动解密，完全违背块设备加密的目的。
- `/boot`已加密。否则，不同安装（包括实时环境（live environment））上的 root 可以从 initramfs 中提取您的密钥，并在没有任何其他身份验证的情况下解锁设备。

此方法允许使用一个特殊命名的密钥文件，该密钥文件将嵌入到 initramfs 中并由`encrypt`钩子拾取以自动解锁根文件系统（`cryptdevice`）。在使用 GRUB 早期加密硬盘功能时应用它可能很有用，以避免在引导期间输入两个密码。

生成密钥文件，为其赋予适当的权限并将其添加为 LUKS 密钥：

```
 # dd bs=512 count=4 if=/dev/random iflag=fullblock | install -m
0600 /dev/stdin /etc/cryptsetup-keys.d/root.key
 # cryptsetup luksAddKey /dev/sdX# /etc/cryptsetup-keys.d/root.key
```

注意：注意：initramfs 由 mkinitcpio 生成，默认权限为`600`，因此普通用户无法通过生成的 initramfs 读取密钥文件。

在 mkinitcpio 的 FILES array中插入密钥：

```
/etc/mkinitcpio.conf
FILES=(/etc/cryptsetup-keys.d/root.key)
```

重新生成initramfs

对于 `encrypt` 钩子，密钥文件由 `cryptkey=` 内核参数指定：在 initramfs，语法为 `rootfs:*/path/to/keyfile*`。默认值为 `/crypto_keyfile.bin`，如果 initramfs 包含此路径的有效密钥，则可以省略 `cryptkey`。请参阅Dm-crypt/系统配置#cryptkey。

对于上述示例，当在 busybox-based initramfs 中使用 `encrypt` 钩子时，设置内核参数：

```
cryptkey=rootfs:/etc/cryptsetup-keys.d/root.key
```

若以 `sd-encrypt` 钩子替代，密钥文件通过 `rd.luks.key=` 内核参数指定。默认值是 `/etc/cryptsetup-keys.d/*name*.key`（其中 `*name*` 是用于 #使用 cryptsetup 加密设备 中解密的 *dm_name*）。且当 initramfs 包含此路径的有效密钥时， `rd.luks.key` 可省略。参阅 Dm-crypt/系统配置#rd.luks.key。

在下一次重新启动时，您应该只需要输入一次容器解密密码。

(资料来源)

2.11 Dm-crypt/加密整个系统

难度 高级 | **篇幅** 71.2 KB | **所属** 静态数据加密

本节将讲到

- 概览
- 在单一分区上配置LUKS
- 使用TPM2和安全启动在单一分区上配置
- 在LUKS上配置LVM
- 在LVM上配置LUKS

本文介绍了几种常见的使用“dm-crypt”加密整个系统的方式，以及相比正常安装过程所需的改动。官方安装镜像已经包含了加密所需的工具。

若要加密的是已有的未加密的文件系统，见 dm-crypt/设备加密#加密现有的未加密文件系统。

概览

“dm-crypt”能为为根文件系统提供安全保护，且相比其它方案具更优的性能。相比仅仅加密部分非根文件系统，加密根文件系统可以隐藏许多敏感信息（如已安装应用列表、用户名等），并能避免通过locate、`/var/log/`等常见方式泄露数据。此外，由

于除了引导加载程序和内核（通常情况下），所有数据都被加密，更难对系统进行未授权的改动。

除了上面提到的共同优点以外，各种加密方案还有如下特有的优缺点：

方案	优点	缺点
#在单一分区上配置LUKS 展示了使用LUKS完全加密的根文件系统的基本方式，最直接。	 - 分区与配置简单。 - 在使用GPT的磁盘上，systemd可自动挂载根分区。	

- 不灵活；用于加密的整个磁盘空间需要预先分配。

与上例相似，并通过使用安全启动、TPM2提高了安全性。

相比上例，本方法还有如下优点：

- 安全启动可抵御邪恶女佣攻击。
- 在安全启动被关闭或配置被修改时，TPM2将阻止解密。
- 与上例缺点相似。

在LUKS加密的单一分区上配置LVM，实现了分区灵活性。

- 通过LVM，可简单地在加密设备上配置多个分区。
- 只需单一密码即可解密所有分区。
- 在未解密时系统的分区结构不可见。
- 若要使用休眠功能，这是最简单的方式。
- LVM增加了系统设备映射结构的复杂性，且需要单独添加钩子。
- 难以对某个分区单独指定密码。
- 若需要配置的LVM卷组包含多个物理卷，则所有物理卷必须分别进行LUKS加密。在系统启动时，必须分别解锁所有物理卷后，才能激活并使用卷组。

在LVM上层使用LUKS。

- 可通过LVM将（单个）加密分区拓展到多个磁盘上。
- 可方便地同时配置加密与未加密分区。
- 若更改了LVM逻辑卷的结构，需要同时更改加密设备与逻辑卷的对应关系，配置复杂。
- 每个逻辑卷需要单独设置密码。
- LVM中逻辑卷的结构在未解密时也是可见的。
- 由于每个加密的逻辑卷都需要单独解密，系统启动较慢。

在软RAID上层使用LUKS。

- 与在LVM上配置LUKS相似。
- 与在LVM上配置LUKS，并使用加密引导分区类似。

使用dm-crypt的plain模式（不使用LUKS头及LUKS的多密钥选项）。 此方案还将`/boot`配置到USB设备上，并通过USB设备储存密钥。该思路也适用于其它方案。

- 在面临可能损坏LUKS头的意外情况时，仍能保证数据完整性。
- 能够实现可抵赖加密。
- 有助于解决在SSD上应用加密的一些问题。
- 需要尤其关注所有使用的加密参数。
- 只能使用单个密码且不能更换密码。
- 即使对于常规系统，配置也非常复杂。

展示了如何在使用GRUB时加密boot分区。 该方案还使用了EFI系统分区，这也适用于其它方案。

- 与选择的加密配置方式相同的优点。（在本例中采用在LUKS上配置LVM）
- 未加密的数据更少，只有引导加载程序和EFI系统分区（若有）未加密。
- 与选择的加密配置方式相同的缺点。（在本例中采用在LUKS上配置LVM）
- 配置更复杂。
- 其它引导加载程序不支持此方案。
- GRUB解密LUKS卷所需的时间较长，将减慢启动速度。
- 若将`zpool`加密，则其中的所有数据集（dataset）都将使用相同的ZFS加密设置加密。同时，由于加密在ZFS层面进行，支持ZFS的系统都可解密，便于配置双系统或在不同系统间共享数据。
- 可将加密的文件系统安全地备份到未加密的ZFS中。产生的文件系统快照将保持原生加密状态。
- ZFS 不会加密有关存储池结构的文件系统元数据（例如，数据集/快照名称，数据集结构/属性/文件大小/文件空洞，deduplication tables(though the deduplicated data itself is encrypted)）。
- 若要追求最佳性能，需要根据磁盘物理属性创建存储池时（甚至需要考虑物理块大小`ashift`）。
- ZFS的`aes`实现存在一些问题；此外，某些加密算法的实现可能存在性能问题。
- 使用`zvol`作为交换分区或者使用数据集中的文件作为交换文件存在问题。只能通过关闭休眠并设置交换分区到其它分区（详见下）解决。

虽然对于外界威胁，相比文件系统层面的加密，上述方案提供了好得多的保护。但也有一个共同的缺点：任何拥有密码的用户都能解密整个设备，进而访问其它用户的数据。若要防范上述风险，可以结合使用块设备加密和文件系统加密。可参见静态数据加密进行提前规划。

对于本文中所有方案的分区策略的概述，参见dm-crypt/准备磁盘#分区。

此外，还需考虑是否设置加密的swap分区及设置方式。详见Dm-crypt/交换分区加密。

加密整个系统可在设备失窃时保护数据，但不能抵御逻辑层面的非法修改（例如，系统上运行的恶意软件修改未加密的boot分区）。若有相关需求，还应参见dm-crypt/特殊应用#保护未加密的boot分区。

在使用固态硬盘时，可能需要启用TRIM支持。但这可能影响安全性，详见dm-crypt/特殊应用#固态硬盘的Discard/TRIM支持。

警告：

- 在任何情况下都不要直接在加密卷上运行文件系统修复工具（例如，fsck），否则，加密密钥将永久丢失，使得所有加密内容无法被解密。只应当在解密后的设备上运行文件系统修复工具。
- Argon2 密钥派生函数的设计方式使得其在运行时将占用大量内存（在默认设置下，每个加密卷将占用1GiB内存）。在设备内存较少或启动时并行解密多个LUKS2分区的情况下，可能导致错误。可使用`--pbkdf-memory`选项控制内存使用。
- GRUB对LUKS2的支持并不完善，详见GRUB#加密的/boot。对于需要由GRUB解密的分区，建议使用PBKDF2密钥派生函数（`cryptsetup luksFormat --pbkdf pbkdf2`）。
- 对于ZFS，即使交换分区并没有放置在zvol上，从休眠中唤醒时也有可能损坏存储池。因此在配置休眠时需要额外注意。参考

在单一分区上配置LUKS

本例说明了如何通过“dm-crypt”与LUKS，将安装在单一分区上的系统进行加密。

```
+-----------------------+-----------------------+-----------------------+
| boot分区              | LUKS加密的根分区      | 可选的用于其它分区的  |
|                       | partition             | 空闲空间              |
|                       |                       |                       |
| /boot                 | /                     |                       |
|                       |                       |                       |
|                       | /dev/mapper/root      |                       |
|                       |-----------------------|                       |
| /dev/sda1             | /dev/sda2             |                       |
<br>+-----------------------+-----------------------+-----------------------+<br>```
```

本例的前几步可在启动到Arch Linux安装镜像后直接进行。

准备磁盘

在创建分区前，应先查看[dm-crypt/准备磁盘](Dm-crypt_准备磁盘.md)，了解安全地擦除整个磁盘的重要性，并掌握相关方法。

接下来，创建本例中的分区（至少需要`/`（`/dev/sda2`）和`/boot`（`/dev/sda1`），详见[分区](../../02_系统管理与内核/系统恢复/分区.md)。

准备非boot分区

在格式化分区时，不要使用[安装指南](../../01_安装与包管理/安装过程/安装指南.md#格式化分区)中的步骤，而是使用本节及下一节的步骤。

使用如下命令创建并挂载加密的根分区，[dm-crypt/设备加密#使用 LUKS 模式加密设备](Dm-crypt_设备加密.md#使用_LUKS_模式加密设备)中详细介绍了进行的操作。若要使用非默认的加密选项（例如，修改加密算法，密钥长度，扇区大小），在执行如下第一条指令前参见相关[加密选项](Dm-crypt_设备加密.md#LUKS_模式的加密选项)。
在解密的LUKS设备上[创建文件系统](../../02_系统管理与内核/文件系统/文件系统.md)。例如，要创建[Ext4](../../02_系统管理与内核/文件系统/Ext4.md)文件系统，执行：
将根分区挂载到`/mnt`：
确保设备映射正常：
若还创建了其它分区（例如，`/home`），则对于**除了**`/boot`的其它分区，执行类似操作。在启动时解密其它分区的方法可参见[dm-crypt/加密非root文件系统#自动解锁并挂载](Dm-crypt_加密非root文件系统.md#自动解锁并挂载)。

注意，对于每个要加密并挂载的块设备，都需要单独设置密钥，这将导致在启动时需要多次输入密钥，造成不便。可通过在根分区中存储密钥文件，并通过修改`crypttab`设置，自动使用密钥文件解锁其它设备。详见[dm-crypt/设备加密#使用LUKS来格式化带有密钥文件的分区](Dm-crypt_设备加密.md#使用LUKS来格式化带有密钥文件的分区)。

准备boot分区

对于加密根分区的配置，需要设置非加密的`/boot`分区。

对于UEFI启动的系统，使用如下命令格式化新创建的[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)：

> **警告：**只有在EFI系统分区是在安装Arch Linux的分区步骤中创建的情况下，才应当格式化该分区。否则，若EFI系统分区是由其它系统创建的，则重新格式化将导致其它系统的引导程序丢失。
对于传统BIOS启动的系统：
之后，创建boot分区的挂载点并挂载分区：
挂载分区

在[安装指南#挂载分区](../../01_安装与包管理/安装过程/安装指南.md#挂载分区)步骤时，应当挂载`/dev/mapper/*`中的相应设备（包含LUKS加密的设备的解密后的版本），而不是LUKS加密的设备自身对应的分区。不过，对于未加密的`/boot`分区，仍应当直接挂载到`/mnt/boot`（假设在安装过程中，根分区被挂载到`/mnt`）。

配置mkinitcpio

在[安装指南#关于_initramfs](../../01_安装与包管理/安装过程/安装指南.md#关于_initramfs)步骤前，需要在新安装的系统中完成如下操作：

若使用基于busybox的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`和`encrypt`钩子。此外，若要使用非标准键盘布局，添加`keymap`钩子；若要自定义控制台字体，添加`consolefont`钩子。

HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt filesystems fsck)

若使用基于systemd的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`和`sd-encrypt`钩子。此外，若要使用非标准键盘布局，或要自定义控制台字体，添加`sd-vconsole`钩子。

```
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt filesystems fsck)
```

保存上述修改后，[重新生成initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md)。其它可能需要用到的钩子以及配置细节参见[Dm-crypt/系统配置#mkinitcpio](Dm-crypt_系统配置.md#mkinitcpio)。

配置引导加载程序

要在启动时解密已加密的根分区，需配置引导加载程序，设置如下[内核参数](../../02_系统管理与内核/内核/内核参数.md)：

```
cryptdevice=UUID=device-UUID:root root=/dev/mapper/root
```

若使用的是[sd-encrypt](Dm-crypt_系统配置.md)，则应该添加如下内核参数：

```
rd.luks.name=device-UUID=root root=/dev/mapper/root
```

`*device-UUID*`指的是LUKS超级块的UUID，在本例中为`/dev/sda2`对应的UUID。详见[块设备持久化命名](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md)。

另见[Dm-crypt/系统配置#内核参数](Dm-crypt_系统配置.md#内核参数)。

> **提示：**若根分区与boot分区在同一硬盘上，并且使用的引导加载程序支持[GPT分区自动挂载](../../01_安装与包管理/初始化/Systemd.md#GPT分区自动挂载)，则可以修改根分区的[分区类型GUID](https://en.wikipedia.org/wiki/GUID_Partition_Table#Partition_type_GUIDs)为"Root partition"并通过[systemd-gpt-auto-generator(8)](https://man.archlinux.org/man/systemd-gpt-auto-generator.8)自动挂载，而不必设置上述内核参数。

使用TPM2和安全启动在单一分区上配置

本例与[#在单一分区上配置LUKS](#在单一分区上配置LUKS)相似，但通过使用[安全启动](../../01_安装与包管理/启动过程/UEFI_安全启动.md)和[可信平台模块](../../04_硬件与外设/硬件/可信平台模块.md)，使启动过程更安全。

在本配置方案中，只有[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)是未加密的，存储经过签名以用于安全启动的[统一内核映像](../../01_安装与包管理/启动加载程序/统一内核映像.md)和[systemd-boot](../../01_安装与包管理/启动加载程序/Systemd-boot.md)。在本配置中，若安全启动功能被关闭，或者安全启动的密钥数据库被修改，则TPM将不会释放用于解密分区的密钥。此时，仍可以使用配置过程中创建的恢复密钥解密数据。此外，还可要求在解密数据前必须输入设置的TPM PIN码。本配置方案类似Windows中的BitLocker和macOS中的FileVault。

在继续之前，确保已经详细阅读了[可信平台模块#LUKS_数据加密](../../04_硬件与外设/硬件/可信平台模块.md#LUKS_数据加密)，并着重留意其中的警告。

本例中的分区按照符合[Systemd#GPT分区自动挂载](../../01_安装与包管理/初始化/Systemd.md#GPT分区自动挂载)的方式创建，因此无需fstab、crypttab文件。

```
+----------------------+--------------------------------+
```

EFI系统分区	LUKS加密的根分区	
/boot	/	
	/dev/mapper/root	

/dev/sda1	/dev/sda2	

EFI系统分区	LUKS加密的根分区	
		 +---------- +----------+ ```

按照安装指南#创建硬盘分区前的步骤进行基本配置。之后，按本文下一节的内容进行磁盘分区。

准备磁盘

在创建分区前，应先查看dm-crypt/准备磁盘，了解安全地擦除整个磁盘的重要性，并掌握相关方法。

使用GUID分区表 (GPT)，并创建需要的分区。

创建大小合适的EFI 系统分区（在本例中，使用`/dev/sda1`）。该分区将被挂载到`/boot`。

在磁盘的剩余空间上创建根分区（在本例中，使用`/dev/sda2`）。根分区将被加密，并挂载到`/`。将根分区的分区类型GUID设置为`4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709`（在fdisk中，设置为"Linux root (x86-64)"类型；在gdisk中，设置为`8304`类型）。

准备根分区

使用如下命令创建并挂载加密的根分区，dm-crypt/设备加密#使用 LUKS 模式加密设备中详细介绍了进行的操作。

若要使用非默认的加密选项（例如，修改加密算法，密钥长度，扇区大小），或者不想使用TPM解密，在执行如下第一条指令前参见相关加密选项。

创建并挂载LUKS卷（由于在配置完成后将擦除密码槽位，可以直接使用空密码）：

```
# cryptsetup luksFormat /dev/sda2
# cryptsetup open /dev/sda2 root
```

在解密的LUKS设备上创建文件系统。例如，要创建Ext4文件系统，执行：

```
# mkfs.ext4 /dev/mapper/root
```

将根分区挂载到 `/mnt`:

```
# mount /dev/mapper/root /mnt
```

准备EFI系统分区

格式化新创建的EFI系统分区，并挂载：

```
# mount --mkdir /dev/sda1 /mnt/boot
```

继续执行安装指南#关于_initramfs前的步骤。注意无需执行安装指南#生成_fstab_文件步骤。

配置mkinitcpio

修改mkinitcpio.conf中的HOOKS=为如下内容，以构建可用的基于systemd的initramfs：

```
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt filesystems fsck)
```

按照统一内核映像#mkinitcpio中的步骤配置统一内核映像。

由于/boot/EFI/Linux目录暂未由启动加载程序创建，暂时不要重新生成initramfs。

配置引导加载程序

可以直接通过UEFI启动内核，而无需引导加载程序。

若仍需使用引导加载程序，可以安装systemd-boot：

```
# bootctl install
```

systemd-boot可自动识别mkinitcpio生成的统一内核映像，无需在/boot/loader/entries/手动添加条目。

其它配置参见systemd-boot#更新 EFI 启动管理器、Systemd-boot#配置。

完成安装

重新生成initramfs，并确保相关镜像成功生成。

不要忘记设置 root 密码，重启以完成安装。

安全启动

为了启用安全启动，需要为引导加载程序、EFI镜像签名。可使用sbctl简单、快速地实现。

注册TPM

在签名引导加载程序并启用安全启动后，可以向TPM注册密钥用以解锁LUKS卷。如下命令将移创建LUKS卷时设置的空密码，创建绑定到TPM PCR 7（安全启动状态和已注册证书）寄存器的密钥，并创建恢复密钥，用于在TPM出现问题时解密卷。若安全启动链完好，TPM将自动释放密钥。详见systemd-cryptenroll#Trusted Platform Module、systemd-cryptenroll(1)。

```
# systemd-cryptenroll /dev/sda2 --recovery-key
# systemd-cryptenroll /dev/sda2 --wipe-slot=empty --tpm2-device=auto --tpm2-pcrs=7
```

提示：若要求在启动时需要输入PIN码解密卷，可添加--tpm2-with-pin=yes参数。

警告：

- 在绑定密钥到PCR7寄存器时，需要确保安全启动已启用且处于用户模式。否则，未授权的启动设备也可能解锁加密卷。
- 若固件证书状态改变，则PCR7寄存器状态也可能改变，导致解锁失败。例如，使用fwupd可能在不经意间改变固件证书状态；此外，轮转安全启动密钥也将改变固件证书状态。

在LUKS上配置LVM

在加密的单个分区上配置LVM是一种较为直接的方案。该方案将LVM设置在一个大的加密块设备中，LVM结构只在块设备解密，扫描并挂载LVM物理卷后才可见。

示例磁盘结构如下：

```
+-----------------------------------------------------------------------+
+-----------------+
| 逻辑卷 1                | 逻辑卷 2                | 逻辑卷 3                | | boot分区
|
|                          |                          |                          | |
|
| [SWAP]                   | /                        | /home                    | | /boot
|
|                          |                          |                          | |
|
| /dev/MyVolGroup/swap   | /dev/MyVolGroup/root   | /dev/MyVolGroup/home   | |
|
|_ _ _ _ _ _ _ _ _ _ _ _|_ _ _ _ _ _ _ _ _ _ _ _|_ _ _ _ _ _ _ _ _ _ _ _| | （可能在其他
|
|                                                                       | | 设备上）
|
|                            使用LUKS加密的分区                          | |
|
|                              /dev/sda1                                | | /dev/sdb1
|<br>+-----------------------------------------------------------------------+
+-----------------+<br>```
```

> **注意：**使用默认的`encrypt`钩子时，不能将逻辑卷分配到多个磁盘上。可使用[sd-encrypt](Dm-crypt_系统配置.md)钩子或参见[dm-crypt/特殊应用#修改encrypt钩子以用于多个分区](Dm-crypt_特殊应用.md#修改encrypt钩子以用于多个分区)。

> **提示：**该配置方案有两种变形：
>
> - [dm-crypt/特殊应用#在加密系统时使用分离的LUKS头](Dm-crypt_特殊应用.md#在加密系统时使用分离的LUKS头)介绍了如何将LUKS头单独存放于USB设备上，实现两步验证。
> - [dm-crypt/Specialties#Encrypted /boot and a detached LUKS header on USB](Dm-crypt_特殊应用.md#Encrypted_/boot_and_a_detached_LUKS_header_on_USB)则将LUKS头、加密的boot分区、加密的密钥文件都存放在USB设备上。

准备磁盘

在创建分区前，应先查看[dm-crypt/准备磁盘](Dm-crypt_准备磁盘.md)，了解安全地擦除整个磁盘的重要性，并掌握相关方法。

> **提示：**若要在BIOS启动的设备上使用[GPT](../../02_系统管理与内核/系统恢复/分区.md)磁盘，且使用[GRUB](../../01_安装与包管理/启动加载程序/GRUB.md)引导加载程序，需创建[BIOS boot partition](../../01_安装与包管理/启动加载程序/GRUB.md#GUID_分区表_(GPT)_特殊操作)。

[创建](../../01_安装与包管理/安装过程/安装指南.md#Partition_the_disks)一个大于1GiB的分区用于`/boot`。

> **提示：**对于UEFI启动的设备，可将[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)用于`/boot`。

创建一个分区，用于存储整个加密的LVM。

在上述创建的分区上配置LUKS作为LVM的容器（若要修改加密选项，参见[dm-crypt/设备加密#LUKS 模式的加密选项](Dm-crypt_设备加密.md#LUKS_模式的加密选项)）：
打开创建的容器：
现在可通过`/dev/mapper/cryptlvm`访问解密的容器。

准备逻辑卷

在解密的LUKS容器上创建物理卷：
创建卷组（本例中使用`MyVolGroup`，也可使用其它名称），并加入先前创建的物理卷：
在卷组中创建逻辑卷：

> **提示：**若需要使用[ext4](../../02_系统管理与内核/文件系统/Ext4.md)格式的逻辑卷，应在卷组中预留至少256MiB的空闲空间，以便能正常使用[e2scrub(8)](https://man.archlinux.org/man/e2scrub.8)。可以在使用`-l 100%FREE`创建占满所有剩余空间的最后一个逻辑卷后，使用类似`lvreduce -L -256M MyVolGroup/home`的命令减少该逻辑卷的大小。
在每个逻辑卷上[创建文件系统](../../02_系统管理与内核/文件系统/文件系统.md#创建文件系统)。例如，若根分区和home分区使用[Ext4](../../02_系统管理与内核/文件系统/Ext4.md)文件系统：
挂载文件系统：
准备boot分区

引导加载程序从`/boot`目录加载内核、[initramfs](../../01_安装与包管理/启动过程/Arch%20的启动流程.md)和自身的配置文件。可使用任何引导加载程序支持的文件系统。

为`/boot`分区创建[文件系统](../../02_系统管理与内核/文件系统/文件系统.md)。对于UEFI启动的系统，使用如下命令格式化新创建的[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)：

> **警告：**只有在EFI系统分区是在安装Arch Linux的分区步骤中创建的情况下，才应当格式化该分区。否则，若EFI系统分区是由其它系统创建的，则重新格式化将导致其它系统的引导程序丢失。
对于传统BIOS启动的系统：
将上述分区挂载到`/mnt/boot`：
继续执行[安装指南#开始安装系统](../../01_安装与包管理/安装过程/安装指南.md#开始安装系统)中的步骤。之后回到本页，执行修改后的[关于 initramfs](../../01_安装与包管理/安装过程/安装指南.md#关于_initramfs)、[安装引导程序](../../01_安装与包管理/安装过程/安装指南.md#安装引导程序)步骤。

配置mkinitcpio

确保已安装[lvm2](https://archlinux.org/packages/?name=lvm2)包。

若使用基于busybox的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`、`encrypt`、`lvm2`钩子。此外，若要使用非标准键盘布局，添加`keymap`钩子；若要自定义控制台字体，添加`consolefont`钩子。

```
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt lvm2 filesystems fsck)
```

若使用基于systemd的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`、`sd-encrypt`、`lvm2`钩子。此外，若要使用非标准键盘布局，或要自定义控制台字体，添加`sd-vconsole`钩子。

```
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt lvm2 filesystems fsck)
```

保存上述修改后，[重新生成initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md)。其它可能需要用到的钩子以及配置细节参见[Dm-crypt/系统配置#mkinitcpio](Dm-crypt_系统配置.md#mkinitcpio)。

注意：若使用[dracut](../../01_安装与包管理/Initramfs/Dracut.md)生成mkinitcpio，则其已经包含必要的模块，无需额外配置。

配置引导加载程序

要在启动时解密已加密的根分区，需配置引导加载程序设置如下[内核参数](../../02_系统管理与内核/内核/内核参数.md)：

cryptdevice=UUID=device-UUID:cryptlvm root=/dev/MyVolGroup/root

若使用的是[sd-encrypt](Dm-crypt_系统配置.md)，则应该添加如下内核参数：

rd.luks.name=device-UUID=cryptlvm root=/dev/MyVolGroup/root

`*device-UUID*`指的是LUKS超级块的UUID，在本例中为`/dev/sda1`对应的UUID。详见[块设备持久化命名](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md)。

若使用[dracut](../../01_安装与包管理/Initramfs/Dracut.md)，一般可使用如下参数：

rd.luks.uuid=device-UUID root=/dev/MyVolGroup/root

有时，需要设置更多参数：

rd.luks.uuid=luks-deviceUUID rd.lvm.lv=MyVolGroup/root rd.lvm.lv=MyVolGroup/swap root=/dev/mapper/MyVolGroup-root rootfstype=ext4 rootflags=rw,relatime

详见[Dm-crypt/系统配置#内核参数](Dm-crypt_系统配置.md#内核参数)。

在LVM上配置LUKS

为了在[LVM](../../02_系统管理与内核/存储虚拟化/LVM.md)上层设置加密，首先要设置LVM卷，用作加密分区的基础。在本方案中，可同时创建加密及未加密的分区。

> **提示：**与[#在LUKS上配置LVM](#在LUKS上配置LVM)不同，可以使用常规方式将逻辑卷分配到多个磁盘上。

下例展示了在LVM上配置LUKS加密卷，并使用密钥文件解锁home分区。对于swap分区，使用临时加密卷，以确保在每次重启后，都重新配置swap分区的加密，防止敏感信息泄露，提升安全性。若熟悉LVM配置，可自行修改LVM分区方案。

若要创建横跨多个磁盘的逻辑卷，或要将已有逻辑卷扩展到其它已配置好的磁盘上，详见[dm-crypt/特殊应用#将LVM拓展到多个磁盘上](Dm-crypt_特殊应用.md#将LVM拓展到多个磁盘上)。注意调整逻辑卷的大小后，LUKS加密容器的大小也需要调整。

准备磁盘

分区方案如下：

+---------------
+----------+

boot分区	dm-crypt plain加密的分区	LUKS加密的分区	LUKS加密的分区
/boot	[SWAP]	/	/home
	/dev/mapper/swap	/dev/mapper/root	/dev/mapper/home
	----------	----------	----------

boot 分区	dm-crypt plain加密的分区	LUKS加密的分区	LUKS加密的分区
	逻辑卷 1	逻辑卷 2	逻辑卷 3
	/dev/MyVolGroup/cryptswap	/dev/MyVolGroup/cryptroot	/dev/MyVolGroup/crypthome
	----------	----------	----------
/dev/sda1	/dev/sda2	 +---------- +----------+ ```	

按照dm-crypt/准备磁盘#dm-crypt 擦除空设备或分区随机擦除/dev/sda2。

准备逻辑卷

```
# pvcreate /dev/sda2
# vgcreate MyVolGroup /dev/sda2
# lvcreate -L 4G -n cryptswap MyVolGroup
# lvcreate -L 32G -n cryptroot MyVolGroup
# lvcreate -l 100%FREE -n crypthome MyVolGroup
# cryptsetup luksFormat /dev/MyVolGroup/cryptroot
# cryptsetup open /dev/MyVolGroup/cryptroot root
```

在解密后的设备上创建文件系统，并挂载到对应挂载点。例如，要创建Ext4文件系统：

```
# mkfs.ext4 /dev/mapper/root
# mount /dev/mapper/root /mnt
```

关于加密选项的更多信息，详见dm-crypt/设备加密#LUKS 模式的加密选项。

注意/home的加密会在#加密/home逻辑卷步骤中进行。

提示：若要在Arch ISO中访问加密的根文件系统，可在LVM逻辑卷出现后执行上述open操作。

准备boot分区

为/boot分区创建文件系统。对于UEFI启动的系统，使用如下命令格式化新创建的EFI 系统分区：

警告：只有在EFI系统分区是在安装Arch Linux的分区步骤中创建的情况下，才应当格式化该分区。否则，若EFI系统分区是由其它系统创建的，则重新格式化将导致其它系统的引导程序丢失。

```
# mkfs.fat -F32 /dev/sda1
```

对于传统BIOS启动的系统：

```
# mkfs.ext4 /dev/sda1
```

之后，创建boot分区的挂载点并挂在分区：

```
# mount --mkdir /dev/sda1 /mnt/boot
```

配置mkinitcpio

确保已安装lvm2包。

若使用基于busybox的initramfs，在mkinitcpio.conf中添加keyboard、encrypt、lvm2钩子。此外，若要使用非标准键盘布局，添加keymap钩子；若要自定义控制台字体，添加consolefont钩子。

```
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block lvm2 encrypt filesystems fsck)
```

若使用基于systemd的initramfs，在mkinitcpio.conf中添加keyboard、sd-encrypt、lvm2钩子。此外，若要使用非标准键盘布局，或要自定义控制台字体，添加sd-vconsole钩子。

```
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt lvm2 filesystems fsck)
```

保存上述修改后，重新生成initramfs。其它可能需要用到的钩子以及配置细节参见Dm-crypt/系统配置#mkinitcpio。

配置引导加载程序

要在启动时解密已加密的根分区，需配置引导加载程序设置如下内核参数：

```
cryptdevice=UUID=device-UUID:root root=/dev/mapper/root
```

若使用的是sd-encrypt，则应该添加如下内核参数：

```
rd.luks.name=device-UUID=root root=/dev/mapper/root
```

*device-UUID*指的是LUKS超级块的UUID，在本例中为/dev/MyVolGroup/cryptroot对应的UUID。详见块设备持久化命名。

若使用的是dracut，需要一系列更复杂的参数：

```
kernel_cmdline="rd.luks.uuid=luks-deviceUUID rd.lvm.lv=MyVolGroup/root rd.lvm.lv=MyVolGroup/swap  root=/dev/mapper/MyVolGroup-root rootfstype=ext4 rootflags=rw,relatime"
```

详见Dm-crypt/系统配置#内核参数。

配置fstab和crypttab

crypttab用于解密设备，fstab用于挂载文件系统。如下配置将在每次重启时用不同的密钥重新加密swap分区：

```
/etc/crypttab
swap   /dev/MyVolGroup/cryptswap   /dev/urandom  swap,cipher=aes-xts-plain64,size=256,sector-size=4096
/etc/fstab
/dev/mapper/root                            /     ext4   defaults  0 1
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /boot ext4   defaults  0 2
/dev/mapper/swap                            none  swap   defaults  0 0
```

加密/home逻辑卷

由于本方案使用LVM作为第一层，dm-crypt作为第二层，每个加密逻辑卷都需要单独进行加密解密配置。此外，显然，和临时文件系统可采用临时随机密钥加密不

同，/home对应的逻辑卷应该采用持久不变的密钥。下列命令假定你已经重启进入了安装好的系统。否则，需要调整相应路径。

为了避免在系统启动时输入两次密码，创建密钥文件：

```
# dd bs=512 count=4 if=/dev/random iflag=fullblock | install -m
0600 /dev/stdin /etc/cryptsetup-keys.d/home.key
```

/home逻辑卷将采用上述密钥文件加密：

```
# cryptsetup luksFormat -v /dev/MyVolGroup/crypthome /etc/cryptsetup-keys.d/home.key
# cryptsetup -d /etc/cryptsetup-keys.d/home.key open /dev/MyVolGroup/crypthome home
```

在解密后的设备上创建文件系统，并挂载到对应挂载点。例如，要创建Ext4文件系统：

```
# mkfs.ext4 /dev/mapper/home
# mount /dev/mapper/home /home
```

解密并挂载该文件系统需要配置crypttab和fstab：

```
/etc/crypttab
home    /dev/MyVolGroup/crypthome    none
/etc/fstab
/dev/mapper/home         /home    ext4         defaults         0        2
```

在软RAID上配置LUKS

下例基于真实的工作站笔记本设置，该工作站有两块大小相同的SSD及一块大容量机械硬盘。在本配置中，对所有磁盘采用LUKS加密（包括/boot）,并将两块SSD组成了RAID0阵列。系统启动时，在GRUB中输入正确的密码后，将用解密出的密钥文件解密所有分区。

本方案采用的分区配置非常简单，组成的RAID阵列挂载在/（没有单独的/boot 分区），解密后的机械硬盘挂载在/data。

注意对于本方案，定期的备份非常重要。若两块SSD中的任何一块损坏，整个RAID阵列中的数据都将丢失。若要求较高的容错能力，应选择其它RAID级别。

本配置中的加密方式不是可抵赖加密。

在之后的命令中，假定采用如下块设备设置：

```
/dev/sda = 第一块SSD
/dev/sdb = 第二块SSD
/dev/sdc = 机械硬盘
+---------------------+-------------------------+-------------------------+
+---------------------+-------------------------+-------------------------+
+-------------------------+
| BIOS boot分区        | EFI系统分区                | LUKS加密的分区             | | BIOS 启动分
区        | EFI系统分区                | LUKS加密的分区             | | LUKS加密的分区
|
|                       |                           |                           | |
|                            |                           | |                           |
|                       | /efi                      | /                         | |
| /efi                       | /                         | | /data                     |
```

```
|                       |                            |                              | |
|                       |                            |                              | |
|                       |                            | /dev/mapper/root             | |
|                       | /dev/mapper/root           |                              | |
|                       +----------------------------+------------------------------+ |
+---------------------------+----------------------------+ |                          |
|                       | RAID1 array (part 1 of 2) | RAID0 array (part 1 of 2) | |
| RAID1 array (part 2 of 2) | RAID0 array (part 2 of 2) | |                          |
|                       |                            |                              | |
|                       |                            |                              | |
|                       | /dev/md/ESP                | /dev/md/root                 | |
| /dev/md/ESP               | /dev/md/root               | | /dev/mapper/data           |
|                       +----------------------------+------------------------------+ |
+---------------------------+----------------------------+ +----------------------------+
| /dev/sda1             | /dev/sda2                  | /dev/sda3                    |
| /dev/sdb1             | /dev/sdb2                  | /dev/sdb3                    |
| /dev/sdc1                 |<br>+---------------------+-----------------------------
+---------------------------+ +---------------------+-----------------------------
+---------------------------+ +---------------------------+<br>```
```

在之后的操作中，需要根据实际情况替换上述配置。

准备磁盘

在创建分区前，应先查看[dm-crypt/准备磁盘](Dm-crypt_准备磁盘.md)，了解安全地擦除整个磁盘的重要性，并掌握相关方法。

对于使用GPT分区表的BIOS启动的系统，需创建一个大小为1MiB的[BIOS启动分区](../../01_安装与包管理/启动加载程序/GRUB.md#GUID_分区表_(GPT)_特殊操作)，GRUB将在此存储第二阶段的BIOS引导加载程序。不要挂载该分区。

对于[UEFI启动的系统](../../01_安装与包管理/启动加载程序/GRUB.md#UEFI_系统)，需创建大小适当的[EFI系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)，并挂载到`/efi`。

对于设备上的剩余空间，创建一个用于"Linux RAID"的分区（本例中为`/dev/sda3`）。对于MBR分区表，选择`fd`分区类型ID；对于GPT分区表，选择`A19D880F-05FC-4D3B-A006-743F0F84911E`分区类型GUID。

一旦在`/dev/sda`上创建好了分区，可使用下列命令将它们的配置克隆到`/dev/sdb`上：
对于机械硬盘，创建单个占据整个磁盘的Linux分区（`/dev/sdc1`）。

构建RAID阵列

创建用于两块SSD的RAID阵列：

> **注意：**
>
> - 对于两块SSD，EFI系统分区必须能在每块SSD上单独访问，因此只能存放在RAID1上。
> - 必须通过`--metadata=1.0`选项将RAID superblock放置在EFI系统分区的后部，否则固件将不能访问EFI系统分区。

对于根分区，本例采用了RAID0。可根据实际偏好与需要选择其它RAID级别。
准备块设备

[dm-crypt/准备磁盘](Dm-crypt_准备磁盘.md)中介绍了通过使用随机密钥创建加密卷，并向加密卷中写入`/dev/zero`产生的全零数据来随机擦除设备的方法。此外，也可使用`dd`与`/dev/random`或`/dev/urandom`来擦除，不过速度较慢。
对本例中的机械硬盘（`/dev/sdc1`）也需重复上述操作。

为`/dev/md/root`设置加密：

> **警告：**GRUB对LUKS2的支持并不完善，详见[GRUB#加密的/boot](../../01_安装与包管理/启动加载程序/GRUB.md#加密的/boot)。对于需要由GRUB解密的分区，建议使用PBKDF2密钥派生函数（`cryptsetup luksFormat --pbkdf pbkdf2`）。

在解密后的设备上[创建文件系统](../../02_系统管理与内核/文件系统/文件系统.md)。例如，要创建[Ext4](../../02_系统管理与内核/文件系统/Ext4.md)文件系统：

将根分区挂载到`/mnt`：

对机械硬盘，类似的：

对于UEFI启动的系统，还需设置EFI系统分区：

> **警告：**只有在EFI系统分区是在安装Arch Linux的分区步骤中创建的情况下，才应当格式化该分区。否则，若EFI系统分区是由其它系统创建的，则重新格式化将导致其它系统的引导程序丢失。

配置GRUB

对于本例中通过LUKS加密的系统，编辑`/etc/default/grub`以配置[GRUB](../../01_安装与包管理/启动加载程序/GRUB.md)：

GRUB_CMDLINE_LINUX="cryptdevice=/dev/md/root:root"
GRUB_ENABLE_CRYPTODISK=y

对于较新的设备，可能需要使用USB键盘在GRUB中输入密码。这需要启用固件中的"legacy USB support"，或在`/etc/default/grub`中添加：

GRUB_TERMINAL_INPUT="usb_keyboard" GRUB_PRELOAD_MODULES="usb usb_keyboard ohci uhci ehci"

否则在提示输入LUKS密码时可能无法使用键盘。详见[Dm-crypt/系统配置#内核参数](Dm-crypt_系统配置.md#内核参数)、[GRUB#加密的/boot](../../01_安装与包管理/启动加载程序/GRUB.md#加密的/boot)。

将GRUB安装到两块SSD上（事实上，仅安装到`/dev/sda`也是可行的）：

配置密钥文件

进行如下步骤可避免在启动时需输入两次密码（第一次发生在GRUB解密LUKS设备，第二次发生在initramfs接管系统时）。首先要创建[密钥文件](Dm-crypt_设备加密.md#密钥文件)，然后将该密钥文件加入到initramfs镜像中以便encrypt钩子解密根分区。详见[dm-crypt/设备加密#在_initramfs_中嵌入密钥文件](Dm-crypt_设备加密.md#在_initramfs_中嵌入密钥文件)。

- 创建[密钥文件](Dm-crypt_设备加密.md#密钥文件)并向其中添加`/dev/md/root`的密码。
- 创建另一个密钥文件以便于在启动时解密机械硬盘（`/dev/sdc1`）。为了便于以后可能的需要恢复的情况，不要移动/删除上述密钥文件。接下来编辑`/etc/crypttab`以在启动时解密机械硬盘。详见[Dm-crypt/系统配置#使用密钥文件解密](Dm-crypt_系统配置.md#使用密钥文件解密)。

配置系统

配置[fstab](../../01_安装与包管理/启动过程/Fstab.md)以挂载根分区、data分区、EFI系统分区：

/dev/mapper/root / ext4 rw,noatime 0 1 /dev/mapper/data /data ext4 defaults 0 2 /dev/md/ESP /efi vfat rw,relatime,codepage=437,iocharset=iso8859-1,shortname=mixed,utf8,tz=UTC,errors=remount-ro 0 2

保存RAID配置：

编辑[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md#配置)，包含密钥文件，并添加所需的钩子：

FILES=(/crypto_keyfile.bin) HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block mdadm_udev encrypt filesystems fsck)

详见[Dm-crypt/系统配置#mkinitcpio](Dm-crypt_系统配置.md#mkinitcpio)。

Plain dm-crypt

与LUKS不同，dm-crypt的*plain*模式无需在加密设备前附加LUKS头，本方案利用该特点在未分区的硬盘上设置加密的系统。对于他人，将无法区分用该方式加密的硬盘与与用随机数据填充的硬盘，即所谓[可抵赖加密](https://en.wikipedia.org/wiki/Deniable_encryption)。详见[wikipedia:Disk encryption#Full disk encryption](https://en.wikipedia.org/wiki/Disk_encryption#Full_disk_encryption)。

注意，除非需要将整个硬盘全部加密，否则，无论是加密整个系统还是部分分区，使用之前小节中介绍的基于LUKS的方法都是更好的选择。因为在*plain*模式中，无法使用[LUKS特有的功能](Dm-crypt_设备加密.md#专门针对LUKS的Cryptsetup操作)，例如支持多个密码、密钥文件的密钥管理，主密钥备份，原位重新加密。

由于不依赖主密钥，可避免主密钥损坏或被销毁导致的单点故障，*plain*模式对数据损坏的承受能力更强。此外，若尤为在意[对SSD的TRIM支持](Dm-crypt_特殊应用.md#固态硬盘的Discard/TRIM支持)，也可以使用*plain*模式。然而，在*plain*模式下，要到达与LUKS模式相同的加密强度，需要手动配置加密选项。详见[静态数据加密#加密的元数据](静态数据加密.md#加密的元数据)。

> **提示：**若需要使用无LUKS头的加密但仍想使用KDF的相关功能，可尝试如下两种替代方案：
>
> - 通过*cryptsetup*的`--header`选项[使用分离的LUKS头](Dm-crypt_特殊应用.md#在加密系统时使用分离的LUKS头)。注意若使用该方案则不能使用标准的"encrypt"钩子，不过可对该钩子进行修改。
> - tcplay 在无LUKS头加密模式下提供了PBKDF2函数。

本方案使用了两个U盘：

- 一个用于引导设备，同时存储用于解密plain加密设备的配置选项；
- 一个用于密钥文件，将按原始比特位存储，故不了解加密方案的攻击者只能看到一串仿佛随机的数据，而非可见的普通文件。详见[Wikipedia:Security through obscurity](https://en.wikipedia.org/wiki/Security_through_obscurity)。要准备密钥文件，参见[dm-crypt/Device encryption#Keyfiles](Dm-crypt_设备加密.md#Keyfiles)。

磁盘布局如下：

```
+--------------------+--------------------+--------------------+ +---------------+ +---------------+
```

提示：

- 也可以只使用一个U盘，如下为两种方式：
- 在U盘（/dev/sdb）的另一个分区（/dev/sdb2）上存储密钥。
- 直接复制密钥文件到initramfs中。例如，在/etc/mkinitcpio.conf中设置FILES=(/etc/cryptsetup-keys.d/root.key)将复制/etc/cryptsetup-keys.d/root.key密钥文件到initramfs镜像中。要使得encrypt钩子从initramfs镜像中读取密钥文件，在文件名前添加rootfs:前缀，例如，cryptkey=rootfs:/etc/cryptsetup-keys.d/root.key。
- 此外，也可使用足够安全的密码解密。

准备磁盘

向解密后的映射设备中填充随机数据是非常重要的。尤其是在本例中。

详见dm-crypt/准备磁盘、dm-crypt/准备磁盘#dm-crypt_专用方案

准备分区（除boot）

详见dm-crypt/设备加密#plain模式的加密选项。

在本例中，使用/dev/sda，并使用aes-xts加密算法，密钥长度为512位，并使用密钥文件加密：

```
 # cryptsetup open --type plain --cipher=aes-xts-plain64 --offset=0 --key-file=/dev/sdc --key-size=512 --sector-size 4096 /dev/sda cryptlvm
```

与使用LUKS不同，每次要解密设备时必须使用上述“完整”的命令，故一定要牢记加密算法、密钥文件相关信息。

检查有关/dev/mapper/cryptlvm的映射条目已建立：

```
# fdisk -l
```

提示：

- 若无需使用LVM提供的功能，cryptsetup FAQ提倡直接在解密后的设备上创建文件系统，而不创建LVM。
- 若LVM卷组中包含ext4文件系统的逻辑卷，为了便于使用e2scrub(8)工具，至少在卷组中保留256MiB的空闲空间。若使用-l 100%FREE参数创建了最后一个逻辑卷，可以使用lvreduce -L -256M MyVolGroup/home留出256MiB的空闲空间。

接下来，在解密后产生的映射设备上设置LVM逻辑卷，详见Install Arch Linux on LVM：

```
# pvcreate /dev/mapper/cryptlvm
# vgcreate MyVolGroup /dev/mapper/cryptlvm
# lvcreate -L 32G MyVolGroup -n root
# lvcreate -L 4G MyVolGroup -n swap
# lvcreate -l 100%FREE MyVolGroup -n home
```

格式化并挂载上述分区并激活swap，详见文件系统#创建文件系统。

```
# mkfs.ext4 /dev/MyVolGroup/root
# mkfs.ext4 /dev/MyVolGroup/home
# mount /dev/MyVolGroup/root /mnt
# mount --mkdir /dev/MyVolGroup/home /mnt/home
# mkswap /dev/MyVolGroup/swap
# swapon /dev/MyVolGroup/swap
```

准备boot分区

可直接将单独的U盘（通常使用FAT32文件系统）作为/boot分区。或者，若要手动分区，使用分区工具创建大小为1GiB的小分区，并建立文件系统。

警告：只有在EFI系统分区是在安装Arch Linux的分区步骤中创建的情况下，才应当格式化该分区。否则，若EFI系统分区是由其它系统创建的，则重新格式化将导致其它系统的引导程序丢失。

```
# mkfs.fat -F32 /dev/sdb1
# mount --mkdir /dev/sdb1 /mnt/boot
```

配置mkinitcpio

确保已安装lvm2包。

若使用基于busybox的initramfs，在mkinitcpio.conf中添加keyboard、encrypt、lvm2钩子。此外，若要使用非标准键盘布局，添加keymap钩子；若要自定义控制台字体，添加consolefont钩子。

```
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt lvm2 filesystems fsck)
```

保存上述修改后，重新生成initramfs。其它可能需要用到的钩子以及配置细节参见Dm-crypt/系统配置#mkinitcpio。

配置引导加载程序

要从加密的根分区启动，需通过引导加载程序设置如下内核参数（注意512位对应64字节）：

```
cryptdevice=/dev/disk/by-id/disk-ID-of-sda:cryptlvm:sector-size=4096
cryptkey=/dev/disk/by-id/disk-ID-of-sdc:0:64 crypto=:aes-xts-plain64:512:0:
```

*disk-ID-of-**disk***表示相应磁盘的id，详见块设备持久化命名。

对于其它可能需要的内核参数，详见Dm-crypt/系统配置#内核参数。

提示：若使用GRUB，可将其安装到/boot分区所在的U盘上。

对于BIOS启动的系统：

```
# grub-install --target=i386-pc --recheck /dev/sdb
```

对于UEFI启动的系统：

```
# grub-install --target=x86_64-efi --efi-directory=/boot --removable
```

安装完成后的操作

或许希望在系统启动后移除启动用的U盘。由于/boot分区并不常用，可在/etc/fstab的相应行中添加noauto选项：

```
/etc/fstab
# /dev/sdb1
UUID=XXXX-XXXX /boot vfat noauto,rw,noatime 0 2
```

然而，若需要更新initramfs、内核、引导加载程序使用的相关文件时，则必须挂载/boot分区。由于fstab中存在相关条目，只需执行如下命令即可挂载：

```
# mount /boot
```

加密的boot分区(GRUB)

本配置使用与#在LUKS上配置LVM相同的分区结构和配置。但使用了GRUB引导加载程序，其能从LVM逻辑卷和LUKS加密的/boot上启动系统。详见GRUB#加密的/boot

本例中的磁盘结构如下：

```
+---------------------+----------------------+----------------------
+----------------------+----------------------+
| BIOS boot分区        | EFI系统分区           | 逻辑卷 1              | 逻辑卷 2
| 逻辑卷 3              |
```

```
|                       |                       |                       |
|                       |                       |                       |
|                       | /efi                  | /                     | [SWAP]
| /home                 |                       |                       |
|                       |                       |                       |
|                       |                       |                       |
|                       |                       | /dev/MyVolGroup/root
| /dev/MyVolGroup/swap  | /dev/MyVolGroup/home  |
| /dev/sda1             | /dev/sda2             |-----------------------
+-----------------------+-----------------------+
| 未加密                | 未加密                | /dev/sda3 使用LUKS加密并配置LVM
|<br>+--------------------+-----------------------
+-------------------------------------------------------------------------+<br>```

> **提示：**
>
> - 所有提供的案例仅用作示例。可按需与其它方案结合使用。此外，可参见[#在LUKS上配置LVM](#在LUKS上配置LVM)中介绍的两种变形。
> - 可以使用[cryptboot](https://aur.archlinux.org/packages/cryptboot/)AUR中的`cryptboot`脚本简化加密boot分区的管理（包括挂载、卸载、软件包更新）。此外，该脚本与[UEFI安全启动](../../01_安装与包管理/启动过程/UEFI_安全启动.md#使用自己的密钥)配合还可抵御[邪恶女佣](https://www.schneier.com/blog/archives/2009/10/evil_maid_attac.html)攻击。更多信息及存在的问题见[cryptboot project](https://github.com/kmille/cryptboot)。

### 准备磁盘

在创建分区前，应先查看[dm-crypt/准备磁盘](Dm-crypt_准备磁盘.md)，了解安全地擦除整个磁盘的重要性，并掌握相关方法。

对于[UEFI启动的系统](../../01_安装与包管理/启动加载程序/GRUB.md#UEFI_系统)，创建大小合适的[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)，将挂载该分区到`/efi`。

对于[BIOS + GPT启动的系统](../../01_安装与包管理/启动加载程序/GRUB.md#GUID_分区表_(GPT)_特殊操作)，创建大小为1MiB的[BIOS启动分区](../../01_安装与包管理/启动加载程序/GRUB.md#GUID_分区表_(GPT)_特殊操作)，GRUB将在该分区中存储第二阶段引导加载程序，不要手动挂载该分区。

对于BIOS + MBR启动的系统，无需创建BIOS启动分区。

创建一个类型为`8309`的分区用于加密的LVM。

创建LUKS加密容器：

> **警告：**GRUB对LUKS2的支持并不完善，详见[GRUB#加密的/boot](../../01_安装与包管理/启动加载程序/GRUB.md#加密的/boot)。对于需要由GRUB解密的分区，建议使用PBKDF2密钥派生函数（`cryptsetup luksFormat --pbkdf pbkdf2`）。
> 关于加密选项的更多信息，在执行上述命令前参见[dm-crypt/设备加密#LUKS 模式的加密选项](Dm-crypt_设备加密.md#LUKS_模式的加密选项)。

分区结构应与下述相似：

... Number Start (sector) End (sector) Size Code Name 1 2048 4095 1024.0 KiB EF02 BIOS boot partition 2 4096 2101247 1024.0 MiB EF00 EFI system partition 3 2101248 69210111 32.0 GiB 8309 Linux LUKS

打开LUKS加密容器：
解密后的容器将在`/dev/mapper/cryptlvm`。

### 准备逻辑卷
```

本例中逻辑卷的结构与[#在LUKS上配置LVM](#在LUKS上配置LVM)中的相同。因此，只需遵循[#准备逻辑卷](#准备逻辑卷)中的步骤并按需调整。

对于UEFI启动的系统，将[EFI 系统分区](../../01_安装与包管理/启动过程/EFI%20系统分区.md)挂载到`/efi`，以与接下来的`grub-install`指令相匹配。
现在，在`/mnt`下应挂载有如下分区与逻辑卷：
$ lsblk

NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT sda 8:0 0 200G 0 disk ├─sda1 8:1 0 1M 0 part ├─sda2 8:2 0 550M 0 part /mnt/efi └─sda3 8:3 0 100G 0 part └─cryptlvm 254:0 0 100G 0 crypt ├─MyVolGroup-swap 254:1 0 4G 0 lvm [SWAP] ├─MyVolGroup-root 254:2 0 32G 0 lvm /mnt └─MyVolGroup-home 254:3 0 60G 0 lvm /mnt/home

继续执行[安装指南#开始安装系统](../../01_安装与包管理/安装过程/安装指南.md#开始安装系统)及之后的步骤。请使用本文中的对应内容替换安装指南的[Initramfs](../../01_安装与包管理/安装过程/安装指南.md#关于_initramfs)、[安装引导加载程序](../../01_安装与包管理/安装过程/安装指南.md#安装引导程序)步骤。

配置mkinitcpio

确保已安装[lvm2](https://archlinux.org/packages/?name=lvm2)包。

若使用基于busybox的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`、`encrypt`、`lvm2`钩子。此外，若要使用非标准键盘布局，添加`keymap`钩子；若要自定义控制台字体，添加`consolefont`钩子。
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt lvm2 filesystems fsck)

若使用基于systemd的initramfs，在[mkinitcpio.conf](../../01_安装与包管理/Initramfs/Mkinitcpio.md)中添加`keyboard`、`sd-encrypt`、`lvm2`钩子。此外，若要使用非标准键盘布局，或要自定义控制台字体，添加`sd-vconsole`钩子。
HOOKS=(base systemd autodetect microcode modconf kms keyboard sd-vconsole block sd-encrypt lvm2 filesystems fsck)

保存上述修改后，[重新生成initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md)。其它可能需要用到的钩子以及配置细节参见[Dm-crypt/系统配置#mkinitcpio](Dm-crypt_系统配置.md#mkinitcpio)。

配置GRUB

配置GRUB以从LUKS加密的`/boot`启动：
/etc/default/grub

GRUB_ENABLE_CRYPTODISK=y

设置内核参数以便initramfs解密加密的根分区。 若使用`encrypt`钩子：
/etc/default/grub

GRUB_CMDLINE_LINUX="... cryptdevice=UUID=device-UUID:cryptlvm ..."

若使用[sd-encrypt](Dm-crypt_系统配置.md)钩子：
/etc/default/grub

GRUB_CMDLINE_LINUX="... rd.luks.name=device-UUID=cryptlvm ..."

详见[Dm-crypt/系统配置#内核参数](Dm-crypt_系统配置.md#内核参数)、[GRUB#加密的/boot](../../01_安装与包管理/启动加载程序/GRUB.md#加密的/boot)。`*device-UUID*`应为LUKS superblock的UUID（在本例中为`/dev/sda3`，该分区包含了含有根分区的LVM）。详见[块设备持久化命名](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md)。

对于UEFI启动的系统，[安装GRUB](../../01_安装与包管理/启动加载程序/GRUB.md#安装)到挂载的EFI系统分区：
对于BIOS启动的系统，[安装GRUB](../../01_安装与包管理/启动加载程序/GRUB.md#安装_2)：
生成GRUB[配置文件](../../01_安装与包管理/启动加载程序/GRUB.md#生成主配置文件)：
若上述所有命令都成功执行，在下次重启后GRUB应提示输入密码解锁`/dev/sda3`。

避免需输入两次密码

尽管在上述操作后，GRUB会要求输入密码解密LUKS加密的分区，但密码并不会传递到initramfs中。因此，需要在启动时输入两次密码：一次用于GRUB，一次用于initramfs。

可通过[在initramfs中嵌入密钥文件](Dm-crypt_设备加密.md#在_initramfs_中嵌入密钥文件)避免输入两次密码。

首先，创建一个密钥文件，将其添加到LUKS key中：
将密钥文件添加到initramfs镜像：
/etc/mkinitcpio.conf

FILES=(/etc/cryptsetup-keys.d/cryptlvm.key)

重新生成initramfs镜像。

设置如下内核参数以通过密钥文件解密LUKS分区。 若使用`encrypt`钩子：
GRUB_CMDLINE_LINUX="... cryptkey=rootfs:/etc/cryptsetup-keys.d/cryptlvm.key"

若使用[sd-encrypt](Dm-crypt_系统配置.md)钩子，将默认使用`/etc/cryptsetup-keys.d/*name*.key`密钥文件，因此无需手动设置内核参数。

若由于某些原因密钥文件解密boot分区失败，systemd会要求手动输入密码。输入正确的密码后，系统启动将正常进行。

> **提示：**若想加密`/boot`分区以抵御离线攻击`/boot`分区的威胁，可使用[dm-crypt/特殊应用#mkinitcpio-chkcryptoboot](Dm-crypt_特殊应用.md#mkinitcpio-chkcryptoboot)钩子。

使用USB设备解锁 /boot

使用存储在单独USB设备上的密钥文件可避免使用密码（若密码太复杂，则难以记忆；若太简单，则容易被猜中）。从安全性角度考虑，在未使用时，存储有密钥文件的USB设备必须与需解密的设备分开保存。

首先，按照[#避免需输入两次密码](#避免需输入两次密码)中的步骤创建新的密钥文件。不要复用initramfs中使用的密钥文件，否则，若USB设备丢失或密钥泄露，需要修改嵌入到initramfs中的密钥文件。

将创建的密钥文件复制到USB设备，并创建GRUB配置文件：
/boot/grub/grub-pre.cfg

set crypto_uuid=UUID-of-the-luks-volume set key_disk=UUID-of-the-volume-with-the-key cryptomount -u $crypto_uuid -k ($key_disk)/the-location-of-the-key-on-your-usb set root=UUID-of-the-unlocked-volume-as-in-grub.cfg set prefix=($root)/boot/grub insmod normal normal

创建并安装GRUB镜像（依文件系统设置而定，可能不需要在镜像中包含某些模块）：

Root on ZFS

要将dm-crypt与[ZFS](../../02_系统管理与内核/文件系统/ZFS.md)一同使用，参见[ZFS#Encryption in ZFS using dm-crypt](../../02_系统管理与内核/文件系统/ZFS.md#Encryption_in_ZFS_using_dm-crypt)。

此外，ZFS具有[原生加密](../../02_系统管理与内核/文件系统/ZFS.md#Native_encryption)支持，可用于加密根分区（不包含引导加载程序和文件系统元数据）。详见：

- OpenZFS上的[Arch Linux Root on ZFS](https://openzfs.github.io/openzfs-docs/Getting%20Started/Arch%20Linux/Root%20on%20ZFS.html)指南。
- [Install Arch Linux on ZFS#Installation](../../01_安装与包管理/安装过程/在%20ZFS%20上安装%20Arch%20Linux.md#Installation).

对于UEFI启动的系统，在安装后可通过[安全启动](../../01_安装与包管理/启动过程/UEFI_安全启动.md)验证引导加载程序。

沙箱

把程序关进笼子里运行，是本章反复出现的同一件事：限制它能看到哪些文件、能拿到哪些能力（Capabilities）、能碰哪些内核接口。代价差别很大，轻则给一个 systemd 服务加几行沙箱指令，重则起一整套容器运行时。

建议先按序读 Systemd/沙箱 与 Bubblewrap，紧接着读 Bubblewrap/示例，它用 dhcpcd、MuPDF、Firefox、Chromium 等演示规则怎么从收紧调到真能用；Firejail 可作同一思路的另一实现对照看。Docker 与 Podman 选读其一，Linux 容器与 Systemd-nspawn 篇幅最长，不做容器运维可以跳过。最容易误解的是把沙箱当成装好即生效的开关：规则松了形同虚设，紧了程序能启动却在运行中失能，Xorg 程序与浏览器是重灾区。

3.1　Systemd/沙箱

难度 进阶 | **篇幅** 12.6 KB | **所属** 沙箱

本节将讲到

- 概述
- 能力 (Capabilities)
- 常见指令
- 进阶指令
- system.conf

翻译状态：

- 本文（或部分内容）译自 Systemd/Sandboxing，最近一次同步于 2026-10-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/Sandboxing_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

systemd 允许用户加固并把系统服务单元放到沙箱中运行。由于技术限制，以及讽刺的是，出于安全方面的考量，用户服务单元不能被加固或放到沙箱中，因为这可能导致权限提升相关的问题。不过这不会影响使用 `User=` 的系统单元。

提示：Firejail 能以显式允许的，低权限的用户运行沙箱中的应用，不过它本身是一个 SUID 程序，因此从理论上来说，它有可能受权限提升漏洞影响。

从传统意义上来说，只有服务单元可以被加固或沙箱化处理。详情请参考 systemd.exec(5) 以及 systemd.resource-control(5)。

概述

由于加固或沙箱化处理会限制应用行为，因此不可能应用所有沙箱化指令。例如网络服务器不应该使用 `PrivateNetwork=true`，因为它一般需要网络访问权限。

可以使用 `systemd-analyze security *unit*` 来列出所有沙箱化指令并生成一个评分，有助于帮您决定接下来可以尝试开启什么设置。

注意：评分只用于参考。一个简单的 `Hello world` 可以获得一个接近完美的分数。没有应用可以应用*所有*沙箱化设置。

不幸的是，systemd 在沙箱化相关的配置有误时给出的错误信息有时是模糊的，甚至具有误导性。临时将日志等级设为 `debug` 可以获得真正有价值的信息。

```
# systemctl log-level debug
```

能力 (Capabilities)

capabilities(7) 可用于授予进程特定的特权。例如，`CAP_NET_BIND_SERVICE` 可以让非特权进程绑定低于 1024 的端口，从而避免以 root 特权启动进程。另一个显著的例子是可以授予备份程序 `CAP_DAC_READ_SEARCH` 以允许其读取任意文件。

提示：

- 建议*始终*同时使用 `CapabilityBoundingSet` 和 `AmbientCapabilities`。
- `systemd-analyze capabilities` 可以列出 systemd 可使用的能力。当您运行新增了能力的实验性内核时，这个命令会很有用。

要想采用上述建议，您可以在编辑服务单元时使用 `AmbientCapabilities=` 来授予其能力，并使用 `CapabilityBoundingSet=` 来确保没有预期外的能力被授予：

```
example.service
[Service]
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
```

还可以在 `/etc/systemd/system.conf` 中设置 `CapabilityBoundingSet=` 或在 `/etc/systemd/systemd.conf.d` 中创建 drop-in 配置文件来移除继承自 PID 1 以及其子进程的能力。不过大部分能力会被实际使用而不应该被移除。

常见指令

大部分指令可以在不引起太多问题的情况下应用到大多数应用中。

注意："影响"以及"潜在破坏性"都是相对于实际的运行环境以及单元文件变化的。

没有特殊配置的指令

简单的布尔设置，可以为启用或禁用。它们不能配置细节。

指令	影响1	破坏性2	备注
`LockPersonality`	中	低	
`MemoryDenyWriteExecute`	中	中	不兼容运行时动态生成代码的程序，例如 JIT、可执行栈以及 C 编译器生成的代码“蹦床”。可以使用 `SystemCallFilter` 增强。
`NoNewPrivileges`	高	低	
`PrivateDevices`	中	低	`/dev/null` 等类似设备不会被禁用。
`PrivateNetwork`	高	非常高	不允许*任何*网络访问。
`PrivateTmp`	中	低	
`PrivateUsers`	高	高	
`RestrictRealtime`	低	低	可能有助于防止拒绝服务情况出现。
`RestrictSUIDSGID`	中	低	建议和 `NoNewPrivileges` 一起使用。

1. 指令产生的效果
2. 指令破坏程序正常运行的可能性

可配置的指令

这些指令大部分都很有效而不会产生过多的影响。推荐考虑使用一些以下的指令。

指令	值	影响1	破坏性2	备注
`ProtectSystem`	`strict`	非常高	非常高	通常和 `ReadWritePaths=` 一起使用。
`full`	高	中	可能会破坏如使用 ACME 来更新密钥的网络服务器等应用，因为它们需要的文件*可能位*于 `/etc`。	
`true`	高	中	理论上基本没有程序会往 `/boot` 和 `/usr` 写入。	
`ProtectHome`	`true`	高	中	一些应用可能需要将持久化数据存储于 `XDG_CONFIG_HOME`3。
`tmpfs`	高	中	家目录包含大量敏感数据，应用 `tmpfs` 或 `true` 也许能够避免泄露。4	
`read-only`	低	低		

指令	值	影响1	破坏性2	备注
			非常合适用于备份服务。	
ProtectProc5	invisible	高	中	
noaccess	中	中		
RestrictAddressFamilies	例如 AF_UNIX AF_INET AF_INET6	低	低	填写用空格分隔的地址族，详情请参考 address_families(7)。

1. 指令产生的效果
2. 指令破坏程序正常运行的可能性
3. StateDirectory= 可以用于缓解一些负面后果。
4. 这也会使 /run/user/ 不可访问，避免使用中的 IPC socket 泄露。理论上来说，sockets 也有可能在另外地方，例如 /tmp。
5. 当指令被省略时，默认为 /proc 的 hidepid 挂载选项的值。通常来说为 0（无限制）。

进阶指令

这些指令对多数单元来说不是很有用，因此很少用到。

指令	值	影响	破坏性	备注
DynamicUser	布尔值，例如 true	中	中	需要和 StateDirectory=、RuntimeDirectory=、CacheDirectory=、LogsDirectory=、ConfigurationDirectory= 结合使用。
ProtectClock1,2	布尔值	低	中	有些用户返回当该指令启用时，smartctl 不能运行，不过该指令应该是安全的。
ProtectControlGroups1	布尔值	中	低	
ProtectHostname1,2	布尔值	低	低	
ProtectKernelLogs1	布尔值	低	低	所有官方内核都将 SECURITY_DMESG_RESTRICT 设置为 y，不过这个选项仍然可以达成纵深防御。
ProtectKernelModules1	布尔值	中	低	
ProtectKernelTunables1	布尔值	低	低	
RestrictFileSystems	例如 ext4 tmpfs	中	中	填写用空格分隔的文件系统列表或指定集合，如 @network。详情请运行 systemd-analyze filesystemd。
SystemCallArchitectures	例如 native	低	低	另请参阅 #禁止非原生架构系统调用；对单元文件设置这个指令可以覆盖系统默认配置，

指令	值	影响	破坏性	备注
				从而把特定单元排除在系统限制之外。
SystemCallFilter	例如 @system-service	中	高	另请参阅 systemd.exec(5) § SYSTEM CALL FILTERING；少写哪怕一个系统调用都可能使您的应用在不合时宜的情况下崩溃。
SocketBindAllow/Deny	例如 SocketBindAllow=ipv4:22 、SocketBindDeny=any	中	中	建议和 CAP_NET_BIND_SERVICE 一起设置以确保特权上下文只能绑定预期的端口。

1. 如果服务单元未以 root 运行或通过 AmbientCapabilities= 等指令被授予特权，则设置该指令是多余且不必要的。如果服务单元以另一个低权限普通 User= 运行，则这些设置完全是多余的并可以被安全地忽略。
2. 仅限制应用执行相关的系统调用，而不限制应用通过 systemd 提供的 IPC 服务（即 *systemd-timedated* 以及 *systemd-hostnamed*）来访问。如果需要绝对安全，务必注意同时限制涉及 D-Bus/Varlink 的方法。

chroot jail

警告：尽管 chroot jail 事实上可以工作，但它一定程度依赖于指令未写入文档的副作用并且可能随时在新版本的 systemd 中失效。因此，通常认为它不受 systemd 的开发者支持。已提交一项功能请求以将其转变为一项官方认可并支持的功能。

通过指定 TemporaryFileSystem=/:ro 并只挂载需要的路径到这种类 chroot jail 以激进地限制进程能看到的文件是可能的。 RootDirectory 要求路径必须存在，而 TemporaryFileSystem 则不需要，且会无缝修改 /。由于没有使用 chroot 系统调用，两者，尤其是后者，看起来都属于安全的类 chroot 指令，无法被轻易逃逸。

警告：ProtectSystem 和 ProtectHome 这两者与 TemporaryFileSystem=/:ro 不兼容且会导致后者失效，导致 / 再次可见。不过这两个指令都不需要，因为可见的路径是由白名单决定的。请参阅上面的警告，出现这个问题的原因是这些指令之间的相互作用并不被支持或纳入考量。

所有需要的路径都需要通过 BindReadOnlyPaths 和 BindPaths 挂载到 jail 中：

```
example_jailed_unit.service
[Unit]
Description=示例单元

[Service]
ExecStart=/home/someuser/executable
User=someuser
Group=someuser
TemporaryFileSystem=/:ro
PrivateTmp=true
BindReadOnlyPaths=/usr/lib /lib64 /lib
BindPaths=/home/someuser/executable
```

以上只是一个最小的示例而大多数应用需要更多的白名单路径。常见的有：

- `/etc/ca-certificates`、`/etc/ssl`
- `/etc/resolv.conf`
- `/usr/share/zoneinfo`
- 任何需要使用的 sockets，如 `/var/run/mysqld/mysqld.sock`

首次尝试把一个单元放到沙箱中运行往往需要调试。如果它完全不能启动或以 `status=203/EXEC` 失败，则说明无法访问可执行文件或需要的库文件。可以从宽泛的路径（如允许整个 `/usr`）开始，逐步缩小可访问的路径范围。

system.conf

`/etc/systemd/system.conf` 的更改是全局性的，因此会影响所有单元。另请参见 systemd-system.conf(5)。

禁止非原生架构系统调用

非原生架构二进制文件，大部分情况下是 32 位二进制文件，可能一定程度上损害系统安全性，因为部分加固并不能良好工作。此前也有过影响非原生系统调用的低危害漏洞，例如 CVE-2009-0835。

注意：以下配置**将会**破坏 32 位程序兼容性。尝试执行 32 位程序将导致 `file not found` 错误。

```
/etc/systemd/system.conf
SystemCallArchitectures=native
```

这个配置在大多数系统上都可以正常工作，但如果安装了 multilib 的软件包的话，就需要在一部分情况下禁用。尤其注意使用 Wine 玩游戏可能会受影响。使用 `systemd-run` 或修改会话切片可以按需覆盖 `SystemCallArchitectures` 配置以禁用限制。

启用更多单元统计

systemd 默认情况下不会跟踪一个单元的所有资源使用情况。开启 `Default*Accounting` 可以在 `systemctl status` 输出和日志中查看更多统计信息。严格意义上来说，这不是一个安全设置，但是它确实可以使调试更简单，并有助于深入了解资源使用情况。

```
/etc/systemd/system.conf
DefaultIOAccounting=yes
DefaultIPAccounting=yes
DefaultMemoryAccounting=yes
DefaultTasksAccounting=yes
```

参考资料

https://docs.rockylinux.org/10/guides/security/systemd_hardening/

1. ↑ 译者注：真有程序还在用这玩意吗?

3.2 Bubblewrap

难度 进阶 | **篇幅** 18.7 KB | **所属** 沙箱

本节将讲到

- 安装
- 配置
- bubblewrap 配置管理器
- 使用示例
- 疑难解答

Bubblewrap 是一款轻量化的沙箱应用，常为 Flatpak 和其他容器工具所用。安装和运行 bubblewrap 所需的资源非常少。虽然软件包名称是 bubblewrap，实际的命令行接口却是 bwrap(1)。Bubblewrap 所值得注意的特性包括支持 cgroup、IPC（进程间通信）、mount、network、PID、user、UTS namespaces 和 seccomp 过滤。注意 bubblewrap 在沙箱内会丢弃所有的 capabilities，因此子任务的权限不能超过其父任务。但 bubblewrap 对文件路径的黑白名单缺乏明确支持。

警告：Bubblewrap 是一种能提供 namespaces 和 seccomp 过滤等沙箱化技术的工具。默认情况下，不提供能够隔离所用技术的缺陷的完整沙箱环境。运行不受信任的代码永远是危险的，沙箱化技术也不会改变这一事实。

安装

安装 bubblewrap包。

注意：

- 有关 Arch Linux 内核所支持的 user_namespaces(7) 的更多信息，参见安全#禁用非特权用户命名空间。
- Bubblewrap 自 0.12.0 版本起不再支持非特权用户命名空间，因此 linux-hardened包 用户需要通过 sysctl 启用 `kernel.unprivileged_userns_clone`。更多相关信息请参阅 FS#63316。

配置

可通过直接在命令行中调用或是作为组成复杂包装器的一部分 shell 脚本中使用 bubblewrap。与一些自动在沙箱内将 `/var` 和 `/etc` 设置为只读的应用程序（例如 Firejail）不同，bubblewrap 不会做这样的操作假设。应该由用户根据想要沙箱化运行的程序来决定需要传入何种配置选项。以 setuid 权限运行的 bubblewrap 不会自动创建用户命名空间（user namespaces），但可以接受典型的环境变量，如 `$HOME` 和 `$USER`。

强烈建议安装 strace包 软件包用于了解要以沙箱运行的程序的所需文件。

bubblewrap 配置管理器

除了手动配置，也可以使用配置管理器并通过更简单的操作来自动化配置 bubblewrap。

- **Bubblejail** — 基于 bubblewrap 的沙箱，有基于资源的权限模型（提供了用于权限配置的图形界面）。

: https://github.com/igo95862/bubblejail || bubblejailAUR

- portableAUR：简单易用的沙箱化框架，包括 D-Bus 代理过滤、数据隔离、访问控制以及其他功能。

使用示例

参阅 Bubblewrap 示例以了解如何使用 bubblewrap。 除此以外，许多项目已证实 bubblewrap 可用于通常应用程序：

- bwscripts
- StandingPad's Bubblewrap scripts

No-op

无操作调用（No-op invocation）bubblewrap 的示例如下：

```
$ bwrap --dev-bind / / bash
```

上述命令会创建一个大多数情况下与沙箱外部表现完全一致的 Bash 进程。如果沙箱化的程序出现异常行为，可以通过上述的无操作调用启动，然后逐步调整至更安全的配置。

注意：如果执行对象的拥有者（owner）和用户组（group）与执行者不一致，这个操作会把所有的拥有者（owner）和用户组（group）更改为 `nobody`，因此运行某些程序如 `sudo`，将不能正常工作。

Bash

创建一个简单的 Bash 沙箱：

- 确定可用的内核命名空间（kernel namespaces）：

```
$ ls /proc/self/ns
cgroup  ipc  mnt  net  pid  user uts
```

注意：`user` 的存在意味着内核已经通过 `CONFIG_USER_NS=y` 暴露出对用户命名空间（user namespaces）的支持。

- 将宿主的整个 `/` 目录以只读模式绑定到沙箱内的 `/`。
- 创建一个新的用户命名空间（user namespace）并将用户标识符设为 `256`，用户组标识符设为 `512`。

```
$ bwrap --ro-bind / / --unshare-user --uid 256 --gid 512 bash
bash-4.4$ id
uid=256 gid=512 groups=512,65534(nobody)
bash-4.4$ ls -l /usr/bin/bash
-rwxr-xr-x 1 nobody nobody 811752 2017-01-01 04:20 /usr/bin/bash
```

桌面项

在桌面项中使用 bubblewrap：

- 将宿主的整个 `/` 目录以读写模式绑定到沙箱内的 `/`。
- 将沙箱内的 `/var` 和 `/etc` 目录重新以只读模式绑定。
- 挂载一个新的 devtmpfs 文件系统到沙箱内的 `/dev`。
- 在沙箱化的 `/run` 目录中创建一个 tmpfs 文件系统。
- 创建一个新的网络命名空间（network namespace）以禁用网络访问。

```
[Desktop Entry]
Name=nano Editor
Exec=bwrap --bind / / --ro-bind /var /var --ro-bind /etc /etc --dev /dev --tmpfs /run
--unshare-net st -e nano -o . %f
Type=Application
MimeType=text/plain;
```

注意：为写入到 `/dev/pty`，`--dev /dev`是必需的。

- MuPDF 的桌面项示例（配合一个 `mupdf.sh` 的 shell 包装器）：

```
[Desktop Entry]
Name=MuPDF
Exec=mupdf.sh %f
Icon=application-pdf.svg
Type=Application
MimeType=application/pdf;application/x-pdf;
```

注意：请确保 `mupdf.sh` 位于 PATH 中，例如 `PATH=$PATH:$HOME/bwrap`

隔离文件系统

警告：Bubblewrap 用户需自行负责定期更新文件系统树。

要进一步隐藏文件系统的内容（例如 `/var`、`/usr/bin` 和 `/usr/lib` 中的文件），甚至是用沙箱保护软件的安装，pacman 可以将软件包安装到隔离的文件系统中。

为能让 pacman 将软件安装到文件系统树中，需要先安装 fakeroot包 和 fakechroot 包。

假设用户想用 pacman 将 `xterm` 软件包安装到一个隔离的文件系统树中，用户应该先准备好文件系统树，就像下面的示例那样：

```
$ MYPACKAGE=xterm
$ mkdir -p ~/sandboxes/${MYPACKAGE}/files/var/lib/pacman
$ mkdir -p ~/sandboxes/${MYPACKAGE}/files/etc
$ cp /etc/pacman.conf ~/sandboxes/${MYPACKAGE}/files/etc/pacman.conf
```

可能需要编辑 `~/sandboxes/${MYPACKAGE}/files/etc/pacman.conf` 并调整 pacman 的配置文件：

- 移除任何不需要的自定义仓库以及仅被宿主系统需要的 `IgnorePkg`、`IgnoreGroup`、`NoUpgrade` 和 `NoExtract` 设置。
- 可能需要移除 `CheckSpace` 选项，这样 pacman 就不会在检查磁盘空间的时候抱怨找不到根文件系统。

然后将 `base` 包组和 fakeroot 安装到隔离的文件系统树中：

```
$ fakechroot fakeroot pacman -Syu \
    --root ~/sandboxes/${MYPACKAGE}/files \
    --dbpath ~/sandboxes/${MYPACKAGE}/files/var/lib/pacman \
    --config ~/sandboxes/${MYPACKAGE}/files/etc/pacman.conf \
    base fakeroot
```

因为接下来要以相同的选项重复调用 bubblewrap，可以设置一个别名（alias）：

```
$ alias bw-install='bwrap                                 \
     --bind ~/sandboxes/${MYPACKAGE}/files/ /     \
     --ro-bind /etc/resolv.conf /etc/resolv.conf \
     --tmpfs /tmp                                 \
     --proc /proc                                 \
     --dev /dev                                   \
     --chdir /                                    '
```

编辑 `~/sandboxes/${MYPACKAGE}/files/etc/locale.gen` 并运行下面的命令以设置 locale：

```
$ bw-install locale-gen
```

然后设置 pacman 的 keyring：

```
$ bw-install fakeroot pacman-key --init
$ bw-install fakeroot pacman-key --populate
```

现在就可以安装所需的 `xterm` 软件包了：

```
$ bw-install fakeroot pacman -S ${MYPACKAGE}
```

如果 pacman 在这一步执行失败，尝试再次运行用于 populate keyring 的命令。

成功后，用户现在拥有一个包含 `xterm` 的隔离的文件系统树。之后可以再次使用 `bw-install`（上面的例子中配置好的别名）来更新文件系统树。

接下来就可以使用 bubblewrap 来运行软件了。本例中的 `*command*` 应该替换成 `xterm`。

```
$ bwrap                                           \
     --ro-bind ~/sandboxes/${MYPACKAGE}/files/ / \
     --ro-bind /etc/resolv.conf /etc/resolv.conf \
     --tmpfs /tmp                                 \
     --proc /proc                                 \
     --dev /dev                                   \
     --chdir /                                    \
     command
```

注意，一些文件会在软件包中共享。用户可以将一个已存在的父文件系统树中的所有文件硬链接到一个新的文件系统树中来重用这些文件：

```
$ cp -al ~/sandboxes/${MYPARENTPACKAGE} ~/sandboxes/${MYPACKAGE}
```

然后像平时那样从 `bw-install fakechroot fakeroot pacman ...` 开始调用 pacman 进行安装。

疑难解答

使用 X11

绑定挂载宿主的 X11 socket 到另一个 X11 socket 可能无法工作：

```
--bind /tmp/.X11-unix/X0 /tmp/.X11-unix/X8 --setenv DISPLAY :8
```

一种解决方法是将宿主的 X11 socket 绑定挂载到沙箱中的同一个 socket：

```
--bind /tmp/.X11-unix/X0 /tmp/.X11-unix/X0 --setenv DISPLAY :0
```

沙箱化 X11

虽然 bwrap 为沙箱化的应用程序提供了一些非常不错的隔离功能，但对于应用程序来说，只要能访问 X11 socket，就仍然有能够逃离沙箱隔离的方法。

X11 下的应用程序之间没有互相隔离，这会让恶意软件能够监听输入、注入按键或记录其他应用程序的图像。

一种应对方式是换用 Wayland 混成器，该混成器不能从沙箱中访问 X 服务。Wayland 实现了不允许应用程序交互的特性。

而如果要继续使用 X11，应该使用 xpra 或 xephyr。这些工具能够创建一个次要且仅用于运行被沙箱化的应用程序的 X11 实例，该实例会在用户目前使用的环境中的一个窗口中显示。以这种方式运行的应用程序窗口无法与其自身 X11 实例以外的环境交互。上述方式同样可与 bwrap 共同使用。

要测试 X11 是否被隔离，运行 `xinput test *id*`（使用 `xinput list` 查找所需的 *id*）。 在没有额外隔离的情况下运行 X11 时，上述命令会显示任何能访问 X11 且够抓取键盘输入的其他应用程序，这种情况下恶意软件将能够记录按键。

使用 portals

**本文或本章节可能需要合并到XDG Desktop Portal。**

附注： 有独立页面。（在 Talk:Bubblewrap 中讨论）

警告：这其实是利用一些方法“欺骗”了 `xdg-desktop-portal`，让它认为被沙箱化的程序是一个 Flatpak 程序。再次提醒，运行不受信任的代码永远是危险的，就算是通过 Portals 在沙箱中运行也是。

通过一些解决方法，或许可以利用 XDG Desktop Portals 来沙箱化一些应用程序。 这么做的优点是，使用文件系统 portals，能够在不给予应用程序对于家目录（home directory）的访问权限的情况下，让应用程序依然可以访问文件。 然而出于安全原

因，使用 portals 需要欺骗 `xdg-desktop-portal`，让它认为被沙箱化的程序是 Flatpak 的一部分。这可以通过在沙箱的根文件系统中添加一个 `.flatpak-info` 文件来实现。

此外，用户还应该运行 `xdg-dbus-proxy` 以更好地颗粒化控制 portals 的访问权限。此命令应该在沙箱环境下运行，因此也需要一个 `.flatpak-info` 文件。但 proxy 至少需要能够有 `org.freedesktop.portal.Flatpak` 的 talk 访问权限。更多 portals 请访问 Flatpak 文档。

常见的案例是允许一个被完全限制 home 目录访问权限的程序仅仅能够访问用户在文件选择器中选择的文件和目录。用以下参数启动 `xdg-dbus-proxy` 可以实现这个目的：

```
--talk=org.freedesktop.portal.Documents
--talk=org.freedesktop.portal.Flatpak
--talk=org.freedesktop.portal.Desktop
--talk=org.freedesktop.portal.FileChooser
```

完整示例如下：

```
APP_NAME=app.application.Name
APP_FOLDER="$XDG_RUNTIME_DIR/app/$APP_NAME"
mkdir -p "$APP_FOLDER"
set_up_dbus_proxy() {
 bwrap \
   --new-session \
   --symlink /usr/lib64 /lib64 \
   --ro-bind /usr/lib /usr/lib \
   --ro-bind /usr/lib64 /usr/lib64 \
   --ro-bind /usr/bin /usr/bin \
   --bind "$XDG_RUNTIME_DIR" "$XDG_RUNTIME_DIR" \
   --ro-bind-data 3 "/.flatpak-info" \
   --die-with-parent \
   -- \
   env -i xdg-dbus-proxy \
   "$DBUS_SESSION_BUS_ADDRESS" \
   "$APP_FOLDER/bus" \
   --filter \
   --log \
   --talk=org.freedesktop.portal.Flatpak \
   --
call="org.freedesktop.portal.Desktop=org.freedesktop.portal.Settings.Read@/org/freedesk
top/portal/desktop" \
   --
broadcast="org.freedesktop.portal.Desktop=org.freedesktop.portal.Settings.SettingChange
d@/org/freedesktop/portal/desktop" 3<<EOF
[Application]
name=$APP_NAME
EOF
}

set_up_dbus_proxy &
sleep 0.1

bwrap \
  ...
  --ro-bind-data 3 /.flatpak-info \
  ...
  3<<EOF
[Application]
name=$APP_NAME
```

```
EOF
```

从被隔离的（wrapped）应用中打开 URL

当被隔离的 IRC 或电子邮件客户端尝试打开一个 URL，通常会尝试启动一个浏览器进程，而被启动的浏览器进程会与被隔离的应用程序一样运行在同一个沙箱中，而一个被良好隔离的应用程序很有可能没法正常工作。Firejail 所用的处理方式是给予被隔离的应用程序所有浏览器也拥有的权限，然而这也意味着分配大量的权限。

对于这个问题，更好的解决方法是将打开 URL 的请求发送到沙箱外。参照下列示例使用 `snapd-xdg-open`：

1. 安装 snapd-xdg-open-gitAUR
2. 在 `bwrap` 命令行中添加：

```
$ bwrap ... \
  --ro-bind /run/user/$UID/bus /run/user/$UID/bus \
  --ro-bind /usr/lib/snapd-xdg-open/xdg-open /usr/bin/xdg-open \
  --ro-bind /usr/lib/snapd-xdg-open/xdg-open /usr/bin/chromium \
  ...
```

对于不使用 XDG 惯例的应用程序而言，绑定 `/usr/bin/chromium` 是必需的，比如 Mozilla Thunderbird。

New session

一个关于 TIOCSTI 的安全问题（CVE-2017-5226）会引起沙箱逃逸。 为防止这个问题，bubblewrap 已引入一个新选项“--new-session”，该选项会调用 setsid()。 然而，这个选项在一些情况下会引起行为问题（behavioural issues）从而难以使用。 例如，这个选项会让 shell 的作业控制（job control）无法应用于 bwrap 命令。

如果应用程序开发人员有其他方式解决上述漏洞（CVE-2017-5226），可以不用这个选项（比如 Flatpak 所处理的用于 SECCOMP 的方式），但除此以外建议尽量使用“--new-session”选项。

嵌套的命名空间

特定应用程序如 Chromium 已经使用 suid 帮助文件实现了其自身的沙箱环境。这个机制会在以 bubblewrap 容器中运行这些软件时被阻止。

一种解决方法是让应用程序使用 bubblewrap 创建的命名空间。比如可以使用 zypakAUR，同时它也是 Flatpak 所使用的方式，用于在额外的命名空间中运行基于 electron 的应用程序。这个链接中有关于如何使用 zypak 运行 Chromium 或 Electron 的示例代码。

没有声音输出

对于直接使用 ALSA 声音系统的程序，如果没有声音输出，请添加如下选项：

```
--dev-bind /dev/snd /dev/snd
```
即可获得声音输出。

另请参见

- GitHub repository
- Seccomp BPF (SECure COMPuting with filters)
- Additional bubblewrap examples
- sandbox-run – A simple bubblewrap-based sandboxing tool written in shell.

3.3 Bubblewrap/示例

难度 高级 | **篇幅** 21.6 KB | **所属** 沙箱

本节将讲到

- dhcpcd
- Unbound
- MuPDF
- p7zip
- Firefox

dhcpcd

创建一个简单的 dhcpcd 沙箱：

- 确定可用的内核命名空间（kernel namespaces）：

```
$ ls /proc/self/ns
cgroup  ipc  mnt  net  pid  uts
```
注意：缺少 `user` 意味着内核以 `CONFIG_USER_NS=n` 参数构建或是被限制了用户命名空间（user namespace）。

- 将宿主的整个 `/` 目录以读写模式绑定到沙箱中的 `/`。
- 将一个新的 devtmpfs 文件系统挂载到沙箱中的 `/dev`。
- 创建一个新的 IPC（进程间通信）和 cgroups（控制组）命名空间。
- 创建一个新的 UTS 命名空间并将 `dhcpcd` 设置为主机名（hostname）。

```
# /usr/bin/bwrap --bind / / --dev /dev --unshare-ipc --unshare-cgroup --unshare-uts --
hostname dhcpcd /usr/bin/dhcpcd -q -b
```

Unbound

创建一个稍颗粒化和复杂的 Unbound 沙箱：

- 将宿主系统的 `/usr` 目录以只读模式绑定到沙箱中的 `/usr`。
- 将宿主系统的 `/usr/lib` 目录软链接到沙箱中的 `/lib64`。
- 将宿主系统的 `/etc` 目录以只读模式绑定到沙箱中的 `/etc`。
- 在沙箱中创建空的 `/var` 和 `/run` 目录。

- 将一个新的 devtmpfs 文件系统挂载到沙箱中的 `/dev`。
- 创建一个新的 IPC（进程间通信）和 PID（进程标识符）以及控制组命名空间（control group namespaces）。
- 创建一个新的 UTS 命名空间并将 `unbound` 设置为主机名（hostname）。

```
# /usr/bin/bwrap --ro-bind /usr /usr --symlink usr/lib /lib64 --ro-bind /etc /etc --
dir /var --dir /run --dev /dev --unshare-ipc --unshare-pid --unshare-cgroup --unshare-
uts --hostname unbound /usr/bin/unbound -d
```

提示：参见 systemd#修改现存单元文件以了解如何启用 systemd 单元文件的 bubblewrapping，包括 `unbound.service`。

MuPDF

**本文或本章节的事实准确性存在争议。**

原因： 转发 X11 socket 可能导致沙箱逃逸。（在 Talk:Bubblewrap/示例 中讨论）

在 shell 包装器（wrapper）中创建环境时，能体现出 bwrap 的强大和灵活性：

- 将宿主系统的 `/usr/bin` 目录以只读模式绑定到沙箱中的 `/usr/bin`。
- 将宿主系统的 `/usr/lib` 目录以只读模式绑定到沙箱中的 `/usr/lib`。
- 将宿主系统的 `/usr/lib` 目录软链接到沙箱中的 `/lib64`。
- 在沙箱中创建一个 tmpfs 文件系统，覆盖 `/usr/lib/gcc`。
- 这么做可以有效地阻止) `/usr/lib/gcc` 中的内容出现在沙箱中。
- 在沙箱中创建一个新的 tmpfs 文件系统，作为 `$HOME` 目录。
- 将 `.Xauthority` 文件和“Documents”目录以只读模式绑定到沙箱。
- 这么做可以有效且递归地允许访问 `.Xauthority` 文件和“Documents”目录。
- 在沙箱中创建一个新的 tmpfs 文件系统，作为 `/tmp` 目录。
- 将 X11 socket 以只读模式绑定到沙箱中以允许访问。
- 复制并创建用于容纳宿主内核所支持的所有命名空间的私有容器（private containers）。
- 如果内核不支持非特权的（non-privileged）用户命名空间，跳过创建过程并继续操作。
- 勿将网络组件放置到私有命名空间内。
- 这么做可以允许联网访问 URI 超链接。

```
#!/bin/sh
#~/bwrap/mupdf.sh
(exec bwrap \
--ro-bind /usr/bin /usr/bin \
--ro-bind /usr/lib /usr/lib \
--symlink usr/lib /lib64 \
--tmpfs /usr/lib/gcc \
--tmpfs $HOME \
--ro-bind $HOME/.Xauthority $HOME/.Xauthority \
--ro-bind $HOME/Documents $HOME/Documents \
--tmpfs /tmp \
```

```
--ro-bind /tmp/.X11-unix/X0 /tmp/.X11-unix/X0 \
 --unshare-all \
--share-net \
/usr/bin/mupdf "$@")
```

提示：使用 `/usr/bin/sh` 执行作为已存在的可执行项目的替代的命令包装器以调试和验证沙箱中的内容和文件系统结构。

```
$ bwrap \
--ro-bind /usr/bin /usr/bin \
--ro-bind /usr/lib /usr/lib \
--symlink usr/lib /lib64 \
--tmpfs /usr/lib/gcc \
--tmpfs $HOME \
--ro-bind $HOME/.Xauthority $HOME/.Xauthority \
--ro-bind $HOME/Desktop $HOME/Desktop \
--tmpfs /tmp \
--ro-bind /tmp/.X11-unix/X0 /tmp/.X11-unix/X0 \
--unshare-all \
--share-net \
 /usr/bin/sh
bash-4.4$ ls -AF
.Xauthority  Documents/
```

或许在构建一个被 bubblewrap 的文件系统时需考虑到的最重要的规则是：“命令应该按出现的顺序依次执行”。 根据上面的 MuPDF 示例：

- 首先创建了一个 tmpfs 文件系统，其次是绑定并挂载 `.Xauthority` 文件和“Documents”目录：

```
--tmpfs $HOME \
--ro-bind $HOME/.Xauthority $HOME/.Xauthority \
--ro-bind $HOME/Documents $HOME/Documents \
bash-4.4$ ls -a
.  ..  .Xauthority  Desktop
```

- 如果在绑定挂载 `.Xauthority` 后再创建 tmpfs 文件系统，结果将覆盖先前文件，因此沙箱中只能够找到“Documents”这个目录：

```
--ro-bind $HOME/.Xauthority $HOME/.Xauthority \
--tmpfs $HOME \
--ro-bind $HOME/Desktop $HOME/Desktop \
bash-4.4$ ls -a
.  ..  Desktop
```

p7zip

还未对已知漏洞进行修补的应用程序是 bubblewrap 的主要候选对象：

- 将宿主系统中的 `/usr/bin/7za` 可执行文件路径以只读模式绑定到沙箱中。
- 将宿主系统的 `/usr/lib` 目录软链接到沙箱中的 `/lib64`。
- 用 tmpfs 文件系统覆盖以禁用沙箱中 `/usr/lib/modules` 和 `/usr/lib/systemd` 的内容。
- 将一个新的 devtmpfs 文件系统挂载到沙箱中的 `/dev`。
- 将宿主系统的 `/sandbox` 目录以读写模式绑定到沙箱中的 `/sandbox`。
- *7za* 仅会在宿主的 `/sandbox` 目录中运行，或在被 shell 包装器调用时，在其子文件夹中运行。

- 为应用程序及其进程创建新的 cgroup、IPC、network、PID 和 UTS 命名空间。
- 如果内核不支持非特权的用户命名空间，跳过创建过程并继续操作。
- 新创建的 network 命名空间将阻止沙箱获取网络访问权限。
- 为沙箱设置计算机名，例如 `p7zip`。
- 取消设置 `XAUTHORITY` 环境变量以隐藏 X11 conection cookie 的位置。
- *7za* 正常运行时不需要连接到 X11 显示服务。
- 启动一个新的终端会话，防止键盘输入从沙箱中逃逸。

```
#!/bin/sh
#~/bwrap/pz7ip.sh
(exec bwrap \
--ro-bind /usr/bin/7za /usr/bin/7za \
--symlink usr/lib /lib64 \
--tmpfs /usr/lib/modules \
--tmpfs /usr/lib/systemd \
--dev /dev \
--bind /sandbox /sandbox \
--unshare-all \
--hostname p7zip \
--unsetenv XAUTHORITY \
--new-session \
/usr/bin/7za "$@")
```

注意：*/usr/bin/sh* 和 */usr/bin/ls* 必须在可执行文件路径中才能遍历和验证沙箱的文件系统。

```
bwrap \
--ro-bind /usr/bin/7za /usr/bin/7za \
--ro-bind /usr/bin/ls /usr/bin/ls \
--ro-bind /usr/bin/sh /usr/bin/sh \
--symlink usr/lib /lib64 \
--tmpfs /usr/lib/modules \
--tmpfs /usr/lib/systemd \
--dev /dev \
--bind /sandbox /sandbox \
--unshare-all \
--hostname p7zip \
--unsetenv XAUTHORITY \
--new-session \
/usr/bin/sh
bash: no job control in this shell
bash-4.4$ ls -AF
dev/  lib64@  usr/
bash-4.4$ ls -l /usr/lib/modules
total 0
bash-4.4$ ls -l /usr/lib/systemd
total 0
bash-4.4$ ls -AF /dev
console  full  null  ptmx@  pts/  random  shm/  stderr@  stdin@  stdout@  tty  urandom
zero
bash-4.4$ ls -A /usr/bin
7za  ls  sh
```

Firefox

面向网络且拥有巨大的潜在被攻击面的应用程序同样是 bubblewrap 的理想候选对象：

- 将 Transmission 包含在沙箱中以响应磁力链接和种子链接并启动。

- 示例的 wrap 在GNOME（Wayland）下支持音频（PulseAudio）和打印服务（CUPS/Avahi）。
- `~/.config/transmission/settings.json` 中的 PATH 应反映 `--setenv HOME` 变量。
- 环境中的按键绑定不支持变量扩展，因此使用了完整路径。
- 同时还支持 WebRender 和硬件加速混成。

```
bwrap \
--symlink usr/lib /lib \
--symlink usr/lib64 /lib64 \
--symlink usr/bin /bin \
--symlink usr/bin /sbin \
--ro-bind /usr/lib /usr/lib \
--ro-bind /usr/lib64 /usr/lib64 \
--ro-bind /usr/bin /usr/bin \
--ro-bind /usr/lib/firefox /usr/lib/firefox \
--ro-bind /usr/share/applications /usr/share/applications \
--ro-bind /usr/share/gtk-3.0 /usr/share/gtk-3.0 \
--ro-bind /usr/share/fontconfig /usr/share/fontconfig \
--ro-bind /usr/share/icu /usr/share/icu \
--ro-bind /usr/share/drirc.d /usr/share/drirc.d \
--ro-bind /usr/share/fonts /usr/share/fonts \
--ro-bind /usr/share/glib-2.0 /usr/share/glib-2.0 \
--ro-bind /usr/share/glvnd /usr/share/glvnd \
--ro-bind /usr/share/icons /usr/share/icons \
--ro-bind /usr/share/libdrm /usr/share/libdrm \
--ro-bind /usr/share/mime /usr/share/mime \
--ro-bind /usr/share/X11/xkb /usr/share/X11/xkb \
--ro-bind /usr/share/icons /usr/share/icons \
--ro-bind /usr/share/mime /usr/share/mime \
--ro-bind /etc/fonts /etc/fonts \
--ro-bind /etc/resolv.conf /etc/resolv.conf \
--ro-bind /usr/share/ca-certificates /usr/share/ca-certificates \
--ro-bind /etc/ssl /etc/ssl \
--ro-bind /etc/ca-certificates /etc/ca-certificates \
--dir "$XDG_RUNTIME_DIR" \
--ro-bind "$XDG_RUNTIME_DIR/pulse" "$XDG_RUNTIME_DIR/pulse" \
--ro-bind "$XDG_RUNTIME_DIR/wayland-1" "$XDG_RUNTIME_DIR/wayland-1" \
--dev /dev \
--dev-bind /dev/dri /dev/dri \
--ro-bind /sys/dev/char /sys/dev/char \
--ro-bind /sys/devices/pci0000:00 /sys/devices/pci0000:00 \
--proc /proc \
--tmpfs /tmp \
--bind /home/example/.mozilla /home/example/.mozilla \
--bind /home/example/.config/transmission /home/example/.config/transmission \
--bind /home/example/Downloads /home/example/Downloads \
--setenv HOME /home/example \
--setenv GTK_THEME Adwaita:dark \
--setenv MOZ_ENABLE_WAYLAND 1 \
--setenv PATH /usr/bin \
--hostname RESTRICTED \
--unshare-all \
--share-net \
--die-with-parent \
--new-session \
/usr/bin/firefox
```

增强隐私保护

- 移除特定条目可以增加访问限制。
- 移除下面的条目可以取消音频支持：

```
--ro-bind "$XDG_RUNTIME_DIR/pulse" "$XDG_RUNTIME_DIR/pulse" \
```

- `/sandbox` 作为用户所定义的目录，并没有特殊含义，仅用于存放所需的个人资料信息。
- 可通过脚本或定时任务，也可手动将清理过的配置文件复制到 `/sandbox`。

```
$ cp -pR ~/.mozilla /sandbox/
```

文件位置可以来自网络共享、USB 挂载设备，或是来自本地文件系统，甚至是 ramfs 或 tmpfs。

- 设置 `/home/r` 以隐藏真实的 `/home/example`。
- 设置新的用户标识符和组标识符。

注意：确保新的用户标识符（UID）和组标识符（GID）不会与 `/etc/passwd` 和 `/etc/groups` 中已存在的值相冲突。

```
bwrap \
....
--bind /sandbox/.mozilla /home/r/.mozilla \
--bind /sandbox/Downloads /home/r/Downloads \
...
--setenv HOME /home/r \
...
--uid 200 --gid 400 \
...
/usr/bin/firefox --no-remote --private-window
```

Chromium

下面是 Wayland 下运行 Chromium 沙箱的简单示例：

```
bwrap \
    --symlink usr/lib /lib \
    --symlink usr/lib64 /lib64 \
    --symlink usr/bin /bin \
    --symlink usr/bin /sbin \
    --ro-bind /usr/lib /usr/lib \
    --ro-bind /usr/lib64 /usr/lib64 \
    --ro-bind /usr/bin /usr/bin \
    --ro-bind /etc /etc \
    --ro-bind /usr/lib/chromium /usr/lib/chromium \
    --ro-bind /usr/share /usr/share \
    --dev /dev \
    --dev-bind /dev/dri /dev/dri \
    --proc /proc \
    --ro-bind /sys/dev/char /sys/dev/char \
    --ro-bind /sys/devices /sys/devices \
    --ro-bind /run/dbus /run/dbus \
    --dir "$XDG_RUNTIME_DIR" \
    --ro-bind "$XDG_RUNTIME_DIR/wayland-1" "$XDG_RUNTIME_DIR/wayland-1" \
    --ro-bind "$XDG_RUNTIME_DIR/pipewire-0" "$XDG_RUNTIME_DIR/pipewire-0" \
    --ro-bind "$XDG_RUNTIME_DIR/pulse" "$XDG_RUNTIME_DIR/pulse" \
    --tmpfs /tmp \
    --dir $HOME/.cache \
    --bind $HOME/.config/chromium $HOME/.config/chromium \
    --bind $HOME/Downloads $HOME/Downloads \
    /usr/bin/chromium --enable-features=UseOzonePlatform --ozone-platform=wayland
```

警告：如果用户正在使用 linux-hardened包 内核，由于 `kernel.unprivileged_userns_clone` sysctl 已被设置为 0，会无法实现沙箱运行 chromium。用户可以自行设置为 1，但不建议这么做（FS#36969）。

一种解决方法是让 chromium 使用 bubblewrap 创建的命名空间。可以用 zypakAUR 实现，这个方法同样被 flatpak 用于在一个额外命名空间中运行基于 electron 的应用程序。这个链接是有关如何使用 zypak 运行 chromium/electron 的示例代码。

- PipeWire：`--ro-bind "$XDG_RUNTIME_DIR/pipewire-0" "$XDG_RUNTIME_DIR/pipewire-0" \`
- 不使用 pipewire 的用户可移除此行内容。
- `--bind $HOME/.config/chromium $HOME/.config/chromium \` 将用户的 chromium 配置目录以读写模式挂载到沙箱中。
- `--bind $HOME/Downloads $HOME/Downloads \` 将用户的“~/Downloads”目录以读写模式挂载到沙箱中。
- 通过增加更多隔离处理可优化本例。

Steam

**本文或本章节的事实准确性存在争议。**

原因： 转发 X11 可能导致沙箱逃逸。（在 Talk:Bubblewrap/示例 中讨论）

Steam 正常运行时需要访问 D-Bus。

下面的示例使用 `xdg-dbus-proxy` 将有限的宿主系统 bus 访问权限暴露给沙箱。一旦 bwrap 退出，此 proxy 将被一同终止。

```
steam.py
#!/usr/bin/env python3
from glob import glob
import os
from pathlib import Path
import subprocess

HOME = Path.home()
XDG_RUNTIME_DIR = Path(os.getenv("XDG_RUNTIME_DIR", "/tmp"))
XDG_CACHE_HOME = Path(os.getenv("XDG_CACHE_HOME", HOME / ".cache"))
XDG_DATA_HOME = Path(os.getenv("XDG_DATA_HOME", HOME / ".local/share"))
DBUS_PROXY_PATH = XDG_RUNTIME_DIR / "bus-proxy"
DBUS_PROXY_PATH.mkdir(exist_ok=True, parents=True)

#
STEAM_HOME = XDG_DATA_HOME / "steam_home"
STEAM_HOME.mkdir(exist_ok=True, parents=True)

# xdg-dbus-proxy
r, w = os.pipe()
session_bus_proxy = DBUS_PROXY_PATH / str(os.getpid())
system_bus_proxy = DBUS_PROXY_PATH / f"{os.getpid()}-system"
subprocess.Popen([
    "/usr/bin/xdg-dbus-proxy",
    f"--fd={w}",
    # session bus
    os.environ["DBUS_SESSION_BUS_ADDRESS"],
    str(session_bus_proxy),
    "--filter",
    "--own=com.steampowered.*",
    "--talk=org.freedesktop.portal.*",
    "--talk=org.gnome.SettingsDaemon.MediaKeys",
    "--talk=org.kde.StatusNotifierWatcher",
```

```
    "--talk=org.freedesktop.ScreenSaver",
    "--talk=org.freedesktop.PowerManagement",
    "--talk=org.freedesktop.Notifications",
    # systrem bus
    "unix:path=/run/dbus/system_bus_socket",
    str(system_bus_proxy),
    "--filter",
    "--talk=org.freedesktop.UPower",
    "--talk=org.freedesktop.UDisks2",  # used by wine
], pass_fds=[w])
# wait xdg-dbus-proxy to start
os.read(r, 1)
os.set_inheritable(r, True)

# bwrap
argv: list[str] = [
    "bwrap",
    "--bind", str(STEAM_HOME), str(HOME),
    "--proc", "/proc",
    "--dev", "/dev",
    "--dir", "/tmp",
    "--unshare-cgroup-try",
    "--unshare-pid",
    "--unshare-user-try",
    "--unshare-uts",
    "--die-with-parent",
    "--sync-fd", str(r), # ensures dbus proxy stops when the bwrap bwrap quits
    "--bind", str(system_bus_proxy), "/run/dbus/system_bus_socket",
    "--bind", str(session_bus_proxy), str(XDG_RUNTIME_DIR / "bus"),
]
def rw(*paths): argv.extend([i for path in paths for i in ("--bind-try", str(path),
str(path))])
def ro(*paths): argv.extend([i for path in paths for i in ("--ro-bind-try", str(path),
str(path))])
def dev(*paths): argv.extend([i for path in paths for i in ("--dev-bind-try",
str(path), str(path))])
def link(*links): argv.extend([i for link in links for i in ("--symlink", str(link),
str(link))])

rw(
    "/usr",
    "/etc",
    "/opt",
    "/run/systemd/resolve/",
    # WARNING: Forwarding the X11 is insecure and might lead to sandbox escapes
    "/tmp/.X11-unix",
    "/tmp/.ICE-unix",
    *glob(str(XDG_RUNTIME_DIR / "wayland*")),
    # audio
    "/var/lib/alsa/",
    *glob(str(XDG_RUNTIME_DIR / "pulse*")),
    *glob(str(XDG_RUNTIME_DIR / "pipewire*")),
    # shader cache
    XDG_CACHE_HOME / "mesa_shader_cache",
    XDG_CACHE_HOME / "mesa_shader_cache_db",
    XDG_CACHE_HOME / "nv",
    XDG_CACHE_HOME / "nvidia",
    XDG_CACHE_HOME / "radv_builtin_shaders",
    XDG_CACHE_HOME / "radv_builtin_shaders64",
    # steam
    XDG_DATA_HOME / "Steam",
)
link(
    ("/usr/bin", "/bin"),
    ("/usr/bin", "/sbin"),
    ("/usr/lib", "/lib"),
    ("/usr/lib64", "/lib64"),
    ("/run", "/var/run"),
)
dev(
    "/dev/dri",
```

```
    "/dev/input",
    "/dev/hugepages",
    *glob("/dev/nvidia*"),
    "/dev/snd",
    "/dev/fuse",
    "/sys/block/",
    "/sys/bus/",
    "/sys/class/",
    "/sys/dev/",
    "/sys/devices/",
    "/sys/module/",
)
os.execvp("bwrap", argv + ["--sync-fd", str(r), "/usr/lib/steam/steam"])
```

- 本例中将 `$XDG_DATA_HOME/steam_home` 挂载为 `$HOME`。用户可以自行将 `STEAM_HOME` 修改为其他位置。
- Use the `rw()` or `ro()` functions to grant read/write or read-only access, respectively, to a host path. For example, `rw("HOME/.config/unity3d")`.
- 使用 `rw()` 或 `ro()` 以授予宿主 path 的读写或只读权限。例如 `rw("HOME/.config/unity3d")`。

NPM, Node Version Manager (NVM), Maven Java

为能在项目的工作目录中使用 bubblewrap 运行“npm”，可以参考下面的命令示例。

Bubblewrap 能够与 Angular、Cypress 和 Maven Java 共同使用并工作良好。X11 和 Wayland 需要被包含在最开始，因为 Cypress 会启动一个基于 Electron 的图形界面。

假设用户在项目工作目录下执行 `npm install`，同时“npm”需要读写 `node_modules`、`package.json` 等文件，这种方式允许在程序启动目录中拥有完整的文件访问权限。同时“npm”和“nvm”（`npm -g install ...`）也可以访问的全局安装目录。此外 Cypress 也可以运行在 X11 或 Wayland 下。

注意：绑定到 zsh、maven 和其他的目录可能与实际情况有所不同。

```
bwrap_arguments=(
    # 避免成为僵尸进程
    --die-with-parent

    # 依赖项需要网络访问
    --unshare-all
    --share-net

    # 创建正确的运行环境
    --tmpfs /
    --tmpfs /run
    --dir /tmp
    --dev /dev
    --proc /proc
    --ro-bind /bin /bin
    --ro-bind /sbin /sbin
    --ro-bind /usr /usr
    --ro-bind /etc /etc
    --ro-bind /lib /lib
    --ro-bind /lib64 /lib64
    --ro-bind /sys /sys
    --ro-bind /var /var
```

```
    # systemd-resolve for dns
    --ro-bind /run/systemd/resolve /run/systemd/resolve

    # npm 初始化仓库时会使用 git，确保已配置好 email 和 username 项
    --ro-bind $XDG_CONFIG_HOME/git/config $XDG_CONFIG_HOME/git/config

    # zsh has to look everywhere cool
    --ro-bind $XDG_CONFIG_HOME/zsh/.zshrc $XDG_CONFIG_HOME/zsh/.zshrc
    --ro-bind $XDG_CONFIG_HOME/zsh/.zshenv $XDG_CONFIG_HOME/zsh/.zshenv
    --ro-bind $HOME/.zshenv $HOME/.zshenv

    # Maven
    --ro-bind /opt/maven /opt/maven
    --ro-bind $HOME/.m2 $HOME/.m2

    # NPM
    --bind "$XDG_DATA_HOME/npm" "$XDG_DATA_HOME/npm"

    # npm、cypress、nvm、maven 等程序需要使用 cache
    --bind "$XDG_CACHE_HOME" "$XDG_CACHE_HOME"

    # x11, needed for cypress
    --ro-bind "$XAUTHORITY" "$XAUTHORITY"

    # wayland, might be useful
    --ro-bind "$XDG_RUNTIME_DIR/$WAYLAND_DISPLAY" "$XDG_RUNTIME_DIR/$WAYLAND_DISPLAY"

    # 假定运行时所在的目录是项目工作目录，并且拥有完全的访问权限
    --bind "$(pwd)" "$(pwd)"
)

# 以上方指定的参数运行 bwrap 处理的用户命令：
$ bwrap "${bwrap_arguments[@]}" "$@"
```

3.4 Firejail

难度 高级 | **篇幅** 26.9 KB | **所属** 沙箱

本节将讲到

- 安装
- 配置
- 使用方法
- Creating custom profiles
- Firejail with Xorg

Firejail 是一个易于使用的 SUID 沙盒程序，它通过使用 Linux 命名空间、seccomp-bpf 和 Linux 功能来限制不受信任的应用程序的运行环境以降低安全漏洞被利用的风险。

警告：运行不受信任的代码永远不安全，沙盒并不能挽救这个局面。

安装

安装 firejail包 或者 firejail-gitAUR 包. 还有可以与 Firejail 一起使用的 GUI 应用程序, firetools包.

注意：有关 Arch Linux 内核中的 user_namespaces(7) 支持的信息，请参阅 Security#Sandboxing applications. Firejail即使被禁用也可以使用。

警告：虽然上游正在逐步采用白名单（参见`/etc/firejail/firefox.profile`），但大多数提供的配置文件仍然严重依赖黑名单。这意味着，任何没有被配置文件明确禁止的东西都会被应用程序访问。例如，如果你在`/mnt/btrfs`中有 btrfs 快照，沙盒里的程序可能被禁止访问`$HOME/.ssh`，但仍能访问`/mnt/btrfs/@some-snapshot/$HOME/.ssh`。请确保检查您的配置文件。见#测试配置文件。

配置

大多数用户不需要任何自定义配置，并且可以继续 #使用方法.

Firejail 使用配置文件为其中执行的每个应用程序设置安全保护 - 您可以在 `/etc/firejail/*application*.profile` 中找到默认配置文件. 如果您需要为未包含的应用程序自定义配置文件，或者希望修改默认值, 您可以在 `~/.config/firejail/` 目录中放置新规则或默认值副本. 一个应用程序可能有多个自定义配置文件，并且您可以在多个应用程序之间共享同一个配置文件.

如果 firejail 没有特定应用程序的配置文件，它会使用其限制性的系统范围默认配置文件。如果没有事先创建自定义且限制较少的配置文件，这可能会导致应用程序无法按预期运行。

参阅 firejail-profile(5)。

使用方法

要使用 firejail 对该应用程序的默认保护（默认配置文件）执行应用程序，请执行以下命令：

```
$ firejail <应用程序名称>
```

一次性添加到默认配置文件可以作为命令行选项添加（参见 firejail(1)）。 例如，要执行带有 seccomp 保护的 *okular*，请执行以下命令：

```
$ firejail --seccomp okular
```

您可以为单个程序定义多个非默认配置文件。 创建配置文件后，您可以通过执行来使用它：

```
$ firejail --profile=/absolute/path/to/profile <program name>
```

默认配置使用 Firejail

默认情况下将 Firejail 用于具有配置文件的所有应用程序，使用 sudo 运行 firecfg 工具:

```
$ sudo firecfg
```

这将在/usr/local/bin中创建指向/usr/bin/firejail，用于 Firejail 有默认或自定配置文件的程序的符号链接。请注意，firecfg(1) 只与/etc/firejail/firecfg.config中列出的程序建立符号链接。某些命令行界面的程序是不存在的，例如 *tar*、*curl* 和 *git*。这些程序需要手动建立符号链接。请参阅 Profiles not in firecfg #2507 了解为什么不包括这些程序。*firecfg* 还将当前用户加入 Firejail 用户访问数据库，并检查/usr/share/applications/*.desktop文件是否包含相应可执行文件的完整路径，然后删除完整路径并将其复制到~/.local/share/applications/。这确保了/usr/local/bin中的符号链接将被使用，从而防止 Firejail 被绕过。如果您的系统中没有安装 sudo，请以 root 身份执行：

```
# firecfg
```

以及以一般用户执行：

```
$ firecfg --fix
```

以修复 *.desktop* 文件

在某些情况下，您可能需要手动修改~/.local/share/applications/中".desktop " 文件的Exec=行，以直接调用 Firejail。

提示：可使用 pacman hook来在 pacman 操作中自动运行*firecfg*。

```
/etc/pacman.d/hooks/firejail.hook
[Trigger]
Type = Path
Operation = Install
Operation = Upgrade
Operation = Remove
Target = usr/bin/*
Target = usr/local/bin/*
Target = usr/share/applications/*.desktop

[Action]
Description = Configure symlinks in /usr/local/bin based on firecfg.config...
When = PostTransaction
Depends = firejail
Exec = /bin/sh -c 'firecfg >/dev/null 2>&1'
```

要手动链接各个应用程序，请执行以下操作：

```
# ln -s /usr/bin/firejail /usr/local/bin/application
```

注意：

- /usr/local/bin must be set before /usr/bin and /bin in the PATH environment variable.
- To run a symbolic program with custom Firejail setting, simple prefix *firejail* as seen in #使用方法.
- For a daemon, you will need to overwrite the systemd unit file for that daemon to call firejail, see systemd#Editing provided units.
- Symbolic links to *gzip* and *xz* interfere with *makepkg*'s ability to preload libfakeroot.so. See BBS#230913.

Use with hardened_malloc

hardened_mallocAUR is a hardened implementation of glibc's `malloc()` allocator, originally written for Android but extended for use on the desktop. While not integrated into glibc yet, it can be used selectively with `LD_PRELOAD`. The proper way to launch an application within firejail using hardened_malloc is demonstrated below. To make it permanent, you would need to create your own entry in /usr/local/bin for the desired application.

```
$ firejail --env=LD_PRELOAD='/usr/lib/libhardened_malloc.so' /usr/bin/firefox
```

Alternatively, add the following to a custom profile:

```
env LD_PRELOAD='/usr/lib/libhardened_malloc.so'
```

The various environment variables and settings that can be used to tune hardened_malloc can be found on its github page.

Enable AppArmor support

Since 0.9.60-1, Firejail has supported more direct integration with AppArmor through a generic AppArmor profile. During installation, the profile, `firejail-default`, is placed in `/etc/apparmor.d` directory, and needs to be loaded into the kernel by running the following command as root:

```
# apparmor_parser -r /etc/apparmor.d/firejail-default
```

See firejail(1) § APPARMOR.

Local customizations of the apparmor profile are supported by editing the file `/etc/apparmor.d/local/firejail-local`

AppArmor is already enabled for a large number of Firejail profiles. There are several ways to enable AppArmor confinement on top of a Firejail security profile:

- Pass the `--apparmor` flag to Firejail in the command line, e.g. `$ firejail --apparmor firefox`
- Use a custom profile and add the `apparmor` command.
- Enable Apparmor globally in `/etc/firejail/globals.local` and disable as needed through the use of `ignore apparmor` in `/etc/firejail/<ProgramName>.local`.

Note that enabling AppArmor by above methods **always** means that `/etc/apparmor.d/firejail-default` is used. If you rather want to use a *specific* AppArmor profile for an application, you have to use the above mentioned `ignore apparmor` command. However, that is not recommended, as using both Firejail and AppArmor for the same applications often creates problems.

Verifying Firejail is being used

```
$ firejail --list
```

A more comprehensive output is produced by

```
$ firejail --tree
```

Creating custom profiles

Whitelists and blacklists

Blacklists are heavily used in various `/etc/firejail/*.inc` files which are included in most profiles. Blacklists are permissive:

- Deny access to a directory or file and permit everything else: `blacklist <directory/file>`
- Disable/undo/ignore blacklisting a directory or file already blacklisted, e.g., in an \.inc* file: `noblacklist <directory/file>`

The order in which they appear in a profile is important: *noblacklist* directives must be added **above** *blacklist* directives.

Whitelists block everything what is not *explicitly* whitelisted. They should not be used in profiles for applications that need access to random locations (e.g., text editors, image viewers/editors).

- Allow access to a directory or file and forbid everything else: `whitelist <directory/file>`
- Disable/undo/ignore whitelisting a directory or file already whitelisted, e.g., in an \.inc* file: `nowhitelist <directory/file>`

The order in which they appear in a profile is important: *nowhitelist* directives must be added **above** *whitelist* directives.

Whitelisting is always done before blacklisting. As mentioned, a *whitelist* directive blacklists everything else. A *blacklist* directive is therefore a fallback if there are no *whitelist* directives or if a *whitelist* directive is too permissive.

(no)blacklist and *(no)whitelist* directives are often used in combination. Example: `/etc/firejail/disable-programs.inc` (which is included in all profiles) contains the directive:

```
blacklist ${HOME}/.mozilla
```

in order to block access to that directory for all applications sandboxed by Firejail. `/etc/firejail/firefox.profile` must disable this directive **and** must add a *whitelist* directive to allow access to that directory (as the Firefox profile is a whitelisted profile):

```
noblacklist ${HOME}/.mozilla
whitelist ${HOME}/.mozilla
```

Profile writing

The basic process is:

1. Copy `/usr/share/doc/firejail/profile.template` to `/etc/firejail/` or `~/.config/firejail/` and rename it to `*ProfileName*.profile` where *ProfileName* should match the name of the executable to be sandboxed

2. Change the line `include PROFILE.local` to `include ProfileName.local`
3. Gradually comment/uncomment the various options while checking at each stage that the application runs inside the new sandbox. Do not change the order of the sections in that template.
4. Detailed explanations of the possible options for a Firejail profile can be found in the firejail-profile(5) man page
5. Test the profile for security holes, see #Testing profiles

If you want to create a whitelisted profile (i.e. a profile which contains *whitelist* directives) you can build a whitelist of permitted locations by executing

```
$ firejail --build application
```

Keep in mind that a whitelisted profile is problematic for applications that need to access random locations (like text editors or file managers).

注意：

- The idea is to be as restrictive as possible, while still maintaining usability. This may involve sacrificing potentially dangerous functionality and a change in cavalier work habits.
- By default, seccomp filters work on a blacklist (which can be found in `/usr/share/doc/firejail/syscalls.txt`). It is possible to use `seccomp.keep` to build a custom whitelist of filters for an application. . A convenient way to automate these steps is to execute `/usr/lib/firejail/syscalls.sh`. If the application is still broken because of missing syscalls, you should follow the instructions at the bottom of `/usr/share/doc/firejail/syscalls.txt`.

Persistent local customisation

The standard profile layout includes the capability to make persistent local customisations through the inclusion of `.local` files. Basically, each officially supported profile contains the lines `include ProgramName.local` and `include globals.local`. These \.local* files might be located in `/etc/firejail` or in `~/.config/firejail`. Since the order of precedence is determined by which is read first, this makes for a very powerful way of making local customisations. For example, with reference this firejail question, to globally enable Apparmor and disable Internet connectivity, one could simply create/edit `/etc/firejail/globals.local` to include the lines

```
# enable Apparmor and disable Internet globally
net none
apparmor
```

Then, to allow, for example, "curl" to connect to the internet, yet still maintain its apparmor confinement, one would create/edit `/etc/firejail/curl.local` to include the lines.

```
# enable internet for curl
ignore net
```

Since `curl.local` is read before `globals.local`, `ignore net` overrides `net none`, and, as a bonus, the above changes would be persistent across future updates.

Testing profiles

In order to test and audit a Firejail profile you may find the following to be useful:

1. `firejail --debug $Program > $PathToOutputFile` Gives a detailed breakdown of the sandbox
2. `firejail --debug-blacklists $Program` and `firejail --debug-whitelists $Program` show the blacklisted and whitelisted directories and files for the current profile.
3. `firejail --debug-caps` gives a list of caps supported by the current Firejail software build. This is useful when building a caps whitelist.
4. `firejail --help` for a full list of `--debug` options
5. `firemon PID` monitors the running process. See `firemon --help` for details
6. Executing `sudo jailcheck` tests running sandboxes. See the jailcheck(1) man page for details.
7. checksec包 may also be useful in testing which standard security features are being used

Firejail with Xorg

**本文或本章节的事实准确性存在争议。**

原因： Why does sandboxing X11 require DNS?（在 Talk:Firejail 中讨论）

On Xorg any program can listen to all keyboard input and record all screens. The purpose of sandboxing X11 is to restrict this behavior, which is especially problematic for complex programs working with potentially malicious input like browsers.

Xephyr and Xpra allow you to sandbox Xorg. Although Xpra provides full clipboard support, it is recommended to use Xephyr due to the very notable and permanent lag with nested X11 sessions.

For a complete setup with (not ideal) clipboard support (clipboard is still always shared), see Sakaki's Gentoo guide, especially the section about the clipboard and automatic rescaling.

Alternatively, if clipboard support is not needed but windows need to be managed, install a standalone window manager such as Openbox.

`xephyr-screen *Width*x*Height*` can be set in `/etc/firejail/firejail.config` where `*Width*` and `*Height*` are in pixels and based on your screen resolution.

To open the sandbox:

```
$ firejail --x11 --net=device openbox
```

device is your active network interface, which is needed to ensure that DNS works. Then right click and select your applications to run.

注意：If you use Unbound, dnsmasq, Pdnsd or any other local resolver on 127.0.0.1, you would leave `--net=*device*` out of the command as DNS should work automatically.

See the Firejail Wordpress site for a simpler guide.

According to the guide:

: The sandbox replaces the regular X11 server with Xpra or Xephyr server. This prevents X11 keyboard loggers and screenshot utilities from accessing the main X11 server.

Note that the statement:

: The only way to disable the abstract socket `@/tmp/.X11-unix/X0` is by using a network namespace. If for any reasons you cannot use a network namespace, the abstract socket will still be visible inside the sandbox. Hackers can attach keylogger and screenshot programs to this socket.

is incorrect, xserverrc can be edited to `-nolisten local`, which disables the abstract sockets of X11 and helps isolate it.

Sandboxing a browser

Openbox can be configured to start a certain browser at startup. *program*.profile is the respective profile contained in `/etc/firejail`, and `--startup "*command*"` is the command line used to start the program. For example, to start Chromium in the sandbox:

```
$ firejail --x11 --profile=/etc/firejail/chromium.profile openbox --startup "chromium"
```

Tips and tricks

Hardening Firejail

The security risk of Firejail being a SUID executable can be mitigated by adding the line

```
force-nonewprivs yes
```

to `/etc/firejail/firejail.config`. However, this can break specific applications. On Arch Linux, VirtualBox doesn't start anymore. With the linux-hardened包 kernel Wireshark and Chromium-based browsers are also affected.

Further hardening measures include creating a special firejail group with adding the user to that group and changing the file mode for the firejail executable. For details see here.

Paths containing spaces

If you need to reference, whitelist, or blacklist a directory within a custom profile, such as with palemoonAUR, you must do so using the absolute path, without encapsulation or escapes:

```
/home/user/.moonchild productions
```

Private mode

Firejail also includes a one time private mode, in which no mounts are made in the chroots to your home directory. In doing this, you can execute applications without performing any changes to disk. For example, to execute okular in private mode, do the following:

```
$ firejail --seccomp --private okular
```

Experimental improved tools

Some of the Firejail developers recognized issues with the tools it ships with and made their own, improved versions of them.

警告：As stated in some of the projects descriptions, these tools might be experimental.

- firecfg.py, an improved version of `firecfg`.
- fjp, a tool to interact with Firejail profiles.
- firejail-handler-http, which helps with opening HTTP(S) links properly when sandboxing applications.
- firejail-handler-extra, like above but handles other protocols.

疑难解答

Firejail 可能很难调试。配置错误或其他不合适设置的症状包括随机分段故障、应用程序挂起和简单的错误信息。

有些应用程序比其他应用程序更难沙箱化。例如，网络浏览器和 Electron 应用程序往往比其他应用程序需要更多的故障排除，因为可能出错的地方很多。首先查看 FAQ 与 open issues 是至关重要的，因为调试可能需要相当长的时间。

提示：另请参阅上游维基，尤其是其中有关调试 Firejail 的页面。

移除 Firejail 的符号链接

要移除 Firejail 创建的符号链接（例如重置为默认值）：

```
# firecfg --clean
```

如果您不想在特定应用程序中使用 Firejail（例如，因为您更喜欢使用 AppArmor 对其进行限制），则必须手动删除相关的符号链接：

```
# rm /usr/local/bin/application
```

由于后续执行 *firecfg* 时会重新添加已删除的符号链接，因此应在 `/etc/firejail/firecfg.config` 中对相应应用程序进行注释。

验证 Firejail 是否仍然覆盖桌面项的任何残留内容。

PulseAudio

注意：使用 PulseAudio 9.0 或更高版本应该可以解决这个问题。

如果 Firejail 会导致沙盒应用程序出现 PulseAudio 有关问题，可以使用以下命令：

```
$ firecfg --fix-sound
```

此命令会为*当前*用户创建一个自定义 `~/.config/pulse/client.conf` 文件，其中包含 `enable-shm = no` 和其他可能的变通方法。

hidepid

如果系统使用 hidepid 内核参数，Firemon 只能以 root 身份运行。这将导致 Firetools GUI 错误报告 "Capabilities"、"Protocols" 和 "Seccomp" 的状态等问题。

Nvidia 专有驱动程序

一些用户报告（例如 、 或 ）称，在使用 Firejail 和 NVIDIA 专有图形驱动程序时会出现问题。通常禁用应用程序的 `noroot` 选项即可解决这一问题。Firejail 选项。

\--net 选项和 Linux 内核 >= 4.20.0

在使用 linux >= 4.20.0 时，firejail 0.5.96 存在一个错误，请参见 和 。

错误信息示例：

```
$ firejail --noprofile --net=eth0 ls
Parent pid 8521, child pid 8522
Error send: arp.c:182 arp_check: Invalid argument
Error: proc 8521 cannot sync with peer: unexpected EOF
Peer 8522 unexpectedly exited with status 1
```

警告： 无法使用 AppArmor 限制应用程序

**本文或本章节可能需要合并到#启用 AppArmor 支持。**

附注： 应在前文中明确说明启用 AppArmor 的要求。请注意，Manjaro 的引用是不相关的。（在 Talk:Firejail 中讨论）

对于某些应用程序（如 Firefox ）启动 Firejail 时可能会出现以下警告：

```
Warning: Cannot confine the application using AppArmor.
Maybe firejail-default AppArmor profile is not loaded into the kernel.
As root, run "aa-enforce firejail-default" to load it.
```

运行建议的命令时，您可能会看到

```
ERROR: Cache read/write disabled: interface file missing. (Kernel needs AppArmor 2.4 compatibility patch.)
```

这意味着 AppArmor 未作为内核参数启用，因此必须根据 AppArmor#安装进行设置。

/usr/bin/patch: \\\\ Can't open patch file

这意味着 `PKGBUILD` 会使用 `patch` 和 `-i` 参数，因此需要在 `/etc/makepkg.conf` 中为 `$SRCDEST` 列出白名单。

用 `$SRCDEST` 的值创建 覆盖 `patch.local`：

```
whitelist /path/to/makepkg/sources
```

将 `PKGBUILD` 改为使用 `stdin` 也同样有效：

```
patch -p1 < ../file.patch
```

使用 AMDGPU 启动图形应用程序时挂起

当使用 AMDGPU 且 Mesa >= 19.3.4 时，某些图形应用程序（如 Firefox 和 mpv）会在启动时挂起。请参见 。该问题已在上游 fixed 解决，因此 firejail-gitAUR 应可正常工作。或者，为所有受影响的应用程序在 `etc/firejail` 中的配置文件中添加 `seccomp !kcmp`。如果它们已经有了 `seccomp` 语句，则可以将其连接为逗号分隔的列表，例如 `seccomp !chroot,!kcmp` 。

守护进程/后台进程挂起

有一个已知问题会阻止进程守护进程化。除了不使用 Firejail 对受影响的应用程序进行沙箱化之外，目前没有其他解决方案。因为这是 Firejail 内部的一个错误，任何配置都无法解决这个问题。幸运的是，问题中提到的应用程序通常没有很大的攻击面，因此在没有沙箱的情况下运行它们的风险相对较低。

参见

- Firejail GitHub 项目页面
- bubblewrap，一个 Firejail 的最小替代方案

3.5 Linux 容器

难度 高级 | **篇幅** 27.9 KB | **所属** 沙箱

本节将讲到

- 特权容器或非特权容器
- 安装
- for xorg
- 管理容器
- 运行Xorg程序

Linux 容器 (LXC) 是一种在单个宿主机（LXC host）运行多个隔离的 Linux 系统（容器）的操作系统级虚拟化方法。它并不提供虚拟机，而是提供了一个具有独立 CPU、内存、块 I/O、网络等空间和资源控制环境的虚拟环境。这是通过 LXC 宿主机上

Linux 内核的 namespaces 和 cgroups 特性实现的。它类似于 chroot ，但是具有更好的隔离性。

LXD 可以被用做 LXC 的管理器，本页面则涉及直接使用 LXC 。

使用容器的替代方法包括 systemd-nspawn 和 Docker 。

特权容器或非特权容器

LXC 支持两种类型的容器：*特权*和*非特权*。

一般来说，*特权*容器被认为是不安全的。

运行*非特权*容器比运行*特权*容器更安全，因为*非特权*容器在设计上具有更高程度的隔离性。其中的关键在于容器内的 root UID 被映射为宿主机上的非 root UID ，这使得容器内部的攻击更难以对宿主机系统施加影响。换句话说，如果攻击者设法逃离容器，它们会发现自己在宿主机上被限制或没有权限。

Arch 的 linux包 、 linux-lts包 和 linux-zen包 内核软件包现在提供了*非特权*容器的开箱即用支持。 类似的，对于 linux-hardened包 软件包，*非特权*容器仅适用于系统管理员；对于普通用户，默认情况下禁用了 namespace ，因此需要进行额外的内核配置更改。

本文包含了用户运行任何类型容器所需的信息，但是为了运行*非特权*容器可能需要额外的操作。

一个例子说明非特权容器

为了说明 UID 映射的作用，考虑以下来自运行中的*非特权*容器的输出。在 `ps` 命令的输出中，我们可以看到容器化的进程由容器化的 root 用户所有：

```
[root@unprivileged_container /]# ps -ef | head -n 5
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 17:49 ?        00:00:00 /sbin/init
root        14     1  0 17:49 ?        00:00:00 /usr/lib/systemd/systemd-journald
dbus        25     1  0 17:49 ?        00:00:00 /usr/bin/dbus-daemon --system --
address=systemd: --nofork --nopidfile --systemd-activation
systemd+    26     1  0 17:49 ?        00:00:00 /usr/lib/systemd/systemd-networkd
```

然而在宿主机中，可以看到这些容器化的 root 进程事实上显示为以映射用户（ID>99999）的身份运行，而不是宿主机上真实的 root 用户：

```
[root@host /]# lxc-info -Ssip --name sandbox
State:          RUNNING
PID:            26204
CPU use:        10.51 seconds
BlkIO use:      244.00 KiB
Memory use:     13.09 MiB
KMem use:       7.21 MiB
[root@host /]# ps -ef | grep 26204 | head -n 5
UID        PID  PPID  C STIME TTY          TIME CMD
100000   26204 26200  0 12:49 ?        00:00:00 /sbin/init
```

```
100000   26256 26204  0 12:49 ?        00:00:00 /usr/lib/systemd/systemd-journald
100081   26282 26204  0 12:49 ?        00:00:00 /usr/bin/dbus-daemon --system --
address=systemd: --nofork --nopidfile --systemd-activation
100000   26284 26204  0 12:49 ?        00:00:00 /usr/lib/systemd/systemd-logind
```

安装

需要的软件

安装 lxc包 和 arch-install-scripts包 使宿主机系统能够运行特权 lxc 容器。

启用非特权容器支持（可选）

更改 `/etc/lxc/default.conf` 使其包含下面的配置行：

```
lxc.idmap = u 0 100000 65536
lxc.idmap = g 0 100000 65536
```

创建 `/etc/subuid` 和 `/etc/subgid` 为每个可以运行容器的用户配置容器化 UID/GID 对的映射。下面的示例仅针对 root 用户（和 systemd 系统单元）：

```
/etc/subuid
root:100000:65536
/etc/subgid
root:100000:65536
```

此外，只有在提前委派一个 cgroup 时，以非 root 用户运行非特权容器才有效（ cgroup2 委派模型强制执行此限制，而不是 liblxc ）。使用以下 *systemd* 命令来委派 cgroup （根据 LXC - Getting started: Creating unprivileged containers as a user）：

```
$ systemd-run --unit=myshell --user --scope -p "Delegate=yes" lxc-start container_name
```

同样的方式也适用于其他 lxc 命令。

**本文或本章节可能需要合并到 cgroups#User delegation。**

附注： 这不是 Linux 容器特有的问题，避免重复。（在 Talk:Linux 容器 中讨论）

或者，您可以通过创建一个 *systemd* 单元来委派*非特权* cgroup （根据 Rootless Containers: Enabling CPU, CPUSET, and I/O delegation ）：

```
/etc/systemd/system/user@.service.d/delegate.conf
[Service]
Delegate=cpu cpuset io memory pids
```

linux-hardened 和自定义内核上的非特权容器

希望在 linux-hardened包 或自定义内核上运行*非特权*容器的用户需要完成几个额外的配置步骤。

首先，内核需要支持用户命名空间（具有 `CONFIG_USER_NS` 配置）。所有 Arch Linux 内核都有 `CONFIG_USER_NS` 的支持。然而，基于更一般的安全考虑， linux-

hardened包 内核仅为 *root* 用户启用了用户命名空间。这里有两个建立*非特权*容器的选项：

- 只以 *root* 用户的身份建立非特权容器。同样为 *sysctl* 的 `user.max_user_namespaces` 配置设置一个正值来满足你的环境要求，如果当前值为 `0` （这将解决运行 `lxc info --show-log *container_name*` 时产生的 `Failed to clone process in new user namespace` 错误）。
- 在 linux-hardened包 & `lxd 5.0.0` 下，你可能需要配置 `/etc/subuid` & `/etc/subgid` 为使用 `root:1000000:65536` 。你还可能需要以特权模式启用第一个容器。这会解决错误 `newuidmap failed to write mapping "newuidmap: uid range [0-1000000000) -> [1000000-1001000000) not allowed"` 。
- 启用 *sysctl* 的 `kernel.unprivileged_userns_clone` 配置来允许普通用户运行非特权容器。可以以 root 身份运行 `sysctl kernel.unprivileged_userns_clone=1` 来为当前会话生效或阅读 sysctl.d(5) 使其永久生效。

宿主机网络配置

LXC 支持多种不同的虚拟网络类型和设备（见 lxc.container.conf(5) § NETWORK）。本节所介绍的虚拟网络类型中，大部分都需要一个宿主机上的网桥设备。

这里主要有几个可供参考的配置：

1. 主机网桥
2. NAT 网桥

主机网桥需要宿主机上的网络设置工具来配置一个共享网桥接口。宿主机和所有 LXC 容器将在同一个网络中被指派 IP 地址（如 192.168.1.x）。当你的目标是将一些暴露在网络上的服务如 web 服务器或 VPN 服务器容器化时，这可能更加便捷。用户可以将 LXC 当作物理 LAN 上的其他 PC，并在路由器上为其配置相应的端口转发。增加的便捷也可以认为是增加的威胁向量，同样的，如果 WAN 流量被转发给 LXC，将其运行在不同的网络范围上将显现更小的威胁面。

NAT 网桥不需要宿主机的网络设置工具来配置网桥。lxc包 自带的 `lxc-net` 将建立一个叫 `lxcbr0` 的 NAT 网桥。这个 NAT 网桥是一个独立的网桥，具有不与以太网设备或物理网络桥接的私有网络。它以宿主机的一个子网的形式存在。

使用主机网桥

见 Network bridge。

使用 NAT 网桥

安装 dnsmasq包，它是 `lxc-net` 的依赖，并在网桥启动前，先为其建立配置文件：

```
/etc/default/lxc-net
# Leave USE_LXC_BRIDGE as "true" if you want to use lxcbr0 for your
# containers.  Set to "false" if you'll use virbr0 or another existing
# bridge, or mavlan to your host's NIC.
USE_LXC_BRIDGE="true"

# If you change the LXC_BRIDGE to something other than lxcbr0, then
# you will also need to update your /etc/lxc/default.conf as well as the
# configuration (/var/lib/lxc/<container>/config) for any containers
# already created using the default config to reflect the new bridge
# name.
# If you have the dnsmasq daemon installed, you'll also have to update
# /etc/dnsmasq.d/lxc and restart the system wide dnsmasq daemon.
LXC_BRIDGE="lxcbr0"
LXC_ADDR="10.0.3.1"
LXC_NETMASK="255.255.255.0"
LXC_NETWORK="10.0.3.0/24"
LXC_DHCP_RANGE="10.0.3.2,10.0.3.254"
LXC_DHCP_MAX="253"
# Uncomment the next line if you'd like to use a conf-file for the lxcbr0
# dnsmasq.  For instance, you can use 'dhcp-host=mail1,10.0.3.100' to have
# container 'mail1' always get ip address 10.0.3.100.
#LXC_DHCP_CONFILE=/etc/lxc/dnsmasq.conf

# Uncomment the next line if you want lxcbr0's dnsmasq to resolve the .lxc
# domain.  You can then add "server=/lxc/10.0.3.1' (or your actual $LXC_ADDR)
# to your system dnsmasq configuration file (normally /etc/dnsmasq.conf,
# or /etc/NetworkManager/dnsmasq.d/lxc.conf on systems that use NetworkManager).
# Once these changes are made, restart the lxc-net and network-manager services.
# 'container1.lxc' will then resolve on your host.
#LXC_DOMAIN="lxc"
```

提示： 确保网桥的 IP 网段不会和本地网络冲突。一种选择可用 IP 地址的方法是使用已经动态分配给容器的地址之一。可以使用 `lxc-ls -f` 命令进行检查。

然后我们需要修改 LXC 容器模板使我们的容器使用我们的网桥：

```
/etc/lxc/default.conf
lxc.net.0.type = veth
lxc.net.0.link = lxcbr0
lxc.net.0.flags = up
lxc.net.0.hwaddr = 00:16:3e:xx:xx:xx
```

作为一个可选配置，创建一个配置文件来手动为任意一个容器指定 IP 地址：

```
/etc/lxc/dnsmasq.conf
dhcp-host=playtime,10.0.3.100
```

防火墙相关

基于宿主机运行的防火墙类型，可能需要允许 `lxcbr0` 的入口流量进入宿主机，以及 `lxcbr0` 的出口流量穿过宿主机进入其他网络。为了测试这是否可以实现，尝试启动一个容器并使用 DHCP 自动获取 IP 地址，检查 `lxc-net` 是否能够为容器注册一个 IP 地址。（如果 IP 地址并没有被成功分配，可以用 `lxc-ls -f` 检查，宿主机有必要更改的策略。

ufw包 用户可以简单运行下面两行命令来放行入口和出口流量:

```
# ufw allow in on lxcbr0
# ufw route allow in on lxcbr0
```

或者，nftables包 用户可以编辑 `/etc/nftables.conf`（并运行 `nft -f /etc/nftables.conf` 重载该配置；运行 `nft -cf /etc/nftables.conf` 来检查格式是否正确）使容器能够具有互联网访问权限（将 `"eth0"` 替换成系统中具有互联网连接的设备；运行 `ip link` 列出所有设备：

```
/etc/nftables.conf
table inet filter {
  chain input {
    ...
    iifname "lxcbr0" accept comment "Allow lxc containers"

    pkttype host limit rate 5/second counter reject with icmpx type admin-prohibited
    counter
  }
  chain forward {
    ...
    iifname "lxcbr0" oifname "eth0" accept comment "Allow forwarding from lxcbr0 to
eth0"
    iifname "eth0" oifname "lxcbr0" accept comment "Allow forwarding from eth0 to
lxcbr0"
  }
}
```

另外，由于 LXC 运行在 10.0.3.x 子网上，需要将对诸如 ssh、httpd 等服务的访问主动转发到 LXC。原则上，主机上的防火墙需要对容器上预期端口的入口流量进行转发。

iptables 规则示例

这个示例规则的功能是允许 ssh 流量转发到 LXC：

```
# iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 2221 -j DNAT --to-destination
10.0.3.100:22
```

这个规则将 2221 端口的 tcp 流量转发到 LXC 的 22 端口上。

注意：确保允许宿主机上 2221/tcp 的流量和 LXC 上 22/tcp 的流量。

为了从 LAN 上的另一台 PC 通过 ssh 连接到容器，需要 ssh 到宿主机的 2221 端口。宿主机将会把流量转发给容器。

```
$ ssh -p 2221 host.lan
```

ufw 规则示例

如果使用 ufw包，将下面的内容添加到 `/etc/ufw/before.rules` 中：

```
/etc/ufw/before.rules
*nat
:PREROUTING ACCEPT [0:0]
-A PREROUTING -i eth0 -p tcp --dport 2221 -j DNAT --to-destination 10.0.3.100:22
COMMIT
```

以非 root 用户运行容器

为了使用非 root 用户创建并启动容器，需要进行额外的配置。

建立 usernet 文件名为 `/etc/lxc/lxc-usernet`。根据 lxc-usernet(5)，每一行配置格式为：

```
user type bridge number
```

为每个需要建立容器的用户添加配置。bridge 需要和 `/etc/default/lxc-net` 中定义的一样。

在非 root 用户的家目录还需要一份 `/etc/lxc/default.conf` 配置文件的拷贝，如 `~/.config/lxc/default.conf` （如果有必要的话请建立该目录）。

以非 root 用户运行容器需要 `~/.local/share/` 具有 `+x` 权限。在启动容器前使用 chmod 命令应用该更改。

创建容器

使用 lxc-create(1) 命令创建容器。在 lxc-3.0.0-1 版本中，上游移除了本地存储的容器模板。

使用像下面这样的调用，来建立一个 Arch 容器：

```
# lxc-create --name playtime --template download -- --dist archlinux --release current --arch amd64
```

使用像下面这样的调用并从支持的列表中选择，来建立其他发行版的容器：

```
# lxc-create --name playtime --template download
```

要查看 *download* 模板的选项列表：

```
# lxc-create -t download --help
```

提示：

- 用户可以选择安装 haveged包 包并 start `haveged.service` ，来避免在设置过程中，等待系统熵种子生成时出现类似挂起的情况。否则 private/GPG 密钥的生成过程可能使整个过程变得更长。
- Btrfs 用户可以应用 `-B btrfs` 来建立一个 Btrfs 子卷以存储容器化的根目录。这在使用 `lxc-clone` 命令进行容器克隆时十分有用。 ZFS 用户可以相应地应用 `-B zfs`。

注意：想要使用传统模板的用户可以在 lxc-templatesAUR 找到它们，或者用户也可以用 distrobuilder包建立自己的模板。

配置容器

下面的例子同时适用于*特权*和*非特权*容器。主要对于非特权容器，默认会出现例子中没有出现的额外配置，包括 `lxc.idmap = u 0 100000 65536` 和 `lxc.idmap = g 0 100000 65536` 这些在启用非特权容器支持（可选）中作为可选配置添加的值。

具有网络支持的基本配置

注意：在 lxc-1:2.1.0-1 版本中，许多配置选项改变了。已有的容器需要被更新；用户需要参考 v2.1 发布注记 中包含这些变化的表格。

当一个进程使用 `/var/lib/lxc/CONTAINER_NAME/config` 中定义的容器时，系统资源将被虚拟化/隔离。在默认情况下，创建容器的进程将应用一个没有网络支持的最小化配置。下面是一个具有 `lxc-net.service` 提供网络支持的示例：

```
/var/lib/lxc/playtime/config
# Template used to create this container: /usr/share/lxc/templates/lxc-archlinux
# Parameters passed to the template:
# For additional config options, please look at lxc.container.conf(5)

# Distribution configuration
lxc.include = /usr/share/lxc/config/common.conf
lxc.arch = x86_64

# Container specific configuration
lxc.rootfs.path = dir:/var/lib/lxc/playtime/rootfs
lxc.uts.name = playtime

# Network configuration
lxc.net.0.type = veth
lxc.net.0.link = lxcbr0
lxc.net.0.flags = up
lxc.net.0.hwaddr = ee:ec:fa:e9:56:7d
```

容器中的挂载点

对于*特权*容器，用户可以选择宿主机上的目录并以 bind 方式挂载在容器中。这在容器化相同架构并希望在容器和宿主机之间分享 pacman 软件包时非常有用。另一个例子是共享文件夹。配置的格式是：

```
lxc.mount.entry = /var/cache/pacman/pkg var/cache/pacman/pkg none bind 0 0
```

注意：若不对文件系统权限进行更改，这在 *非特权* 容器上将无法使用。

图形化程序相关（可选）

为了在宿主机上显示容器中的程序窗口，需要定义一些挂载点，使容器化的程序可以获取宿主机上的资源。向 `/var/lib/lxc/playtime/config` 添加下面的段落：

```
## for xorg
lxc.mount.entry = /dev/dri dev/dri none bind,optional,create=dir
lxc.mount.entry = /dev/snd dev/snd none bind,optional,create=dir
lxc.mount.entry = /tmp/.X11-unix tmp/.X11-unix none bind,optional,create=dir,ro
lxc.mount.entry = /dev/video0 dev/video0 none bind,optional,create=file
```

**本文或本章节的事实准确性存在争议。**

原因： 设置 `xhost +` 是非常不安全的，参考 cookie-based authentication 作为替代。（在 Talk:Linux 容器 中讨论）

如果 LXC 客户机依然出现 *permission denied* 的报错，在宿主机调用 `xhost +` 来允许客户机连接宿主机的显示服务器。注意这种完全放开显示适配器权限可能带来的安全风险。 另外，在上面的挂载点之**前**添加下面的配置。

```
lxc.mount.entry = tmpfs tmp tmpfs defaults
```

VPN 相关

运行容器化的 OpenVPN 或 WireGuard, 见 Linux 容器/使用 VPN 。

管理容器

基本使用方法

列出所有已安装的LXC容器：

```
# lxc-ls -f
```

可以使用Systemd的`lxc@CONTAINER_NAME.service`来启动或停止LXC。启用`lxc@CONTAINER_NAME.service`可以在宿主系统启动时连带启动LXC。

也可以不使用Systemd来启动或停止LXC。启动一个LXC容器：

```
# lxc-start -n CONTAINER_NAME
```

停止容器：

```
# lxc-stop -n CONTAINER_NAME
```

登录容器：

```
# lxc-console -n CONTAINER_NAME
```

登录容器后，它就像普通的Linux系统一样，可以进行设置root账户密码、创建用户、下载软件包等操作。

执行容器内命令：

```
# lxc-attach -n CONTAINER_NAME --clear-env
```

它和lxc-console相似，用于以root权限在容器内执行一条命令。若不使用`--clear-env`选项，容器将携带宿主机的环境变量值（例如`$PATH`，容器使用其他发行版的环境变量可能无法执行某些指令）。

高级使用方法

LXC容器克隆

通过快照运行多个容器可以减少管理容器权限的负担（如用户管理和系统更新等）。该策略是以一个最新版本的容器为基础，需要使用容器时，克隆一个基础容器并在这个克隆的基础容器上构建应用。容器使用overlayfs挂载，overlayfs只会存储和基本容器有差异的数据，故这种策略能尽可能减少对系统和磁盘的占用。基本容器是只读的，但是通过overlayfs，由基本容器衍生的其他容器可以进行修改。

注意：因为安全原因，目前的主线Arch Linux内核不支持非特权容器使用overlayfs

例如，设置一个基础容器，并通过以下命令以基础容器为基础创建两个快照（snapshot）容器，称之为“snap1”和“snap2”.

```
# lxc-copy -n base -N snap1 -B overlayfs -s
# lxc-copy -n base -N snap2 -B overlayfs -s
```

注意：如果在基础容器上设定了静态IP，在启动快照容器前应为它手动设定IP。如果涉及到自动化流程，可以在自动化脚本内使用sed对快照容器进行IP设置。

这些快照可以像其他容器一样被启动和关闭。用户可以通过以下命令选择性的删除某些快照和其中的数据。该删除指令不会影响基础容器。

```
# lxc-destroy -n snap1 -f
```

在pi-hole和OpenVPN管理快照的脚本及系统组件可在lxc-service-snapshots获得。

将一个特权容器转换为非特权容器

当一个系统被设置为使用非特权容器时（见 #Enable support to run unprivileged containers (optional))\[损坏的链接：无效的章节\]），nsexec-bzrAUR包含一个被称为`uidmapshift`的工具将某个已经存在的特权容器转换为非特权容器从而避免重新构建新容器。

警告：

- 强烈建议在使用这个工具前备份容器！！！
- 这个工具不会更改ACL内的的UID和GID，请手动变更。

可以使用以下命令来进行转换：

```
# uidmapshift -b /var/lib/lxc/foo 0 100000 65536
```

直接运行`uidmapshift`可查看使用帮助。

运行Xorg程序

使用lxc-attach或SSH调用目标容器，并在需要使用的程序前加上X程序的DISPLAY ID。对于大多数简单的设置，显示编号通常为0。

一个简单的例子，在容器内运行Firefox并在主机屏幕上显示：

```
$ DISPLAY=:0 firefox
```

或者为避免直接连接容器，以下命令可以在宿主容器使这个过程自动化：

```
# lxc-attach -n playtime --clear-env -- sudo -u YOURUSER env DISPLAY=:0 firefox
```

提示与技巧

在非特权容器中 ping 无法工作

在非特权容器中，如果没有额外的配置步骤，ping 很可能无法工作。示例错误：

```
% ping www.google.com
ping: socktype: SOCK_RAW
ping: socket: Operation not permitted
ping: -> missing cap_net_raw+p capability or setuid?
```

要在主机上修复容器 foo 中的此问题：

```
# lxc-attach -n foo -- chmod u+s /usr/bin/ping
```

故障排除

Root用户登录失败

如果在使用lxc-console登录时显示以下错误：

```
login: root
Login incorrect
```

以及容器的 journal 显示:

```
pam_securetty(login:auth): access denied: tty 'pts/0' is not secure !
```

删除容器内文件系统的 `/etc/securetty` 和 `/usr/share/factory/etc/securetty`。或者将它们加入`/etc/pacman.conf`的NoExtract中以阻止它们被重新下载，见 **FS#45903**。

或者使用lxc-attach创建一个新用户并用它来登陆系统，在登录后切换为root账户。

```
# lxc-attach -n playtime
[root@playtime]# useradd -m -Gwheel newuser
[root@playtime]# passwd newuser
[root@playtime]# passwd root
[root@playtime]# exit
# lxc-console -n playtime
[newuser@playtime]$ su
```

容器配置中使用 veth 时无网络连接

如果你通过`/etc/lxc/*containername*/config`将网络接口设定为**veth**后无法访问局域网或广域网，请检查这个虚拟网口是否获得ip地址并正确连接到网络。

```
ip addr show veth0
inet 192.168.1.111/24
```

你可以关闭所有有关的静态ip集，并像平常一样通过已启动的容器设置这些ip

例如 `*container*/config`

```
...
lxc.net.0.type = veth
lxc.net.0.name = veth0
lxc.net.0.flags = up
lxc.net.0.link = bridge
...
```

然后通过容器的首选方法分配 IP，见网络配置#Network management.

无法找到命令

当在相对于宿主机系统使用不同Linux发行版的容器中（例如在Arch Linux宿主机系统中运行Debian容器）执行基本命令（如*ls*、*cat*等）时，可能会出现此错误。在附加容器时，请使用参数`--clear-env`：

```
# lxc-attach -n container_name --clear-env
```

Failed at step KEYRING spawning...

在非特权容器内执行的服务可能会出现以下错误：

```
some.service: Failed to change ownership of session keyring: Permission denied
some.service: Failed to set up kernel keyring: Permission denied
some.service: Failed at step KEYRING spawning ....: Permission denied
```

在容器内添加文件`/etc/lxc/unpriv.seccomp`，并写入以下配置：

```
/etc/lxc/unpriv.seccomp
2
blacklist
[all]
keyctl errno 38
```

并在容器配置文件的lxc.idmap属性后添加：

```
lxc.seccomp.profile = /etc/lxc/unpriv.seccomp
```

已知的问题

缺失lxc.init.static会导致lxc-execute执行失败

`lxc-execute` 指令运行失败并显示以下错误：`Unable to open lxc.init.static`。参见 FS#63814 了解详情。

使用 `lxc-start` 命令重启即可。

参见

- 官方网站
- 入门指南
- 文档
- 论坛
- 官方 GitHub 仓库

3.6 Podman

难度 高级 | **篇幅** 31.2 KB | **所属** 沙箱

本节将讲到

- 安装
- 配置
- Quadlet
- 镜像
- 故障排除

Podman 是 Docker 的替代品，提供类似的接口。它支持无根容器和为 *docker-compose* 提供的垫片服务。

安装

安装 podman包 软件包。

Podman 依赖 netavark包 作为有根容器的默认网络后端（参见 podman-network(1)）。Netavark 依赖 aardvark-dns包 实现同一网络中容器间的名称解析。对替代网络后端（CNI，cni-plugins包）的支持已弃用。

若想替代 Docker，可以安装 podman-docker包 来模拟 docker 二进制文件及手册页。

与 Docker 不同，Podman 不需要守护进程，但可通过 cockpit-podman包 为 cockpit 等服务提供 API。

关于构建容器的高级用法，请参阅基于 Buildah 的 podman-build(1)。

配置

容器行为配置文件位于 `/usr/share/containers/`。编辑前需将必要文件复制到 `/etc/containers`。要配置 Podman 使用的网络桥接接口，请参阅 `/etc/cni/net.d/87-podman.conflist`。

镜像仓库

Arch Linux 默认未配置容器镜像仓库 ，这意味着类似 `podman search httpd` 这种未限定仓库的搜索命令将无法工作。要使 Podman 行为与 Docker 一致，需配置 containers-registries.conf(5)：

```
/etc/containers/registries.conf.d/10-unqualified-search-registries.conf
unqualified-search-registries = ["docker.io"]
```

**本文或本章节的事实准确性存在争议。**

原因： 由 containers-common包 安装的列表位于 `/etc/containers/registries.conf.d/00-shortnames.conf`。（在 Talk:Podman 中讨论）

提示：为使未限定仓库的搜索对知名镜像更安全，可考虑将 https://github.com/containers/shortnames 的简称列表 集成到配置中，例如复制到 `/etc/containers/registries.conf.d/shortnames.conf`。

用户命名空间模式

默认情况下，Podman 容器中的进程在调用者的用户命名空间中运行，即容器未通过 user_namespaces(7) 功能隔离。这是 `--userns=host` 的行为，参见 podman-run(1)。

`--userns=auto` 标志会自动为容器创建唯一的用户命名空间，使用未分配的 UID 和 GID 范围：

- 对于 root 启动的容器，`--userns=auto` 需要在 `/etc/subuid` 和 `/etc/subgid` 文件中指定 `containers` 用户并分配未使用的 ID 范围。例如：`containers:2147483647:2147483648`。
- 对于其他用户启动的容器，将使用 `/etc/subuid` 和 `/etc/subgid` 中该用户的 ID 范围。参见 #Rootless Podman 了解必要配置。

`--userns` 标志还有其他有效值，详见 podman-run(1)。用户命名空间模式也可通过 containers.conf(5) 在全局或用户级别配置。

无根 Podman

警告：无根 Podman 依赖非特权用户命名空间（`CONFIG_USER_NS_UNPRIVILEGED`），存在严重安全隐患，详见 安全#沙盒程序。

默认只有 `root` 可运行容器（内核术语中的命名空间）。使用无根 Podman 可提升安全性（攻击者无法获得系统 root 权限），并允许多个非特权用户在同一机器上运行容器。另见 podman(1) § Rootless mode 和官方无根教程（可能已过时）。

启用 kernel.unprivileged_userns_clone

首先检查 `kernel.unprivileged_userns_clone` 的值：

```
$ sysctl kernel.unprivileged_userns_clone
```

若当前值为 `0`，可通过 sysctl 或内核参数设为 `1` 来启用。

注意：linux-hardened包 默认将 `kernel.unprivileged_userns_clone` 设为 `0`。

设置 subuid 和 subgid

用户要运行无根 Podman，必须在 subuid(5) 和 subgid(5) 中为其创建配置条目。使用 useradd(8) 创建的新用户默认已有这些条目。

为 shadow 4.11.1-3 之前版本创建的用户迁移

在 shadow包 4.11.1-3 之前版本创建的用户默认没有 `/etc/subuid` 和 `/etc/subgid` 条目。可使用 usermod(8) 命令或手动修改文件为其创建条目。

以下命令允许 `*username*` 用户和组运行 Podman 容器（或其他类型容器），为其分配指定范围的 UID 和 GID：

```
# usermod --add-subuids 100000-165535 --add-subgids 100000-165535 username
```

上述范围可能已被系统首个用户的默认范围占用。如有疑问，请先查阅 `/etc/subuid` 和 `/etc/subgid` 文件确认已分配范围。

注意：许多镜像需要 65536 个 UID/GID 进行映射（特别是基础 *busybox* 和 *alpine* 镜像）。建议为每个用户至少分配该数量的 UID/GID 以保持与 Docker 的最大兼容性。

为 homed 管理用户的变通方案

Homed 似乎不会为其用户分配 *gid* 和 *uid* 条目。可手动运行：

```
# usermod --add-subuids 524288-589823 --add-subgids 524288-589823 username
```

或直接以 root 身份编辑以下配置文件并添加：

```
/etc/subuid
username:524288:65536
/etc/subgid
username:524288:65536
```

这将为 `*username*` 用户分配 `524288-589823` 的 UID/GID 范围。若范围已被其他用户占用，需相应调整。

可能需要重启以应用更改。

注意：

- 此为临时方案，Podman 未官方支持 homed。
- 这是 *systemd-homed* 的 已知问题。
- 使用 Docker 似乎可行（将用户加入 `docker` 组，但有 安全隐患）。

使 subuid 和 subgid 变更生效

无根 Podman 使用暂停进程保持非特权命名空间活动，这会阻止 `/etc/subuid` 和 `/etc/subgid` 的更改在暂停进程运行时生效。要使更改生效需运行：

```
$ podman system migrate
```

此后，上述文件中指定的用户/组即可启动和运行 Podman 容器。

启用原生无根 overlay

过去需要使用 fuse-overlayfs包 在无根环境中进行 FUSE overlay 挂载。但现代 Podman 和 Linux 内核支持 *原生*无根 overlay，可获得更好性能。

注意：当使用修改后的 UID/GID 映射启动无根容器，且尚未用指定容器镜像和 UID/GID 映射创建过容器时，*原生 overlay* 相比 *fuse-overlayfs* 会有性能损失，因为必须更新磁盘上容器文件的所有 UID/GID。这对 `--userns auto` 尤其明显，因为每次调用可能使用不同的 UID/GID 映射。详情参见 Podman 性能指南。

要从 fuse-overlayfs包 迁移，运行以下命令（会删除所有已拉取镜像）：

```
$ podman system reset
```

同时确保 Podman 使用 `overlay` 驱动且 containers-storage.conf(5) 中未定义 `mount_program` 参数。遵循 Docker#启用本地覆盖差异引擎(native overlay diff engine)) 的说明。

验证原生无根 overlay 是否启用：

```
$ podman info | grep -i overlay
```

应显示 `graphDriverName: overlay` 和 `Native Overlay Diff: "true"`。

网络

Podman 依赖 passt包，其提供的 pasta 是默认的无根网络后端。

另一种无根网络后端是 slirp4netns包，在 Podman 5 之前是默认选项。

两者主要区别在 Podman 5.0 重大变更%20and%20copies%20the%20ip%20addresses%20from%20your%20main%20interface%20into%20the%20container%20namespace.)中概述：

: *Pasta 默认不进行网络地址转换（NAT），而是将主接口的 IP 地址复制到容器命名空间。*

上游的无根 Podman 缺陷解释了此变更的影响：

: *由于 pasta 复制主接口 IP 地址，容器无法通过该 IP 连接宿主机。这意味着除非有多个接口，否则必须显式传递 pasta 网络配置（通过 `containers.conf` 或运行时参数）才能实现容器间连接。*

提示：容器到宿主的通信问题已在 Podman 5.3 修复。

在 "Podman 5.0 重大变更" 博客中给出了模拟 slirp4netns 行为的示例：

```
containers.conf
[network]
pasta_options = ["-a", "10.0.2.0", "-n", "24", "-g", "10.0.2.2", "--dns-forward",
"10.0.2.3"]
```

此外，可在 `containers.conf` 的 `[network]` 部分通过 `default_rootless_network_cmd` 选择默认无根网络工具，可设为 `pasta` 或 `slirp4netns`。因此遇到问题时，可回退到 slirp4netns（需已安装）：

```
containers.conf
[network]
default_rootless_network_cmd = "slirp4netns"
```

存储

容器镜像和实例的存储配置位于 `/etc/containers/storage.conf`。

注意：使用 无根 Podman 时，可在 `$XDG_CONFIG_HOME/containers/storage.conf` 中按用户覆盖存储设置。

默认的 `overlay` 驱动经过充分测试，支持在具备该功能的文件系统（Btrfs、XFS、ZFS 等）上使用 reflink 复制 。

有关可用选项和其他配置的详细信息，请参阅 containers-storage.conf(5) § STORAGE_TABLE。

外部架构

Podman 可通过 Wikipedia:binfmt_misc 系统运行与宿主机不同 CPU 架构的镜像。

安装 qemu-user-static包 和 qemu-user-static-binfmt包 以启用该功能。

systemd 提供 `systemd-binfmt.service` 服务来启用新规则。

验证 binfmt 规则是否已添加：

```
$ ls /proc/sys/fs/binfmt_misc
DOSWin         qemu-cris         qemu-ppc       qemu-sh4eb         status
qemu-aarch64   qemu-m68k         qemu-ppc64     qemu-sparc
qemu-alpha     qemu-microblaze   qemu-riscv64   qemu-sparc32plus
qemu-arm       qemu-mips         qemu-s390x     qemu-sparc64
qemu-armeb     qemu-mipsel       qemu-sh4       register
```

现在 Podman 应能运行外部架构镜像。大多数命令通过 `--arch` 选项指定架构。

示例：

```
# podman run --arch arm64 'docker.io/alpine:latest' arch
aarch64
```

Docker Compose

Podman 的 *compose* 子命令是 compose 提供器的薄封装层，支持 docker-compose包 或 podman-compose包。若两者都安装，优先使用 *docker-compose*。可通过 `PODMAN_COMPOSE_PROVIDER` 环境变量覆盖此行为。

若使用 *docker-compose*，需为当前用户启用 `podman.socket` 用户单元并设置 docker socket 环境变量：

```
$ export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock
```

使用 *podman-compose* 时无需此操作，因其直接调用 *podman*。

注意：

- 若在 docker 中启用了 buildkit，集成将无法工作。需通过设置 `DOCKER_BUILDKIT=0` 环境变量禁用 buildkit。
- podman-compose包 存在兼容性问题，例如 环境变量传递行为与 docker-compose 不一致。

NVIDIA GPU

NVIDIA Container Toolkit 为 NVIDIA GPU 提供容器运行时。安装 nvidia-container-toolkit包 软件包，其包含的 pacman hook 会生成 GPU 的 CDI 规范并保存至 `/etc/cdi/nvidia.yaml`。

测试配置：

```
$ podman run --rm --gpus all archlinux nvidia-smi -L
```

注意：NVIDIA CDI hook 无法与 `--userns nomap` 和 `--userns auto` podman run 参数共同使用 。

带重启策略的容器

要自动启动带重启策略的容器，请启用 `podman-restart.service`。

Quadlet

Quadlet 允许通过 systemd 管理 Podman 容器。

对于无根(rootless) Podman，将 Quadlet 文件放置于以下目录之一：

- `$XDG_CONFIG_HOME/containers/systemd/` 或 `~/.config/containers/systemd/`
- `/etc/containers/systemd/users/*UID*` 对应指定 `*UID*` 的用户
- `/etc/containers/systemd/users/` 适用于所有用户

对于有根(rootful) Podman，目录为 `/etc/containers/systemd/`。

Podman 会读取扩展名为 *.container*、*.volume*、*.network*、*.kube*、*.image* 和 *.pod* 的 Quadlet 文件，并通过 systemd.generator(7) 生成对应的 *.service* 文件。Quadlet 文件会在系统启动时读取，或通过运行 daemon-reload 手动加载。

您也可以使用 podletAUR 从 Podman 命令生成 Quadlet 文件。

例如，以下命令用于运行 LinuxServer.io 的 Syncthing 容器：

```
$ podman run \
    --rm \
    --replace \
    --label io.containers.autoupdate=registry \
    --name syncthing \
    --hostname=syncthing \
    --uidmap 1000:0:1 \
    --uidmap 0:1:1000 \
    --uidmap 1001:1001:64536 \
    --env PUID=1000 \
    --env PGID=1000 \
    --env TZ=Etc/UTC \
    --publish 127.0.0.1:8384:8384/tcp \
    --publish 22000:22000/tcp \
    --volume /path/to/syncthing/config:/config \
    --volume /path/to/data1:/data1 \
    lscr.io/linuxserver/syncthing:latest
```

要将其作为 systemd 服务管理，创建以下 Quadlet 文件：

```
~/.config/containers/systemd/syncthing-lsio.container
[Unit]
Description=Syncthing 容器

# 容器之间可以使用 systemd 依赖项进行关联，但需添加 ".service" 后缀。
# 例如：若要让另一个容器等待本容器启动，可在其 [Unit] 部分添加 "After=syncthing-lsio.service"

[Container]
ContainerName=syncthing
Image=lscr.io/linuxserver/syncthing:latest

# 启用容器自动更新
```

```
AutoUpdate=registry

Volume=/path/to/syncthing/config:/config
Volume=/path/to/data1:/data1

HostName=syncthing
PublishPort=127.0.0.1:8384:8384/tcp
PublishPort=22000:22000/tcp

Environment=PUID=1000
Environment=PGID=1000
Environment=TZ=Etc/UTC

# UID 映射是运行 linuxserver.io 无根容器所必需的。
# 这会将容器内的 UID=1000 映射到中间 UID=0。
# 对于无根 Podman，中间 UID=0 将被映射到当前用户的 UID。
UIDMap=1000:0:1
UIDMap=0:1:1000
UIDMap=1001:1001:64536

[Service]
Restart=on-failure

# 延长超时时间以允许拉取镜像
TimeoutStartSec=300

# [Install] 部分用于启用生成的服务
[Install]
WantedBy=default.target
```

可通过以下命令验证 Quadlet 文件：

```
$ /usr/lib/podman/quadlet -dryrun -user
```

然后 reload 并启动`syncthing-lsio.service`。

注意：

若无根容器未按预期在启动时运行，请检查 `podman-user-wait-network-online.service` 的状态。若因超时失败，可能是由于缺少激活 `network-online.target` 的服务，此时可create、reload 并 enable 一个虚拟服务解决。参考 和 。

```
/etc/systemd/system/podman-network-online-dummy.service
[Unit]
Description=此服务仅用于激活 network-online.target
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/bin/echo 激活 network-online.target 中...

[Install]
WantedBy=default.target
```

参阅 Systemd/User#Automatic_start-up_of_systemd_user_instances 了解如何在没有开放会话时启动无根容器。

`Container` 部分的可用选项列于 [podman-systemd.unit(5) § Container units \[Container\]](https://man.archlinux.org/man/podman-systemd.unit.5#Container_units_%5BContainer%5D)。`PodmanArgs=` 可用于添加没有对应文件选项的其他 Podman 参数。

更多示例（包括 `Pod`、`Volume`、`Network` 和 `Image` 单元）请参阅 podman-systemd.unit(5) § EXAMPLES。

镜像

注意：可以省略镜像的注册表前缀，因为 Podman 会自动在 `/etc/containers/registries.conf` 中 `unqualified-search-registries` 列出的所有注册表中按顺序搜索镜像。以下示例将始终包含前缀，以兼容未配置 `docker.io` 的情况。

Arch Linux

以下命令从 Docker Hub 拉取 Arch Linux x86_64 镜像：

```
# podman pull docker.io/archlinux
```

完整可用标签列表（包含带/不带构建工具的版本）请参阅 Docker Hub 页面。

另见 README.md。

Alpine Linux

Alpine Linux 是小型容器镜像的热门选择，尤其适合静态编译的软件。以下命令从 Docker Hub 拉取最新 Alpine Linux 镜像：

```
# podman pull docker.io/alpine
```

Alpine Linux 使用 musl 作为 C 库实现，而非大多数 Linux 发行版使用的 glibc。由于 Arch Linux 使用 glibc，宿主机与 Alpine Linux 容器间存在许多功能差异，可能影响软件性能和正确性。这些差异列于 https://wiki.musl-libc.org/functional-differences-from-glibc.html。

注意：在 Arch Linux（或其他 glibc 系统）上构建的动态链接软件在 Alpine Linux（或其他不同 C 库系统）上运行时可能出现错误和性能问题。示例参见 、 和 。

CentOS

以下命令从 Docker Hub 拉取最新 CentOS 镜像：

```
# podman pull docker.io/centos
```

各 CentOS 版本的完整标签列表请参阅 Docker Hub 页面。

Debian

以下命令从 Docker Hub 拉取最新 Debian 镜像：

```
# podman pull docker.io/debian
```

各 Debian 版本的标准版和精简版标签列表请参阅 Docker Hub 页面。

故障排除

为进程添加暂停

```
WARN Failed to add pause process to systemd sandbox cgroup: Process
org.freedesktop.systemd1 exited with status 1
```

可通过以下方式解决：https://github.com/containers/crun/issues/704

```
# echo +cpu +cpuset +io +memory +pids > /sys/fs/cgroup/cgroup.subtree_control
```

容器在 Shell 退出后终止

部分用户在注销后 Podman 容器会停止。要避免此问题，请为运行容器的用户启用 lingering。

也可按 podman-auto-update(1) § EXAMPLES 创建用户 systemd 单元。

无根模式提交时报错

```
Error committing the finished image: error adding layer with blob
"sha256:02823fca9b5444c196f1f406aa235213254af9909fca270f462e32793e2260d8": Error
processing tar file(exit status 1) permitted operation
```

检查存储配置中存储驱动是否为 overlay。

在无根模式下使用桥接网络创建容器时报错

若使用 AppArmor，在启用 `dnsname` 插件的情况下创建桥接网络容器时可能遇到问题：

```
$ podman network create foo
/home/用户/.config/cni/net.d/foo.conflist
$ podman run --rm -it --network=foo docker.io/library/alpine:latest ip addr
Error: command rootless-cni-infra [alloc
89398a9315256cb1938075c377275d29c2b6ebdd75a96b5c26051a89541eb928 foo
festive_hofstadter    ] in container
1f4344bbd1087c892a18bacc35f4fdafbb61106c146952426488bc940a751efe failed with status 1,
stdout="", stderr="exit status 3\n"
```

解决方法是在 `/etc/apparmor.d/local/usr.sbin.dnsmasq` 中添加：

```
owner /run/user/[0-9]*/containers/cni/dnsname/*/dnsmasq.conf r,
owner /run/user/[0-9]*/containers/cni/dnsname/*/addnhosts r,
owner /run/user/[0-9]*/containers/cni/dnsname/*/pidfile rw,
```

然后重新加载 AppArmor 配置：

```
# apparmor_parser -R /etc/apparmor.d/usr.sbin.dnsmasq
# apparmor_parser /etc/apparmor.d/usr.sbin.dnsmasq
```

找不到镜像

**本文或本章节可能需要合并到#Registries。**

附注： 相关内容应合并至配置章节。（在 Talk:Podman 中讨论）

默认情况下注册表列表未填充（包内文件来自上游）。这意味着若未指定注册表，尝试拉取任何镜像都会出现如下错误：

```
Error: short-name "archlinux" did not resolve to an alias and no unqualified-search
registries are defined in "/etc/containers/registries.conf"
```

初始配置示例：

```
/etc/containers/registries.conf.d/00-unqualified-search-registries.conf
unqualified-search-registries = ["docker.io"]
/etc/containers/registries.conf.d/01-registries.conf
[[registry]]
location = "docker.io"
```

此配置等价于 Docker 的默认设置。

另一种兼容性更高但不太便捷的方式是在 `Containerfile` 或 `Dockerfile` 中使用完整注册表路径：

```
Containerfile
FROM docker.io/archlinux/archlinux
```

权限被拒绝: OCI permission denied

```
$ podman exec openvas_openvas_1 bash
Error: crun: writing file
`/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/user.slice/libpod-
b3e8048a9b91e43c214b4d850ac7132155a684d6502e12e22ceb6f73848d117a.scope/container/cgroup
.procs`: Permission denied: OCI permission denied
```

解决方法见 BBS#253966：

```
$ env DBUS_SESSION_BUS_ADDRESS= podman ...
$ env DBUS_SESSION_BUS_ADDRESS= podman-compose ...
```

推送镜像至 Docker Hub: 访问被拒/需身份验证

使用 `podman push` 推送镜像至 Docker Hub 时可能遇到 `Requested access to the resource is denied` 或 `Authentication required` 错误。以下提示可能有助解决：

- 标记本地镜像：

```
    # podman tag <本地镜像> docker.io/<DockerHub用户名>/<DockerHub仓库>:<标签>
```

- 推送标记的镜像：

```
    # podman push docker.io/<DockerHub用户名>/<DockerHub仓库>:<标签> docker://docker.io/
<DockerHub用户名>/<DockerHub仓库>:<标签>
```

- 登录相关服务：

:```

```
- 登录前先登出所有注册表：
- 在 Docker Hub 仓库的 Collaborators 标签页添加 `<DockerHub用户名>` 为协作者

### WARN\[0000\] "/" is not a shared mount, this could cause issues or missing mounts
with rootless containers

无根 Buildah/Podman 要求绑定挂载为共享模式，检查是否设置为私有：
$ findmnt -o PROPAGATION /

PROPAGATION private

若如此，参考 [mount(8) § Shared_subtree_operations]
(https://man.archlinux.org/man/mount.8#Shared_subtree_operations) 并**临时**设为共享：
要**永久**生效，请编辑 [/etc/fstab](../../01_安装与包管理/启动过程/Fstab.md#Usage) 并为对应挂载点添加 *shared* 选项后重启。示例条目：
/etc/fstab

UUID=0a3407de-014b-458b-b5c1-848e92a327a3 / ext4 defaults,shared 0 1

### 容器内网络问题
```

IP 网络

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-inaccurate.png)**本文或本章节的事实准确性存在争议。**
>
> **原因：** 此段关于防火墙影响的描述过于冗长。Netavark 使用 iptables 自动创建防火墙规则，因此 iptables 链的默认 `DROP` 策略不会造成问题。（在 [Talk:Podman](https://wiki.archlinuxcn.org/wiki/Talk:Podman) 中讨论）

Podman 容器默认通过虚拟网络接口桥接到主机。

例如容器内的虚拟接口 `eth0@if6` 拥有 IP 10.89.0.3（具体 IP 可能不同）：

容器内# ip addr

... 2: eth0@if6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000 ... inet 10.89.0.3/24 brd 10.89.0.255 scope global eth0 valid_lft forever preferred_lft forever

主机上，容器的数据包通过虚拟接口（此处为 `podman1`，路由 IP 10.89.0.1）传出：

宿主机# ip addr

... 4: podman1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000 ... inet 10.89.0.1/24 brd 10.89.0.255 scope global podman1

尽管使用虚拟 IP，数据包仍需经过内核的包过滤系统，可能被 iptables/nftables 规则拦截。特别是 iptables 过滤链中的默认 `DROP` 策略和运行的防火墙（[ufw](../../03_网络与服务/防火墙/Uncomplicated%20Firewall.md)、[firewalld](../../03_网络与服务/防火墙/Firewalld.md)）可能影响容器。若怀疑此类问题，请检查配置（如使用 `iptables -L -n -v` 或 `nft list ruleset`）。

修改 `docker-compose.yml` 后，注意使用 `podman compose down` 销毁环境时，`networks:` 部分创建的网络可能不会被删除。如需删除，请使用 `podman network ls` 和 `podman network rm` 手动操作。

DNS 与名称解析

名称解析由 Podman 子系统（如 [aardvark-dns](https://archlinux.org/packages/?name=aardvark-dns)包）处理，既提供外部 DNS（通常通过宿主的 DNS 解析器）也支持容器间名称解析（例如 `webserver.dns.podman` 访问 `database.dns.podman`）。

上例中，容器通过 `/etc/resolv.conf` 被自动配置为向宿主侧端口 53 发送 DNS 请求：

容器内# cat /etc/resolv.conf

search dns.podman nameserver 10.89.0.1

检查宿主是否运行其他占用 53 端口的 DNS 解析器（如 [Systemd-resolved](../../03_网络与服务/网络/Systemd-resolved.md) 或 [Unbound](../../03_网络与服务/DNS/Unbound.md)），这可能会干扰 Podman 名称解析。若有此情况，可将 Podman 在宿主上的监听端口更改为其他可用端口，Podman 会自动转发容器请求：

宿主# # cat /etc/containers/containers.conf

... dns_bind_port = 20053

内核不支持 overlay 文件系统：'overlay' 不支持在 <文件系统> 上使用

请按照[常规故障排除#内核升级后部分外设无法使用](../../02_系统管理与内核/系统恢复/常规故障排除.md#内核升级后部分外设无法使用)中的说明重启系统。

另请参阅

- [官方网站](https://podman.io/)
- [Podman 教程](https://github.com/containers/podman/blob/main/docs/tutorials/README.md)
- [Podman 与 Docker 对比：你需要知道的一切](https://phoenixnap.com/kb/podman-vs-docker)

3.7 Docker

难度 高级 | **篇幅** 45.8 KB | **所属** 沙箱

本节将讲到

- 安装
- 使用
- 配置
- 镜像
- 有用的建议

Docker) 是一种打包、传输和运行任何程序作为轻量级容器的实用工具。

安装

要拉取Docker镜像并运行Docker容器，你需要安装Docker引擎，其包含包括一个守护进程来管理容器，以及一个docker命令行界面前端。安装 docker包 包。下一步启用/启动 docker.service 或者 docker.socket。两者的差距在于docker.service将会在开机时启动，而docker.socket将会在第一次启动Docker时启动，使用后者可以减少开机启动时间。然后验证操作:

```
# docker info
```

注意, 如果你有一个活动的 VPN 连接, 那么 docker 服务的启动可能失败, 因为 VPN 和 Docker 的网桥 IP 冲突以及网络覆盖。如果发生了这种事, 尝试在启动 docker 服务之前断开 VPN 连接. 你可以在之后立刻重连 VPN。你也可以尝试手动解决网络冲突(也可参见或)。

你也可以尝试验证是否可以运行容器。以下命令行将会下载一个最新的Arch Linux image，并使用其在这个容器中运行一个Hello World程序:

```
# docker run -it --rm archlinux bash -c "echo hello world"
```

要删除下载的Arch Linux Docker镜像，请参见#移除 Docker 和镜像。

如果你想以普通用户身份运行docker的话，添加你自己到 docker 用户组，重新登录并重启docker.service

警告：任何加入到 `docker` 组的用户都和root用户等价，因为他们可以通过运行`# docker run --privileged`来以root权限启动容器。 参见 此处 以及 此处。

如果你还要使用 Docker 构建容器镜像，请安装docker-buildx包以使用新版构建器(否则会使用已弃用的旧版构建器)。

Docker Compose

Docker Compose是另一种Docker引擎的CLI前端,它使用`compose.yaml`YAML文件来指定容器的属性,这样就可以不使用附带指令的`docker run`脚本了。如果你需要经常设置或者使用具有复杂选项的容器,可能使用docker-compose更为方便。你需要安装docker-compose包来使用。

Docker Desktop

Docker Desktop是一个专有的桌面应用程序,它在一个Linux虚拟机中运行Docker。它还包括Kubernetes集群以及一个漏洞扫描器。这个应用程序对于在macOS或Windows上进行开发Docker容器的团队非常友好。Docker Desktop适配的Linux版本相对较新，同时也保持了对Docker CLI的良好兼容。

Docker直接为Arch Linux提供了一个实验性的软件包（参见其官方文档手动安装Docker一节）。需要注意其手动下载的软件包会与docker-compose包以及docker-buildx包冲突，你需要在安装前手动移除这两个包。如果你想保留现有的软件包，你也可以从AUR安装docker-desktopAUR，它将不会与现有软件包发生冲突。

此外，在运行Docker Desktop之前，你需要确保你已经安装了所有的在Linux上运行的最小系统要求,包括使用KVM进行虚拟化技术支持。对于Gnome用户，你还需要安装gnome-shell-extension-appindicator包以显示托盘图标。

最后，请注意文件共享功能是通过`/etc/subuid`和`/etc/subgid`映射用户和组ID完成的。详细参见Docker Desktop的Linux文件共享说明。

注意：在Linux上的Docker Desktop会在一个虚拟机(VM)上启动，这个虚拟机会单独创建并使用一个自定义的Docker上下文，即`desktop-linux`。

这意味着在安装前，部署在Linux Docker引擎上的所有镜像与容器都无法在Docker Desktop中使用。

更多信息请参见Docker Desktop for Linux和Docker Engine的区别。

此外，由于Docker Desktop是在VM上启动的，相比直接使用Docker引擎，预期性能会下降并且 CPU 使用率会提高。

默认情况下，Docker Desktop会启用一个用户级别的systemd服务，该服务会在启动时自动启动应用程序。在Docker Desktop仪表板中禁用“自动启动”设置并不能阻止此服务启动，您需要手动禁用 `docker-desktop.service`。

前端工具

- **Ducker** — 一个用于管理Docker容器的终端应用程序。

: https://github.com/robertpsoane/ducker || ducker包

- **goManageDocker** — 一个用于管理Docker对象的TUI工具。

: https://github.com/ajayd-san/gomanagedocker || gomanagedockerAUR

- **Lazydocker** — 一个简单的终端UI，用于管理docker和docker-compose，使用Go语言编写，基于gocui库。

: https://github.com/jesseduffield/lazydocker || lazydocker包

- **oxker** — 一个简单的TUI，用于查看和控制Docker容器。

: https://github.com/mrjackwills/oxker || oxker-binAUR

- **Podman Desktop** — 从单一UI和托盘中管理Podman和其他容器引擎。

: https://github.com/containers/podman-desktop || podman-desktop包

- **Portainer** — 一个轻量级的Docker管理UI。

: https://github.com/portainer/portainer || portainer-binAUR

- **Whaler** — 为Pantheon设计的Docker容器管理工具。

: https://github.com/sdv43/whaler || whaler-gitAUR

使用

Docker由多个部分组成:

- Docker守护进程(也称Docker引擎),这是一个以`docker.service`形式运行的进程。其提供了Docker API接口并管理Docker容器。
- `docker` CLI命令,其允许用户使用命令行来与Docker API交互，并控制 Docker 守护进程。
- Docker容器，这是一种命名进程，由Docker守护进程通过Docker API的请求进行管理。

一般来说，用户通过使用`docker`命令行来对Docker进行操作,命令行又通过Docker API对Docker守护进程发起请求以执行对容器的相关操作。掌握客户端 (`docker`), 服务端(`docker.service`)和容器之间的关系是很必要的。

请注意，如果Docker守护进程停止/重启，那么当前运行的所有Docker容器也会停止/重启。

你也可以不借助`docker` CLI来对Docker API发起请求来控制容器，参见Docker API开发指南。

更多使用文档请参见Docker入门指南。

配置

Docker守护进程可以通过修改配置文件`/etc/docker/daemon.json`或者直接在`docker.service`中添加命令行标志来进行配置。根据Docker官方文档, 推荐使用修改配置文件的方法进行配置。如果你想使用添加命令行标志的方法进行配置，使用Systemd#附加配置片段覆盖`docker.service`中的`ExecStart`部分。

对于`daemon.json`中的选项，参见守护进程配置文件参考。

存储驱动程序

存储驱动程序控制着Docker主机上的镜像与容器的储存与管理方式。默认的`overlay2`驱动在大部分情况下都具有良好的性能。

如果你的文件系统使用的是btrfs或者ZFS，你可以使用对应的`btrfs`或者`zfs`驱动,它们可以利用这些文件系统独有的功能，要使用这些驱动请参见btrfs驱动或zfs驱动文档。

启用守护进程TCP套接字

默认情况下,Docker守护进程使用位于`/var/run/docker.sock`的Unix套接字来提供Docker API。大部分情况下，这是一个合适的选择。

你可以将设置守护进程设置为额外监听TCP套接字,这样就能使Docker API被远程访问了(参见允许远程访问Docker API)。如果你在Windows或macOS上使用Arch虚拟机，你可以在完成设置后使用宿主机上直接使用`docker`命令行访问虚拟机中允许的Docker守护进程。

警告：Docker API默认情况下既没有加密也没有身份验证。除非你在附加Docker配置中启用了 使用SSH或TLS进行连接，否则对Docker守护进程的TCP访问等同于不安全的远程root访问。

注意默认的`docker.service`设置了`-H`标志，如果选项同时存在于标志与`/etc/docker/daemon.json`文件中，Docker将不会启动，因此最简单的更改监听TCP套接字设置的方法是使用一个附加文件。例如，如果你想在端口2376添加一个TCP套接字:

```
/etc/systemd/system/docker.service.d/docker.conf
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H unix:///var/run/docker.sock -H tcp://0.0.0.0:2376
```

重载systemd守护进程并重启 `docker.service`以应用更改。

HTTP代理

要想在Docker中使用HTTP代理，你需要同时对Docker守护进程以及Docker容器进行配置。

Docker守护进程代理设置

参见Docker文档：配置Docker守护进程使用HTTP代理。

Docker容器代理设置

参见Docker文档：如何配置代理，使用`docker`CLI来自动为所有容器配置代理。

配置DNS

参见Docker中网络配置了解Docker容器内部的DNS行为以及如何自定义Docker的DNS配置信息。一般来说，主机上的配置也会直接配置到容器中。

大部分托管在`127.0.0.0/8`上的DNS解析器都是不被支持的(由于容器和主机网络命名空间之间的冲突)。这些解析器会在容器中的/etc/resolv.conf中删除。如果这导致了`/etc/resolv.conf`为空文件,容器将会使用Google DNS。

此外，如果`127.0.0.53`是唯一的名称服务器，在这种特定的情况下Docker会假设解析器是systemd-resolved并使用来自`/run/systemd/resolve/resolv.conf`的上游DNS解析器。

如果你使用dnsmasq来提供一个本地解析器，考虑为dnsmasq添加一个虚拟接口(使用`169.254.0.0/16`网段的链路本地IP地址来绑定，而不是`127.0.0.1`)以避免网络命名空间冲突。

镜像位置

注意，从 版本 29（2025 年 11 月）开始，新安装的**默认**镜像存储后端已变更为 containerd。

当使用 containerd 镜像存储时，镜像内容和容器快照默认存储
在 /var/lib/containerd/ 目录下。如果是从旧版本升级而来，Docker 镜像仍位于 /var/lib/docker/ 目录中，直到您手动启用 containerd 镜像存储。

您可以使用以下命令来确认当前使用的镜像储存后端：

```
docker info -f '{{ .DriverStatus }}'
```

使用 containerd 镜像存储时，输出将为：

```
[[driver-type io.containerd.snapshotter.v1]]
```

您可以通过修改 配置文件 来将Docker镜像放置在其他位置(如其他分区或磁盘中)。

首先, 停止docker.service,注意，这也会停止所有当前运行的容器并卸载任何正在运行的镜像。

然后根据镜像存储后端编辑相应的配置文件：

- 对于 containerd 储存后端：

```
/etc/containerd/config.toml
version = 2
root = "/path/to/containerd-data"
```

- 对于传统Docker(overlay)储存后端：

```
/etc/docker/daemon.json
{
  "data-root": "/path/to/docker-data"
}
```

随后将 /var/lib/containerd 或 /var/lib/docker 中的内容移至目标位置。

注意：请注意，对于传统Docker(overlay)储存后端，无法仅移动镜像，必须同时移动镜像和卷。

重启docker.service以应用更改。

不安全的自建仓库

如果你使用自签名证书的仓库(registries)，或该自建仓库未使用TLS加密（即通过http连接）, docker会拒绝连接。你需要将其添加到信任列表中。 例如，要信任托管于myregistry.example.com:8443上的镜像,在文件/etc/docker/daemon.json中配置insecure-registries的值：

```
/etc/docker/daemon.json
{
  "insecure-registries": [
    "my.registry.example.com:8443"
  ]
}
```

随后重新加载 docker.service配置。

IPv6

为了开启Docker中的IPv6支持,参见\[失效链接 2025-03-15 ⓘ\]与。

首先，将`/etc/docker/daemon.json`中的`ipv6`设置为启用并设置一个特定的IPV6子网(即使用私有的`fd00::/80`子网)。请确保至少使用80位的子网，因为这样可以使容器的IPv6地址以容器的MAC地址结尾，这有助于解决NDP邻居缓存失效的问题。

```
/etc/docker/daemon.json
{
  "ipv6": true,
  "fixed-cidr-v6": "fd00::/80"
}
```

重启 `docker.service`以应用更改。

最后，为了让容器能够访问主机网络，你需要添加IPv6 NAT以解决使用私有IPv6子网时出现的路由问题:

```
# ip6tables -t nat -A POSTROUTING -s fd00::/80 ! -o docker0 -j MASQUERADE
```

现在Docker应该已经开启了IPv6支持，你可以使用以下指令来进行测试：

```
# docker run curlimages/curl curl -v -6 archlinux.org
```

如果你使用firewalld,你还需要添加防火墙规则，例如：

```
# firewall-cmd --zone=public --add-rich-rule='rule family="ipv6" destination not address="fd00::1/80" source address="fd00::/80" masquerade'
```

如果你使用ufw,你还需要根据Uncomplicated Firewall#转发策略创建Ipv6转发。

首先，你需要编辑`/etc/default/ufw`并取消以下几行的注释：

```
/etc/ufw/sysctl.conf
net/ipv6/conf/default/forwarding=1
net/ipv6/conf/all/forwarding=1
```

现在你可以使用以下命令添加iptables规则:

```
# ip6tables -t nat -A POSTROUTING -s fd00::/80 ! -o docker0 -j MASQUERADE
```

如果你使用*docker-compose*来创建的容器,你可能还需要在`networks`中对应的部分设置`enable_ipv6: true`。另外，你可能还需要手动指定IPv6子网，参见compose-file中的ipv6地址设置。

用户命名空间隔离

默认情况下，Docker中的进程和`dockerd`主守护程序运行在同一用户命名空间中，即容器不会通过用户命名空间隔离(参见user_namespaces(7))。这将会允许进程根据用户和用户组#权限与属主在主机上来访问已配置的资源。这样提升了容器运行的兼容性，但是一旦出现了一个允许容器中进程访问非预期资源的漏洞，这会带来很大的安全隐患。(一个这样的漏洞在2019年2月发布并修补。)

启用用户命名空间隔离可以降低此类漏洞的影响。其将会在单独的用户命空间中运行每个容器，并将这个空间中的UIDs/GIDs映射到主机上不同的(通常情况下也是非特权的)UIDs/GIDs。

注意：

- 主dockerd守护程序依然是以root身份在主机上运行的，在非root身份下运行docker(rootless mode)是另一个功能。
- 容器中的进程将会以Dockerfile中定义的USER指令定义的用户身份启动。
- 所有容器都会映射到相同的UID/GID范围，这是为了让容器之间的共享卷功能生效。
- 在一些情况下无法启用用户命名空间。
- 由于 Docker 需要调整这些资源的所有权，因此启用用户命名空间隔离会有效屏蔽现有的映像层和容器层，以及 /var/lib/docker/ 中的其他 Docker 对象。上游文档建议仅在新安装的Docker上启用此功能，而不是在现有的Docker上启用。

在/etc/docker/daemon.json中配置userns-remap的值。default是一个特殊值，其会自动创建名为dockremap的用户与用户组用于重映射。

```
/etc/docker/daemon.json
{
  "userns-remap": "default"
}
```

在/etc/subuid和/etc/subgid中配置用户名/组名，UID/GID的范围。在这个例子中，dockremap用户/用户组分配为从165536开始的65536个UIDs/GIDs。

```
/etc/subuid
dockremap:165536:65536
/etc/subgid
dockremap:165536:65536
```

重启docker.service以应用更改。

应用此更改后，默认情况下所有容器都将在隔离的用户命名空间中运行。你也可以在docker命令中加上添加标志--userns=host来在特定的容器中禁用用户命名空间隔离，参见。

非特权模式运行Docker守护程序(Docker rootless)

注意：非特权模式下运行Docker守护程序(Docker rootless)依赖于非特权用户命名空间(CONFIG_USER_NS_UNPRIVILEGED)。在linux包, linux-lts包, 和linux-zen包内核中默认启用这一功能。如果你使用其他版本的内核，你可能需要手动启用这一功能。这可能带来一些安全隐患，参见安全#沙盒程序。

要将Docker守护程序作为普通用户运行，安装 docker-rootless-extrasAUR软件包。

随后在/etc/subuid与/etc/subgid中配置用户名/用户组名，起始 UID/GID 和 UID/GID 范围大小，以分配重新映射的用户和组。以下是一个示例：

```
/etc/subuid
your_username:165536:65536
/etc/subgid
your_username:165536:65536
```

启用 docker.socket systemd/用户单元: 这将会使用systemd的套接字激活来启动docker。

最后设置docker套接字的环境变量:

```
$ export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
```

启用本地覆盖差异引擎(native overlay diff engine)

默认情况下Docker无法在Arch Linux上使用本地覆盖差异引擎(native overlay diff engine),这会导致构建Docker镜像很慢。如果你经常构建镜像，请按照以下步骤配置:

```
/etc/modprobe.d/disable-overlay-redirect-dir.conf
options overlay metacopy=off redirect_dir=off
```

随后停止`docker.service`, 重新加载`overlay`内核模块:

```
# modprobe -r overlay
# modprobe overlay
```

重载内核模块后，启动`docker.service`。

要验证是否成功启用，你可以运行`docker info`检查`Native Overlay Diff`值是否为`true`。

镜像

Arch Linux

下面的命令会拉取 archlinux x86_64 image。这是一个arch内核的剥离版本，没有网络等功能。

```
# docker pull archlinux
```

另请参阅 README.md。

对于完整的arch镜像，可以从下面克隆镜像并且建立你自己的基础镜像。

```
$ git clone https://gitlab.archlinux.org/archlinux/archlinux-docker.git
```

请确保devtools包, fakechroot包以及fakeroot包软件包已被安装。

随后编辑构建文件并运行：

```
# make docker-image
```

Alpine Linux

Alpine Linux是一个热门的小型容器镜像，其比较适合运行静态二进制形式软件。使用以下命令来拉取最新的Alpine Linux镜像:

```
# docker pull alpine
```

Alpine Linux使用musl libc实现，这有区别与大部分的Linux发行版使用的glibc libc实现。 由于Arch Linux使用的glibc,因此Arch Linux主机与Alpine Linux容器之间存在有功能差异，这可能会影响软件性能或正确性。你可以在此处查看存在的差异。

注意，在Arch Linux(或其他没有使用musl libc实现的发行版)上编译的动态链接软件在Alpine Linux (或其他使用musl libc的镜像)上可能会出现错误或性能问题。参见, 和。

Debian

下面的命令以拉取Debian镜像 debian x86_64。

```
# docker pull debian
```

请参阅Docker Hub页面查看可用标签的完整列表，包括每个Debian版本的标准版与精简版。

手动

用 debootstrap包建立Debian镜像:

```
# mkdir jessie-chroot
# debootstrap jessie ./jessie-chroot http://http.debian.net/debian/
# cd jessie-chroot
# tar cpf - . | docker import - debian
# docker run -t -i --rm debian /bin/bash
```

Distroless

Google 维护着distroless镜像，这是没有基本操作系统组件(例如shells和包管理器)的最小化镜像。其最小镜像`gcr.io/distroless/static-debian11`仅有2MiB左右，可以用于打包软件并生成非常小的镜像。

请参阅GitHub中的README以获得镜像列表以及在不同编程语言中的使用方法。

有用的建议

抓取运行容器的IP地址

抓取运行容器的IP地址:

```
$ docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
<container-name OR id>
172.17.0.37
```

每个正在运行的容器，它们的名字和相关IP地址都能被列出来在 `/etc/hosts`里用:

```
#!/usr/bin/env sh
for ID in $(docker ps -q | awk '{print $1}'); do
    IP=$(docker inspect --format="{{range .NetworkSettings.Networks}}{{.IPAddress}}
{{end}}" "$ID")
    NAME=$(docker ps | grep "$ID" | awk '{print $NF}')
    printf "%s %s\n" "$IP" "$NAME"
done
```

运行容器中的图像程序

本节介绍了允许在主机的X服务器上运行图形程序（包括依赖于OpenGL或Vulkan的程序）所需的步骤。

首先，需要在容器内安装与**主机**图形硬件兼容的正确驱动程序。如果容器使用的Arch Linux镜像，请参见OpenGL#安装或 Vulkan#安装来安装对应的驱动。

接下来，你需要授予容器访问主机上X服务的权限。在单用户环境中，你可以通过在主机中运行Xhost来完成这一操作。该命令将非网络本地连接添加到访问控制列表：

```
$ xhost +local:
```

最后，你需要在`docker run`中传递以下参数:

- `-e "DISPLAY=$DISPLAY"`将环境变量`DISPLAY`设置为主机显示器;
- `--mount type=bind,src=/tmp/.X11-unix,dst=/tmp/.X11-unix`将主机X服务器套接字挂载到相同路径下的容器内;
- `--device=/dev/dri:/dev/dri`允许容器直接访问主机上的直接渲染(DRI)设备。

为了验证设置是否生效，请在容器中运行mesa-utils包中的`glxgears`命令(或者vulkan-tools包中的`vkcube`命令)。

开机启动 Docker Compose 项目

**本文或本章节的事实准确性存在争议。**

原因： 没有必要在`compose.yml`中启用`restart: always`(讨论请参见)。（在 Talk:Docker#"开机启动 Docker Compose 项目" 是否多余? 中讨论）

首先，创建一个用于Docker Compose的Systemed单元，并通过服务名称进行参数化(参见systemd.service(5) § SERVICE TEMPLATES):

```
/etc/systemd/system/docker-compose@.service
[Unit]
Description=%i service with docker compose
Requires=docker.service
After=docker.service

[Service]
WorkingDirectory=/opt/%i
ExecStartPre=-/usr/bin/docker compose pull
ExecStart=/usr/bin/docker compose up --remove-orphans
ExecStop=/usr/bin/docker compose down
ExecReload=/usr/bin/docker compose pull
ExecReload=/usr/bin/docker compose up --remove-orphans

[Install]
WantedBy=multi-user.target
```

随后，对于你想运行的每一个服务，在`/opt/*project_name*`目录下新建一个包含Compose文件以及其他所需文件(例如`.env`文件)。

最后, 启用/启动 `docker-compose@*project_name*.service`。

使用buildx进行交叉编译

buildx CLI 插件使用了新的BuildKit构建工具包。安装docker-buildx包，buildx接口支持构建多平台镜像(包括与主机不同的框架)。

交叉编译镜像也需要QEMU。如果你想在Docker中设置静态版本的QEMU，请参见multiarch/qemu-user-static 镜像。否则请在主机上设置QEMU与Docker共同使用(参见QEMU#从 x86_64 环境中 Chroot 至 arm/arm64 环境)。无论哪种情况，你的系统都将配置为对客户端架构进行用户模式模拟。

```
$ docker buildx ls
```

```
NAME/NODE DRIVER/ENDPOINT STATUS  PLATFORMS
default * docker
  default default         running linux/amd64, linux/386, linux/arm64, linux/riscv64,
linux/s390x, linux/arm/v7, linux/arm/v6
```

用NVIDIA GPU运行GPU加速的Docker容器

从19.03版本开始，Docker原生支持NVIDIA GPU作为Docker设备。 推荐使用NVIDIA Container Toolkit来运行需要操作NVIDIA显卡的容器。 安装 nvidia-container-toolkit包包并重启Docker。之后可以用`--gpus`选项来运行使用NVIDIA显卡的容器

使用 --gpus 选项（推荐）

```
# docker run --gpus all nvidia/cuda:12.1.1-runtime-ubuntu22.04 nvidia-smi
```

指定容器内可使用多少GPU：

```
# docker run --gpus 2 nvidia/cuda:12.1.1-runtime-ubuntu22.04 nvidia-smi
```

指定使用哪一个GPU：

```
# docker run --gpus '"device=1,2"' nvidia/cuda:12.1.1-runtime-ubuntu22.04 nvidia-smi
```

或

```
# docker run --gpus '"device=UUID-ABCDEF,1"' nvidia/cuda:12.1.1-runtime-ubuntu22.04
nvidia-smi
```

**本文或本章节的事实准确性存在争议。**

原因： More information on when the following error happens is needed. It should work, see \[失效链接 2023-04-23 ⓘ\]. （在 Talk:Docker#GPU accelerated Docker Nvidia 中讨论）

如果在执行指令时收到错误`Failed to initialize NVML: Unknown Error`,你可以尝试详细指定GPU来解决这个问题:

```
# docker run --gpus all --device /dev/nvidiactl:/dev/nvidiactl --device /dev/nvidia-
uvm:/dev/nvidia-uvm --device /dev/nvidia0:/dev/nvidia0 nvidia/cuda:12.1.1-runtime-
ubuntu22.04 nvidia-smi
```

指定需要的具体功能（图像、计算等）

```
# docker run --gpus all,capabilities=utility nvidia/cuda:12.1.1-runtime-ubuntu22.04
nvidia-smi
```

参阅 container-toolkit文档与安装指南。

使用 NVIDIA Container Runtime

编辑`/etc/docker/daemon.json`以注册NVIDIA运行时环境。

```
/etc/docker/daemon.json
{
  "runtimes": {
    "nvidia": {
      "path": "/usr/bin/nvidia-container-runtime",
      "runtimeArgs": []
    }
  }
}
```

之后重启 Docker。

运行时也可以通过*dockerd*的一个命令行选项来注册。

```
# /usr/bin/dockerd --add-runtime=nvidia=/usr/bin/nvidia-container-runtime
```

完成后可通过命令启动GPU加速的容器:

```
# docker run --runtime=nvidia nvidia/cuda:9.0-base nvidia-smi
```

或（要求 Docker 版本19.03或更高）

```
# docker run --gpus all nvidia/cuda:9.0-base nvidia-smi
```

参阅 README.md。

有CUDA的 Arch Linux 镜像

可使用以下`Dockerfile` 构建自定义的有CUDA的 Arch Linux 镜像。它使用 Dockerfile frontend syntax 1.2 在宿主机上缓存pacman包。请注意，你必须在构建镜像之前设置环境变量`DOCKER_BUILDKIT=1`。

```
Dockerfile
# syntax=docker/dockerfile:1.2

FROM archlinux:base-devel

# 安装包
RUN --mount=type=cache,sharing=locked,target=/var/cache/pacman \
    pacman -Syu --noconfirm --needed cuda

# 配置 nvidia container runtime
# https://github.com/NVIDIA/nvidia-container-toolkit/tree/main/cmd/nvidia-container-
runtime#environment-variables-oci-spec
ENV NVIDIA_VISIBLE_DEVICES=all
ENV NVIDIA_DRIVER_CAPABILITIES=compute,utility
```

移除 Docker 和镜像

如果你想完全移除 Docker，你可以通过下面的步骤完成：

注意：不要仅仅只是复制粘贴下面的命令而不知道你在干什么！

检查正在运行的容器：

```
# docker ps
```

列出在主机运行的所有容器，为删除做准备：

```
# docker ps -a
```

停止一个运行的容器：

```
# docker stop <CONTAINER ID>
```

杀死还在运行的容器：

```
# docker kill <CONTAINER ID>
```

通过 ID 删除列出的所有容器：

```
# docker rm <CONTAINER ID>
```

列出所有的 Docker 镜像：

```
# docker images
```

通过 ID 删除所有镜像：

```
# docker rmi <IMAGE ID>
```

删除所有与容器没有关联的镜像、容器、卷与网络（悬空）：

```
# docker system prune
```

要删除所有停止的容器和所有未使用的镜像（而不只是悬空镜像），添加 `-a` 标志：

```
# docker system prune -a
```

删除所有 Docker 数据（清除目录）：

```
# rm -R /var/lib/docker
```

故障排除

使用systemd-networkd时,docker0 网桥无法获取 IP / Internet 到容器

Docker会自己启用IP转发，但是默认 systemd-networkd 会覆盖对应的sysctl设置，在网络配置文件里设置 `IPForward=yes`。查阅网络分享#启用包转发获取细节。

当systemd-networkd尝试管理由Docker创建的网络时，如果你在`Match`部分设置了`Name=*`或`Type=ether`，这可能会导致网络连接出现问题。请更为具体地配置接口，即尽可能避免使用`Name=*`, `Type=ether`等通配符来匹配Docker管理的接口。你可以验证 `networkctl list` 是否在 Docker 创建的所有网络的 SETUP 栏中设置为 `unmanaged`。

注意：

- 你可能需要在每次 重启 `systemd-networkd.service` 或者 `iptables.service` 之后，再手动重启 `docker.service`。
- 注意nftables默认情况下可能会禁止Docker的连接，你可以使用命令 `nft list ruleset` 来检查规则集。
- 你可以使用 `nft flush chain inet filter forward` 来临时移除所有转发规则，或者编辑 `/etc/nftables.conf` 文件使其永久生效。在更改了配置文件，请 重启 `nftables.service` 以应用更改。详细了解Docker的nftables支持请参见 。

默认的允许的进程/线程数太少

如果你允许时得到下面的错误信息

```
# e.g. Java
java.lang.OutOfMemoryError: unable to create new native thread
# e.g. C, bash, ...
fork failed: Resource temporarily unavailable
```

那么你可能需要调整被systemd允许的进程数, 编辑并添加下面片段 `docker.service`：

```
# systemctl edit docker.service
[Service]
TasksMax=infinity
```

对于更多参数,例如 `DefaultLimitNPROC`，请参阅 systemd-system.conf(5) § OPTIONS。对于 `TasksMax` 请参阅 systemd.resource-control(5) § OPTIONS。

无法创建到某文件的路径: 设备没有多余的空间了

如果你在创建或者运行Docker镜像时获取到类似的错误信息:

```
ERROR: Failed to create some/path/to/file: No space left on device
```

如此时磁盘还有多余的空间，请确保:

- Tmpfs 被禁用了并且有足够的内存分配. Docker可能会尝试写入文件到 `/tmp` 但是失败了因为内存使用的限制和磁盘空间不足.
- 如果你在使用 XFS, 你可能得从相关入口移除 `noquota` 挂载选项在 `/etc/fstab`里 (通常是 `/tmp` 和/或 `/var/lib/docker` 在的地方). 查阅 Disk quota 获取更多信息, 特别是你计划使用和调整 `overlay2` Docker 存储驱动.
- XFS 的配额挂载选项在文件系统重新挂载时 (`uquota`, `gquota`, `prjquota`, 等等.) 失败了. 为了为root文件系统启用配额挂载选项必须作为内核参数 `rootflags=`传递到initramfs. 之后, 它就不应该在 `/etc/fstab`中的挂载选项中列出root (`/`) 文件系统.

注意：XFS配额和标准LinuxDisk quota, 是有区别的。这里值得一读。

Docker-machine无法使用virtualbox驱动程序创建虚拟机

如果docker-machine 无法使用 virtualbox 驱动程序创建虚拟机并提供了以下信息:

```
VBoxManage: error: VBoxNetAdpCtl: Error while adding new interface: failed to
open /dev/vboxnetctl: No such file or directory
```

尝试在CLI中使用`vboxreload`重启virtualbox。

启动Docker后会破坏KVM的桥接网络

这是因为启动Docker的脚本会添加一些iptables规则,其会阻止除自身外的其他接口进行转发。这是一个已知问题。

您可以尝试以下的解决方案(请将br0替换为您自己的桥接网络名称):

- 最快的解决方案(但是这会关闭所有Docker自动添加的iptables规则):

```
/etc/docker/daemon.json
{
  "iptables": false
}
```

- 如果您已经配置好了用于KVM的桥接网络，您也可以通过修改Docker的配置文件来解决这个问题，参见将配置文件修改为：

```
/etc/docker/daemon.json
{
  "bridge": "br0"
}
```

- 如果上述方法无效，或者您希望直接通过iptables或者类似于UFW的管理器来解决问题，请添加以下内容:

```
iptables -I FORWARD -i br0 -o br0 -j ACCEPT
```

更详细的解决方案可以参见此处。

从Docker Hub拉取的镜像受到速率限制

从2020年11月1日开始，Docker Hub限制了匿名用户以及免费账户用户的下载速率，详细请参见速率限制文档。

匿名用户的速率限制通过IP进行跟踪，免费账户用户的速率限制通过账户进行跟踪。

如果你需要更高的下载速率，你可以注册付费计划或者将你需要的镜像拉取到不同的镜像仓库。你可以自建镜像仓库或者使用云托管的镜像站点，例如 Amazon ECR, Google Container Registry, Azure Container Registry 或 Quay Container Registry。

使用Docker CLI 中的 `pull`, `tag` 以及 `push` 命令来镜像一个镜像。例如，将 Nginx 标签(tag)为 `1.19.3` 的镜像镜像到托管于 `cr.example.com` 上的仓库:

```
$ docker pull nginx:1.19.3
$ docker tag nginx:1.19.3 cr.example.com/nginx:1.19.3
$ docker push cr.example.com/nginx:1.19.3
```

随后你可以从镜像仓库中拉取并运行镜像:

```
$ docker pull cr.example.com/nginx:1.19.3
$ docker run cr.example.com/nginx:1.19.3
```

错误提示：iptables (旧版): 未知选项 "--dport"

**本文或本章节的事实准确性存在争议。**

原因： Nftables#与 Docker 一起工作 建议不要使用 iptables-nft包。（在 Talk:Docker 中讨论）

如果你在运行容器时收到了这样的错误： `iptables (legacy): unknown option "--dport"`

安装 iptables-nft包 而不是 iptables包 (旧版) 然后重启。

注意：使用iptables-nft包可能会导致docker 混合 nftables 和 iptables 规则，详细参见Nftables#与 Docker 一起工作。

运行docker login时提示"密码以非加密形式储存"

默认情况下 Docker 会尝试使用 `pass` 或者 `secretservice` 的二进制形式文件来储存你的注册表密码。如未能找到这些文件，注册表秘密将以明文形式(base64编码)储存在 `$HOME/.docker/config.json` ，并在登录成功后提示:

```
$ WARNING! Your password will be stored unencrypted
in /home/username/.docker/config.json.
```

如果你在使用支持 Secret Service Freedesktop DBUS API 的密码管理器,例如KDE的kwallet包 或着 GNOME的 gnome-keyring包，你可以安装 docker-credential-secretserviceAUR 来让你的注册表密码储存在密码管理器中。

"无法在默认选项中找到可用的，不重叠的IPv4地址池进行分配"

如果你在使用大量的 Docker 项目 (例如使用 docker-compose)，可能会出现Docker容器的可用IP地址不足的情况：

```
Could not find an available, non-overlapping IPv4 address pool among the defaults to assign to the network
```

参见这个 Docker issue, 默认选项是:

Type	Default Size	Default Pool
local	/16	172.17.0.0/12
local*	/20	192.168.0.0/16

你可以在 `/etc/docker/daemon.json` 文件中修改 `default-address-pools`，将其中第一个IP范围的值从16改为24解决这个问题。为了避免本地网络发生IP冲突，请不要修改第二个IP范围的值。

```
/etc/docker/daemon.json
{
  ...
  "default-address-pools" : [
    {
      "base" : "172.17.0.0/12",
      "size" : 24
    },
    {
      "base" : "192.168.0.0/16",
      "size" : 24
    }
  ]
}
```

重启 `docker.service` 以应用更改。

更多详细信息和技术解释请参见文章: Docker默认地址池选项的权威指南。

Golang编译速度过慢

由于ulimit配置的原因，使用makepkg构建docker镜像以及其依赖时会非常缓慢（会在"Entering fakeroot environment..."处卡住）。

这是因为 ，你可以尝试将 `--ulimit "nofile=1024:524288"` 添加到你的docker构建选项中来解决问题：

```
/etc/docker/daemon.json
{
  "default-ulimits": {
    "nofile": {
      "Name": "nofile",
      "Soft": 1024,
      "Hard": 524288
    }
  }
}
```

另请参阅

- 官方网站
- Arch Linux on docs.docker.com
- Docker 容器真的安全吗？——opensource.com
- Awesome Docker
- 为什么在 Docker 中运行特权容器是个坏主意

3.8 Systemd-nspawn

难度 高级 | **篇幅** 50.6 KB | **所属** 沙箱

本节将讲到

- 安装
- 用例
- 管理
- 配置
- 提示和技巧

systemd-nspawn 跟 chroot 命令类似，是个终极版的 chroot。

systemd-nspawn 可以在轻量的命名空间容器中运行命令或系统。它比 chroot 强大的地方是，它完全虚拟了文件系统层次结构、进程树、各种 IPC 子系统以及主机名。

systemd-nspawn 将容器中各种内核接口的访问限制为只读，像是 `/sys`, `/proc/sys` 和 `/sys/fs/selinux`。网络接口和系统时钟不能从容器内更改，不能创建设备节点。不能从容器中重启宿主机，也不能加载内核模块。

相比 LXC 或 Libvirt， systemd-nspawn 更容易配置。

安装

systemd-nspawn 是 systemd 的一部分并被打进 systemd包 软件包中。

用例

创建并启动一个最小的 Arch Linux 容器

首先创建一个目录来保存容器。在这个示例中我们将使用 `~/MyContainer`.

接下来，我们使用 arch-install-scripts包 中的 ***pacstrap*** 在容器中安装一个基本的Arch系统。其中至少需要安装包组 base包。

```
# pacstrap -K -c ~/MyContainer base [additional packages/groups]
```

提示：base包 已经不再依赖内核包 linux包，这已为容器化做好准备。

注意：如果是从一个 *pacstrap* 不可用的操作系统中创建，可以使用如下方法替代容器镜像 从现有_Linux_发行版安装_Arch_Linux#方法一：使用_Bootstrap_镜像（推荐）。pacman 密钥环需要在容器中进行初始化，参阅 从现有_Linux_发行版安装_Arch_Linux#初始化_pacman_密匙环。

一旦安装完成后，进入容器中并设置root密码：

```
# systemd-nspawn -D ~/MyContainer
# passwd
# logout
```

提示：设置根密码是可选的。你可以运行 `machinectl shell root@MyContainer` 直接获取一个 root shell ，并不需要登录。

最后, 启动容器:

```
# systemd-nspawn -b -D ~/MyContainer
```

参数 `-b` 将会启动这个容器（比如：以 PID=1 运行 `systemd`）, 而不是仅仅启动一个 shell, 而参数 `-D` 指定成为容器根目录的目录。

容器启动后，输入密码以"root"身份登录。

注意：如果登录失败显示 "Login incorrect", 问题可能是 `securetty` TTY 设备白名单导致的。 请查看 #Root登录失败。

可以在容器内运行 `poweroff` 来关闭容器。在主机端，容器可以通过 machinectl 工具进行控制。

注意：要从容器内终止 *session*，请按住 `Ctrl` 并快速地按 `]` 三下。

创建 Debian 或 Ubuntu 环境

安装 debootstrap包 以及 debian-archive-keyring包 或 ubuntu-keyring包 中的一个或两个（当然要安装你想要的发行版的keyring）。

然后通过以下结构式调用 *deboostrap*：

```
# debootstrap codename container-name repository-url
```

- *codename*：对于 Debian，有效的代码名称要不然是稳定性别名如 `stable`、`testing` 或 `unstable`，要不然就是版本名称如 `bookworm` 或 `sid`。你可以从这里获取名称列表：。对于 Ubuntu：只有像是 `jammy` 或 `noble` 这样的名称应该被使用而不是版本号。你可以从 或者 获取一份代码名称与版本号的对应表格。
- *container-name* 是会包含操作系统文件树的目录，如果它不存在的话将会被创建。
- *repository-url*：下载操作系统文件树的镜像地址。目前 Debian 发布在任何有效镜像站，比如说 CDN 支持的 https://deb.debian.org/debian 。 Ubuntu 的镜像站在 ，比如说 https://archive.ubuntu.com/ubuntu 。

注意：

- 如果你需要一个以归档的 Debian 版本（指的是任何在2024年8月，Debian 10/Buster 之前的版本），请使用一个特殊的 debian-archive 镜像地址：https://archive.debian.org/debian/ 。Ubuntu 应该使用 https://old-releases.ubuntu.com/ubuntu/ 。
- *systemd-nspawn* 要求容器中的操作系统使用 *systemd* 作为其 init 程序（以 PID 1 运行）。从 Debian 8 （“jessie”） 以及 Ubuntu 15.04 （“vivid”） 之后，这是两者的默认 init 程序。但请注意，对于不包含在上述密钥包中的版本，会出现与 “未知签名密钥（unknown signing key）”相关的问题，比如任何早于 Debian 9 （“Stretch”） 的发行版本。这使得后者成为成为无需额外操作即可安装的最老版本。

与 Arch 相同，Debian 和 Ubuntu 不会让您在首次登录时无密码登录。为了设置root密码登录，要不使用"-b"参数并设置密码：

```
# cd /var/lib/machines
# systemd-nspawn -D ./container-name
# passwd
# logout
```

提示：如果计划使用 #machinectl 管理容器，请确保使用合适的包名在目标操作系统容器中安装 dbus 包，否则容器将不会工作。可以安装 `systemd-container` 时使用 `--include=` 选项，这始终是 dbus 和 systemd 的依赖项：

```
# debootstrap --include=systemd-container codename /var/lib/machines/container-name
repository-url
```

Create a Fedora or AlmaLinux environment

Install dnf包, and edit the `/etc/dnf/dnf.conf` file to add the required Fedora repositories.

```
/etc/dnf/dnf.conf
[fedora]
name=Fedora $releasever - $basearch
metalink=https://mirrors.fedoraproject.org/metalink?repo=fedora-
$releasever&arch=$basearch
gpgkey=https://getfedora.org/static/fedora.gpg

[updates]
name=Fedora $releasever - $basearch - Updates
metalink=https://mirrors.fedoraproject.org/metalink?repo=updates-released-
f$releasever&arch=$basearch
gpgkey=https://getfedora.org/static/fedora.gpg
```

The *fedora.gpg* file contain the gpg keys for the latest Fedora releases https://getfedora.org/security/. To set up a minimal Fedora 37 container:

```
# mkdir /var/lib/machines/container-name
# dnf --releasever=37 --best --setopt=install_weak_deps=False --repo=fedora --
repo=updates --installroot=/var/lib/machines/container-name install dhcp-client dnf
fedora-release glibc glibc-langpack-en iputils less ncurses passwd systemd systemd-
networkd systemd-resolved util-linux vim-default-editor
```

注意： If you want to install a different Fedora release, keep in mind that different releases will have distinct package requirements.

If you are using btrfs filesystem create a subvolume instead of creating a directory.

An Enterprise Linux derivative like AlmaLinux has three repositories enabled by default, *BaseOS* wich contains a core set that provides the basis for all installations,

AppStream that includes additional applications, language packages, etc and *Extras* that contains packages not included in RHEL. So for a minimal container we only need to add the BaseOS repository to `/etc/dnf/dnf.conf`

```
/etc/dnf/dnf.conf
[baseos]
name=AlmaLinux $releasever - BaseOS
mirrorlist=https://mirrors.almalinux.org/mirrorlist/$releasever/baseos
gpgkey=https://repo.almalinux.org/almalinux/RPM-GPG-KEY-AlmaLinux-$releasever
```

To create an AlmaLinux 9 minimal container:

```
# dnf --repo=baseos --releasever=9 --best --installroot=/var/lib/machines/container-name --setopt=install_weak_deps=False install almalinux-release dhcp-client dnf glibc-langpack-en iproute iputils less passwd systemd vim-minimal
```

This will install the latest minor version of AlmaLinux 9, you can choose to install a specific point release, but you will need to change the *gpgpkey* entry to manually point to *RPM-GPG-KEY-AlmaLinux-9*

Just like Arch, Fedora or AlmaLinux will not let you log in as root without a password. To set up the root password, run *systemd-nspawn* without the `-b` option:

```
# systemd-nspawn -D /var/lib/machines/container-name passwd
```

编译与测试包

请参阅 Creating packages for other distributions 寻找更多用例.

管理

位于 `/var/lib/machines/` 的容器可以被 *machinectl* 命令所控制，它可以内部控制 `systemd-nspawn@.service` 单元的实例。 `/var/lib/machines/` 下的子目录对应着容器的名字， 比如 `/var/lib/machines/*container-name*/`。

注意：如果因为某些原因容器不能移进 `/var/lib/machines/` ， 可以使用符号链接。 详情看 machinectl(1) § FILES AND DIRECTORIES 。

默认 systemd-nspawn 选项

要明白非常重要的一点是通过 *machinectl* 与 `systemd-nspawn@.service` 启动的容器所使用的默认选项与通过 *systemd-nspawn* 命令手动启动的有所不同。 通过服务启动所使用的额外选项有：

- `-b`/`--boot` – 管理的容器会自动搜索一个init程序，并以PID 1的形式调用它。
- `--network-veth` 关联于 `--private-network` – 管理的容器获得一个虚拟网络接口，并与主机网络断开连接。 详情看 #网络。
- `-U` – 如果内核支持，管理的容器默认使用 user_namespaces(7) 特性。 解释请看 #无特权容器。
- `--link-journal=try-guest`

这些行为可以在每个容器配置文件中被覆盖， 详情看 #配置。

machinectl

注意：*machinectl* 工具要求在容器中安装 systemd 和 dbus包 。有关详细讨论，请参阅 。

容器可以被 `machinectl *subcommand* *container-name*` 命令管理， 比如说，启动容器：

```
$ machinectl start container-name
```

注意：*machinectl* 要求 container-name 只包含 ASCII 字符，数字与连字符。这样才能保证 hostname 的合法性。比如，如果 container-name 包含了下划线，这将不会被识别并且当你运行 `machinectl start container_name` 时将会报错 `Invalid machine name container_name`。详情请看 与 。

相似的， 还有其他的子命令 `poweroff`， `reboot`， `status` 和 `show`. 详细解释请看 machinectl(1) § Machine Commands 。

提示：关机与重启操作可以在容器内通过 `poweroff` and `reboot` 命令执行。

其他常见命令：

- `machinectl list` – 显示现在正在运行的容器
- `machinectl login *container-name*` – 在一个容器中打开一个交互式登录会话
- `machinectl shell *[username@]container-name*` – 在容器中打开一个交互式shell会话（这将立即调用一个用户进程，而不需要通过容器中的登录进程）
- `machinectl enable *container-name*` 与 `machinectl disable *container-name*` – 启用或禁用容器的随开机启动，详见 #启用容器的随开机启动

machinectl 也有管理容器（或虚拟机）镜像和镜像传输的子命令。 详情见 machinectl(1) § Image Commands 与 machinectl(1) § Image Transfer Commands 。

systemd 工具链

大部分核心的 systemd 工具链已更新为对容器有效。工具们通常提供了 `-M`， `--machine=` 选项并把容器名称作为参数。 用例：

查看特定容器的 journal 日志：

```
# journalctl -M MyContainer
```

显示控制组的内容：

```
$ systemd-cgls -M MyContainer
```

查看容器的启动时间：

```
$ systemd-analyze -M MyContainer
```

查看资源使用情况的概况：

```
$ systemd-cgtop
```

配置

容器前设置

要指定容器前设置而不是全局覆盖，可以使用*.nspawn*文件。 详情见 systemd.nspawn(5) 。

注意：

- 当执行`machinectl remove` 的时候，*.nspawn* 文件可能会从 `/etc/systemd/nspawn/` 被意外移除。
- 当有`--settings=override`（在`systemd-nspawn@.service`文件内被指定）时，*.nspawn* 内与命令行中所指定的为网络选项的交互不能正常工作。 作为一个变通方法，你需要引入`VirtualEthernet=on`选项，即使服务指定了`--network-veth`。

启用容器的随开机启动

当频繁使用一个容器时，你可能希望在系统启动时启动它。

首先确保 `machines.target` 已经 enabled。

容器被 machinectl 的可发现可以 enabled 或者 disabled。

```
$ machinectl enable container-name
```

注意：

- 这样做的效果是启用 `systemd-nspawn@*container-name*.service` systemd单元。
- 如#默认 systemd-nspawn 选项中提到的，由*machinectl*启动的容器会获得一个虚拟的以太网接口。要禁用私有网络，请参见#使用主机网络。

资源控制

您可以使用控制组来使用 `systemctl set-property` 实现对容器的限制和资源管理，请参阅 systemd.resource-control(5)。例如，您可能希望限制内存量或 CPU 使用率。要将容器的内存消耗限制为 2 GiB：

```
# systemctl set-property systemd-nspawn@myContainer.service MemoryMax=2G
```

或者将 CPU 时间使用率限制为大约相当于 2 个内核：

```
# systemctl set-property systemd-nspawn@myContainer.service CPUQuota=200%
```

这将在 `/etc/systemd/system.control/systemd-nspawn@*myContainer*.service.d/` 中创建常驻文件}。

根据文档，`MemoryHigh` 是控制内存消耗的首选方法，但也不会像 `MemoryMax` 那样进行强制限制。您可以使用这两个选项将 `MemoryMax` 作为您最后的一道防线。还要

考虑到如果您不会限制容器可以“看到”的 CPU 数量，但通过限制容器相对于总 CPU 时间的最大时间，也可以获得类似的结果。

提示：如果不希望在重新启动后保留此更改，则可以传递选项 `--runtime` 使更改成为临时性的。您可以使用 `systemd-cgtop` 检查其结果。

网络

systemd-nspawn 容器可以使用*主机网络*或者*私有网络*:

- 在主机网络模式下，容器可以完全访问主机网络。这意味着容器将能够访问主机上的所有网络服务，来自容器的数据包将在外部网络中显示为来自主机（即共享同一IP地址）。
- 在私有网络模式下，容器与主机的网络断开连接，这使得容器无法使用所有网络接口，但环回设备和明确分配给容器的接口除外。为容器设置网络接口有多种不同的方法：
- 可以将现有接口分配给容器（例如，如果您有多个以太网设备）。
- 可以创建一个与现有接口（即VLAN接口）相关联的虚拟网络接口，并将其分配给容器。
- 可以创建主机和容器之间的虚拟以太网链接。

: 在后一种情况下，容器的网络是完全隔离的（与外部网络以及其他容器），由管理员来配置主机和容器之间的网络。这通常涉及创建一个网桥network bridge来连接多个（物理或虚拟）接口，或者在多个接口之间设置一个NATNetwork Address Translation。

主机网络模式适用于*应用程序容器*，它不运行任何网络软件来配置分配给容器的接口。当你从shell运行*systemd-nspawn*时，主机联网是默认模式。

另一方面，私有网络模式适用于应与主机系统隔离的 "系统容器"。创建虚拟以太网链路是一个非常灵活的工具，可以创建复杂的虚拟网络。这是由*machinectl*或`systemd-nspawn@.service`启动的容器的默认模式。

下面的小节描述了常见的情况。关于可用的*systemd-nspawn*选项，请参见systemd-nspawn(1) § Networking Options。

使用主机网络

要禁用私有网络和创建由*machinectl*启动的容器使用的虚拟以太网链接，请添加*.nspawn*文件，其中包含以下选项：

```
/etc/systemd/nspawn/container-name.nspawn
[Network]
```

`VirtualEthernet=no`
这将覆盖在`systemd-nspawn@.service`中使用的`-n/--network-veth`选项，新启动的容器将使用主机网络模式。

使用虚拟以太网链接

如果使用`-n/--network-veth`选项启动容器，*systemd-nspawn*将在主机和容器之间创建一个虚拟的以太网链接。链接的主机侧将作为名为`ve-*容器名称*`的网络接口提供。链路的容器侧将被命名为`host0`。请注意，这个选项意味着 `--private-network`。

注意：

- 如果容器名称太长，接口名称将被缩短（例如`ve-long-conKQGh`而不是`ve-long-container-name`），以适应15个字符的限制。全名将被设置为接口的`altname`属性（见ip-link(8)），并且仍然可以用来引用接口。
- 当检查带有`ip link`的接口时，接口名会显示一个后缀，例如`ve-*容器名*@if2`和`host0@if9`。`@if*N*`实际上并不是接口名称的一部分，相反，`ip link`附加了这个信息，以指示虚拟以太网电缆连接到另一端的哪个 "槽"。

: 例如，显示为`ve-*foo*@if2`的主机虚拟以太网接口与容器`*foo*`相连，在容器内部与第二个网络接口--在容器内部运行`ip link`时，显示索引为2的接口。同理，容器内名为`host0@if9`的接口连接到主机上的第9个网络接口。

当您启动容器时，必须为两个接口（主机上和容器中）分配一个 IP 地址。如果您在主机上和容器中都使用systemd-networkd，这就是开箱即用：

- 主机上的`/usr/lib/systemd/network/80-container-ve.network`文件与`ve-*container-name*`接口相匹配，并启动一个DHCP服务器，该服务器为主机接口和容器分配IP地址，
- 容器中的`/usr/lib/systemd/network/80-container-host0.network`文件与`host0`接口相匹配，并启动一个DHCP客户端，该客户端从主机接收一个IP地址。

如果不使用 systemd-networkd，可以配置静态 IP 地址或在主机接口上启动 DHCP 服务器，在容器中启动 DHCP 客户端。详情请参见网络配置。

要让容器访问外部网络，您可以按照 Internet sharing#Enable NAT 中的描述配置 NAT。如果您使用 systemd-networkd，这将通过 `/usr/lib/systemd/network/80-container-ve.network` 中的 `IPMasquerade=yes` 选项自动（部分）完成。然而，这只会发出一个iptables规则，比如说：

```
-t nat -A POSTROUTING -s 192.168.163.192/28 -j MASQUERADE
```

`filter`表必须手动配置，如Internet sharing#Enable NAT所示。您可以使用通配符来匹配所有以`ve-`开头的接口：

```
# iptables -A FORWARD -i ve-+ -o internet0 -j ACCEPT
```

注意：*systemd-networkd*使用libiptc库与iptables交互。如果您使用nftables，请安装 iptables-nft包 转译层。参见 systemd issue 13307。

并且，为了接收DHCP服务器的入站连接，你需要在 `ve-+` 接口上开放 UDP 67 端口（由 *systemd-networkd* 操控）：

```
# iptables -A INPUT -i ve-+ -p udp -m udp --dport 67 -j ACCEPT
```

使用网络桥接

如果您已在主机系统上配置了网桥network bridge，则可以为容器创建一个虚拟以太网链路，并将其主机侧添加到网桥中。这可以通过`--network-bridge=*bridge-name*`选项来完成。请注意，`--network-bridge`意味着`--network-veth`，即虚拟以太网链路是自动创建的。然而，链路的主机端将使用`vb-`前缀而不是`ve-`，因此用于启动DHCP服务器和IP伪装的systemd-networkd选项将不会被应用。

网桥的管理由管理员负责。例如，网桥可以用一个物理接口连接虚拟接口，也可以只连接几个容器的虚拟接口。参见systemd-networkd#Network bridge with DHCP和 systemd-networkd#Network bridge with static IP addresses，了解使用systemd-networkd进行配置的例子。

还有一个`--network-zone=*zone-name*`选项，它与`--network-bridge`类似，但网桥由*systemd-nspawn*和*systemd-networkd*自动管理。当第一个用`vz-*zone-name*`配置的容器启动时，会自动创建名为`--network-zone=*zone-name*`的网桥接口，当最后一个用`--network-zone=*zone-name*`配置的容器退出时，会自动删除。因此，这个选项可以方便地将多个相关的容器放置在一个共同的虚拟网络上。请注意，`vz-*`接口由systemd-networkd管理的方式与`ve-*`接口相同，使用`/usr/lib/systemd/network/80-container-vz.network`文件中的选项。

使用 "macvlan" 或者 "ipvlan" 接口

您可以在现有的物理接口（即VLAN接口）上创建一个虚拟接口，并将其添加到容器中，而不是创建一个虚拟的以太网链路（其主机端可能被添加到桥接中，也可能没有）。该虚拟接口将与底层主机接口进行桥接，从而使容器暴露在外部网络中，从而使其能够通过DHCP从与主机相连的同一局域网中获得一个独特的IP地址。

systemd-nspawn 提供两个选项:

- `--network-macvlan=*interface*` – 虚拟接口的MAC地址将与底层物理`*interface*`不同，并被命名为`mv-*interface*`。
- `--network-ipvlan=*interface*` – 虚拟接口的MAC地址将与底层物理`*interface*`相同，并命名为`iv-*interface*`。

所有选项都意味着 `--private-network`.

使用现有接口

如果主机系统有多个物理网络接口，可以使用`--network-interface=*interface*`将`*interface*`分配给容器（并使它在容器启动时对主机不可用）。请注意，`--network-interface`意味着`--private-network`。

提示：自 v256 版本起，systemd-nspawn 容器支持传递无线网络接口。

端口映射

当启用私有网络时，可以使用`-p`/`-port`选项或使用*.nspawn*文件中的`Port`设置将主机上的各个端口映射到容器上的端口。这可以通过向`nat`表发出iptables规则来完成，但需要手动配置`forward`表中的`filter`链，如#使用虚拟以太网链接所示。

例如，将主机上的TCP端口8000映射到容器中的TCP端口80：

```
/etc/systemd/nspawn/container-name.nspawn
[Network]
Port=tcp:8000:80
```

注意：

- *systemd-nspawn*在映射端口时明确地排除了`loopback`接口。因此，在上面的例子中，`localhost:8000` 连接到主机而不是容器。只有到其他接口的连接才会受到端口映射的影响。详情请参见 。
- 端口映射只对IPv4连接有效。。

域名解析

容器中的域名解析 Domain name resolution 可以以主机系统中相同的方式配置。另外，*systemd-nspawn* 提供了管理容器中 `/etc/resolv.conf` 文件的选项：

- `--resolv-conf` can be used on command-line
- `ResolvConf=` can be used in *.nspawn* files

这些相应的选项有许多可能的值，在 systemd-nspawn(1) § Integration Options 的描述中会提到。 默认值为 `auto`，这意味着：

- 如果启用了`--private-network`，`/etc/resolv.conf`就会保持容器中的原样。

- 否则，如果systemd-resolved在主机上运行，它的存根文件`resolv.conf`将被复制或绑定到容器中。
- 否则，`/etc/resolv.conf` 文件就会被从主机复制或绑定到容器上。

在后两种情况下，如果容器根部是可写的，则复制文件，如果是只读的，则绑定挂载。

对于第二种 systemd-resolved 在宿主机上运行的情况，*systemd-nspawn* 希望它也能在容器中运行，所以容器将会从宿主中 `/etc/resolv.conf` 文件建立桩符号链接。如果不是这样，那么默认值 `auto` 将不会生效，你应该使用任意 `replace-*` 选项来替换这个链接。

提示和技巧

运行 non-shell/init 命令

来自 systemd-nspawn(1) § Execution_Options:

: \[选项\] `--as-pid2` \[调用\] shell 或以进程ID (PID) 2 而不是 PID 1 (init) 来运行特定程序。 \[...\] 建议使用这种模式来调用容器中的任意命令，除非它们已经被修改为以 PID 1的形式正确运行。**或者换句话说：这个改变应该被用于几乎所有的命令**，除非该命令是指一个init或shell的实现。

无特权容器

*systemd-nspawn*支持无特权的容器，不过容器需要以root身份启动。

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Very little of Linux Containers#Enable support to run unprivileged containers (optional)) applies to systemd-nspawn.（在Talk:Systemd-nspawn讨论）

注意：此功能需要 user_namespaces(7), 更多信息请参见 Linux Containers#Enable support to run unprivileged containers (optional))。

最简单的方法是通过`-U`选项让*systemd-nspawn*自动选择一个未使用的UID/GID范围。

```
# systemd-nspawn -bUD ~/MyContainer
```

如果内核支持用户命名空间，`-U`选项相当于`--private-users=pick --private-users-ownership=auto`。详情请参见systemd-nspawn(1) § User Namespacing Options。

注意：您也可以手动指定容器的UID/GID范围，但是，这很少有用。

如果一个容器曾使用 `--private-users-ownership=chown` 选项（或者在一个 `-U` 需要 `--private-users-ownership=chown` 的文件系统中）在私有UID/GID范围启动，就需要一直这样使用它以避免权限错误。另外，也可以在一个ID范围从0开始的容器文件系统中撤销`--private-users-ownership=chown`的效果：

```
# systemd-nspawn -D ~/MyContainer --private-users=0 --private-users-ownership=chown
```

使用 X 环境

本文或本章节的事实准确性存在争议。

原因： 本节末尾有关 systemd 版本的注释似乎已经过时。对于我（译者注：原作者）来说（systemd 版本为239） 如果 `/tmp/.X11-unix` 权限是 rw，X应用程序可以工作运行。（在 Talk:Systemd-nspawn#/tmp/.X11-unix contents have to be bind-mounted as read-only - still relevant? 中讨论）

详情见 Xhost 和 Change root#Run graphical applications from chroot.

您需要设置您容器会话中 `DISPLAY` 以连接到外部 X 服务器。

X 在 `/tmp` 文件夹中存储一些必要的文件。为了使容器能显示任何内容，它需要访问这些文件。为此，当启动容器时，请追加 `--bind-ro=/tmp/.X11-unix` 选项。

注意：从 systemd 版本 235 开始, `/tmp/.X11-unix` 的内容 必须以只读方式装入，否则它们将从文件系统中消失。只读挂载标志不会阻止在套接字上使用 `connect()`。如果你还绑定了`/run/user/1000` 那么你有可能希望显式绑定 `/run/user/1000/bus` 为只读，以防止 dbus 套接字被删除。

避免 xhost

`xhost` 仅提供对 X 服务器相当粗糙的访问权限。更细节的访问可通过`$XAUTHORITY` 文件控制。遗憾的是, 仅使 `$XAUTHORITY` 文件在容器中可被访问并无法正常的执行工作: 这是因为您的 `$XAUTHORITY` 文件只特定于您的主机，但是容器是另一台主机。 根据 stackoverflow 下面这个技巧可以让你的X服务器接受来自于你容器中的X应用的 `$XAUTHORITY` 文件：

```
$ XAUTH=/tmp/container_xauth
$ xauth nextract - "$DISPLAY" | sed -e 's/^..../ffff/' | xauth -f "$XAUTH" nmerge -
# systemd-nspawn -D myContainer --bind=/tmp/.X11-unix --bind="$XAUTH" -E
DISPLAY="$DISPLAY" -E XAUTHORITY="$XAUTH" --as-pid2 /usr/bin/xeyes
```

上面第二行将连接的组设定为 "FamilyWild"，GID为`65535`，会使输入匹配你的每一个显示。 更多信息请参考 Xsecurity(7) 。

使用 X 嵌套/Xephyr

另一个运行X应用程序并避免共享X桌面风险的简单方法是使用X嵌套。 这里的优点是完全避免了容器内应用程序和非容器内应用程序之间的交互，并且能够运行不同的 desktop environment 或 window manager，缺点是使用 Xephyr 时性能较差，并且缺乏硬件加速。 在容器外启动 Xephyr：

```
# Xephyr :1 -resizeable
```

然后在使用下面的参数启动容器：

```
--setenv=DISPLAY=:1 --bind-ro=/tmp/.X11-unix/X1
```

不需要其他绑定。 在某些情况下，你可能仍然需要在容器中手动设置 `DISPLAY=:1`（主要是在与 `b` 一起使用时）。

运行 Firefox

以PID 1运行

```
 # systemd-nspawn --setenv=DISPLAY=:0 \
                --setenv=XAUTHORITY=~/.Xauthority \
                --bind-ro=$HOME/.Xauthority:/root/.Xauthority \
                --bind=/tmp/.X11-unix \
                -D ~/containers/firefox \
                --as-pid2 \
                firefox
```

注意： 因此，如果不使用 #无特权容器，firefox会以根用户的身份运行，这就带来了自身的风险。在这种情况下，你可以先选择在容器内 添加一个用户，然后在 *systemd-nspawn* 调用中增加 `--user <username>` 选项。

或者你可以启动容器，让systemd-networkd等设置虚拟网络接口：

```
# systemd-nspawn --bind-ro=$HOME/.Xauthority:/root/.Xauthority \
                --bind=/tmp/.X11-unix \
                -D ~/containers/firefox \
                --network-veth -b
```

一旦你的容器被启动，就像这样运行Xorg二进制文件：

```
# systemd-run -M firefox --setenv=DISPLAY=:0 firefox
```

3D图形加速

为了启用加速的3D图形，可能有必要在 *.nspawn* 文件中加入以下一行，将 `/dev/dri` 挂载到容器中。

```
Bind=/dev/dri
```

上述技巧来自patrickskiba.com。这将明显解决了以下问题

```
libGL error: MESA-LOADER: failed to retrieve device information
libGL error: Version 4 or later of flush extension not found
libGL error: failed to load driver: i915
```

你可以通过运行 `glxinfo` 或 `glxgears` 来确认它已经被启用。

Nvidia GPUs

如果你不能在容器上安装与主机上相同的nvidia驱动版本，你可能需要同时绑定驱动库文件。你可以在主机上运行 `pacman -Ql nvidia-utils` 来查看它包含的所有文件。你不需要把所有文件都复制过来。当容器通过 `machinectl start *container-name*` 运行时，下面的systemd覆盖文件将把所有必要的文件绑定过来。

**本文或本章节的事实准确性存在争议。**

原因： 将 `/usr/lib/` 绑定到 `/usr/lib/x86_64-linux-gnu/` 是没有必要的。（在 Talk:Systemd-nspawn 中讨论）

```
/etc/systemd/system/systemd-nspawn@.service.d/nvidia-gpu.conf
[Service]
ExecStart=
ExecStart=systemd-nspawn --quiet --keep-unit --boot --link-journal=try-guest --
machine=%i \
--bind=/dev/dri \
--bind=/dev/shm \
--bind=/dev/nvidia0 \
--bind=/dev/nvidiactl \
--bind=/dev/nvidia-modeset \
--bind=/usr/bin/nvidia-bug-report.sh:/usr/bin/nvidia-bug-report.sh \
--bind=/usr/bin/nvidia-cuda-mps-control:/usr/bin/nvidia-cuda-mps-control \
--bind=/usr/bin/nvidia-cuda-mps-server:/usr/bin/nvidia-cuda-mps-server \
--bind=/usr/bin/nvidia-debugdump:/usr/bin/nvidia-debugdump \
--bind=/usr/bin/nvidia-modprobe:/usr/bin/nvidia-modprobe \
--bind=/usr/bin/nvidia-ngx-updater:/usr/bin/nvidia-ngx-updater \
--bind=/usr/bin/nvidia-persistenced:/usr/bin/nvidia-persistenced \
--bind=/usr/bin/nvidia-powerd:/usr/bin/nvidia-powerd \
--bind=/usr/bin/nvidia-sleep.sh:/usr/bin/nvidia-sleep.sh \
--bind=/usr/bin/nvidia-smi:/usr/bin/nvidia-smi \
--bind=/usr/bin/nvidia-xconfig:/usr/bin/nvidia-xconfig \
--bind=/usr/lib/gbm/nvidia-drm_gbm.so:/usr/lib/x86_64-linux-gnu/gbm/nvidia-drm_gbm.so \
--bind=/usr/lib/libEGL_nvidia.so:/usr/lib/x86_64-linux-gnu/libEGL_nvidia.so \
--bind=/usr/lib/libGLESv1_CM_nvidia.so:/usr/lib/x86_64-linux-gnu/libGLESv1_CM_nvidia.so
\
--bind=/usr/lib/libGLESv2_nvidia.so:/usr/lib/x86_64-linux-gnu/libGLESv2_nvidia.so \
--bind=/usr/lib/libGLX_nvidia.so:/usr/lib/x86_64-linux-gnu/libGLX_nvidia.so \
--bind=/usr/lib/libcuda.so:/usr/lib/x86_64-linux-gnu/libcuda.so \
--bind=/usr/lib/libnvcuvid.so:/usr/lib/x86_64-linux-gnu/libnvcuvid.so \
--bind=/usr/lib/libnvidia-allocator.so:/usr/lib/x86_64-linux-gnu/libnvidia-allocator.so
\
--bind=/usr/lib/libnvidia-cfg.so:/usr/lib/x86_64-linux-gnu/libnvidia-cfg.so \
--bind=/usr/lib/libnvidia-egl-gbm.so:/usr/lib/x86_64-linux-gnu/libnvidia-egl-gbm.so \
--bind=/usr/lib/libnvidia-eglcore.so:/usr/lib/x86_64-linux-gnu/libnvidia-eglcore.so \
--bind=/usr/lib/libnvidia-encode.so:/usr/lib/x86_64-linux-gnu/libnvidia-encode.so \
--bind=/usr/lib/libnvidia-fbc.so:/usr/lib/x86_64-linux-gnu/libnvidia-fbc.so \
--bind=/usr/lib/libnvidia-glcore.so:/usr/lib/x86_64-linux-gnu/libnvidia-glcore.so \
--bind=/usr/lib/libnvidia-glsi.so:/usr/lib/x86_64-linux-gnu/libnvidia-glsi.so \
--bind=/usr/lib/libnvidia-glvkspirv.so:/usr/lib/x86_64-linux-gnu/libnvidia-glvkspirv.so
\
--bind=/usr/lib/libnvidia-ml.so:/usr/lib/x86_64-linux-gnu/libnvidia-ml.so \
--bind=/usr/lib/libnvidia-ngx.so:/usr/lib/x86_64-linux-gnu/libnvidia-ngx.so \
--bind=/usr/lib/libnvidia-opticalflow.so:/usr/lib/x86_64-linux-gnu/libnvidia-
opticalflow.so \
--bind=/usr/lib/libnvidia-ptxjitcompiler.so:/usr/lib/x86_64-linux-gnu/libnvidia-
ptxjitcompiler.so \
--bind=/usr/lib/libnvidia-rtcore.so:/usr/lib/x86_64-linux-gnu/libnvidia-rtcore.so \
--bind=/usr/lib/libnvidia-tls.so:/usr/lib/x86_64-linux-gnu/libnvidia-tls.so \
--bind=/usr/lib/libnvidia-vulkan-producer.so:/usr/lib/x86_64-linux-gnu/libnvidia-
vulkan-producer.so \
--bind=/usr/lib/libnvoptix.so:/usr/lib/x86_64-linux-gnu/libnvoptix.so \
--bind=/usr/lib/modprobe.d/nvidia-utils.conf:/usr/lib/x86_64-linux-
gnu/modprobe.d/nvidia-utils.conf \
--bind=/usr/lib/nvidia/wine/_nvngx.dll:/usr/lib/x86_64-linux-gnu/nvidia/wine/_nvngx.dll
\
--bind=/usr/lib/nvidia/wine/nvngx.dll:/usr/lib/x86_64-linux-gnu/nvidia/wine/nvngx.dll \
--bind=/usr/lib/nvidia/xorg/libglxserver_nvidia.so:/usr/lib/x86_64-linux-
gnu/nvidia/xorg/libglxserver_nvidia.so \
--bind=/usr/lib/vdpau/libvdpau_nvidia.so:/usr/lib/x86_64-linux-
gnu/vdpau/libvdpau_nvidia.so \
--bind=/usr/lib/xorg/modules/drivers/nvidia_drv.so:/usr/lib/x86_64-linux-
gnu/xorg/modules/drivers/nvidia_drv.so \
--bind=/usr/share/X11/xorg.conf.d/10-nvidia-drm-
outputclass.conf:/usr/share/X11/xorg.conf.d/10-nvidia-drm-outputclass.conf \
--bind=/usr/share/dbus-1/system.d/nvidia-dbus.conf:/usr/share/dbus-1/system.d/nvidia-
dbus.conf \
```

```
--
bind=/usr/share/egl/egl_external_platform.d/15_nvidia_gbm.json:/usr/share/egl/egl_exter
nal_platform.d/15_nvidia_gbm.json \
--
bind=/usr/share/glvnd/egl_vendor.d/10_nvidia.json:/usr/share/glvnd/egl_vendor.d/10_nvid
ia.json \
--bind=/usr/share/licenses/nvidia-utils/LICENSE:/usr/share/licenses/nvidia-
utils/LICENSE \
--bind=/usr/share/vulkan/icd.d/nvidia_icd.json:/usr/share/vulkan/icd.d/nvidia_icd.json
\
--
bind=/usr/share/vulkan/implicit_layer.d/nvidia_layers.json:/usr/share/vulkan/implicit_l
ayer.d/nvidia_layers.json \
DeviceAllow=/dev/dri rw
DeviceAllow=/dev/shm rw
DeviceAllow=/dev/nvidia0 rw
DeviceAllow=/dev/nvidiactl rw
DeviceAllow=/dev/nvidia-modeset rw
```

注意：每当你在主机上升级nvidia驱动时，你将需要重启容器，并可能需要在容器中运行 `ldconfig` 以更新库。

访问主机文件系统

请见 `--bind` 和 `--bind-ro` 于 systemd-nspawn(1).

如果主机和容器都是 Arch Linux，则例如，可以共享 pacman 缓存：

```
# systemd-nspawn --bind=/var/cache/pacman/pkg
```

或许你还可以使用文件来进行指定的先于容器的绑定：

```
/etc/systemd/nspawn/my-container.nspawn
[Files]
Bind=/var/cache/pacman/pkg
```

详情见 #容器前设置.

要将该目录绑定到容器内的不同路径，请添加路径，并用冒号分隔。例如：

```
# systemd-nspawn --bind=/path/to/host_dir:/path/to/container_dir
```

如果是 #无特权容器，产生的挂载点将由nobody用户拥有。这可以通过 `idmap` 挂载选项来修改。

```
# systemd-nspawn --bind=/path/to/host_dir:/path/to/container_dir:idmap
```

在非systemd系统上运行

详情见 Init#systemd-nspawn。

使用Btrfs子卷作为容器的根

要使用 Btrfs subvolume 作为容器根目录的模板，请使用 `--template` 标志。这将获取子卷的快照，并以它填充容器的根目录。

注意：如果指定的模板路径不是子卷的根，则会复制整个树。这将非常耗时。

例如，要使用位于`/.snapshots/403/snapshot`的快照：

```
# systemd-nspawn --template=/.snapshots/403/snapshots -b -D my-container
```

其中*my-container*是将为容器创建的目录的名称。关机后，会保留新创建的子卷。

使用容器的临时Btrfs快照

可以使用--ephemeral或-x标志来创建容器的临时btrfs快照，并将其作为容器的根。在容器中启动时所作的任何更改都将丢失。例如：

```
# systemd-nspawn -D my-container -xb
```

其中*my-container*是**现有**容器或系统的目录。例如，如果/是一个btrfs子卷，我们可以通过以下操作创建一个当前运行的主机系统的短暂容器：

```
# systemd-nspawn -D / -xb
```

关闭容器电源后，创建的btrfs子卷会立即被删除。

在 systemd-nspawn 中运行 docker

自 Docker 20.10以来，在启用 *cgroups v2* （Arch Linux默认）的情况下，可以在无特权的 *systemd-nspawn* 容器内运行Docker容器，而不会因为禁用cgroups和用户名字空间而破坏安全措施。要做到这一点，请编辑或创建 /etc/systemd/nspawn/myContainer.nspawn，并添加以下配置。

```
/etc/systemd/nspawn/myContainer.nspawn
[Exec]
SystemCallFilter=add_key keyctl bpf
```

然后，Docker 应该在容器内按原样工作。

注意：上述配置将系统调用 *add_key* 、*keyctl* 和 *bpf* 暴露给容器，而这些调用是没有命名间隔的。这仍然可能是一个安全风险，尽管它比完全禁用用户命名空间要低得多，就像人们在 cgroups v2 之前必须做的那样。

由于 *overlayfs* 不能用于用户命名空间，并且在 *systemd-nspawn* 中不可用，默认情况下，Docker 会退回到使用低效的 *vfs* 作为其存储驱动，每次启动容器都会创建一个镜像的副本。这一点可以通过使用 *fuse-overlayfs* 作为其存储驱动来解决。要做到这一点，我们需要首先将 *fuse* 暴露给容器。

```
/etc/systemd/nspawn/myContainer.nspawn
[Files]
Bind=/dev/fuse
```

然后允许容器读取和写入设备节点。

```
# systemctl set-property systemd-nspawn@myContainer DeviceAllow='/dev/fuse rwm'
```

最后，在容器中安装 fuse-overlayfs包 包。你需要重新启动容器以使所有配置生效。

映射本地用户并将其主目录绑定挂载到容器中

按照#创建并启动一个最小的 Arch Linux 容器中的说明，在 /var/lib/machines/MyContainer/ 中创建一个容器。

创建一个包含 `BindUser=` 的配置文件，以将选定的本地用户名映射到容器中。请注意，这需要 `PrivateUsers=`，详见 systemd-nspawn(1) 手册页。在容器内外绑定挂载的主目录中创建的文件将具有相同的 UID 和 GID。

```
/etc/systemd/nspawn/MyContainer.nspawn
[Exec]
User=username
PrivateUsers=pick

[Files]
BindUser=username
```

在配置中同时使用 `User=` 指定了用于在容器内运行命令（例如交互式 shell）的默认用户：

```
# systemd-nspawn -M MyContainer bash
```

这对于从另一个 Linux 发行版测试 Arch Linux 软件包很有帮助。

疑难解答

Root登录失败

如果您在尝试登录时（即使用`machinectl login <name>`）得到以下错误:

```
arch-nspawn login: root
Login incorrect
```

并且容器中`journal`显示:

```
pam_securetty(login:auth): access denied: tty 'pts/0' is not secure !
```

**本文或本章节的事实准确性存在争议。**

原因： 用户不应编辑 `/usr/lib` 中的文件，`/usr/lib/tmpfiles.d/arch.conf` 中的改动会在 filesystem包 升级后丢失。（在 Talk:Systemd-nspawn 中讨论）

只删除容器文件系统中的`/etc/securetty` 和`/usr/share/factory/etc/securetty`，或者也可以根据需要只简单地向*容器*文件系统中 `/etc/securetty` 添加所需的pty终端设备（如 `pts/0`）。

但是任何改变都会在下一次启动时被覆盖，因此有必要同时在*容器*文件系统上的 `/usr/lib/tmpfiles.d/arch.conf` 文件中删除 `/etc/securetty` 条目，见 FS#63236。如果你选择删除，也可以在 `/etc/pacman.conf` 中选择将这些文件列入黑名单（NoExtract），以防止它们被重新安装。详见 FS#45903。

execv(...) failed: Permission denied

当试图通过`systemd-nspawn -bD */path/to/container*`来启动容器时（或在容器中执行一些东西），出现以下错误：

```
execv(/usr/lib/systemd/systemd, /lib/systemd/systemd, /sbin/init) failed: Permission denied
```

即使有关文件的权限（即 `/lib/systemd/systemd`）是正确的，这也可能是由于以非root用户的身份挂载容器所在的文件系统造成的。例如，如果您在fstab中手动挂载了具有`noauto,user,...`选项的磁盘，*systemd-nspawn*将不允许执行文件，即使它们是由root用户拥有的。

TERM中的终端类型不正确（破损的颜色）

当通过`machinectl login`登录容器时，容器内的终端中的颜色和按键可能会被破坏。这可能是由于`TERM`环境变量中的终端类型不正确。除非明确配置，否则环境变量不会从主机上的 shell 继承，而是回到 systemd 中固定的默认值 (`vt220`)。要配置，在容器内为 `container-getty@.service` systemd 服务创建一个配置覆盖，启动 `machinectl login` 的登录 getty，并将 `TERM` 设置为与您登录的主机终端匹配的值：

```
/etc/systemd/system/container-getty@.service.d/term.conf
[Service]
Environment=TERM=xterm-256color
```

或者使用`machinectl shell`。它正确地继承了终端的`TERM`环境变量。

在容器内挂载NFS共享

目前（2019年6月）还不可能

曲线救国的方式是使用主机挂载NFS后通过目录映射到容器目录，NFS挂载方式请查看 NFS#Client。挂载后会遇到权限错误，这是因为容器的用户ID是通过主机映射过去的，详情请查看：， 解决这个问题的方法有很多种，如下面几种：

1. 在主机上使用管理员权限将NFS挂载目录所属用户ID修改为容器所对应的实际用户ID。
2. 在容器启动时使用`--private-users`参数指定使用用户作为特权用户启动。
3. 在挂载目录时使用`--bind-user`参数指定NFS目录所属用户映射到容器中。

See also

- Automatic console login
- Creating containers with systemd-nspawn
- Presentation by Lennart Poettering on systemd-nspawn
- Running Firefox in a systemd-nspawn container
- Graphical applications in systemd-nspawn

加密

单机上的数据保护无非两件事：让内容看不懂，或者确认内容没被动过。加密卷、密钥备份、散列校验，以及网络查询这一侧，这一章各有现成工具。

前三节建议按顺序读完：Vault 篇幅最短，讲安装与服务，可先建立手感；Tomb 是日常主力，安装、使用与高级功能都要照着敲；Paperkey 处理密钥丢失后的恢复，技巧别跳过。SHA 散列可随时回查；Dnscrypt-proxy 篇幅最长，讲网络查询而非磁盘文件，时间紧可略过。

最容易踩的坑是把散列当成加密：SHA 不可逆，保证的是完整性，藏不住数据。

4.1　Vault

难度 入门 | **篇幅** 1.5 KB | **所属** 加密

本节将讲到

- 安装
- 服务

安装

安装 vault包 包。

服务

Vault 由 systemd 单元 `vault.service` 控制。启动并启用这个单元。

配置

编辑 `/etc/vault.hcl` 文件。

在开发人员模式下运行服务器

遵循 https://www.vaultproject.io/docs/concepts/dev-server.html 。 注意 API 的更改：https://stackoverflow.com/questions/49872480/vault-error-while-writing 。

4.2　Tomb

难度 入门 | **篇幅** 3.2 KB | **所属** 加密

本节将讲到

- 安装
- 使用
- 高级功能
- 参见

根据官方网站:

: Tomb 是 100% 免费和开源的软件，使强大的加密变得易于使用。 : 一个 Tomb 就像一个锁定的文件夹，可以安全地传输和隐藏在文件系统中。 : 密钥可以分开保存：例如，在您计算机上储存 Tomb 和在 U 盘上储存密钥。

Tomb 旨在成为一个非常简单易用的软件来管理称为 tombs 的“加密目录”。只有当您拥有密钥文件且知道密码时，才能打开 Tomb。它还具有高级功能，例如隐写术。

安装

安装 tombAUR 或 tomb-gitAUR。

使用

Tomb 旨在作为单个非交互式脚本从控制台使用。它还提供**tomb-open**，这是一个简单的交互式脚本，可帮助您创建 tomb、打开 tomb、从 USB 检索密钥。

Tomb 是从终端命令行操作的，并且需要 root 访问权限（或者只是对脚本的 sudo 访问权限）。

要创建一个名为“secret”的 100MB tomb，请执行以下操作：

```
# tomb dig -s 100 secret.tomb
# tomb forge secret.tomb.key
# tomb lock secret.tomb -k secret.tomb.key
```

要打开它，请执行以下操作：

```
# tomb open secret.tomb -k secret.tomb.key
```

完成后：

```
# tomb close
```

有关更多信息，请参阅 `tomb -h` 和 `man tomb`。

高级功能

- 隐写术（将密钥隐藏在 jpeg/wav 文件中）
- 绑定挂钩：您可以把它的一些子目录作为 "绑定" 挂到其他目录上。例如，假设你想对你的.Mail、.firefox 和 Documents 目录进行加密。那么，您可以创建一个包含这些子目录的 tomb（如果你愿意，也可以创建其他子目录），并在 tomb 中创建一个简单的配置文件；当您运行`tomb open`时，它会自动将这些目录绑定到正确的位置。这样，您就可以很容易地得到一个加密的 firefox 配置文件，或 maildir。
- 后挂钩：当 tomb 打开或关闭时运行的命令。您可以为此实现很多事情：打开 tomb 内的文件，将您的计算机置于“偏执”状态（例如，禁用交换），等等。

参见

- 手册页
- 首页
- 快速开始
- 高级功能

4.3 Paperkey

难度 入门 | **篇幅** 4.3 KB | **所属** 加密

本节将讲到

- 安装
- 用法
- 提示和技巧

Paperkey 是一个在纸上导出 GnuPG 密钥的命令行工具。它通过从私钥中删除公钥部分来减小导出密钥的大小。Paperkey 还在密钥中包含 CRC-24 校验和，以允许用户检查其私钥是否已正确恢复。

安装

安装软件包 paperkey包。

用法

备份

警告：恢复纸质密钥备份时，您需要有可用的公钥！由于使公钥公开是安全的，故请考虑将其上传到公钥服务器。

要创建 GnuPG 密钥的备份，请将私钥通过管道符传输到 paperkey：

```
$ gpg --export-secret-key key-id | paperkey --output secret-key-paper.asc
```

恢复私钥

要恢复密钥，您需要有一个包含 paperkey 数据和公钥的文件。然后运行以下命令将私钥导入 `~/.gnupg`：

```
$ paperkey --pubring public-key.gpg --secrets secret-key-paper.asc | gpg --import
```

或者，将私钥恢复到一个文件：

```
$ paperkey --pubring public-key.gpg --secrets secret-key-paper.asc --output secret-key.gpg
```

Error: unable to parse OpenPGP packets (is this armored data?)

如果在恢复密钥时收到此错误，则需要先解除（dearmor）公钥：

```
$ gpg --dearmor public-key.asc
```

提示和技巧

直接打印私钥

如果没有说明 `--output` 参数，paperkey 会将其输出打印到 `stdout`。可以直接打印密钥而无需中间文件，这可能会产生安全隐患。为此，请安装 CUPS，并通过管道符传输到 `lpr`：

```
$ gpg --export-secret-key key-id | paperkey | lpr
```

将私钥转换成二维码

默认情况下，paperkey 会将密钥输出为可读的文本格式。虽然这种格式保证了有能力读取和恢复已打印出的信息，但它不是很方便。`--output-type raw` 选项告诉 paperkey 要输出原始（raw）密钥数据。这允许了其他编码方式的使用，包括计算机可读的编码，例如二维码。

qrencode包 程序可以这样使用：

```
$ gpg --export-secret-key key-id | paperkey --output-type raw | qrencode --8bit --output secret-key.qr.png
```

可以使用 `--level H` 选项将纠错级别提升到最大值。这可以恢复约 30% 的丢失数据，但代价是减少了二维码的容量。如果密钥不适合（高纠错级别的）二维码，也可以使用较低的 `Q` 和 `M` 纠错级别，恢复率分别约为 25% 和 15%。默认纠错级别为 `L`，允许恢复约 7% 的丢失数据。

从二维码恢复私钥

使用 zbar包，可以使用相机恢复密钥：

```
$ zbarcam -1 --raw -Sbinary | paperkey --pubring public-key.gpg | gpg --import
```

相同的选项也适用于 `zbarimg`：

```
$ zbarimg -1 --raw -q -Sbinary secret-key.qr.png | paperkey --pubring public-key.gpg | gpg --import
```

提示：使用 `-q` 选项运行 zbarimg 会在打印解码数据后禁止打印其状态文本。不使用该选项，传递到 paperkey 中的数据可能会被污染。

如果您使用的是扫描的图像，则可能必须通过此命令对其进行模糊处理。

```
$ convert secret-key.qr.png -blur 0 secret-key-blurred.qr.png
```

4.4 SHA 散列算法

难度 入门 | **篇幅** 4.9 KB | **所属** 加密

本节将讲到

- SHA 密码散列

翻译状态：

- 本文（或部分内容）译自 SHA hashes，最近一次同步于 2026-10-29，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

安全散列算法(Secure Hash Algorithms, SHA)是一组散列函数（台湾地区又称杂凑函数）的集合，常用于各种用途。

以下是 Arch Linux 中的一些使用示例：

- 验证 Arch Linux ISO 和软件包的完整性。
- 使用 Git 固定软件版本。
- 加密密钥的安全保护和验证，例如磁盘加密。
- 密码验证，详见安全#密码的散列值以及本文中的相关 SHA 散列的介绍。

SHA 密码散列

自 shadow包 软件包的 `4.1.4.3-3` 版本（参见 bug 13591）到 `4.14.0` 版本（参见这则公告）内，Arch Linux 默认将 SHA-512 作为密码散列函数。它取代了较为老旧的、因存在碰撞攻击漏洞而已经被攻破的 MD5 散列函数。有关更多信息，参见 zhwp:SHA家族#SHA函数对比。

配置

可以使用 `rounds=N` 选项来配置密码散列函数，这可以提升 {{zhwp:密钥延伸|密钥延伸(Key strengthening)}} 。例如 `rounds=65536` 意味着对于每一个要测试的密码，攻击者均需要重复计算 65536 次散列值才能与 `/etc/shadow` 中的密码散列值进行比对。

因此，攻击者的破解速度将会延缓 65536 倍。这也意味着登录时也需要计算 65536 次散列值，但是即便在慢速计算机上，这个流程也不会超过一秒。

从 pam包 1.6.0 版本开始，可以通过以下两种方式配置 SHA-512 的 `rounds` 选项：编辑 `/etc/login.defs` 文件，为 `SHA_CRYPT_MAX_ROUNDS` 参数设置一个值；或者编辑 `/etc/pam.d/passwd` 并添加合适的值的 `rounds` 选项。

如果不使用 `rounds` 选项，那么 PAM 将使用一个与 `/etc/login.defs` 中注释掉的 `5000` 轮示例不同的默认值，因为与 PAM 走独立验证栈和独立配置文件 `/etc/pam.d` 不同的是，shadow包 源码中硬编码了默认值。

例如：

```
/etc/pam.d/passwd
password      required      pam_unix.so sha512 shadow nullok rounds=65536
```

警告：如果在上述配置示例中显式指定了 `sha512` 等散列算法，它将会覆盖后续运行 *passwd* 命令时默认使用的 `yescrypt` 散列算法。

如需了解 `/etc/pam.d/passwd` 密码选项的详细说明，参阅 pam_unix(8)。

使用方法

即便更新了加密设置，在此之前设置的密码也并不会自动重新计算散列值。若需解决这个问题，需要重置所有用户的密码，以便于使用新的算法重新计算散列值。

以 `root` 身份执行以下命令

```
# passwd username
```

其中 `*username*` 是需要修改密码的用户名，随后重新输入他们当前的密码，系统就会使用新算法对重新计算散列值。

若需验证密码是否已经重新计算散列值，以 `root` 身份检查 `/etc/shadow` 文件。

- 使用 SHA-256 的密码散列值以 `$5` 开头。
- 使用 SHA-512 的密码散列值以 `$6` 开头。
- 使用 yescrypt 的密码散列值以 `$y` 开头。

4.5 Dnscrypt-proxy

难度 进阶 | **篇幅** 11.1 KB | **所属** 加密

本节将讲到

- 安装
- 配置
- 技巧

dnscrypt-proxy 可以加密和认证用户和 DNS 解析服务器之间的数据传输，支持 DNS over HTTPS 和 DNSCrypt，可以避免中间人攻击和窃听。*dnscrypt-proxy* 兼容 DNSSEC。

安装

安装软件包 dnscrypt-proxy包。

配置

配置文件位于 `/etc/dnscrypt-proxy/dnscrypt-proxy.toml`。

启动

服务有两种启动方式，但是只能二选一：

- 启用 `.service`

注意：必须指定 `listen_addresses` (即在配置文件中写好 `listen_addresses = ['127.0.0.1:53', '[::1]:53']`)

- 启用 `.socket`

注意：必须留空 `listen_addresses` (即 `listen_addresses = [ ]`)，systemd 会自己配置好 socket

选择解析服务器

如果留空配置文件 `/etc/dnscrypt-proxy/dnscrypt-proxy.toml` 内的 `server_names`, ***dnscrypt-proxy*** 将会自动从 `[sources]` 下选择最快的服务器 。服务器列表会自动下载、更新和验证 ，因此无须指定服务器即可使用。

如果要指定 DNS 服务器，编辑 `/etc/dnscrypt-proxy/dnscrypt-proxy.toml` 然后取消注释里面的 `server_names`，填写一个服务器名。使用 Cloudflare 服务器的配置如下：

```
server_names = ['cloudflare', 'cloudflare-ipv6']
```

upstream page 和 Github 有所有 DNS 服务器的列表。如果 *dnscrypt-proxy* 之前成功运行过，`/var/cache/dnscrypt-proxy/public-resolvers.md` 文件里也会有一个服务器列表。最好找支持 DNSSEC、无记录且无审查的服务器。使用 `require_dnssec`, `require_nolog`, `require_nofilter` 选项可以强制使用 DNSSEC、禁止记录及禁止审查。

禁用其他占用53端口的服务

提示：如果使用 #Unbound 提供本地 DNS 缓存，请跳过这一节，因为 *unbound* 默认监听 53。

查看占用 53 端口的服务：

```
 $ ss -lp 'sport = :domain'
```

如果输出除了表头还有其他内容，你就需要禁用占用53端口的那些服务，比如 `systemd-resolved.service` 和其他网络管理器包含的类似服务。把它们全部干掉，直到上面的命令的输出只剩这个样子：

```
 Netid               State                Recv-Q                Send-Q
Local Address:Port                               Peer Address:Port
```

修改 resolv.conf

修改 resolv.conf，把现有的`nameserver`全部清掉，添一行指向 *localhost* 的地址（一般是 127.0.0.1）的`nameserver`；并增添一行 *options* 。示例如下：

```
nameserver ::1
nameserver 127.0.0.1
options edns0 single-request-reopen
```

其他程序可能会修改 resolv.conf，详见：resolv.conf#Overwriting of /etc/resolv.conf。

启动 systemd 服务

根据上面你选择的配置来 start/enable `dnscrypt-proxy.service` 或者 `dnscrypt-proxy.socket`

技巧

本地 DNS 缓存配置

提示：*dnscrypt* can cache entries without relying on another program. This feature is enabled by default with the line `cache = true` in your dnscrypt configuration file

It is recommended to run DNSCrypt as a forwarder for a local DNS cache if not using *dnscrypt's* cache feature; otherwise, every single query will make a round-trip to the upstream resolver. Any local DNS caching program should work. In addition to setting up *dnscrypt-proxy*, you must setup your local DNS cache program.

修改端口

In order to forward queries from a local DNS cache, *dnscrypt-proxy* should listen on a port different from the default `53`, since the DNS cache itself needs to listen on `53` and query *dnscrypt-proxy* on a different port. Port number `53000` is used as an example in this section. In this example, the port number is larger than 1024 so *dnscrypt-proxy* is not required to be run by root.

There are two methods for changing the default port:

Socket method

Edit `dnscrypt-proxy.socket` with the following contents:

```
[Socket]
ListenStream=
ListenDatagram=
ListenStream=127.0.0.1:53000
ListenDatagram=127.0.0.1:53000
```

When queries are forwarded from the local DNS cache to `53000`, `dnscrypt-proxy.socket` will start `dnscrypt-proxy.service`.

Service method

Edit the `listen_addresses` option in `/etc/dnscrypt-proxy/dnscrypt-proxy.toml` with the following:

```
listen_addresses = ['127.0.0.1:53000', '[::1]:53000']
```

本地 DNS 缓存配置举例

The following configurations should work with *dnscrypt-proxy* and assume that it is listening on port `53000`.

Unbound

Configure Unbound to your liking (in particular, see Unbound#Local DNS server) and add the following lines to the end of the `server` section in `/etc/unbound/unbound.conf`:

```
  do-not-query-localhost: no
forward-zone:
  name: "."
  forward-addr: 127.0.0.1@53000
```

提示：If you are setting up a server, add `interface: 0.0.0.0@53` and `access-control: *your-network*/*subnet-mask* allow` inside the `server:` section so that the other computers can connect to the server. A client must be configured with `nameserver *address-of-your-server*` in `/etc/resolv.conf`.

Restart `unbound.service` to apply the changes.

dnsmasq

Configure dnsmasq as a local DNS cache. The basic configuration to work with DNSCrypt:

```
/etc/dnsmasq.conf
no-resolv
server=127.0.0.1#53000
listen-address=127.0.0.1
```

If you configured DNSCrypt to use a resolver with enabled DNSSEC validation, make sure to enable it also in dnsmasq:

```
/etc/dnsmasq.conf
proxy-dnssec
```

Restart `dnsmasq.service` to apply the changes.

pdnsd

Install pdnsd. A basic configuration to work with DNSCrypt is:

```
/etc/pdnsd.conf
global {
    perm_cache = 1024;
    cache_dir = "/var/cache/pdnsd";
    run_as = "pdnsd";
    server_ip = 127.0.0.1;
    status_ctl = on;
    query_method = udp_tcp;
    min_ttl = 15m;       # Retain cached entries at least 15 minutes.
    max_ttl = 1w;        # One week.
    timeout = 10;        # Global timeout option (10 seconds).
    neg_domain_pol = on;
    udpbufsize = 1024;   # Upper limit on the size of UDP messages.
}

server {
    label = "dnscrypt-proxy";
    ip = 127.0.0.1;
    port = 53000;
    timeout = 4;
    proxy_only = on;
}

source {
    owner = localhost;
    file = "/etc/hosts";
}
```

Restart `pdnsd.service` to apply the changes.

沙盒隔离

Edit `dnscrypt-proxy.service` to include the following lines:

```
[Service]
CapabilityBoundingSet=CAP_IPC_LOCK CAP_SETGID CAP_SETUID CAP_NET_BIND_SERVICE
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
PrivateTmp=true
PrivateDevices=true
MemoryDenyWriteExecute=true
NoNewPrivileges=true
RestrictRealtime=true
RestrictAddressFamilies=AF_INET AF_INET6
SystemCallArchitectures=native
SystemCallFilter=~@clock @cpu-emulation @debug @keyring @ipc @module @mount @obsolete
@raw-io
```

See systemd.exec(5) and Systemd#Sandboxing application environments for more information. Additionally see upstream comments.

启用 EDNS0

Extension Mechanisms for DNS that, among other things, allows a client to specify how large a reply over UDP can be.

Add the following line to your `/etc/resolv.conf`:

```
options edns0
```

You may also wish to append the following to `/etc/dnscrypt-proxy.conf`:

```
EDNSPayloadSize <bytes>
```

Where *<bytes>* is a number, the default size being **1252**, with values up to **4096** bytes being purportedly safe. A value below or equal to **512** bytes will disable this mechanism, unless a client sends a packet with an OPT section providing a payload size.

测试 EDNS0

Make use of the DNS Reply Size Test Server, use the *drill* command line tool to issue a TXT query for the name *rs.dns-oarc.net*:

```
$ drill rs.dns-oarc.net TXT
```

With **EDNS0** supported, the "answer section" of the output should look similar to this:

```
rst.x3827.rs.dns-oarc.net.
rst.x4049.x3827.rs.dns-oarc.net.
rst.x4055.x4049.x3827.rs.dns-oarc.net.
"2a00:d880:3:1::a6c1:2e89 DNS reply size limit is at least 4055 bytes"
"2a00:d880:3:1::a6c1:2e89 sent EDNS buffer size 4096"
```

TLS

TLS 在 Arch 上不是一个库，而是几个并存的实现，本章要解决的是分清各自负责什么、该装哪一个，以及证书与配置放哪里。《OpenSSL》一节的体量超过其余三节之和，安装、配置、用法与排错俱全；《Network Security Services》次之；《GnuTLS》与《Mbed TLS》各只有安装与基本用法。

建议先读《OpenSSL》，四节里只有它需要单独配置，把配置位置和排错手法拿下，必读；《Network Security Services》次之，需要时再精读；《GnuTLS》《Mbed TLS》查到即可，手上没有对应软件栈就跳过。最容易踩的坑是把这四者当成可以互相顶替的替代品，以为装哪个都行：它们并存于同一台机器，装上一个并不等于要用它的软件就用上了它，也不能把《OpenSSL》的配置与排错当成另外三节也有的内容。

5.1　Mbed TLS

难度 入门 | **篇幅** 1.8 KB | **所属** TLS

本节将讲到

- 安装
- 用法
- 参见

根据 Wikipedia:

: **mbed TLS**（以前的 **PolarSSL**）是 TLS 和 SSL 协议以及相应的加密算法和所需支持代码的实现。它具有 Apache License 版本 2.0 和 GPLv2 的双重许可。网站上的声明是 mbed TLS 旨在“易于理解，使用，集成和扩展”。

安装

安装 mbedtls包 软件包。

用法

命令名称以“mbedtls_”开头，关于用法示例，请参见知识库。

生成 RSA 私钥

```
$ mbedtls_gen_key rsa_keysize=keysize filename=filename
```

生成证书签名请求

```
$ mbedtls_cert_req filename=private_key subject_name=subject output_file=filename
```

相关方法

生成自签名证书

```
$ mbedtls_cert_write selfsign=1 issuer_key=private_key issuer_name=subject
not_before=YYYYMMDDHHMMSS not_after=YYYYMMDDHHMMSS is_ca=1 max_pathlen=0
output_file=file
```

相关方法

参见

- 官方网站
- API 文档

5.2 GnuTLS

难度 入门 | **篇幅** 2.0 KB | **所属** TLS

本节将讲到

- 安装
- 用法
- 另请参见

根据 Wikipedia:

: **GnuTLS** (the **GNU Transport Layer Security Library**) is a free software implementation of the TLS, SSL and DTLS protocols. It offers an application programming interface (API) for applications to enable secure communication over the network transport layer, as well as interfaces to access X.509, PKCS #12, OpenPGP and other structures.

安装

安装 gnutls包 包。

要与 Apache HTTP Server 集成，请安装 mod_gnutls。

用法

有关以下各节中使用的命令，请参见 certtool(1)；有关 API 文档，请参见信息文档。

生成 RSA 私钥

```
$ certtool -p --rsa --bits=keysize
```

生成证书签名请求

```
$ certtool -q --load-privkey private_key --outfile file
```

生成自签名证书

```
$ certtool -s --load-privkey private_key --outfile file
```

另请参见

- 官方网站

5.3 Network Security Services

难度 入门 | **篇幅** 4.4 KB | **所属** TLS

本节将讲到

- 安装
- 用法
- 参见

Network Security Services(*NSS*)是一组旨在支持跨平台开发具有安全功能的客户端和服务器应用程序的库。

使用NSS构建的应用程序可以支持SSL v2和v3、TLS、PKCS #5、PKCS #7、PKCS #11、PKCS #12、S/MIME、X.509 v3证书以及其他安全标准。

许多软件包都需要NSS，包括，例如，Chromium和Firefox。

安装

安装 nss包包。

用法

使用NSS提供的*certutil*工具来管理你的证书。

列出证书数据库

要获得所有证书的列表。

```
$ certutil -d sql:$HOME/.pki/nssdb -L
```

要获得证书的详细信息。

```
$ certutil -d sql:$HOME/.pki/nssdb -L -n certificate_nickname
```

生成一个RSA私钥

```
$ certutil -G -d database_directory -g keysize -n nickname
```

生成一个证书签名请求

```
$ certutil -S -s subject -n nickname -x -t C,C,C -o file
```

生成一个自签名的证书

```
$ certutil -S -s subject -n nickname -x -t C,C,C -o file
```

导入证书

要添加一个证书，请指定`-A`选项。

```
$ certutil -d sql:$HOME/.pki/nssdb -A -t "'TRUSTARGS'" -n certificate_nickname -
i /path/to/cert/filename
```

`TRUSTARGS`是由零个或多个字母组成的三个字符串，用逗号分隔，例如，`"TCu,Cu,Tuw"`。它们定义了证书在SSL、电子邮件和对象签名中应如何被信任，在certutil documentation或Meena's blog post中对信任标志进行了解释。

要为SSL客户认证添加个人证书和私钥，请使用以下命令。

```
$ pk12util -d sql:$HOME/.pki/nssdb -i /path/to/PKCS12/cert/filename.p12
```

这将导入一个存储在PKCS #12文件中的个人证书和私钥。个人证书的TRUSTARGS将被设置为"u,u,u"。

编辑证书

使用-M选项调用*certutil*来编辑证书。例如，要编辑TRUSTARGS。

```
$ certutil -d sql:$HOME/.pki/nssdb -M -t "'TRUSTARGS'" -n certificate_nickname
```

删除证书

使用-D选项来删除证书。

```
$ certutil -d sql:$HOME/.pki/nssdb -D -n certificate_nickname
```

添加一个受信任的CA证书

chromium包、firefox包、thunderbird包、evolution包和seamonkey包使用NSS检索可信的CA证书。

nss包已经集成p11-kit包，它将使用system-wide自动安装所有证书。但如果你更喜欢分发 "纯净" NSS，可以将证书安装到你自己的浏览器配置文件中。

```
certutil -d database -A -i myCA.cert -n "Honest Achmed's CA" -t C, 。
```

chromium包和evolution包使用一个位于-d "sql:$HOME/.pki/nssdb"的 "共享式" 数据库。

对于firefox包、thunderbird包和seamonkey包，要制定浏览器专用的配置文件目录（例如-d ~/.mozilla/firefox/ov6jazas.default）。

参见

- 网络安全服务 - Mozilla
- 使用证书数据库工具--Mozilla
- 管理NSS数据库中的证书信任标志 - Meena Vyas, Oracle

5.4 OpenSSL

难度 进阶 | **篇幅** 10.1 KB | **所属** TLS

本节将讲到

- 安装
- 配置
- 用法
- Troubleshooting
- See also

翻译状态：

- 本文（或部分内容）译自 OpenSSL，最近一次同步于 2023-06-01，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。

附注： 请完成翻译。大部分内容尚未翻译。（在 Talk:OpenSSL# 中讨论）

OpenSSL是 SSL 和 TLS 协议的开源实现，旨在尽可能灵活。OpenSSL 支持各种平台，包括 BSD、Linux、OpenVMS、Solaris 和 Windows。

警告：2015 年 5 月发布的对 OpenSSL 协议使用情况的合作研究显示 SSL 连接存在重大风险 “Logjam”。可在 https://weakdh.org/ 查看结果，在 https://weakdh.org/sysadmin.html 查看建议的服务端配置。

安装

Arch Linux 默认安装 openssl包（作为 coreutils包 的依赖）。

有许多 OpenSSL 库绑定可供开发者使用：

- python-pyopenssl包
- perl-net-ssleay包
- lua-sec包、lua52-sec包 和 lua51-sec包
- haskell-hsopenssl包
- haskell-openssl-streams包

配置

在 Arch Linux 中 `OPENSSLDIR` 为 `/etc/ssl`。

OpenSSL 配置文件通常位于 `/etc/ssl/openssl.cnf`，乍一看可能很复杂。注意在赋值中可以展开变量，这与 Shell 脚本的工作方式很相似。配置文件格式的详细解释，请见 config(5ssl)。

req 部分

本文或本章节可能需要合并到#生成证书签名请求。

附注： 主题相同。（在 \\[[en:Talk:OpenSSL#Plan\]\] 中讨论）

与生成密钥、请求和自签名证书有关的设置。

The req section is responsible for the DN prompts. A general misconception is the *Common Name* (CN) prompt, which suggests that it should have the user's proper name as a value. End-user certificates need to have the **machine hostname** as CN, whereas CA should *not* have a valid TLD, so that there is no chance that, between the possible combinations of certified end-users' CN and the CA certificate's, there is a match that could be misinterpreted by some software as meaning that the end-user certificate is self-signed. 某些 CA 证书甚至没有 CN，例如 Equifax：

```
$ openssl x509 -subject -noout < /etc/ssl/certs/Equifax_Secure_CA.pem
subject= /C=US/O=Equifax/OU=Equifax Secure Certificate Authority
```

用法

请先阅读 Transport Layer Security#Obtaining a certificate。

生成 Curve25519 私钥

```
$ openssl genpkey -algorithm x25519 -out 文件名
```

生成 ECDSA 私钥

```
$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out 文件名
```

生成 RSA 私钥

使用（根据 openssl(1ssl)，代替 *genrsa* 的）genpkey(1ssl)：

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:私钥大小 -out 文件名
```

如果需要加密密钥，使用 `-aes-256-cbc` 选项。

生成证书签名请求

使用 req(1ssl)：

```
$ openssl req -new -sha256 -key 私钥 -out 文件名
```

显示证书签名请求

证书签名请求以编码形式存储。要以人类可读的形式查看请求：

```
$ openssl req -noout -text -in 文件名
```

生成自签名证书

```
$ openssl req -key private_key -x509 -new -days days -out filename
```

Generate a self-signed certificate with private key in a single command

You can combine the above command in OpenSSL into a single command which might be convenient in some cases:

```
$ openssl req -x509 -newkey rsa:4096 -days days -keyout key_filename -out cert_filename
```

Generate Diffie–Hellman parameters

See Diffie–Hellman key exchange for more information.

Current best practice is to use one of the standard DH groups from RFC:7919, eg. ffdhe2048.

Alternatively you can generate a random group of your own:

```
$ openssl dhparam -out filename 2048
```

提示：To speed up generating, especially when not on high-end hardware, add the `-dsaparam` option .

显示证书信息

```
$ openssl x509 -text -in 证书文件名
```

显示证书指纹

```
$ openssl x509 -noout -in 证书文件名 -fingerprint -digest
```

`*-digest*` is optional and one of `-md5`, `-sha1`, `-sha256`, or `-sha512`. See "-digest" in x509(1ssl) § Input, Output, and General Purpose Options for when the digest is unspecified.

转换证书格式

Use `openssl x509` to convert certificates from binary (DER) format to PEM format (the text format with `BEGIN CERTIFICATE` headers)：

```
$ openssl x509 -inform DER < myCA.crt > myCA_pem.crt
```

Use third-party providers

OpenSSL 3 introduced providers as a new concept for OpenSSL plugability. It is possible to use algorithms not included in OpenSSL without having to recompile it. For example, to test the NIST Post-Quantum Cryptography algorithms, you can install the Open Quantum Safe provider oqsproviderAUR. As an example, you can generate a quantum-safe self-signed certificate with private key using one of the variants of the Dilithium signature algorithm:

```
$ openssl req -provider oqsprovider -x509 -newkey dilithium3 -days days -keyout key -
out cert
```

Troubleshooting

"bad decrypt" while decrypting

OpenSSL 1.1.0 changed the default digest algorithm for the dgst and enc commands from MD5 to SHA256.

Therefore if a file has been encrypted using OpenSSL 1.0.2 or older, trying to decrypt it with an up to date version may result in an error like:

```
error:06065064:digital envelope routines:EVP_DecryptFinal_ex:bad
decrypt:crypto/evp/evp_enc.c:540
```

Supplying the `-md md5` option should solve the issue:

```
$ openssl enc -d -md md5 -in encrypted -out decrypted
```

Python 3.10 and "ca md too weak" errors

In Python 3.10 by default there is a hardcoded list of allowed OpenSSL ciphers. Some of the less secure, like MD5, have been disabled at the `ssl` module level, ignoring the system-wide configuration of OpenSSL. It results sometimes in strange errors on older certificates, sometimes even when establishing `https` connections, like:

```
requests.exceptions.SSLError: HTTPSConnectionPool(host='a.kind.of.example.com',
port=443): Max retries exceeded with url: / (Caused by SSLError(SSLError(398, '[SSL:
CA_MD_TOO_WEAK] ca md too weak (_ssl.c:3862)')))
```

To make Python follow the system configuration, you may have to rebuild it, adding `--with-ssl-default-suites=openssl` parameter to `./configure`. The issue has been also reported as FS#73549.

See also

- 维基百科的 OpenSSL 页面，包含背景信息。
- OpenSSL 项目页面。
- FreeBSD Handbook
- Step-by-step guide to create a signed SSL certificate
- OpenSSL Certificate Authority: A guide demonstrating how to act as your own certificate authority.
- Bulletproof SSL and TLS by Ivan Ristić, a more formal introduction to SSL/TLS

身份验证

登录入口最先被试探。本章讲在 PAM 上叠加验证手段，以及把身份验证交给中心的域。《Pam oath》与《Kerberos》是主干，必读：前者最短，设置 OATH、接上 PAM、用密码登录跑通，宜先读；后者最长，服务器到 keytab、跨域信任依次展开，需跨机统一身份时再细读。《Pam autologin》只看安全性一节，《Howdy》接人脸时再翻。

最大的坑是把"装上模块"当成"配好验证"：OATH 与 Howdy 的成败都在 PAM 与配置细节上；自动登录的便利，代价写在安全性一节。

6.1　Pam oath

难度 进阶 | **篇幅** 6.0 KB | **所属** 身份验证

本节将讲到

- 安装
- 设置OATH
- 设置 PAM
- 使用 OATH 密码登录
- 参见

OATH Toolkit 为认证系统提供一次性密码（OTP）组件。它包含一个 PAM 认证模块，支持的技术包括基于消息的 HOTP 算法 (RFC 4226）和基于时间的 TOTP 算法 (RFC 6328）。OTP 生成器应用程序可用于 Android、iOS、Blackberry 和其他设备。与 Google Authenticator 类似，该认证机制整合在 Linux PAM 系统中。本指南展示该机制的安装和配置。

安装

安装 oath-toolkit包。

设置OATH

OATH 种子是一个十六进制数字，每个用户应该唯一。为了给一个用户生成一个新的种子，可以使用下面的命令：

```
$ openssl rand -hex 10
12345678909876543210
```

注意：本文中上述输出种子是作为例子使用，**不要**直接使用。

每个用户需要有一个 OATH，并在配置文件 `/etc/users.oath` 中链接到它。使用根用户创建该文件并插入用户种子：

提示：在一个终端登录到具有root用户权限的会话中，并在另一个终端中修改 PAM 配置。这样，如果 PAM 设置出现问题，你仍然有一个账户登录并修复，这在远程连接时特别有用。

```
/etc/users.oath
# Option User Prefix Seed
HOTP/T30/6 user - 12345678909876543210
```

警告：不要设置 T 大于60，否则会有报错：（OATH_UNKNOWN_USER: 无法找到用户的信息）。

如果需要 HOTP，请使用此配置：

```
/etc/users.oath
# Option User Prefix Seed
HOTP user - 12345678909876543210
```

确保该文件只能由 Root 用户访问：

```
# chmod 600 /etc/users.oath
# chown root /etc/users.oath
```

设置 PAM

要想只为某个特定的服务启用 OATH，比如 OpenSSH，可以编辑 `/etc/pam.d/sshd` 文件，并在文件开头添加以下一行：

```
auth     sufficient pam_oath.so usersfile=/etc/users.oath window=30 digits=6
```

如果只输入正确的 OATH 代码即可认证通过。如果使用下面这行，可以把 OATH 作为一个要求，并让 PAM 堆栈的其他部分得到处理：

```
auth     required pam_oath.so usersfile=/etc/users.oath window=30 digits=6
```

为了让 SSH 登录工作，确保这些选项在文件中启用 `/etc/ssh/sshd_config`：

```
ChallengeResponseAuthentication yes
UsePAM yes
```

重启 `sshd.service` 来启用这些更改。

如果想强制 OATH 请求-响应，即使有一个有效的公钥认证和密码认证，把以下内容加入到 `/etc/ssh/sshd_config`：

```
AuthenticationMethods publickey,keyboard-interactive:pam
KbdInteractiveAuthentication yes
PasswordAuthentication yes
```

提示：用于本地登录，编辑 `/etc/pam.d/login` 即可。

使用 OATH 密码登录

对于用 TOTP 登录的情况：

```
$ oathtool -v --totp -d 6 12345678909876543210
```

警告：当 TOTP 模式设置为 `SHA512` 会有一处程序错误，但开发者没有修复。参见 Gitlab 的公开问题。

对于用 HOTP 登录的情况：

```
$ oathtool -v -d 6 12345678909876543210
```

将`*12345678909876543210*`替换为用户对应的种子。将输出类似如下内容：

```
Hex secret: 1ab4321412aebc
```

```
Base32 secret: DK2DEFASV26A====
Digits: 6
Window size: 0
Start counter: 0x0 (0)

820170
```

最后一串数字即可现在用于登录的代码，但 Base32 密钥更加有趣，因为可转换成 QR 码方便转移密钥。安装 qrencode包 并运行以下命令转换：

```
$ qrencode -o user.png 'otpauth://totp/user@machine?secret=DK2DEFASV26A====
```

相应改变*用户*、*机器*和`*DK2DEFASV26A====*`。一旦完成，可以用你喜欢的图像显示程序来显示 QR 码。另外，也可以用以下方法在终端上直接生成 QR 码：

```
$ qrencode -t UTF8 'otpauth://totp/user@machine?secret=DK2DEFASV26A====
```

使用 Aegis Authenticator 或 FreeOTP+，然后对该 `*user*.png`（或类似 ASCII-art 的图像）截图，并在需要时让它显示 OTP 密码，这样十分方便。

注意：你的用户密匙是这个系统中最重要的信息。一旦设置手机来提供 OTP，它就会有这个密钥，`*user*.png` 文件中的 QR 码也有这个密钥。需要特别注意这个文件，它应该只存储在加密的媒介上（你的手机需要使用加密，以达到任何合理的安全水平）。如果没有，可以限制在像三星 Knox 这样的沙盒中，以防止第三方应用程序潜在地访问它们。

参见

- 使用 pam_oath 和 Google Authenticator 的双因素时间（TOTP）SSH 认证
- pam_oath 手册

6.2 Pam autologin

难度 进阶 | **篇幅** 6.6 KB | **所属** 身份验证

本节将讲到

- 安装
- 配置
- 安全性
- 另见

pam_autologin 可以自动登录某个用户。与其他自动登录方式不同，该模块会保存密码，并在正常的认证流程中使用它。随后，密码可以被 pam_mount（英语：pam_mount） 用于挂载加密的家目录，或由 GNOME/Keyring 用于解锁登录密钥环。

安装

安装 pam_autologinAUR 软件包。

要为特定应用启用该模块，请编辑其在 `/etc/pam.d/` 中的配置，例如：

```
/etc/pam.d/login
#%PAM-1.0
```

```
auth     required pam_autologin.so
auth     include  system-local-login
account  include  system-local-login
session  include  system-local-login
```

自动登录这一行应当是 `auth` 部分中的第一个模块，这样它才能设置用户名和密码。如果排在前面的是另一个需要用户名的模块，该模块会提示你输入用户名。

每个 TTY 上只会自动登录一次，因为当你注销时，多半是想登录另一个账户。该模块会在 `/var`、`/log`、`/wtmp` 这些路径中查找本次开机以来在该 TTY 上的登录记录。若要每次都自动登录，可以指定 `always` 选项：

```
auth required pam_autologin.so always
```

如果登录过程由基于 PAM 的应用执行，例如显示管理器，或是像 loginxAUR 那样的 getty/login 组合，那就无需进一步配置。但普通的 agetty 并不是 PAM 应用，它总会提示输入用户名。要为它启用自动登录，需要给 getty 加上 `--skip-login` 选项以便直接启动 login，具体做法是为 systemd 的 `getty@.service` 添加一个附加代码片段：

```
/etc/systemd/system/getty@.service.d/override.conf
[Service]
ExecStart=
ExecStart=-/usr/bin/agetty --skip-login --noreset --noclear - ${TERM}
```

配置

通过创建配置文件来让自动登录模块保存下一次登录：

```
# touch /etc/security/autologin.conf
```

重启后，你应该会看到一条消息，告知下一次登录将被保存。正常登录。再次重启后，你应该会被自动登录。

当你想要停止自动登录时，删除该配置文件：

```
# shred -u /etc/security/autologin.conf
```

请使用 shred 工具在删除文件之前将其内容清零。该文件中包含你的用户名和密码，否则它们可能留在未分配的存储区域中，某些黑客有可能在那里找到它们。尽管该文件用随机密钥做了混淆，但为了让自动登录模块能够自动读取它，密钥必须存放在文件本身之中，因此这只能起到混淆作用，而无法提供保护。

安全性

**本文或本章节的事实准确性存在争议。**

原因： 本节对该主题的论述相当片面，只是展开说明了与模块仓库相同的那些要点，而本应引用它们并论述与 Arch 相关的要点。或许可以补充指向 w:Access control#Computer security、安全、静态数据加密，以及 w:Security token、Smartcards（英语：Smartcards） 之类替代方案的链接。（在 Talk:Pam autologin 中讨论）

自动登录常常被不应得地讥为不安全，因此似乎有必要就此做一些说明。要判断它对你来说是否不安全，你必须问一句：“要防范什么？”对绝大多数用户而言，值得担心的威胁只有两类：恶意软件和失窃。

恶意软件和各种远程入侵威胁，是通过以用户权限运行的程序中的缺陷进来的，通常就是网页浏览器。本模块保存的登录信息只有 root 可读，所以恶意软件首先需要借助一个提权漏洞。以 root 权限运行的恶意软件本来就能用键盘记录器记录你的登录来拿到密码，因此在那种情况下，自动登录并不会降低你的安全性。

失窃是另一种常见威胁，而自动登录确实会让窃贼完全访问一切。失窃对你来说有多大的威胁，因人而异。如果你在家中使用台式机，在大多数地方它被盗的可能性微乎其微。在大城市之外，入室盗窃并不常见。如果你在家工作，那就尤其安全；窃贼不喜欢和人打交道。在台式机上启用自动登录，你的安全程度基本与未启用时相当，而不用输入密码带来的便利很可能超过任何残余的顾虑。

另一方面，笔记本电脑被盗的可能性要大得多。每当你在公共场所把它放在无人看管之处，回来时发现它不见了都不足为奇。在笔记本电脑上使用自动登录通常不是一个好主意，但笔记本电脑并不总是在公共场所使用。如果你只在家中使用笔记本电脑，它遭遇失窃的风险并不比台式机更高，在它上面使用自动登录同样安全。如果你在家以外的地方使用笔记本电脑，但只是偶尔，那你可以在出门前先禁用自动登录。这样你仍然可以让一切保持加密，同时在安全的地点仍能享受自动登录的便利。

在大多数情况下，除了随身携带于公共场所的笔记本电脑之外，使用 pam_autologin 是相当安全的，没什么理由害怕使用它。

另见

- SourceForge 项目页面

6.3 Howdy

难度 进阶 | **篇幅** 7.7 KB | **所属** 身份验证

本节将讲到

- 安装
- 配置
- 疑难杂症

Howdy 是 Linux 上一个类似 Windows Hello，通过电脑的红外传感器识别人脸，解锁电脑的程序。

安装

安装 howdy-gitAUR 软件包。或者您也可以从archlinuxcn仓库安装 howdyCNRepo(howdy包已无法在最新的archlinux上正常使用，因此推荐安装howdy-gitAUR)。

注意：在该软件包的使用过程中会从 GitHub 的 raw 服务器下载文件，而中国大陆用户连接该服务器网络不通畅，请确保你的网络能正常连接。

配置

把 Howdy 设置为当需要时才使用

为了让 Howdy 能够对用户进行身份验证，您需要在使用了 Howdy 的 PAM 配置文件中做一些小改动。请把以下内容添加到对应的配置文件中：

```
auth sufficient pam_python.so /lib/security/howdy/pam.py
```

注意：当使用 Howdy 3.0.0 BETA 及以上版本时(howdy-gitAUR)，这一行应该为：

```
auth sufficient /lib/security/pam_howdy.so
```

为 `howdy/pam.py` (或 `pam_howdy.so`) 添加 ***sufficient*** 的控制标记后，将只提示进行人脸身份验证。如果您无法中断（`Ctrl+C`）面部识别过程（比如在解锁屏幕和Polkit验证窗口中），那您将无法使用密码进行验证。想要既能人脸识别，又能用密码验证，请把以下内容添加到 `/etc/pam.d/` 下对应的 PAM 配置文件的**最顶部**。

```
auth sufficient pam_unix.so try_first_pass likeauth nullok
```

添加正确的红外传感器

可以在诸如 cheese包、fswebcamAUR 或者 v4l-utils包 的程序里找到并确定能连接到红外传感器的正确 `/dev/videoX` 文件，

以内置于 v4l-utils包 软件包的工具为例：

```
$ v4l2-ctl --list-devices
Integrated_Webcam_HD: Integrate (usb-0000:00:14.0-11):
        /dev/video0
        /dev/video1

EyeChip: Tobii Video (usb-0000:00:14.0-3.4.3):
        /dev/video4
        /dev/video5

HD Webcam C525 (usb-0000:00:14.0-3.4.4):
        /dev/video2
        /dev/video3
```

在上述例子中，有不止一个网络摄像头，而且每一个摄像头都有好几个不同的 `/dev/videoX` 路径，一般来说选第一或第二个路径会比较好。

如果摄像头或红外传感器不止一个，使用 `/dev/videoX` 可能会随着时间的推移而变得不稳定，因为它们的路径可能会随着某些设备的拆卸和装入而改变。这时可以用由

Video 4 Linuxwikipedia:Video4Linux 提供的，更加稳定的 `/dev/v4l/by-id/` 路径。

```
$ ls -l /dev/v4l/by-id
total 0
lrwxrwxrwx 1 root root 12 Dec  3 15:01 usb-046d_HD_Webcam_C525_BE4703F0-video-index0 -
> ../../video2
lrwxrwxrwx 1 root root 12 Dec  3 15:01 usb-046d_HD_Webcam_C525_BE4703F0-video-index1 -
> ../../video3
lrwxrwxrwx 1 root root 12 Dec  3 14:47 usb-CNFGH19N306021000582_Integrated_Webcam_HD-
video-index0 -> ../../video0
lrwxrwxrwx 1 root root 12 Dec  3 14:47 usb-CNFGH19N306021000582_Integrated_Webcam_HD-
video-index1 -> ../../video1
lrwxrwxrwx 1 root root 12 Dec  3 14:47 usb-
Tobii_Technology_AB_EyeChip_IS404-100109244721-video-index0 -> ../../video4
lrwxrwxrwx 1 root root 12 Dec  3 14:47 usb-
Tobii_Technology_AB_EyeChip_IS404-100109244721-video-index1 -> ../../video5
```

先验证这些 `v4l` 路径不随设备的拆卸和装入而改变，然后重列设备目录（re-listing the directory）。

找到正确的文件名后，您就可以用自己喜欢的编辑器编辑 `/lib/security/howdy/config.ini` 文件，也可以以 root 用户权限使用 `howdy config` 命令来编辑。

设定 `EDITOR` 变量来自定义 `howdy config` 使用的编辑器：

```
# EDITOR=editor howdy config
```

向 Howdy 添加面部数据

要想添加面部模型，运行 `sudo howdy add`。

加固 Howdy

有些版本的 Howdy 会在识别用户时拍照，然后保存到 `/lib/security/howdy/snapshots`。这某种程度来说是一个安全漏洞。攻击者能够轻易找到某次成功登录的照片并用它来冒充您进行登录，从而达到提权的目的。当然，攻击者还可以使用目标用户的任何其他照片，但是 Howdy 简化了该过程。

为了避免这样的攻击，以及突然发现磁盘空间骤减，可以在 `/lib/security/howdy/config.ini` 中取消拍照：

```
[snapshots]
save_failed = false
save_successful = false
```

疑难杂症

红外发射器不工作

已启用红外摄像头而红外发射器不工作，没准是因为你选错了文件。打个比方，可能 `/dev/video0` 和 `/dev/video2` 都能识别你的脸部，但是只有 `/dev/video2` 会

启用红外发射器，所以要试试所有的 `/dev/video*X*` 路径，看看有没有哪个能正常工作。

如果还不行，试着跟从 linux-enable-ir-emitter 的指导启用红外发射器。

注意：由于上游代码的原因，linux-able-ir-emitterAUR包的最新版本通过在安装完成后会运行自己的脚本来生成 Systemd 服务文件。这种行为可能不是您想要的，因为它的自定义脚本会修改文件系统根目录。目前已知在版本 `3.2.0-2` 之前，该软件包都带有 Systemd 服务文件，所以您可能需要选择适合自己的版本。

测试红外摄像头

提前确认红外摄像头能正常运作是个挺不错的主意，用 gstreamer包 软件包的以下命令十连拍来测试设备（别忘了用你的红外摄像机的位置替代下面的 `*IR camera*`）：

```
gst-launch-1.0 v4l2src device=IR camera num-buffers=10 ! image/jpeg ! multifilesink location="frame-%02d.jpg"
```

Howdy 无法正常运转

用 root 权限运行 `howdy test` 来确保 Howdy 正常工作，如果看起来一切正常，就检查 PAM 配置并确认其正在工作。例如 SDDM 的某些程序不能在 PAM 下正常工作，会造成一些出乎意料的结果。

如果您安装的是2.x版本，您也可以尝试使用beta版本howdy-beta-gitAUR，参见。

输入设备识别错误

例如 Thinkpad T480 的某些红外传感器需要在配置文件中定义框架高度和宽度：

```
frame_width = 400
frame_height = 400
```

获取传感器输出的高度和宽度：`v4l2-ctl --list-devices --all`

6.4 Kerberos

难度 进阶 | **篇幅** 16.5 KB | **所属** 身份验证

本节将讲到

- 安装
- 服务器配置
- 客户端配置
- 配置 kadmin
- Service principals and keytabs

Kerberos) 是一个网络认证系统. 见 krb5 文档.

安装

在客户端和服务端上安装 krb5包 软件包.

注意：强烈建议使用时钟同步守护进程来保持客户端/服务器时钟同步

如果尚未配置主机名解析，则可以手动将客户端和服务器添加到每台计算机的 hosts(5)文件中。

注意：FQDN（myclient.example.com）必须是hosts文件中IP地址之后的第一个hostname。

服务器配置

域创建

编辑 `/etc/krb5.conf` 来配置你的域:

```
/etc/krb5.conf
[libdefaults]
    default_realm = EXAMPLE.COM

[realms]
    EXAMPLE.COM = {
        admin_server = kerberos.example.com
        # use "kdc = ..." if the kerberos SRV records aren't in DNS (see Advanced
section)
        kdc = kerberos.example.com
        # This breaks krb4 compatibility but increases security
        default_principal_flags = +preauth
    }

[domain_realm]
    example.com  = EXAMPLE.COM
    .example.com = EXAMPLE.COM

[logging]
    kdc          = SYSLOG:NOTICE
    admin_server = SYSLOG:NOTICE
    default      = SYSLOG:NOTICE
```

MIT Kerberos中描述了此文件的格式 documentation

创建数据库:

```
# kdb5_util -r EXAMPLE.COM create -s
Loading random data
Initializing database '/var/lib/krb5kdc/principal' for realm 'EXAMPLE.COM',
master key name 'K/M@EXAMPLE.COM'
You will be prompted for the database Master Password.
It is important that you NOT FORGET this password.
Enter KDC database master key: ***
Re-enter KDC database master key to verify: ***
```

最后，启用并启动 Kerberos 服务:

```
# systemctl enable --now krb5-kdc krb5-kadmind
```

添加主体

使用本地身份验证启动Kerberos管理工具

```
# kadmin.local
Authenticating as principal root/admin@EXAMPLE.COM with password.
```

```
kadmin.local:
```

将用户主体添加到Kerberos数据库：

```
kadmin.local: addprinc myuser@EXAMPLE.COM
WARNING: no policy specified for myuser@EXAMPLE.COM; defaulting to no policy
Enter password for principal "myuser@EXAMPLE.COM": ***
Re-enter password for principal "myuser@EXAMPLE.COM": ***
Principal "myuser@EXAMPLE.COM" created.
```

将KDC主体添加到Kerberos数据库:

```
kadmin.local: addprinc -randkey host/kerberos.example.com
WARNING: no policy specified for host/kerberos.example.com@EXAMPLE.COM; defaulting to
no policy
Principal "host/kerberos.example.com@EXAMPLE.COM" created.
```

最后，将KDC主体添加到服务器的键表中:

```
kadmin.local: ktadd host/kerberos.example.com
Entry for principal host/kerberos.example.com with kvno 2, encryption type aes256-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
Entry for principal host/kerberos.example.com with kvno 2, encryption type aes128-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
```

退出Kerberos本地验证工具:

```
kadmin.local: quit
```

您现在应该能够获得Kerberos凭证了:

```
$ kinit
Password for myuser@EXAMPLE.COM: ***
$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: myuser@EXAMPLE.COM

Valid starting       Expires              Service principal
08/30/2017 14:26:09  08/31/2017 14:26:09  krbtgt/EXAMPLE.COM@EXAMPLE.COM
```

防火墙

Add ALLOW rules to your firewall for any applicable ports/protocols:

- 88, TCP and UDP for Kerberos v5
- 749, TCP and UDP for kadmin if you plan to configure it
- 750, TCP and UDP for Kerberos v4 if you need backwards compatibility

DNS记录

注意：如果在每台机器的krb5.conf文件中都指定了kerberos和kadmin服务器，则不需要这样做

```
db.example.com
kerberos.example.com.           A     1.2.3.4
_kerberos.example.com.          TXT   "EXAMPLE.COM"
_kerberos._udp.example.com.     SRV   0 0  88 kerberos.example.com.
_kerberos-adm._udp.example.com. SRV   0 0 749 kerberos.example.com.
```

注意：不要忘记反向DNS.

客户端配置

编辑客户端的 `/etc/krb5.conf` 来适配你服务器的配置. 您可以从服务器复制此文件，或者只设置所需的领域信息.

测试

您现在应该能够在客户端上获得Kerberos凭证:

```
$ kinit
Password for myuser@EXAMPLE.COM: ***
$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: myuser@EXAMPLE.COM

Valid starting       Expires              Service principal
08/30/2017 15:36:10  08/31/2017 15:36:10  krbtgt/EXAMPLE.COM@EXAMPLE.COM
```

配置 kadmin

You will need /etc/krb5.conf configured on the kadmin client, and the server's firewall configured for kadmin.

Configuring kadmin ACL

Create a principal for administration:

```
kadmin.local:  add_principal myuser/admin@EXAMPLE.COM
WARNING: no policy specified for myuser/admin@EXAMPLE.COM; defaulting to no policy
Enter password for principal "myuser/admin@EXAMPLE.COM": ***
Re-enter password for principal "myuser/admin@EXAMPLE.COM": ***
Principal "myuser/admin@EXAMPLE.COM" created.
```

Add the user to the kadmin ACL file:

```
/var/lib/krb5kdc/kadm5.acl
myuser/admin@EXAMPLE.COM *
```

This file's format is described in the MIT Kerberos documentation

Configure kdc.conf:

```
/var/lib/krb5kdc/kdc.conf
[kdcdefaults]
    kdc_ports = 750,88

[realms]
    EXAMPLE.COM = {
        database_name = /var/lib/krb5kdc/principal
        acl_file = /var/lib/krb5kdc/kadm5.acl
        key_stash_file = /var/lib/krb5kdc/.k5.EXAMPLE.COM
        kdc_ports = 750,88
        max_life = 10h 0m 0s
        max_renewable_life = 7d 0h 0m 0s
    }
```

This file's format is described in the MIT Kerberos documentation

Restart `krb5-kdc.servce` and `krb5-kadmind`.

You can now use kadmin as your own user, authenticating with kerberos:

```
$ kadmin
Authenticating as principal myuser/admin@EXAMPLE.COM with password.
Password for myuser/admin@EXAMPLE.COM: ***
kadmin:
```

Service principals and keytabs

First, ensure you have configured krb5.conf on all involved machines.

A kerberos principal has three components, formatted as \primary/instance@REALM\. For user principals, the primary is your username and the instance is omitted or is a role (eg. "admin"): \myuser@EXAMPLE.COM\ or \myuser/admin@EXAMPLE.COM\. For hosts, the primary is "host" and the instance is the server FQDN: \host/myserver.example.com@EXAMPLE.COM\. For services, the primary is the service abbreviation and the instance is the FQDN: \nfs/myserver.example.com@EXAMPLE.COM\. The realm can often be omitted, the local computer's default realm is usually assumed.

With remote kadmin

This is the easier method, but requires you to have configured kadmin.

Open kadmin as root (so we can write the keytab) on the client, authenticating with your admin principal:

```
client# kadmin -p myuser/admin
Authenticating as principal myuser/admin with password.
Password for myuser/admin@EXAMPLE.COM:
kadmin:
```

Add a principal for any services you will be using, eg. "host" for SSH authentication or "nfs" for NFS:

```
kadmin: addprinc -randkey host/kbclient.example.com
WARNING: no policy specified for host/kbclient.example.com@EXAMPLE.COM; defaulting to
no policy
Principal "host/kbclient.example.com@EXAMPLE.COM" created.
```

Save each key to the local keytab:

```
kadmin: ktadd host/kbclient.example.com
Entry for principal host/kbclient.example.com with kvno 2, encryption type aes256-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
Entry for principal host/kbclient.example.com with kvno 2, encryption type aes128-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
```

Without remote kadmin

Start kadmin on the Kerberos server, using either unix or kerberos authentication:

```
# kadmin.local
Authenticating as principal root/admin@EXAMPLE.COM with password.
kadmin.local:
```

Add a principal for any services you will be using, eg. "host" for SSH authentication or "nfs" for NFS:

```
kadmin.local: addprinc -randkey host/kbclient.example.com
WARNING: no policy specified for host/kbclient.example.com@EXAMPLE.COM; defaulting to
no policy
Principal "host/kbclient.example.com@EXAMPLE.COM" created.
```

Save each key to a new keytab to be transferred to the client:

```
kadmin.local: ktadd -k kbclient.keytab host/kbclient.example.com
Entry for principal host/kbclient.example.com with kvno 2, encryption type aes256-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
Entry for principal host/kbclient.example.com with kvno 2, encryption type aes128-cts-
hmac-sha1-96 added to keytab FILE:/etc/krb5.keytab.
```

Finally, copy kbclient.keytab from the server to the client using SCP or similar, then put it in place with correct permissions:

```
# install -b -o root -g root -m 600 kbclient.keytab /etc/krb5.keytab
```

Finally, delete kbclient.keytab from the server and client.

Cross-Realm Trust

Set up a second server as shown above, then create the cross-realm principal on both KDCs. Cross-realm principals must be created with strong passwords, not `-randkey`, and the same password must be used on both KDCs. The principal must have the same key version number (kvno) in both KDCs.

To grant EXAMPLE.COM principals access to EXAMPLE.ORG resources, you would use the following principal:

```
kadmin# addprinc krbtgt/EXAMPLE.ORG@EXAMPLE.COM
```

The `[capaths]` section of `krb5.conf` can be used to further control cross-realm trust relationships.

SSH authentication

Use the instructions in Service principals and keytabs to create a principal for the "host" service for both client and server, then put the client's keys in the client's keytab and the server's keys in the server's keytab.

Modify your SSH server configuration to enable GSSAPI authentication:

```
/etc/ssh/sshd_config
# GSSAPI Options
GSSAPIAuthentication yes
GSSAPICleanupCredentials yes
```

And modify your client configuration to send GSSAPI requests:

```
/etc/ssh/ssh_config
Host *
  GSSAPIAuthentication yes
  GSSAPIDelegateCredentials yes
```

Get a ticket-granting ticket on the client before using ssh:

```
$ kinit myuser@EXAMPLE.COM
Password for myuser@EXAMPLE.COM: ***
```

Pass the -v option to ssh to watch what's happening:

```
$ ssh sshserver.example.com -v
debug1: Authentications that can continue: publickey,gssapi-with-mic,password
debug1: Next authentication method: gssapi-with-mic
debug1: Delegating credentials
debug1: Delegating credentials
debug1: Authentication succeeded (gssapi-with-mic).
Authenticated to sshserver.example.com ([192.168.100.136]:22).
debug1: channel 0: new [client-session]
debug1: Requesting no-more-sessions@openssh.com
debug1: Entering interactive session.
debug1: pledge: network
debug1: client_input_global_request: rtype hostkeys-00@openssh.com want_reply 0
Last login: Wed Aug 30 15:52:41 2017 from 192.168.100.1
```

And you should now see a host ticket on the client:

```
client$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: myuser@EXAMPLE.COM

Valid starting       Expires              Service principal
08/30/2017 15:37:40  08/31/2017 15:37:40  krbtgt/EXAMPLE.COM@EXAMPLE.COM
```

```
08/30/2017 15:53:04  08/31/2017 15:37:40  host/sshserver.example.com@EXAMPLE.COM
```

Authorize other principals

To allow a different kerberos principal to authenticate to a user account, add the principal name to the target account's `.k5login` file. For example, to allow `robert@EXAMPLE.COM` to SSH to alice's account:

```
/home/alice/.k5login
robert@EXAMPLE.COM
```

NFS security

First, configure your NFS server server. Also see NFS Troubleshooting. Configuring a 时钟同步 daemon on both the clients and the server is strongly recommended. Clock drift will cause this to break, and the error message will not be helpful.

Use the instructions in Service principals and keytabs to create a principal for the "nfs" service for both client and server, then put the client's keys in the client's keytab and the server's keys in the server's keytab.

NFS server

Add a Kerberos export option. Multiple options can be specified using a colon as a delimiter:

- sec=krb5 uses kerberos for authentication only, and transmits the data unauthenticated and unencrypted.
- sec=krb5i uses kerberos for authentication and integrity checking, but still transmits data unencrypted.
- sec=krb5p uses kerberos for authentication and encryption.
- sec=sys does not use kerberos

```
/etc/exports
/srv/export *(rw,async,no_subtree_check,no_root_squash,sec=krb5p:krb5)
```

And reload the exports:

```
# exportfs -arv
```

NFS client

Mount the exported directory:

```
# mount nfsserver:/srv/export /mnt/
```

You can add -vv for verbose information, and may need -t nfs4 and -o sec=krb5p or your chosen security option.

Check that it worked with the `mount` command:

```
mount | grep krb5
nfsserver:/srv/export on /mnt type nfs4
(rw,relatime,vers=4.1,rsize=131072,wsize=131072,namlen=255,hard,proto=tcp,port=0,timeo=
600,retrans=2,sec=krb5,clientaddr=192.168.100.139,local_lock=none,addr=192.168.100.136)
```

Browsers

Some browsers have support for Kerberos protocol but disable it by default. Here are the instructions how to enable it:

Chromium

Chromium needs to be run with a command line parameter that specifies a list of sites where Kerberos authentication is allowed. The easiest way is to add persistent flag to the config file:

```
/etc/chromium/policies/managed/test_policy.json
{
  "AuthServerWhitelist": "*.mycompany.com",
  "DisableAuthNegotiateCnameLookup": true
}
```

Firefox

To configure Firefox with trusted sites visit `about:config` and set `network.negotiate-auth.trusted-uris` property to FOO.COM (Note: for Firefox there is no "*."; for Chrome, there is).

Troubleshooting

Cannot set GSSAPI authentication names

```
Cannot set GSSAPI authentication names, aborting
```

Your realm is missing either the `kadmin/admin` or `kadmin/changepw` principal.

For clients, invalid arguments/options may happen on first setup if rpc-gssd is not loaded. Loading it is usually acomplished by enabling and starting `nfs-client.target`, but after first setup this target will need a restart.

SSH authentication fails while connecting to a server requiring GSSAPI with KeyExchange

If any of the following errors are encountered:

```
$ ssh -v -o GSSAPIDelegateCredentials=yes -o GSSAPIAuthentication=yes <user>@<IP
address>
Unable to negotiate with <IP address> port 22: no matching key exchange method found.
Their offer: gss-group14-sha1-...
$ ssh -v -o GSSAPIDelegateCredentials=yes -o GSSAPIKeyExchange=yes -o
GSSAPIAuthentication=yes <user>@<IP address>
command-line: line 0: Bad configuration option: gssapikeyexchange
```

it means that package openssh包 is not configured with GSSAPI patch for OpenSSH. You can install openssh-gssapiAUR or follow this method.

See also

- RHEL7: Configure a Kerberos KDC
- RHEL7: Configure a system to authenticate using Kerberos

OpenPGP

软件包的信任终究是人的信任：包是谁打的，他的 PGP 密钥是什么，这条链又怎样落到 makepkg 的配置里。本章把维护者的规矩与构建的操作接起来：新维护者的待办清单、AUR 与 extra 各自的准则、辞职时的交代事项，以及 makepkg 的配置、使用、优化与问题处理。

建议先读《Makepkg》，配置与问题处理必须吃透，优化和小技巧留到需要提速时再翻；《软件包维护者指导原则》只需精读与密钥指纹有关的那一条，待办清单与辞职事项除非要长期维护仓库的包，可直接跳过。最容易踩的坑是拿默认密钥凑合：PGP 密钥不是默认密钥时，必须在配置里写明正确指纹，不然取用的仍是默认密钥而非你指定的那一把；另一个误解是把 AUR 与 extra 的准则当成同一套。

7.1　软件包维护者指导原则

难度 进阶 | **篇幅** 19.5 KB | **所属** OpenPGP

本节将讲到

- 新软件包维护者的 TODO 列表
- 软件包维护者与 AUR
- 软件包维护者与 extra，软件包维护准则
- Optional
- If your PGP key is not the default, specify the right fingerprint:

软件包维护者是负责使 AUR 正常工作的 Arch Linux 工作人员。他们负责维护热门软件包（并在必要时与上游项目沟通或者向上游项目提交补丁），并且参与管理事务的表决。新的软件包维护者由现任包维护者从活跃的社区成员中民主选举产生，包维护者是唯一有权决定 AUR 发展方向的成员群体。

软件包维护者们根据软件包维护者细则来管理社区。

新软件包维护者的 TODO 列表

1. 仔细阅读本页面
2. 阅读软件包维护者细则
3. 确认自己在 AUR 的账户信息是最新的
4. 让一个推荐人在 AUR 中给您软件包维护者状态
5. 提醒 bureaucrat 将你的维基账号添加到 ⟨group-archpackager⟩ 组
6. 提醒 BBS 管理员更改你在论坛上的账户
7. 让推荐人提供 #archlinux-staff 和 #archlinux-packaging 密码，并加入频道（非必须，但有助于团队成员相互认识并合作）
8. 如果需要 bouncer，请找 heftig 申请加入 Matrix

9. 如果需要 `@archlinux/trusteduser/*username*`，请连续群组联络人 group contacts
10. 如果想要 `@archlinux/package-maintainer/*username*` cloak，请找群组联系人进行协助
11. 让一个赞助者在基础设施仓库问题追踪中使用 `Onboarding` 模板创建工单，并提供如下信息：
12. 一个 SSH 公钥，如果还没有，参考 SSH 密钥#生成密钥对生成一个。
13. SSO 帐号要使用的用户名，此名称也会用来创建 `@archlinux.org` 邮箱地址。
14. 全名
15. 个人邮箱地址和对应的有效 PGP 公钥 ID，该信息将用于提供链接到 SSO 账户的开发者界面（archweb）初始密码
16. 是否将你的私人或（尚未创建的）`*username*@archlinux.org` 邮箱地址用在非公开邮件清单，并允许发布到 arch-dev-public 邮件清单
17. 按照 arch:DeveloperWiki:Staff Services#Email 配置 `@archlinux.org` 邮箱的密码
18. 参考添加新打包人密钥的流程创建用于软件包签名的新 PGP 密钥对（使用新的 `*username*@archlinux.org` 邮箱地址作为 uid）
19. 让一位推荐人在 archlinux-keyring 代码库问题追踪页使用 `New Packager Key` 模板创建工单，以使你的 PGP 密钥受至少三个主密钥持有者签名
20. 安装 devtools包 软件包
21. 为 `repos.archlinux.org` 配置你的 ssh 私钥
22. 获得权限后，ssh 到 `*yourname*@repos.archlinux.org`
23. 开始你的贡献吧!

初级维护者

Since the ratification of RFC 0014, new Package Maintainers will be marked as "junior" for their first two months of packaging. During this time, the new Package Maintainer may only push to the *extra-testing* repository. Your sponsors can review your packages as-needed and move them to *extra*.

软件包维护者与 AUR

软件包维护者应检查被提交到 AUR 中的软件包是否含有恶意代码，以及 PKGBUILD 是否符合标准。80% 左右的 PKGBUILD 都非常简单，包维护者团队可以很快地检查其可用性以及安全性。

软件包维护者也应该检查 PKGBUILD 的一些小错误，提出修正和改进建议。软件包维护者应该努力确认所有的软件包是否遵循了 Arch 打包指导准则/标准，并与其他打包者分享他们的技能和经验来提升 archlinux 的总体软件打包质量。

另外，也建议软件包维护者撰写文档来记录一些推荐做法。

Rewriting git history

In some cases rewriting the history of an AUR repository is required, for example when a user inadvertently uses their legal name in a published commit. This can be automated with git-filter-branch(1).

To force push the new history, forward the `AUR_OVERWRITE=1` environment variable to git-push(1).

In detail this includes adding `SendEnv AUR_OVERWRITE` to your AUR SSH config and setting the env var on your push command: `AUR_OVERWRITE=1 git push --force`. See for details.

警告：It is recommended to create a backup of the repository before rewriting history.

Modify committer or author identity

Install git-filter-repo包 and run:

```
$ git-filter-repo --name-callback 'return name.replace(b"Old name", b"New name")' --email-callback 'return email.replace(b"old@email.com", b"new@email.com")'
```

提示：If the username contains special characters you may need to encode the strings `name.replace("Bás Ssze".encode("utf-8"), b"newname")'`

Alternatively, use `git filter-branch --env-filter` with the `GIT_AUTHOR_NAME`, `GIT_AUTHOR_EMAIL`, `GIT_COMMITTER_NAME` and `GIT_COMMITTER_EMAIL` environment variables. For example:

```
git filter-branch --env-filter '
if test "$GIT_AUTHOR_EMAIL" = "lepetit@prince.com"; then
  GIT_AUTHOR_EMAIL=user@users.noreply.github.com
fi
if test "$GIT_AUTHOR_NAME" = "Antoine de Saint-Exupéry"; then
  GIT_AUTHOR_NAME=user
fi'
```

注意：git-log(1) only displays the git *author* by default. Use `git log --pretty=fuller` to display the *author* and *committer*.

Handling AUR requests

Package Maintainers should periodically check the requests filed on the AUR. For that there are some generic rules what to check for each request type:

Orphan request

- check if the request is older then 14 days (the date column turns red in the overview) (you cannot accept it before that anyway)

- check if there was no update to the package itself (commit or release) done in the past 14 days
- check if there was no comment from the AUR package maintainer done in the past 14 days

If all of the above points are true then you can accept the Orphan Request.

软件包维护者与 extra，软件包维护准则

软件包进入 \[extra\] 仓库的规则

- 软件包不能存在于任何 Arch Linux 仓库中。你还需要确保没有其他打包者同时提交该软件包。检查 AUR 的包评论，阅读 aur-general 中新文章的标题，grep git-log(1) 日志，并向私密打包 IRC 频道发送消息。
- Pacman 封装是一个特例，不管投票数有多高，都不会被批准进入。如果是 AUR 助手，可以写封邮件到 `arch-dev-public` 进行提议，并尊重团队成员提出的反对意见。

**本文或本章节的事实准确性存在争议。**

原因： The following rule is arbitrary and not enforced in practice. The next two points need to be adjusted as well.（在 Talk:软件包维护者指导原则 中讨论）

- 只有“流行”的软件包才能进入仓库（指 pkgstats 有 1% 使用数或者 AUR 中 10 票以上
- 自动略过该规则的软件包有：
- i18n 包
- 辅助功能软件包
- 驱动程序
- 满足“流行”定义的软件包的依赖包，包括 makedeps 和 optdeps
- 属于同一集合并一起发布的软件包，前提是这一集合的某些软件包满足“流行”定义
- 任何不属于上述例外的其它软件包都需首先在 aur-general 邮件列表中提出申请，并解释成为例外的原因（例如重命名的软件包、新软件包等）。软件包进入 *extra* 仓库需要其他三名软件包维护者同意。维护大量“不流行”软件包的软件包维护者的申请更有可能被拒绝。
- 强烈建议软件包维护者移除 *extra* 仓库中他们维护的低使用率软件包。该项非强制，但在有人接手前离职维护者的软件包有可能会被过滤掉
- 软件包从 AUR 提升到其它仓库时，应该将 **pkgrel** 的数值加 *1*（即设为 *n* + *1*），这样已安装软件包的用户可以从官方仓库继续收到自动更新。还有一个好处就是避免 **pkgrel** 重置为 *1* 时用户收到本地软件包比仓库更新的警告

- 所有官方软件包的构建脚本都以 0BSD 许可证提供。如果 AUR 包的构建脚本没有显式以 0BSD 进行许可，则需要进行重写

访问并更新仓库

参考软件包维护者指引。

停止维护软件包

如果一个软件包维护者不想再维护一个软件包了，需要在 AUR 邮件列表中发出一个通知，以便其他的软件包维护者可以接手。即使没有人接手，软件包维护者也可以停止维护，但应避免放弃大量软件包（即不应接手超出能力范围的包数量）。如果一个软件包已经过时或不再被使用，那么可以将其完全移除。

如果一个软件包已经被完全移除，可以将其重新上传到 AUR，使得普通用户可以替软件包维护者进行维护。

将软件包从 AUR 移到 extra

遵循打包者指引中将软件包添加到 *extra* 的正常步骤即可，但记得将对应的包从 AUR 中移除！

将软件包从 extra 移至 AUR

按照打包者指引移除软件包，并将源代码上传到 AUR 即可。

将软件包从 extra-testing 移至 extra

Move the package from the *extra-testing* to the *extra* repository using the instructions in the packager guide.

在 build.archlinux.org 上远程编译

警告：The following procedures defeats the Web Of Trust model: a user with root access to PKGBUILD.com could alter the package and/or the signature before it gets published.

软件包维护者和开发者可以 SSH 到 build.archlinux.org，并使用 devtools 编译软件包。 和本地编译相比，这样做有如下好处：

- 编译快，网速快
- 环境仅需要配置一次
- 本地系统可以不是 Arch Linux

The process is similar to that of a local setup with devtools. Your GnuPG private is required for signing but you do not want to upload it for obvious security reasons. As such, you will need to forward the GnuPG agent socket from your local machine to the server: this will allow you to sign packages on the build server without

communicating your key. This also means that we need to disable the agent on the server before we can run anything.

First, connect to build.archlinux.org and disable

```
$ ssh build.archlinux.org
$ systemctl --user mask gpg-agent.service
```

Make sure gpg-agent is not running (`systemctl --user stop gpg-agent.service`). At this point, make sure that no sockets exist in the folder pointed by `gpgconf --list-dir socketdir`. If they do, remove them or log out and in again. If you have a custom $GNUPGHOME (eg. to move it to `~/.config/gnupg`), you will need to unset that, as it is not possible in gnupg to set the homedir without setting the socketdir. On build.archlinux.org, `StreamLocalBindUnlink yes` is set in `sshd_config`, therefore removing the sockets manually on logout is not necessary.

While the PGP private keys remain on your local machine, the public keys **must** be on the build server. Export your public ring to the build server, e.g. from you local machine

```
$ scp ~/.gnupg/pubring.gpg build.archlinux.org:~/.gnupg/pubring.gpg
```

SSH is required to checkout and commit to the Git repository. You can either set up a new SSH key pair on the server (it is highly discouraged to put your local private key on a server for security reasons) or reuse your local keys via socket forwarding. If you opt for the latter, make sure to disable ssh-agent on the build server if you had enabled it previously (it is not running by default).

Configure you build environment on the build server:

```
~/.makepkg.conf
PACKAGER="John Doe <john@doe.example>"
## Optional
PKGDEST="/home/johndoe/packages"
SRCDEST="/home/johndoe/sources"
SRCPKGDEST="/home/johndoe/srcpackages"
LOGDEST="/home/johndoe/logs"
## If your PGP key is not the default, specify the right fingerprint:
GPGKEY="ABCD1234..."
```

警告：Forwarding your gpg-agent socket to a remote machine makes it possible for anyone with root access to that system to use your unlocked GPG credentials. To circumvent this issue, we need to disable passphrase caching.

Disable passphrase caching with the following settings:

```
gpg-agent.conf
default-cache-ttl 0
max-cache-ttl 0
```

Because we will want to keep our usual GPG agent running with its current settings, we are going to run another GPG agent dedicated to the task at hand. Create a `~/.gnupg-archlinux` folder and symlink everything from `~/.gnupg` there, except `~/.gnupg/gpg-agent.conf`. Configure the new GPG agent:

```
~/.gnupg-archlinux
extra-socket /home/doe/.gnupg-archlinux/S.gpg-agent.extra
default-cache-ttl 0
```

```
max-cache-ttl 0
pinentry-program /usr/bin/pinentry-gtk-2
```
The `gpg-agent-extra.socket` will be forwarded to build.archlinux.org.

Start the dedicated agent with

```
$ gpg-agent --homedir ~/.gnupg-archlinux --daemon
```
Connect with:

```
$ ssh -R REMOTE_SSH_AUTH_SOCK:$SSH_AUTH_SOCK -R /run/user/REMOTE_UID/gnupg/S.gpg-agent:/home/doe/.gnupg-archlinux/S.gpg-agent.extra build.archlinux.org
```
or, if using GnuPG as your SSH agent:

```
$ ssh -R /run/user/REMOTE_UID/gnupg/S.gpg-agent.ssh:/run/user/LOCAL_UID/gnupg/S.gpg-agent.ssh -R /run/user/REMOTE_UID/gnupg/S.gpg-agent:/home/doe/.gnupg-archlinux/S.gpg-agent.extra build.archlinux.org
```
Replace `*REMOTE_UID*` and `*LOCAL_UID*` by your user identifier as returned by `id -u` on the build server and locally, respectively. If using ssh-agent, replace `*REMOTE_SSH_AUTH_SOCK*` by the path to the SSH socket on the remote host (it can be anything).

You can make the forwarding permanent for that host. For instance with gpg-agent.ssh:

```
~/.ssh/config
Host build.archlinux.org
  RemoteForward /run/user/REMOTE_UID/gnupg/S.gpg-agent /run/user/%i/gnupg/S.gpg-agent.extra
  RemoteForward /run/user/REMOTE_UID/gnupg/S.gpg-agent.ssh /run/user/%i/gnupg/S.gpg-agent.ssh
```
Again, replace `*REMOTE_UID*` with the user UID on the build server.

From then on, the procedure should be exactly the same as a local build:

```
$ ssh build.archlinux.org
$ pkgctl repo clone existing-package
$ ...
```
注意：pinentry-curses might not work with socket forwarding. If it fails for you, try using a different pinentry.

软件包维护者辞职需要完成的事项

当一个软件包维护者辞去自己的职务时，需要完成以下清单。如果仅辞去软件包维护者，但仍是开发者，就不适用以下步骤。

1. 此软件包维护者的所有包都需要被重新签名（重新打包）。软件包维护者编译的软件包可以通过下面 URL 从 Archweb 查询 https://archlinux.org/packages/?sort=&q=&packager=$packager&flagged= ，将 packager 替换成包维护者在 Archweb 的用户名即可。
2. Archweb 需要禁用此维护者的帐号，并将其添加到 'Retired Package maintainers' 组。需从 'Package Maintainers' 移除此软件包维护者，并收回所有软件仓库权限。

3. 从服务器上禁用此帐号的 shell 访问权限（尤其是 repos.archlinux.org, pkgbuild.com）。
4. 移除此用户的 GPG key，并向仓库提交新的 archlinux-keyring 软件包。在 keyring 项目中创建 bug 工单，删除辞职人员的密钥。
5. 将包维护者的 AUR 帐号移出 Package Maintainer 组。
6. bureaucrat 应将维护者的维基账户移出 ⟨group-archpackager⟩ 组。
7. BBS 管理员需要修改包维护者的论坛帐号。

7.2 Makepkg

难度 高级 | **篇幅** 29.5 KB | **所属** OpenPGP

本节将讲到

- 配置
- 使用
- 优化
- 小技巧
- 问题处理

翻译状态：

- 本文（或部分内容）译自 makepkg，最近一次同步于 2026-01-21，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

makepkg 是一个软件包自动化构建脚本，使用时需要一个具备构建条件的 Unix 环境和一个 PKGBUILD 文件。

makepkg 是由 pacman包 包提供的。

配置

全局配置位于 `/etc/makepkg.conf`，还可以通过 `$XDG_CONFIG_HOME/pacman/makepkg.conf` 或 `~/.makepkg.conf` 进行用户特定配置。你还可以添加 `/etc/makepkg.conf.d/makepkg.conf` 文件进行系统全局配置更改。在构建软件包前，建议先检查 makepkg 配置。

提示：devtools包 用于从干净的 chroot 构建软件包的助手脚本会使用 `/usr/share/devtools/makepkg.conf.d/*arch*.conf` 配置文件。

更多信息请参考 makepkg.conf(5)。

打包人信息

每个软件包都会有元数据信息，其中就包含 *packager*。默认情况下，用户自己打包的软件标记为 `Unknown Packager`。如果多个用户会在系统上编译，或者需要发布软件包给其他人，最好提供真实的联系人。打包人信息可以通过 `makepkg.conf` 中的 `PACKAGER` 变量进行设置。

要检查已安装软件包的打包人：

```
$ pacman -Qi package
...
Packager        : John Doe <john@doe.com>
...
```

要自动化签名过程，请同时在 `makepkg.conf` 中设置 `GPGKEY` 变量.

包输出

makepkg 默认会在工作目录创建软件包，并把源代码下载到 `src/` 目录。可以配置到自定义的路径，比如将所有构建出的软件包放到 `~/build/packages/` 目录，所有源文件放到 `~/build/sources/` 目录下。

根据需求不同，可以配置 `makepkg.conf` 下的这些变量：

- `PKGDEST` - 软件包构建输出目录
- `SRCDEST` - 用于存放源数据的目录（在指向其它位置的情况下，软链接将被放置到 `src/` 目录下）
- `SRCPKGDEST` - 源代码包构建输出目录（通过 `makepkg -S` 构建）

提示：可以参考 pacman#清理软件包缓存 用命令（例如 `paccache -c ~/build/packages/`）清理 `PKGDEST` 目录

你还可以#在包目录下使用相对路径。

验证签名

注意：*makepkg* 中的签名验证并不使用 pacman 的密钥环, 而是使用用户的密钥。

如果签名文件是以 *.sig* 或 *.asc* 形式作为 PKGBUILD 源码数组的一部分提供，*makepkg* 会自动验证软件包。如果用户未提供需要的签名公钥，*makepkg* 会停止安装过程并提示用户说无法验证 PGP 密钥。

如果缺少软件包所需的公钥，那么 PKGBUILD 很可能带有 validpgpkeys 项，其中包含了所需的密钥 ID。你可以手动进行导入，也可以在在公钥服务器上进行查找，然后导入。运行 *makepkg* 时使用 `--skippgpcheck` 选项可以临时禁用签名检查。

使用

继续之前，确保已安装了 base-devel包组 软件包组。属于这个组的软件包不会被要求列在 PKGBUILD 文件的构建时依赖（*makedepends*）中。

注意：

- 确保为传入 pacman 的命令正确配置了 sudo。另外，可以通过 makepkg.conf(5) 配置文件的 `PACMAN_AUTH` 参数指定不同的认证命令。
- 不允许以根用户（root）身份运行 `makepkg`。 原因除了 `PKGBUILD` 可能包含恶意代码外，以 root 身份构建软件一般都被认为是不安全的。 无法使用普通用户时需以 nobody 用户的身份运行 `makepkg`（例如 `runuser -u nobody makepkg`）。

要构建软件包，用户必须首先创建一个 PKGBUILD 文件或编译脚本（在创建软件包中有详细描述），或者从 Arch 构建系统（*ABS*）、AUR 或其他来源获取。获取到 `PKGBUILD` 后，切换到该文件所在文件夹中，然后执行以下命令构建软件包：

```
$ makepkg
```

如果需要的依赖不满足，*makepkg* 会输出一个警告然后失败。想要编译软件包并自动安装必须的依赖，只需添加 `-s`/`--syncdeps` 参数：

```
$ makepkg --syncdeps
```

如果添加了 `-r`/`--rmdeps` 选项，*makepkg* 会在结束后删除不再需要的编译依赖。如果需要持续编译软件包，请考虑定期删除未使用软件包。

注意：

- 这些依赖必须在已配置的软件源之中存在，具体信息请参考 pacman#软件仓库。另外，用户也可以在编译前手动安装需要的依赖：(`pacman -S --asdeps *dep1* *dep2*`)。
- 在安装依赖包时，只使用全局值，即不会使用拆分包打包功能中的任何覆盖值。

在满足所有依赖并成功编译软件包后，一个软件包文件（`*pkgname*-*pkgver*.pkg.tar.zst`）将会被创建在工作目录下。如需安装，请使用 `-i`/`--install` 参数（与 `pacman -U *pkgname*-*pkgver*.pkg.tar.zst` 相同）：

```
$ makepkg --install
```

要清理残余的文件和目录（如解压到 `$srcdir` 的文件），请使用 `-c`/`--clean` 选项。这对于在使用同一个文件夹多次编译同一个软件包或者升级软件包版本时很有用。它可以防止过期或残余的文件被呈递到新的构建任务中：

```
$ makepkg --clean
```

更多信息请阅读 makepkg(8)。

优化

默认选项与 devtools包 为官方仓库构建软件包使用的选项一致。因此，用户可以通过调整以下选项来适应本地环境，以获得或多或少的收益。

优化编译结果

通过启用针对主机的编译器优化，可以提高打包后软件的性能。缺点是，为特定处理器架构编译的二进制文件无法在其他机器上正常运行。在 x86_64 机器上，重新构建官方软件包通常不会有显著的性能提升，因此不值得为此投入时间。

不过，使用 “非标准” 编译器标志很容易降低性能。许多编译器优化仅在某些情况下有用，不应对软件包随意应用。除非有评测数据证明某项优化更快，否则很有可能会导致性能下降！Gentoo 的 GCC 优化和安全的 CFLAGS 文章提供了有关编译器优化的更多信息。

C 和 C++

传递到 C/C++ 编译器（例如 gcc包 或 clang包）的选项是通过 `CFLAGS`、`CXXFLAGS` 和 `CPPFLAGS` 环境变量传入的。在使用 Arch 构建系统时，*makepkg* 会将 `makepkg.conf` 中的配置项作为这些环境变量进行传递。默认值会生成通用的二进制文件，可安装在各种机器上。

注意：

- 记住，不是所有的构建系统都会使用 `makepkg.conf` 中设置的变量。例如，*cmake* 不会遵循 `CPPFLAGS` 预处理器选项环境变量。因此，很多 PKGBUILD 都包含了针对打包软件所使用的构建系统而设置的选项。
- 源码提供的 `Makefile` 或编译命令行中指定的参数优先级更高，并有可能会取代掉 `makepkg.conf` 中的配置。

GCC 可以自动检测架构，并应用安全的架构特定优化项。要使用该特性，首先需要移除所有 `-march` 和 `-mtune` 标志，然后添加 `-march=native`，例如：

```
/etc/makepkg.conf
CFLAGS="-march=native -O2 -pipe ..."
CXXFLAGS="${CFLAGS} ..."
```

要查看该操作会启用的选项，执行：

```
$ gcc -march=native -v -Q --help=target
```

注意：如果没有指定 `-march=native`，`-Q --help=target` 将**不会**起作用。具体启用的选项需要编译后才能知道。详细步骤请参考。

进一步优化需要启用对内存占用和编译时间有较大负面影响的选项，做法为将优化级别从 `-O2` 设为 `-O3`。使用该选项一般都能提升二进制文件的性能，但具体效果取决于文件本身。更多信息请参考 Gentoo:GCC optimization#-O 和 GCC 优化选项页。

Rust

从 pacman 5.2.2 开始，makepkg.conf 还包含了对 RUSTFLAGS 环境变量的覆写，用于替换 Rust 编译器的标志。通过在 RUSTFLAGS 中添加 -C target-cpu=native，Rust 编译器还能检测并应用架构特定优化项：

```
/etc/makepkg.conf.d/rust.conf
RUSTFLAGS="-C force-frame-pointers=yes -C target-cpu=native"
```

查看该操作会已启用的 CPU 特性：

```
$ rustc -C target-cpu=native --print cfg
```

在不包含 -C target-cpu=native 的情况下运行 --print cfg 将输出默认配置。

下面为一些可以添加到 RUSTFLAGS 的优化参数：

- -C opt-level=3：可以根据需要将该值更改为 3、s 或 z。针对分发的构建默认使用 opt-level=3。
- -C codegen-units=*n*：选择小于 16 的 *n* 值可以优化二进制文件，但会增加构建耗时。
- -C link-arg=-z -C link-arg=pack-relative-relocs 可以降低二进制文件的大小。

更多信息请参考 Rust 编译器文档。

减少编译时间

并行编译

make包 编译系统使用 MAKEFLAGS 环境变量指定 *make* 的额外选项。这个值也可以在 makepkg.conf 文件中进行设置。

使用多核/多处理器系统的用户可以设定同时运行的任务数。可以用 nproc(1) 获得可用处理器的个数，例如：MAKEFLAGS="--jobs=$(nproc)"。

有些 PKGBUILD 会强制使用 -j1，因为某些版本会产生冲突或者软件包本身并不支持。如果出现软件包因为此原因无法编译，请在确认错误是由 MAKEFLAGS 引起的前提下，在 bug 跟踪系统中进行报告（如果是 AUR 包，则向包维护者报告）。

完整可用选项请参考 make(1)。

使用内存文件系统进行编译

编译过程需要大量的读写操作，要处理很多小文件。将工作目录移动到 tmpfs 可能会减少编译时间。

使用 BUILDDIR 变量可以临时将 *makepkg* 的编译目录设置到现有 tmpfs，例如：

```
$ BUILDDIR=/tmp/makepkg makepkg
```

将 makepkg.conf 中的 BUILDDIR 选项取消注释可以永久变更编译目录，具体位于 /etc/makepkg.conf 文件末尾的 BUILD ENVIRONMENT 一节。设置此变量为 BUILDDIR=/tmp/makepkg 可以利用 Arch 默认的 /tmp 临时文件系统。

注意：

- 在 tmpfs 中构建大型软件包时可能会内存不足。
- 挂载 tmpfs 目录时不能使用 noexec 选项，否则编译命令无法执行。
- 在 tmpfs 中构建好的包重启后会消失，设置 PKGDEST 选项可以将构建结果保存到其它目录。

使用编译缓存

ccache 可以将编译结果缓存起来供下次编译使用，减少编译时间。

使用 mold 链接器

mold包 是 ld/lld 链接器的直接替代，据称其速度稍微较快。

要使用 *mold*，需添加 -fuse-ld=mold 到 LDFLAGS，例如：

```
/etc/makepkg.conf
LDFLAGS="... -fuse-ld=mold"
```

要向 *mold* 添加额外选项，可以将它们添加到 LDFLAGS 中，例如：

```
/etc/makepkg.conf
LDFLAGS="... -fuse-ld=mold -Wl,--separate-debug-file"
```

要为 Rust 软件包使用 *mold*，可以将 -C link-arg=-fuse-ld=mold 添加到 RUSTFLAGS 中，例如：

```
/etc/makepkg.conf.d/rust.conf
RUSTFLAGS="... -C link-arg=-fuse-ld=mold"
```

禁用调试包和 LTO

包含在 2024 年 2 月 pacman 6.0.2-9 版本的 90bf367e 号提交默认启用了 debug 和 lto 选项。

构建调试包能让官方仓库为用户提供更多的问题排查工具（archlinux/packaging/packages/pacman#23#note_173528），但在自行构建软件包时，调试包不是必需的，而且会减慢构建进程。参见 archlinux/packaging/packages/pacman#23#note_173782。

链接时优化可以生成更优化的二进制文件，但会增加构建耗时（archlinux/packaging/packages/pacman#23#note_173678），不一定是一个理想的权衡。

如需禁用这些选项，可以在 `OPTIONS=()` 数组中的这些选项前直接添加 `!` 符号，例如：`OPTIONS=(...!debug !lto...)`

压缩

使用其它压缩算法

为了加快打包和安装速度，您可以更改 `PKGEXT`，代价是生成的软件包文件更大。

例如，以下命令可以跳过压缩步骤，使得安装时无需解压：

```
$ PKGEXT='.pkg.tar' makepkg
```

另一个例子使用 lz4 算法，可以提升速度：

```
$ PKGEXT='.pkg.tar.lz4' makepkg
```

要使这些设置之一永久生效，请在 `/etc/makepkg.conf` 中设置 `PKGEXT` 。

在压缩时使用多个 CPU 核心

zstd包 支持对称多处理（SMP），通过添加 `-T`/`--threads` 标志可以加速压缩。默认情况下，`/etc/makepkg.conf` 中的 `COMPRESSZST` 数组包含了 `-T0` 标志，使得 *zstd* 尽可能使用多的物理 CPU 核心来压缩软件包。可以通过 `--auto-threads=logical` 标志让 *zstd* 根据逻辑 CPU 核心数进一步提升调用数：

```
COMPRESSZST=(zstd -c -T0 --auto-threads=logical -)
```

lz4包 和 xz包 默认使用多线程，不需要对 `/etc/makepkg.conf` 进行修改。

pigz包 是 gzip包 的一个替代、并行实现，它默认使用所有可用的CPU核心（可以使用 `-p`/`--processes` 标志来使用较少的核心）：

```
COMPRESSGZ=(pigz -c -f -n)
```

pbzip2包 是 bzip2包 的一个替代、并行实现，它也默认使用所有可用的CPU核心。可以使用 `-p#` 标志来使用较少的核心（注意：`-p` 和核心数之间没有空格）：

```
COMPRESSBZ2=(pbzip2 -c -f)
```

lbzip2包 是 bzip2包 的另一个替代、并行实现，它也默认使用所有可用的CPU核心。可以使用 `-n` 标志来使用较少的核心。

```
COMPRESSBZ2=(lbzip2 -c -f)
```

plzipAUR 是 lzip包 的一个多线程实现，它也默认使用所有可用的 CPU 核心。可以使用 `-n`/`--threads` 标志来使用较少的核心。

```
COMPRESSLZ=(plzip -c -f)
```

修改压缩等级

有几种压缩算法（包括 zstd 和 xz）支持设定压缩等级，以在速度、内存占用和压缩效率间进行取舍。

小技巧

减少下载和解压时间

设定源文件位置

特别是在编译 VCS 软件包时，使用 `SRCDEST` 可以缩短重新构建时获取和解压源代码的时间。

自定义 DLAGENTS

可以使用支持并行下载的下载器，以 axel包 为例：

```
/etc/makepkg.conf
DLAGENTS=('file::/bin/curl -gqC - -o %o %u'
          'ftp::/bin/axel -n 2 -o %o %u'
          'http::/bin/axel -n 2 -o %o %u'
          'https::/bin/axel -n 2 -o %o %u')
```

另请参考 Nonfree applications package guidelines#Custom DLAGENTS。

生成新校验和

安装 pacman-contrib包，然后在 PKGBUILD 文件所在目录中执行以下命令来生成新校验和：

```
$ updpkgsums
```

`updpkgsums` 使用 `makepkg --geninteg` 生成校验和，查看此论坛了解详情

也可以用如 `sha256sum` 命令生成校验和并手动加入到 `sha256sums` 数组中。

从本地源文件生成

如果您想对源代码进行更改，可以使用`-o`、`--nobuild`下载源代码，而无需构建软件包，仅进行下载和解压缩文件选项。

```
$ makepkg -o
```

您现在可以对源代码进行更改，然后使用`-e`，`--noextract` 不提取源文件（使用现有的$srcdir/ dir）选项构建软件包。使用`-f`选项覆盖已经生成的和现有的包。

```
$ makepkg -ef
```

显示具有特定打包程序的包

expac包 是一个pacman数据库提取工具。此命令显示系统上安装的所有打包程序名为 *packagername*的包：

```
$ expac "%n %p" | grep "packagername" | column -t
```

这将显示系统上安装的所有软件包， `/etc/makepkg` 变量 `PACKAGER`仅显示在中定义的存储库中的包 `/etc/pacman.conf`。

```
$ . /etc/makepkg.conf; grep -xvFf <(pacman -Qqm) <(expac "%n\t%p" | grep "$PACKAGER$" | cut -f1)
```

在 64 位系统上构建 32 位软件包

参考32位软件包打包准则。

软件包签名托管

**本文或本章节可能需要合并到 GnuPG#Unattended_passphrase。**

附注： 非 *makepkg* 特定。（在 Talk:Makepkg 中讨论）

在使用类似 Jenkins 的自动构建环境时，用户可能无法为用于软件签名的 gpg 私钥提供密码。另外，不建议在没有密码保护的系统上存放 GPG 私钥。

使用 *makepkg* 创建的 zst 软件包可以在创建后通过如下命令进行签名：

```
$ gpg --detach-sign --pinentry-mode loopback --passphrase --passphrase-fd 0 --output NewlyBuilt.pkg.tar.zst.sig --sign NewlyBuilt.pkg.tar.zst
```

其中 GPG 密码由你选择的自动化套件安全地提供和隐藏。

生成的 `zst` 和 `sig` 文件可被需要有效签名的 pacman 客户端和使用 `repo-add --sign` 创建的软件仓库引用。

磁力链接 URI

使用 transmission-dlagentAUR 下载器可以支持在 `source` 字段使用磁力链接 URI（前缀为 `magnet://`）。

Running makepkg in a systemd control group

If the package you are building takes too many resources to build with your default *make* flags, which are otherwise set properly for most packages, you can try running it in its own control group. makepkg-cgAUR is a wrapper for *makepkg* that achieved this via systemd control groups (see systemd.resource-control(5)).

Running with idle scheduling policy

Package build process can lead to high CPU utilization, especially in case of #Parallel compilation. Under heavy CPU load, the system can issue a significant slowdown up to becoming unusable, even with the highest nice(1) value. User interface and foreground applications may stutter or even became unresponsive.

This can be worked around by changing the scheduling policy to `SCHED_IDLE` before running *makepkg*. It ensures that package building process does not interfere with regular tasks and only utilizes remaining unused CPU time.

From sched(7) § SCHED_IDLE: Scheduling very low priority jobs:

: This policy is intended for running jobs at extremely low priority (lower even than a +19 nice value with the `SCHED_OTHER` or `SCHED_BATCH` policies).

The `SCHED_IDLE` policy can be set by running chrt(1) command with the `-i` flag, specifying priority 0 (the only valid option for `SCHED_IDLE`) and specifying the PID of the current shell.

For most shells:

```
$ chrt -iap 0 $$
```

提示：You can apply this command for every build by placing it into `makepkg.conf`.

For the fish shell, where `$$` is not set:

```
$ chrt -iap 0 %self
```

在包目录下使用相对路径

Instead of using absolute paths for the package output options, you can also configure relative paths inside each package directory.

注意：The following options might cause problems with some AUR helpers, as they might use `makepkg.conf` in a context where `$startdir` is not defined. So be careful.

For example, you can define target paths in your `makepkg.conf` file as follows. The `$startdir` variable refers to the directory where a `PKGBUILD` is located when you build a package.

```
PKGDEST="$startdir/build/packages/"
SRCDEST="$startdir/build/sources/"
SRCPKGDEST="$startdir/build/srcpackages/"
LOGDEST="$startdir/logs/"
```

This will result in:

- Built packages will be stored in: `"package directory"/build/packages/`
- All downloaded source files will be stored in: `"package directory"/build/sources/`
- Built source packages will be stored in: `"package directory"/build/srcpackages/`
- All logs will be stored in: `"package directory"/logs/`

makepkg will still create `src/` and `pkg/` directories as usual, so this is expected behaviour.

问题处理

Specifying install directory for QMAKE based packages

The makefile generated by *qmake* uses the environment variable `INSTALL_ROOT` to specify where the program should be installed. Thus this package function should work:

```
PKGBUILD
...
package() {
	cd "$srcdir/${pkgname%-git}"
	make INSTALL_ROOT="$pkgdir" install
```

```
}
...
```

Note, that qmake also has to be configured appropriately. For example put this in the corresponding *.pro* file:

```
YourProject.pro
...
target.path = /usr/local/bin
INSTALLS += target
...
```

WARNING: Package contains reference to $srcdir

由于某种原因，有时 `$pkgdir` 或 `$srcdir` 中的字面字符串会进入到软件包中的文件。

可以在 *makepkg* 构建文件夹中执行以下命令进行检测：

```
$ grep -R "$PWD/src" pkg/
```

一个可能是 C/++ 代码使用了 `__FILE__` 宏并将完整路径传递给了编译器。

Dotnet 二进制文件有时也会在默认配置文件中包含 `.pdb` 文件的完整路径。

Makepkg fails to download dependencies when behind proxy

When *makepkg* calls dependencies, it calls pacman to install the packages, which requires administrative privileges via *sudo*. However, *sudo* does not pass any environment variables to the privileged environment, and as well excludes the proxy-related variables ftp_proxy, http_proxy, https_proxy, and no_proxy.

In order to have *makepkg* working behind a proxy, invoke one of the following methods.

Enable proxy by setting its URL in XferCommand

The XferCommand can be set to use the desired proxy URL in `/etc/pacman.conf`. Add or uncomment the following line in `pacman.conf`:

```
/etc/pacman.conf
...
XferCommand = /usr/bin/curl --proxy http://username:password@proxy.proxyhost.com:80 --location --continue-at - --fail --output %o %u
...
```

Enable proxy via sudoer's env_keep

Alternatively, one may want to use sudoer's `env_keep` option, which enables preserving given variables the privileged environment. See Pacman#pacman does not honor proxy settings for more details.

Makepkg fails, but make succeeds

If something successfully compiles using *make*, but fails through *makepkg*, it is almost certainly because `/etc/makepkg.conf` sets an incompatible compilation variable. Try adding these flags to the PKGBUILD `options` array:

`!buildflags`, to prevent its default `CPPFLAGS`, `CFLAGS`, `CXXFLAGS`, and `LDFLAGS`.

`!makeflags`, to prevent its default `MAKEFLAGS`.

`!debug`, to prevent its default `DEBUG_CFLAGS`, and `DEBUG_CXXFLAGS`, in case the PKGBUILD is a debug build.

If any of these fix the problem, this could warrant an upstream bug report assuming the offending flag has been identified.

参阅

- makepkg(8)
- makepkg.conf(5)
- Makepkg 流程概览
- makepkg 源码

密码学

本章只处理两个具体问题：口令的哈希究竟能扛住多少暴力猜测，以及密钥材料应当从哪里取得。建议按此顺序读：Hashcat 一节篇幅极短，讲安装与用法，是两节里唯一的实操入口，必读，读后应能把一份哈希交给它去跑并看懂结果；随机数生成一节较长，讲内核内置的随机数生成器与替代方案，凡需自行产出密钥材料者必读，只用现成工具、不碰密钥材料的人可跳过。最容易踩的坑是把哈希当成可解密的密文：Hashcat 并不还原原文，只是反复猜测再比对，所以跑不出结果只说明这一轮猜得不够，不代表原文受到了更强的保护。

8.1　Hashcat

难度 入门 | **篇幅** 1.3 KB | **所属** 密码学

本节将讲到

- 安装
- 用法
- 参见

Hashcat 是功能强大的密码恢复工具，支持 200 多种 hash 算法。它使用 OpenCL 来增强性能。

安装

安装 hashcat包 包。

如果没有 OpenCL，Hashcat 将无法运行，因此您需要为 CPU 或 GPU 安装 GPGPU#OpenCL 软件包。

用法

对于来自 *hash_file* 与 *hash_type* 获得密码使用 *dictionary_file*：

```
hashcat -m hash_type hash_file dictionary_file
```

在 Hashcat wiki 上可以找到更多示例和用法详细信息。

参见

- 官方网站

8.2　随机数生成

难度 进阶 | **篇幅** 8.4 KB | **所属** 密码学

本节将讲到

- 内核内置随机数生成器
- 替代方案
- 另见

根据wikipedia:Random number generation条目的定义：

: 随机数生成器（random number generator, RNG）是用于生成数字/符号序列的计算设备或物理设备。其生成的序列应当看起来是随机的，不含任何规律性特征。

生成随机数据对某些应用至关重要。随机数据可用于生成密码学密钥（密钥可用于静态数据加密）、安全地擦除磁盘、运行软件接入点等用途。

内核内置随机数生成器

注意：近年来，在4.x和5.x内核版本中，内核内置的随机数生成器有了重大改进。因此，现存的许多信息都已过时。本文介绍的是当前内核版本的实现方式与行为，内容适用于Arch Linux官方支持的内核以及其它较新版本的内核。

Linux内核内置的随机数生成器可生成密码学安全的伪随机数据。其从多种来源搜集熵)，用于随机数生成。熵的来源包括硬件随机数生成器、中断、CPU抖动熵（CPU-based jitterentropy），不依赖单一来源。熵将通过BLAKE2s 密码学哈希函数#BLAKE2)提取，并用作一组ChaCha20密码学随机数生成器（Cryptographic Random Number Generator， CRNG）的种子。最终的随机数据由ChaCha20密码学随机数生成器生成。内核在运行时将不断搜集熵值，并周期性地更新随机数生成器的种子。

若要在用户空间访问内核内置随机数生成器，可使用内核提供的三个接口：

- getrandom(2)系统调用
- `/dev/random`
- `/dev/urandom`

在过去，一般认为`/dev/random`生成的随机数随机性比`/dev/urandom`的好。然而，随着内核的更新，`/dev/random`和`/dev/urandom`的行为越来越相似。甚至，在目前的x86-64系统上，它们的行为是完全相同的。由于Arch Linux只支持x86-64架构，因此可以说在Arch Linux中，两者的行为是相同的。（这是由于所有x86-64 CPU都支持RDTSC 指令，因而总是可以使用CPU抖动熵算法搜集熵值；此外，大多数x86-64 CPU还支持RDRAND指令，可用于随机数生成。）

对于其它架构（特别是不支持快速周期计数器（fast cycle counter）的架构），`/dev/random`与`/dev/urandom`仍有区别：在内核认为密码学随机数生成器完成初始化之前，`/dev/random`将阻塞，而`/dev/urandom`不阻塞。因此，在通常

情况下，应用若要生成长期使用的密码学密钥，仍应当依照传统，使用/dev/random，或直接使用getrandom(2)系统调用（其默认行为与/dev/random相似）。

注意，由于/dev/random只保证生成密码学安全的随机数据（已经能满足所有现实需求），而不再保证生成“真”随机数据，从其中读取数据不再会消耗内核熵池中的熵，/proc/sys/kernel/random/entropy_avail的内容应当始终是256（ChaCha20密钥的比特长度）。因此，若看到认为256“太低”，需要手动提高该值的过时文档，不必采取相应操作。

替代方案

不需要密码学安全随机数的应用程序可以使用非密码学随机数生成器，例如random(3)。

对于需要密码学安全随机数的应用程序，在现在，使用内核内置的随机数生成器即可，一般无需使用其它方式。曾经，内核的随机数生成器生成速度较慢，且没有使用许多本可以使用的熵源。但现在，经过改进，其使用了更多的熵源，在x86-64架构上吞吐量能达到大约400 MB/s。即使在需要非常高吞吐量的场景中（例如，安全地擦除磁盘），也可以直接使用/dev/urandom。

对于如下情况，可能需要使用其它方式，而不是直接使用内核随机数生成器：

- 在某些特定情况下，应用程序需要获取密码学安全随机数，且要求**非常高**的吞吐量/非常低的延迟（以至于不能容忍系统调用的开销）。此时可以使用运行在用户空间密码学安全随机数生成器。要保证安全性，应当使用内核随机数生成器生成的随机数作为用户空间生成器的种子，并考虑种子的更新问题。
- 某些应用的工作环境中已有用户空间密码学安全随机数生成器，且提供了完善的使用该生成器生成随机数的API。例如，若应用程序使用了OpenSSL，则可以使用其RAND_bytes(3)函数；若应用程序使用Java编写，则可以使用C0。一般而言，这些API底层的用户空间密码学安全随机数生成器将自动使用内核随机数生成器生成的随机数作为种子。
- 需要使用内核未包含的额外熵源。例如，Haveged可通过抖动熵（jitterentropy）生成随机数。若可能，不应使用额外的熵源完全替代内核的随机数生成器，而应当与内核随机数生成器共同使用，为用户空间密码学安全随机数生成器提供种子。此外，也可以将高熵数据写入/dev/random，写入的数据最终将影响内核随机数生成器（由于内核密码学安全随机数生成器的更新机

制，可能需要至多60秒才能生效）。需要注意，内核默认已经使用RDSEED等其它硬件随机数源作为熵源。

另见

- Kernel RNG improvements:
- random: replace non-blocking pool with a Chacha20-based CRNG - introduced the ChaCha20 CRNG (2016)
- random: try to actively add entropy rather than passively wait for it - made CPU-based jitterentropy be used as a fallback (2019)
- Removing the Linux /dev/random blocking pool - made `/dev/random` use a CRNG, just like `/dev/urandom`, and not try to provide "true" randomness (2020)
- Random number generator enhancements for Linux 5.17 and 5.18 (2022)
- random: use simpler fast key erasure flow on per-cpu keys - simplified and optimized the CRNG design (2022)
- random: do not pretend to handle premature next security model - made new entropy be used more quickly (2022)
- random: opportunistically initialize on /dev/urandom reads - made `/dev/urandom` as safe as `/dev/random` on x86 (2022)
- Randomness - A popular science article explaining different RNGs
- ENT - A simple program for testing random sequences (entropy, Chi square test, Monte Carlo, correlation, etc.)
- An Analysis of OpenSSL's Random Number Generator - Paper on RNG reseeding risks in OpenSSL functionality

认证

认证要解决的是谁在登录、凭什么被信任的问题，本章从可选的软件方案开始，最终落到一套能真正跑起来的 LDAP 认证。身份管理一节给出的只是软件清单，价值在于横向对比与选型，方案已定的读者可以直接跳过；LDAP 认证一节才是必读的主体，其中服务端配置与客户端配置前后相连，不能拆开看。

最容易踩的坑是把认证当成单侧配置：只把客户端指过去，或只在服务端建好目录数据，任缺一端都登录不上，而报错往往指向缺失的那一侧。建议按先服务端、后客户端的顺序读，参见留到最后需要补充时再查。

9.1　身份管理

难度 入门 | **篇幅** 2.3 KB | **所属** 认证

本节将讲到

- 软件清单

翻译状态：

- 本文（或部分内容）译自 Identity management，最近一次同步于 2025-07-12，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

身份管理（Identity Management，IDM），又称作身份与访问管理（Identity and Access Management，IAM），负责控制用户用户如何获得数字身份及其角色，并有时负责为身份授予权限。

软件清单

有许多软件可以帮助进行身份管理，下列是其中一部分。

- Google 身份验证器，使用一次性密码(OTP)进行两步验证
- Initiative for Open Authentication，对网页 API 调用获取访问令牌的方法进行了标准化。包含了密码方法，但未来会转用 WebAuthn
- kanidm，相关信息可参考此处，是 LDAP 和 Kerberos 的后继者，实现了服务器端的 WebAuthn
- Kerberos, with Active Directory integration, FreeIPA
- LDAP 认证，可将密码存放在中心化服务器上
- PAM Linux 可插拔认证模块（Pluggable Authentication Module），可将 Kerberos，LDAP，kanidm 等接入到 Linux

- 通用第二因素（U2F），被 WebAuthn 替代
- 用户和用户组，可存放在系统上或中心化的 LDAP 和 kanidm 平台上
- WebAuthn 浏览器网页认证 API（Web Authentication API），用于替代密码认证，可向后兼容 U2F

9.2 LDAP 认证

难度 进阶 | **篇幅** 17.1 KB | **所属** 认证

本节将讲到

- 服务端配置
- 客户端配置
- 参见

这是关于如何配置 Arch Linux 安装以针对 LDAP 目录进行身份验证的指南。LDAP 目录服务可以是本地的（安装在同一台计算机上）或网络中的（例如，在需要集中身份验证的实验环境中）。

本指南分为两部分。第一部分介绍如何安装和配置 OpenLDAP 服务端。第二部分是如何在客户端上配置所需的 NSS 和 PAM 模块。如果您只想将 Arch 加入到已存在的 LDAP 服务器进行身份验证，可以跳到第二部分。

服务端配置

安装

服务端实现软件如下：

- **389 Directory Server** — 这是一个高性能的开源企业级LDAP服务器，用于存储和管理身份认证、用户、群组、组织结构等信息。

: https://www.port389.org/ || 389-ds-base包

- **OpenLDAP** — 这是一个开源的轻量级LDAP服务器，用于存储和管理身份验证、用户、组、服务配置等信息。

: https://www.openldap.org/ || openldap包

- ApacheDS——这是Apache软件基金会下的一个开源项目，提供LADP服务，用于存储、管理和组织分布式信息如用户、群组、权限的呢个，支持高度安全性和可扩展性。
- https://directory.apache.org/apacheds/

本文档使用OpenLDAP进行演示，请安装 OpenLDAP 服务器并配置服务器和客户端。完成此操作后，请返回此处。

访问控制

为确保LDAP服务中存储的（已加密）密码不被他人轻易读取，但又允许用户可更改自己的某些属性（例如自己的密码或图片等），需要LDAP配置访问控制策略，请创建临时 LDIF 文件 `allowpwchange.ldif`。

注意：请根据实际情况将 **exmaple** 和 **org** 修改成对应域名。

```
allowpwchange.ldif
dn: olcDatabase={1}mdb,cn=config
changetype: modify
replace: olcAccess
olcAccess: {0}to
attrs=cn,givenName,sn,userPassword,shadowLastChange,mail,loginShell,photo by self write
by anonymous auth by dn.base="cn=Manager,dc=example,dc=org" write by * none
olcAccess: {1}to * by self read by dn.base="cn=Manager,dc=example,dc=org" write by *
read
```

在数据库编号 0 （cn=config） 上导入它：

```
$ slapmodify -n 0 -l allowpwchange.ldif
```

然后重新启动 `slapd.service`。

导入基本组织架构

创建临时文件`base.ldif`

```
base.ldif
# 域
dn: dc=example,dc=org
dc: example
o: Example Organization
objectClass: dcObject
objectClass: organization

# 域管理员账户
dn: cn=Manager,dc=example,dc=org
cn: Manager
description: LDAP administrator
objectClass: organizationalRole
objectClass: top
roleOccupant: dc=example,dc=org

# 用户
dn: ou=People,dc=example,dc=org
ou: People
objectClass: top
objectClass: organizationalUnit

# 用户组
dn: ou=Group,dc=example,dc=org
ou: Group
objectClass: top
objectClass: organizationalUnit
```

导入到域服务中

```
$ ldapadd -D "cn=Manager,dc=example,dc=org" -W -f base.ldif
```

使用下面命令检测是否导入成功

```
$ ldapsearch -x -b 'dc=example,dc=org' '(objectclass=*)'
```

导入用户信息

按照下面模板创建文件`user_joe.ldif`

```
user_joe.ldif
dn: uid=johndoe,ou=People,dc=example,dc=org
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: johndoe
cn: John Doe
sn: Doe
givenName: John
title: Guinea Pig
telephoneNumber: +0 000 000 0000
mobile: +0 000 000 0000
postalAddress: AddressLine1$AddressLine2$AddressLine3
userPassword: {CRYPT}xxxxxxxxxx
labeledURI: https://archlinux.org/
loginShell: /bin/bash
uidNumber: 9999
gidNumber: 9999
homeDirectory: /home/johndoe/
description: This is an example user
```

userPassword 条目中的 `xxxxxxxxxx` 应替换为 `/etc/shadow` 中的值或使用 `slappasswd` 命令。使用下面命令导入用户：

```
$ ldapadd -D "cn=Manager,dc=example,dc=org" -W -f user_joe.ldif
```

添加同名的POISX用户组

```
group_joe.ldif
dn: cn=joe,ou=Group,dc=example,dc=org
objectClass: top
objectClass: posixGroup
cn: joe
gidNumber: 9999
```

提示：还可以使用 PADL 软件的 openldap-migrationtoolsAUR 自动将所有本地帐户（和组等）迁移到 LDAP 服务中。

客户端配置

按照 OpenLDAP 中所述安装 OpenLDAP 客户端。确保可以使用 ldapsearch 查询服务。

根据需求，选择仅在线身份验证或在线和离线身份验证。

在线认证

NSS 配置

NSS 是一种系统工具，它以配置数据库的形式管理不同的源。例如:`/etc/passwd` 是 `passwd` 数据库的文件类型源，用于存储用户帐户。

安装 nss-pam-ldapd包 软件包。

修改/etc/nsswitch.conf，它是 NSS 的主要配置文件。它定义 NSS 哪些源用于哪些系统数据库。我们需要将ldap指令添加到passwd、group和shadow数据库中，请将配置文件修改至如下所示：

```
/etc/nsswitch.conf
passwd: files ldap
group: files ldap
shadow: files ldap
```

修改/etc/nslcd.conf配置文件，并将base和uri修改为LDAP服务对应的值。

```
/etc/nslcd.conf
...
uri ldap://ldap-server.example.org/
base dc=example,dc=org
binddn cn=Manager,dc=example,dc=org
bindpw YourPassword
...
```

重启服务nslcd.service。

若配置正确，在客户端上执行命令:getent passwd将会列出LDAP服务上面的用户。

PAM 配置

简单的PAM配置原则是将pam_ldap.so配置在任何包含pam_unix.so的配置文件中。Arch在使用pambase包后有效地减少了编辑量，有关于PAM的更多信息，请查阅RedHat文档，如果需要可查看nss-pam-ldapd官方文档。

提示：若想防止 UID 与系统上的本地用户发生冲突，需要在pam_ldap.so行的末尾配置包含 minimum_uid=10000 或类似值。并必须确保 LDAP 服务可返回与限制匹配的uidNumber字段。

注意：每个阶段（auth、session、password、account）都构成一个独立的链，其顺序至关重要。带有sufficient的行有时会“短路”，会跳过该阶段的其余内容，因此对于*auth*、*password*和*account*阶段来说，需要在*required*行之前添加*sufficient*行，而在*session*阶段则是放在required行之后；*optional*几乎是放在最后。添加pam_ldap.so行时，请不要无故改变其他行的相对顺序！只需将LDAP插入到现有链中即可。

首先修改/etc/pam.d/system-auth。此文件包含在pam.d的大多数其他文件中，因此在此处的更改可很好地分发配置。当pambase包更新时可能会更改此文件。

在每个阶段的开始位置添加pam_ldap.so，并将其配置为sufficient，但*session*阶段需要配置为optional。

```
/etc/pam.d/system-auth
auth      sufficient pam_ldap.so
auth      required  pam_unix.so     try_first_pass nullok
auth      optional  pam_permit.so
auth      required  pam_env.so

account   sufficient pam_ldap.so
```

```
account    required  pam_unix.so
account    optional  pam_permit.so
account    required  pam_time.so

password  sufficient pam_ldap.so
password  required  pam_unix.so     try_first_pass nullok sha512 shadow
password  optional  pam_permit.so

session    required  pam_limits.so
session    required  pam_unix.so
session    optional  pam_ldap.so
session    optional  pam_permit.so
```

然后以相同的方式修改文件/etc/pam.d/su和/etc/pam.d/su-l，当用户执行命令:su --login时，会用到su-l文件。

在每个阶段开始位置添加pam_ldap.so,但不要放在pam_rootok的前面，并配置为sufficient，而在auth阶段中的pam_unix.so模块后面则需要添加use_first_pass。

```
/etc/pam.d/su
#%PAM-1.0
auth       sufficient    pam_rootok.so
auth       sufficient    pam_ldap.so
# 取消对以下行的注释，以默认信任"wheel"组中的用户。
#auth      sufficient    pam_wheel.so trust use_uid
# 取消对以下行的注释，以必须要求用户位于"wheel"组中。
#auth      required      pam_wheel.so use_uid
auth       required   pam_unix.so use_first_pass
account    sufficient    pam_ldap.so
account    required   pam_unix.so
session    sufficient    pam_ldap.so
session    required   pam_unix.so
```

若要使用户能修改自己密码，需要修改文件/etc/pam.d/passwd

```
/etc/pam.d/passwd
#%PAM-1.0
password        sufficient      pam_ldap.so
password        required        pam_unix.so sha512 shadow nullok
```

登录时创建主文件夹

若希望在登录时自动创建主文件夹（注：不打算使用NFS存储主文件夹情况下），请修改文件/etc/pam.d/system-login并将pam_mkhomedir.so添加到任何session阶段的"sufficient"项目上方。若需从 ssh、xdm、sddm、gdm 等或在 tty 登录时会自动创建主文件夹。请以相同的方式修改其他文件，例如当执行命令:su和su --login时会使用文件/etc/pam.d/su和/etc/pam.d/su-l。如果不想在 ssh 登录是执行此操作，请修改文件system-local-login，而不是文件system-login。

```
/etc/pam.d/system-login
...未显示文件头部...
session    optional   pam_loginuid.so
session    include    system-auth
session    optional   pam_motd.so          motd=/etc/motd
session    optional   pam_mail.so          dir=/var/spool/mail standard quiet
-session   optional   pam_systemd.so
session    required   pam_env.so
session    required   pam_mkhomedir.so skel=/etc/skel umask=0077
/etc/pam.d/su-l
```

```
...未显示文件头部...
session         required        pam_mkhomedir.so skel=/etc/skel umask=0077
session         sufficient      pam_ldap.so
session         required        pam_unix.so
```

启用 sudo

要从 LDAP 用户启用 sudo，请编辑 /etc/pam.d/sudo。并相应地修改 sudoers。

```
/etc/pam.d/sudo
#%PAM-1.0
auth       sufficient    pam_ldap.so
auth       required      pam_unix.so  try_first_pass
auth       required      pam_nologin.so
```

还需要在`/etc/openldap/ldap.conf`中添加以下内容：

```
/etc/openldap/ldap.conf
...
sudoers_base ou=sudoers,dc=example,dc=org
```

用SSSD完成在线和离线认证

SSSD 是一个系统守护程序。它的主要功能是通过一个通用框架提供对身份和身份验证以及远程资源的访问，该框架可以为系统提供缓存和离线支持。它提供PAM和NSS模块，未来将提供基于D-BUS的接口，用于扩展用户信息。它还提供了一个数据库来存储本地用户以及扩展用户数据。

注意：必须为 LDAP 服务器配置TLS加密，否则 SSSD 将不起作用。

安装 sssd包 包

配置sssd

修改sssd配置文件`/etc/sssd/sssd.conf`，如果文件不存在请自行创建。

```
/etc/sssd/sssd.conf
[sssd]
config_file_version = 2
services = nss, pam
domains = LDAP

[domain/LDAP]
cache_credentials = true
enumerate = true

id_provider = ldap
auth_provider = ldap

ldap_uri = ldap://server1.example.org, ldap://server2.example.org
ldap_search_base = dc=example,dc=org
ldap_id_use_start_tls = true
ldap_tls_reqcert = demand
ldap_tls_cacert = /etc/openldap/certs/cacerts.pem
chpass_provider = ldap
ldap_chpass_uri = ldap://server1.example.org
entry_cache_timeout = 600
ldap_network_timeout = 2

# 若 OpenLDAP 支持 posixGroup，可取消注释以下两行
# 获取用户组成员身份支持（需要注释下面冲突配置）
#ldap_schema = rfc2307
#ldap_group_member = memberUid
```

```
# 其他 LDAP 服务可能支持的组成员身份功能
ldap_schema = rfc2307bis
ldap_group_member = uniqueMember
```

以上只是一个示例。参见sssd.conf(5) 了解完整的细节。

最后将文件权限`chmod 600 /etc/sssd/sssd.conf`，否则 sssd 将无法启动。

配置 NSS

修改`/etc/nsswitch.conf`配置文件如下：

```
/etc/nsswitch.conf
passwd: files systemd sss
group: files [SUCCESS=merge] systemd sss
shadow: files systemd sss
gshadow: files systemd
sudoers: files sss

publickey: files

hosts: mymachines resolve [!UNAVAIL=return] files myhostname dns
networks: files

protocols: files
services: files
ethers: files
rpc: files

netgroup: files
```

配置 PAM

首先配置系统认证`/etc/pam.d/system-auth`，所有与系统相关的流程都会经过当前文件,每当用户登录时，使用sssd模块进行认证。

```
/etc/pam.d/system-auth
#%PAM-1.0

auth      sufficient     pam_sss.so forward_pass
auth      required         pam_unix.so try_first_pass nullok
auth      optional         pam_permit.so
auth      required         pam_env.so

account   [default=bad success=ok user_unknown=ignore authinfo_unavail=ignore]
pam_sss.so
account   required         pam_unix.so
account   optional         pam_permit.so
account   required         pam_time.so

password  sufficient     pam_sss.so
password  required         pam_unix.so try_first_pass nullok sha512 shadow
password  optional         pam_permit.so

session   required       pam_mkhomedir.so skel=/etc/skel/ umask=0077
session   required         pam_limits.so
session   required         pam_unix.so
session   optional         pam_sss.so
session   optional         pam_permit.so
```

注意：如果在用 GNOME/Keyring，当在 PAM 文件的开头添加 'sufficient' 会导致 Gnome Keyring 无法解锁。有关解决方案，请查看高级 PAM 配置。

若向要使用su命令时通过sssd进行认证，则请按照下面示例修改文件/etc/pam.d/su。

```
/etc/pam.d/su
#%PAM-1.0
auth            sufficient      pam_rootok.so
auth           sufficient      pam_sss.so      forward_pass
auth            required        pam_unix.so
account        [default=bad success=ok user_unknown=ignore authinfo_unavail=ignore]
pam_sss.so
account         required        pam_unix.so
session         required        pam_unix.so
session        optional        pam_sss.so
```

启用 sudo

按照下面示例修改文件/etc/pam.d/sudo

```
/etc/pam.d/sudo
#%PAM-1.0
auth           sufficient     pam_sss.so
auth           required        pam_unix.so try_first_pass
auth           required        pam_nologin.so
```

配置/etc/sssd/sssd.conf文件，添加sudo服务和sudo用户搜索路径。

```
/etc/sssd/sssd.conf
[sssd]
...
services = nss, pam, sudo
...

[domain/LDAP]
...
ldap_sudo_search_base = ou=sudoers,dc=example,dc=org
...
```

或者配置 sudo 以允许所需的 LDAP 用户使用 sudo。

密码管理

若要在使用su登录时能修改用户过期密码，请参照下面示例修改文件/etc/pam.d/su。

```
/etc/pam.d/su
#%PAM-1.0
...
auth        include     system-auth
account     include     system-auth
session     include     system-auth
password    include    system-auth
```

重启sshd.service服务。

若配置正确，使用命令getent passwd *LDAP用户名*或id *LDAP用户名*能查看到用户信息。

使用用户登录后，凭据将被缓存，当 ldap 服务器脱机或不可用时，仍然能够使用缓存的凭据进行登录。

参见

1. ↑ Name Service Switch (NSS) 是类Unix系统中用于配置如何查找用户和组信息的服务。它通过 /etc/nsswitch.conf 配置文件定义了系统应如何查询各种类型的信息，例如用户账户、组账户、主机名解析等。 | 博客园:Linux NSS简介

访问控制

属主、属组、其他三组身份分不开权限时，本章补上两层：访问控制列表把文件权限细化到具体用户与具体操作，AppArmor 用规则限定程序能访问的路径与能力。

建议按顺序读：访问控制列表篇幅短，安装与启用后即可上手；AppArmor 是重点，使用方法与配置必读，缓存与排错留到出问题时再翻。最容易踩的坑是把两者当成同一层——文件权限再细，也管不住已被攻破进程到手的权限；程序规则再严，也不会替你把文件权限改对。另一个误解是装完即生效，ACL 不启用、配置不加载，看着齐全却什么都没拦。

10.1　访问控制列表

难度 进阶 | **篇幅** 7.2 KB | **所属** 访问控制

本节将讲到

- 安装
- 启用 ACL
- 使用
- 例子
- 参阅

访问控制列表 (ACL) 为文件系统提供了一种额外的、更加灵活的权限机制，用于完善 UNIX 的文件权限机制。ACL 允许您将任意磁盘资源授权给任意用户组或用户。

安装

acl包 是 systemd 的依赖之一，应该已经安装好了。

启用 ACL

要启用 ACL，文件系统在挂载时必须要加入 `acl` 参数。要开机时自动启用 ACL，你可以通过编辑 fstab 文件来实现。

默认情况下 Btrfs、Ext2/3/4 已经启用了 `acl` 参数，因此无需额外指定。对于 Ext* 文件系统你可以使用以下命令检查 `acl` 是否真的已经启用：

```
# tune2fs -l /dev/sdXY | grep "Default mount options:"
Default mount options:    user_xattr acl
```

还需要检查 `/proc/mounts` 文件中是否存在 `noacl` 。以防止默认挂载选项被覆盖。

你可以通过 `tune2fs -o *option* *partition*` 命令设置文件系统的默认挂载选项。例如：

```
# tune2fs -o acl /dev/sdXY
```

这将导致此磁盘以后在被其他 Linux 挂载时依然会使用相同的挂载选项。

注意：

- `acl` 选项在创建 ext2/3/4 文件系统时将被默认启用，该行为由 `/etc/mke2fs.conf` 控制。
- `/proc/mounts` 文件中不会列出默认挂载选项。

使用

设置 ACL

ACL 可以通过 *setfacl* 命令被设置。

提示：

- 你可以通过添加 `--test` 参数来测试该命令的结果
- 要递归地将操作应用到所有的文件和目录上，使用 `-R`/`--recursive` 参数。

为用户设定权限(`*user*` 是用户名或 UID)：

```
# setfacl -m "u:user:permissions" <file/dir>
```

为用户组设定权限 (`*group*`是用户组名或 GID)：

```
# setfacl -m "g:group:permissions" <file/dir>
```

为其他用户设定权限：

```
# setfacl -m "other:permissions" <file/dir>
```

允许所有*新创建的文件*或*目录*从父目录继承条目(这不会影响将*复制*到目录中的文件)：

```
# setfacl -dm "条目" <目录>
```

删除权限：

```
# setfacl -x "用户/用户组" <文件/目录>
```

删除默认权限

```
# setfacl -k <文件/目录>
```

删除所有ACL权限（拥有者、用户组和其他用户的权限将被保留）

```
# setfacl -b <file/dir>
```

**本文或本章节的事实准确性存在争议。**

原因： The original note about the `--mask` option (which was taken from setfacl(1)) was determined as inaccurate, but the new note does not seem correct either. See the talk page for details.（在 Talk:访问控制列表#ACL mask entry 中讨论）

注意：The default behavior of *setfacl* is to recalculate the ACL mask entry, unless a `--mask` entry was explicitly given. The mask entry indicates the maximum permissions allowed for users (other than the owner) and for groups. Unless explicitly set, this will match the permissions of the default group. To clarify what this means, suppose the group owning a directory has `r-x` permissions. If you add an ACL user or group with `rwx` permissions, the *effective* permissions of this user or group will be `r-x`. The

reason for this is so that there are no surprises when a file from a system which does not support ACLs is made available on a system which does..

查看 ACL

要显示 ACL 权限，使用:

```
# getfacl <file/dir>
```

例子

给予 `johnny` 对 `abc` 文件的所有访问权：

```
# setfacl -m "u:johnny:rwx" abc
```

检查权限：

```
# getfacl abc
# file: abc
# owner: someone
# group: someone
user::rw-
user:johnny:rwx
group::r--
mask::rwx
other::r--
```

改变 `johnny` 的权限：

```
# setfacl -m "u:johnny:r-x" abc
```

检查权限：

```
# getfacl abc
# file: abc
# owner: someone
# group: someone
user::rw-
user:johnny:r-x
group::r--
mask::r-x
other::r--
```

删除所有已添加的 ACL 权限信息：

```
# setfacl -b abc
```

检查权限：

```
# getfacl abc
# file: abc
# owner: someone
# group: someone
user::rw-
group::r--
other::r--
```

ACL 对 ls 命令的影响

当使用 `ls -l` 命令时，含有ACL权限的文件将会显示一个 `**+**`

```
$ ls -l /dev/audio
crw-rw----+ 1 root audio 14, 4 nov.   9 12:49 /dev/audio
$ getfacl /dev/audio
getfacl: Removing leading '/' from absolute path names
# file: dev/audio
# owner: root
# group: audio
```

```
user::rw-
user:solstice:rw-
group::rw-
mask::rw-
other::---
```

私有文件的执行权限

下面的例子演示了如何对 WEB 服务器这样的进程赋予其访问用户目录的权限，以便防止其越权处理文件。

假设运行该 WEB 服务器进程的用户为 `http`，授予其访问 `/home/geoffrey` 的权限。

首先授予 `http` 执行权限：

```
# setfacl -m "u:http:--x" /home/geoffrey
```

注意：要获取目录下的文件列表，必须要对目录拥有执行权

由于 `http` 现在已经对 `/home/geoffrey` 拥有执行权，下面移除 other 对 `/home/geoffrey` 的权限：

```
# chmod o-rx /home/geoffrey
```

使用`getfacl` 查看更改：

```
$ getfacl /home/geoffrey
getfacl: Removing leading '/' from absolute path names
# file: home/geoffrey
# owner: geoffrey
# group: geoffrey
user::rwx
user:http:--x
group::r-x
mask::r-x
other::---
```

正如上面的输出所示，`other` 不再具有任何权限，但是用户 `http` 仍然能够访问文件，因此可以认为安全性有所提高。

注意：假设需要为用户`http` 授予特定目录或文件的写访问权：

```
# setfacl -dm "u:http:rwx" /home/geoffrey/project1/cache
```

参阅

- getfacl(1)
- setfacl(1)
- POSIX Access Control Lists on Linux
- How to set default file permissions for all folders/files in a directory?

10.2 AppArmor

难度 进阶 | **篇幅** 18.3 KB | **所属** 访问控制

本节将讲到

- 安装

- 使用方法
- 配置
- 提示与技巧
- Turn creating/updating of the cache on by default

翻译状态：

- 本文（或部分内容）译自 AppArmor，最近一次同步于 2026-06-27，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

AppArmor 是一种强制访问控制（Mandatory Access Control, MAC）系统，基于 Linux 安全模块（Linux Security Modules, LSM）实现。

就像其他大多数的 Linux 安全模块一样，AppArmor 是对默认的的自主访问控制（Discretionary Access Control, DAC）的补充，而非替代。因此，程序通过 AppArmor 获取到的权限不可能大于原本在 DAC 下拥有的权限。

Ubuntu 和许多其他的发行版默认使用 AppArmor，而 RHEL（及其衍生发行版）和 SUSE 所使用的 SELinux 需要特定的用户空间工具才能正常使用。SELinux 通过附加标签的方式管理所有的文件、进程和对象，因而十分灵活。不过，普遍认为 SELinux 难以配置，且需要支持扩展属性的的文件系统才能运作。相反，AppArmor 基于文件路径，配置文件也相对简单易懂。

AppArmor 通过对特定应用执定专用的规则集，主动保护操作系统和应用程序免受来自内外部的威胁，包括部分零日漏洞攻击。安全策略决定了每个应用所能够访问的系统资源及其权限。没有在安全配置文件中允许的访问，将被默认拦截。AppArmor 随附了一些默认策略，再结合使用高级静态分析与基于学习的工具，即使是极其复杂的应用程序，用户也可以在数小时内完成其 AppArmor 策略的编写。

任何违反策略的访问都将在系统日志中报告。也可以通过配置 AppArmor，令其在越权访问发生时，在用户桌面实时弹出警告通知。

安装

AppArmor 可以在所有的官方支持的内核中使用。

安装 apparmor包 以获取管理 AppArmor 的用户空间工具与库。要在启动时加载所有 AppArmor 安全配置文件，启用 `apparmor.service`。

要在每次引导时将 AppArmor 作为默认的安全模型启用，设置以下的内核参数：

```
lsm=landlock,lockdown,yama,integrity,apparmor,bpf
```

注意：`lsm=` 内核参数决定了 Linux 安全模块的初始化顺序。内核已配置的 `lsm=` 值可以通过 `zgrep CONFIG_LSM= /proc/config.gz` 查看，当前值可以通过 `cat /sys/kernel/security/lsm` 查看。

- 确保 `apparmor` 是参数列表中的第一个“主要”模块。合法的值和顺序的样例可以在 security/Kconfig 查看。
- `lsm=` 中不应包含 `capability`，因为它始终会被自动隐式包含。

自定义内核

自行编译内核时，需要启用以下选项：

```
CONFIG_SECURITY_APPARMOR=y
CONFIG_AUDIT=y
```

要让内核在无需设置内核参数的前提下默认启用 AppArmor Linux 安全模型，需要额外添加 `CONFIG_LSM` 选项，并将 `apparmor` 设定为列表中的第一个“主要”模块。

```
CONFIG_LSM="landlock,lockdown,yama,integrity,apparmor,bpf"
```

使用方法

显示当前状态

检测 AppArmor 是否成功启动：

```
$ aa-enabled
Yes
```

显示当前运行状态，执行 aa-status(8)：

```
# aa-status
apparmor module is loaded.
44 profiles are loaded.
44 profiles are in enforce mode.
 ...
0 profiles are in complain mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
```

在 *complain* 模式下，违反策略的访问将被允许，但该行为会被记录在日志中。*complain* 模式常用于了解程序正常运行所需要的权限，或是测试新撰写的配置文件是否合适。但是，需要注意的是，在配置文件中被设定为 *deny* 的规则在 *complain* 模式下仍然会强制执行。

在 *enforce* 模式下，违反策略的访问将被拒绝，并记录到日志中。

解析配置文件

使用 `apparmor_parser` 以加载、卸载、重启、缓存或是统计配置文件。默认行为（`-a`）是以 *enforce* 模式加载配置文件。要以 *complain* 模式加载，请使用 `-C` 参数。要覆盖已有配置文件，使用 `-r` 参数。要删除配置文件，使用 `-R` 参数。每次操作也可能应用到多个配置文件上。参见 apparmor_parser(8) 手册页面以了解详情。

停用

要在当前会话临时停用 AppArmor，执行以下命令以停用所有配置文件：

```
# aa-teardown
```

要永久停用 AppArmor 配置文件，请停用 `apparmor.service`。要阻止内核加载 AppArmor，添加`apparmor=0`到 内核参数、或者删除在安装 AppArmor 时添加的 `lsm=` 内核参数。

配置

审计与生成配置文件

要创建新的配置文件，确保 Audit 框架（英语：Audit framework）正在运行，因为 Arch Linux 使用 systemd，内核日志默认不会保存到文件中。AppArmor 可以从用户空间的 auditd 进程抓取内核日志，从而协助编写配置文件。

注意：AppArmor audit 消息使用 AVC 记录格式，使用 ausearch 搜索时可能破坏 audit 日志的解析。在此查看漏洞报告：，，

可以使用 aa-genprof(8) 或 aa-autodep(8) 创建新的 AppArmor 配置文件。新创建的配置文件默认处于 *complain* 模式：此模式下，违反规则的行为仅会被记录，不会被拒绝。要交互式地创建配置文件，可以使用 apparmor包 包中的 aa-logprof(8)。配置完成后，请使用 aa-enforce(8) 将配置文件设置为 *enforce* 模式。此模式下，配置文件中的规则将被严格执行。此外，可以重复执行 aa-logprof(8) 以添加额外的规则，也可以使用 aa-complain(8) 将配置文件重新设置为 *complain* 模式。可以在此找到更为详细的教程：AppArmor wiki - Profiling with tools。

需要注意的是，aa-logprof(8) 也提供 *deny* 规则。不过根据 AppArmor 的设计逻辑，除非规则明确允许，否则任何行为都将被拒绝，因此严格来说没有必要使用 *deny* 规则。不过，*deny* 规则有以下两个用途：

1. *deny* 规则的优先级比 *allow* 规则更高，
   在 `/etc/apparmor.d/abstractions` 中的许多 abstraction 中都有使用，用来阻断任何对重要文件（夹）的访问。这样做能防止创建 *allow* 规则时可能的疏忽，使得配置文件不会太宽松。
2. *deny* 规则执行时不会在日志中记录，使得后续执行 *aa-logprof* 时产生更为简洁的日志。需要注意的是，即使是在 *complain* 模式下，*deny* 规则也会被严格执行——因此，如果程序在 *complain* 模式下还是会出错，应该检查在某一配置文件或其包含的 abstraction 中，是否有一条 *deny* 规则导致了此问题。

另外，也可以手动编写配置文件， 在此查看教程：AppArmor wiki - Profiling by hand。

除了 /etc/apparmor.d/ 中的默认配置文件外，
在 /usr/share/apparmor/extra-profiles/ 中还有更多预定义的配置文件。不过，这些配置文件并非一定适用于生产环境，因此可能需要手动修改或使用 aa-logprof(8)。

也可以在 apparmor.d 项目处找到额外的 AppArmor 规则集。不过，直到撰写本文时，该项目并非稳定可用。

解读配置文件

配置文件是可读性很高的文本文件，保存在 /etc/apparmor.d/ 下，定义了某个可执行文件的权限。以下是一个典型的配置文件：

```
/etc/apparmor.d/usr.bin.test
#include <tunables/global>

profile test /usr/lib/test/test_binary {
    #include <abstractions/base>

    # Main libraries and plugins
    /usr/share/TEST/** r,
    /usr/lib/TEST/** rm,

    # Configuration files and logs
    @{HOME}/.config/ r,
    @{HOME}/.config/TEST/** rw,
}
```

以 @ 符号开头的字符串是在
abstraction（/etc/apparmor.d/abstractions/），
tunables（/etc/apparmor.d/tunables/或配置文件本身中定义的变量。
#include 将其他配置文件中的内容包含到此配置文件中来。路径后的字符是访问权限。使用 AppArmor 的通配符语法以实现模式匹配。

以下权限涵盖了最常见的使用场景：

- r——读取：读取数据
- w——写入：创建，删除，写入文件，或是附加内容到文件尾部
- m——映射：将文件映射为可执行内存
- x——执行：执行文件。需要在前面加上修饰符，形如 px（执行时切换配置文件）

注意，定义的权限始终无法使程序获得大于原本在 DAC 中拥有的权限。

此处仅提到了很浅显的内容。要查询更为详细的文档，请查看 apparmor.d(5) 手册页和官方文档。

提示与技巧

拒绝访问后发出桌面通知

当 AppArmor 拒绝了程序的访问后，通知进程可以显示桌面通知。通过以下步骤，可以在登录时自动启动 *aa-notify* 进程：

安装 Audit framework 并且启动/启用用户空间的Linux Audit进程。之后将你的桌面用户添加到 `audit` 用户组，从而允许其读取审计日志（audit logs）：

```
# groupadd -r audit
# gpasswd -a user audit
```

将 `audit` 用户组添加到 `auditd.conf`：

```
/etc/audit/auditd.conf
log_group = audit
```

注意：自 audit包 4.1.2-1 版本起，Arch Linux 有了一组 `tmpfiles.d` 配置文件，其中包含以下行:

```
/usr/lib/tmpfiles.d/audit.conf
[...]
d /var/log/audit 700 root root - -
[...]
z /var/log/audit 700 root root - -
```

这会让仅属于 `audit` 用户组的用户，在 `systemd-tmpfiles` 被调用后——例如使用pacman升级系统时，将不能读取目录 `/var/log/audit`。你可通过创建以下文件覆盖此行为：

```
/etc/tmpfiles.d/audit.conf
z /var/log/audit 750 root audit - -
```

提示：也可以使用已有的系统用户组，例如 `wheel` 或 `adm`。

安装 python-notify2包 、 python-psutil包 和 tk包

创建一个桌面配置项并填入以下内容：

```
~/.config/autostart/apparmor-notify.desktop
[Desktop Entry]
Type=Application
Name=AppArmor Notify
Comment=Receive on screen notifications of AppArmor denials
TryExec=aa-notify
Exec=aa-notify -p -s 1 -w 60 -f /var/log/audit/audit.log
StartupNotify=false
NoDisplay=true
```

重启并检查进程 `aa-notify` 是否运行：

```
$ pgrep -ax aa-notify
```

注意：在一些特定的系统配置下，可能会有**海量**的消息弹出。

要查询更多信息，参见 aa-notify(8)。

缓存配置文件以提高 AppArmor 启动速度

启动时，AppArmor 需要将配置文件转换为二进制格式，这一步可能会显著延长启动时间。可以用以下命令检查当前 AppArmor 的启动耗时：

```
$ systemd-analyze blame | grep apparmor
```

要启用 AppArmor 配置文件缓存功能，取消注释以下内容：

```
/etc/apparmor/parser.conf
## Turn creating/updating of the cache on by default
write-cache
```

要改变默认的缓存存储位置，添加以下内容：

```
/etc/apparmor/parser.conf
cache-loc=/path/to/location
```

注意：自版本 2.13.1 起，默认的缓存存储位置已经由 `/etc/apparmor.d/cache.d/` 变为 `/var/cache/apparmor/`。

重启并再次检查 AppArmor 启动耗时，观察有无改善：

```
$ systemd-analyze blame | grep apparmor
```

排错

无法启动 Samba SMB/CIFS 服务

参见 Samba#AppArmor_权限问题。

升级到 AppArmor v4 后无法登录

在非常罕见的情况下，升级到 AppArmor v4 会导致无法登录到任何账户。

系统日志中可能包含类似如下内容的错误：

```
unix_chkpwd: check pass; user unknown
unix_chkpwd: password check failed for user (john)
gdm-password]: pam_unix(gdm-password:auth): authentication failure; logname= uid=0
euid=0 tty=/dev/tty1 ruser= rhost=  user=john
kernel: audit: type=1400 audit(1730844640.468:171): apparmor="DENIED"
operation="capable" class="cap" profile="unix-chkpwd" pid=1612 comm="unix_chkpwd"
capability=2  capname="dac_read_search"
kernel: audit: type=1400 audit(1730844640.468:172): apparmor="DENIED"
operation="capable" class="cap" profile="unix-chkpwd" pid=1612 comm="unix_chkpwd"
capability=1  capname="dac_override"
```

这可能是由于 `root` 账户无法读取 `/etc/shadow` 和/或 `/etc/gshadow` 导致的（例如，这些文件的权限位（Permission bits）尚未设定）。因此，可以试试以下解决办法：

1. 重启，然后停用 AppArmor（在启动时编辑 Bootloader 参数，或者使用尚未启用 AppArmor 的 fallback 启动项）。
2. 以 `root` 身份登录，然后设定正确的文件权限：`chmod 600 /etc/shadow /etc/gshadow`
3. 再次重启。

不同 Linux 发行版之间的区别

大多数现有的资料是为运行在 Ubuntu 上的 AppArmor 编写的，而这些并不完全适用于 Arch Linux，因为 Ubuntu 为 AppArmor 应用了许多内核补丁。其他的发行版也可能使用自己的内核补丁，但 Arch Linux 使用的是尽可能接近主线的内核。

例如，apparmor.d(5) 手册早在 v6.17 主线内核支持的前几年就已经记载了 `dbus` 规则。但是，实际的支持程度也受到 D-Bus 实现方式 的影响：dbus包 软件包在构建时未启用 AppArmor 支持，而该支持是应用 dbus AppArmor 规则所必需的。

Ubuntu 所使用的 AppArmor 特定内核补丁也可以在此处找到（将下文中的 `*jammy*` 替换为指定的 Ubuntu 版本代号）：

```
https://git.launchpad.net/~ubuntu-kernel/ubuntu/+source/linux/+git/jammy/log/?qt=grep&q=UBUNTU%3A+SAUCE%3A+apparmor
```

受用户空间工具支持的 ABI 版本也可以在 `/etc/apparmor.d/abi/` 处找到。当前运行的内核所支持的 ABI 可以使用以下命令查看：

```
$ aa-features-abi --extract
```

或者

```
$ ls /sys/kernel/security/apparmor/features/
```

参见

- Wikipedia:AppArmor
- AppArmor wiki
- AppArmor Core Policy Reference——Detailed description of available options in a profile
- Ubuntu Wiki——Basic command overview
- AppArmor Versions——Version overview and links to the respective release notes
- Kernel Interfaces——Low level interfaces to the AppArmor kernel module
- wikipedia:Linux Security Modules——Linux kernel module on which basis AppArmor is build upon
- AppArmor in openSUSE Security Guide

Smartcards

本章要解决的是把 YubiKey 这类智能卡硬件真正接入 Arch 并让它按预期工作的问题，内容围绕安装、输入、输出、USB 连接模式、一次性密码与 OATH 展开。全章只有一节，没有可跳的分支：安装、输入输出与 USB 连接模式是前提，必须按顺序读完；一次性密码与 OATH 属具体应用形态，用不上可以先略过，需要时再补。

最容易踩的坑是把 USB 连接模式当成单纯的插拔问题，模式没对，后面的输入输出与一次性密码都无从谈起；另一处误解是以为装上软件就算完成，实际结果要按输出侧的真实反馈验证。

11.1　YubiKey

难度 高级 | **篇幅** 37.0 KB | **所属** Smartcards

本节将讲到

- 安装
- 输入
- 输出
- USB 连接模式
- 一次性密码

翻译状态：

- 本文（或部分内容）译自 YubiKey，最近一次同步于 2025-02-01，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

YubiKey 是一个小型 USB 安全令牌。取决于型号不同，它可以：

- 作为智能卡使用（通过 CCID 协议）- 可以存储 PGP 和 PIV 私钥
- 处理通用第二因素（U2F）请求
- 存储和读取约 30 条开放身份验证（OATH）凭证
- 以 Yubico OTP 或 HMAC-SHA1 模式处理质询-应答请求
- 生成一次性密码（OTP）- 基于 AES 的 Yubico 标准.
- “输入”最长 63 个字符的固定密码

虽然功能丰富，但新版 YubiKey 并不开源。其它替代品有 Solo，TKey 和 Nitrokey。

安装

管理工具

- **YubiKey Manager** — 通过 USB 配置并读取 YubiKey 的 Python 库和命令行工具（`ykman`）。带有可选图形界面。

: https://developers.yubico.com/yubikey-manager/ || yubikey-manager包，yubikey-manager-qt包

: > **注意**：安装后，启用 `pcscd.service`。 : > **警告**：在新的 Arch/python/yubikey-manager 版本上，yubikey-manager-qt包 已经完全无法使用，且 Yubico 已废弃该工具。

- **YubiKey Personalization** — 通过 OTP USB 配置并读取 YubiKey 的库和工具。比 ykman 更强，但更难用，带有可选图形界面。

: https://developers.yubico.com/yubikey-personalization/ || yubikey-personalization包，yubikey-personalization-gui包

认证工具

- **Yubico PAM** — 使用 Yubico OTP 或质询-应答进行 PAM 用户认证。

: https://developers.yubico.com/yubico-pam/ || yubico-pam包

- **Yubico PAM-U2F** — 使用 U2F 进行 PAM 用户认证。

: https://developers.yubico.com/pam-u2f/ || pam-u2f包

- **Yubico Authenticator for Desktop** — 通过 USB 从 YubiKey 读取 OATH 代码的图形工具。支持新的 OATH 实现（例如 YubiKey NEO 和 4）和旧版基于槽位的实现（例如 YubiKey Standard 和 Edge）。注意：archlinux/packaging/packages/yubioath-desktop#2

: https://developers.yubico.com/OATH/YubiKey_OATH_software.html || yubioath-desktop包

- **Yubico Authenticator 6.0+ for Desktop** — Yubico Authenticator 6.0+（6.0 及更高版本）是用于管理 YubiKey 双因素凭证的工具。从 6.0 版本开始，其使用 Flutter 框架完全重写。

: https://developers.yubico.com/yubioath-flutter/ || yubico-authenticator-binAUR

- **libfido2** — 客户端侧 U2F 支持，使浏览器可以通过 U2F 协议使用 YubiKey 进行认证。

: https://developers.yubico.com/libfido2/ || libfido2包

- **YubiKey 全盘加密** — 使用质询-响应模式创建强 LUKS 密码。支持全盘加密。

: https://github.com/agherzan/yubikey-full-disk-encryption || yubikey-full-disk-encryption包

输入

YubiKey 接受通过 USB 传入的 API 调用和按钮输入。

YubiKey 的按钮非常灵敏。根据场景不同，触碰按钮会出现以下效果之一：

- 触发固定密码输入或一次性密码（OTP）（短按是槽 1，长按是槽 2）。这是默认行为，很容易会不小心触发。
- 确认 / 允许功能或访问，LED 会在这时发光提示用户。
- 插入 / 弹出智能卡

输出

YubiKey 会将这些输入转换为输出：

- 按键输入（模拟为 USB 键盘），用于输入固定密码和 OTP。（注意，固定密码无法防范键盘窃听器。）
- 内置 LED：
- 插入时闪烁，方便排障
- 在需要触碰按钮对 API 响应授权时闪烁
- 通过 USB 发送 API 响应，用于：
- 质询-响应请求（使用 Yubico OTP 或 HMAC-SHA1 模式进行计算）
- U2F 质询-响应请求
- CCID 智能卡相关请求

USB 连接模式

取决于 YubiKey 型号不同，设备提供了最多三种不同的 USB 界面。其中两种实现了 USB HID（人机交互设备）类，第三种是智能卡接口（CCID）。三种接口可以分别被启用或禁用，以控制对应协议。

以下表格展示了接口对应的协议：

协议	接口
OTP	键盘 HID
FIDO	其它 HID
PIV	CCID
OpenPGP	CCID
OATH	CCID

`ykman` 使用了“模式”一词，将其命名为 OTP、FIDO 和 CCID。

注意：ykman 从 0.6.1（2018-04-16）版本开始将“U2F”改名为“FIDO”。
https://developers.yubico.com/yubikey-manager/Release_Notes.html

获取已启用的模式

对于版本 5 前的 YubiKey：

```
$ ykman config mode
Current connection mode is: OTP+FIDO+CCID
```

注意：`ykman mode` 命令已被废弃，后续可能会被移除。

对于 YubiKey 5：

```
$ ykman info
Device type: YubiKey 5 NFC
Serial number: XXXXXXXXX
Firmware version: 5.4.3
Form factor: Keychain (USB-A)
Enabled USB interfaces: OTP, FIDO, CCID
NFC transport is enabled.

Applications    USB     NFC
FIDO2           Enabled Enabled
OTP             Enabled Enabled
FIDO U2F        Enabled Enabled
OATH            Enabled Enabled
YubiHSM Auth    Enabled Enabled
OpenPGP         Enabled Enabled
PIV             Enabled Enabled
```

配置模式

出厂时所有模式都会被启用。要修改配置，请使用如下命令：

```
$ ykman mode [OPTIONS] MODE
```

- *`MODE`* 可以是字符串（如 `OTP+FIDO+CCID`）或是简写形式（如 `o+f+c`）。
- *`MODE`* 也可以是模式编号，编码了数种启用的模式。

以下为模式编号表：

<table class="wikitable"><tbody><tr><td>0</td><td>仅作为 OTP 设备
</td></tr><tr><td>1</td><td>仅作为 CCID 设备
</td></tr><tr><td>2</td><td>OTP/CCID 复合设备</td></tr><tr><td>3</td><td>仅作为 U2F 设备</td></tr><tr><td>4</td><td>OTP/U2F 复合设备
</td></tr><tr><td>5</td><td>U2F/CCID 复合设备
</td></tr><tr><td>6</td><td>OTP/U2F/CCID 复合设备
</td></tr><tr><td>7</td><td>仅作为 OATH 设备
</td></tr><tr><td>8</td><td>仅作为 YubiHSM Auth 设备
</td></tr><tr><td>9</td><td>OTP/CCID 复合设备
</td></tr><tr><td>10</td><td>FIDO/CCID 复合设备
</td></tr><tr><td>11</td><td>OTP/FIDO/CCID 复合设备
</td></tr></tbody></table>

</td></tr><tr><td>81</td><td>仅作为 CCID 设备，附带触摸弹出功能
</td></tr></tbody></table>

注意：有些范例使用了无效的模式编号 86。80 会被忽略掉，并作为编号 6 使用。

选项：

- `--touch-eject` - 按钮将用于插入和弹出智能卡，只在仅 CCID 模式可用，需要禁用 FIDO 和 OTP
- `--autoeject-timeout *SECONDS*` - 一定时间后自动弹出智能卡，限制与 `--touch-eject` 相同
- `--chalresp-timeout *SECONDS*` - 设置质询-响应超时时间

更多信息请参考 `ykman mode --help`。

一次性密码

该功能的名称有些误导，因为它还包括静态密码和质询-响应功能。

该功能有两个槽位，可以分别通过短按和长按进行访问。两者都可以被配置为以下功能之一：

- Yubico OTP
- OATH-HOTP
- OATH-TOTP
- 质询-响应
- 固定密码

每个功能在创建时都有多个配置选项，但在创建完成后无法被再次读取。使用 `ykman otp swap` 可以互换槽 1 和槽 2.

出厂配置

对于新出厂的 YubiKey，槽 1 默认配置为 Yubico OTP。该初始 AES 对称密钥同时被存储在 YubiKey 和 Yubico 认证服务器上，这样就可以通过 YubiCloud 进行验证，以在 Yubico 论坛或 https://demo.yubico.com 等使用 Yubico OTP 进行认证。

警告：如果你覆写掉了槽 1 的出厂密钥，就无法再次创建同等信任度的新密钥。出厂生成的 Yubico OTP 凭证以 `cc` 开头，用户生成的凭证以 `vv` 开头。两者在安全和功能性上没有区别，但有的服务只信任 `cc` 开头的凭证。更多信息请参考该论坛帖子。

Yubico OTP

Yubico OTP 基于对称密钥加密。具体来说，每个 YubiKey 都包含了该设备独有的 AES 密钥，并同时存储在验证服务器上。当被要求输入密码时，YubiKey 将使用如密钥 ID、计数器和随机数等不同字段创建一个令牌，并对结果进行加密。

然后该 OTP 将被发送到目标系统，之后传到验证服务器上。接着，验证服务器（其同时拥有私钥）会将其解密并验证内容，之后将结果发送到目标系统上，由目标系统决定是否授权。

YubiCloud 和验证服务器

Yubico 提供了被称为 YubiCloud 的验证服务器，提供了免费无限制访问。YubiCloud 记录了所有 YubiKey 的出厂配置，且是 yubico-pam包 等的“默认”验证服务。Yubico 还提供了服务器的开源实现。

注意：要验证 Yubico 验证服务器，可以使用：

- **HMAC**：通过 https://upgrade.yubico.com/getapikey/ 获取一个 HMAC 密钥和 ID
- **HTTPS**：验证服务器的证书由 GoDaddy 进行签名，因此默认受 Arch 信任（前提是至少安装了 ca-certificates包）。

配置和用法

在槽 2 生成新密钥，然后上传到 YubiCloud（会在浏览器中打开）：

```
$ ykman otp yubiotp --generate-key --upload 2
```

更多信息请参考 `ykman otp yubiotp --help`。

安全风险

AES 密钥泄漏

可想而知，AES 密钥必须保密，该密钥无法从 YubiKey 上被读取（或者说至少软件上不可行）。密钥同时被存放在验证服务器上，因此必须确保该服务器的安全。

篡改验证请求/响应

鉴于目标系统依赖于验证服务器，因此攻击方式之一是假冒该服务器。因此，目标系统必须使用 HMAC 或 HTTPS 对验证服务器进行验证。

质询-应答

质询会发送到 YubiKey，然后基于 secret 计算出应答。同一个质询会得出相同的应答。就算有大量的质询-响应对，在没有 secret 的情况下反推计算操作是不可能的。

这可以被用于：

- 事实上的双因素认证：质询被提供给用户，用户必须与密码一起提供正确的响应。双方都需要持有私钥。

- “半”双因素认证：质询被用作密码，然后服务器会存储正确的响应。该操作与 OTP 不同，且持有正确响应的人都能获得授权；但由于不需要私钥，实现难度较低。

有两种质询-响应算法：

- HMAC-SHA1
- Yubico OTP

你可以使用 yubikey-personalization-gui包 通过图形界面进行配置，也可以遵循以下步骤：

HMAC-SHA1 算法

将槽 2 配置为质询-响应模式，并生成一个密钥：

```
$ ykman otp chalresp --generate 2
```

可以省略掉 `--generate` 并手动指定密钥（参考 `ykman otp chalresp --help`），这么做的好处是可以用同一密钥配置其它备用设备。可以使用 `openssl rand -hex 20` 生成可用密钥。

Yubico OTP 算法

`ykman` 貌似不支持配置 chal-yubico 算法，但可以使用 `ykpersonalize` 进行配置。在槽 2 生成一个随机密钥：

```
$ ykpersonalize -2 -ochal-resp -ochal-yubico
```

更多信息请参考 ykpersonalize(1)。

发送质询

可以使用 `ykchalresp -*slot* *challenge*` 发送质询并获得响应，例如：

```
$ ykchalresp -2 archie
12a19763be77d75af46fb76f0b737c117fa47205
```

将返回特定于槽 2 的 40 字节 SHA1 哈希值。不同的质询将返回不同的响应。

固定密码

你也可以生成固定密码：

```
$ ykman otp static --generate slot
```

或者手动提供一个：

```
$ ykman otp static slot password
```

该命令有多个选项：可以设置密码的长度和字符，也可以设置最后是否会按下确认键。具体选项请参考 `ykman otp static --help`。

提示：Most YubiKeys provide only limited (2) slots to save static passwords. A setup challenge-response slot provides static hash responses for unlimited challenges. While these are numeric and lower alphabet only, the response length provides considerable entropy.

模拟 USB 键盘的限制（又称“为什么我的密码看着很弱？”）

为使 YubiKey 与大多数键盘布局兼容，密码默认只能由 ModHex 字母表（`cbdefghijklnrtuv`）、数字 `0-9` 和 `!` 构成。这些字符在大量键盘布局中使用相同的扫描代码，以确保与大多数计算机兼容。

相关信息可参考 Yubico 提供的白名单。

OATH

YubiKey 提供了两种 OATH 实现：

OATH API：新实现，取决于型号不同，可以存储约 30 个凭证。（YubiKey 4、NEO 及更新版本） OTP 槽：旧实现，两个 OTP 槽都分别只能存储单个凭证。（所有支持质询-响应的型号都可用）

OATH API

如果更偏好图形界面，可以使用 yubioath-desktop包。

`ykman` 可以通过 `ykman oath uri` 以 URI 形式添加代码。以下为从二维码图片添加凭证的示例：

```
$ zbarimg qr_code.png --quiet --raw | xargs ykman oath accounts uri
```

你也可以手动进行添加。配置 TOTP 密钥，并要求触摸按钮后才能生成代码：

```
$ ykman oath accounts add --touch name secret
```

配置 HOTP 密钥：

```
$ ykman oath accounts add --oath-type HOTP name secret
```

列出凭证：

```
$ ykman oath accounts list
```

生成代码：

```
$ ykman oath accounts code query
```

所有可用的子命令请参考 `ykman oath --help`。要查看单个命令的选项，请使用 `ykman oath *subcommand* --help`。

OTP 槽实现

在槽 2 中配置 HOTP：

```
$ ykman otp hotp 2 key
```

配置 TOTP：

```
$ ykman otp chalresp --totp slot key
```

生成 HOTP 代码：

```
$ ykman otp calculate slot
```

生成 TOTP 代码：

```
$ ykman otp calculate --totp slot
```

另请参考 `ykman otp --help` 和 https://developers.yubico.com/OATH/ 。

U2F

通过 YubiKey 使用通用第二因素 (U2F) 非常简单，不需要对密钥本身进行配置。注意，在部分文档和工具中，该模式也被称作“FIDO”。`ykman` 上的管理配置项比较少：

- 设置 PIN 码：`ykman fido access change-pin`
- 删除单个凭证：`ykman fido credentials delete *QUERY*`
- 重置所有凭证和 PIN 码：`ykman fido reset`

使用 U2F 进行认证的方法请参考 U2F。

另请参考 WebAuthn。

CCID 智能卡

CCID（芯片卡接口设备）是一个 USB 标准设备类，适用于 USB 智能卡读卡器或类似通过 YubiKey 等通过 USB 连接的安全令牌。HID（人机交互设备）和 CCID 都是 USB 设备类，即它们都位于 USB 标准的同一类别。HID 是类似键盘等计算机外设的标准。在 OTP 和 FIDO 模式下，YubiKey 类似于 USB（HID）键盘；在使用 PIV 应用或作为 OpenPGP 设备时，会切换到 CCID 协议。

从 2015 年 11 月开始，所有新出厂的 YubiKey 都应默认启用了 CCID 模式。Enable at least the CCID mode. Please see #获取已启用的模式.

PIV

从 YubiKey NEO 开始，YubiKey 在芯片中内置了 PIV（个人身份验证）应用。PIV 是一个美国政府标准（FIPS 201），指定了使用 RSA 或 ECC（椭圆曲线密码学）的令牌如何被用于个人电子身份认证。YubiKey NEO 只支持 RSA 加密，后续型号（包括 YubiKey 4 和 5）支持 RSA 和 ECC，具体支持的算法取决于固件。例如，只有 5.7 及更新版本的 YubiKey 支持 RSA 3072、RSA 4096、Ed25519 和 X25519 密钥。PIV 令牌的特性是可以保护私钥并在芯片片上执行操作，私钥在安装到令牌上后就无法被导出。私钥也可以通过芯片内置的随机数生成器直接在芯片上生成，在这种情况下，私钥完全不会暴露出芯片外，也无法从令牌上恢复。在使用 PIV 特性时，YubiKey 是作为 CCID 设备工作的。

OpenPGP 智能卡

YubiKey 可以作为标准 OpenPGP 智能卡工作，具体搭配 GnuPG 进行配置和使用的步骤请参考 GnuPG#智能卡。Yubico 也提供了一些文档：https://developers.yubico.com/PGP/ 。

如果你不想使用其它功能（如 U2F 和 OTP），可以将按钮配置为插入和弹出智能卡，也可以配置自动弹出时间。具体信息请参考 #USB 连接模式。

默认用户 PIN 码是 `123456`，默认管理员密码和 PUK 是 `12345678`。别忘了改掉这三个密码。

用例

本节展示了如何使用 YubiKey 进行多种身份验证，实际上能做到的比这还多。

使用 LUKS 进行全盘加密

有数种实现方案：

- **质询-响应**：将对质询的响应用作 LUKS 密钥。质询可被用作双因素认证中的密码，也可为单因素认证以纯文本形式存储。
- **GnuPG**：使用 Yubikey 的 PGP 智能卡功能，在不需要强密码的同时提供强双因素认证。
- **FIDO HMAC Secret**：如果拟定 YubiKey 支持 U2F，可让其返回对称 secret。

注意：硬盘的加密安全性仅与其最弱的密钥等同。在配置完这几项之一后，考虑移除初始密码或替换为超长密码。

常见依赖

- 一个可启动的 LUKS 加密系统，使用 `encrypt` mkinitcpio 钩子，并有至少一个可用槽位。
- mkinitcpio-ykfdeAUR 除外，`sd-encrypt` 钩子不受上述任一方案支持。
- 备份 LUKS header（可选，但建议）

质询-应答

完整步骤请参考 yubikey-full-disk-encryption包 的官方文档。简单来讲：

1. 安装 yubikey-full-disk-encryption包
2. 配置 `/etc/ykfde.conf`
3. 注册硬盘：`# ykfde-enroll -d /dev/*DISK* -s *LUKS_SLOT*`
4. 在 `encrypt` 钩子前添加 `ykfde` mkinitcpio 钩子

5. 重新生成 initramfs

: > **注意**：对于 Plymouth 用户：将 `plymouth-encrypt` 替换为 `ykfde` 钩子。

该方案有数个变种：

- 2FA：默认行为，在注册设备和启动时需要将质询作为密码提供.
- 1FA：在 `ykfde.conf` 中设置 `YKFDE_CHALLENGE`。注意，这是以纯文本保存的，请考虑为该文件禁用非根用户读取权限。
- NFC 支持（实验性）
- 待机和唤醒支持（实验性）在待机时自动锁定加密卷，恢复时解锁

为使配置生效，需要重新生成 initramfs。

基于 systemd 的 initramfs

`sd-encrypt` 钩子的用户可以安装 mkinitcpio-ykfdeAUR 或 mkinitcpio-ykfde-gitAUR，然后遵循项目文档中的操作描述。具体步骤与 yubikey-full-disk-encryption包大致类似。

GnuPG encrypted keyfile

One tool to accomplish this is initramfs-scencrypt; see its docs for complete instructions. Note that as of October 2022 this package is not in the AUR and is not thoroughly tested, though the GitHub repository offers a PKGBUILD.

The dm-crypt pages offer a few alternatives, though they are mostly links to old forum posts.

HMAC secret extension of FIDO2 protocol

Yet another way of using YubiKey for full disk encryption is to utilize HMAC Secret Extension to retrieve the LUKS password from YubiKey. This can be protected by a passphrase. This functionality requires at least YubiKey 5 with firmware 5.2.3+. For a passphrase protected solution, install khefinAUR and follow instructions available in project documentation. For single factor (optionally PIN-protected) solution and starting with systemd 248, it is possible to use your FIDO2 key as LUKS2 keyslot. Instructions available in the author's blog post.

KeePass

可以为 KeePass 配置 YubiKey 支持，具体步骤请参考 YubiKey 一节。

SSH 密钥

CCID

如果你的 YubiKey 支持 CCID 智能卡，可以基于 GPG 或 PIV 密钥将其用作基于硬件的 SSH 密钥。Yubico 提供了详细的文档：

- 两者用途和优缺点的概述
- Instructions for PGP 认证
- Instructions for 通过用户证书进行 PIV 认证
- Instructions for 通过 #PKCS11 进行 PIV 认证

: > 注意：YubiKey 的 PIV 应用默认 PIN 码是 `123456`，你可能会想修改它和默认管理密码。详细信息请参考设备配置方法。

U2F

你也可以通过 YubiKey 的 U2F 功能创建基于硬件的 SSH 密钥。具体步骤请参考 SSH 密钥#FIDO/U2F。

PIV

yubikey-agentAUR 以 PIV 令牌的显示存储 SSH 密钥。配置步骤请参考 https://github.com/FiloSottile/yubikey-agent#readme 。

使用 PAM 进行 Linux 用户认证

PAM 及所有使用 PAM 进行用户验证的软件都可使用 YubiKey 作为用户验证的一环，其中包括 sudo、su、ssh、锁屏、显示管理器及几乎所有 Linux 系统上需要对用户进行验证的地方，灵活的配置项使你可以为整个系统、特定应用或一组应用设定认证需求。例如，你可以在本地会话上用 YubiKey 取代密码，但在远程会话上要求同时使用二者。除了 Arch Wiki 外，建议额外阅读 pam(8) 和 pam.conf(5) 来理解工作原理的配置方法。

有多个模块可用于将 YubiKey 支持的协议集成到 PAM 中：

- pam-u2f包 - 通过 FIDO2 标准支持 #U2F。如果你不确定要用哪种，建议用这个
- 通用第二因素#用于用户会话认证
- Yubico 的官方文档，提供了支持的模块参数清单
- Man Page：pam_u2f(8), pamu2fcfg(1)
- oath-toolkit包 - 提供 #OATH 一次性密码支持（HOTP 或 TOTP）
- pam_oath
- yubico-pam包 - 提供 #Yubico OTP 和质询-响应支持。注意，Yubico OTP 模式需要联网到验证服务器，而质询-响应模式没有网络要求
- Yubico 的官方文档
- pam_yubico(8) - 注意 `mode` 参数，需要用于配置质询-响应模式

警告：修改 PAM 配置文件需要小心，错误配置会导致系统不再安全，又或是过于安全使得无法通过认证。

PAM 配置超出了本文讨论范围，大致描述如下：

- 在用户 home 目录下或中心化创建包含密钥的文件
- 在对应 PAM 配置文件的合适位置添加一行，格式如下：

```
auth [required|sufficient] [module_name].so [module arguments]
```

- 对于多因素认证使用 `auth required`，单因素认证使用 `auth sufficient`
- `module_name` - 例如：`pam_u2f.so`。要获取已安装的模块清单，请执行：`ls /usr/lib/security`
- 模块配置参数用于如指定密钥位置、模块在认证时使用的方法等操作。

SSH 附注

- Yubico 提供了一些额外说明。虽然是写给旧版 Ubuntu 的，但在最新的 Arch 系统上也可用。
- 如果你在为远程系统配置使用 YubiKey，需要至少额外提供一个备份 SSH 会话，以免配置失败时被锁在系统外。
- 检查 `/etc/ssh/sshd_config` 是否包含以下设置。openssh包 提供的 `sshd_config` 默认包含以下正确配置。

```
ChallengeResponseAuthentication no
UsePAM yes
```

浏览器/网页集成

很多网页服务都开始支持 FIDO 硬件令牌。详细信息请参考 U2F 和 WebAuthn，但通常只需要安装 libfido2包，然后就可以尝试一下。

小技巧

在插入/拔出 YubiKey 时执行操作

例如，如果你想在拔出 YubiKey 时执行操作，可以创建 `/etc/udev/rules.d/80-yubikey-actions.rules` 并添加以下内容：

```
ACTION=="remove", ENV{ID_VENDOR}=="Yubico", ENV{ID_VENDOR_ID}=="1050", ENV{ID_MODEL_ID}=="0010|0111|0112|0113|0114|0115|0116|0401|0402|0403|0404|0405|0406|0407|0410", RUN+="/usr/local/bin/script args"
```

注意，有些版本的 YubibKey 可能无法使用该示例，你需要看下 lsusb 的 vendor 和 model ID 及设备描述，也可以用 udevadm 来获取这些信息。如果要改成在插入时执行，需要将 ACTION 中的 remove 改成 add。

在插入时启动 Yubico Authenticator

The authenticator is a long-running GUI process. If run directly in a udev rule, the process would block udev's processing. If forked, udev would unconditionally kill the process after the event handling finishes. Thus you can't start the authenticator from udev rules. However, systemd.device may be used to handle this case.

与上文类似，创建 `/etc/udev/rules.d/80-yubikey-actions.rules` 并添加以下内容：

```
ENV{ID_VENDOR}=="Yubico", ENV{ID_VENDOR_ID}=="1050", ENV{ID_MODEL_ID}=="0010|0111|0112|0113|0114|0115|0116|0401|0402|0403|0404|0405|0406|0407|0410", SYMLINK+="yubikey", TAG+="systemd"
```

然后创建一个 systemd 用户单元：

```
~/.config/systemd/user/yubioath-desktop.service
[Unit]
Description=Autostart Yubico Authenticator
# Uncomment if you want to stop the authenticator when unplugged.
#StopPropagatedFrom=dev-yubikey.device

[Install]
WantedBy=dev-yubikey.device

[Service]
Type=oneshot
ExecStart=/usr/bin/yubioath-desktop
```

并将其启用。*systemctl* 会提示它被添加为不存在单元的 `dev-yubikey.device` 依赖，但对应单元会在 YubiKey 插入时创建。

维护 / 升级

Installing the OATH Applet for a YubiKey NEO

These steps will allow you to install the OATH applet onto your YubiKey NEO. This allows the use of Yubico Authenticator in the Google Play Store.

注意：These steps are only for NEOs with a firmware version <= 3.1.2. The current generation NEOs (with U2F) come with the OpenPGP applet already installed)

Configure the NEO as a CCID device

1. Install yubikey-personalization-gui包 (yubikey-personalization-gui-gitAUR).
2. Add the udev rules and reboot so you can manage the YubiKey without needing to be root
3. Run `ykpersonalize -m82`, enter `y`, and hit enter.

Install the applet

1. Install gpshellAUR, gppcscconnectionpluginAUR, globalplatformAUR, and pcsclite包.
2. 启动 `pcscd.service`.
3. Download the most recent CAP file from the ykneo-oath site.
4. Download `gpinstall.txt` from GitHub.

5. Edit the line in gpinstall.txt beginning with `install -file` to reflect the path where the CAP file is located.
6. Open a terminal and run `gpshell *path/to/gpinstall.txt*`.
7. Ideally, a bunch of text will scroll by and it ends saying something like

```
    Command -->
80E88013D7C000C400BE00C700CA00CA00B400BE00CE00D200D500D700B000DB00C700DF00BEFFFF00BE00E
400AC00AE00AE00DB00E700A

A00EA00ED00ED00ED00BE00EF00F100F400F100F700FA00FF00BE00F700AA01010103010700CA00C400B400
AA00F700B400AA00B600C7010C

010C00AA0140012001B0056810B00130056000000056810E0011006B4B44304B44404B44106B44B4405B4434
00343B002410636810E06B4B44

407326810B004B43103441003334002B102B404B3B403BB4003B440076820A4100221024405B4341008B446
0000023106682OA100
    Wrapped command -->
84E88013DFC000C400BE00C700CA00CA00B400BE00CE00D200D500D700B000DB00C700DF00BEFFFF00BE00E
400AC00AE00AE00DB00E700A

A00EA00ED00ED00ED00BE00EF00F100F400F100F700FA00FF00BE00F700AA01010103010700CA00C400B400
AA00F700B400AA00B600C7010C

010C00AA0140012001B0056810B00130056000000056810E0011006B4B44304B44404B44106B44B4405B4434
00343B002410636810E06B4B44

407326810B004B43103441003334002B102B404B3B403BB4003B440076820A4100221024405B4341008B446
0000023106682OA15D848CB77
    27D0EDA00
    Response <-- 009000
    Command -->
80E60C002107A000000527210108A0000005272101010800003C901000000
    Wrapped command -->
84E60C002907A000000527210108A0000005272101010800003C9010000B4648127914
A4C7C00
    Response <-- 009000
    card_disconnect
    release_context
```

1. Unplug the NEO and try it with the Yubico Authenticator app.

（可选）安装 Yubico Authenticator 桌面客户端

You can get the desktop version of the Yubico Authenticator by installing yubioath-desktop包.

While `pcscd.service` is running, run `yubioath-desktop` and insert your YubiKey when prompted.

排障

不论部分功能正常与否，先重启试试，特别是如果上次 YubiKey 能正常使用后更新过系统。

YubiKey not acting as HID device

注意：These steps are no longer necessary after systemd since v244 added native support for this functionality.

Add udev rule as described in this article:

```
/etc/udev/rules.d/10-security-key.rules
KERNEL=="hidraw*", SUBSYSTEM=="hidraw", MODE="0664", GROUP="users", ATTRS{idVendor}
=="2581", ATTRS{idProduct}=="f1d0"
```

Run `udevadm trigger` afterwards.

ykman 无法连接到 YubiKey

如果管理器无法连接到 YubiKey，先检查 `pcscd.service` 或 `pcscd.socket` 是否已经启动。

Error: Failed connecting to YubiKey 5 \[OTP+FIDO+CCID\]. Make sure the application have the required permissions.

在 `scdaemon` 独占了 YubiKey 控制权的情况下，如果 `ykman` 尝试读取 OATH 凭证，就会出现该报错。

要修复该问题，可以在 `~/.gnupg/scdaemon.conf` 中为设备配置 `reader-port` 选项。

注意：该操作会导致每次 ykman 访问 YubiKey 时 gpgagent 都会提示你解锁 YubiKey。

对于 YubiKey NEO 和 YubiKey 4：

```
reader-port Yubico Yubikey
```

对于 YubiKey 5：

```
reader-port Yubico Yubi
```

无法将 YubiKey 绑定到虚拟机

假设 YubiKey 可用于虚拟机上，该问题是由宿主机上的驱动绑定了设备导致的。要解绑设备，需要在宿主机上通过 dmesg 获取总线和端口信息：

```
# dmesg | grep -B1 Yubico | tail -n 2 | head -n 1 | sed -E 's/^\[[^]]+\] usb
([^:]*):.*/\1/'
```

输出的 USB ID 格式为 `X-Y.Z` 或 `X-Y`。接着，在宿主机上使用 `find` 搜索 `/sys/bus/usb/drivers` 来查找哪一个驱动在占用 YubiKey（例如 `usbhid` 或 `usbfs`）：

```
$ find /sys/bus/usb/drivers -name "*X-Y.Z*"
```

要解绑 YubiKey，需用到之前获取的信息（例如 `/sys/bus/usb/drivers/*DRIVER*/X-Y.Z:1.0`）：

```
# echo 'X-Y.Z:1.0' > /sys/bus/usb/drivers/DRIVER/unbind
```

Error: \[key\] could not be locally signed or gpg: No default secret key: No public key

在插入 YubiKey 时使用非标准密钥环对密钥进行签名时出现（如 Pacman 的 `pacman-key --populate` 命令）。解决方法是拔出 YubiKey 后重试。

YubiKey 在 Yubico Authenticator 中消失后又重新出现

这是因为缺少 CCID 驱动，你需要安装 ccid包。

YubiKey core error: timeout

可能是用错槽位了，换一个试试。

TOMOYO Linux

TOMOYO Linux 走的是与 SELinux 一类标签式方案不同的强制访问控制路线，本章要解决的是它究竟是什么、为何这样设计，以及怎样真正用起来。只有一节，顺序上建议先读简介，吃透它的定位与基本模型；开发分支、1.x、AKARI 三部分讲的是历史沿革，时间紧可以略过；2.x 是现行主线，要看清它与 1.x 的分界；最后的「使用」是唯一的动手部分，必须读完。最容易踩的坑有两个：一是拿标签式 MAC 的直觉直接套过来，规则与预期不符就断定是配置写错；二是照着 1.x 时代的旧资料去操作 2.x，混淆了两者的差别。

12.1　TOMOYO Linux

难度 进阶 | **篇幅** 14.5 KB | **所属** TOMOYO Linux

本节将讲到

- 简介
- 开发分支
- TOMOYO Linux 1.x
- AKARI
- TOMOYO Linux 2.x

翻译状态：

- 本文（或部分内容）译自 TOMOYO Linux，最近一次同步于 2026-06-14，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

TOMOYO Linux是针对 Linux 的一种强制访问控制（Mandatory Access Control, MAC）实现。该项目于 2003 年 3 月启动，并由NTT Data公司资助。TOMOYO Linux 专注于系统行为，允许每个进程声明实现其目的所需的行为和资源。它既能作为系统分析工具使用，也能作为访问限制工具使用。

TOMOYO Linux 的安全目标是提供“既能覆盖绝大多数用户的实际使用需求，又能保持让绝大多数管理员易于使用的强制访问控制(MAC)”。TOMOYO Linux 不仅是面向安全专业人员的工具，也同样适用于普通用户和系统管理员。

有关强制访问控制概念，参见：zhwp:强制访问控制

注意：本条目仅作上游用户文档补充说明。有关更多信息，参见上游用户文档

提示：TOMOYO Linux 2.x最终将在功能特性上接近 1.x 分支，但对于追求快速上手的用户而言，2.x 分支更易于安装。TOMOYO Linux 1.x 分支适用于对安全性要求极高的场景，而 AKARI 则介于两者之间。

简介

TOMOYO Linux 试图构建一个让所有配置都以易于理解的方式预先安排妥当的系统：

- 提前获取在内核生命周期内至少会发生一次的所有访问请求。
- 允许管理员编写仅放行预期内且合规的访问请求策略。

与AppArmor不同，TOMOYO Linux 旨在保护整个系统免受攻击者利用应用程序漏洞带来的威胁。TOMOYO Linux 应对该威胁的方式是：先在测试环境中记录所有应用的行为，随后在生产环境中强制所有应用只能在这些已记录的范围内运行。

TOMOYO Linux 并不适用于希望直接使用他人提供的现成策略文件的用户。它需要从零开始构建策略，并通过“学习模式(learning mode)”提供辅助，该模式可以针对特定系统自动生成具备必要且充分权限的策略文件。TOMOYO Linux 能够报告 Linux 系统内部正在发生的情况，因此也可以作为系统分析工具使用。它类似于 *strace*，能够报告每个程序正在执行的操作以及访问了哪些文件或网络。

该对比表提供了 TOMOYO Linux与AppArmor、SELinux以及SMACK)的全面对比。

开发分支

TOMOYO Linux 1.x是最初的开发分支。TOMOYO Linux 于 2005 年 11 月 11 日首次发布。它以可应用于 Linux 内核的补丁形式实现，目前仍在积极开发中。该分支可以与 SELinux、SMACK 和 AppArmor 等其他安全模块并存。

TOMOYO Linux 2.x是 Linux 主线内核(Mainline Kernel)的开发分支。2009 年 6 月，TOMOYO 被合并至 Linux 2.6.30 内核版本中，并使用标准的 Linux 安全模块(LSM)钩子(Hooks)。然而，若要将 TOMOYO Linux 的完整 MAC 功能移植到 Linux 内核中，则必须进一步扩展 LSM 钩子。因此，该分支尚未提供与 1.x 开发分支对等的功能。该图表对比了各分支之间的差异。

AKARI基于 TOMOYO Linux 1.x 分支，并可以加载内核模块(LKM)的形式实现。因此，在无需针对内核应用补丁并重新编译的前提下，该分支具有更显著的使用优势。该表格提供了 AKARI 与 TOMOYO Linux 1.x 及 2.x 分支的全面对比。

TOMOYO Linux 1.x

通过应用 *ccs-patch* 补丁的内核来实施 TOMOYO Linux 1.x，以获取该项目所能提供的完整功能。然而，由于必须使用ccs-patch对内核进行补丁处理并随后重新编译，因此在实施该分支时需要克服多重困难。

必须同时安装打过补丁的内核以及用户空间工具。AUR 中提供了ccs-toolsAUR软件包。

初始化配置

首先必须初始化策略：

```
# /usr/lib/ccs/init_policy
```

策略文件会保存在/etc/css/目录中，可以通过运行以下命令进行编辑：

```
# ccs-editpolicy
```

AKARI

AKARI 的局限性

在纯粹将 TOMOYO Linux 项目用于系统分析的前提下。AKARI 是实现该目标最简便的方法。而在将其用于系统限制的前提下，这是以最小的工作量获取 TOMOYO Linux 1.x 分支绝大部分功能的方法。然而，有下述局限性：

- 其取决于发行版提供的内核版本与具体配置：

```
CONFIG_SECURITY=y [必需]
CONFIG_KALLSYMS=y [必需]
CONFIG_PROC_FS=y [必需]
CONFIG_MODULES=y [必需]
CONFIG_SECURITY_PATH=y [可选：用于支持绝对路径名]
CONFIG_SECURITY_NETWORK=y [可选：用于提供网络限制]
```

- 由于缺少所需的 LSM 钩子，部分高级网络操作的限制功能会受制约或无法使用。
- 无法限制内核功能(capabilities)的使用。
- 为了保持内核二进制接口(KABI)不发生改变，每个任务的变量(pre-task variables)被管理在*struct task_struct*之外，这导致其查找速度较慢。不过，由于基于路径名的权限检查所带来的性能下降占主导地位，因此对于典型的最终用户而言，此项性能损耗应该并不明显。

该表格提供了 AKARI 与 TOMOYO Linux 1.x 及 2.x 分支的全面对比。

安装

必需同时安装 AKARI 以及用户空间工具。AUR 中提供了akariAUR和ccs-toolsAUR软件包。

若要激活 AKARI，请设置以下内核参数：

```
init=/usr/bin/ccs-init
```

初始化配置

首先必需初始化策略：

```
# /usr/lib/ccs/init_policy --module_name=akari
```

策略文件会保存在`/etc/css/`目录中，可以通过运行以下命令进行编辑：

```
# ccs-editpolicy
```

TOMOYO Linux 2.x

TOMOYO Linux 2.x 的局限性

TOMOYO Linux 2.x 在 Linux 主线内核中的实现尚未完全结束，但自 2.5.x 版本以来已非常接近 1.x 分支。与 1.x 分支相比，仍有少数特性有待实现。该图表提供了各开发分支之间差异的全面对比。

安装

所有官方支持的内核均已内置对 TOMOYO 的支持。

针对自定义内核，请确保设置了以下内核选项：

```
CONFIG_SECURITY=y
CONFIG_SECURITYFS=y
CONFIG_SECURITY_NETWORK=y
CONFIG_SECURITY_PATH=y
CONFIG_SECURITY_TOMOYO=y
CONFIG_SECURITY_TOMOYO_POLICY_LOADER="/usr/bin/tomoyo-init"
CONFIG_SECURITY_TOMOYO_ACTIVATION_TRIGGER="/usr/lib/systemd/systemd"
 CONFIG_LSM="landlock,lockdown,yama,integrity,tomoyo,bpf"
```

安装用户空间工具包的 2.6 分支软件包tomoyo-toolsAUR。

激活

若要激活 TOMOYO Linux，请设置以下内核参数以启用该 Linux 安全模块(LSM)：

```
lsm=landlock,lockdown,yama,integrity,tomoyo,bpf
```

注意：`lsm=`内核参数用于设置 Linux 安全模块的初始化顺序。内核配置的`lsm=`可以通过`zgrep CONFIG_LSM=/proc/config.gz`确定，当前生效的值可通过`cat /sys/kernel/security/lsm`查看。

- 确保`tomoyo`放在列表第一个“主要(major)”模块的位置有关有效值及其顺序的示例，可以在security/Kconfig中找到。
- 由于`capability`总是会被自动包含，因此应从`lsm=`中省略

在`/usr/lib/systemd/systemd`执行时，TOMOYO 将从`/etc/tomoyo/policy/current`中加载所有已保存的策略。

接下来，检查激活是否成功。通过dmesg应该会看到如下所示（或类似）的输出行：

```
# dmesg | grep -A 1 -B 1 TOMOYO
[    0.003375] Security Framework initialized
[    0.003387] TOMOYO Linux initialized
[    0.003396] AppArmor: AppArmor disabled by boot time parameter
--
[    6.829798] Calling /usr/bin/tomoyo-init to load policy. Please wait.
```

```
[    6.833709] TOMOYO: 2.5.0
[    6.833712] Mandatory Access Control activated.
```

首次使用时，如果需要在计算机关机/重启时将内存中的策略自动保存到文件系统，请创建以下systemd服务单元：

```
/etc/systemd/system/tomoyo-savepolicy.service
[Unit]
Description=Tomoyo savepolicy

[Service]
Type=oneshot
ExecStart=/bin/true
ExecStop=/usr/bin/tomoyo-savepolicy
StandardInput=tty
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

随后，启用`tomoyo-savepolicy.service`。

禁用

从`lsm=`内核参数中移除`tomoyo`，或者完全移除`lsm=`参数。

初始化策略

首先必需初始化策略：

```
# /usr/lib/tomoyo/init_policy
```

策略文件会保存在`/etc/tomoyo`目录中，可以通过运行以下命令进行编辑：

```
# tomoyo-editpolicy
```

默认情况下，TOMOYO 启动时会使用*Disabled*（已禁用）配置集（参见下方的配置集表格）。如果现在就要为所有实体启用学习模式，只需要在`etc/tomoyo/policy/current/domain_policy.conf`中`*<kernel>*`命名空间的配置集：

```
<kernel>
use_profile 1
use_group 0
```

在无法确定是否需要如此广泛的学习模式的前提下，直接忽略此步骤即可。稍后可以通过*tomoyo-editpolicy*工具，在*Domain transition editor*（域迁移编辑器）中对任何选定的域（或多个域）按下`S`键来切换配置集。

完成上述配置后，需要重启计算机。

日志守护进程

TOMOYO 拥有专属的日志守护进程`tomoyo-auditd`。它对于监控闭源应用程序的行为非常实用。初始化配置文件中包含详实的注释说明，位于`/etc/tomoyo/tools/auditd.cconf`，而日志文件则存放在`/var/log/tomoyo`目录中

若需配合 Systemd 使用该进程，请按照官方文档第 4.6 章所述内容创建`/etc/systemd/system/tomoyo-auditd.service`文件。

使用

为能够高效使用 TOMOYO Linux 或 AKARI，必需查阅相关文档：

- TOMOYO Linux 文档
- AKARI 文档

运行策略编辑器以开始编辑。在运行 TOMOYO Linux 1.x 或 AKARI 的前提下，应使用*ccs-tools*：

```
# ccs-editpolicy
```

如果使用 TOMOYO Linux 2.x，则应使用：

```
# tomoyo-editpolicy
```

随着系统的运行，TOMOYO Linux 将创建域(domains)并将其添加到树状结构中。TOMOYO Linux 中的访问分析/限制是通过域来实现的。每个进程都属于一个唯一的域，并且每当进程执行一个程序时，就会迁移到一个不同的域。域的名称是表示进程执行历史的级联字符串表达式。例如，内核所属的域为`*<kernel>*`；由内核调用的`/sbin/init`所属的域名为`*<kernel> /sbin/init*`；如果`/sbin/init`调用了`/etc/rc.d/rc`，则其所属的域名为`*<kernel> /sbin/init /etc/rc.d/rc*`。可以根据需要抑制或初始化域迁移。

可以为每个域分配配置集(profiles)。共有四种默认配置集：

<table class="wikitable"><tbody><tr><td>Disabled（已禁用）</td><td>效果与普通内核相同。</td></tr><tr><td>Learning（学习）</td><td>即使访问请求违反了策略，也不予拒绝，而是将该请求追加到策略中。</td></tr><tr><td>Permissive（宽容）</td><td>即使访问请求违反了策略，也不予拒绝，且不将该请求追加到策略中。</td></tr><tr><td>Enforcing（强制）</td><td>若请求违反策略，予以拒绝，不将该请求追加到策略中。</td></tr></tbody></table>

学习(learning)配置集可用于分析系统或有特定需求的应用程序。一旦确定了某个域的所有预期访问请求，即可在切换到强制(enforcing)配置集之前，根据需要编辑该域的策略。此操作可以从系统启动开始，针对任何及所有域进行设置。

参见

- Wikipedia:Tomoyo Linux
- TOMOYO Linux 主页

- TOMOYO Linux 维基
- AKARI 主页
- AKARI 文档
- AKARI/TOMOYO 功能对比表
- TOMOYO Linux 1.8.x : 官方指南
- TOMOYO Linux 2.6.x : 官方指南
- TOMOYO Linux 安全目标
- 规则示例
- Embedded Linux Wiki上的 TOMOYO Linux 条目
- 来自 PacSec 2007 的演示文稿幻灯片

入侵检测

机器被人动过手脚，靠翻日志和看进程往往看不出来，留下的痕迹多是替换过的系统命令和被植入的后门。本章用 rkhunter 把这件事做成一次可重复的例行检查：装好、配准、跑通，结果才有可比性。

本章只有一节，必读，没有可跳的部分。安装与配置决定扫描结果是否可信；使用与故障排除放在第一次扫描后按报错回查，附录当报警时的对照表。最易误解的是把 rkhunter 当杀毒软件：它比的是特征与文件属性，报警只代表需要人工确认，不等于已被入侵；而不做配置就开扫，满屏警告会把真正的异常淹掉。

13.1　Rkhunter

难度 入门 | **篇幅** 2.9 KB | **所属** 入侵检测

本节将讲到

- 安装
- 配置
- 使用
- 故障排除
- 附录

rkhunter (Rootkit Hunter) 是一款用于POSIX兼容系统的安全监控工具。它通过搜索（rootkit的）默认目录、权限配置错误、隐藏文件、包含可疑字符串的内核模块，以及将重要文件的哈希值与已知正常文件哈希值进行比较，来检查rootkit和其他可能的漏洞。

其使用 Bash 编写，具有良好的可移植性，可在大多数基于UNIX的系统上运行。

安装

安装 软件包 rkhunter包。

配置

初始化

首次运行rkhunter前，请更新*文件属性数据库*:

```
# rkhunter --propupd
```

相关文件

主配置文件位于 `/etc/rkhunter.conf`。

默认情况下，最近一次系统检查的日志将存放在 `/var/log/rkhunter.log`。

使用

详细请参见 rkhunter(8)。

基本命令

更新文件属性数据库：

```
# rkhunter --propupd
```

在扫描前，务必运行以下命令以确保rkhunter数据文件保持最新：

```
# rkhunter --update
```

执行系统检查：

```
# rkhunter --check --sk
```

校验配置文件：

```
# rkhunter --config-check
```

故障排除

误报

默认情况下，Rootkit Hunter在文件属性检查期间会产生一些误报警告。这是因为部分核心程序已被脚本替代，您可通过白名单机制消除这些警告：

```
/etc/rkhunter.conf
SCRIPTWHITELIST=/usr/bin/egrep
SCRIPTWHITELIST=/usr/bin/fgrep
SCRIPTWHITELIST=/usr/bin/ldd
SCRIPTWHITELIST=/usr/bin/vendor_perl/GET
```

附录

额外文档

- Rootkit Hunter Homepage
- Rootkit Hunter README
- Rootkit Hunter FAQ

相关维基页面

- rkhunter
- Host-based intrusion detection system (HIDS)
- Intrusion detection system (IDS)

密码管理器

密码管理器的任务很具体：把散落在脑子和笔记里的口令收进一个结构化、可备份的加密仓库，并把旧口令搬进去。本章内容集中在一节：安装、基本用法与信息格式是必读主干，走完即可覆盖日常查改；迁移到 pass 决定你手上现成的口令怎么进来，务必先读完再动手；扩充与进阶用法可按需跳读，用不上就先放过。

最容易踩的坑是把信息格式当成可有可无的排版：它是后续检索与自动处理的基础，迁移时随手塞旧文件而不按格式重写，省下的时间后面要加倍还。另一个误解是装完就急着扩充：主干没跑顺就堆功能，排错时反而不知该怀疑哪一层。

14.1　Pass

难度 进阶 | **篇幅** 11.3 KB | **所属** 密码管理器

本节将讲到

- 安装
- 基本用法
- 信息格式
- 迁移到pass
- 扩充

官网提到:

: 密码管理应该要简易且遵照Unix哲学。Pass将你的密码保存在由gpg加密的文件中，并以相关的网站和资源的名称来命名文件。这些加密文件会被组织成合理的文件体系，你可以从一台装置复制到另一台装置，并用命令行程序来管理和操作它们。

Pass是一款简易的命令行密码管理器，本质上，它其实是利用GnuPG、tree包、和Git的脚本。

安装

安装 pass包 软件包.

另外还有图形Qt界面软件包可供安装：qtpass包

基本用法

注意：在使用Pass前，请先配置好GnuPG。Pass所使用的密钥信任程度(trust level)必须为"ultimate"。

初始化：

```
$ pass init <gpg-id or email>
```

若要创建一组新密码，提供一个文件名，注意文件名需要能体现出文件层次，如：*archlinux.org/wiki/username*。

```
$ pass insert archlinux.org/wiki/username
```

以文件组织的方式查看储存的密码：

```
$ pass
Password Store
└── archlinux.org
    └── wiki
        └── username
```

生成一组随机的新密码，执行如下命令，其中，正整数*n*代表想要的密码长度。

```
$ pass generate archlinux.org/wiki/username n
```

若要取得一组密码，执行如下命令，并在弹出窗口输入你的gpg密码短语(passphrase)，如使用以上范例：

```
$ pass archlinux.org/wiki/username
```

若您是Xorg用户并安装了xclip包，您可以直接将取得的密码暂时的复制到剪贴板(clipboard)上；若您是Wayland用户，pass-gitAUR 则会使用 wl-clipboard包，如使用以上范例：

```
$ pass -c archlinux.org/wiki/username
```

注意：如果您喜欢以点击鼠标滚轮的方式来贴上密码，您可以在自己的 `~/.shellrc` 中添加：`export PASSWORD_STORE_X_SELECTION=primary`

pass 也有附加的功能可与dmenu包相结合，让用户可以轻松的搜索和复制粘贴。若要使用它，安装如下可选依赖dmenu包后，执行：

```
$ passmenu
```

当您选择一组密码时，dmenu将会复制密码到剪贴版上。dmenu(1) 有更多相关的自定义选项。为了更快的取得密码，您可以把这个命令绑定在一组系统快捷键上。

信息格式

由`pass insert`新建的信息文件预设只会包含您的密码，有时这仍旧不太足够，因为一些应用可能会要求取得您的其他信息如：用户名、网站地址等。这时，您可以用以下命令，编辑一个已存在的信息文件：

```
$ pass edit password_name
```

如下是由官网推荐的信息排版格式。使用这种格式时，选项`-c`或`--clip`仅会复制第一行的密码。

```
YwrZSNH35z164ym9pI
URL: *.amazon.com/*
Username: AmazonianChicken@example.com
Secret Question 1: What is your childhood best friend's most bizarre superhero fantasy?
Oh god, Amazon, it's too awful to say...
Phone Support PIN #: 84719
```

迁移到pass

在这里可以找到相当多脚本可将其他应用的密码导入pass。

扩充

自版本1.7起，pass开始支援由社区开发的扩充，这些扩充包含一些新的命令，用以延伸pass的功能。

- pass-tomb (pass-tombAUR)

可将信息文件以tomb加密

- pass-otp (pass-otp包)

一次性密码(OTP)支援

- pass-import (pass-importAUR)

从其他管理器导入密码的综合工具

- pass-update (pass-updateAUR)

一种更新密码的快捷方式

- pass-audit (pass-auditAUR)

一款用以审查密码安全性的扩充

进阶用法

可使用环境变量修改pass设定中执行存取和git命令的地方：

```
PASSWORD_STORE_DIR=/path/to/store
```

若想进一步了解怎么修改变量使pass支持存取多个密码仓库，参阅此处

以下的`pw()`别名范例将信息文件中第一行复制到剪贴板上，五秒后再复制第二行，再五秒后复制一组一次性密码(OTP)。如果信息文件中的第一行为密码(password)，第二行为用户名(username)，并包含一组一次性密码(OTP)URI，此范例可按照*username* > *password* > *otp code*的顺序将三者贴入空白栏位中(如浏览器的登入注册表)。

```
pw() {
export PASSWORD_STORE_CLIP_TIME=8
export PASSWORD_STORE_X_SELECTION=primary
pass -c2 $1; sleep 5; pass -c $1; sleep 5; pass otp -c $1; exit
}
```

Multiple pass Contexts (e.g. Teaming)

One can use aliases to set up different pass contexts, which helps when collaborating with different teams. We have gotten this working in bash as follows:

Add aliases to your *~/.bashrc*:

```
alias passred="PASSWORD_STORE_DIR=~/.pass/red pass"
alias passblue="PASSWORD_STORE_DIR=~/.pass/blue pass"
```

Add these for bash-completion to your *~/.bash_completion* and make sure bash-completion包 is installed:

```
source /usr/share/bash-completion/completions/pass
_passred(){
    PASSWORD_STORE_DIR=~/.pass/red/ _pass
}
complete -o filenames -o nospace -F _passred passred
_passblue(){
    PASSWORD_STORE_DIR=~/.pass/blue/ _pass
}
complete -o filenames -o nospace -F _passblue passblue
```

Now you can initialize into *~/.pass/red* and *~/.pass/blue* and have two pass contexts with the *passred* and *passblue* aliases. You can generalize this further into as many contexts as you like.

Git integration

Git helper usage

You can use `pass` as a credentials helper for `git`. Install the pass-git-helperAUR or pass-git-helper-gitAUR package. Detail are described in the github README file.

`git` Configuration

Install `pass-git-helper` as a git credentials helper by calling:

```
git config --global credential.helper /usr/bin/pass-git-helper
```

Mapping File

Create the file `~/.config/pass-git-helper/git-pass-mapping.ini`. It is used to map git remote hosts to your `pass` database. The format is something like this:

```
[github.com]
target=dev/github

[*.fooo-bar.*]
target=dev/fooo-bar
```

You can use wildcards in the host part, as shown in the example.

Password Store Layout

As usual with pass, the helper assumes that the password is contained in the first line of the passwordstore entry. Additionally, if a second line is present, this line is interpreted as the username.

For this to work, you have to use `pass insert --multiline` to create a multi line password store entry.

Central Git server for pass in combination with GnuPG (SSH example)

You are able to setup a password management system by setting up a central Git server for Pass. This allows you to synchronize your central password repository through multiple client environments.

Install a bare Git repository for Pass on the server

On the server run `git init --bare ~/.password-store` to create a bare repository you can push to.

Import authorized public SSH keys

See SSH keys#Copying the public key to the remote server

On the client

This section assumes you have configured GnuPG and have a key pair to encrypt passwords. On your local client ensure you have a local password store on the client, then enable management of local changes through Git, add your remote Git repository, and push your local Pass history.

```
# Create local password store
pass init <gpg key id>
# Enable management of local changes through Git
pass git init
# Add the the remote git repository as 'origin'
pass git remote add origin user@server:~/.password-store
# Push your local Pass history
pass git push -u --all
```

Now you can use the standard Git commands, prefixed by `pass`. For example: `pass git push`, or `pass git pull`. Pass will automatically create commits when you use it to modify your password store.

Troubleshooting

Encryption failed: Unusable public key

The following error can occur when attempting to insert a new entry:

```
$ pass insert archlinux.org/wiki/username
Enter password for archlinux.org/wiki/username:
Retype password for archlinux.org/wiki/username:
gpg: XXXXXXXXX: There is no assurance this key belongs to the named user
gpg: [stdin]: encryption failed: Unusable public key
Password encryption aborted.
```

This occurs if the trust level of the GnuPG key is set to anything other than "ultimate." Edit the key used for `pass` to set its trust level to "ultimate."

See also

- A more comprehensive pass tutorial
- Pass home page
- List of Compatible clients and possibilities for migration to Pass

指纹识别

指纹识别把生物特征接入验证流程。读完这一章，你应当能判断手上的读取器在 Arch Linux 下能不能用，完成安装与指纹注册，并把它接入登录与解锁入口。

本章只有 Fprint 一节，需要通读：使用前提与安装决定能否继续，配置是设备真正可用的关键一步，都属必读；故障排除与参见偏备查，初读可略过，注册失败或识别失灵时再回头翻。最容易踩的坑是顺序颠倒：不先核对硬件支持就装软件，毫无反应还以为配错了；其次是期待错位——指纹只是日常解锁的便利补充，不该当成比口令更可靠的唯一凭据，识别不准时先回前提与配置核对，而不是急着换设备。

15.1　Fprint

难度 进阶 | **篇幅** 13.5 KB | **所属** 指纹识别

本节将讲到

- 使用前提
- 安装
- 配置
- 故障排除
- 参见

fprint 旨在填补开源桌面环境在生物特征认证方面的空白，尤其是通过为指纹扫描仪提供标准支持来增强安全性。该项目的核心是 **libefprint** 库，它作为一个中间件层，为指纹识别硬件提供统一的接口，让各个桌面环境和应用软件能够便捷地利用指纹识别功能。

该项目是在一些支持内置指纹识别器的笔记本中使用 PAM 登录。此教程还将介绍如何使用常规密码作为备用登录的方法（由于某些原因，不建议仅使用指纹模块进行验证登录）。

使用前提

注意：兼容设备列表并没有定期更新，也不完整。在使用AUR软件包之前，即使设备没有出现在该兼容清单列表中，仍然可以尝试使用本页教程测试指纹识别设备是否可用。

通过查看兼容设备清单和不兼容设备清单来确定是否支持指纹识别设备。 使用下面命令检测是否识别指纹设备信息:

```
$ lsusb
```

命令 *lsusb* 在软件包usbutils包中提供。

安装

安装 fprintd包 软件包。 可能也需要软件包imagemagick包.

某些设备需要使用 libfprint包 的分支版本，但这些分支目前还没有合并到上游*libfprint*, 清单如下：

- **libfprint-tod** — 用于基于传感器的设备。

: https://gitlab.freedesktop.org/3v1n0/libfprint/-/tree/tod || libfprint-tod-gitAUR

- **libfprint-elanmoc2** — 用于ELAN `04f3:0c4c`,其版本在等待合并中。

: https://gitlab.freedesktop.org/Depau/libfprint/-/tree/elanmoc2 || libfprint-elanmoc2-gitAUR

- **libfprint-elanmoc2-newdrvs** — 针对`04f3:0c4c`或`04f3:0c00`**实验性**的功能，其版本也在等待合并。

: https://gitlab.freedesktop.org/geodic/libfprint/-/tree/elanmoc2 || libfprint-elanmoc2-newdrvs-gitAUR

清单并非包含所有，更完整的分支清单需在AUR中查找。

配置

登录配置

注意：

- 如果桌面管理器为 GDM ，指纹选项默认可以用于重新认证（例如在锁屏界面解锁），而在初次认证时默认不可用（例如在登陆界面登陆账号）。在进行下面完的配置后则可以在初次认证时使用指纹认证。
- 如果桌面管理器为 SDDM, 请查看 使用指纹识别器。

将`pam_fprintd.so`模块添加在`/etc/pam.d/system-local-login`文件 auth 字段的开头位置。

```
/etc/pam.d/system-local-login
auth      sufficient pam_fprintd.so
auth      include    system-login
...
```

这里首先会尝试使用指纹认证登录，如果验证失败或找不到用户指纹数据，则进行密码认证登录。

警告：此设置是su和sudo的安全漏洞，因为它允许后台进程在不提示用户的情况下获取权限。请参阅：CVE-2024-37408

将此项放在`auth fully pam_fprintd.so`之前，以便在指纹无法显示提示时使用sudo/su禁止指纹。

```
# 不允许在没有tty的情况下在sudo/su中使用指纹
```

```
auth       [success=1 default=ignore]  pam_succeed_if.so    service in sudo:su:su-l tty
in :unknown
```

也可以用相同的方式修改 `/etc/pam.d/{login,su,sudo,gdm,lightdm}` 中的其他文件， 如 `/etc/pam.d/polkit-1` 用于基于 polkit 身份验证（GNOME和其他桌面环境）。如果 `/etc/pam.d/polkit-1` 不存在，请从`/usr/lib/pam.d/polkit-1`复制。

KDE已经在`/etc/pam.d/KDE fingerprint`中配置了指纹身份验证，因此不需要编辑该文件。对于最低限度的工作设置，它首先要求您的指纹，如果在KDE Plasma上失败，则需要密码验证，只需将以下行更改为`/etc/pam.d/system-auth`即可：

```
-auth      [success=3 default=ignore]   pam_systemd_home.so
auth       [success=2 default=ignore]   pam_fprintd.so
auth       [success=1 default=bad]     pam_unix.so          try_first_pass nullok
```

当完成录入指纹后，如果将 `pam_fprintd.so` 以*sufficient*作为条件添加到 `/etc/pam.d/` 中的配置文件中时，只会进行指纹身份验证，此时因无法按 `Ctrl+c` 中断指纹身份验证（由于缺少shell环境），导致无法使用密码进行认证。如果想在图形界面中使用密码或指纹认证其中一项完成认证，请在相关文件的开头位置添加下列内容：

```
auth          sufficient    pam_unix.so try_first_pass likeauth nullok
auth          sufficient    pam_fprintd.so
...
```

这里首先会提示输入密码，如果在空白字段上按 `Enter` 将跳过密码认证，继续进行指纹认证。

如果需要同时使用输入指纹和密码，可以使用 pam-fprint-grosshackAUR。对于某些不允许空白密码输入的图形程序，例如 Gnome 的内置 polkit 代理，可能需要这样做。若要使用此包，请将以下行添加到所需任何文件的开头位置：

```
auth          sufficient    pam_fprintd_grosshack.so
auth          sufficient    pam_unix.so try_first_pass nullok
...
```

录入指纹

需要运行 Polkit 身份认证组件才能录入。

开始录入指纹，请运行：

```
$ fprintd-enroll
```

或为所有手指录入指纹：

```
$ fprintd-delete "$USER"
$ for finger in {left,right}thumb,{index,middle,ring,littlefinger}; do fprintd-enroll -
f "$finger" "$USER"; done
```

运行后将要求扫描给定的手指。用右手食指滑动“五次”。之后，在 `/var/lib/fprint/` 中创建指纹数据。

也可以直接指定用户录入指纹：

```
# fprintd-enroll user
```

验证指纹

如果您需要验证录入的指纹能否正确识别，请运行以下命令：

```
$ fprintd-verify
```

删除指纹

如果不满意已录入的指纹数据，可以使用下面命令进行删除，例如需要删除左手拇指指纹数据：

```
$ fprintd-delete -f left-thumb
```

有关更多信息，请参阅 fprintd(1).

限制用户录入指纹

**本文或本章节可能需要合并到polkit_(简体中文)。**

附注： 部分内容重复（在 Talk:Fprint 中讨论）

默认情况下，允许每个用户注册新指纹，而无需提示输入密码或指纹。可以使用 polkit 规则更改此行为。

有两个位置包含 polkit 配置文件

- `/etc/polkit-1/rules.d/`
- `/usr/share/polkit-1/rules.d/`

注意：不推荐直接修改 `/usr/share/polkit-1/rules.d/` 目录下的文件，因为该目录下的文件在程序更新时将会被覆盖，应该复制文件到 `/etc/polkit-1/rules.d/` 目录。

在以下示例中，只有 root 可以录入指纹：

```
/etc/polkit-1/rules.d/50-net.reactivated.fprint.device.enroll.rules
polkit.addRule(function (action, subject) {
  if (action.id == "net.reactivated.fprint.device.enroll") {
    return subject.user == "root" ? polkit.Result.YES : polkit.Result.NO
  }
})
```

故障排除

指纹设备无法使用

如果在兼容设备清单没有找到您的设备, 请通过journalctl查看`fprintd.service`的日志信息。

可能会可能到如下日志条目：

```
fprintd: Corrupted message received
```

```
fprintd: Ignoring device due to initialization error: unsupported firmware version
```
这时请确保设备的固件是最新的 Fwupd。

系统休眠后gdm在显示登录提示时挂起

这个问题在libfprint仓库中有描述。开发人员的答案是:

: 我现在的猜测是，我们正在断开USB蓝牙设备的连接，而它正在初始化。然后，当蓝牙usb设备试图加载固件时，一切都被卡住了（这有一个10秒的超时，解释了我们看到的不到10秒的挂起）。像这样断开USB蓝牙设备的连接预计会在rfkill开关切换时发生，所以这是正常的。只是设备突然断开连接的情况似乎没有得到正确处理并超时。

建议的修复方案是将禁用该驱动模块，添加内核模块配置文件：

```
/etc/modprobe.d/bluetooth-blacklist.conf
blacklist btusb
```
或者直接执行：

```
# rmmod btusb
```
完成上面操作后，系统将不会尝试初始化该设备。

挂起设备时出现意外错误

这个问题在libfprint仓库中有描述：

: Set your laptop to not suspend to RAM but to do s2idle. You might need to switch the BIOS into "Windows mode"。

BIOS中设置成"Windows mode"

指纹认证在polkit代理不工作

如果在录入指纹的时候提示类似下面信息：

```
Using device /net/reactivated/Fprint/Device/0
Enrolling right-index-finger finger.
EnrollStart failed: GDBus.Error:net.reactivated.Fprint.Error.PermissionDenied: Not
Authorized:
net.reactivated.fprint.device.enroll
```
这是因为polkit策略的限制，可以尝试在`/usr/share/polkit-1/rules.d/`添加一条策略`reactivated.fprint.device.verify.rules`，以下实例中允许`users`用户组可以使用指纹识别器：

```
/etc/polkit-1/rules.d/reactivated.fprint.device.verify.rules
polkit.addRule(function (action, subject) {
  if (action.id == "net.reactivated.fprint.device.enroll"
    && subject.isInGroup("users")) {
    return polkit.Result.YES
  }
})
```

调试模式

使用下面环境变量来获取运行时详细日志。

```
# G_MESSAGES_DEBUG=all /usr/lib/fprintd -t
```

fprintd在从睡眠唤醒前启动

创建并启动一个fprintd.service 守护进程，将 `3-3` 替换成实际指纹Bus ID，可通过命令 `lsusb -t` 获取。

```
/etc/systemd/system/fingerprint.service
[Unit]
Description=Kill fprintd before sleep
Before=sleep.target

[Service]
ExecStart=killall fprintd

[Install]
WantedBy=sleep.target
```

能录入指纹却无法验证

某些指纹识别设备扫描的图像太小，数据不足以支撑 fprint 正常工作。一种常见的解决方法是滑动而不是触摸传感器，但生成好图像的速度可能会有所不同。有些设备需要较慢的滑动速度，有些则需要较快的滑动速度。好的图像示例请参考github。

如果想练习不同速度以查看哪种速度生成更好的图像，请尝试使用脚本examples/img-capture来导出图像，并与上面的示例进行比较（需要从源代码编译libfprint）。

相关讨论见gitlab。

用户无法录入指纹

如果运行报错

```
EnrollStart failed:
GDBus.Error:net.reactivated.Fprint.Error.PermissionDenied: Not
Authorized: net.reactivated.fprint.device.enroll
```

最简单的方式是用root身份执行该命令：

```
sudo fprintd-enroll "$(whoami)"
```

这里需要提供用户，否则该指纹将会录入到root账户上。

如果录入指纹时提示 "enroll-duplicate" 的报错，则表明该手指指纹已被注册到其他用户了。要么更换其他手指，要么使用`fprintd delete`从用户中删除该指纹。

参见

1. ↑ https://github.com/iafilatov/libfprint?tab=readme-ov-file

2. ↑ https://gitlab.freedesktop.org/libfprint/libfprint/-/blob/master/examples/img-capture.c
3. ↑ https://gitlab.freedesktop.org/libfprint/libfprint/-/issues/174

通用第二因素

只有口令时，网页账号、用户会话、远端登录和磁盘都只剩一层防护。本章把同一套通用第二因素接入四个场景：网页认证、用户会话认证、OpenSSH 和 LUKS 静态数据加密，在口令之外补上一份独立凭据。

网页认证与用户会话认证是机制基础，建议必读；OpenSSH 与 LUKS 只是同一机制在远端登录和静态数据上的落地，用不上可跳过。最常见的误解是把第二因素当成口令的替代品——它只是叠加的一层，口令失守后仍要靠它拦住登录；另一个是以为配好一次便长期有效，凭据或设备一变就验证不通。

16.1　通用第二因素

难度 进阶 | **篇幅** 10.1 KB | **所属** 通用第二因素

本节将讲到

- 用于网页认证
- 用于用户会话认证
- OpenSSH
- 使用 LUKS 进行静态数据加密

通用第二因素（U2F）是一个开放标准，它基于类似智能卡中的安全技术，使用专门的 USB 或 NFC 设备来加强和简化双因素身份验证（2FA）。

该标准最早由谷歌和 Yubico 开发，并包含了来自 NXP 的贡献，现在由 FIDO 联盟进行维护。

所有与 U2F 和 U2F 设备相关的文章请参考分类:通用第二因素。

当前最新的标准是 WebAuthn。

用于网页认证

谷歌、Facebook、Twitter 和 GitHub 等主流网站都支持使用 U2F，更多网站的清单和设置文档请参考 2fa.directory 或 dongleauth.com。对于所有支持该功能的浏览器，很可能只需要安装 libfido2包。Yubico 提供了一个测试页用于进行测试。

Firefox

Firefox/微调#Fido U2F 身份验证

Chromium/Chrome

Chromium#U2F 身份验证

用于用户会话认证

YubiKey 的制造商 Yubico 开发了一个 U2F PAM 模块，可用于在登陆时进行二次验证，也可以完全替代密码。

注意：虽然本文提到了 Yubico/YubiKey，基本上所有 U2F 和/或 FIDO2 合规密钥都受 pam-u2f包 支持。

安装 PAM 模块

该模块是 pam-u2f包 的一部分。

添加密钥

注意：下面使用的目录是默认路径。对于**单用户**实现，可以将文件移动到你选择的目录下，然后在 pam_u2f.so 一行末添加 `authfile=/path/to/u2f_keys`，该操作可用于将 `u2f_keys` 移动到文件系统上的受保护位置。对于多用户实现，可参考 pam-u2f 官方文档使用中心化映射文件。

首先使用 `pamu2fcfg` 添加密钥：

```
$ mkdir ~/.config/Yubico
$ pamu2fcfg -o pam://hostname -i pam://hostname > ~/.config/Yubico/u2f_keys
```

输入 PIN 码后，通过按下 U2F 密钥上的按钮进行确认。

注意：如果由于不同网络下的 DHCP 等原因导致系统主机名被更改，你将无法进行登录。为避免该情况发生，建议指定上述选项，并将 `*hostname*` 替换为正确的主机名。

如果你有多个密钥，参考下方进行添加：

```
$ pamu2fcfg -o pam://hostname -i pam://hostname -n >> ~/.config/Yubico/u2f_keys
```

注意：以上命令必须使用 `-n` 选项。对于同一用户的后续条目，它将按照规范要求省略生成条目中的用户名部分。多个相同用户名的条目会导致 PAM 出现不可预测的行为。另外，不要手动进行分行，`u2f_keys` 文件中只能存在一行内容。

无密码 sudo

警告：在保存配置更改前，先单独打开一个具有 root 权限的 shell 环境（如 `sudo -s`），这样可以保证出问题后能够撤回更改。

在文件中加入第一行：

```
/etc/pam.d/sudo
auth            sufficient      pam_u2f.so cue origin=pam://hostname
appid=pam://hostname
```

别忘了替换掉 `*hostname*` 一项。接下来打开一个新的命令行窗口，执行 `sudo ls`。密钥上的 LED 应该会开始闪烁，按下后命令就会开始执行。`cue` 选项用于提示用户所需进行的操作（`Please touch the device`）。

要让密钥作为使用 sudo 的唯一方法（即不回退使用密码），需要将其它认证方式注释掉（默认一般只有 system-auth 一项）。

```
/etc/pam.d/sudo
#auth           include         system-auth
```

你还需要将 `pam_u2f.so` 一行的 `sufficient` 替换为 `required`。

警告：这会导致用户在没有配置 U2F 密钥的情况下无法使用 sudo。

GDM 登录

在现有的 `auth` 行后添加以下内容：

```
/etc/pam.d/gdm-password
auth           required      pam_u2f.so nouserok origin=pam://hostname
appid=pam://hostname
```

注意 `nouserok` 选项，该选项允许在用户未配置密钥时规则失效，适用于多用户环境下其中只有部分用户使用 U2F 密钥的情况。

注意：该方法不适用于用户主目录（home）加密的情况，该目录在登录流程完成前尚未完全解密，导致无法使用 `u2f_keys` 文件。在这种情况下，可以参考 pam-u2f 官方文档使用中心化映射文件。

有些多功能密钥（例如 Trezor Model T）带有 U2F/PAM 功能，但根据 CTAP 2.0 标准，在系统启动时其可能不会启用该功能。在有多个 U2F 密钥的情况下，pam-u2f 会依次查找并等待设备（Trezor）超时，之后才提示提供或触碰其它 U2F 密钥，导致 GDM 出现长达 2 分钟的延时 。You could try adding the `nodetect` option alongside `debug` and finish any device specific login (ex. screen PIN) before GDM loads.

SDDM/KDE

SDDM 似乎在用户初始登录时不支持 pam_u2f。可以使用自动登录，然后编辑 `/etc/pam.d/kde` 来控制屏幕锁定。

如果你使用的是带有生物识别验证功能的 U2F 密钥（如 Yubikey Bio），并希望使用 1FA 功能，请使用 `/etc/pam.d/kde-fingerprint`，注释掉 pam_fprintd.so 行，并在其位置上进行修改。这样可以避免在身份验证后显示解锁按钮，而不是立即解锁 。

以下为参考示例：

```
/etc/pam.d/kde-fingerprint
#%PAM-1.0

auth        required                    pam_shells.so
auth        requisite                   pam_nologin.so
auth        requisite                   pam_faillock.so      preauth

# take over fprintd for u2f since yubikey bio is a fingerprint reader and bypasses
kscreenlocker's "unlock" button
#-auth      required                     pam_fprintd.so
auth required pam_u2f.so cue pinverification=0 userverification=1

auth        optional                    pam_permit.so
auth        required                    pam_env.so

account     include                     system-local-login

password    required                    pam_deny.so
```

```
session    include                     system-local-login
```

警告：不要像其它 PAM 配置文件一样在 `/etc/pam.d/kde-fingerprint` 中使用 `auth sufficient`，这会导致即使认证失败也能绕过锁屏界面。

其它认证方式

可以参考上文为其它服务启用 PAM 模块。以 Cinnamon 屏保为例，可以修改 `/etc/pam.d/cinnamon-screensaver`。

对于 Polkit，需要先将 `/usr/lib/pam.d/polkit-1` 复制到 `/etc/pam.d/polkit-1`，然后再进行修改。

排障

如果你把自己锁在系统外了，需要先启动到恢复模式或 U 盘，然后撤回 PAM 的配置并重启。

如果 pam-u2f 模块静默失败了，可以在 `/etc/pam.d/` 下文件对应的行中添加 *debug* 关键字。

OpenSSH

8.2 以及上版本的 OpenSSH 原生支持 FIDO/U2F 密钥，详情请参考 SSH 密钥#FIDO/U2F。

使用 LUKS 进行静态数据加密

**此页面或章节适合移动到 systemd-cryptenroll。**

附注： *systemd-cryptenroll* 不止支持 FIDO。（在 Talk:Trusted Platform Module#move systemd-cryptenroll to separate article 讨论）

从 248版本开始，可以通过 systemd 使用 FIDO2 密钥解锁 LUKS 分区。

首先需要配置 `/etc/crypttab` 文件（具体参考下方），如果需要解锁根分区，也可以配置 initramfs。配置步骤与使用 TPM 芯片解锁类似，具体请参考 systemd-cryptenroll#Trusted Platform Module。

注册密钥需要使用 *systemd-cryptenroll*。首先使用以下命令列出检测到的密钥：

```
$ systemd-cryptenroll --fido2-device=list
```

然后在 LUKS 槽中注册密钥，指定 `auto` 值（如果有多个设备，也可以 FIDO2 设备的路径，例如 `/dev/hidrawX`）：

```
$ systemd-cryptenroll --fido2-device=auto /dev/sdX
```

非根分区

对于非根分区，crypttab 如下：

```
/etc/crypttab
data /dev/sdX none fido2-device=auto
```

对于加密分区是由 LVM 管理的逻辑卷的情况同样可用：

```
/etc/crypttab
data /dev/vg1/data none fido2-device=auto
```